

THE JAVASCRIPT BLUEPRINT

By Gustav Ndamukong

Copyright © 2026 Gustav Ndamukong

All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in reviews and certain other non-commercial uses permitted by copyright law.

The code examples in this book are provided for educational purposes. They may be used freely in your own projects.

The information in this book is provided on an "as is" basis, without warranty. While every effort has been made to ensure its accuracy, neither the author nor any distributor shall be liable for any loss or damage caused, or alleged to have been caused, directly or indirectly by the information it contains.

JavaScript is a trademark of Oracle Corporation. Other product and company names mentioned may be the trademarks of their respective owners, and are used here in an editorial fashion only, with no intention of infringement.

Published by Nolimit Media Press Toronto, Ontario, Canada nolimitmedia.ca

First edition

ISBN: 000-0-0000000-0-0

For Gustavine & Jean-Steve, my first coding students

CONTENTS AT A GLANCE

PREFACE: How to make the most of this book

Chapter 1 - Introduction

Chapter 2 - Variables

Chapter 3 - Arrays

Chapter 4 - Constants

Chapter 5 - Control Flow

Chapter 6 - Regular Expressions

Chapter 7 - Functions

Chapter 8 - Cookies

Chapter 9 - Strings

Chapter 10 - Data Types

Chapter 11 - Data Structures

Chapter 12 - Object Oriented Programming (OOP)

Chapter 13 - Databases and Storage

Chapter 14 - Dates and Time

Chapter 15 - DOM and URL Manipulation

Chapter 16 - The Canvas Element

Chapter 17 - Design Patterns

Chapter 18 - File Management

Chapter 19 - Forms and Email

Chapter 20 - Images

Chapter 21 - Error Debugging and Testing

Chapter 22 - Extensions (APIs & Libraries)

Chapter 23 - Templating

Chapter 24 - Events Handling

Chapter 25 - Asset Management

What next

(Every chapter ends with its own Quiz — the questions first, then the answers listed below them, numbered to match.)

CONTENTS

- THE JAVASCRIPT BLUEPRINT 1**
- CONTENTS AT A GLANCE 4**
- HOW TO MAKE THE MOST OF THIS BOOK 10**
- Chapter 1 — INTRODUCTION 12**
 - LANGUAGE SYNTAX 15
 - Terminating statements 16
 - Where to place the script tags in a web document 18
 - Commenting code 20
 - Dealing with browsers that do not support JavaScript (rare) 21
 - Debugging JavaScript errors 21
 - Creating a local development server 22
 - QUIZ — Introduction 27
- Chapter 2 — VARIABLES 32**
 - Variable creation and naming rules 33
 - Initialising a variable 34
 - QUIZ — Variables 51
- Chapter 3 — ARRAYS 58**
 - THE TWO TYPES OF ARRAYS 60
 - Multi-dimensional arrays 73
 - Looping through arrays 82
 - Array properties 93
 - Array methods 95
 - True array methods and associative arrays 106
 - QUIZ — Arrays 109
- Chapter 4 — CONSTANTS 119**
 - QUIZ — Constants 122
- Chapter 5 — CONTROL FLOW 125**
 - QUIZ — Control Flow 149
- Chapter 6 — REGULAR EXPRESSIONS 159**
 - GROUPING 162
 - CLASSES 162
 - NEGATION 163
 - QUIZ — Regular Expressions 168

Chapter 7 — FUNCTIONS	174
INTRODUCTION	174
FUNCTIONS DEMONSTRATED	176
THE ARGUMENTS OBJECT	178
ARGUMENTS WITH DEFAULT VALUES	180
TO RETURN SOMETHING OR TO DO SOMETHING	181
FUNCTIONS AND VARIABLE SCOPE	182
ANONYMOUS AND ARROW FUNCTIONS	183
Quick object literals from function arguments	185
Passing arguments to an arrow function	188
QUIZ — Functions	193
Chapter 8 — COOKIES	201
How cookies work	201
Security considerations	201
Managing cookies	202
A cookie consent solution	204
QUIZ — Cookies	208
Chapter 9 — STRINGS	213
QUIZ — Strings	220
Chapter 10 — DATA TYPES	225
Why data types differ across languages	226
Similarities across Languages	227
Understanding strong and weak typing	230
Understanding static and dynamic typing	232
The best way to study data types	233
Explicit casting	234
Types of data in JavaScript	237
BOOLEANS	247
QUIZ — Data Types	251
Chapter 11 — DATA STRUCTURES	257
ARRAYS	259
LINKED LISTS	259
STACKS	260
QUEUES	261
TUPLES	262
Dictionaries (Maps/HashMaps)	263
Sets	263

Structs	264
Trees (Binary Trees, AVL Trees, etc.)	265
COLLECTIONS	266
QUIZ — Data Structures	267
Chapter 12 — OBJECT ORIENTED PROGRAMMING (OOP)	273
Introduction	275
OBJECTS IN DEPTH	306
QUIZ — OOP	334
Chapter 13 — DATABASES AND STORAGE	343
QUIZ — Databases and Storage	356
Chapter 14 — DATES AND TIME	361
Date and time in JavaScript	361
Create a date	362
Formatting dates	362
Working with Time Zones	363
Working with Timers: setTimeout() and setInterval()	369
A datepicker library in vanilla JS & Bootstrap	375
QUIZ — Dates And Time	377
Chapter 15 — DOM AND URL MANIPULATION	383
Introduction to the DOM	385
Traversing the DOM	402
Manipulating elements	405
Creating New Elements	408
Working with Attributes & Classes	413
Handling styling	414
Working with positioning	415
Working with events	416
XPath and selecting DOM Elements	417
The DOMParser	431
Examples of DOM manipulation	432
THE WINDOW OBJECT	449
Modifying URL components	457
Generating links to assets created in memory	458
QUIZ — DOM and URL Manipulation	462
Chapter 16 — THE CANVAS ELEMENT	471
The Canvas element—for drawing on the web	471
QUIZ — The Canvas Element	500

Chapter 17 — DESIGN PATTERNS	508
ADAPTER PATTERN	517
DECORATOR PATTERN	519
FACADE PATTERN	521
COMPOSITE PATTERN	523
PROXY PATTERN	525
OBSERVER PATTERN	529
STRATEGY PATTERN	530
TEMPLATE METHOD PATTERN	532
COMMAND PATTERN	534
ITERATOR PATTERN	537
QUIZ — Design Patterns	539
Chapter 18 — FILE MANAGEMENT	548
The File Reader API	549
Use-cases for the File Reader API	550
Key methods	551
Upload a text file and display its contents	553
Generate CSV from a JavaScript Array	561
Read data from a CSV file and inject into the DOM	562
Reading a file's raw binary data with readAsArrayBuffer()	564
Validating File Signatures (a.k.a. Magic Numbers)	565
Sending binary file content over WebSocket	566
Previewing a PDF file	579
Reading XML files with JavaScript	580
Limitations of the FileReader API	594
QUIZ — File Management	595
Chapter 19 — FORMS AND EMAIL	602
Sending Emails	610
QUIZ — Forms and Email	615
Chapter 20 — IMAGES	620
JavaScript and images	620
What JavaScript can do with images	621
Image manipulation	621
QUIZ — Images	648
Chapter 21 — Error Debugging and Testing	654
Displaying values on screen	654
The browser developer tools	659

Throwing and handling exceptions	663
TESTING	673
QUIZ — Error, Debugging and Testing	675
Chapter 22 — Extensions (APIs & Libraries)	681
APIs	682
AJAX	703
WEBSOCKETS	731
Where networking actually lives	732
LIBRARIES	733
QUIZ — Extensions (APIs and Libraries)	735
Chapter 23 — TEMPLATING	741
String Literals & Template Strings	741
JavaScript Templating Engines	741
Popular Modern Approaches	742
Implementing a template with Handlebars.js	742
QUIZ — Templating	745
Chapter 24 — EVENTS HANDLING	748
EVENT LISTENERS	749
Common JavaScript Events	752
Keyboard key detection	754
Triggering events dynamically	756
Conclusion	766
QUIZ — Events Handling	767
Chapter 25 — ASSET MANAGEMENT	773
What next	774
Practice, practice, and practice	775
ABOUT THE AUTHOR	777

HOW TO MAKE THE MOST OF THIS BOOK

What many beginners aren't told loudly enough is this: you won't become a great programmer by just reading about code—you become great by writing it. You can read every book on programming cover to cover, but until you roll up your sleeves and start coding, true understanding will remain out of reach. That's why this book takes a hands-on, example-driven approach. Every chapter ends with a quiz—a set of questions designed to help reinforce key concepts and accelerate your learning. Some of them are small exercises where you write a little code yourself to demonstrate a point. These questions are not random—they're deliberately structured to build your skill step by step, and every one of them carries a clue, so you should never find yourself stuck. The answers are listed immediately below the questions, at the end of that same chapter, numbered to match, with detailed explanations. Alongside the quizzes, you'll also find concise, well-explained code examples throughout every chapter. These are just as valuable as the quiz questions because they demonstrate real-life scenarios in which the concepts are applied. They're not artificially complex or overly long, but are written to be clear and focused—so you can easily test them in your browser, see them in action, and truly understand what's happening. The uniqueness of this book lies in its many practical examples and the thorough, beginner-friendly explanations that follow them. These examples are inspired by real-world programming problems, so you'll always understand why the code matters and when to use it. As you progress, you'll notice how the chapters build upon each other, connecting the dots and helping you grow from a novice into a confident, capable JavaScript developer. This approach is grounded in my own experience. When I first started out, I read many books, but when it came time to actually write code, I couldn't produce anything useful. The problem wasn't with the books themselves—it was with the lack of practical, interactive learning. Once I started solving real problems, documenting my own solutions, and revisiting them over time, everything began to make sense. That's when I truly started to learn. This book follows the method I wish I had from the start: clear, example-rich, and practical. In every chapter, you're encouraged to take your time, run the code, and digest the explanations. This isn't a book to skim through—it's one to explore, code-along with, and return to. Here's how to get the most out of it:

- For the quiz at the end of each chapter: pause and work through every question yourself first, before looking at the answers listed below them.

- Throughout each chapter: run the code examples in your browser, play with the values, and observe how they work.
- You should set up a simple local project with an index.html file, using the following skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>JavaScript Master</title>
</head>
<body>

  <script>
    // Your JavaScript code here
  </script>
</body>
</html>
```

The book is designed so you can try everything in real time, right in your browser. Whether you're completely new to JavaScript or revisiting it after a while, this book is your companion. I've resolved the very challenges I faced when learning over a decade ago, and now I'm handing you a smoother, smarter path forward. Let's dive in—and start building something amazing together.

Chapter 1 —INTRODUCTION

- Welcome to the World of JavaScript
 - What Is Programming?
 - Why JavaScript
 - A Quick Look Back: The History of JavaScript
 - Why learn JavaScript today?
 - What About Frameworks?
 - What this book will teach you
- Language syntax
- Terminating statements
- Where to place the script tags in a web document
- Commenting code
- Dealing with browsers that do not support JavaScript (rare)
- Debugging JavaScript errors
- Creating a local development server

Imagine opening a webpage and seeing more than just plain text and images. Imagine buttons that react, forms that validate instantly, animations that respond to your actions, and content that changes without reloading the page. All this magic? That's JavaScript. This book is your journey into that world—a world where websites come alive and programming becomes exciting, practical, and even fun.

What Is Programming?

At its heart, programming is simply the act of telling a computer what to do. But it's not just about giving orders—it's about thinking logically, solving problems, and building things from scratch. And just like you'd use different tools for different jobs, programming gives you a variety of tools (called programming languages) to create all kinds of software.

Why JavaScript

JavaScript is one of the most popular programming languages in the world today—especially when it comes to web development.

A web page is built from three separate technologies, each with its own job. HTML (HyperText Markup Language) provides the content and the structure—the headings, paragraphs, images, buttons and form fields. CSS (Cascading Style Sheets) controls how all of that looks—the colours, fonts, spacing and layout. JavaScript is the third of the three, and it is the one that makes the page do things. This book assumes you have seen a little HTML before, but you do not need to be an expert at it. We will explain any HTML we use as we go.

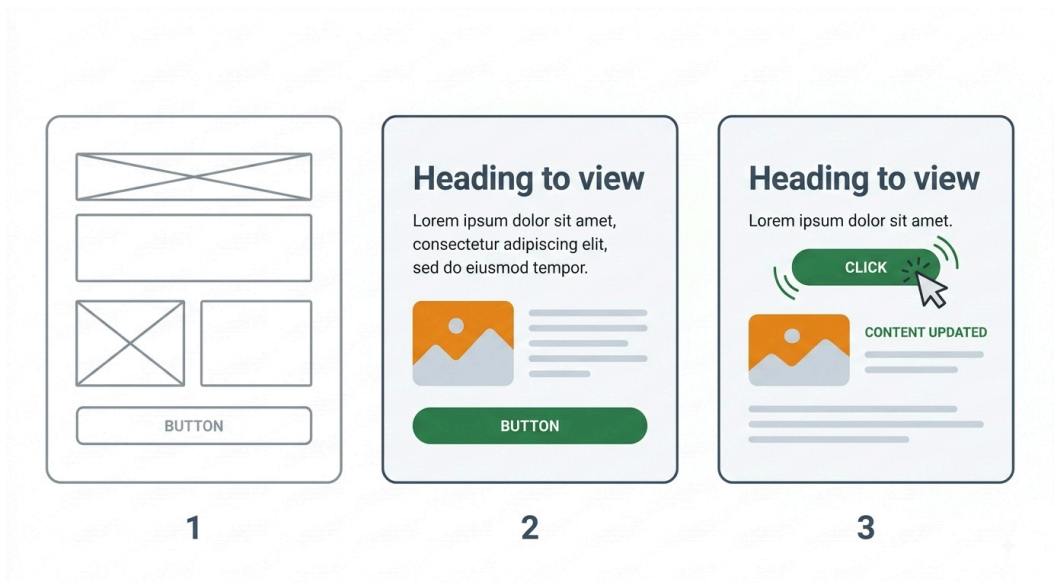


Figure 1.1 — The three technologies that build a web page

If HTML is the skeleton of a webpage and CSS is the skin and clothes, JavaScript is the muscle. It gives websites power, movement, and interactivity. Without JavaScript, websites are static. They just sit there. You can click a link and go to another page, but that's about it. With JavaScript, you can make pages respond to a user's actions, show a message, fetch more data without reloading, change what's on the page in real time, and so much more. JavaScript runs inside your web browser, which is why it's known as a client-side scripting language. It has direct access to every part of the webpage through something called the DOM (Document Object Model), a structured map of all the HTML elements on the page. This gives JavaScript immense power.

A Quick Look Back: The History of JavaScript

JavaScript was created in 1995 by Brendan Eich while he was working at Netscape. Back then, it was built quickly to add a bit of interactivity to web pages. But from those humble beginnings, JavaScript exploded into something massive.

At first, developers faced a tough challenge—different web browsers supported JavaScript in different ways. This meant code that worked perfectly in one browser could fail in another. To solve this, libraries like jQuery were created to smooth over browser differences and make JavaScript easier to use. Basically, jQuery was a library built on top of JavaScript, to solve the discrepancies between different browsers. It was successful in achieving that, which was great. With jQuery, you were assured that the JavaScript code you wrote would work on all browsers. That was peace of mind for developers who had to previously test their code on all browsers and write extra functionality to handle browsers that may not support the code they wrote. This is why jQuery quickly became very popular.

Then came a major evolution: ECMAScript, the standard version of JavaScript. Starting with ES5 (2009), then the game-changing ES6 (2015) and beyond. JavaScript has grown into a mature, modern, and powerful language—complete with new features, better syntax, and (thankfully) solid support across all major browsers.

Why learn JavaScript today?

This is the perfect time to learn JavaScript. It's no longer the wild west. JavaScript is now stable, widely supported, and used everywhere—from websites and mobile apps to backend servers, desktop apps, and even smart devices. Better yet, the job market for JavaScript developers is booming. Whether you're learning to build your own apps or preparing for a career, JavaScript is an essential language to know.

What About Frameworks?

You will hear the words 'library' and 'framework' used a great deal, and often quite loosely, so here is the difference. A library is a collection of ready-made code that you call when you need it. You stay in charge, and you reach for the library like a tool out of a toolbox. A framework works the other way round: it provides the whole structure of your application, and it calls your code at the points where you have filled in the blanks. The framework is in charge, and you build inside it. jQuery, which we met earlier, is a library. Angular is a framework. React sits somewhere between the two, which is why you will hear it described as both.

You might have heard of React, Vue, Angular, Backbone, or TypeScript. These are JavaScript frameworks and tools created to make certain types of web development easier and faster. For example:

- React (by Meta, formerly Facebook) helps build user interfaces efficiently using components. Strictly speaking React is a library rather than a full framework, but you will often hear it spoken of as one.
- Vue (a community-driven framework) is loved for its simplicity and flexibility.
- Angular (by Google) offers a complete toolkit for building large, complex apps.
- Backbone is one of the earliest of these tools. It brought structure to JavaScript applications back when the language had none of its own.
- TypeScript (by Microsoft) adds types to JavaScript to help catch bugs early. Note that TypeScript is not a framework at all. It is a superset of JavaScript, which means it is JavaScript with extra features built on top.

These frameworks are powerful, but they all sit on top of vanilla JavaScript—the core language we’re about to dive into. ‘Vanilla’ simply means plain: JavaScript on its own, with nothing added—no frameworks, no libraries, no extra tools. It is the real language underneath every one of them. Learning vanilla JavaScript gives you the foundation to understand, master, and even switch between frameworks with confidence.

What this book will teach you

This book will take you from a complete beginner to a confident JavaScript developer. You’ll learn everything from the basics like variables, loops, functions, to advanced concepts like objects, arrays, event handling, working with APIs and more. We’ll focus on clarity, simplicity, and hands-on examples, so you’re not just reading, you’re doing. You’ll write real code, solve real problems, and build things that work. So, if you’re ready to learn a powerful language that runs in every web browser, builds beautiful user experiences, and opens doors to all kinds of opportunities, you’re in the right place. The JavaScript Blueprint is meant to be the only book you will ever need to master the JavaScript programming language.

LANGUAGE SYNTAX

JavaScript has to be written within script tags, or in a separate .js file that is linked from a script tag, for it to be parsed by the JavaScript interpreter in the browser.

Two words are going to come up a lot from here on, so let’s define them now. To parse something means to read it through and work out its structure—exactly what you do when you read a sentence and work out which word is the verb and which is the subject. The JavaScript interpreter is the part of your browser that does this

for your code. It reads your JavaScript from top to bottom, works out what each instruction means, and then carries it out. Every major browser has one built in, and this is why you do not need to install anything special to run JavaScript. Your browser is already the engine.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Hello world</title>
  </head>
  <body>
    <script>
      // Your JavaScript code here
      document.body.innerHTML = "Hello world";
    </script>
  </body>
</html>
```

Do not worry about understanding that line of code just yet. In plain English it says: "find the body of this page, and replace everything inside it with the words Hello world". We will take it apart properly when we reach the DOM in Chapter 15.

Terminating statements

Before we talk about terminating them, let's be clear about what they actually are. A statement is a single complete instruction to the computer—one thing you are telling it to do. An expression is a piece of code that produces a value. The difference is easiest to see side by side:

```
2 + 3          <- an expression. It produces the value 5.
let x = 2 + 3;  <- a statement. It instructs the computer to
                store that value in x.
```

So expressions are the ingredients, and statements are the instructions that use them. A statement will often have one or more expressions inside it. Think of a statement as a full sentence, and an expression as a phrase within that sentence. And just as a sentence ends with a full stop, a JavaScript statement ends with a semicolon.

Expressions and statements are terminated by a semicolon. For example:

```
let x = 5;
console.log(x);
```

That second line is your first sight of `console.log()`, so let's say what it is before we go any further. It is a built-in JavaScript command that prints a value out so that you can look at it. What it prints does not appear on the web page itself; it goes

into the browser console, which you open using your browser's developer tools (in most browsers, press F12, or right-click the page and choose Inspect, then click the Console tab). Keep that console open while you work through this book, because `console.log()` is the tool you will reach for most often to check what your code is actually doing. We will look at it, and at the developer tools, more closely in Chapter 21 (Error Debugging and Testing).

You do not have to end every statement with a semicolon because JavaScript has a feature called Automatic Semicolon Insertion (ASI). This allows you to omit semicolons in many cases, as the interpreter automatically adds them where it thinks they are needed. So this code will still work:

```
let x = 5
console.log(x)
```

Generally, the JavaScript parser considers a new line as a new statement. So, theoretically you only 'have to' put a semicolon at the end of a statement if you are placing more than one statement in the same line, in which case you would mark the end of each command by ending it with a semicolon, except for the last command. However, the interpreter will not always be accurate in determining where your statements end or start, and so you might sometimes get unexpected outcomes. For example, the following code

```
let x = 5
let y = x
(2 + 3).toString()
```

will be wrongly interpreted as follows, resulting in an error:

```
let x = 5;
// Error: TypeError: x is not a function
let y = x(2 + 3).toString();
```

It is therefore recommended to be safe by ending all your statements with semicolons. This will still work well, and have the added benefit of eliminating the risk of any misunderstanding by the parser about where one statement ends and where another starts. Also, while semicolons are not always needed to mark the end of statements, there is an exception that if a statement ends with a variable or function where the first character of the next line is a left parenthesis or a square bracket, then you must finish that line with a semicolon or the code will not work. So, this reinforces the advice that whenever in doubt just use a semicolon.

Where to place the script tags in a web document

JavaScript can be included in an HTML document in several ways: directly within the page (inline), as a reference to a separate local file, or by linking to a file hosted on an external server. The third method is commonly used to load third-party libraries or services like jQuery, Google Analytics, or frameworks such as React or Vue. Examples:

i) Referencing a local script file:

```
<script src="/filePath/script.js"></script>
```

ii) Loading an external script from the internet:

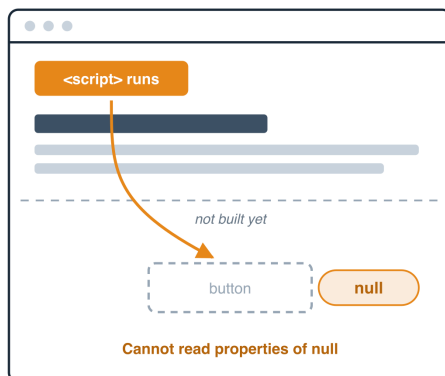
```
<script src="https://example.com/scriptName.js"></script>
```

(Note: You don't include JavaScript code between the opening and closing `<script>` tags when using the `src` attribute.)

However, whether the JavaScript code being used on your web page is being pulled from an external file in your local file system, or from an entirely different website on the internet; knowing where to place the JavaScript code on your web page is important. Where you place your script in the document has significant performance and behaviour implications. So, where should you place script tags? Placing the script just before the closing `</body>` tag is generally considered best practice, especially for scripts that manipulate or depend on elements in the DOM. This ensures the HTML is parsed and elements are available before the script runs, which avoids errors and improves perceived load speed. To know why this happens, one has to understand how a web page is loaded. Imagine a web page being loaded as a sheet of paper coming out of a fax machine. The paper is slowly ejected from the machine in a head-first manner, as it is written to by the fax machine. If the head of this page has the JavaScript code to work on this page, then you will end up with a situation where, the code then tries to read HTML elements below on the page (maybe to add event listeners to them, or select them etc), but these elements may not be available yet because the page is not fully loaded yet. (An event listener is simply code that sits and waits for something to happen, such as a user clicking a button. We will cover them properly in Chapter 24.) You may therefore get errors for elements which you are trying to select but which do not yet exist on the page, because they have not been loaded yet. What the browser hands back when it cannot find an element is null, which simply means 'nothing found', so the error message you will actually see reads something like 'Cannot read properties of null'.

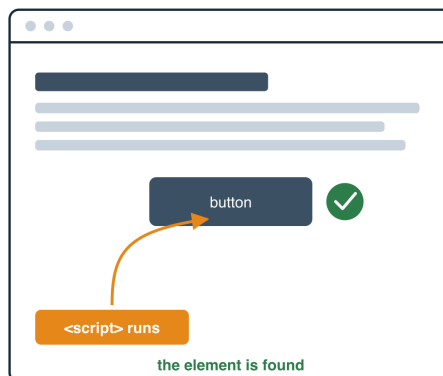
Script in the <head>

the page is only half built when the code runs



Script before </body>

the whole page exists before the code runs



- Figure 1.2 — Why a script in the head cannot find elements further down the page*

If you have to place code within the head tag section of your web page, then it's wise to add some kind of code to make the script wait until your HTML page has loaded before it runs. Let's look at two ways to do this:

- a) Add the `defer` attribute to the `<script>` tag itself, so that the script waits until the HTML has finished being parsed before it runs. Here is how to do that:

```
<script src="/your-script.js" defer></script>
```

Notice the 'defer' attribute in the opening `<script>` tag. It is the HTML5 short way of declaring attributes. The full way will look like this: `defer=""`. We leave out the `'=""` if it has no value.

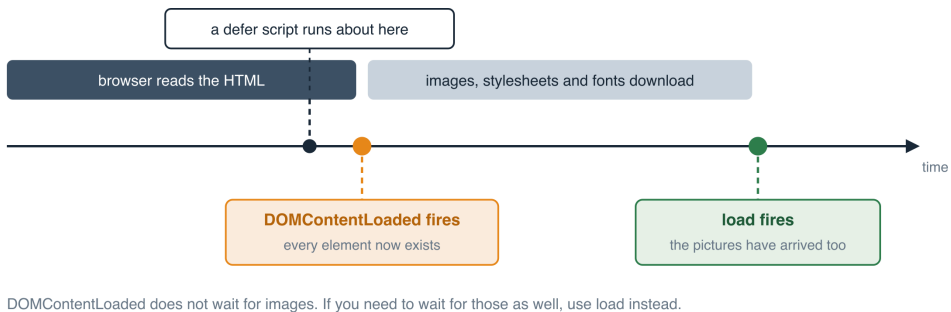
One thing to note is that `defer` only works on a script that loads an external file using the `src` attribute. It has no effect on code written directly between the opening and closing script tags.

- b) Use the `DOMContentLoaded` event. Here is how to use it:

```
document.addEventListener('DOMContentLoaded', function () {  
    // your code here. Only runs once the HTML has been fully  
    // parsed and all the elements exist on the page  
});
```

Be careful not to confuse this with the page being 'fully loaded'. `DOMContentLoaded` fires as soon as the HTML has been read and all the elements exist, but it does not wait for images, stylesheets or fonts to finish downloading. If you need to wait for those too, use the 'load' event instead.

What happens, and in what order, as a page loads



DOMContentLoaded does not wait for images. If you need to wait for those as well, use load instead.

- Figure 1.3 — When defer, DOMContentLoaded and load each happen*

Commenting code

This is text that the browser will not treat as code, and will therefore ignore. It's usually greyed out. It is always good to start learning a language by knowing how to comment out code so that it is not interpreted when your code is run. All programming languages have this feature, and it is very handy in placing notes to yourself and to your fellow developer colleagues when working in a team environment. These notes-to-self (so to speak), can make your life easy as convoluted code written and understood now can seem like Greek when you come back to read it months, or even years later. You can use comments to help yourself or another developer who picks up the project quickly understand what was going on. There are two ways to comment out code, single line (also known as inline commenting), and multi-line comments. To comment out a single line, begin the line with a double forward slash like so:

```
let userName = "John";  
// let userName = "Peter";
```

The value of `userName` will be "John". The line that sets a different value, "Peter", is not read. This is because it is commented out so it is ignored by the browser. We will learn more about variables shortly.

To comment out multiple lines, begin the first line with a forward slash and an asterisk, then end the last line with an asterisk and a forward slash like so:

`/* This code is commented out and so it will not be executed. Note how it also spans multiple lines of code. Do not forget to end the comment like so below.`

- `/`

Dealing with browsers that do not support JavaScript (rare)

While JavaScript is essential for creating dynamic web applications, not all users have it enabled. The `<noscript>` tag provides a way to display fallback content for those rare cases. What It Does The `<noscript>` tag tells the browser:

“If JavaScript is disabled or not supported, show this content within this tag instead.” It’s useful for displaying a message or a simplified version of your feature. Here is an example:

```
<noscript>
  <p>JavaScript is required to use this website. Please enable it in
your
  browser settings.</p>
</noscript>
```

One rule of thumb is to use the `<noscript></noscript>` tag for browsers not supporting JavaScript, then provide static HTML alternatives to the functionality you build using JavaScript.

These days, however, you will rarely need to use this because most modern browsers support JavaScript, and most users have it enabled. But there are times when `<noscript>` is still helpful. Here are examples of scenarios when you may need it:

- To show a warning message when JavaScript is required.
- For critical features where accessibility or SEO is a concern.
- When you want to provide a basic static version of a feature.

However, if your app fully depends on JavaScript like many modern single-page applications (SPAs), and your audience is mostly using up-to-date browsers, you may not need `<noscript>` at all. So in conclusion, use `<noscript>` as a fallback for better accessibility or critical communication, but it’s no longer something that must be used in every project.

Debugging JavaScript errors

Debugging means being able to find where a fault is in code, for example a typo. This is a very handy skill that every developer should have—the ability to think critically and resolve issues that come about in code as the application is being used. I will give you the three most important tools available to you for debugging in JavaScript. These are:

- Displaying values on screen
- Browser developer tools
- Handling exceptions

We will talk about how to use them in Chapter 21 (Error Debugging and Testing).

Creating a local development server

When learning programming, you always want to test out what you learn. This is the best way to learn. This is why the book is packed full of code examples. I urge you to set up a local development server on your computer to actually test out most, if not all, of the examples I provide you in this book. Do not worry about your speed, just worry about how much you are understanding. The time you put in learning will be worth it in the end, when you end up being an expert programmer. In all my years of experience, if I am to give you one rule to follow, it would be this:

"Never, ever rush over your coding lessons. Rather, document stuff, and prioritise having working code that you fully understand in the end, over tons of code that you do not understand".

You should therefore not read a coding book like a novel, or watch a coding video tutorial as a movie. Once you have passed the beginner level and have grasped the concepts of programming, you may do this occasionally, to refresh your mind on something, maybe a syntax or approach. But programmers know that you will only get real reward from coding when you type out code and get something working. This brings me to the next rule I would like to give you:

"Whilst learning to code, only working code that you typed yourself can count as a skill that you have a mastery of".

Having said that, let's now talk about how to set up for yourself a local development server where you can quickly write and test your JavaScript code. This ensures your browser behaves more like a real-world scenario, especially when working with files like JavaScript, images, and AJAX requests. AJAX is the technique of fetching fresh data from a server in the background, without reloading the whole page—it is how a page can update a part of itself while you are still looking at it. We will come to it in Chapter 23. This should be easy to do because we are dealing with vanilla JavaScript—which is JavaScript with no frameworks or third-party libraries or software needed. You therefore do not need to install much on your computer. All you need is your browser, and an IDE (Integrated Development Environment). There are many free IDEs out there, of which VS Code is one. VS

Code has an extension known as Live Server which you can use to serve your JavaScript web pages in your browser locally as you develop. This is perfect for local testing. One more thing that is great about Live Server is that it will watch your JavaScript files and automatically detect any changes you make in the code and refresh the page in the browser for you. That is good because it saves you having to refresh your browser every time you save any changes. It is one of the most popular and beginner-friendly ways to set up a quick local test server in VS Code for running and testing HTML/CSS/JavaScript files. Here's a step-by-step guide on how to set up a local development environment in Visual Studio Code (VS Code):

Step 1: Create a Project Folder:

- Go to your computer's desktop (or any other location you prefer).
- Create a folder and name it something like js-projects.
- Inside that folder, you can create other folders for each project, for example: js-test, first-project, calculator-app, loops-practice, etc.

Step 2: Install Visual Studio Code (VS Code)

If you don't have VS Code installed already:

- Visit <https://code.visualstudio.com>
- Download and install VS Code for your operating system.

Step 3: Open Your Project Folder in VS Code

- Open VS Code
- Go to File > Open Folder
- Select the folder you just created (e.g., js-projects or any of its subfolders).
- Click Open.

Step 4: Install the “Live Server” Extension

If you had not already installed the Live Server extension in your VS Code, do it like so:

- Click on the Extensions icon (left sidebar with the group of tiny squares) to open the Extensions view, search for:

Live Server

- Many options will come up. Choose the one made by Ritwick Dey. This is the original, it is by far the most installed, and it is the one this book assumes you

ensions exist with very similar names, and some of them have been removed for being malicious. Always check the publisher name, and check the number of downloads and star reviews, before you install anything.

- Click on Install.

Step 5: Create an HTML File to Run

Live Server can open any .html file that you right-click on, but it is a good habit to have a file named 'index.html' in your project folder. This is the file a web server looks for by default when someone visits a folder without naming a particular file, so it is the one Live Server will reach for when you click 'Go Live'.

- So go ahead and create one within your project folder. The code in your index.html file can look like this:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>The JavaScript Blueprint</title>
  </head>
  <body>
    <h1>The JavaScript Blueprint</h1>

    <script src="index.js"></script>
  </body>
</html>
```

- Next, create a new file that will contain your JavaScript code in the same folder. It is a JavaScript file and so should have the '.js' extension, for example 'index.js', although it could be named anything, for example 'script.js'. Just make sure it is referenced correctly within the <script> tag as seen above in the body tag of your index.html file like so:

```
<script src="index.js"></script>
```

- This index.js file is where your JavaScript will live, so put some code like the following, just to test that it is working:

```
alert("JavaScript is working!");
```

This will display a popup box saying: "JavaScript is working!" when your index.html page loads. alert() is one of JavaScript's built-in commands, and its whole job is to pop up a message box like that.

Step 6: Start the Live Server

- Finally, right-click on your index.html file's tab in the VS Code editor.

- Choose "Open with Live Server". You will get a message saying:

Server is Started at port: 5500

Give it a few seconds and it should open a new browser tab with a URL like this:

<http://127.0.0.1:5500/yourFolder/index.html>

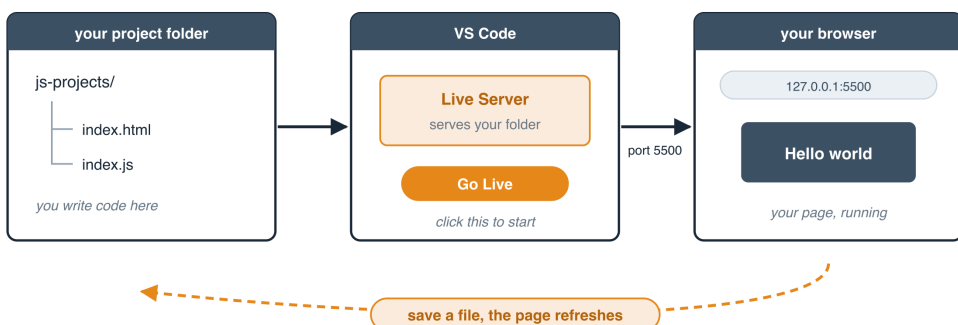
This is because Live Server has launched a web server, exposed the port number 5500 on your computer, to serve your project folder files beginning with index.html at the above URL. A port is simply a numbered door on your computer that a program can listen at. Your computer has thousands of them, and giving Live Server port 5500 means 'send anything for this project to door number 5500'. That is why the port number turns up in the URL. If the browser tab fails to open, just paste <http://127.0.0.1:5500/yourFolder/index.html> into your browser's address bar yourself. Replace 'yourFolder' with any folder path that sits between the folder you opened in VS Code and your index.html file. If your index.html sits directly inside the folder you opened, then there is nothing to replace and the URL is simply <http://127.0.0.1:5500/index.html>.

- Visit that URL: <http://127.0.0.1:5500/yourFolder/index.html> in your browser to see the contents of your index.html web page displayed. You should now see a browser popup displayed with this text:

JavaScript is working!

- Any time you update your code and save the file, the page will automatically refresh.
- You only need to do this setup once per project.
- You can also start the Live Server by clicking "Go Live" in the bottom-right corner of VS Code.

How your local development setup fits together



- Figure 1.4 — How your local development setup fits together*

QUIZ — Introduction

This page contains the Q & A (questions and answers) for this chapter — Chapter 1: Introduction. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. A web page is built from three separate technologies. Name all three, and say in one short sentence what each one is responsible for.

Clue: one gives the page its content and structure, one controls how it looks, and one makes it do things.

2. Look at this line taken from a web page:

```
<script src="/js/main.js"></script>
```

Why is there nothing written between the opening `<script>` and the closing `</script>` ?

Clue: look at what the `src` attribute is pointing to.

3. Where in an HTML document is it usually best to place your `<script>` tag, and why?

Clue: think back to the fax machine, and to which parts of the page actually exist at the moment your code runs.

4. Here are two lines of JavaScript with no semicolons anywhere:

```
let x = 5  
console.log(x)
```

Will this still work? What is the name of the JavaScript feature that makes this possible, and what is this book's advice on the matter?

Clue: the feature's initials are ASI.

5. Write one single line of JavaScript that is commented out. Then write a block of three lines that are all commented out together.

Clue: one of them starts the line with two characters. The other wraps the whole block, opening with one pair of characters and closing with another.

6. What is the difference between an expression and a statement? Give one tiny example of each.

Clue: one of them produces a value. The other is a complete instruction to the computer.

7. A friend tells you their web page is broken. Their JavaScript sits inside the `<head>`, and when it runs they get the error "Cannot read properties of null". What has gone wrong, and what are two different ways they could fix it?

Clue: one fix is a single word added to the script tag. The other is an event you can listen for.

8. What is the `<noscript>` tag for, and would you say every website today must have one?

Clue: think about what the browser does with the content inside it, and about how common it is nowadays for a visitor to have JavaScript switched off.

9. What does "vanilla JavaScript" mean, and why does this book teach it before teaching any framework?

Clue: think about what every framework is built on top of.

10. You want to test your code on your own computer. Name the VS Code extension this book recommends and say who made it. Then explain what the 5500 refers to in this address:

<http://127.0.0.1:5500/index.html>

Clue: the chapter describes that last part as a numbered door.

ANSWERS

1. HTML, CSS and JavaScript.

- HTML (HyperText Markup Language) provides the content and the structure — the headings, paragraphs, images, buttons and form fields.
- CSS (Cascading Style Sheets) controls how all of that looks — the colours, fonts, spacing and layout.
- JavaScript makes the page do things — it gives the page movement and interactivity.

Or, as the chapter puts it: HTML is the skeleton, CSS is the skin and clothes, and JavaScript is the muscle.

2. Because the `src` attribute is already telling the browser where to find the code. The JavaScript lives in the separate file at `/js/main.js`, so there is nothing to write

inside the tags. When you use `src`, you leave the space between the tags empty.

3. Just before the closing `</body>` tag.

The browser reads an HTML page from top to bottom, in order. If your script sits at the top in the `<head>`, it runs before the rest of the page has been built, so any element it tries to reach further down does not exist yet. Putting the script at the bottom guarantees the whole page has been read first, so every element your code needs is already there.

4. Yes, it will still work.

The feature is called Automatic Semicolon Insertion (ASI). The JavaScript interpreter generally treats a new line as the end of a statement and quietly adds the semicolon for you.

The book's advice, though, is to end all your statements with semicolons anyway. ASI is not always accurate, and it can join two lines together that you meant to keep apart, which produces confusing errors. When in doubt, use a semicolon.

5. A single commented-out line uses two forward slashes at the start:

```
// let userName = "Peter";
```

A commented-out block opens with a forward slash and an asterisk, and closes with an asterisk and a forward slash:

```
/*  
    let a = 1;  
    let b = 2;  
    let c = 3;  
*/
```

6. An expression is a piece of code that produces a value. A statement is a single complete instruction telling the computer to do something.

```
2 + 3          <- an expression. It produces the value 5.  
let x = 2 + 3;  <- a statement. It instructs the computer to store  
that value in x.
```

A statement will often contain one or more expressions inside it. Think of a statement as a full sentence and an expression as a phrase within that sentence.

7. The script in the `<head>` is running before the elements it wants have been built. When the browser cannot find an element, it hands back null, which means "nothing found" — hence the error message.

Two ways to fix it:

- Add the defer attribute to the script tag, so the script waits until the HTML has finished being parsed:

```
<script src="/your-script.js" defer></script>
```

- Wrap the code in a DOMContentLoaded listener, so it only runs once every element exists:

```
document.addEventListener('DOMContentLoaded', function () {  
    // your code here  
});
```

A third option, of course, is simply to move the script down to just before `</body>`.

8. The `<noscript>` tag holds content that the browser will display only if JavaScript is disabled or unsupported. It is a fallback — typically a short message, or a simplified version of whatever your JavaScript would have provided.

```
<noscript>  
    <p>JavaScript is required to use this website. Please enable it in  
    your  
        browser settings.</p>  
</noscript>
```

No, not every site needs one today. Almost all modern browsers support JavaScript and most people have it enabled. It is still worth using where accessibility matters, or where you want to show a warning that JavaScript is required. But it is no longer something that must appear in every project.

9. "Vanilla" simply means plain. Vanilla JavaScript is JavaScript on its own, with nothing added — no frameworks, no libraries, no extra tools.

The book teaches it first because every framework, React, Vue, Angular and the rest, is built on top of it. Once you understand the core language, you have the foundation to pick up any framework, and to move between them with confidence.

10. The extension is Live Server, made by Ritwick Dey. (Take care to pick that one — copycat extensions exist with very similar names, and some have been removed for being malicious.)

The 5500 is a port number. A port is a door into your computer that programs can use. The port number is a numbered door on your computer that a program running on your computer can listen at for any other program trying to connect with or communicate with it, or through which it can make information available for other programs to connect to and make use of. Those other programs are often somewhere else on a network, but they can just as easily be sitting on the very same

computer — which is exactly what is happening here, since it is your own browser doing the connecting. Live Server has opened door number 5500 and is serving your project files through it, which is why that number appears in the address.

Chapter 2 — VARIABLES

- Variable creation and naming rules
- Initialising a variable
 - Assigning values to variables
 - Variable scope and blocks
 - What is a block
 - Types of blocks
 - Blocks are not objects
 - Blocks are not data structures
 - The practical use of blocks
 - Global scope
 - Two differences between var vs let and const in the global scope
 - Hoisting and the temporal dead zone
 - Best practices for variables
 - JavaScript modules and variable scope

A variable is like a virtual container in computer memory in which you can store things for later use while your program is running. It is far more efficient to work something out once and keep it in a variable than to work it out all over again every time you need it. You can only store one item in a variable at a time. This means an attempt to store an item in a variable that already contains something will result in the value of that variable being reset to the new value and the old value being discarded. Some programming languages will try to help you by erroring about the variable already having been set, but others will not and let you override the variable. You just have to be careful and make sure that is your intention, because JavaScript will let you reassign the value of any variable you created with var or let. (There is a third kind, created with const, which cannot be reassigned once it has been given a value. We will come to it shortly.) A variable can contain any of the data types, for example, a number, a string of text, a boolean, an array, a function, or the value held by another variable. (Do not worry if some of those words mean nothing to you yet. We will meet arrays in Chapter 3 and the full list of data types in Chapter 10. Functions get a chapter of their own, Chapter 7, though we will need a quick working idea of them a little later on in this one.)

Variable creation and naming rules

Variable names should adhere to these rules:

- i) Variables may include a-z, A-Z, 0-9, the \$ symbol and the underscore.
- ii) No other characters are allowed. That means no spaces, and no punctuation.
- iii) The first character of the variable name must be a letter (a-z or A-Z), the \$ symbol, or the underscore. It may NOT be a digit, so a name like 1stName is not allowed. Digits are fine anywhere after the first character, as in name1. However, the \$ character should be avoided so that it is not mixed up with the \$ character used in other scripting languages like PHP for variables, or any third party library you might be using now or in the future like jQuery which uses it too.
- iv) Names are case sensitive. It has to be, as variables are meant to identify resources and so must be unique.

This means that these two variables are not the same:

```
let myCaseDetails;  
let Mycasedetails;
```

- v) There is no limit on variable name length.

By convention, programmers usually name variables using camel casing. This means that the variable name should start with a lowercase letter, and any other word that makes up the name will be started in uppercase, with no spaces in-between. For example, the following are potential variable names:

myRoundBall nationalId myCaseDetails etc

A variable can be declared in three ways in JavaScript: -The var keyword One way is to use the var keyword, for example:

```
var firstName = "John";
```

-The let keyword There is the let keyword. Here is an example.

```
let mySurname = "Doe";
```

-The const keyword There is the const keyword. Here is an example:

```
const title = "The Day of The Jackal";
```

As for the const keyword, it was introduced in ES6; it is a way to create a block-scoped constant variable. A constant is a variable whose value cannot be re-assigned once it has been set. We will talk more about it in Chapter 4 (Constants).

Initialising a variable

Declaring a variable means creating it. Initialising it means giving it its first value. You can do both in one go, or you can declare a variable now and give it a value later on. Here is a variable being declared and initialised at the same time:

```
let greeting = "hello";
```

And here is a variable (a) being declared with no value given to it yet:

```
let a;
```

```
console.log(a); // will return undefined
```

A variable that has been declared but not yet initialised holds the special value `undefined`. It can also be referred to as an empty variable, since it contains no value. Take note that `undefined` is not the same thing as `null`. They are two different values, and we will look at the difference between them in Chapter 10 (Data Types). You can assign a value to it later using an assignment operator like so:

```
a = 'dog';
```

```
console.log(a);  
// will return the string value 'dog'
```

As a quick reminder from Chapter 1, `console.log()` is the built-in command that prints a value out so you can look at it, and what it prints goes into the browser console rather than onto the page itself. Open it with your browser's developer tools (in most browsers, press F12, or right-click the page and choose Inspect, then click the Console tab). It is the tool you will reach for most often when you want to check what a variable is actually holding, so it is worth keeping that console open as you work through this chapter.

Assigning values to variables

We do so using the assignment operator—more on operators in Chapter 5 (Control Flow). Here is an example of assigning a value to a variable:

```
let firstName = "John";
```

As we can see above, the assignment operator is `=`, and we used it to assign the value of `"John"` to the `firstName` variable.

You can however declare a variable without assigning it a value yet. The variable will then exist but its value will be `undefined`. You can then assign a value to it later. Here is how to declare a variable with no value:

```
let userName;  
let age;
```

Once you have declared the variable, when assigning a value to it later, you will no longer need to use the var or let keyword. For example, to assign values to the declared variables `userName` and `age` above, do this:

```
userName = "John Doe";  
age = 30;
```

Notice that `"John Doe"` is wrapped in quotes but `30` is not. The quotes are what make something a string, that is to say a piece of text. A number is written without quotes. If you wrote `age = "30"`; you would be storing the text `"30"` rather than the number `30`, and the two behave very differently once you start doing sums with them.

Variable scope and blocks

A quick word about functions first, because they are about to matter a great deal. A function is a parcel of code that you give a name to, so that you can run it whenever you like, rather than only at the moment you wrote it. You create one with the word `function`, a name, a pair of round brackets, and then the code itself inside curly braces:

```
function sayHello() {  
    console.log("Hello");  
}
```

Writing that out does not run anything. It only puts the parcel aside under the name `sayHello`. To actually run it, you call it, by writing its name followed by round brackets:

```
// now it runs, and prints Hello  
sayHello();
```

Those round brackets can also carry values into the function. A value you hand over in this way is called an argument, and the name the function uses for it on the inside is called a parameter:

```
// name is the parameter  
function greet(name) {  
    console.log("Hello " + name);  
}  
  
greet("John");  
// "John" is the argument
```

JavaScript also comes with a good many functions already built in, ready for you to call. You have met `console.log()` already. Two more you will see shortly are `alert()`, which pops up a message box, and `prompt()`, which pops up a box asking the user to type something in and hands back whatever they typed. That is as much as you need for this chapter. Functions have a great deal more to them, and Chapter 7 is devoted entirely to them.

Now, before we understand the scope of variables, we have to first of all understand the concept of blocks in JavaScript.

What is a block

A block in JavaScript is any section of code enclosed within curly braces `{}`. Blocks are used to group multiple statements together, and they define the scope for `let` or `const` variables. So, the two types of variables which can have their scope limited by blocks are variables created with the keywords `'let'` or `'const'`. These two types of variables are therefore said to be block-scoped variables. There is a third type of variable whose scope is slightly different from `let` and `const` variables; and this is a `var` variable. Variables created using the `'var'` keyword can only be scoped by a function. Though a function as we will see below is also a block, also referred to as a function block; it is the only block that can limit the scope of a `var` variable. To summarise the behaviour of these three types of variables within blocks, we can say that while `let` and `const` variables are block-scoped, `var` variables are function-scoped. Here are the characteristics of a block: - A block is created by enclosing a body of code within a pair of opening and closing `{}` (curly braces). The code within these curly braces then become the block. This is where you can group multiple statements together. - A block creates a new scope, which in other words means that besides allowing you a space to group multiple statements together, a block also creates an area of visibility or reach for your variables. However, the types of variables that respect blocks, in terms of being limited in scope by blocks are only variables created using the keywords `'let'` and `'const'`. This means that when these variables are defined, they cannot be accessed from outside the block they are defined in. Any attempt will lead to a `ReferenceError`, which is the browser telling you it cannot find that name at all. Variables created using the `'var'` keyword on the other hand are not block scoped, or in other words, do not respect blocks. This means that variables created with the `'var'` keyword inside a block can be accessed from outside that block they are defined in, unless the block is a function block. In other words, their values leak through the block.

Here is an example of how `var` being used inside a block leaks out:


```

if (true) {
  // Declared inside the block
  var x = 10;
}

// x is still accessible outside the if block
console.log(x);

```

Even though `x` is declared inside the `if` block, it is still accessible outside because `var` does not respect block scope. Using `let` or `const` resolves the issue:

```

if (true) {
  let y = 20; // Block-scoped
  const z = 30; // Block-scoped
}

// Error: ReferenceError: y is not defined
console.log(y);

// Error: ReferenceError: z is not defined
console.log(z);

```

Here, both `y` and `z` are restricted to the block because `let` and `const` follow block scope rules.

This tells us that there is no point creating a variable using `'var'` within a block. Its value will leak out anyway. The only time `var` is useful is when you actually want function-scoped variables, like inside a function:

```

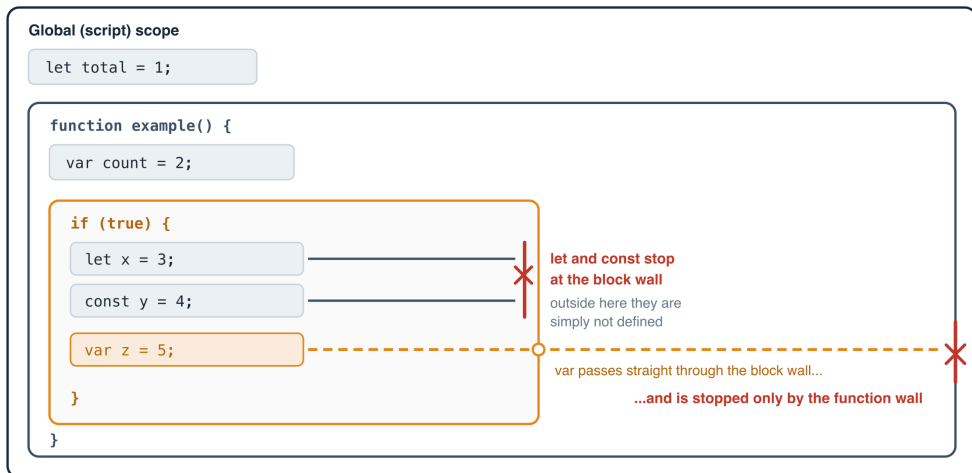
function example() {
  // Function-scoped
  var a = 100;
}

// Error: ReferenceError: a is not defined
console.log(a);

```

Here, `a` is restricted to the function, so it behaves as expected.

How far each kind of variable can be seen



let and const are held by every set of curly braces. var ignores them all, and is stopped only by a function.

- Figure 2.1 — How far each kind of variable can be seen*

Types of blocks

Remember we have established that a block in JavaScript is a group of code enclosed within a pair of curly braces. The main types of block you will meet in JavaScript are: a standalone block, an if statement, a loop (for or while loop), and a function. There are others besides these, such as try/catch and switch blocks, which we will meet in later chapters. Here they are with examples:

a) Standalone block

```
{  
  let x = 10;  
  console.log(x); // 10  
}
```

```
// ReferenceError: x is not defined  
console.log(x);
```

The variable x only exists inside {} and is not accessible outside.

b) If statement block

```
if (true) {  
  let y = 20;  
  console.log(y); // 20  
}
```

```
// ReferenceError: y is not defined
console.log(y);
```

c) Loop (for or while loop) block

Both the for loop and the while loop of JavaScript are blocks. Let's see an example of a for loop:

```
for (let i = 0; i < 3; i++) {
  // 0, 1, 2
  console.log(i);
}

// ReferenceError: i is not defined
console.log(i);
```

In this example; *i* is only available inside the for loop block.

d) Function block

```
function example() {
  let z = 50;
  console.log(z); // 50
}

example();

// ReferenceError: z is not defined
console.log(z);
```

Functions are also considered blocks, and variables declared inside them are scoped to the function (function-scoped).

Blocks are not objects

An object literal { key: value } is NOT a block—it's just an object. Don't worry, we will talk more on objects in Chapter 12 (Object Oriented Programming). Blocks are structural elements of JavaScript's syntax, while objects are data structures. For example:

```
let obj = {
  name: "Alice",
  age: 25
};

console.log(obj.name); // Alice
```

Even though an object uses {}, it does not create a new scope. The variables inside the object are properties, not block-scoped variables.

Blocks are not data structures

A block `{}` is not a data structure in JavaScript. A block is just a syntactic structure used to group code together and define scope, particularly for `let` and `const`. It does not store data like arrays or objects do. Let's demonstrate that a little bit:

```
{
  let x = 10;
  const y = 20;
}

// ReferenceError: x is not defined
console.log(x);
```

This `{}` simply defines a scope. It doesn't store values like an object or an array. Basically, an object `{ key: value }` is a data structure because it stores and organises data. In programming in general, there are different types of data structures, with each one having its unique way of storing and organising data. In the case of objects, its data is stored as key-value pairs. See Chapter 11 (Data Structures) to learn more about data structures in JavaScript. Here is an example of an object:

```
let person = {
  name: "Alice",
  age: 25
};

// This will write to the console: Alice
console.log(person.name);
```

This `{}` is an object literal, which is used to store data. Let's look at some key differences to help us distinguish between a block and an object:

- A block does not have keys—it just defines a scope for `let` and `const` variables.
- An object has keys (properties), but these are not variables. They are stored inside the object and accessed using dot notation (`obj.key`) or bracket notation (`obj["key"]`).

The practical use of blocks

One might be tempted to question the usefulness of standalone blocks! At first glance, it might seem like they don't serve much purpose, but they do have practical applications in real-world programming. While they aren't the most common feature, they can be useful in certain scenarios—especially for keeping your code clean and avoiding variable pollution (overriding of one another). Here are some instances when you need a block in your application code:

- When you need temporary variables that shouldn't affect the rest of the code.

- When you want to isolate (separate) execution logic without defining a function. This means, the code will be parsed and run at the line where it is, without you having to call it explicitly.
- When you want to logically group code within a function for better readability.

Let us see some code examples:

- 1. Using a Block to Process Temporary Data Without Polluting Scope

Here is an example of fetching and processing API data in an isolated block. Before you read it, a word of reassurance. This example, and the one after it, deliberately reach ahead of where we are. They use tools we have not covered yet, such as `fetch`, `async` and `await` for talking to a server (Chapters 21 and 22), and commands for reaching into the web page itself (Chapter 15). Do not try to understand every line. Look only at the curly braces, and at which variables survive outside them. That is the single point being made here.

```
async function fetchUserData()
{
    console.log("Fetching user data...");

    // start of block
    {
        let response = await fetch("https://api.example.com/user");
        let data = await response.json();

        let username = data.name;
        let age = data.age;

        console.log(`User: ${username},
                    Age: ${age}`);
    } // end of block

    // Now, outside the above block, the
    // variables 'response', 'data',
    // 'username' and 'age' are all out of
    // scope, and that is the idea.

    console.log("Finished processing.");
}

fetchUserData();
```

The `fetchUserData()` function has a block inside of it. Outside that block, the blocked-scoped variables `'response'`, `'data'`, `'username'`, and `'age'` no longer exist. This is because they are out of scope, and that is the idea. The benefit is a tidier program, where a name only exists for as long as it is actually needed.

- 2. Isolating Temporary DOM Elements Without Affecting Global Scope. This is about temporary DOM Manipulation Without Polluting Scope.

```
document.querySelector("#btn")
  .addEventListener("click", () => {

    // block start
    {
      let messageBox =
        document.createElement("div");
      messageBox.textContent =
        "Action completed!";
      messageBox.style.color =
        "green";

      // place the div on the DOM
      document.body.appendChild(
        messageBox);

      setTimeout(() => {
        // Remove it after 3 seconds
        messageBox.remove();
      }, 3000);
    } // end of block

  });
```

This example demonstrates JavaScript's prowess in manipulating the DOM. Here we create an HTML div element and store it in a variable named `messageBox`, and insert it into the DOM so it can be seen in the browser. It then sets a timer so that the div is removed from the DOM again after 3 seconds. Working with the Document is something that may be new to you now, but we will dive deeper into it and demystify everything when we come to talk about DOM Manipulation in Chapter 15. In this code, the `messageBox` element is only used inside the block. Once it's appended and scheduled for removal, there's no need to keep the variable around.

Therefore you can see how when using a block, you can place code in them that performs actual logic like fetching data, modifying the DOM, handling calculations, etc. The key idea is:

- a) Do the necessary work inside the block.
- b) Let the variables automatically disappear when they are no longer needed.

Global scope

As a reminder of all what we have learned so far about variables and scopes, here are the take-away points: -Var variables are always global unless they are used

inside a function (they are function-scoped). This means that they only adhere to the rules of function blocks (blocks that are functions). Using them in blocks (non-function) will not stop them from being global, unless the block is within a function, in which case they will be scoped to the function and not the block anyway. There is therefore no point putting them in a block inside a function. The bottom line is that using them in a function is the only thing that will stop them from being global and they will be local to that function. -Let and const variables will be global unless they are used within a block (they are block-scoped). Note that functions themselves are blocks (see types of blocks above), therefore let and const declared inside a function behave as function-scoped. So, using them within a function will stop them from being global, and they will be local to that function. Also, unlike variables declared with var; if you placed let and const variables inside a block within the function, they will only be defined and local to that block within the function, and reaching for them outside that block gives a ReferenceError.

Regardless of the limitations the three types of variables can have when they are scoped, they can all still be used as global variables. Do this simply by not scoping them. So, if a variable is declared with var, let or const outside of any block or function, they are global and can be used throughout your entire script. Let's demonstrate in code.

```
const fee = 20; // Global constant
const price = 100; // Global constant
let count = 0; // Global variable
// Global variable
var shopName = "OvalFoods";

function getAmount(quantity = 1)
{
    // Using global price and fee
    var total = (price * quantity) + fee;

    console.log("Total paid at " +
        shopName + " is: " + total);

    return total;
}

getAmount(5);
```

The fee and price variables are declared with const at the top level, making them global constants.

The count and shopName variables are declared with let and var respectively at the top level, making them global variables as well. Notice that getAmount() names its own parameter quantity rather than count. Had we named it count too, it would

have hidden the global count from view inside the function, which is a trap we look at in a moment under shadowing.

The function `getAmount()` is able to access these global variables, and so will any other code on this page whether they are in a function, a block or neither.

We have not talked about a variable declared with neither of the keywords `var`, `let` or `const`. This is also possible when you are running JavaScript in non-strict mode. Strict mode is a stricter set of rules that you can switch on by putting the line `"use strict"`; at the top of your file, and which JavaScript modules turn on automatically. Normally strict mode will force you to add those keywords (declarations), but non-strict mode will not. So in non-strict mode, a variable declared without any of those keywords is automatically a global variable.

```
var appStatus = true;

function updateStatus() {
    testLocalVar = "wild";
}

console.log(appStatus);

updateStatus();

console.log(testLocalVar);
```

In this example, `appStatus` is declared outside of any function, making it a global variable. This means it can be accessed both inside and outside functions. That's why logging `appStatus` in different places results in the same output: `true`.

However, the variable `testLocalVar` inside `updateStatus()` is not explicitly declared using `var`, `let`, or `const`. In this case, JavaScript implicitly assigns it as a global variable when `updateStatus()` is called. This happens because, in non-strict mode, any variable assigned a value without a declaration (`var`, `let`, or `const`) is automatically added to the global scope. That's why `console.log(testLocalVar)` works without throwing an error.

Another thing we need to remember about global variables is that it is possible to declare a local variable with the same name as a global variable. When this happens, the local variable shadows the global one within its function. That means it overrides the global one locally. Consider this example:

```
var item = 'shoe';

function displayItem() {
    var item = 'dress';
```



```
displayItem();
console.log(item);
```

The output will be : dress shoe

Inside `displayItem()`, the local `item` variable with the value 'dress' shadows the global `item` within that function's scope. However, the global `item` remains 'shoe' when accessed outside the function.

Two differences between `var` vs `let` and `const` in the global scope

There are two important differences between the behaviour of `var` and `let` or `const` in the global scope. Even though `let` and `const` can be used globally, they do not behave exactly like `var` in the global scope. Here are the differences:

a) `var` attaches to the window object (in browsers), `let` and `const` do not. The window object is the browser's own global object, where it keeps everything that is available everywhere on the page. We look at it properly in Chapter 15.

```
var x = 10;
let y = 20;
const z = 30;

// OK: 10 (var attached to the global obj)
console.log(window.x);
// undefined (let is not attached)
console.log(window.y);
// undefined (const is not attached)
console.log(window.z);
```

`var` becomes a property of window, while `let` and `const` do not. They still work everywhere on the page, but they are not stored on the window object.

b) Re-declaring `var` globally is allowed, but `let` and `const` are not. For example:

```
var a = 100;
var a = 200; // OK: works fine

let b = 300;

// Error: SyntaxError: Identifier 'b' has
// already been declared
let b = 400;
```

Note that re-declaring is not re-assigning. Re-declaring is the same as in the above example where we use the `var`, or `let` or `const` keyword (declaration) as if declaring the variable for the first time like so:

```
let b = 300;
let b = 400;
```

Re-assigning is when you simply update the value of an already existing variable, for example:

```
let count = 0;
count = 1;
```

The assigning code references the variable name (count) without needing to re-declare it (using let) and updates its value.

Hoisting and the temporal dead zone

There is one more difference between var and let or const, and it explains a lot of otherwise baffling behaviour. It is called hoisting. Before your code runs, JavaScript takes a quick first pass over it and makes a note of every variable you have declared. In effect, the declarations are lifted to the top of their scope. This is what is meant by hoisting: the declaration is hoisted up, even though the line you wrote it on stays where it is. The catch is that var and let behave very differently when this happens. A var variable is hoisted and given the value undefined straight away. So you can refer to it before the line that declares it, and instead of an error you simply get undefined:

```
console.log(price);
// undefined, not an error
var price = 100;
```

That is rarely what anybody wants. It hides mistakes, because a typo or a line in the wrong order fails quietly instead of telling you about it. A let or const variable is also hoisted, but it is NOT given a starting value. It sits in what is called the temporal dead zone: it exists, but it refuses to be touched until the line that declares it has actually run. Reaching for it before then gives you a clear error:

```
// Error: ReferenceError: Cannot access
// 'total' before initialization
console.log(total);
let total = 100;
```

"Temporal" simply means "to do with time", and the dead zone is the stretch of time between the top of the block and the line where the variable is declared. This is a good example of let being stricter than var, and of that strictness being a kindness. The var version lets a mistake slide by silently. The let version stops and tells you exactly what went wrong, which is far easier to fix. The practical lesson is a simple one: declare your variables before you use them, and prefer let and const so that JavaScript tells you when you have not.

Reaching for a variable before you declare it

Before a single line runs, JavaScript has already noted every declaration in the scope. This is hoisting.

with var

price already exists here,
holding undefined

```
console.log(price);
```

```
var price = 100;
```

That console.log prints:

undefined

no error at all

with let

temporal dead zone

total exists, but cannot be touched

```
console.log(total);
```

```
let total = 100;
```

That console.log throws:

ReferenceError: Cannot access 'total'

The error is the better outcome.

var lets the mistake slide past quietly. let stops and tells you exactly what went wrong, which is far easier to fix.

- Figure 2.2 — Reaching for a variable before you declare it*

Best practices for variables

It is recommended to always declare variables using `let` or `const` instead of relying on `var`, as they provide better scoping rules and avoid accidental global variable leaks. Use `const` for a value that should never be reassigned, and `let` when you do need to update the value later on. One word of warning about `const`, because it catches almost everybody out: `const` stops you from reassigning the variable, but it does not freeze what is inside it. If a `const` holds an array or an object, you can still change the contents; what you cannot do is point that name at something else entirely. We look at this properly in Chapter 4 (Constants). `let` allows reassignments without polluting the window object (useful for things like counters, flags which are only needed temporarily). Let variables as we have seen also prevent accidental re-declaration, which `var` allows. Avoid implicit global variables (i.e., variables declared without `var`, `let`, or `const`), as they can lead to unintended side effects. Keep variable scope as limited as possible to avoid conflicts and unexpected behaviour.

By following these practices, you can write cleaner, more maintainable JavaScript code.

JavaScript modules and variable scope

As your code grows larger, it's common to split it into multiple files to keep things organised. These files can be treated as modules — reusable, self-contained pieces of JavaScript code. (Take care not to confuse this use of the word with the

les modules a little differently than regular scripts. To declare a JavaScript file as a module, you use the `type="module"` attribute in your HTML in the `<script>` tag as you reference the specific JavaScript file:

```
<script type="module" src="index.js"></script>
```

This tells the browser, “This file uses modular JavaScript — isolate its variables and functions from the global scope.” That last part is important:

When a file is loaded as a module, its variables and functions are private by default. They won’t be accessible globally from your HTML or from other script files unless you explicitly export or attach them to the global window object. Let us look at an example that will cause an error of a missing function:

Let’s say you define a function like this in your `index.js` file:

```
// index.js
async function fetchPhotos() {
  const response = await
  axios.get('https://jsonplaceholder.typicode.com/photos', {
    params: { albumId: 1 }
  });

  document.getElementById('output').textContent =
    JSON.stringify(response.data, null, 2);
}
```

Do not worry about understanding what the code does for now. It uses an external library called Axios to make an AJAX request to fetch photos from the URL `'https://jsonplaceholder.typicode.com/photos'`. We will come back to this same example in Chapter 22 (Extensions - APIs & Libraries), where I will explain how it all works. So, let’s say in your HTML code you do:

```
<script type="module" src="index.js"></script>

<button onclick="fetchPhotos()">Fetch Photos</button>
<pre id="output"></pre>
```

If you run this code in your browser, you’ll get an error like:

Uncaught ReferenceError: fetchPhotos is not defined

That’s because `fetchPhotos()` exists inside the module, not in the global scope where HTML’s `onclick` can see it. So how do you resolve this issue? You have two options:

Option 1: Attach the function to the global window object. Inside your `index.js`, just add:

```
window.fetchPhotos = fetchPhotos;
```

This explicitly exposes the `fetchPhotos()` function to the global scope, so HTML can use it.

Option 2: Remove the `type="module"` attribute from your `<script>` tag. If you're not using modern features like `import` and `export`, you can simply treat your script as a regular script:

```
<script src="index.js"></script>
```

Now everything defined in `index.js` becomes globally available by default.

Both of those options answer the same question: how do I reach this function from my HTML? There is a third case, and it is the one modules were really built for — reaching it from another JavaScript file.

For that, neither `window` nor removing `type="module"` is the answer. The file that owns the value has to hand it out, using the `export` keyword:

```
// index.js
export async function fetchPhotos() {
  // ...the same code as before
}
```

And the file that wants to use it has to ask for it by name, using the `import` keyword:

```
// main.js
import { fetchPhotos } from './index.js';

fetchPhotos();
```

The two always come as a pair. One file offers the value with `export`, the other asks for it with `import`, and neither one works on its own.

Notice also what this does NOT do. Adding `export` to `index.js` would not have fixed the onclick error above, because `export` makes a value available to other MODULES, not to HTML. That is what Option 1 was for. It is worth keeping the two apart in your mind: `window` is how you reach JavaScript from HTML, and `import/export` is how one JavaScript file reaches another.

So, here are the take-away points of learning:

- If you want to use `import/export` and keep code modular, use `type="module"` and export what you need. The `import` and `export` keywords are the commands modules use to share code between files: `export` in the file that owns the value, `import` in the file that needs it. Do not worry, we will cover modules and the importing and exporting of values between files in Chapter 12 (Object Oriented Programming), specifically under the sub-heading: 'Sharing data between files' from the 'Objects in depth' heading.

- If you need HTML or other scripts to access your functions, attach them to window, or don't use `type="module"`.

Using modules is recommended for modern apps, but if you're just starting out or building simple pages, you can safely skip `type="module"` for now. As you advance, understanding how scope works in modules will help you write cleaner, more secure code.

QUIZ — Variables

This page contains the Q & A (questions and answers) for this chapter — Chapter 2: Variables. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 10 to 12 are proper little exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Which of these are valid JavaScript variable names, and why are the others not allowed?

myAge my age 1stPrize _total \$price first-name

Clue: think about spaces, about punctuation, and about what a name is not allowed to begin with.

2. What is the difference between declaring a variable and initialising it? And if you declare a variable but never give it a value, what does it hold?

Clue: it is not null. It is the other one.

3. What is a block, and which of these three create one?

```
if (true) { ... }  
let person = { name: "Alice" };  
function greet() { ... }
```

Clue: all three use curly braces, but one of them is storing data rather than grouping code.

4. Look at this code and say what each `console.log()` prints:

```
if (true) {  
  var a = 10;  
  let b = 20;  
}  
  
console.log(a);  
console.log(b);
```

Clue: one of these two keywords respects blocks and the other does not.

5. What does this print, and what would happen if you swapped `var` for `let`?

```
console.log(price);  
var price = 100;
```

Clue: the name of the behaviour begins with H, and the let version gives you an error message rather than a value.

6. You have written this:

```
const colours = ["red", "green"];
```

Can you do `colours.push("blue")` ? Can you do `colours = ["blue"]` ?
Explain the difference.

Clue: const protects the label on the box, not what is inside the box.

7. Of var, let and const declared at the top level of a script in a browser, which one becomes a property of the window object?

Clue: it is the oldest of the three.

8. What is the difference between re-declaring a variable and re-assigning it? Give a one-line example of each.

Clue: one of them uses a keyword like let, and the other does not.

9. What does this code print, and what is the name for what is happening?

```
var item = 'shoe';  
  
function displayItem() {  
    var item = 'dress';  
    console.log(item);  
}  
  
displayItem();  
console.log(item);
```

Clue: the inner variable hides the outer one, but only inside the function.

10. EXERCISE. This exercise will teach you how to generate HTML elements in JavaScript and insert them into the HTML section of your code.

a) Create a div element in your HTML with a specific ID b) Within your JavaScript tag, create a variable assigning it a paragraph HTML element c) Dynamically (using JavaScript) grab the div and insert that <p> tag inside of it

Clue: you will need `document.getElementById()` to grab the div, and `innerHTML` to put something inside it. Remember that an HTML tag written in JavaScript is just a string, so it goes in quotes.

11. EXERCISE. This exercise will teach you how to get the user to supply some information to your application, and then take and use that information.

- a) Display a prompt on screen asking the user to enter their forename
- b) Display a prompt on screen asking the user to enter their surname
- c) Store those values in variables, then
- d) Show an alert telling them what their forename and surname are

When you refresh your web page in the browser, you should see two prompt dialog popups one after the other; one asking you to enter your forename, and if you enter the value for your forename and press enter or hit OK, another popup will appear asking you to enter your surname. After entering the value for your surname, you will get an alert popup on screen with text saying something like "Your forename is theForenameYouEntered, and your surname is theSurnameYouEntered".

Clue: `prompt()` asks the user for something and hands back what they typed. `alert()` shows a message. Join your text and your variables together with the `+` sign.

12. EXERCISE. This one will show you how to modify the value of an HTML element.

- a) Create a div in your HTML and give it an ID
- b) Place a `p` tag with some text manually in the div in your HTML. Refresh your web page in the browser and you should see the text in the `p` tag displayed on screen
- c) Next, create a variable in JavaScript and assign an image tag to it
- d) Dynamically grab the div, remove the `p` tag with text inside of it, and replace it with the image you have created in JavaScript

When you refresh your web page, the image should be displaying in the place of the text that was being displayed before.

Clue: emptying an element is just a matter of setting its `innerHTML` to an empty string. Watch your quotes carefully on this one — the `img` tag needs quotes of its own inside the string.

ANSWERS

1. Valid: `myAge`, `_total`, `$price`.

- `my age` is not allowed, because names may not contain spaces.
- `1stPrize` is not allowed, because a name may not begin with a digit. Digits are fine anywhere after the first character, so `prize1st` would be fine.
- `first-name` is not allowed, because the hyphen is punctuation. JavaScript would read it as "first minus name".

Remember that `$` is allowed but is best avoided, as other tools and libraries such as jQuery use it for their own purposes.

2. Declaring a variable means creating it. Initialising it means giving it its first value.

```
let a;           // declared only
// declared and initialised in one go
let b = 5;
```

A variable that has been declared but never given a value holds the special value `undefined`. It does not hold `null`. `undefined` means "nothing has been put in here yet", whereas `null` is a value you deliberately assign yourself. There is more on the difference in Chapter 10 (Data Types).

3. A block is any section of code enclosed within curly braces `{}`. It groups statements together and creates a scope for `let` and `const` variables.

The `if` statement and the function both create blocks. The object literal does not — even though it uses curly braces, it is a data structure that stores values as key-value pairs, and it creates no scope at all.

4. It prints:

```
10 Error: ReferenceError: b is not defined
```

`var` does not respect block scope, so `a` leaks out of the `if` block and is still reachable afterwards. `let` is block-scoped, so `b` only exists inside the braces. Reaching for it outside gives a `ReferenceError`, which is JavaScript telling you it cannot find that name at all.

5. It prints `undefined` — not an error.

This is hoisting. Before your code runs, JavaScript makes a note of every variable you have declared, and a `var` variable is given the value `undefined` straight away. So the name already exists by the time `console.log()` runs, it just has nothing useful in it yet.

Swap `var` for `let` and you get an error instead:

```
Error: ReferenceError: Cannot access 'price' before initialization
```

because a `let` variable sits in the temporal dead zone until the line declaring it has actually run. The error is the more helpful outcome of the two, which is a good reason to prefer `let`.

6. Yes to the first, no to the second.

```
// fine - colours is now ["red", "green", "blue"]
colours.push("blue");
// TypeError: Assignment to constant variable
colours = ["blue"];
```

`const` stops you from pointing the name at something else. It does not freeze what is inside. With an array or an object, the contents can still be changed.

This catches almost everybody out at least once. See Chapter 4 (Constants) for more.

7. var.

```
var x = 10;
let y = 20;
const z = 30;

console.log(window.x);    // OK: 10
console.log(window.y);
// undefined
console.log(window.z);
// undefined
```

A top-level var becomes a property of the window object. let and const do not. They still work everywhere on the page, but they are not stored on window.

8. Re-declaring means using the keyword again, as though creating the variable for the first time. Re-assigning means simply updating the value of a variable that already exists.

```
let count = 0;
// re-declaring - SyntaxError with let
let count = 1;

let total = 0;
// re-assigning - perfectly fine
total = 1;
```

var allows re-declaration, which is one of the ways it lets mistakes slip through unnoticed. let and const do not.

9. It prints:

dress shoe

This is called shadowing. The item inside displayItem() is a separate, local variable that hides the global item for as long as we are inside that function. Outside the function, the global item is untouched and still holds 'shoe'.

10. Here is the whole page:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Exercise 1</title>
</head>
<body>
  <div id="myDivElem"></div>
```

```

let pTag = "<p>This is my awesome p tag</p>";
    let div = document.getElementById('myDivElem');

    div.innerHTML = pTag;
</script>
</body>
</html>

```

Take note of how you need to put HTML tags within quotes when you create them in JavaScript, as in:

```
let pTag = "<p>This is my awesome p tag</p>";
```

As far as JavaScript is concerned that is simply a piece of text. It only becomes real HTML at the moment you hand it to innerHTML.

When you refresh your web page, you will see the text "This is my awesome p tag" displayed, and it is coming from the p tag you dynamically created and injected into the myDivElem div.

11. Here is the code:

```

let forename = prompt('Please enter your forename', 'Enter forename');
let surname = prompt('Please enter your surname', 'Enter surname');

alert('Your forename is ' + forename + ' and your surname is ' +
surname);

```

prompt() takes two arguments. The first is the message shown to the user, and the second is the text that starts off in the input box. Whatever the user types is handed back, and here we store it in a variable.

Notice how the alert message is built by joining pieces together with the + sign: some text, then the value of a variable, then more text, then another variable. This is called concatenation, and we will look at it properly in Chapter 9 (Strings).

12. Here is the whole page:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Exercise 3</title>
</head>
<body>
  <div id="myImgDiv"><p>My manual p tag here</p></div>

  <script>
    let image = "<img src='visa.png' />";
    let div = document.getElementById('myImgDiv');

```

```
div.innerHTML = '';
    div.innerHTML = image;
</script>
</body>
</html>
```

Notice how we dynamically create an image tag in JavaScript and insert it as a value into another HTML element, in this case the myImgDiv div. Again, when we create the HTML element, we must enclose it within quotes as a string.

There is one more thing worth studying in that image tag. Because we used double quotes around the whole tag, the src attribute inside it cannot also use double quotes. You cannot nest the same type of quote inside itself, or JavaScript will think the string has ended early and the code will fail. There are two ways round this. Either use the other type of quote for the inner ones, as we did above with single quotes, or escape the inner quotes using a backslash ().

So either of these lines would work perfectly well:

```
let image = "<img src='visa.png' />";
let image = "<img src=\"visa.png\" />";
```

The escape character tells JavaScript that the quote coming straight after it is not the matching partner of the one that opened the string. It should simply be included as part of the text. There is more on quotes and escaping in Chapter 9 (Strings).

Chapter 3 — ARRAYS

- Definition
- The two types of arrays
 1. Numeric (index)
 - the `Array()` constructor
 - square brackets
 - Assigning values to a numeric array
 - Retrieve values from a numeric array
 - Modify an array using indexes
 2. Associative arrays (also known as hash maps)
 - Retrieve values from an associative array
 - Assign and update values in an associative array
 - Using Dot notation
 - Using Bracket notation
 - Initialising the array with data
 - Updating values
 - The difference between an associative array and a JSON object
- Multi-dimensional arrays
 - Multi-dimensional numeric arrays
 - Multi-dimensional associative arrays
 - How to assign values to a multi-dimensional array
 - How to retrieve values from a multi-dimensional array
- Looping through arrays
 - Looping through a numeric array
 - Looping through an associative array
 - Looping through a multi-dimensional array
 - a) Loop through a multi-dimensional index array
 - b) Loop through an associative multi-dimensional array
- Rest parameters and the spread operator
- Array properties
 - `length`

- prototype
- constructor
- prototype.length
- Array methods
- True array methods and associative arrays

Definition

An array is similar to a variable in that it is like a virtual container in computer memory to store things, with the only difference that you can store multiple things at once. The elements you store in an array can be any data type e.g. Numbers, Strings, Booleans, Functions, Arrays, Objects. These elements as well as their keys (indexes) can also be stored in variables and passed into the array dynamically. Multiple elements should be separated by commas, with the trailing (last) comma (after the last element) being optional. When an array contains another array, we have an array structure known as a multi-dimensional array. In such a structure, we refer to the inner array as a child, nested or sub array, while the outer array can be referred to as the parent array. There are two types of arrays and their differences lie in how the indexes (also known as keys) of the items they contain (known as elements) are allocated.

1. Numeric arrays aka indexed arrays
2. Associative arrays

Each element in the array is referenced by a key which marks its spot in the array. For a numeric array, the key is a number (also referred to as an index) and for an associative array, that key is a small string of text, also referred to as a name. Let's talk about the syntax, structure and application of the two array types.

What marks the spot of each item

Numeric array — the key is a number

```
let fruits = ['mango', 'apple', 'guava', 'kiwi'];
```



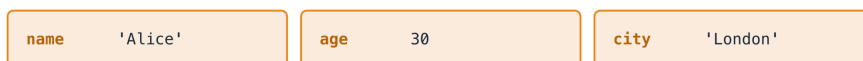
JavaScript numbers
these for you

`fruits[2]` gives you 'guava'. Counting starts at 0, so the last item is at index 3.

`fruits.length` is 4 — a plain count, not zero-based.

“Associative array” — the key is a name (really a plain object)

```
let person = { name: 'Alice', age: 30, city: 'London' };
```



you choose these names yourself

`person.age` is 30. But `person.length` is **undefined** — objects have no `.length`. Use `Object.keys(person).length` instead.

- Figure 3.1 — What marks the spot of each item*

THE TWO TYPES OF ARRAYS

1) Numeric arrays aka indexed arrays

Numerically-indexed arrays, also referred to as numeric arrays are arrays whose indexes, or keys are number based. JavaScript uses the so-called zero-based numbering of array indexes, which means that the numbering of the keys of arrays starts from zero. When creating an array, you do not need to specify the keys as JavaScript will do that for you automatically. This numeric array is the true array of JavaScript, and its parent is `Array.prototype` i.e. the prototype of the `Array` constructor. Do not worry if you do not know what a constructor means right now. I will explain that in Chapter 12 where you will learn all about objects and classes. Just bear with me, follow the code examples I provide as we go along now, and though they may include some objects, I will break it all down so that by the time you get to Chapter 12, you will be a master of it. If you prefer to just go to Chapter 12 to read the introduction before coming back here, that will also be a wise idea, but you do not have to do so to follow along here. I need you however, to remember this important point; that the parent of a numeric array in JavaScript is the prototype (blueprint from which other objects are formed) property of JavaScript's built-in `Array` object. This makes numeric arrays the true array in JavaScript, and I will explain more shortly how that makes numeric arrays behave differently from the other type of array—associative arrays—which are not true arrays in JavaScript because their parent is the prototype property of JavaScript's (built-in) `Object.prototype`. Do not mistake this to mean that they are not related, because

they are very related. Let me show you how.

When we speak of objects in programming, also referred to as object-oriented programming (OOP), if an object extends another, it is said to be the child of the object it is extending, and it inherits properties from that parent. The parent of associative arrays is `Object.prototype` which is the parent of objects in JavaScript, while the parent of numeric arrays is `Array.prototype`. However, `Array.prototype` in turn inherits from `Object.prototype`. See associative arrays in JavaScript as the uncle of numeric arrays. There is no such official relationship in JavaScript or any programming language as uncle or nephew, but I am giving you this as a symbolic way to understand the relationship between these two array types better. If `Array.prototype` and associative arrays both inherit from (have the same parent) `Object.prototype`, then the two of them are like siblings. Also, since `Array.prototype` has a child that is a numeric array, a good way to remember their relationship is to see a numeric array as the nephew of an associative array, and a grand child of `Object.prototype`. Hold this thought as we proceed, and all will become crystal clear, I promise. Let's proceed with learning how to create a numeric array. There are two ways to create a numeric array, and these are either by using the constructor of the built-in `Array` class, or the easy shorthand way which is by using square brackets.

With the `Array()` constructor

```
let fruits = new Array('mango', 'apple', 'guava');  
  
console.log(fruits);  
  
Outputs: ['mango', 'apple', 'guava']
```

With square brackets

```
let fruits = ['mango', 'apple', 'guava'];  
  
console.log(fruits);  
  
Outputs: ['mango', 'apple', 'guava']
```

Assigning values to a numeric array

In the above examples, we initialised the variable with elements already added to it. But you can create an empty array and then dynamically add elements to it later when you need to. You will find that this is a very common way of working. This is because you will not always have the data immediately when you create the array, but rather, you will create the array in anticipation of data being available, and add them as they become available. Once you have created the empty array, there is a very handy method called `push()` which you will use to insert items into your array.

(A method is simply a function that belongs to something, in this case to the array itself, which is why you write it after a dot: `fruits.push()`.) The `push()` method can take a single item or several of them, separated by commas, and they can be of any type: strings, numbers, booleans, even other arrays. Let's see how to do that:

```
// create an empty array as normal
// using the Array() constructor
// or the square brackets as we have seen above
let fruits = new Array();
// OR
let fruits = [];

// add single or multiple items to the array
fruits.push('mango');
fruits.push('apple', 'guava', 'kiwi');

console.log(fruits);
```

Outputs: ['mango', 'apple', 'guava', 'kiwi']

You can also target a specific index/key to add a element to in an existing array. If there is no element at that key, the element will be added to the key, but if there is already an element at that key, the new element will replace the previous element. For example:

```
let fruits = ['mango'];
fruits[0] = 'apple';

console.log('One item: '+fruits);

fruits[1] = 'guava';

console.log('Two items: '+fruits);
```

The output of the above code is:

One item: apple Two items: apple,guava

We start by creating an array of fruits with one item 'mango' in it. Next, we add an item 'apple' specifying that we need it at the first index. However, that first index already has the 'mango' since we know that being the only item in fruits, it will automatically be occupying index 0 (the first index). So, 'apple' replaces 'mango' at that index. When we check what's inside fruits array, we see it has only one item, 'apple'. Next, we add a new item 'guava' and specify that we want it at the index 1, since there is nothing in fruits at that index (apple is at index 0), guava is added at index 1. Now when we view the contents of fruits, we see it has both apple and guava in it. Adding items to specific indices is okay when you know the positions of existing elements in the array, but there are times when it is hard to tell. You run the

risk of unintentionally overriding an item in the array if you are not sure. In these cases, it's better to use `push()` to have that item appended as the last element in the array.

Retrieve values from a numeric array

To retrieve values from a numeric array, it depends on how you wish to retrieve the data. You may want to retrieve only a single element or you may wish to loop through the array and retrieve or display every element in it. If you just want to grab only a single element from an index in the array, it's quicker to use the bracket notation and the index number. The bracket notation will also work with strings too—more on this in Chapter 9 (Strings). Here is how to use the bracket notation to retrieve array values:

```
let fruits = [];  
  
fruits.push('mango');  
fruits.push('apple', 'guava', 'kiwi');  
  
console.log(fruits);  
  
Outputs: ['mango', 'apple', 'guava', 'kiwi']
```

To grab the item at index 3, do it like so:

```
// this will give you 'kiwi'.  
fruits[3];
```

If you wish to grab everything in the array, whose content and length you may not always know, you have to use a loop statement to do so. You can look under the looping section for a full explanation and demonstration of how looping works and the types of loops. For now here is how to use the for loop to extract all the items from the fruits array.

```
let fruits = [];  
  
fruits.push('mango');  
fruits.push('apple', 'guava', 'kiwi');  
  
// prepare new array to store your retrieved fruit items in  
let fruitBasket = [];  
  
//console.log(fruits.length);  
for (let i = 0; i < fruits.length; i++)  
{  
  console.log("adding " + fruits[i] + " to my basket");  
  fruitBasket.push(fruits[i]);  
}
```

```
// view the contents of your fruit basket
console.log(fruitBasket);
```

Outputs:

```
['mango', 'apple', 'guava', 'kiwi'];
```

Modify an array using indexes

This will not work with strings, but it will work with arrays. Here is how you can do that:

```
let myArray = [1, 5, 25];
myArray[1] = 45;
```

This means you have targeted the myArray array's value at the index of 1, which is 5, and changed its value to 45. The array myArray will now contain: [1, 45, 25];

2) Associative arrays

Unlike a numeric array which is basically an ordered (numbered) list; an associative array is an array whose keys are named properties (strings), meaning, instead of numbers, the keys are strings. Unlike most other programming languages which have real associative arrays—arrays whose keys are strings, JavaScript does not have associative arrays. At least it does not have it in the real sense of the word. JavaScript arrays are designed for ordered lists using numbers as keys (called indexes). If you want to store values by name instead of number, you should use an object, not an array. This is a very common misconception that confuses programmers new to JavaScript. So what has come to be known as an associative array in JavaScript is actually an object. That, and the fact that the parent of the associative array is JavaScript's built-in `Object.prototype` and not `Array.prototype`, is the reason why associative arrays are not true arrays in JavaScript. The true arrays are numeric-indexed arrays, whose parent is `Array.prototype`. Once again, associative arrays are simple plain objects and the keys are not numbers. Respect this distinction between the two, and you will be fine. For example, do not create an array in the numeric style using strings instead of numbers. Remember I mentioned earlier that because of their different parents, both array types behave differently. Let me explain why not understanding the difference between the two can cause problems for you. While JavaScript would still let you assign named properties to an array, they don't behave like real array elements. For example, they won't show up when you loop through the array, and no matter how many elements there are in the array, that number will not be reflected in the `.length` property. This is better demonstrated than explained. Here is an example of an associative array being created in

the same way you would create a numeric (indexed) array, instead of using an object:

```
let student = [];

// Adding "associative" (named) keys
student["name"] = "Amina";
student["age"] = 12;
student["grade"] = "6";

// Adding one real array element
student[0] = "Math";

// Let's examine the array
// Output is: [ 'Math', name:
// 'Amina', age: 12, grade: '6' ]
console.log(student);

// Output is: 1 – only the numeric item is counted
console.log(student.length);

// Loop through the array
for (let i = 0; i < student.length; i++) {
  // Output: only "Math" gets printed
  console.log(student[i]);
}
```

What happens is:

- Only the numeric index 0 (the subject "Math") is stored as a real array element.
- The "name", "age", and "grade" properties are stored as custom properties, not actual array properties.
- Checking the length of the array with `student.length` returns 1, where `.length` is a property of arrays that shows you the number of elements in an array. The value here is 1 because only index 0 is counted.
- Even worse, for loops, `.map()`, `.forEach()`, etc., which are meant to work on real arrays, will ignore those named keys.

The correct way to create an array with named keys in JavaScript is to use objects. We will learn all about objects in Chapter 12 when we learn about Object-Oriented Programming (OOP). For now just understand that to create a plain object in JavaScript, you assign a variable to a pair of curly braces (`{}`) inside which you list key-value pairs, where the keys are on the left and their values are on the right. Each key is separated from its value by a colon (e.g. `myKey: myValue`), and multiple key-value pairs are separated by commas. Take care not to confuse this with a block, which also uses curly braces but groups code rather than storing data. We drew that distinction in Chapter 2, under "Blocks are not objects".

Let's convert the above faulty student example into an object:

```
let student = {  
  name: "Amina",  
  age: 12,  
  grade: "6",  
  subjects: ["Math", "English"]  
};
```

The name of the array (object) is student. An example of a key is name, and its value is the string "Amina". Notice how the values are separated by commas, and the last element may or may not have a trailing comma.

```
// Output is: Amina  
console.log(student.name);  
  
// Output is: 2  
console.log(student.subjects.length);  
  
// Output is Math and English,  
// each printed on its own line.  
// (The => arrow is a short way of writing a function.  
// We come to those in Chapter 7.)  
student.subjects.forEach(sub => console.log(sub));
```

The length property will not work on this object, because it is a property of real (numeric) arrays, not objects. Notice that though we cannot use the .length property on the whole student object itself, we use it on the subjects property, which is a numeric array. If you ever find yourself giving an array "named properties" (like array["name"] = "value"), stop and ask: "Should this really be an object instead?" Arrays are for ordered lists (a list with numbers), and objects are for key-value pairs. Mixing them leads to bugs where your data won't behave the way you expect. In some JavaScript references, you will see associative arrays being defined as an array whose properties are strings, and in others, as an array with named properties, or arrays that have properties by name. These are all essentially describing JavaScript objects, not true arrays, and they're all saying the same thing in different words. Having said all that, let us agree that when people say "associative array" in JavaScript, they are essentially speaking of a plain object with key-value pairs, even though it's not really an array. The keys of this object are strings (or symbols), and the values can be of any type. Enough of this repetition, for I know you have got it now. Congratulations, you have hit a milestone and are well on your way towards JavaScript mastery. Let's forge onwards and look at the syntactical aspects of an associative array. The property name of the object must be a valid identifier. Here are the syntax rules of an identifier:

- It cannot contain spaces

- The named properties may or may not have quotes
- It cannot start with a number (but can have a number in the body)
- Cannot contain spaces or symbols (special characters) like -, ., !, # etc, unless quoted
- The only special characters allowed are _ and \$
- If your property name has spaces or special characters like - (hyphen), it's not a valid identifier, and must be quoted:

Here is a valid example:

```
const person = {
  "name": "Alice",
  "age": 30,
  occupation: "Developer",
  _id: 12345,           // valid
  $status: "active",    // valid
  "other-name": "Gray", // valid (quoted)
  "home town": "London", // valid (quoted)
};
```

The property name "other-name" would otherwise be invalid if it was not quoted, because it contains a hyphen. Similarly, the "home town" has a space in it, so it is quoted. These two properties break the identifier rules, and are fixed by quotes. However, because they are fixed by quotes, it changes the way their values can be retrieved. I will address that shortly when I talk about retrieving and updating the values of associative array properties.

With JavaScript objects, whether you quote the properties or not, it makes no difference — they're all stored as strings. That is why, to retrieve their values, doing it like so:

```
person.name or person['name'] or person["name"]
```

will all work just fine.

If you are wondering why `person.name` and `person["name"]` both work, it's because in JavaScript you can use both bracket or dot notations to access object values. Notice that in this person object, some keys are quoted, like "name", and some are not, like occupation. However, retrieving their values will work in the same way—which is by using the bracket or the dot notation. The only exception is with the keys that break the identifier rules and are quoted, whose values I will talk about how to retrieve shortly.

Let's look at how to retrieve values from objects (associative arrays) next.

Retrieve values from an associative array

In JavaScript, associative arrays are just objects with named properties — so we can use the same rules for accessing object values to access associative array values. We can therefore do so by either using the dot (.) notation, or the bracket notation ([]). Both of these lines will work:

```
person['name'];
person.name;
```

Let's talk about these notations, and when to use which. Remember, above we saw how if your property name has spaces or special characters like - (hyphen), it's not a valid identifier, and must be quoted. Even if you use quotes to create the property (like "other-name"), it still isn't a valid identifier—so in that case, you cannot use dot notation to access it. Rather, you should use bracket notation, like this:

```
obj["other-name"]
```

Let's look at the differences between the two types of notations.

Dot Notation

- Uses a period (.) followed by the property name, like `obj.propertyName`.
- It is simpler and more readable.
- The property name as we know, must be a valid identifier (i.e., it cannot contain spaces, start with a number, or have special characters other than `_` and `$`).

Bracket Notation (`obj['property']`)

- More flexible because the property name can be a string or a dynamic expression.
- It is required:
- When the property name contains special characters or spaces. For example:

```
const obj = {
  "first-name": "Tom",
  "aunt": "Polly"
};
```

```
console.log(obj["first-name"]);
// will return: Tom
```

- When the property name is stored in a variable. For example:

```
let key = "aunt";
console.log(obj[key]);
// will return Polly
```


-When you need to access properties dynamically inside a loop or function. For example:

```
// Iterates over all values
// displaying the value at each key
for (let key in obj)
{
    console.log(obj[key]);
}
```

When to Use Which

- Prefer dot notation when possible, as it is more readable and concise.

Use bracket notation when:

- The property name is not a valid identifier.
- You need dynamic property access.

A dynamic key access is if you need to reference a key of the object by referencing it not directly, like using a variable.

```
const person = {
    "name": "Alice",
    "age": 30,
};
```

```
let key = "age";
console.log("Dynamic age value is: "+person[key]);
console.log(person.name);
```

The output will be:

Dynamic age value is: 30 Alice

We referenced the key dynamically from a variable: `person[key]`. For this, we had to use the bracket notation as the dot notation like so:

`person.key`

would not have worked.

Here is another example demonstrating how and when to use the two notations:

```
const person = {
    "name": "Alice",
    "age": 30,
    occupation: "Developer",
    _id: 12345,           // valid
    $status: "active",   // valid
}
```

```

"other-name": "Gray",    // valid (quoted)
"home town": "London",  // valid (quoted)
};

console.log(person.age);
console.log(person.occupation);
console.log(person._id);
console.log(person.$status);
console.log(person["other-name"]);
console.log(person["home town"]);

```

Will work fine and produce the output:

30 Developer 12345 active Gray London

But `console.log(person.other-name);`

Will not throw an error, but will hand you back NaN, which in JavaScript is a special value meaning “Not a Number”. You can use dot notation (`object.key`) only if the property name is a valid JavaScript identifier (no spaces, no hyphens, no starting numbers). If the name contains symbols or spaces, you must use bracket notation. JavaScript interprets `obj.other-name` as a math expression:

`person.other - name`

and since you're trying to subtract a variable name (which likely isn't a number) from `person.other`, the result is not a number — hence NaN. Instead, you must use a bracket notation, like this:

```
console.log(person["other-name"]);
```

If you're ever unsure which to use, try bracket notation. It always works, even when dot notation does not.

Assign and update values in an associative array

Same as with retrieving values, you can assign and update values in an associative array using either the Dot or Bracket operators. Which operator you use will depend on the nature of the key. Remember in the introduction to associative arrays above we saw how keys made of invalid identifiers should be wrapped in quotes, and how these have to therefore be retrieved or referenced using square brackets. Well, ‘referencing’ applies to both when assigning and when updating the values of these keys.

Using Dot notation

```

let associativeArray = {};

associativeArray.key = "VALUE";
console.log("The value of key is: "+associativeArray.key);

```

The output will be: The value of key is: VALUE

Using Bracket Notation

```
let associativeArray = {};  
  
associativeArray["key"] = "VALUE";  
  
// example with a dynamic key  
let dynamicKey = "age";  
associativeArray[dynamicKey] = 25;  
  
console.log(associativeArray.key);  
console.log(associativeArray.age);
```

The output will be: VALUE 25

The 'age' key is dynamic because we are assigning the key from a variable (dynamicKey).

Initialising the array with data

The above examples start an array from a blank slate, but you can also create an array that already has elements in it from the outset.

```
const person = {  
  "name": "Alice",  
  "age": 30,  
  "favourite colour": "blue",  
};  
  
person.country = "Holland";  
console.log(person);
```

The output will be:

```
{  
  name: 'Alice',  
  age: 30,  
  'favourite colour': 'blue',  
  country: 'Holland'  
}
```

In this example, we create an array with some data already in it, then add a new key-value pair, country, that did not exist before. Updating the value of a key that already exists is what we look at next.

Updating values

Let's look at examples of how to update associative arrays:

```
//modify "favourite colour" using Bracket notation
person["favourite colour"] = "black";
console.log(person);

// modify age and country using Dot notation
person.age = 40;
person.country = "England";
console.log(person);
```

The output will be:

```
{
  name: 'Alice',
  age: 30,
  'favourite colour': 'black',
  country: 'Holland'
}

{
  name: 'Alice',
  age: 40,
  'favourite colour': 'black',
  country: 'England'
}
```

Let me point out a few things which you probably already noticed. When updating the value of the "favourite colour" key, we use a bracket notation

```
person["favourite colour"] = "black";
```

This is because using a Dot operator will not work, since the "favourite colour" key contains an invalid identifier; a blank space. The other identifiers like 'country' and 'age' are valid identifiers and are therefore updated using the Dot operator e.g:

```
person.age = 40;
person.country = "England";
```

The difference between an associative array and a JSON object

There is a difference between a JSON object and an associative array in JavaScript, though they can look similar. As we learned above, an associative array in JavaScript is just a regular JavaScript object. JSON (JavaScript Object Notation) is a data format used for representing structured data as text. JSON is often used for data exchange between a server and a client. Here is an example of a JSON object as a string:

```
{
  "name": "Alice",
  "age": 30,
```

```
"occupation": "Developer"
}
```

While a JSON object must use double quotes for keys and strings, for example:

```
{"key": "value"}
```

JavaScript objects can use single or double quotes for strings, and their keys can be unquoted as long as they are valid identifiers.

Multi-dimensional arrays

The structure of the arrays we have seen so far—whether numeric or associative—is single-dimensional, which means they are flat arrays, or arrays containing no other arrays themselves, just items. In JavaScript, a multi-dimensional array is simply an array that contains one or more arrays as its elements. You can think of it as a “nested” array—an array within an array. Just like single-level arrays, multi-dimensional arrays can be made up of:

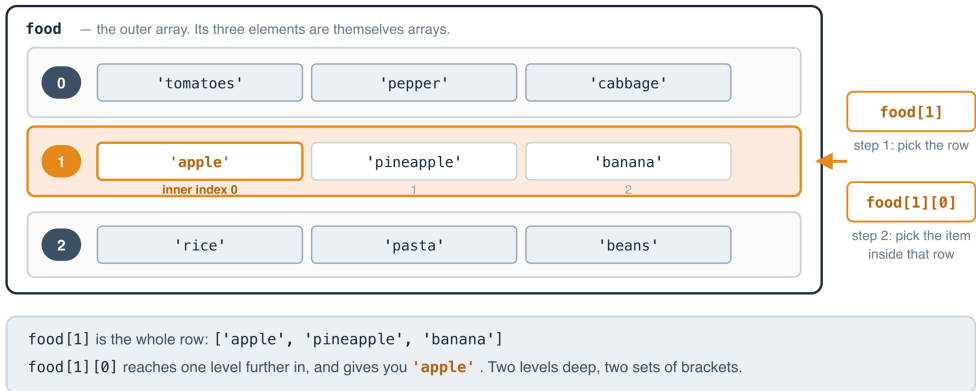
- Numeric (indexed) arrays—arrays where items are accessed by number.
- Associative arrays—objects where values are accessed using named keys.
- A mix of both numeric and associative arrays

So you can have a mix of both types of arrays in the same multi-dimensional structure. But it’s helpful to understand the difference before combining them in your code so you don’t get confused on how to handle them, since they all behave differently. Let me introduce to you certain terms often used when it comes to arrays and their levels, so you will understand them whenever you hear or read about them. Multi-dimensional arrays are also sometimes referred to as multi-level arrays. The regular array we dealt with above is a single-dimensional array, which is a flat array containing elements, none of which are arrays themselves. This has only one level, so-to-speak. But when we talk of multi-dimensional (multi-level) arrays, the levels can go deep, and we can have two-dimensional (2D), three-dimensional (3D) arrays and so on. A two-dimensional array as you can probably guess from the name, is an array that contains elements, one of which is an array itself. Now we are talking about an array that goes two levels deep—one out, one in—hence the name two-dimensional. Any array (nested) inside another can also be referred to as a sub-array or a child array of the one it is inside of. There is also the three-dimensional array which is a structure where the sub array of the first array contains an element, and this element is also an array. It then becomes three levels deep. We can keep going deeper and deeper and the idea stays the same. It’s simple; with each level within the first outer array you need to go to find another (nested) array,

the deeper the dimensions go. I will now break down what a multi-dimensional array looks like for both numeric and associative arrays.

One set of square brackets for each level you go down

```
let food = [['tomatoes', 'pepper', 'cabbage'], ['apple', 'pineapple', 'banana'], ['rice', 'pasta', 'beans']];
```



- Figure 3.2 — One set of square brackets for each level you go down*

Multi-dimensional numeric

With a numeric multi-dimensional array, each element inside it is itself an array of elements, and each of those rows (arrays) of elements is automatically assigned an index number beginning from zero (0). The key phrase to remember here is this: “each row (element) within this array is itself an array”. Here is an example:

```
const numeric_2d = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9],  
];
```

```
console.log(numeric_2d[1]);
```

This console statement writes the value of the second element in the `numeric_2d` array (`numeric_2d[1]`), counting the index from 0, to the console. The output will be

[4, 5, 6]

Let's dig deeper. The following code:

```
console.log(numeric_2d[1][2]);
```

Will output: 6

This is because `numeric_2d[1][2]` gets the value of the row at index 1 (`numeric_2d[1]` counting from 0), which we know is the second element `[4, 5, 6]`, and then gets the element at index 2 from that. This element at index 2 in `[4, 5, 6]` is 6 if we count the index from 0. This is why `numeric_2d[1][2]` results in 6. In the example above, `numeric_2d` is a 2D numeric array. It is said to be 2D (two-dimensional) because it is two levels deep—the array outside, and the ones inside. A 3D array will mean 3 levels deep. The following is a 3D array:

```
const matrix_3d = [
  [1, 2, 3],
  [4, 5, 6],
  [
    [1, 2, 3, 4],
    [1, 8, 3, 9],
  ]
];

console.log(matrix_3d[2][1][2]);
```

This will output: 3

Multi-dimensional associative

A multi-dimensional associative array in JavaScript is simply an object with named keys. The outer/parent object has a child or children (nested) objects that are referenced by named keys inside of it. Here is an example:

```
let assoc_2d = {
  john: {
    name: "John",
    country: "England"
  },
  david: {
    name: "David",
    country: "USA"
  }
};
```

A very common confusion among developers is to mistake a structure like the following for a multi-dimensional array:

```
let clients = [
  { name: "John", country: "England" },
  { name: "David", country: "USA" }
];
```

But that is wrong. This is a real JavaScript array, where each element is an object. You can loop over it directly using loops like `for`, `for...of`, `.forEach()`, `.map()`, etc. This is because each item in the array is indexed with a number: `clients[0]`,

clients[1], etc. It is not an "associative array" in the technical sense, but rather just an array of objects. Whereas the first structure above

```
let assoc_2d = {
  john: {
    name: "John",
    country: "England"
  },
  david: {
    name: "David",
    country: "USA"
  }
};
```

is a plain object, and not an array. It is a plain object ({}) with named keys (john, david). Its keys are strings, not numbers. You cannot directly loop over it with `.forEach()` or `for...of` etc. To do so, you have to convert it into an array first of all, before you can loop through it. See [Looping through arrays](#) below for how this conversion of associative arrays and looping is done. Where the confusion comes from is the fact that programmers sometimes refer to plain objects as associative arrays—especially programmers from other languages, but in JavaScript they’re just plain objects with named keys. If you always keep that in mind, you will know how to manipulate the data, because the type of structure it has will determine how you interact with it. Knowing that the last structure above is an “associative array”, we can therefore consider it to be a two-dimensional array. This is because it has two levels; the first level being the `assoc_2d` parent object itself, and the second level consisting of the two objects inside of it denoted (marked) by the `john` and `david` keys.

To create a 3D associative array—that is, a group of nested objects, where the value of one of the keys of any of the nested (children) objects is itself another object—here is how to do it:

```
let assoc_3d = {
  john: {
    name: "John",
    country: "England",
    parents: {
      dad: "Peter",
      mum: "Jacqueline"
    }
  },
  david: {
    name: "David",
    country: "USA",
    parents: {
      dad: "Mark",

```



```
}  
};
```

```
console.log("His dad is: " + assoc_3d["david"].parents.dad);
```

When using an object literal (`{ ... }`)—which is what an associative array is—you must have key-value pairs inside the curly brackets to denote new elements (objects). Here, the second element is marked by the `david` key, and this second object David has a property (key) `parents`, whose value is also an object. This makes the `parents` sub-array the third level that makes the `assoc_3d` array a 3D array. Running the above code results in the following being printed to the console:

His dad is: Mark

Mixed multi-dimensional arrays

In practice, you will not always get your data structured as plain objects, or as all numeric arrays. Quite often, you will come across data with complex structures, like an outer array, with objects inside of it, or nested plain objects that have arrays as the values of some of their keys. JavaScript is so flexible that it will let you combine numeric arrays and associative arrays as much as you like, as long as you follow the written syntax correctly. Here are some examples:

```
const mixed_2d = [  
  { name: "Alice", age: 30 },  
  { name: "Bob", age: 25 },  
  { name: "Charlie", age: 35 },  
];
```

Here, `mixed_2d` is an array of objects—each object holds `name` and `age` as key-value pairs. We use a numeric index from the outer array just like a numeric array to select an object, and then we use a named key to get the info we need from that specific object. For example:

```
console.log(mixed_2d[1].name);
```

The output of this will be Bob

Here is another mixed example that involves a numeric array that contains a mix of both numeric arrays and objects:

```
const data = [  
  ["Math", 95],  
  { subject: "Science", score: 88 },  
  ["English", 76],  
  {  
    john: { name: "John", country: "England" },  
    david: { name: "David", country: "USA" }  
  }  
];
```

```
];

console.log(data[0][0]);
console.log(data[1].score);
```

This structure mixes both styles—the first and third items are numeric arrays, and the second is an object. The first `console.log()` displays the value of the first key of the first element in the data array (`data[0][0]`), which will be `Math`. The second `console.log()` statement displays the value of the `'score'` key of the second element (`data[1].score`). Running the above code results in the following being printed to the console:

Math 88

Let's look at a bigger structure of mixed data

```
let clients = [
  {
    name: "John",
    country: "England",
    parents: {
      dad: "Peter",
      mum: "Jacqueline"
    },
    children: [
      "Jim",
      "Olly"
    ]
  },
  {
    name: "David",
    country: "USA",
    parents: {
      dad: "Mark",
      mum: "Joyce"
    },
    children: [
      "Linda",
      "Nina"
    ]
  }
];
```

The structure of this `clients` array is complex because it is mixed on several levels. However, it is nothing complicated once you take a closer look at the structure. It is simply a mix of plain objects and arrays, having the same syntax structure we have learned in this chapter. The key in mastering arrays lies in identifying the structure so you know how to manipulate it to get out the data you need. That is all it is about. The only confusion as I mentioned above, is when programmers fail to make that distinction in data types and go ahead and put objects and arrays in the

same pot, so-to-speak. Basically, as long as you understand that you can loop through a (true) array but you cannot directly loop through an object ('associative' array), then you will be fine. I can hear you asking me how you would then loop through an array that has mixed structures, like an outer array, and a nested group of objects which in turn have properties (keys) whose values are arrays etc. The answer is the purpose of this section, and it is simple. The key is in knowing the structure of the data, and in programming, you will never have to worry about that because you will always be told what structure to expect your data to be in. Once you know the structure—say the outer-most part is an array, then you would start by looping through it like you would any array. Then as you iterate through each deeper level, if you know the data at that level is an object, you would reference the data as you would do with an object, or if you need to loop over it, you know you need to convert it to an array before running a loop on it. You have the tools at your disposal for looping through, and converting the data. Again, to see how to convert objects to arrays, visit the "Looping through arrays" section further below.

How to assign values to a multi-dimensional array

Working with multi-dimensional arrays is as simple as viewing them as a parent-child structure—with the children being the nested arrays, and the outer array being the parent. Use the bracket notation to reference the keys and sub keys. Basically, you have to use one square bracket for each nested array, beginning with the outer (parent) array. Once you can access an element within an array, assigning a value to it is done in the same way as you would do for a normal (single-dimension) array. This is easier demonstrated than explained. What I will explain here will cover how to assign values as well as how to update values in multi-dimensional arrays. Take the following 2D array example:

```
let myArray = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

myArray[1].push(1, 2, 3);

let myData = myArray[1];

// will return [4, 5, 6, 1, 2, 3]
console.log(myData);

// OR

// will return [4, 5, 6, 1, 2, 3]
console.log(myArray[1]);
```

In this case, we added three elements (numbers) to the second element (index 1) of the myArray array, which is a sub array. By placing 1 in the bracket, we indicate that we want the element at index 1 (counting from 0) of our myArray array, which is the array [4, 5, 6]. Having selected that child array, we then use the push() method to add some elements to it. When we then view the contents of that child array, we find that it has an updated value of [4, 5, 6, 1, 2, 3].

To change the value of a child array entirely, we can do it like so:

```
myArray[2][2] = 10;
```

```
console.log(myArray[2]);
```

This will update the value of the 3rd element (index 2 counting from 0) of the 3rd child array of myArray.

```
let myArray = [  
  // ...  
  // ...  
  [7, 8, 9]  
];
```

The value of that element which was previously 9 will now be updated to 10. Hence

```
console.log(myArray[2]);
```

will return [7, 8, 10]

How to retrieve values from a multi-dimensional array

You may have noticed that I have already touched on how to access/retrieve data from multi-dimensional arrays when I introduced them above. Even when we looked at "Mixed multi-dimensional arrays", I showed you how to access the data in the mixed structure. It was only normal for me to show how to access the nested arrays in order to show you what the parent-child hierarchy structure looked like. Let us refresh our minds again. I will start with a numeric nested array.

```
let myArray = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];
```

```
let myData = myArray[1][0];
```

```
console.log(myData);
```

The output of this code will be 4

Notice how we use two square brackets. The first one references the index of 1, and this means we want the second element of our myArray array (counting index

from 0), which is the array [4, 5, 6]. We then use a second square bracket which references the key 0. This tells the JavaScript engine that you wish to retrieve the value in this array that is at the key 0. The first element (at key 0) of the [4, 5, 6] array is 4.

Let me now repeat the mixed example from above, an array whose elements are objects. Note that I am naming it `arrayOf2dObjects` rather than `assoc_2d`, because as we established earlier this is a real numeric array, not an associative array. Only the elements inside it are objects.

```
const arrayOf2dObjects = [
  { name: "Alice", age: 30 },
  { name: "Bob", age: 25 },
  { name: "Charlie", age: 35 },
];

console.log(arrayOf2dObjects[1].name);
```

The output here will be Bob. Here we get the second element of the `arrayOf2dObjects` array (`arrayOf2dObjects[1]`), counting index from 0, which is the object { name: "Bob", age: 25 }. Next, we get the value of its name property `arrayOf2dObjects[1].name` using the Dot operator. It's time to look at a structure that goes one level deeper, an array of objects where one of those objects holds an array of its own.

```
const arrayOf3dObjects = [
  { name: "Alice", age: 30 },
  { name: "Bob", age: 25 },
  {
    children: [
      { name: "John", age: 40 },
      { name: "Peter", age: 35 }
    ]
  },
];

console.log(arrayOf3dObjects[2].children[0].name);
```

The output: John.

This should be very clear to you now how to retrieve the values. In this example, we get the third element in the `arrayOf3dObjects` array, which is the following object containing an array. This is the first level with two more levels to go:

```
{
  children: [
    { name: "John", age: 40 },
```

```
{ name: "Peter", age: 35 }  
  ],  
},
```

Basically, this object contains a key 'children' whose value is an array. Next, we grab that children array, and say that we want to get the name property of its first element (children[0].name). That first element (counting from 0) is this object:

```
{ name: "John", age: 40 },
```

We can see that the value of its name property is 'John', which is why we got John written to the console.

Looping through arrays

Looping through a numeric array

There are multiple ways to loop through an indexed (numeric) array in JavaScript. You can use:

- for loop (different from for...in)
- forEach()
- for...of
- map() (Not common, but possible)

The three Object methods below also work on a numeric array, though they were really designed for objects, and that is where you will normally reach for them. They are shown here so you can see the whole picture in one place:

- Object.values() and forEach()
- Object.entries() and forEach()
- Object.keys() and forEach()

Take for example the following numeric array:

```
let arr = [  
  'Tom Sawyer',  
  10,  
  'Sid',  
  'Polly'  
];
```

i) Using a for loop

This is the traditional for loop, and it should not be confused with the for...in loop that is meant for looping through objects. Let's see how to use it. Here is the

syntax:

```
for (initialization; condition; increment/decrement) {  
  // Code to execute  
}
```

For example:

```
for (let i = 0; i < arr.length; i++)  
{  
  console.log(arr[i]);  
}
```

The output is: Tom Sawyer 10 Sid Polly

ii) Using a `forEach()` loop

This works well and it's more readable than the `for` loop. The syntax is:

```
arr.forEach(value => { console.log(value); });
```

The output is the same as that of the `for` loop above.

iii) Using a `for...of` loop

This works well and is more readable than the `forEach()` loop.

```
for (let value of arr)  
{  
  console.log(value);  
}
```

The output is the same as the examples above.

iv) Using `Object.values()` and `forEach()`

This works well and returns only the values.

```
Object.values(arr).forEach(  
value => {  
  console.log(value);  
});
```

The values without the keys actually result in the same output as the other loops above:

Tom Sawyer 10 Sid Polly

v) Using `Object.entries()` and `forEach()`

This works well and also provides the indexes of the values. The example below uses something you have not met before, so let us name it. The backticks in

`ex}`: `${value}` mark what is called a template literal. It is simply another way of writing a string, but with one very handy extra: anywhere you put `${...}` inside it, JavaScript works out what is in the braces and drops the result into the text for you. It saves a good deal of joining strings together with `+` signs. We will look at template literals properly in Chapter 9 (Strings).

```
Object.entries(arr).forEach(
  ([index, value]) => {
    console.log(`${index}: ${value}`);
  }
);
```

The result is:

0: Tom Sawyer 1: 10 2: Sid 3: Polly

These last two are not very common, though they work

vi) Using `Object.keys()` and `forEach()`

Works, but this returns the index numbers (as strings), so it behaves similarly to `for...in`. Here is an example:

```
Object.keys(arr).forEach(
  key => {
    console.log(arr[key]);
  }
);
```

vii) Using the `map()` method

It is used more to transform data than to loop. It returns a new array.

```
arr.map(value => console.log(value));
```

Looping through an associative array

In JavaScript, real arrays use numbered indexes and support array methods like `.forEach()` and `.map()` and many other methods which make looping through numeric arrays to get data from them easy and effective. On the other hand, if you are working with associative arrays, these are actually plain objects and not true arrays, so you cannot use these methods designed for (true) arrays on the objects. But there is a way to work around that limitation. To loop through associative arrays, you have to convert the object to an array, then loop through it as you would do on an array. JavaScript provides you with three special methods to achieve this. These methods are:

- `Object.keys()`,

- `Object.values()`, and
- `Object.entries()`.

Remember them—they will make your life easier. Learn more about how to use them in the section "True array methods and associative arrays" later in this chapter. I am going to discuss below how you can use any of the following ways to loop through associative arrays. So whenever you come across the use of the three methods listed above, just know that they are being used to first of all convert the object into an array, then another kind of array method for example `forEach()` or `map()` etc is used to loop over the resulting array. We are going to talk about the following approaches to perform the loops.

- `for...in` loop
- `Object.keys()` and `forEach()`
- `Object.values()` and `forEach()`
- `Object.entries()` and `forEach()`
- `for...of` with `Object.entries()`
- `map()` with `Object.entries()` (Not common, but possible)

i) Using a `for...in` loop

Note that this should not be confused with the traditional `for` loop which is meant to be used with numeric (indexed) arrays.

```
let arr = {
  name: 'Tom Sawyer',
  age: 10,
  brother: 'Sid',
  aunt: 'Polly'
};
```

Loop through all the elements and display the value at each key:

```
for (let key in arr)
{
  console.log(arr[key]);
}
```

The result of this will be:

Tom Sawyer 10 Sid Polly

ii) Using `Object.keys()` and `forEach()`

The `Object.keys()` method returns an array of an object's keys, which you can then iterate over using `forEach()`. Here is an example:

```
let arr = {
  name: 'Tom Sawyer',
  age: 10,
  brother: 'Sid',
  aunt: 'Polly'
};

Object.keys(arr).forEach(key => {
  console.log(arr[key]);
});
```

The result will be: Tom Sawyer 10 Sid Polly

iii) Using `Object.values()` and `forEach()`

If you only need the values and don't care about the keys, you can use `Object.values()`. It also returns an array, which you can then loop over using any of the (true) array methods e.g. `forEach()` in this case. Here's an example:

```
Object.values(arr).forEach(
  value => { console.log(value); });
```

The result will be: Tom Sawyer 10 Sid Polly

iv) Using `Object.entries()` and `forEach()`

If you want both keys and values, `Object.entries()` returns an array of `[key, value]` pairs. Here is an example:

```
Object.entries(arr).forEach(
  ([key, value]) => {
    console.log(`${key}: ${value}`);
  }
);
```

The result will be:

name: Tom Sawyer age: 10 brother: Sid aunt: Polly

v) Using `for...of` with `Object.entries()`

The `for...of` loop works well with `Object.entries()`. Here's an example:

```
for (let [key, value] of Object.entries(arr))
{
  console.log(`${key}: ${value}`);
}
```

The result will have the keys and their values like so:

name: Tom Sawyer age: 10 brother: Sid aunt: Polly

vi) Using map() (Not common, but possible)

Although map() is typically used for arrays, you can use it with Object.entries() to create a transformed array. Here is an example:

```
Object.entries(arr).map(
  ([key, value]) => console.log(value)
);
```

The result will be: Tom Sawyer 10 Sid Polly

Which of all these types of loops should you use? - Use for...in if you just want a simple loop. - Use Object.keys() if you only need keys. - Use Object.values() if you only need values. - Use Object.entries() if you need both keys and values. - Use for...of with Object.entries() for a cleaner approach.

Looping through a multi-dimensional array

a) Loop through a multi-dimensional index array

There are three ways to loop through a multi-dimensional indexed array. You can do it in any of the following ways:

- a nested for loop,
- a forEach() method, or
- the flat() method.

i) Nested for loop

```
let myArray = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

for (let i = 0; i < myArray.length; i++)
{
  for (let j = 0; j < myArray[i].length; j++)
  {
    console.log(myArray[i][j]);
  }
}
```

The result is each number printed on its own line:

1 2 3 4 5 6 7 8 9

ii) Using the forEach() method

```

myArray.forEach(
  row => {
    row.forEach(element => {
      console.log(element);
    });
  }
);

```

The result is the same, each number on its own line.

iii) Using the `flat()` method This method is for when you just need a single loop, that is, if you don't need to maintain the structure and just want to iterate over all the elements. This method flattens the array into a single-dimensional array before iterating through it.

```

myArray.flat().forEach(element => console.log(element));

```

The result is again the same, each number on its own line.

b) Loop through an associative multi-dimensional array

JavaScript does not have true multi-dimensional associative arrays like other programming languages like PHP. However, you can use an array of objects or a nested object to achieve a similar structure. That is basically what we did under the "Multi-dimensional associative" array section above. If you use an array of objects, you need to iterate differently than an indexed multi-dimensional array. Let's see some examples:

An array of objects

The outer array is a true array (with number indexes) while the inner elements are objects (associative arrays). The way to loop through this as we saw before in the Mixed multi-dimensional arrays section is to interact with the data as its structure guides you. Take for example the following array:

```

let myArray = [
  { a: 1, b: 2, c: 3 },
  { a: 4, b: 5, c: 6 },
  { a: 7, b: 8, c: 9 }
];

```

In this case, you should do your regular loop on the first (outer) array using any of the ways you know how to loop through an array on it. Within the iteration, because you know the inner elements will be objects, act accordingly and loop through the objects using any of the ways to loop through an object. In this case we use a `forEach()` loop for the true outer array, and a `for...in` loop to loop through the inner objects.

```
// Loop through the array and access object properties
myArray.forEach(obj => {
  for (let key in obj) {
    // Display each value
    console.log(obj[key]);
  }
});
```

The result is each number printed on its own line, 1 through 9.

A nested object

If you structure it as a nested object, you'd need a different approach. You should use the `for...in` loop designed for looping through objects. We are going to nest the loop for each nested level. Here is an example:

```
let myObject = {
  row1: { a: 1, b: 2, c: 3 },
  row2: { a: 4, b: 5, c: 6 },
  row3: { a: 7, b: 8, c: 9 }
};

// Loop through the outer object
for (let row in myObject)
{
  // Loop through the inner object
  // properties & display each value
  for (let key in myObject[row])
  {
    console.log(myObject[row][key]);
  }
}
```

Rest parameters and the Spread operator

When working with arrays in JavaScript, understanding rest parameters (`...rest`) and the spread operator (`...spread`) is crucial. These two concepts provide a flexible way to handle array elements, whether you're collecting values into an array or expanding an array into individual elements.

a) Rest Parameters (`...rest`) – Collecting Values into an Array

Rest parameters are used in function parameters to collect multiple arguments into a single array. They are useful when you don't know how many values will be passed to a function. Example: Using Rest Parameters to Collect Arguments into an Array

```
function combineStrings(...words) {
  // Joins collected words into a
  // sentence
```

```

return words.join(" ");
}

// Output: "Hello world from
// JavaScript"
console.log(combineStrings("Hello",
    "world", "from", "JavaScript"));

```

Rest parameters collect multiple values and turn them into an array, making it easier to work with them using array methods like `.map()`, `.filter()`, and `.reduce()` etc.

- When to Use Rest Parameters?
 - When a function needs to accept a variable number of arguments.
 - When you want to work with arguments as an array instead of the older arguments object.

b) The Spread Operator (...spread) – Expanding an Array into Individual Elements

The spread operator allows you to take an array and spread its elements as separate values. This is useful when passing arrays into functions, copying arrays, or merging arrays.

Example: Using Spread to Pass an

Array as Function Arguments

```

const numbers = [10, 20, 30];

// Output: 30
console.log(Math.max(...numbers));

```

Without the spread operator in the above example, `Math.max(numbers)` would return NaN because it expects individual arguments, not an array.

Example: Using Spread to Merge Arrays

```

const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

const mergedArray = [...arr1, ...arr2];

// Output: [1, 2, 3, 4, 5, 6]
console.log(mergedArray);

```

As you can see, spread allows easy array merging without modifying the original arrays.

Example: Using Spread to copy one array to another.

This example will put it into perspective, pay attention. If you try to copy an array `array1` by assigning it to another array `array2`, thinking you have a new array in `array2` which you can modify independently of `array1`, you will be mistaken. You will find that though you end up with two arrays alright, it will not work the way you think it will. Let's try it:

```
let array1 = ['a', 'b', 'c', 'd', 'e', 'f'];

let array2 = [1, 2, 3, 4, 5, 6];

(function() {
  array2 = array1;

  // modify array2
  array2[0] = "word";
})();
console.log(array2);
console.log(array1);
```

You would think that `array2` will now be: `['word', 'b', 'c', 'd', 'e', 'f']` while `array1` will remain: `['a', 'b', 'c', 'd', 'e', 'f']`

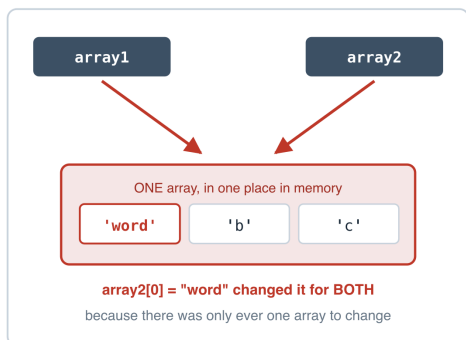
But surprisingly, both arrays `array1` and `array2` are now: `['word', 'b', 'c', 'd', 'e', 'f']` This is simply because of the way arrays work. Arrays are what is called a reference type, which means that when you assigned `array1` to `array2`, `array1` was not copied to `array2`. Rather, both arrays were simply made to point to the same memory location where the `array1` data is stored. That is why changing one of them will change both of them at the same time. (There is more on reference types in Chapter 10, Data Types.)

To fix that and create an independent copy of `array1` into `array2`, change the above line: `array2 = array1;` to: `array2 = [...array1];`

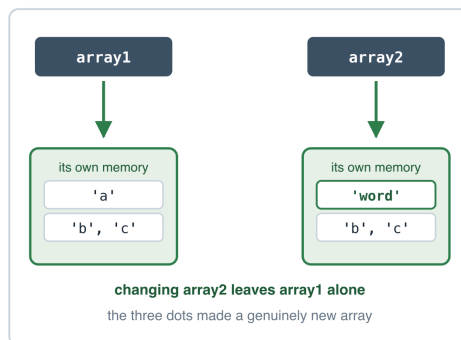
Now changing `array2` will not change `array1` because `array2` is a new array with its own new memory location, albeit with the copied over elements of `array1`.

Why copying an array is not as simple as it looks

array2 = array1;
one array, two names



array2 = [...array1];
two separate arrays



An array is a reference type. The name does not hold the array itself, only a pointer to where it lives. Copy the name and you copy the pointer, not the array.

- Figure 3.3 — Why copying an array is not as simple as it looks*

You can also use the Spread operator to extract the contents of an object literal in the same way you would with true arrays. This is just as well, since objects are used as associative arrays. Here is an example of copying the contents of one object literal to another:

```
const obj1 = { a: 1, b: 2 };  
const obj2 = { ...obj1, c: 3 };  
  
console.log(obj2);
```

This will log to the console the following: `{ a: 1, b: 2, c: 3 }`

- When to Use the Spread Operator?
 - When converting a string into an array of its characters, e.g. `[..."abc"]`.
 - In function calls. When passing array elements as individual arguments to a function.
 - When merging or copying arrays without modifying the original.
 - When merging or copying object literals

Are Rest Parameters and the Spread Operator Opposites? Yes! They can be thought of as performing opposite functions. They use the same triple dots (...) syntax, but they serve opposite purposes depending on where they are used. Here are their key characteristics:

Rest Parameters (...rest) - collect or capture values into an array. They only work in function parameter lists. Think of the word 'Rest' in 'Rest parameters' to mean

er of arguments), Whatever arguments are passed in, the three dots (...) sweep up all the rest of them into an array. - The rest parameter must be the last one in the function's parameter list. Spread Operator (...spread) - Expands an array into individual values. Think of the word 'Spread' to mean 'Spread out', for spreading out the contents of an array into individual items. That's how I visualised it to help myself understand it. - Actually, it can be used in various contexts to expand iterables (array literals, object literals).

Example showing both concepts together

```
function sum(...numbers) {  
  return numbers.reduce(  
    (acc, num) => acc + num, 0);  
}
```

This `sum()` function uses the rest operator (in its parameters) so whatever is passed into it will be converted into an array. Internally, what is passed to it will become the `numbers` array, which it uses to do its job.

```
const nums = [5, 10, 15];  
  
// Output: 30 (Spread converts array  
// into function arguments)  
console.log(sum(...nums));
```

At the point where we need to call the `sum()` function, we have an array of numbers `nums`. However, we know that the `sum()` function needs numbers, not an array. We therefore use the spread operator on the `nums` array as we pass it as the arguments to `sum()` when we call it.

As you can see, Spread (`...nums`) expands an array, while rest (`...numbers`) collects function arguments into an array.

Array properties

JavaScript arrays come with several built-in properties that help in working with them efficiently. Here, we will cover the most commonly used ones that every programmer should know. Let's look at four of them: `length`, `prototype`, `constructor` and `prototype.length`.

length

The `length` property returns the number of elements in an array. It tells us how many elements are in an array. It is modifiable, meaning we can use it to truncate

an array. It can be useful with looping through and managing array size. Let's get the length of an array:

```
const numbers = [10, 20, 30, 40];

// Output: 4
console.log(numbers.length);
```

The number returned by the `.length` property is not based on a zero-based count. So if it is 4, then the array literally contains 4 items and not 5 items. This is worth remembering, so you do not confuse that with the index/key numbering of arrays which are numbered from zero (0).

Let's use it to truncate an array, or in other words, modify the length of the array:

```
// Truncate the array
numbers.length = 2;

// Output: [10, 20]
console.log(numbers);
```

Notice how this chops off the rest of the array elements, retaining only the first two.

prototype

prototype allows you to add properties and methods to arrays. It lets us add new methods that apply to all arrays, thereby extending the functionality of arrays. Example: Adding a Custom Method to Arrays:

```
Array.prototype.last = function () {
  return this[this.length - 1];
};

const nums = [1, 2, 3];

// Output: 3
console.log(nums.last());
```

A word of caution though. Adding your own methods to `Array.prototype` affects every array in your whole program, including ones created by any library you are using. It is generally frowned upon for that reason. It is worth knowing that it is possible, but reach for an ordinary function instead.

constructor

The constructor property identifies the constructor function that created the array. It can help verify if an object is an array. Example: Checking the Constructor of

an Array:

```
const arr = [1, 2, 3];

// Output: true
console.log(arr.constructor === Array);
```

prototype.length

The `prototype.length` property refers to the length of the default array's prototype. It returns the length of the JavaScript (default) array prototype. It is rarely modified but it's part of the JavaScript array behaviour. Example: Let's check the prototype length:

```
// Output: 0 (default)
console.log(Array.prototype.length);
```

Array methods

These are built-in functions provided in JavaScript for use in manipulating arrays. A method is a function that is defined on an object. If the concept of methods or functions is new to you, do not worry. Chapter 7 is devoted entirely to them. Right now, we will look at the most important array methods and how they work. I will describe them, and demonstrate their use with examples. To fully grasp array methods, there are a few things to know about them. They are all similar in the way they work. For example, most of them take a function to be run on every item in the array they are called on. This is logical because there is not much else to do with an array if not to do something with each of its elements. They are all therefore some sort of loop, and as array functions, they have to be called on an array. Here is the syntax of their use:

```
arrayName.arrayMethod();
```

- `concat()` Joins multiple arrays into a new array (does not modify original arrays). Used to join two arrays together into a bigger array. The array passed to it as its argument will be joined to the right side of the array it is called on. The result is one array containing elements from the two arrays. The following is an example:

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5];
const result = arr1.concat(arr2);

// Output: [1, 2, 3, 4, 5]
console.log(result);
```

copyWithin()

Modifies the array by copying values within itself.

```
const arr = [1, 2, 3, 4, 5];
arr.copyWithin(2, 0, 2);

// Output: [1, 2, 1, 2, 5]
console.log(arr);
```

In this example, it copies elements from index 0 to 2 and places them starting at index 2. Elements from index 0-2 will grab 1, and 2 (two elements), then placing them from index 2 means the two copied elements (1, 2) will replace two elements from the original array ([1, 2, 1, 2, 5]) and thus end up with this output array: [1, 2, 1, 2, 5]

every()

It is similar to the `some()` method further down, except that rather than just one element, it checks whether every element in an array passes the test or condition in the function you pass to it. The syntax is the same as with the `some()` method. Pass it a function and it will run that function on all the elements in the array you call it on. If any of the elements fails the test, it will return false, otherwise it will return true. For example:

```
let ages = [32, 33, 16, 40];

function checkAdult(age) {
  return age >= 18;
}

console.log(ages.every(checkAdult));
```

This example will return false because not every age inside ages is 18 or over.

fill()

Fills an array with a value. The new value overwrites whatever was in those positions before.

```
const arr = [1, 2, 3, 4];
arr.fill(0, 1, 3);

// Output: [1, 0, 0, 4]
console.log(arr);
```

Replaces values from index 1 to 3 with 0. Note that 'index 1 to 3' means up to, but not including the element at index 3, which is why 4 from the original array is

not overridden.

filter()

`filter()` returns a new array of elements that pass a condition. This means that it 'filters out' values that don't meet the condition.

```
const numbers = [1, 2, 3, 4, 5];
const even = numbers.filter(
  num => num % 2 === 0);
```

```
// Output: [2, 4]
console.log(even);
```

find()

Returns the First Element That Meets a Condition. It stops at the first match.

```
const numbers = [10, 15, 20, 25];
const found = numbers.find(
  num => num > 12);
```

```
// Output: 15
console.log(found);
```

findIndex()

Returns the Index of the first matching Element. It works just like `find()` except that, it returns the index instead of the value.

```
const numbers = [10, 15, 20, 25];
const index = numbers.findIndex(
  num => num > 12);
```

```
// Output: 1
console.log(index);
```

flat()

This method flattens the multi-dimensional array into a single-dimensional array.

```
const nested = [
  1,
  [
    2,
    [
      3,
      4
    ]
  ],
  5
```

```
];
```

```
// Output: [1, 2, 3, 4, 5]  
console.log(nested.flat(2));
```

The argument 2 specifies how deep to flatten.

forEach()

It loops through an array, and executes a function for each array element.

```
const arr = [1, 2, 3];  
  
// Prints 2, then 4, then 6,  
// each on its own line  
arr.forEach(  
  num => console.log(num * 2)  
);
```

includes()

It checks if an array contains a value and returns true if so, or false if not.

```
const colors = [  
  "red",  
  "blue",  
  "green"  
];  
  
// Output: true  
console.log(colors.includes("blue"));
```

indexOf()

Finds the first occurrence of a value. It returns the index if found, or -1 if not.

```
const arr = [10, 20, 30, 40];  
  
// Output: 1  
console.log(arr.indexOf(20));  
  
// Output: -1  
console.log(arr.indexOf(50));  
  
// A common way to test for presence.  
// This prints false, because 100 is  
// not in the array  
console.log(arr.indexOf(100) !== -1);
```

isArray()

The `Array.isArray()` method checks whether a given value is an array. For example:

```
// Output: true
console.log(Array.isArray([1, 2, 3]));

// Output: false
console.log(Array.isArray("Hello"));
```

join()

It converts an array into a string. It joins the array elements with a specified separator. This means that you have to tell it what to separate the elements in the array by, and you do so by passing the separator as an argument to `join()`, e.g. " " for a space, "," for a comma, or "" for no separator at all.

```
const words = ["Hello", "World"];

// Output: "Hello World"
console.log(words.join(" "));
```

lastIndexOf()

It finds the last occurrence of a value and returns its index or -1 if not found. It works just like `indexOf()` except that it searches from the end.

```
const arr = [10, 20, 30, 20, 40];

// Output: 3
console.log(arr.lastIndexOf(20));
```

map()

`map()` runs a function on every element of an array and hands back a brand new array of the results. The original array is left untouched, and the new array always has the same number of elements as the old one. It is the method you reach for when you want to turn a list of one thing into a list of another thing.

Here it is turning a list of book objects into a block of HTML:

```
const books = [
  { id: 1, name: 'Macbeth' },
  { id: 2, name: 'Oliver Twist' }
];
```

```

<script>
    const bookList =
        document.querySelector('#books');

    const bookHtml = books.map((book) =>
        `<p>${book.name}</p>`).join('');

    bookList.innerHTML = bookHtml;
</script>

```

push()

This function is used to put things in an array. It's very useful, because people create arrays to hold values or data, so when you are working with arrays, you will sooner or later want to place items inside them, and push is one of the fastest ways to do so. It is a member of the array object, so you call it on the array that you wish to place items into (the target array). It accepts the element or item you wish to put in the array in question. The good thing about it is, it does not override anything previously in the target array, but rather adds the new data to the END of it. Here is an example:

```

let myArray = [
    1, 2, 3
];

myArray.push("John", "Peter");

console.log(myArray);

```

This will have added an array containing the two strings 'John' and 'Peter' and so the output of the contents of the target array myArray will now be:

```
[[1, 2, 3], ['John', 'Peter']]
```

-pop() This pop() function is a built-in function in JavaScript used to remove the last element in an array. Here is an example:

```

let myArray = [
    1, 2, 3,
    ['John', 'Peter']
];

myArray.pop();

console.log(myArray);

```

This will have removed the last element from myArray, which is the sub array containing names (['John', 'Peter']), and so the output of the contents of the myArray will end up being:


```
[[1, 2, 3]]
```

You may decide to pop the last item out and capture it in a separate variable of its own if you need to use it. For example:

```
let myArray = [
  [1, 2, 3],
  ['John', 'Peter']
];

let names = myArray.pop();
```

The new array `names` will now contain `['John', 'Peter']`.

reduce()

Reduce the values of an array to a single value (from left-to-right). The `reduce()` method accepts a function or closure that takes not just the item to iterate over like most of the other array methods do, but it takes a property of what you want to reduce everything into (the final result—let's call it the snowball), and then the item (each element in the iteration). With each iteration over the array, the value of the snowball (first argument) of the closure is updated, and it is this value that is returned at the end of the method. Let us see it in action:

```
const items = [
  {name: 'Bike', price: 100},
  {name: 'TV', price: 200},
  {name: 'Book', price: 5},
  {name: 'Computer', price: 1000}
];

const total = items.reduce((currentTotal, item) => {
  return item.price + currentTotal;
}, 0);

// Output: 1305
console.log(total);
```

The second argument of `reduce()` is your starting point, in this case, it is the initial starting price of 0. To explain again one more time how it works, the function or closure you pass to `reduce()` as its first argument is run on all the array elements—in this case the array of items. The closure takes two arguments; the first argument will be updated with what the previous iteration returned, we can call this argument the snowball since it accumulates as it goes along. The second argument of the closure is each item in the array during the iteration. This is the same item that is used in closures in other array methods like `map()` or `forEach()` etc. The second argument of the `reduce()` method (which comes after the closure) is the initial value of the snowball to start the iteration with, and this is the value that will be incremented and

stored in the snowball argument (in this case `currentTotal`) at every iteration. For this method to work however, you need to remember to always explicitly update the value of the snowball within the closure body. In the end, it is the final value of the snowball that will be returned by `reduce()`.

reverse()

The `reverse()` method in JavaScript is used to reverse the elements of an array in place, meaning it modifies the original array instead of creating a new one. No arguments are needed. It returns the same array, but with the order of elements reversed.

```
const numbers = [1, 2, 3, 4, 5];

// Modifies the original array
numbers.reverse();

// Output: [5, 4, 3, 2, 1]
console.log(numbers);
```

shift()

The `shift()` function is a direct opposite of the `pop()` function because unlike `pop()` which is used to remove the last item in an array, `shift()` is used to remove the first element in an array. Here is an example:

```
let myArray = [
  [1, 2, 3],
  ['John', 'Peter']
];

myArray.shift();

console.log(myArray);
```

This will have removed the first element from `myArray`, which is the sub array containing numbers ([1,2,3]), and so the output of the contents of the `myArray` will now end up being:

```
[['John', 'Peter']]
```

You may decide to remove that first item and capture it in a separate variable of its own if you need to use it. For example:

```
let myArray = [
  [1, 2, 3],
  ['John', 'Peter']
];
```

```
];
```

```
let numbers = myArray.shift();
```

The new array numbers will now contain [1, 2, 3].

unshift()

The `unshift()` function is a direct opposite of the `push()` function because unlike `push()` which adds an item to the end of an array, `unshift()` adds an element at the beginning of an array. Here is an example:

```
let names = [  
  ['John', 'Peter']  
];  
  
names.unshift(["Jimmy", "Jack"]);  
  
console.log(names);
```

This will have added a sub array containing more names (["Jimmy", "Jack"]) as the first element of the names array. The output of the contents of the names array will now be:

```
[["Jimmy", "Jack"], ['John', 'Peter']]
```

Note that similarly to the `push()` function, the `unshift()` function accepts an argument, which should be the element or item you wish to add to the target array it was called on.

slice()

It extracts a portion of an array, then returns the extracted portion as a new array without modifying the original.

```
const arr = [1, 2, 3, 4, 5];  
  
// Output: [2, 3, 4]  
console.log(arr.slice(1, 4));
```

This slices the array from index 1 up to but not including index 4.

some()

It checks if at least one element in the given array passes a condition and returns true if any element passes the test or false if none passes.

```
const numbers = [1, 3, 5, 7, 8];  
  
// Output: true
```

```

console.log(
  numbers.some(
    num => num % 2 === 0
  )
);

```

sort()

Sorts an array alphabetically or numerically. By default, JavaScript's `sort()` method converts all elements to strings and sorts them alphabetically in ascending order. But when it comes to numbers, it has no way of detecting which number is bigger than the other. The solution with numbers is to pass to `sort()` a compare function. It will run this function on elements in pairs from left to right, subtracting one number from the other and thereby determining which is greater than the other. Basically, when sorting numbers without a compare function, the results can be unexpected. It's easier when you see an example:

```
[1, 100, 2, 20].sort();
```

The above expression is first converted to `["1", "100", "2", "20"]` and then to `[1, 100, 2, 20]` instead of `[1, 2, 20, 100]`

JavaScript doesn't know that "100" should come after "2", because it compares them as strings, character-by-character. The fix is to pass a compare function to `sort()`. This allows you to control how items are compared—typically by subtracting one from the other for numeric sorting:

```
[1, 100, 2, 20].sort((a, b) => a - b);
```

This will result in the correct sorting: `[1, 2, 20, 100]`

This works because: If `a - b` results in a negative number, `a` comes before `b`. If `a - b` results in a positive number, `b` comes before `a`. If `a - b` results in 0, then their order doesn't change.

Let's do some examples:

Sort in alphabetical order:

```

const names = [
  "Bob", "Alice", "Charlie"
];

// Output: ["Alice", "Bob", "Charlie"]
console.log(names.sort());

```

Sort numbers in an array in ascending order

```
const numbers = [10, 5, 20];

// Output: [5, 10, 20]
console.log(

    // numbers.sort(function(a, b){return a-b});
    // OR
    numbers.sort((a, b) => a - b)
);
```

Sort numbers in an array in descending order

Pass a function to `sort()` that assumes that the later number is greater than the one before it—take note of the `b-a` it returns.

```
let points = [20, 80, 1, 8, 25, 10];
points.sort(function(a, b) { return b - a; });
```

This will produce `[80, 25, 20, 10, 8, 1]`

Get the highest or lowest value in an array

To get the highest number in an array of numbers, the simple trick is to sort them in descending order and grab the number at the first index. You would do the opposite to get the lowest number. For example:

```
let points = [20, 80, 1, 8, 25, 10];
points.sort(function(a, b) { return b - a; });
```

The highest number will be found at `points[0]` The lowest number can be found using any of the following techniques:

```
points[points.length - 1]
// or
points.slice(-1)[0]
// or
points.slice(-1).pop()
```

splice()

It adds to, or removes elements from an array and returns the removed items. In doing so, it modifies the original array. It takes one required argument and any number of optional ones after it:

- i) The index at which to remove an element (required)
- ii) The number of elements to remove (optional). If it's 0, nothing will be removed from the array
- iii) Items to be put into that index (optional).

```
const arr = [1, 2, 3, 4];
arr.splice(1, 2, "a", "b");

// Output: [1, "a", "b", 4]
console.log(arr);
```

In this example, it removes 2 elements beginning from index 1 and inserts "a" and "b", hence we end with the original array having the new value of: [1, "a", "b", 4]. Any number of arguments after the first two will be added to the array starting from the index specified in the first argument.

```
const arr = [1, 2, 3, 4];
arr.splice(1, 2, "a", "b", "c", "d", "z");

// will result in arr having the value:
[1, "a", "b", "c", "d", "z", 4]
```

toString()

It converts an array to a string. It is similar to `join()`, but always uses commas as the separator.

```
const arr = [1, 2, 3];

// Output: "1,2,3"
console.log(arr.toString());
```

True array methods and associative arrays

By now, we've seen that JavaScript has many useful methods designed for true arrays, like:

- `forEach()`
- `filter()`
- `map()`
- `reduce()`
- And others...

But here's something very important to remember. Array methods only work on real arrays. JavaScript objects (which we often use as associative arrays) do not have these methods. That's because they're plain objects, not arrays under the hood. So what do we do when we want to loop through or work with objects the way we do with arrays? The trick is to use three very special methods that JavaScript provides for this purpose. These three methods are:

- `Object.keys()`
- `Object.values()`
- `Object.entries()`

I came up with the abbreviation KVE (for Keys, Values, Entries), to help me remember them better. These special methods convert objects into arrays, so we can then use regular array methods on them. You probably remember them from looping through arrays above. Here's a quick look at how you can use each to convert an object (associative array) into a true array:

```
const person = {  
  name: "Alice",  
  age: 30,  
  city: "London"  
};
```

Object.keys()

```
console.log(Object.keys(person));
```

The output of this is the array:

```
["name", "age", "city"]
```

Object.values()

```
console.log(Object.values(person));
```

The output is the array:

```
["Alice", 30, "London"]
```

Object.entries()

```
console.log(Object.entries(person));
```

The output is the array:

```
[["name", "Alice"], ["age", 30], ["city", "London"]]
```

Having converted an object into an array, you can now use `.filter()`, `.map()`, or any array method on these results. For example, associative arrays or objects do not have the `.length` property, so if you tried to use it on an object like so:

```
console.log(person.length);
```

The output will be undefined. But not to worry—now that you have learned about the 'big' KVE (`Object.keys()`, `Object.values()` and `Object.entries()`), you are covered, and you can use them effectively and easily to work around that. If you guessed

that you can use the `.length` property and any true array method after converting the object into an array using any of the KVE methods, you are absolutely right. To count how many properties are in an object, you can use `Object.keys()` like this:

```
console.log(Object.keys(person).length);
```

The output will be 3. This counts how many keys (i.e. properties) are inside the object. Hence the output is 3 for the 3 properties 'name', 'age', and 'city' of the person object.

QUIZ — Arrays

This page contains the Q & A (questions and answers) for this chapter — Chapter 3: Arrays. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 11 to 14 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. JavaScript has two types of array. Name them, and say what the difference between them is.

Clue: the difference is not in what they store, but in what marks the spot of each item.

2. Your friend writes this and expects `student.length` to be 4:

```
let student = [];  
student["name"] = "Amina";  
student["age"] = 12;  
student["grade"] = "6";  
student[0] = "Math";
```

What does `student.length` actually print, and what should your friend have used instead?

Clue: only one of those four lines added a real array element.

3. An array has 4 elements in it. What is the index of the last one, and what does `.length` report? Explain why those two numbers are not the same.

Clue: one of them starts counting at zero and the other does not.

4. Given this object, which two of these three lines will work, and why does the third one fail?

```
const person = { "first-name": "Tom", aunt: "Polly" };  
  
person.aunt person["first-name"] person.first-name
```

Clue: think about what JavaScript sees when it meets a hyphen in the middle of an expression.

5. What is the difference between `slice()` and `splice()`?

Clue: one of them leaves the original array exactly as it found it. The other does not.

6. Explain in your own words what `reduce()` does, and what its second argument is for.

Clue: the chapter calls the running result a snowball.

7. Four methods add or remove items at the ends of an array: `push()`, `pop()`, `shift()` and `unshift()`. Say which end each one works on, and whether it adds or removes.

Clue: two of them work at the end, two at the beginning.

8. Why does `[1, 100, 2, 20].sort()` give you `[1, 100, 2, 20]` rather than `[1, 2, 20, 100]`, and how do you fix it?

Clue: `sort()` compares them as though they were words, not numbers.

9. Look at this code:

```
let array1 = ['a', 'b', 'c'];
let array2 = array1;
array2[0] = "word";

console.log(array1);
```

What does it print, and why? How would you make `array2` a genuinely independent copy?

Clue: the fix uses three dots.

10. Objects do not have array methods like `.filter()` or `.map()`, and they have no `.length` either. Name the three methods JavaScript gives you to get around this, and say what each one hands back.

Clue: the chapter gives them the nickname KVE.

11. EXERCISE. Loop through an array running a function on each element.

Write a function of your own that takes a number and returns that number multiplied by two. Then use it on every element of an array of numbers, and display the results.

Clue: think about which array method takes a function and runs it on every element of an array, handing you back a new array of the results. You met it in this chapter.

12. EXERCISE. Loop through an array of images and display the images on screen.

- Create a div in your HTML code and give it an ID
- Within your JavaScript code, create an array

- Put in this array a number of elements which should be names of images as strings, including their image extensions. Make sure these are the names of actual images on your computer
- Select the div you created in your HTML. This is where you will be displaying the images whose names you have stored in the array. Let us refer to it as the target div
- Loop through the array grabbing the names of the images one by one, placing them within an `<img />` tag, and then placing the image tag in the target div, so that the image displays
- Use a timer function so that only one image is displayed at a time, and the image changes every 5 seconds

Clue: `setInterval()` takes two arguments — a function to run, and how long to wait between runs, in milliseconds. Keep a counter of which image you are up to, and send it back to 0 when it reaches the end.

13. EXERCISE. Create an array of the numbers 1, 3, 2, 5, 2, 9, 2, 9, 2, 1.

- Then create another, empty array
- Loop through the array of numbers and check each one. If the number is a 2 or a 9, put it into the other array
- Finally, display the contents of that second array in an alert popup

The popup should display 2, 2, 9, 2, 9, 2

This exercise should teach you how to loop through an array and scan its contents for specific values, filtering out what is not needed. In doing so, you will use the comparison operator to check whether a value is equal to another value. We have not covered comparison operators yet, so treat this as a gentle introduction. When we come to them in Chapter 5 (Control Flow), and you see those and many other operators in action, you will grasp their essence and power in programming.

Clue: `==` checks whether two values are equal. `push()` adds to an array.

14. EXERCISE. Finally, we should not proceed without exercising with multi-dimensional arrays too.

- Create an array called `food` which contains three arrays of different food types. Each of those three arrays should have three elements in it
- Loop through this `food` array and display the elements of each of the inner arrays in the console, on separate lines

Clue: one level of array needs one loop. Two levels need a loop inside a loop.

ANSWERS

1. Numeric arrays (also called indexed arrays) and associative arrays.

The difference lies in what marks the spot of each element. In a numeric array the key is a number, called an index, and JavaScript assigns those for you starting from zero. In an associative array the key is a name, a short piece of text that you choose yourself.

Numeric arrays are the true arrays of JavaScript. What people call an associative array in JavaScript is really a plain object.

2. It prints 1.

Only `student[0] = "Math"` added a real array element. The other three added named properties to the array object, and `.length` counts only the numbered elements. Those named properties are also skipped by for loops, `.map()` and `.forEach()`.

Your friend wanted an object:

```
let student = {
  name: "Amina",
  age: 12,
  grade: "6",
  subjects: ["Math"]
};
```

The rule of thumb: arrays are for ordered lists, objects are for key-value pairs. If you find yourself writing `array["name"] = "value"`, stop and ask whether you really wanted an object.

3. The last element is at index 3, but `.length` reports 4.

Indexes are zero-based, so a 4-element array uses indexes 0, 1, 2 and 3. The `.length` property is not zero-based; it is a plain count of how many items there are. This is why the last element of any array sits at index `length - 1`.

4. `person.aunt` and `person["first-name"]` both work. `person.first-name` fails.

Dot notation only works when the key is a valid identifier — no spaces, no hyphens, not starting with a digit. "first-name" has a hyphen, so JavaScript does not read it as one name. It reads it as a subtraction:

`person.first - name`

which gives you NaN rather than "Tom".

Bracket notation always works, so when in doubt, reach for it.

5. `slice()` copies. `splice()` changes the original.

```
let arr = [1, 2, 3, 4];

// slice - takes a copy of part of the array
let newArr = arr.slice(1, 3);
// newArr is [2, 3], and arr is still [1, 2, 3, 4]

// splice - cuts items out of the array itself
arr.splice(1, 2);
// removes [2, 3], so arr is now [1, 4]
```

In short: `slice()` leaves the original array untouched and hands you a new one. `splice()` modifies the array it is called on, and hands you back whatever it removed. It can also insert new items at the same time.

6. `reduce()` boils a whole array down to a single value.

```
let nums = [1, 2, 3, 4];

let sum = nums.reduce((acc, curr) => {
  return acc + curr;
}, 0);

console.log(sum); // 10
```

The function you give it receives two things: the running result so far (here called `acc`, short for accumulator — the chapter's snowball), and the current element. Whatever you return becomes the running result for the next element.

The second argument to `reduce()` itself — the 0 above — is the starting value of that running result. Think of it as the size of the snowball before it starts rolling.

7.

- `push()` adds an item to the END of the array
- `pop()` removes the item at the END of the array
- `unshift()` adds an item to the BEGINNING of the array
- `shift()` removes the item at the BEGINNING of the array

`pop()` and `shift()` both hand back the item they removed, so you can catch it in a variable if you still need it.

8. Because `sort()` converts everything to strings before comparing.

As text, "100" comes before "2", in the same way that "apple" comes before "b" in a dictionary — it compares character by character, and "1" is lower than "2".

The fix is to pass `sort()` a compare function:

```
[1, 100, 2, 20].sort((a, b) => a - b);
```

```
// [1, 2, 20, 100]
```

It works because if $a - b$ is negative, a is placed before b ; if positive, b goes first; and if 0, their order stays as it was. Swap it to $b - a$ to sort in descending order.

9. It prints ['word', 'b', 'c'] — the change to array2 also changed array1.

Arrays are a reference type. Writing `array2 = array1` did not copy anything; it simply made both names point at the same array in memory. Changing it through one name changes what you see through the other, because there is only one array.

To get a genuinely independent copy, use the spread operator:

```
let array2 = [...array1];
```

Now array2 is a brand new array with its own place in memory, holding copies of the elements. Changing it leaves array1 alone. There is more on reference types in Chapter 10 (Data Types).

10. `Object.keys()`, `Object.values()` and `Object.entries()` — KVE.

```
const person = { name: "Alice", age: 30, city: "London" };

console.log(Object.keys(person));
// ["name", "age", "city"]

console.log(Object.values(person));
// ["Alice", 30, "London"]

console.log(Object.entries(person));
// [{"name", "Alice"}, {"age",
// 30}, {"city", "London"}]
```

All three hand you back a real array, which means you can then use any array method on the result. That includes `.length`, which is how you count an object's properties:

```
console.log(Object.keys(person).length);
// 3
```

11. When you meet a problem in programming, the first thing to do is think about what tool the language already gives you. Here we are dealing with an array, so ask what JavaScript provides for arrays — the array methods come to mind. Is there one that runs a function on every element? Yes: `map()`. Once you have found the tool, the problem is half solved.

`map()` can be given a built-in function or one you write yourself. We will write our own, taking a number as its argument and returning that number doubled:

```
function multiplier(num)
{
    return num * 2;
}

let numbers = [1, 2, 3, 4, 5];
let newNumbers = numbers.map(multiplier);

alert(newNumbers);
```

This displays a popup saying 2,4,6,8,10.

Notice that you pass `multiplier` without brackets after it. You are handing `map()` the function itself, not the result of calling it. At each turn of the loop, `map()` passes the current element into `multiplier()` as `num`.

One more thing worth knowing while we are here. `map()` itself takes just one argument, the function. But the function it hands your elements to can accept a second parameter, and JavaScript will fill that one in with the index of the current element. This is handy when you need to know where in the array you are:

```
let names = ['Ada', 'Grace', 'Alan'];

let numbered = names.map(function(name, i) {
    return (i + 1) + ": " + name;
});

console.log(numbered);
// ["1: Ada", "2: Grace", "3: Alan"]
```

We add 1 to `i` because the index starts at 0, and a numbered list that starts at 0 would look odd to a reader. The same second parameter is available in `forEach()` and `filter()` too.

12. Here is the whole page:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Image slideshow</title>
</head>
<body>
    <div id="myImgDiv"></div>

    <script>
        let images = new Array();

        // put the images in the array
        images.push(
            "amex.png",
            "visa.png",
```

```

    "pexels-photo.jpeg",
    "pexels-photo2.jpeg",
    "logo.jpeg"
  );

  // grab the target div
  let div = document.getElementById("myImgDiv");
  let pos = 0;
  let numOfImages = (images.length - 1);

  setInterval(function()
  {
    div.innerHTML = "<img src='" + images[pos] + "' />";

    if (pos < numOfImages)
    {
      pos++;
    }
    else
    {
      pos = 0;
    }
  }, 5000);
</script>
</body>
</html>

```

The code loops through the array of image names, puts each one inside an `<img />` tag, and inserts that into the div.

The looping is done by `setInterval()`. It takes two arguments: a function to perform, and the time to wait between each run. We have passed it an anonymous function to do the work, and 5000 milliseconds, which is 5 seconds. So the images appear in the target div and keep changing every five seconds.

The `pos` counter keeps track of which image we are showing. When it reaches the last one, we set it back to 0 so the slideshow starts over. Congratulations, you have just built your first image slideshow.

13. Here is the code:

```

let arr1 = [1, 3, 2, 5, 2, 9, 2, 9, 2, 1];
let arr2 = [];

for (let i = 0; i < arr1.length; i++)
{
  if (arr1[i] == 2)
  {
    arr2.push(arr1[i]);
  }
}

```



```

    {
        arr2.push(arr1[i]);
    }
}

alert(arr2);

```

As expected, the alert popup displays 2, 2, 9, 2, 9, 2.

The `if ()` block we use to check each item is a conditional statement, which we look at properly in Chapter 5. Notice the word *conditional*: a condition is checked, and what happens next depends on the answer.

Instead of two separate `if` statements, we could check both conditions in one, by joining them with the OR operator (`||`):

```

let arr1 = [1, 3, 2, 5, 2, 9, 2, 9, 2, 1];
let arr2 = [];

for (let i = 0; i < arr1.length; i++)
{
    if ((arr1[i] == 2) || (arr1[i] == 9))
    {
        arr2.push(arr1[i]);
    }
}

alert(arr2);

```

This still displays 2, 2, 9, 2, 9, 2. The double pipe (`||`) means "if the test on either side is true, do the thing". Notice we end up with rather less code than before.

14. Here is the code:

```

let food = [
    ['tomatoes', 'pepper', 'cabbage'],
    ['apple', 'pineapple', 'banana'],
    ['rice', 'pasta', 'beans']
];

for (let i = 0; i < food.length; i++)
{
    let sub = "";

    for (let j = 0; j < food[i].length; j++)
    {
        sub += ' ' + food[i][j];
    }

    console.log(sub);
}

```

The result in the console is:

```
tomatoes pepper cabbage apple pineapple banana rice pasta beans
```

The key to mastering loops is this: for every level you move down into the array, you need another loop block. This array is two levels deep, so we use two for loops, one nested inside the other.

Notice that the outer loop uses a counter called `i`, and the inner one uses `j`. It has to be a different name, because `i` is already in use by the outer loop. There is nothing magic about `i` and `j` — any names would work — but by convention those are the ones programmers reach for. If the array went deeper still, we would carry on in the same way.

Remember too that you use one set of square brackets for each level. `food[1]` gives you the second inner array, `['apple', 'pineapple', 'banana']`, and `food[1][0]` then gives you the first element of that, `'apple'`. That is exactly what `food[i][j]` is doing inside the loop. The sub `+= ' ' +` part is simply building up a string of all three items so we can print them on one line.

Chapter 4 — CONSTANTS

- Mutating a const array
- Mutating a const object

A constant is like a variable, but with one key difference: once a constant has been given a value, you cannot point it at a different one. It is created by declaring it with the `const` keyword, for example:

```
const fee = 20;
```

- Like a variable, a constant stores a single value in the computer's memory.
- However, unlike variables, constants cannot be reassigned after their initial definition.
- Constants are useful for storing fixed values that should remain the same throughout the program, such as tax rates, company names, or configuration settings.
- Here are the key points about constants:
 - Use `const` when you never want the value to change.
 - Constants help make your code more predictable and error-free.
 - Trying to reassign a constant will cause an error.
- Here's a real-world example using constants in a shopping cart scenario to calculate a total price with a constant tax rate. Imagine you're building an online store. You want to calculate the final price of an item after adding tax. Since the tax rate never changes, it's a perfect use case for a constant.

```
// 15% tax rate (constant)
const TAX_RATE = 0.15;

function calculateTotalPrice(price) {
  // Calculate tax
  const taxAmount = price * TAX_RATE;

  // Add tax to price
  const totalPrice = price + taxAmount;

  return totalPrice;
}

// Example usage
const itemPrice = 100;
```

```
console.log(`Final price: ${calculateTotalPrice(itemPrice)}`);

// Output: Final price: $115
```

Why Use a Constant Here?

- The tax rate shouldn't change throughout the program.
- It makes the code clearer—you instantly know what `TAX_RATE` represents.
- It prevents accidental changes that could cause calculation errors.

You may have noticed something about the name `TAX_RATE`. Back in Chapter 2 we said that programmers normally name variables using camel casing, like `taxAmount` or `itemPrice`, and that is still true. Look at the example above and you will see that `taxAmount`, `totalPrice` and `itemPrice` all follow that rule, even though they are declared with `const`. `TAX_RATE` is different because it is a fixed setting: a value written into the program once and never worked out from anything else. For those, the convention is to use capital letters with underscores between the words. It is a signal to anyone reading the code that this is a dial someone chose, not a value the program calculated. You will see the same style used for things like `MAX_LOGIN_ATTEMPTS` or `API_URL`. So the rule of thumb is: camel case for almost everything, including most of your `consts`, and `CAPITALS_WITH_UNDERSCORES` only for fixed settings of this kind.

Mutating a const array

We said above that you cannot point a `const` at a different value. That is true, and it is worth being precise about what it does and does not protect. `const` guards the name, not the contents. So although you cannot re-assign a `const` variable, if the value it holds is an array, you can still change the values at specific keys inside that array.

```
const testData = [5, 10, 20, 25];

function tryToMutateConstVar() {
  testData = [15, 110, 120, 125];

  console.log(testData);
}

tryToMutateConstVar();
```

The output of this code will be an error like so:

`TypeError: Assignment to constant variable.`

But since the `const` variable holds an array, you can target and change the values at specific keys inside it. Here is how: to modify the array key values, you use the bracket notation.

```
const testData = [5, 10, 20, 25];

function tryToMutateConstVar() {
  testData[0] = 200;
  testData[1] = 0;

  console.log(testData);
}

tryToMutateConstVar();
```

The output will be the changed contents of the array, like so:

```
[200, 0, 20, 25]
```

Mutating a const object

The very same thing is true of objects. If a `const` holds an object, you cannot point the name at a different object, but you can change what is inside the one it already holds. Trying to replace the whole object fails:

```
const person = { name: "Alice", age: 30 };

// TypeError: Assignment to constant variable.
person = { name: "Bob", age: 25 };
```

But changing a property inside it works perfectly well:

```
const person = { name: "Alice", age: 30 };

person.name = "Bob";
person.age = 25;

// Output: { name: 'Bob', age: 25 }
console.log(person);
```

So it is worth holding on to this one sentence, because it catches almost everybody out at least once: `const` protects the name, not the contents. If you genuinely need the contents frozen too, JavaScript has a separate tool for that called `Object.freeze()`, which we will meet when we come to objects in Chapter 12.

QUIZ — Constants

This page contains the Q & A (questions and answers) for this chapter — Chapter 4: Constants. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What exactly does `const` protect? Finish this sentence in your own words:

"`const` protects the _____, not the _____."

Clue: it is the reason the next two questions have the answers they do.

2. Which of these two lines throws an error, and what does the error say?

```
const scores = [10, 20, 30];
```

```
scores[0] = 99;  
scores = [99, 20, 30];
```

Clue: one of them is changing what is inside the box. The other is trying to swap the box for a different one.

3. Now the same question for an object. Which line fails?

```
const person = { name: "Alice", age: 30 };
```

```
person.age = 31;  
person = { name: "Alice", age: 31 };
```

Clue: the rule is exactly the same as for arrays.

4. Look at this code from the chapter:

```
const TAX_RATE = 0.15;  
  
function calculateTotalPrice(price) {  
  const taxAmount = price * TAX_RATE;  
  const totalPrice = price + taxAmount;  
  return totalPrice;  
}
```

All four of those names use `const`, but `TAX_RATE` is written differently from the others. Why?

Clue: one of them is a setting somebody chose. The other three are worked out by the program while it runs.

5. You want a value that genuinely cannot be changed at all, contents included. Is `const` enough on its own? If not, what would you reach for?

Clue: the chapter names the tool at the very end, and points you to Chapter 12.

6. EXERCISE. Write a small piece of code that proves the rule from question 1 to yourself.

-Create a `const` holding an array of three of your favourite foods -Change the first one to something else, and print the array to show that it worked -Then try to replace the whole array, and see what the browser tells you

Clue: keep the second part last, because once it throws an error the lines after it will not run.

ANSWERS

1. "`const` protects the NAME, not the CONTENTS."

In other words, `const` stops you pointing that name at a different value. It does not freeze whatever the value happens to be. If the value is a simple one, like a number or a piece of text, there is nothing inside it to change, so `const` feels completely locked. But as soon as the value is an array or an object, the difference matters.

2. The second line throws.

```
// fine - scores is now [99, 20, 30]
scores[0] = 99;

// TypeError: Assignment to constant variable.
scores = [99, 20, 30];
```

Changing `scores[0]` reaches inside the array and changes one of its elements. The array itself is still the same array, sitting in the same place in memory, so `const` has nothing to object to.

The second line is different. It is trying to make the name `scores` point at a brand new array, and that is exactly what `const` forbids.

3. The second line again.

```
// fine
person.age = 31;

// TypeError: Assignment to constant variable.
person = { name: "Alice", age: 31 };
```

Setting `person.age` changes a property inside the object that `person` already points at. The second line tries to swap in a different object altogether, which `const` will not allow.

4. `TAX_RATE` is a fixed setting. It is a value somebody decided on and typed into the program, and it never changes while the program runs.

The convention for those is capital letters with underscores between the words, so that anyone reading the code can see at a glance that it is a dial someone chose. You will meet the same style in names like `MAX_LOGIN_ATTEMPTS` or `API_URL`.

`taxAmount`, `totalPrice` and `itemPrice` are different. They are worked out by the program as it goes along, so even though they are declared with `const` they follow the ordinary camel casing convention from Chapter 2.

The short version: camel case for almost everything, including most of your `const`s, and `CAPITALS_WITH_UNDERSCORES` only for fixed settings.

5. No, `const` is not enough on its own.

`const` only stops the name being pointed somewhere else. To freeze the contents as well, you would use `Object.freeze()`, which we come to in Chapter 12 (Object Oriented Programming).

6. Here is one way to do it:

```
const foods = ["rice", "beans", "plantain"];

// this works, because we are
// changing what is inside the array
foods[0] = "yam";
console.log(foods);
// Output: ["yam", "beans", "plantain"]

// this fails, because we are
// trying to swap the whole array
foods = ["bread", "eggs", "tea"];
// TypeError: Assignment to
// constant variable.
```

Run it and watch the console. The first change prints happily. The second one stops the program with a `TypeError`, and nothing after it runs — which is why it is worth putting that line last while you are experimenting.

If you want to see both halves work without the error cutting things short, comment out the last line once you have seen the message, and run it again.

Chapter 5 — CONTROL FLOW

- OCLS (Operators, Conditionals, Loops, Switch)
 - Operators
 - 1. Mathematical operators
 - Addition operator
 - minus operator
 - division operator
 - multiplication operator
 - modulus operator
 - assignment operator
 - incremental operator
 - decremental operator
 - 2. Assignment operators
 - 3. Comparison operators
 - Equality operator
 - Strict equality operator
 - greater than operator
 - less than operator
 - greater than or equal to operator
 - less than or equal to operator
 - 4. Logical operators
 - Not operator
 - OR operator
 - And operator
 - Ternary operator
 - Nesting multiple ternary operators
 - Combining math operators with the assignment operator
 - Conditionals
 - if statements
 - Checking Multiple Conditions
 - a) Independent Conditions

- b) Mutually Exclusive Conditions (Two Choices)
 - c) Mutually Exclusive Conditions (More Than Two Choices)
- Additional notes on conditionals
- Loops (6)
 - The For loop
 - The While loop
 - The Do...while loop
 - The For...in loop (objects and arrays)
 - The For...of loop (for Arrays, Strings, Maps, Sets, etc)
 - The forEach() loop (for arrays)
- Loop control statements
 - the break statement
 - the continue statement
- Switch statement

Control flow in programming refers to the logic that guides how your program runs. Think of your program like a river. A river doesn't just move in a straight line—it flows around bends, curves through valleys, navigates around trees and rocks, and might even change its course if it hits a huge obstacle, like a mountain. In the same way, your program might face different situations as it runs. Instead of you always having to tell it what to do every time, the program should be smart enough to decide for itself based on what's happening. That's what control flow is all about-making your program intelligent enough to respond to different events. Let's say a robot is programmed to walk forward. What happens if it walks up to a wall? A well-written program will allow it to detect the wall and decide to turn left or right to find a new path. If the robot trips and falls, it should realise it's lying down and maybe try to stand back up. If it reaches a rock, it should figure out whether to climb over or walk around it. All of these situations are examples of events. Your program's job is to detect when events happen and respond with the right logic. In simple terms, logic means looking at a situation, recognising what's going on, and choosing what to do next. Here's a real-world example: When you're typing in a Word document and try to close it without saving, you get a message asking if you want to save your changes first. That's control flow at work—smart logic built into the app to help the user avoid mistakes. For your program to handle events well, it needs two main abilities:

- To detect that an event has happened

- To decide what to do about it (and maybe consider other things too) This is what control flow is all about: giving your program the power to respond to things in smart, useful ways. In JavaScript, there are four main tools that help you build this kind of logic. I remember them by the abbreviation OCLS:
 - 1. Operators
 - 2. Conditionals
 - 3. Loops
 - 4. Switch

Strictly speaking, a switch is itself a kind of conditional, and you will see it described that way elsewhere. I have given it a section of its own because its syntax is different enough from if...else to be worth studying separately. Operators are in this list for a similar practical reason: they are not really control flow either, but you cannot write a condition without them.

OPERATORS

Operators are very useful for performing things like mathematical operations, assigning values to variables, performing logical operations like comparing two values, adding up values etc.

1) Mathematical operators

Operators give you the ability to perform calculations so you can make accurate logical decisions. The following is a list of mathematical operators with practical examples to show how they work. Here is a list of operator characters and their meanings:

=	assignment
==	equal
===	identical
+	addition
-	subtraction
*	multiplication
/	division
%	modulo (used to find remainders)
++	increment by 1
--	decrement by 1
+=	short form of addition
-=	short form of subtraction
*=	short form of multiplication
/=	short form of division
%=	short form of modulo
>	greater than
<	less than
>=	greater than or equal to

<code><=</code>	less than or equal to
<code>!=</code>	not equal
<code>!==</code>	strictly not equal
<code>!</code>	not
<code>&&</code>	and
<code> </code>	or

There is also the ternary operator, which needs a little more explaining than a single line, so it has a section of its own further down.

Addition operator

The addition operator `+` is like the addition sign in mathematics (represented by the plus sign) and it is used to add two numbers together. It should not be mistaken for the concatenation operator that binds variables to a string. For example:

```
let sum = 2 + 2;
```

The value of `sum` will be 4.

Minus operator

The minus operator `-` is the same as the minus operator we are familiar with in math. It is used to subtract one number from the other. For example:

```
let num = 3 - 2;
```

The value of `num` will be 1.

Division operator

The division operator `/` is used to divide one number by the other. It divides the value on the left by the value on the right. The result is called the quotient. For example:

```
let num = 4 / 2;
```

The value of `num` will be 2.

Multiplication operator

The multiplication operator `*` is used to multiply a number by a number. It multiplies the value on the left by the value on the right. The result is called the product. For example:

```
let num = 2 * 2;
```

The value of `num` will be 4.

Modulo operator The modulus operator `%` is used to get the remainder after dividing one number by another. Basically, it divides the number on the left by the

number on the right, and returns whatever is left over. For example:

```
let num = 10 % 5;
```

The value of num will be 0.

```
let num = 10 % 2;
```

The value of num will be 0.

```
let num = 11 % 2;
```

The value of num will be 1.

A practical application of this is to check whether a number is even or odd. Here is an example:

```
let number = 7;

if (number % 2 == 0) {
  console.log("Even number");
} else {
  console.log("Odd number");
}
```

Incremental operator

An incremental operator `++` is used to quickly add 1 to a number. Normally, there are two ways to increase a number's value:

a) By using the addition operator

```
let count = 1;
let value = count + 1; // value is 2
```

b) By using the incremental operator

```
let count = 1;
// value is 1, but count becomes 2
let value = count++;
```

Note: In `count++` where the operator is on the right side of the variable being incremented (count), the value is assigned to the new variable (in this case value) before the incrementing happens. The value of the value variable will now still be 1, while that of count is 2. However, if you want to increment the value of count first before assigning it to value so that value will also have the incremented (updated) value of count, use `++count` with the `++` operator on the left side of the count variable.

Decremental operator

This is the direct opposite of the incremental operator. It is used to subtract 1 from a number. There are two ways to decrease a number's value:

a) By using the subtraction operator

```
let count = 2;
let value = count - 1; // value is 1
```

b) By using the decremental operator

```
let count = 2;
// value is 2, but count becomes 1
let value = count--;
```

Note: In `count--` where the operator is on the right side of the variable being decremented (`count`), the value is assigned to the new variable (in this case `value`) before the decrementing happens. The value of the `value` variable will now still be 2, while that of `count` is 1. However, if you want to decrement the value of `count` first before assigning it to `value` so that `value` will also have the decremented (updated) value of `count`, use `--count` with the `--` operator on the left side of the `count` variable.

2) Assignment operators

The assignment operator is used to assign a value to something. We saw this when learning about variables. It is how we assign values to variables. It is used to assign the value on the right of the operator to the operand on the left of it.

```
let number = 10;
```

This assigns the value of 10 to the `number` variable. It looks like the equal sign (`=`) in mathematics, but in programming, it's used to assign a value.

```
let name;
name = "John Doe";
```

3) Comparison operators

Equality operator

The equal operator `==` is used to check if a value is equal to another value. It checks if the values of both operands match (but not their types). A common mistake is to mistake this for the assignment operator. This equal operator is usually used in conditional statements to verify if the value of a variable for example is equal to a specific value. We will learn all about conditional statements shortly,

when we come to look at conditionals. For now, just remember that you need two mathematical equal signs, not just one, to check if two values are the same. It looks like this:

```
let colour = "green";

if (colour == "green") {
    console.log("The colour is green");
}
```

In this example, we have assigned the value of “green” to a variable `colour`, then used the equal operator (`==`) to check if its value is actually equal to “green”.

Strict equality operator

Also sometimes referred to as the identical operator, it goes beyond the equal operator which only checks if their values are the same, and checks if both values are of the same type as well. It is therefore more strict, and is handy for situations when you need to know that both operands are not only of the same type, but have the same value as well. Here is an example of how their differences can be deceiving:

```
let fiveString = '5';
let fiveNumber = 5;

if (fiveString == fiveNumber)
{
    alert('fiveString and fiveNumber are the same');
}
else
{
    alert('fiveString and fiveNumber are NOT the same');
}
```

The above example will display an alert popup saying 'fiveString and fiveNumber are the same'. JavaScript has a good ability to convert a string of digits into a number when it sees you are trying to use it as a number. This is handy, but it is not what you want in situations where you need to be precise. In such circumstances, just change the expression in the if statement to use an identical operator instead of an equal operator. For example:

```
let fiveString = '5';
let fiveNumber = 5;

if (fiveString === fiveNumber)
{
    alert('fiveString and fiveNumber are the same');
}
else
```

```
{
    alert('fiveString and fiveNumber are NOT the same');
}
```

The above example will display an alert popup saying 'fiveString and fiveNumber are NOT the same'. This is because now, '5' is being taken for what it is; a string, and 5 for what it is; a number, and both are not the same. So always remember that when the == (equal) operator does not seem to work, just use the === (identical) operator and it will save the day as === checks both the value and the type.

Greater than operator

The greater than operator > is the same as the greater than sign in math. For example:

```
let greaterThan = 2 > 1;
```

The value of greaterThan is true.

Less than operator

The less than operator < is the same as the less than sign in math. For example:

```
let lessThan = 2 < 1;
```

The value of lessThan is false because 2 is not less than 1.

Greater than or equal to operator

The greater than or equal to operator >= specifies that the value on the left is either greater than, or equal to the number on the right. It is the same as the greater than or equal to sign in math. For example:

```
let greaterThanOrEqualTo = 2 >= 1;
```

The value of greaterThanOrEqualTo is true because 2 is greater than 1.

Less than or equal to operator

The less than or equal to operator <= specifies that the value on the left is either less than, or equal to the number on the right. It is the same as the less than or equal to sign in math. For example:

```
let lessThanOrEqualTo = 2 <= 2;
```

The value of lessThanOrEqualTo is true because 2 is equal to 2.

- 4. Logical operators Logical operators are used to determine if the value of an expression is true or false.
 - Not operator

- OR operator
- And operator
- Ternary operator

Not operator

The `!` operator, also known as the Not or Logical Not operator, is used to determine whether an expression is not equal to a value. It comes in three forms: `!` Logical NOT `!=` Not Equal `!==` Strict Not Equal (value and type)

For example:

```
let two = 2;
let isTwo = two != 1;
```

The value of the variable `isTwo` will be `true`. This is because the value of the variable `two` is 2 and not 1, so `isTwo` which states that two is not equal to 1 is correct, hence the result is `true`. `!=` is known as the “Not Equal” operator. In the same way, you can flip a `true` or `false` result round using the Logical Not (`!`), or check that two things differ in value or in type using the Strict Not Equal (`!==`).

OR operator

The OR operator `||` works with two conditional expressions, one on either side of the double pipe characters. It states that if the conditional on either side of its double pipe characters (`||`) is `true`, then the result is `true`. It will only return `false` if both conditionals are `false`. Basically, as long as one conditional on either side of its pipe characters is `true`, then the whole expression is `true`. For example:

```
if (trafficLightColor == 'green' || trafficLightColor == 'amber')
{
    console.log("You can go!");
}
```

And operator

The And or `&&` operator is very similar to the OR operator. The only difference is that unlike with the OR operator where if just one conditional is `true`, it passes, with the And operator, both conditionals on either side of it must be `true` for it to return `true`. For example:

```
if (trafficLightColor == 'red' && carStops == false)
{
    console.log("That car has committed a traffic offence!");
}
```

Ternary operator

The ternary operator is a quick way to assign a value to a variable based on a conditional statement. It is a shorthand version of the if...else statement that allows quick assignments. A ternary operator is very powerful and handy. You would typically use it in situations where there is not much code you need to write if an expression is true. The conditional could be as short as one line of code. If you had to write a lot of code, then the block ({}) that an if statement provides would be preferable. With a ternary operator, you probably just need to set a variable's value depending on some condition, so it is quick and short. Here is the syntax:

```
condition ? runIfTrue : runIfFalse;
```

Ternary operators are great when you need to quickly choose between two options. Here is an example:

```
let action = trafficLightColor == "red" ? "Stop" : "Go";
```

The code after the colon acts as the else clause in an if statement. You can see how a ternary operator is simple yet powerful. Using it will make your code very concise and readable.

Nesting multiple ternary operators

You can also chain multiple ternary conditions, just like nesting if...else blocks. Here is the syntax:

```
condition1 ? value1 :  
condition2 ? value2 :  
defaultValue;  
  
let light = 'red';  
  
light == 'green' ? console.log('You may go') :  
light == 'amber' ? console.log('Hurry') :  
console.log('Stop');
```

Here, the value of light is 'red', so the last message, 'Stop', will be printed.

Combining math operators with the assignment operator

Sometimes, you will come across two operators being combined like this: += for example

```
let price = 10;  
let tax = 3;  
let totalPrice = price += tax;
```

It is completely valid, and `+=` means that the value on the right (after the `=` character) is added to the value on the left (before the `+` character), rather than used to replace it. Note and remember that the `+=` operator also works for strings and adds to (extends) a string.

= e.g. <code>j = 3</code>	which means <code>j = 3</code>
<code>+= e.g. <code>j += 2</code></code>	which means <code>j = j + 2</code>
<code>+= e.g. <code>j += 'text'</code></code>	which means <code>j = j + 'text'</code>
<code>-= e.g. <code>j -= 2</code></code>	which means <code>j = j - 2</code>
<code>*= e.g. <code>j *= 4</code></code>	which means <code>j = j * 4</code>
<code>/= e.g. <code>j /= 4</code></code>	which means <code>j = j / 4</code>
<code>%= e.g. <code>j %= 6</code></code>	which means <code>j = j % 6</code>

It is basically a shorthand way of using mathematical operators to assign values to variables, and it is worth knowing, as you are sure to come across it in code. It is effectively a compact and quick way to run an expression and assign its result to a variable in one operation. They are also sometimes referred to as short form operators, or compound operators. It is better demonstrated than explained. For example, instead of saying

```
let num = 2;
num = num * 2;           // num is now 4
```

You can quickly do that like so:

```
let num = 2;
num *= 2;                // num is now 4
```

Other examples:

```
let a = 1;
a += 5;                  // a is now 6
```

```
let b = 6;
b /= 2;                  // b is now 3
```

```
let c = 6;
c *= 2;                  // c is now 12
```

Note that these operators always work on a variable. You cannot write `1 += 5`; because there is nothing there to update - `1` is just the number `1`, and it will always be the number `1`.

The operator always goes before the equal sign: `+=`, `-=`, `*=`, `/=`, `%=` are compound assignment operators.

CONDITIONALS

If statements

In programming, the powerful concept behind writing intelligent systems is the ability of code to perform logic. It does this by evaluating parameters and conditions, then taking action based on the value of those conditions. The topic "conditionals" comes from this idea of a program taking an action based on a condition. This condition is usually an expression that returns true, and so your if code takes an action when that expression evaluates as true. You set up the expression to return true based on whatever value you're checking for. A conditional is therefore used to make decisions in code. The main tool for implementing conditionals in programming is known as an if statement. All programming languages have them. No programming language would be useful without the ability to perform logic and make decisions based on conditions. In JavaScript, an if statement is a block, and its syntax is similar to that of a function. It is written with the word `if`, followed by a pair of parentheses `()`, and then a pair of curly braces `{}`.

Here is the syntax:

```
if (condition) {  
    // code to run (action) if condition is true  
}
```

The condition is always some kind of expression that evaluates to either true or false. It is the answer to that expression which decides whether the code inside the braces runs at all. The above syntax is the shortest version and works well if you have just one condition to check. Let's consider a real-life example. Imagine your program controls a school bell that should ring when the time is exactly 9:00 AM. You'd use an if statement to check that time:

```
if (time == '09:00') {  
    // ring the bell then stop  
}
```

Note: This is not real code for checking the current time in JavaScript. It's simply a demonstration of how the logic would work. Such demonstrations are called pseudo code—a way of showing how code behaves without writing it in a specific programming language.

Checking Multiple Conditions

There will be times when you have more than one condition to check. These multiple conditions can play out in two main ways:

a) Independent Conditions

Let's say the bell should ring not only at 9:00 AM but also at 10:00 AM. The bell ringing at 10:00 has nothing to do with whether it rang at 09:00 or not. The ringing just happens at both times, quite independently of each other. There are different ways to write the if statement, and they will all work. You can either decide to make the one conditional expression check for both times in a single if statement or you can write two separate if statements, one for each time check, with each taking the action necessary.

Recommended – Single if statement using OR (||):

```
if (time == '09:00' || time == '10:00') {  
    // ring the bell then stop  
}
```

Also works – Two separate if statements:

```
if (time == '09:00') {  
    // ring the bell then stop  
}  
  
if (time == '10:00') {  
    // ring the bell then stop  
}
```

While both work, the first approach is preferred because:

- It avoids code duplication
- It's more readable
- It's easier to maintain

Note that here I show just two if statements, but if you are dealing with more than two conditions, you will write as many if statements as there are conditions to handle (check for).

b) Mutually Exclusive Conditions (Two Choices)

In this case, you have two conditions that are mutually exclusive. It means if one event or condition happens, the other cannot happen at the same time. Say for example, you are working with a barrier that opens when a light shows green, and closes when the light is red. In this case the light is the condition (expression) and the behaviour of the barrier (the code you run) is the action you take based on the value of that light—which is what the expression resolves to. In this green or red light scenario, you are dealing with multiple conditions. For each condition, there is an action to be taken, which are to open or close the barrier. JavaScript provides a

system for checking for conditions in such multiple, mutually exclusive scenarios. That is the 'else' clause of the if statement. Here is the syntax:

```
if (condition) {
    // code to run if the condition is true
}
else
{
    // code to run if the condition is false
}
```

Let's see an example of how to use an if...else statement:

```
if (light == 'green') {
    // raise the barrier
} else {
    // light is red, so close the barrier
}
```

c) Mutually Exclusive Conditions (More Than Two Choices)

Now let's say the barrier is controlled by three lights:

- Red: Close the barrier
- Amber: Start closing the barrier
- Green: Open the barrier

Use an if...else if...else chain:

```
if (light == 'green') {
    // raise the barrier
}
else if (light == 'amber')
{
    // start to close the barrier
}
else
{
    // no need to check again - the light is red
    // close the barrier
}
```

Note: when it comes to multiple conditions, whether it is two of them or more, the last conditional check should always be an else clause. This else clause is the 'catch-all' clause. It takes no parentheses because it needs no condition (expression) to check for. It stands for whatever is left over once all the conditions above it have failed to match. In the example above, once you have checked the colour of the light and found it is neither green nor amber, then it can only be red.

Additional notes on conditionals

Here is a list of important points to keep in mind in order to understand how to work with conditionals.

- You can nest as many else if blocks as needed.
- In JavaScript, else if must have a space between the else and the if keywords.
- The final else block is optional but recommended to handle any unexpected or unhandled cases.
- Curly braces {} are optional for single-line statements:

```
if (true) console.log("This works without braces.");
```

But if you have multiple lines, you must use curly braces:

```
if (true) {  
    console.log("This is line 1.");  
    console.log("This is line 2.");  
}
```

Best Practice: Always use curly braces—even for one-liners—for readability and to avoid bugs.

LOOPS

In programming, a loop gives your code the power to repeat actions over and over—without you having to write them again and again. This is super useful when you're working with multiple pieces of data, like items in an array, characters in a string, or keys in an object. For example, imagine having a list of 100 fruits—you wouldn't want to write the same line of code 100 times just to print them all out. A loop saves you from that by doing the repetition for you.

Why are loops important? Loops are one of the most powerful features in any programming language because they allow your program to:

- Do something repeatedly as long as a condition is true.
- Go through a list of items one by one and take action on each.
- Look through data to find something or change things.
- Handle different layers of information, especially in nested arrays or complex data.

Here is how loops work. When you create a loop, you usually define a counter variable—this is most often called *i*. If your loop is going through a simple list, you'll use *i*. If it's a loop inside another loop (like in a multi-level array), the seco-

nd counter is often called j. And if you need more, you can keep going with k, l, and so on. Here's a quick look at how you might retrieve an item from an array using a counter:

```
let myArray = ['boy', 'girl'];  
let girl = myArray[1];  
// 'girl' is at index 1
```

Remember, arrays in JavaScript always start counting from zero (0), so the first element is at index 0, the second at index 1, and so on. JavaScript gives you several types of loops, and each one is useful in different situations. You've already seen some loops when we talked about arrays, but now let's look at all the main loop types available in JavaScript. There are 6 main types of loops to master:

- 1. For loop – The most common type, great for running a block of code a specific number of times.
- 2. While loop – Keeps running as long as a condition is true.
- 3. Do...while loop – Similar to while, but runs the code at least once, even if the condition is false.
- 4. For...in loop – Used to loop through the properties of an object.
- 5. For...of loop – Used to loop through values in arrays, strings, and other iterable objects.
- 6. forEach loop – A special array method that loops through array items and lets you run a function on each one.

Each of these loop types has its own structure and best use case, and we'll go through all of them one by one in this chapter.

1) The For loop

Use a for loop when you know how many times the loop should run. It is by far the most commonly used loop, and it has a very simple and easy to read syntax. The action inside its curly braces is only run while the condition in its parentheses is true. That condition is entirely up to you: very often it counts up to the number of elements in an array, but as the example below shows, it does not have to involve an array at all. Here is the syntax:

```
for (initialisation; condition; increment or decrement) {  
  // Code to execute  
}
```

Example:


```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

The output will be each number printed on its own line:

0 1 2 3 4

2) The While loop

Used when you don't know how many times the loop should run, because that depends on a condition. The while loop is very similar to the for loop in that it runs only when a condition is true. The key difference is flexibility: unlike the for loop, which often checks the number of elements in an array, the while loop can check any condition at all. That means you can use it in a wide variety of situations—not just for arrays, but for any kind of condition you want to keep checking.

Here is the syntax:

```
while (condition) {  
  // code to run while condition is true  
}
```

Here's how it works:

- The loop checks the condition.
- If it's true, it runs the code inside the curly braces.
- Then it checks the condition again, and runs the code again if it's still true.
- It keeps going until the condition becomes false.

Be Careful: Infinite Loops! One important thing to remember is that you must change something inside the loop to make the condition false eventually. If the condition never becomes false, the loop will run forever! This is called an infinite loop, and it can crash your browser or freeze your computer.

Example: A loop that never ends

```
let trafficLightColor = 'green';  
  
while (trafficLightColor !== 'red') {  
  console.log('You can go');  
}
```

In this example, the condition is always true because `trafficLightColor` never changes—so the loop keeps running forever.

Fixed Version (Simulating the Light Changing)

```
let trafficLightColor = 'green';
let timesChecked = 0;

while (trafficLightColor !== 'red') {
  console.log('You can go');

  // Simulate the light changing after 3 checks
  timesChecked++;

  if (timesChecked === 3) {
    trafficLightColor = 'red';
  }
}
```

This version will print "You can go" three times, and then stop once the light changes to red. Much better!

Let us look at an example of using a while loop to loop through an array. You can also use a while loop to go through an array, just like a for loop. Here's an example:

```
let i = 0;
let fruit = ['banana', 'apple', 'kiwi'];

while (i < fruit.length) {
  document.write('The fruit is ' + fruit[i] + '<br>');
  i++;
}
```

What it prints on the screen:

The fruit is banana The fruit is apple The fruit is kiwi

Here, the while loop runs while *i* is less than the number of fruits in the array. It prints each fruit one by one and then stops when there are no more fruits left.

3) The Do...while loop

do...while loops are less common than other types of loops, but they are useful in situations where a block of code needs to run at least once before a condition is checked. A good example would be a website where every user must have at least one profile picture, but can optionally upload more. You might want to display the first picture by default, and then show any extra ones only if they exist. A do...while loop is perfect for this, because it will always run once—showing the first picture—before checking whether additional pictures are available to display. Syntax The syntax for a do...while loop is slightly different from a regular while loop. In a while

loop, the condition comes before the block of code. In a do...while loop, the block runs first, and then the condition is checked afterwards:

Here is the syntax:

```
do {  
    // code to run at least once  
} while (condition);
```

It might feel unusual at first, but it makes sense once you realise the purpose: run something once, then decide if it should run again.

Example:

```
let i = 0;  
let fruit = ['banana', 'apple', 'kiwi'];  
  
do {  
    console.log('The fruit is ' + fruit[i]);  
    i++;  
} while (i < fruit.length);
```

The output will be:

The fruit is banana The fruit is apple The fruit is kiwi

This loop runs once to display the first fruit, then continues checking if more items are available in the array before each additional run.

4) The For...in loop (Used for Objects and Arrays)

Used to loop over object properties. A word of caution: it can also loop over arrays, but it is not recommended because it may include inherited properties.

Here is the syntax:

```
for (let key in object) {  
    // Code to execute  
}
```

Example (Object Iteration):

```
let person = { name: "Alice", age: 25, city: "New York" };  
  
for (let key in person) {  
    console.log(`${key}: ${person[key]}`);  
}
```

The output will be:

name: Alice age: 25 city: New York

5) The For...of loop (Used for Arrays, Strings, Maps, Sets, etc.)

It offers a cleaner way to iterate over iterable objects like arrays and strings.

Here is the syntax:

```
for (let value of iterable) {  
    // Code to execute  
}
```

Example (Array Iteration):

```
let fruits = ["Apple", "Banana", "Cherry"];  
  
for (let fruit of fruits) {  
    console.log(fruit);  
}
```

The output will be:

Apple Banana Cherry

6) The forEach() Loop (Used for Arrays)

The `forEach()` method is an array method that executes a function on each element.

Here is the syntax:

```
array.forEach((value, index, array) => {  
    // Code to execute  
});
```

Example (Array Iteration):

```
let numbers = [10, 20, 30];  
  
numbers.forEach(num => {  
    console.log(num);  
});
```

The output will be:

10 20 30

Here is a conclusive summary of the various loops and when they should be used:

Loop type	When to use it
<code>for</code>	When the number of iterations is known.
<code>while</code>	When the number of iterations is unknown and depends on a condition.
<code>do...while</code>	When you need to run the loop at least once.
<code>for...in</code>	When iterating over objects (not recommended for arrays).
<code>for...of</code>	When iterating over arrays, strings, maps, or sets.
<code>forEach()</code>	When iterating over an array with a function callback.

Loop control statements

In a loop, things usually run over and over until a condition says “stop.” But what if you want to take control of the loop and tell it to either:

- Exit early, even if the condition hasn’t finished yet, or
- Skip one turn in the loop and go to the next round?

This is where loop control statements come in. These are special keywords in JavaScript that give you, the programmer, more control over how a loop behaves. There are two main loop control statements, and these are the `break`, and the `continue` statements.

The `break` statement

The `break` statement tells the loop to stop running completely and jump out of the loop when a certain condition is met. This is useful when you're looking for something, and once you've found it, there's no need to keep looping. It exits the loop immediately when a condition is met. You as the programmer will place this statement inside the loop at the spot where you expect the loop to have achieved its objective—usually in a conditional statement in which case the condition would have matched a target case. You therefore want the program to exit the loop.

Here is an example:

```
for (let i = 0; i < 10; i++) {
  if (i === 5) {
    // Exit the loop when i is 5
    break;
  }

  console.log(i);
}
```

The output will be:

0 1 2 3 4

As you can see, the loop stops as soon as *i* becomes 5. The number 5 is never printed because the loop exited right before that.

The continue statement

The continue statement is a little different. It doesn't stop the whole loop—it just skips the current iteration and goes straight to the next one. This is helpful when you want to ignore certain values but still finish the loop.

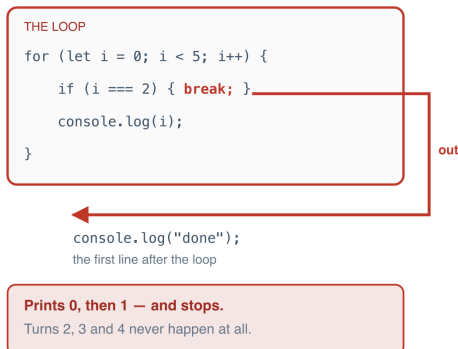
```
for (let i = 0; i < 5; i++) {  
  if (i === 2) {  
    // Skip printing when i is 2  
    continue;  
  }  
  
  console.log(i);  
}
```

The output will be:

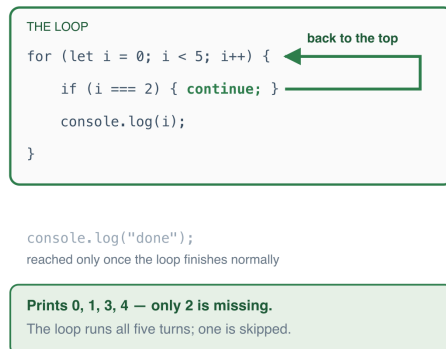
0 1 3 4

Where each one sends you next

break — jumps right out of the loop



continue — skips just this one turn



- Figure 5.1 — Where break and continue send you next*

Switch statement

The switch statement in JavaScript is used for conditional branching. It allows you to execute different blocks of code based on the value of an expression, making it a cleaner alternative to multiple if...else if...else statements.

Here is the syntax:

```

switch (expression) {
  case value1:
    // Code to execute if expression ===
    //value1
    break;
  case value2:
    // Code to execute if expression ===
    // value2
    break;
  default:
    // Code to run if no case matches
}

```

This is how it works:

- The expression is evaluated once.
- Its value is compared with each case.
- If a match is found, the block of code in that matching case runs.
- The break statement prevents fall-through (executing the next case unintentionally). This will cause the code execution to stop checking the other case statements when a match has already been found.
- The default case (optional) runs if no match is found. It acts as the fallback default, just as its name indicates.

Here is an example switch statement. Let's write one that checks for days of the week.

```

// Example: 3 means Wednesday
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  case 4:
    console.log("Thursday");
    break;
  case 5:
    console.log("Friday");
    break;
  case 6:
    console.log("Saturday");
}

```

```
    console.log("Sunday");  
        break;  
    default:  
        console.log("Invalid day");  
}
```

This will be the output:

Wednesday

We got that output because we initialised the value of the 'day' variable to 3. Note that you could place this switch statement inside a function that accepts the variable to test for in the case statements as its argument. In such a case you would create or set a variable within the case blocks so that it stores some data that you need in the case that a match is made. You would then return that variable at the end of the function.

QUIZ — Control Flow

This page contains the Q & A (questions and answers) for this chapter — Chapter 5: Control Flow. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 11 to 15 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What does this print, and why?

```
let fiveString = '5';
let fiveNumber = 5;

console.log(fiveString == fiveNumber);
console.log(fiveString === fiveNumber);
```

Clue: one of these two operators cares about the type of the value as well as the value itself.

2. What does the % operator give you, and how would you use it to test whether a number is even?

Clue: it hands back what is left over after a division.

3. What are the values of value and count after each of these two lines?

```
let count = 1;
let value = count++;
```

And what would change if you wrote ++count instead?

Clue: it depends on which side of the variable the ++ sits.

4. Rewrite each of these using a compound assignment operator:

```
j = j + 2;
j = j * 4;
j = j % 6;
```

Clue: the operator always goes before the equal sign.

5. Rewrite this if...else as a single line using the ternary operator:

```
let action;

if (light == 'green') {
```

```

    action = 'Go';
  } else {
    action = 'Stop';
  }

```

Clue: the shape is condition ? runIfTrue : runIfFalse.

6. In an if...else if...else chain, what is the else clause for, and why does it take no parentheses?

Clue: the chapter calls it the catch-all.

7. Match each job to the loop you would reach for:

- a) You want to run something exactly 10 times
- b) You want to keep going until something happens, and you have no idea how many turns that will take
- c) You want the code to run at least once before the condition is even checked
- d) You want to go through the named properties of an object
- e) You want to go through the values in an array, simply and readably

Clue: five of the six loop types from this chapter, one each.

8. This loop never stops. Why, and how would you fix it?

```

let trafficLightColor = 'green';

while (trafficLightColor !== 'red') {
  console.log('You can go');
}

```

Clue: ask yourself what would ever make the condition false.

9. What is the difference between break and continue?

Clue: one of them leaves the building. The other just skips a turn.

10. What does this print, and what is the name for what has gone wrong?

```

let day = 2;

switch (day) {
  case 1:
    console.log("Monday");
  case 2:
    console.log("Tuesday");
  case 3:
    console.log("Wednesday");
  default:
    console.log("Invalid day");
}

```

Clue: look very carefully at what is missing from every single case.

11. EXERCISE. Create an array in a constant containing people's names. Use a while loop to loop through the array and display the elements on screen. This will help you practise working with while loops.

Clue: you will need a counter starting at 0, and the loop should keep going while that counter is less than the array's length. Do not forget to increase the counter inside the loop, or you will meet question 8 again.

12. EXERCISE. Create a numbers array containing the numbers 1 to 10, stored in a constant. In a while loop, loop through the numbers and use the modulus operator to check whether each number is even, and if so display it on screen.

This will help you practise using while loops and the less commonly used but powerful modulus operator.

Clue: a number is even when dividing it by 2 leaves nothing over.

13. EXERCISE. Create a numbers array containing the numbers 1 to 10, stored in a constant. Create three empty arrays: `evenNumbers`, `divisibleBy3` and `otherNumbers`. Loop through the numbers array with a while loop and, using an `if...else` if conditional and the modulus operator, do the following checks:

- i) check whether the number is divisible by 2, and put it in `evenNumbers` if it is
- ii) check whether the number is divisible by 3, and put it in `divisibleBy3` if it is
- iii) otherwise put the number in `otherNumbers`

Once out of the loop, display the contents of all three arrays.

This exercise is meant to help you master how `if...else if` statements work. When you look at your results, check the `divisibleBy3` array carefully — there is a lesson hiding in it.

Clue: use `push()` to add to an array, and `join(', ')` to display one neatly.

14. EXERCISE. Practise the `do...while` loop.

Create an array called `names` containing some names. Use a `do...while` loop to go through the `names` array and display them on screen.

Clue: remember the block runs first and the condition is checked afterwards.

15. EXERCISE. Practise using the ternary operator.

Create an array named `light` with one string element, `'green'`, in it. Create an action variable and use a ternary operator to give it the value `'Go'` if the string in the `light` array is `'green'`, or `'Stop'` if it is `'red'`. Use an alert popup to display the value of action.

Clue: the array has only one element, so it sits at index 0.

ANSWERS

1. It prints:

```
true false
```

The `==` operator compares only the values. JavaScript is happy to convert the string `'5'` into the number `5` in order to make the comparison, so it reports them as equal.

The `===` operator, sometimes called the identical or strict equality operator, compares the type as well as the value. `'5'` is a string and `5` is a number, so they are not the same, and it reports false.

The rule of thumb: when `==` does not seem to behave, reach for `===`, because it checks both the value and the type.

2. The `%` operator, called modulo, gives you the remainder left over after dividing one number by another.

```
console.log(10 % 5);  
// 0, because 5 goes into 10 exactly  
console.log(11 % 2);  
// 1, because 2 goes into 11  
// five times with 1 left over
```

To test whether a number is even, divide it by 2 and see whether anything is left over. If nothing is, the number is even:

```
let number = 7;  
  
if (number % 2 == 0) {  
    console.log("Even number");  
} else {  
    console.log("Odd number");  
}
```

3. After those two lines, value is 1 and count is 2.

With the `++` on the right of the variable (`count++`), the current value is handed over first, and only then is count increased. So value receives the old value, 1.

With the `++` on the left (`++count`), the increase happens first, and the new value is handed over. So value would be 2, and count would be 2 as well.

The decrement operator `--` works in exactly the same way, only downwards.

4. `j += 2;`
`j *= 4;`
`j %= 6;`

These are called compound assignment operators, or sometimes short form operators. They are a compact way of running an operation and storing the result back into the same variable in one go.

Remember that they only work on a variable. You cannot write `1 += 5`, because there is nothing there to update — 1 is just the number 1, and it will always be the number 1.

5. `let action = light == 'green' ? 'Go' : 'Stop';`

The part before the question mark is the condition. The part between the question mark and the colon runs if it is true, and the part after the colon acts as the else clause.

Ternaries are ideal when all you are doing is choosing between two values. If either branch needs more than that, use a full `if...else`, which is far easier to read.

6. The else clause is the catch-all. It stands for whatever is left over once every condition above it has failed to match.

It takes no parentheses because it has nothing to check. Every other branch in the chain needs a condition inside parentheses; else is simply "in all other cases, do this".

So in the traffic light example, once you have checked whether the light is green and then whether it is amber, and neither matched, the else branch can only mean red.

7.

Job	Loop
a) exactly 10 times	→ the for loop
b) unknown number of turns	→ the while loop
c) run at least once first	→ the do...while loop
d) properties of an object	→ the for...in loop
e) values in an array, readably	→ the for...of loop

The sixth, `forEach()`, is an array method rather than a loop keyword. You would use it when you want to run a function on every element of an array.

8. It never stops because nothing inside the loop ever changes `trafficLightColor`. The condition is true when the loop starts and stays true forever. This is an infinite loop, and it can freeze your browser.

The fix is to make sure something inside the loop can eventually make the condition false:

```
let trafficLightColor = 'green';
let timesChecked = 0;

while (trafficLightColor !== 'red') {
  console.log('You can go');

  timesChecked++;

  if (timesChecked === 3) {
    trafficLightColor = 'red';
  }
}
```

This prints "You can go" three times and then stops. Whenever you write a while loop, ask yourself straight away: what will eventually make this condition false?

9. `break` leaves the loop altogether. Execution jumps to the first line after the loop, and no further turns happen at all.

`continue` skips only the current turn. The loop carries on with the next one.

```
for (let i = 0; i < 5; i++) {
  if (i === 2) { break; }
  console.log(i);
}
// prints 0, 1

for (let i = 0; i < 5; i++) {
  if (i === 2) { continue; }
  console.log(i);
}
// prints 0, 1, 3, 4
```

10. It prints:

Tuesday Wednesday Invalid day

What has gone wrong is called fall-through, and it is the single most common mistake made with switch statements.

Every case is missing its `break`. Once a case matches, JavaScript runs its code and then carries straight on into the next case, and the next, all the way to the bottom, without testing any of them again. It matched case 2, and then simply fell through everything below it.

Adding `break` to each case fixes it, and the output becomes just Tuesday:

```
case 2:
  console.log("Tuesday");
  break;
```

This is exactly why the break statement matters so much in a switch, and it is worth checking for whenever a switch behaves strangely.

11. Here is the code:

```
const people = ['Alice', 'Bob', 'Charlie', 'Diana'];
let i = 0;

while (i < people.length) {
    document.body.innerHTML += people[i] + '<br>';
    i++;
}
```

This displays the names like so:

Alice Bob Charlie Diana

We loop through the people array with a while loop and print each name to the screen. Note the use of += on document.body.innerHTML, which keeps adding each name to what is already there instead of wiping it out and replacing it.

12. Here is the code:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
let i = 0;

while (i < numbers.length) {
    if (numbers[i] % 2 === 0) {
        document.body.innerHTML += numbers[i];
    }

    i++;
}
```

We loop through the numbers and use the modulus operator (%) to check whether each one is divisible by 2, which is to say even. If it is, we print it. This displays only the even numbers:

246810

They run together because we are printing each one straight after the last with nothing in between. If you would rather see them separated, add something to the end:

```
document.body.innerHTML += numbers[i] + ' ';
```

13. Here is the code:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const evenNumbers = [];
const divisibleBy3 = [];
const otherNumbers = [];
```

```

let i = 0;

while (i < numbers.length) {
  if (numbers[i] % 2 === 0)
  {
    evenNumbers.push(numbers[i]);
  }
  else if (numbers[i] % 3 === 0)
  {
    divisibleBy3.push(numbers[i]);
  }
  else
  {
    otherNumbers.push(numbers[i]);
  }

  i++;
}

document.body.innerHTML += 'Even Numbers: ' + evenNumbers.join(', ') +
'<br>';
document.body.innerHTML += 'Divisible by 3: ' + divisibleBy3.join(', ')
+ '<br>';
document.body.innerHTML += 'Other Numbers: ' + otherNumbers.join(', ')
+ '<br>';

```

The result on screen is:

Even Numbers: 2, 4, 6, 8, 10 Divisible by 3: 3, 9 Other Numbers: 1, 5, 7

Now, the lesson hiding in there. The `divisibleBy3` array should arguably contain 6 as well, and it does not. That is because 6 is divisible by 2, so it matched the very first condition, and once a branch of an `if...else if` chain matches, none of the branches below it are even looked at. 6 went into `evenNumbers` and never got the chance to be tested against 3.

That is the whole point of an `else if` chain: at most one branch ever runs.

If you want a number to be able to land in more than one array, use a series of separate `if` statements instead, with no `else` at all, so that every check runs regardless of what the ones before it did:

```

while (i < numbers.length)
{
  if (numbers[i] % 2 === 0)
  {
    // it is an even number
    // (exactly divisible by 2)
    evenNumbers.push(numbers[i]);
  }
}

```



```

{
    // it is divisible by 3
    divisibleBy3.push(numbers[i]);
}

if ((numbers[i] % 2 !== 0) && (numbers[i] % 3 !== 0))
{
    otherNumbers.push(numbers[i]);
}

i++;
}

```

This is perfectly legal code. It just means every if statement is checked every time, regardless of what any of the others did. Now 6 appears in both evenNumbers and divisibleBy3.

14. Here is the code:

```

const names = ['John', 'Susan', 'Mary', 'Ann'];
let i = 0;

do {
    document.body.innerHTML += names[i] + '<br>';
    i++;
} while (i < names.length);

```

This prints:

John Susan Mary Ann

To prove to yourself that a do...while really does run its first turn before checking anything, try adding a condition that says to stop when the name 'John' is met:

```

const names = ['John', 'Susan', 'Mary', 'Ann'];
let i = 0;

do {
    document.body.innerHTML += names[i] + ',';
    i++;
}
while ((i < names.length) && (names[i] !== 'John'));

```

This still runs through all four names and displays John, Susan, Mary, Ann. The first turn happens regardless, without the condition being looked at, and by the time it is checked we are past John, so nothing else matches it.

Change the name in the condition to one further along and you will see the loop stop:

```

while ((i < names.length) && (names[i] !== 'Mary'));

```

This runs only twice, because 'Mary' is the third name, so only John and Susan are printed.

15. Here is the code:

```
const light = ['green'];  
const action = (light[0] === 'green') ? 'Go' : 'Stop';  
  
alert(action);
```

As it stands, the alert popup displays 'Go'. Change the string in the light array to 'red' and refresh the page, and the popup will say 'Stop' instead.

Note that light is an array with a single element, which is why we read it with light[0] rather than just light. Comparing the whole array to 'green' would never be true.

Chapter 6 — REGULAR EXPRESSIONS

Intro Features Grouping Classes Range Negation Anchor characters Shorthand meta characters Quantifiers Modifiers The difference between g and m Regular expressions in JavaScript

A regular expression (also referred to as regex) is a sequence of characters that forms a search pattern. It is used for string matching, searching, and manipulation by defining specific patterns that help identify and extract parts of a string.

Here are its features:

- i) Pattern Matching: Regular expressions allow you to define patterns that match specific sequences in text. For example, finding email addresses, phone numbers, or validating formats like dates.
- ii) Search and Replace: Regex can be used to find occurrences of patterns in text and replace them with new strings.
- iii) Flexible and Powerful: It provides a flexible way to define complex string patterns using literals, wildcards, quantifiers, and special characters.

Some common use cases for regular expressions are:

- Validation: To validate email addresses, URLs, phone numbers, etc.
- Text Search: To find specific patterns within large text files.
- String Manipulation: To extract or replace certain parts of a string, like removing extra spaces or converting formats.

Regular expressions are a powerful tool for working with strings in various programming languages, including PHP, Python, JavaScript, and many others. They are widely used in text processing tasks for pattern matching and validation. One thing worth knowing early on is that regular expressions are not a JavaScript invention. The patterns themselves are much the same wherever you go, which is why learning them once pays off in every language you ever pick up afterwards. What differs from language to language is the wrapper around them: how each one hands you the tools to run a pattern against a string. JavaScript gives you built-in functions such as `test()` and `replace()`, which we will come to later in this chapter. Other languages have their own equivalents. The pattern in the middle stays the same; only the way you reach for it changes. Here are some of the most popular characters available:

* . + [] () {}

With regular expressions, you can construct a powerful pattern-matching algorithm using just a single expression. The syntax is simple: it starts with a forward slash and ends with a forward slash. You have to place the pattern to be matched between them. The pattern is made up of special characters called meta characters. We will proceed by listing the patterns with their meanings, and separate them under topics according to their behaviours.

`/.`

/ An asterisk means "zero or more of whatever comes just before it". This is important, because it never works on its own. Written by itself, /

`/` is not even a valid pattern, since there is nothing in front of it to repeat. Put it after a dot, as in `./`, and you get what people usually mean when they say "match anything, or nothing at all".

`./` A dot character. It matches all kinds of characters except a newline (`\n`). It has a limitation, however; it will match only a single character, so `./` will match everything on the page or subject string by singling them out one by one. That means, if you use it on a string of characters, it will match and stop at the first character. That is why it is better to use it in a combination with for example `+` or `*` to expand its capability.

`./+` Means "one or more of whatever comes just before it". Like the asterisk, it needs something in front of it to work on, so it is always used in a combination. Unlike the asterisk, it will not match nothing—as in, if no character appears in the subject string at all, it will not match.

`/<.>/` It will match anything that comes between '`<`' and '`>`' characters. This will match the beginning names of an HTML tag. It won't match blank tags (`<>`). Another limitation which we already know is; it will match only tags with single letters. For example, it will match `<p>` and `<b>` but will not match `<em>`.

`/<.*>/` Extends the above pattern, fixing its limitations. It will match blank tags, and also tags with multiple letters. This is because the `*` (asterisk) character can also match nothing as well as anything. Combining meta characters like this therefore is useful because they are all combined and executed. Your aim is to have as intelligent an app as possible that will not leave potential matches unmatched. The effort to match as much as possible is known as 'fuzzy character matching'.

`/<.+>/` It can be used instead of `/<.*>/` if you do not want to match blank tags (`<>`). This `+` character has a limitation however. In this case, it will keep matching till the last '`>`' on the line of text. This means that you may end up with a match like this:

```
<p><b>Hello</b></p>
```

This is not ideal, and not what you may want. There is a better way to use the + character. Here it is:

```
/<[^>]+>/
```

Read it as: match an opening angle bracket (<), then one or more characters that are NOT a closing angle bracket, then a closing one. Because [^>] refuses to match a >, the match is forced to stop at the very first > it meets, rather than running on to the last one on the line. That is exactly what we wanted. Note that the [...] and the + characters are a combination.

/?/ Matches an element zero or only 1 time. It is used to make matches on a specific element optional in the subject string.

If you ever need to literally match a character which happens to be regex meta character, you can do that by simply placing a backslash in front of the character. This is known as escaping the character so that your code parser does not treat the character as a regex pattern to use for matching, but to match the character in the subject string literally as it is given. For example, to match the digits 5.0, you need to escape the dot (.) in it like so:

```
/5.0/
```

You may not be sure of how many characters or digits will come after the dot character. For example it could be 5.0, or 5.00 or 5.000 etc. The solution to this is to simply add an asterisk after the last zero like so

```
/5.0*/
```

The backslash can escape anything including a backslash itself, which is useful in case there is a backslash in the string you are matching. Note that there are some so-called shorthand matching characters that start with a backslash. I will provide a list of these later below.

In regular expressions, there are the following concepts:

- grouping
- classes
- ranges
- negation

GROUPING

Grouping is done with a combination of a pair of parentheses followed by a meta character like + or * The parentheses enclose the thing in the subject string to be matched, while the meta characters after the parentheses tell the system how many times to group the matched string. The matched string could be a string of numbers. Here is an example:

```
/1(,000)+/ will match any of these:  
- 1,000  
- 1,000,000  
- 1,000,000,000 etc
```

The parentheses hold the group, and the + after them says "one or more of that group". So the pattern matches a 1 followed by any number of ,000 groups. Take care not to leave a space before the closing slash. A space inside the pattern is not decoration; it is a character that has to be matched like any other. Writing /1(,000)+ / would mean "...and then a space", so it would no longer match 1,000 at all unless a space happened to follow it. If you do want to allow an optional space after each comma, say for numbers written as 1,000, 000, then ask for it explicitly:

```
/1(,\s?000)+/
```

where \s means a space character and the ? after it makes that space optional.

CLASSES

A class is simply a pair of square brackets with the string to be matched between them. There will be a successful match if any of the characters in the bracket is found in the subject string. For example:

/gr[ae]y/ This will match 'gray' 'grey'. Any meta characters after the square brackets will tell the system to repeat that match any number of times. If you modify the pattern like /gr[ae]+y/ then it will match 'greedy' and 'graay' and 'greay' and 'graey'. Just understand that without the + character after the square brackets, each character between the brackets will be matched only once.

An alternative way to achieve the same outcome is to use a pipe character like so: /a|e/ You simply place a pipe character between the characters you want to match either of. This approach is not a class, but I am showing you here just for informative purposes.

RANGE

A range will match any character within a range. `/[0-9]/` will match any number between 0 and 9. Just place a hyphen between the two numbers. `[5-8]` will match any number between 5 and 8.

`/\d/` is a shorthand to match any single digit number. It is exactly the equivalent of the pattern above, `/[0-9]/`.

NEGATION

You use this to create a pattern that the match must not be. Basically, you are saying that a match should be everything but not this negation pattern. You create a negation pattern by placing a caret character as the first thing inside a pair of square brackets. Every other character that follows the caret character inside the brackets is a negation pattern. If the square bracket is followed by a meta character, for example `+` or `*`, then the square bracket and the meta character are a combination. Here is one you have already seen:

`/[^0-9]/` matches any single character that is NOT a digit

And the tag pattern from earlier:

```
/<[^>]+>/ matches a < , then one or
more characters that are not
a > , then a >
```

ANCHOR CHARACTERS (^ and \$)

Outside of the use of square brackets, there are two other characters:

- the ‘anchor’ (to establish start position of) the search string, for a match to occur. The anchor is made up of both the caret (^) and the dollar (\$) characters.

The caret does two quite different jobs depending on where you put it. Inside a pair of square brackets, as we saw a moment ago, it means negation. Outside them, at the very start of a pattern, it means "the match must begin at the start of the subject text". So if the caret character appears at the start of your regex pattern, then the text to be matched must be at the start of the subject text for a match to occur. On the other hand, if the \$ character is placed at the end of your regex, then the string being matched must be at the end of the line of text for a match to be made. For example, to match a subject string that has the text: “Le Guin” and nothing else, just anchor the two ends to make sure our text starts and finishes the line like so:

`/^Le *Guin$`

The `^` pins the match to the start of the text and the `$` pins it to the end, so the subject must be exactly "Le Guin" and nothing more. The `*` after the space allows for any extra spaces between the two words, so "Le Guin" matches too. Leave the `^` off and the pattern would happily find "Le Guin" at the end of a longer sentence, which is not what we asked for.

Shorthand meta characters

These are shorthand characters for frequently used characters, built into regexes.

`\b`/ matches a word boundary

`\B`/ matches anywhere that is NOT a word boundary

`\d`/ single digit

`\D`/ single non-digit

`\n`/ newline character

`\s`/ white space character

`\S`/ non-white space character

`\t`/ tab character

`\w`/ word characters (a-zA-Z0-9_)

`\W`/ non-word character, so anything but any of these: a-z, A-Z, 0-9 and _

Note that every one of these begins with a backslash, and that there is no space after the opening slash. A space inside a pattern is a character like any other, so `/\d/` would mean "a space followed by a digit", which is not the same thing at all.

Quantifiers

In regular expression (regex) terminology, the curly braces `{ }` are referred to as quantifiers. They are used to specify the exact number of occurrences, or a range of occurrences, of a preceding element. You can use a quantifier in 3 ways:

i) `/[w]{3}/` with ONE digit. This means you wish to match a word character if it appears exactly three times.

ii) `{3,}` with ONE digit and a comma. This means you wish to match the preceding character if it appears 3 or more times.

iii) {2,3} with TWO digits separated by a comma. This means you wish to match the preceding character if it appears anything from two to three times.

Take care not to put a space after that comma. {2,3} is a quantifier, but {2, 3} is not one at all — JavaScript gives up on reading it as a quantifier and looks for those exact characters in the text instead.

Modifiers

These are special characters that you place outside, but at the end of a regex pattern to influence how the complete match will be made. There are three (3) modifiers, global (g), case-insensitive (i), and multiline (m). Let's dive straight into some examples, assuming that `/.../` contains your regex pattern:

`/.../g` This will match in a global manner, rather than at the first match that is encountered. You could end up with multiple matches rather than just one. Say you are matching any occurrences of the word 'cat' in a subject string, if the string had 4 occurrences of the word 'cat', it will return all 4 occurrences rather than just 1 for the first one.

`/.../i` This makes the match case-insensitive. Matches are normally case-sensitive, so having this will make the following regex patterns work in the same way:

`/[a-zA-Z]/` and `/[a-z]/i`

or

`/[A-Z]/i` and `/[a-z]/i`

`/.../m` This enables multiline mode. It is easy to misread this one, so it is worth being careful. It does NOT mean the match can run across several lines of text. What it changes is the meaning of the two anchors: with m in place, `^` matches at the start of every line rather than only the start of the whole text, and `$` matches at the end of every line rather than only the end. So it is about where each line begins and ends, not about matching through them.

The difference between g and m

These two get confused constantly, because both of them sound as though they mean "match more". They do quite different jobs, and it is worth pinning down which is which.

g is about HOW MANY matches you get back. m is about WHERE the `^` and `$` anchors apply.

That is the whole of it. Here is a subject string of three lines to try them on:

```
const text = "cat sat here\ndog barked\ncat slept";
```

First, g. Without it you get the first match and nothing more. With it you get all of them:

```
text.match(/cat/) // ["cat"] text.match(/cat/g) // ["cat", "cat"]
```

Now m. Our text has the word "here" at the end of the FIRST line, but the string as a whole ends with "slept". So \$ on its own finds nothing, because \$ means the end of the whole string. Add m and \$ starts meaning the end of any line, so it finds it:

```
text.match(/here$/) // null text.match(/here$/m) // ["here"]
```

And because they do separate jobs, they combine. Our text has "cat" at the start of two different lines:

```
text.match(/^cat/g) // ["cat"] text.match(/^cat/gm) // ["cat", "cat"]
```

With g alone you get one, because ^ still means the start of the whole string, and there is only one of those. Add m as well and ^ applies to every line, so both are found. A useful way to remember it: g asks "how many times?", m asks "what counts as a line?".

g and m answer two different questions

g asks HOW MANY matches. m asks what ^ and \$ MEAN.

The text (three lines):

```
cat sits
cat naps
cat runs
```

Pattern	Finds	Why
<code>/^cat/</code>	<code>["cat"]</code>	<code>^</code> means start of the WHOLE text; stops at one
<code>/^cat/g</code>	<code>["cat"]</code>	keep looking, but <code>^</code> still means the very start
<code>/^cat/m</code>	<code>["cat"]</code>	<code>^</code> now means start of ANY line, but stops at one
<code>/^cat/gm</code>	<code>["cat", "cat", "cat"]</code>	both: every line start, and keep going

Neither flag replaces the other. Most of the time you want both.

• Figure 6.1 — g and m answer two different questions*

Regular expressions in JavaScript

Regular expressions are used in JavaScript only with two methods. These methods are:

- i) `test()` — — — returns boolean
- ii) `replace()`

i) `test()`

You call this `test()` function on your regex pattern string. You pass to it as an argument, the subject string (the string you want to run the test on) and it will tell you if there is a match with your regex pattern or not. Basically, it returns boolean (true or false). For example:

```
console.log(
  /cats/i.test("Cats are fun. I like cats.")
);
```

This will return true to the console.

If you want to see what was actually found rather than just true or false, use `match()` on the string instead:

```
console.log(
  "Cats are fun. I like cats.".match(/cats/i)
);
```

That one prints an array whose first item is the text it matched, in this case "Cats". Note the capital C: the `i` modifier made the match case-insensitive, but what comes back is the text exactly as it appeared in the subject string.

ii) `replace()`

This function is used to match and replace a string from within another text (subject string). Unlike the `test()` function which you have to call on your regex pattern string, you call the `replace()` on the subject string (the string to make the match on). It accepts two arguments; first, the regex pattern string to match with, and what to replace the matched string with. For example, to do a case-insensitive global match on all occurrences of the string 'cats' and replace them with 'dogs', do it like so:

```
console.log(
  "Cats are fun. I like cats".replace(/cats/gi, "dogs")
);
```

The limitation of `replace()` is that it replaces the subject with exactly the text you give, even though you may have wished to use uppercases in some instances and lowercases in others.

QUIZ — Regular Expressions

This page contains the Q & A (questions and answers) for this chapter — Chapter 6: Regular Expressions. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 13 are proper exercises where you write real patterns. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. A friend tells you that the asterisk in a regular expression "matches any number of characters". Are they right? What does `/go`

d/ match, and why is /

/ on its own not even a valid pattern?

Clue: the asterisk always has its eye on whatever sits immediately before it.

2. What does the dot (`.`) match, and what is the one common character it will not match?

Clue: it matches a single character of almost any kind, but it stops at the end of a line.

3. What will `/gr[ae]y/` match? And what changes if you write `/gr[ae]+y/` instead?

Clue: square brackets mean "any one of these". The `+` is a quantifier, and you know from question 1 what quantifiers attach themselves to.

4. What does the caret (`^`) mean inside square brackets, and what does it mean outside them? Give an example of each.

Clue: it does two completely unrelated jobs depending on where you put it.

5. Both of these try to match an HTML tag. Run them in your head against the text `<p><b> Hello </b></p>` and say what each one grabs:

`/<.+>/`
`/<[^>]+>/`

Clue: one of them is greedy and runs on as far as it can. The other is stopped in its tracks by something it is forbidden to match.

6. The dot is a special character in regular expressions. So how would you write a pattern that matches the literal text `5.0`, with a real full stop in it?

Clue: you need to tell the pattern to stop treating the dot as special.

7. What is the difference between the `g` modifier and the `m` modifier?

Clue: one of them is about how many results come back. The other is about what counts as a line.

8. Name the three JavaScript methods this chapter uses for working with regular expressions, and say what each one hands back.

Clue: one gives you true or false, one gives you what it found, and one gives you a changed copy of the string.

9. EXERCISE. Write a regex pattern to match a social security number in the format:

123-45-6789

Then say how your pattern would need to differ depending on whether you are validating a form field that should contain nothing else, or searching for an SSN inside a longer sentence.

Clue: `\d` matches a digit, and the curly braces let you say how many of them you want.

10. EXERCISE. Write a regex pattern to match five word characters in a row. Then write a second pattern that matches a genuine five-letter word standing on its own.

Clue: for the second one you will need to rule out digits, and mark both ends.

11. EXERCISE. Write a regex pattern to match a word of one or more characters.

Clue: one quantifier is all you need.

12. EXERCISE. Write a regex pattern that matches 'my' only when it begins the subject string. Then say what it does with the text "I love my cat", and why.

Clue: this is one of the two jobs the caret does.

13. EXERCISE. Write a regex pattern that matches 'cats' only when it ends the subject string. Then say what it does with "cats are fun. I like cats too", and why.

Clue: the mirror image of the previous question.

ANSWERS

1. Your friend is not right, and this is one of the most common misunderstandings about regular expressions.

The asterisk is a quantifier. It means "zero or more of whatever comes just before it". It does not mean "any characters" on its own.

`/go*d/` matches "gd", "god", "good", "goood" and so on

because the `*` is attached to the letter `o`, and asks for zero or more of them.

Written by itself, `/*/` is not a valid pattern at all. JavaScript rejects it with the error "Nothing to repeat", because there is nothing in front of the asterisk for it to work on.

What your friend was probably thinking of is `./*` — a dot followed by an asterisk, meaning "zero or more of any character". That really does match anything, or nothing at all. The same reasoning applies to `+`, which means "one or more of the thing before it".

2. The dot matches any single character, with one exception: it will not match a newline (`\n`).

The word "single" matters. `./` on its own matches exactly one character, which is why it is almost always combined with a quantifier, as in `./*` or `./+/,` when you want to cover a run of them.

3. `/gr[ae]y/` matches "gray" and "grey". The square brackets say "any one of the characters inside me", so the pattern is "g, r, then either an a or an e, then y". Adding the `+` turns it into a quantifier applying to the whole class, so `/gr[ae]+y/` asks for one or more of those letters in any combination. It still matches "gray" and "grey", but it now also matches "graay", "greey", "greay" and "graey".

4. Inside square brackets, at the start, the caret means negation:

`/[^0-9]/` matches any single character that is NOT a digit

Outside square brackets, at the start of a pattern, it is an anchor meaning "the match must begin at the start of the text":

`/^my/` matches "my cat", but not "I love my cat"

Two completely different jobs, told apart only by where the caret sits.

5. `/<.+>/` grabs the whole thing: "`<p><b>Hello</b></p>`"
`/<[^>]+>/` grabs just the first tag: "`<p>`"

The first is greedy. The `.+` is happy to match any characters at all, including the closing angle brackets in the middle, so it runs on to the last `>` on the line.

The second cannot do that. `[^>]` means "any character that is not a `>`", so the moment the pattern reaches the first `>`, it can go no further and the match ends there. That is exactly why the negated class fixes the greedy version.

6. Put a backslash in front of the dot to escape it:

```
/5.0/
```

The backslash tells the pattern to stop treating the next character as special and match it literally. It works on any special character, including a backslash itself.

If you also wanted to allow 5.00 or 5.000, add a quantifier to the zero:

```
/5.0*/
```

7. They do quite different jobs, though both sound as though they mean "match more".

g is about how many matches you get back. m is about where ^ and \$ apply.

Take this three-line string:

```
const text = "cat sat here\ndog barked\ncat slept";
```

Without g you get the first match only. With it, you get all of them:

```
text.match(/cat/)    // ["cat"]
text.match(/cat/g)    // ["cat", "cat"]
```

The word "here" sits at the end of the first line, but the string as a whole ends with "slept". So \$ on its own finds nothing, because \$ means the end of the whole string. Add m and \$ comes to mean the end of any line:

```
text.match(/here$/)  // null
text.match(/here$/m) // ["here"]
```

And because they are independent, they combine:

```
text.match(/^cat/g)  // ["cat"]
text.match(/^cat/gm) // ["cat", "cat"]
```

A useful way to remember it: g asks "how many times?", m asks "what counts as a line?".

8.

- **test()** is called on the pattern and hands back true or false:

```
/cats/i.test("Cats are fun.") // true
```

- **match()** is called on the string and hands back what it found:

```
"Cats are fun.".match(/cats/i) // ["Cats"]
```

- **replace()** is called on the string and hands back a changed copy of it:

```
"I like cats".replace(/cats/gi, "dogs") // "I like dogs"
```

Note that `replace()` gives you a new string. It does not alter the one you called it on.

9. `/^\d{3}-\d{2}-\d{4}$/`

`\d` matches a digit, and the curly braces say how many: three, then a hyphen matched literally, then two, another hyphen, then four.

Which version you want depends on the job:

- **Validating a form field** that should contain nothing but an SSN — keep the anchors. The `^` and `$` demand that the whole string is the number and nothing else.
- **Searching for an SSN inside a sentence** — drop them:

```
const pattern = /\d{3}-\d{2}-\d{4}/;  
const text = "My SSN is 123-45-6789";  
  
if (pattern.test(text)) {  
  console.log("Valid SSN!");  
} else {  
  console.log("Invalid SSN.");  
}
```

This catches people out. Use the anchored pattern on that sentence and you will be told "Invalid SSN.", because the string does not consist solely of the number.

10. Five word characters in a row:

`/\w{5}/`

A genuine five-letter word on its own:

`/\b[a-zA-Z]{5}\b/`

The first is looser than its description suggests. `\w` covers letters, digits and the underscore, and nothing marks where the word begins or ends, so it matches the first five characters of "helloworld", and matches "abc12" too.

The second fixes both problems. `[a-zA-Z]` rules out the digits, and `\b` marks a word boundary at each end, so the run of five must stand alone. It matches the "hello" in "say hello now" but finds nothing in "helloworld".

11.

`/\w+/`

The `+` asks for one or more word characters.

You will often see this written as `/[\w]+/`, with square brackets round it. That works just as well, but the brackets add nothing here, because `\w` is already a class in its own right. Worth recognising both forms, since you will meet them out in the wild.

12.

`/^my/`

The `^` pins the match to the very start of the text.

On "I love my cat" it matches **nothing at all**, even though there is a perfectly good "my" sitting in the middle. The anchor is about position, not about which occurrence.

If what you want is the first "my" wherever it happens to fall, use `/my/` with no anchor — a pattern without the `g` modifier stops at the first match anyway.

13.

`/cats$/`

The `$` pins the match to the very end of the text.

On "cats are fun. I like cats too" it matches **nothing**, because that string ends with "too". It would match "I like cats", where the word really is at the end.

Same lesson as the previous question, mirrored: `$` does not mean "the last cats you come across". It means "cats, and only if it finishes the text".

Chapter 7 — FUNCTIONS

- Introduction
 - Giving Functions Tools to Work With (Parameters)
 - Built-in Functions vs Custom Functions
 - Quick Note on `document.write()`
 - Rules for Naming Functions
- Functions demonstrated
- The arguments object
- Handling varied arguments as Rest Parameters
- Arguments with default values
- To return something or to do something
- Functions and variable scope
- Anonymous and arrow functions
- Convert an anonymous function into an arrow function
- Quick object literals from function arguments
- Anonymous and arrow functions and the `this` keyword
- Passing arguments to an arrow function
- Immediately Invoked Function Expression (IIFE)
 - Why should you use an IIFE
 - Private Variables with IIFE
 - IIFE with Parameters
 - Arrow Function IIFE

INTRODUCTION

In programming, we often need to perform certain tasks over and over again. Instead of writing the same code multiple times, we can put that code into a function and just call it whenever we need it.

What Is a Function?

A function is simply a block of code that does a specific job. Think of it as a mini-program inside your program. You write it once, and then you can use (or "call") it whenever you want to carry out that task. For example:

```
function greet() {  
    console.log("Hello!");  
}  
  
greet(); // Calls the function
```

Giving Functions Tools to Work With (Parameters)

Sometimes, just like a person needs tools to do a job, a function may also need some information to do its task. This information is passed into the function through parameters, which go inside the parentheses () when you define the function. The actual values you hand over when you call it are known as arguments.

```
function greetUser(name) {  
    console.log("Hello, " + name + "!");  
}  
  
greetUser("Sam"); // Output: Hello, Sam!
```

Here, "Sam" is an argument, and name is the parameter. You can pass multiple parameters by separating them with commas:

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(3, 4)); // Output: 7
```

The arguments you pass can be:

- Literal values (like "hello" or 5)
- Variables (like name or age)
- Even other functions!

Yes, in JavaScript, you can pass a function to another function, especially when one task depends on another already-defined task.

Built-in Functions vs Custom Functions

In JavaScript (and most programming languages), there are two kinds of functions:

- 1. Built-in functions: These are provided by JavaScript itself to help you do common tasks.
 - Example: `document.write("Hello")` displays text in the browser.
 - Another example: `Math.max(3, 10, 7)` gives you the highest number.

- 2. Custom functions: These are the functions you create yourself, like greet-User() above.

While built-in functions are helpful for routine tasks, custom functions allow you to build powerful and unique behaviours into your program.

Quick Note on document.write()

Although document.write() is a built-in JavaScript function, it is no longer recommended in modern web development. It can cause issues with page rendering and often doesn't work well after the page has finished loading. Instead, use this modern approach:

```
document.body.innerHTML = "Hello, world!";
```

This method is safer and works well in all modern browsers.

Rules for Naming Functions

When you create your own functions, JavaScript has some rules for naming them:

- A function name must start with a letter, an underscore (_) or a dollar sign (\$). It may not start with a digit.
- After the first character, the name can include:
 - Letters (a-z, A-Z)
 - Digits (0-9)
 - Underscores (_)
 - Dollar signs (\$)
- Function names are case-sensitive (myFunction is different from myfunction).
- Use camelCase naming (also called bumpyCaps), where the first word is lower-case and each following word starts with a capital letter. For example:

```
function calculateTotal() { }
```

FUNCTIONS DEMONSTRATED

The syntax of a function is as follows: the keyword function, followed by the name of the function, followed by a pair of opening and closing parentheses (), followed by another pair of opening and closing curly brackets {}. Here is an example:

```
function greeting() {  
    console.log("Hello");  
}
```

The code that you place in between the opening and closing curly braces is the thing/task that the function is meant to do.

Explained in layman terms, a function is like a program you create to perform a task for you. That function then becomes in charge of that task. It will be the function you will call whenever you need that task doing. That is why we give functions names, and that is why you will always hear the terminology ‘call’ or ‘function call’ when you hear programmers speak of functions. If the function were a person, a task master, then depending on your agreement with them you may need to tell them what task to perform whenever you call them, unless they have a specific task, in which case you would not need to pass the task to them whenever you call them. The task you pass to a function when you call it is known as an argument. Imagine you asking your mathematician or calculator to add some numbers up for you. You would pass to them the numbers you want adding up, and then expect the results back. The numbers you pass in are the arguments. A function argument can be one or multiple depending on what you want done. So in brief, a function is a program you write to do a task for you, while function arguments are the values you provide to the function to help it accomplish what it needs to do for you. Here is the same function above, given an argument. As I mentioned above, the task meant to be performed by the `greeting()` function is the code that you put within the curly brackets of the function, also known as the body of the function. Our `greeting()` function above does not need an argument because it has a specific task, to respond with a “Hello”, and it will do just that as often as you call it. You can call a function from anywhere in your code by simply writing its name followed by the pair of parentheses. Make the function call at exactly the part of your application where you want its response to be output. To call our `greeting()` function, do it like so:

```
greeting();
```

Notice how we put a pair of parentheses as we call the function. This has to be done even if you are not passing any arguments to the function, else the function call will not work.

If you want the function to do anything other than return the text “Hello”, and maybe say “Hello” addressed to whatever name you pass to it as its argument, you can modify the function by passing it an argument. You would also need to then modify its body to use that argument in its result. Here is how to do it:

```
let username = "John";

function greeting(userName) {
  console.log("Hello " + userName);
}

// call the function
greeting(username);
```

The output of this function call will be:

Hello John

Notice how the function takes the value handed to it and, in combining it with the string “Hello”, closes that string first and then joins it to the value using the “+” character. That is the same + you would use to add two numbers together, but when it is given strings rather than numbers it does something different: it joins them end to end. Used this way it is called the string concatenation operator. In this case, it combines the string “Hello” with the value of the username variable, which is also a string. So string concatenation means combining two or more strings into one, whether one or all of those strings are literal strings, or they are strings stored in a variable.

THE ARGUMENTS OBJECT

In JavaScript, there is a concept known as the `arguments` object. It's an array-like object that is automatically available inside all functions, and contains all the arguments passed to a function, even if the function doesn't explicitly list them as parameters. Eg:

```
function sumAll() {
  let sum = 0;
  for (let i = 0; i < arguments.length; i++) {
    sum += arguments[i];
  }
  return sum;
}

// result will be 10
let result = sumAll(1, 2, 3, 4);
```

Key Points:

- `arguments` Object: Inside any function, `arguments` is an array-like object that holds all the arguments passed to the function.
- I say “array-like” because it behaves like an array (you can access elements with `arguments[0]`, `arguments[1]`, etc.), but it doesn't have all the array

methods like `.map()`, `.forEach()`, etc.

HANDLING VARIED ARGUMENTS AS REST PARAMETERS

Besides the arguments object, there is a more modern and flexible way to handle a variable number of arguments passed to a function when it is called—using rest parameters (...). Rest parameters (...rest) collect multiple arguments into an array. They are written inside the parentheses of a function, such as ...numbers or ...args, though ...rest is commonly used as a placeholder name. However, rest parameters do not "convert" function arguments into an array in the same way as `Array.from(arguments)`. Instead, they create a true array of the collected arguments, making it more convenient to use array methods like `.map()`, `.filter()`, and `.reduce()`. See the notes in Chapter 3 (Arrays), where we discuss how rest parameters are useful for handling a variable number of function arguments and how they differ from the spread operator (...spread). The spread operator is used to expand an array (or other iterable) into individual elements rather than collecting elements into an array. It's important to note that the spread operator does not "convert" an array into a string of comma-separated items—instead, it breaks an array into separate values that can be passed into functions, array literals, or object literals. (For converting an array into a string, we use `.join(",")`.)

In a way, rest parameters can be seen as the opposite of the spread operator, since rest parameters collect multiple values into a single array while the spread operator takes an array and expands it into multiple values. The main thing however, is to understand these points about rest parameters:

- They are used in function parameters to collect multiple arguments into a single array.
- They are always placed at the end of the function parameter list.

Here is an example:

```
function sum(...numbers) {  
  return numbers.reduce(  
    (acc, num) => acc + num, 0);  
}  
  
console.log(sum(1, 2, 3, 4)); // 10  
console.log(sum(5, 10));     // 15
```

Rest gathers the multiple arguments passed to the function into an array for use within the function (numbers becomes [1, 2, 3, 4] in the first call). Now that numbers is an array, that's why in this example, the array function `reduce()` is called on numb-

ers. Again, be reminded that ...numbers could just as well be ...data or ...rest and it will all still work in the same way.

-A JavaScript function like this that accepts rest parameters can be considered a variadic function. A variadic function in any programming language is one which is capable of accepting a varying (or unknown) number of arguments. Hence the name variadic. Rest parameters are the specific JavaScript syntax used to implement this capability. The keyword 'rest' in 'rest parameters' comes from the significance of the '...' characters. They hint at the fact that the rest of the arguments will follow, thereby indicating that there can be multiple and an unknown number of arguments.

-In conclusion; the `arguments` object is useful, but in modern JavaScript, rest parameters (`...`) are generally preferred for better readability and functionality.

ARGUMENTS WITH DEFAULT VALUES

There are times when you write a function and want one or more of the arguments you pass into it to have default values. These are values that will be used by the function even if no argument is passed in. Here is a simple example:

```
let member = {
  name: "",
  type: "member",
  setMember: function(name, type = "member") {
    this.name = name;
    this.type = type;
  }
};

member.setMember("Dolph");
console.log(member.type); // "member"

member.setMember("Dolph", "admin");
console.log(member.type); // "admin"
```

This code has a member object named member. Once a new member is created by calling setMember on the object, if a desired member type is not given, the type will be set to 'member'. The creator of this code would do this because they want a minimum type of 'member' to be applied to all new users. The first time we run the function like so:

```
member.setMember("Dolph");
```

we get 'member' written to the console.

The second time we run the function like so:


```
member.setMember("Dolph", "admin");
```

we get 'admin' written to the console. Note that we call it as `member.setMember(...)` and not just `setMember(...)`. The function belongs to the `member` object, so you have to reach for it through that object. Calling it on its own would give you a `ReferenceError`. You will also have spotted the word "this" inside the function. That is a keyword meaning "the object I belong to", and it has a section of its own later in this chapter.

TO RETURN SOMETHING OR TO DO SOMETHING

Most often, you want a function to return some kind of data to you, which you need in your program. This could be the result of a mathematical operation which the function has performed, or just some data that the function has fetched for us. In this case we use a 'return' keyword within the body of the function to return the data after the function has completed its job. It is very common to capture that returned data in a variable so that you can use it further in your program. Here is an example of a function returning its result.

```
function addNumbers(numOne, numTwo)
{
    return numOne + numTwo;
}

// capture/store the result of calling addNumbers() in
// a variable
let sum = addNumbers(5, 5);

//use the result
console.log('The sum of the two numbers is: ' + sum);
```

However, there are times when you do not want a function to return anything. In these scenarios, you just want the function to do something and hand nothing back. Here is an example of a function that changes the value of a variable and does not return anything:

```
let appStatus = true;

function updateStatus()
{
    // if the appStatus is true, set it to false & vice versa
    if (appStatus == true) {
        appStatus = false;
    }
    else
```

```

}
}

// no need to capture/store the result of calling the
// function updateStatus() since nothing is returned
updateStatus();

```

FUNCTIONS AND VARIABLE SCOPE

Scope relates to the visibility of a variable. We cover everything about scopes in Chapter 2 (Variables), but I will briefly hint at how it applies to code used within functions. There are three scopes of variables you can use within a script. There is the global scope, there is the function scope (which is still a type of block) and there is the block scope. Let and const variables can only be made local (scoped) within blocks—and remember that a function is itself a kind of block—because let and const variables are block-scoped. Variables declared with the var keyword can only be made local (scoped) within functions because they are function-scoped. Any variable (var, let or const) declared outside of a function or block is a global variable. Please see Chapter 2 (Variables) for a thorough explanation of the concept of scopes and blocks. Let's see this in an example:

```

var appStatus = true;

function updateStatus()
{
    var testLocalVar = "wild";
}

console.log(appStatus);

updateStatus();

console.log(testLocalVar);

```

Because the variable appStatus is declared outside of any function, it is a global variable and can be accessed from anywhere in your code, whether that code is inside of a function or outside of it. That is why appStatus can be read from inside the updateStatus() function just as easily as from outside it. Wherever you log it from, you get the same value, true, displayed in the console. If a variable is declared inside a function like the testLocalVar variable that is declared using the var keyword inside the function updateStatus(), it will always be a variable local to the updateStatus() function. This is because var variables are function-scoped. If the testLocalVar variable had not been declared with the var keyword at all, it would instead have become a global variable. When the variable is local, it cannot be seen

from outside that function, so any attempt to use it from outside will result in JavaScript throwing a `ReferenceError`, saying that `testLocalVar` is not defined. It is always recommended not to declare global variables inside of functions or blocks, because that defeats the whole purpose of encapsulating code within functions or blocks. To respect the integrity of your code therefore, it is the popular convention among JavaScript developers to always declare variables with `let` or `const`, so that the variables respect the scope (local area) in which they have been declared. You will still meet the older `var` keyword in existing code, which is why it is worth knowing, but it is not what you should reach for in new code. To learn everything about variable scopes, see Chapter 2 (Variables).

ANONYMOUS AND ARROW FUNCTIONS

An anonymous function is a function without a name. It is often assigned to a variable or passed as an argument.

```
const greet = function(name) {  
  return `Hello, ${name}!`;  
};
```

```
// Output: "Hello, Alice!"  
console.log(greet("Alice"));
```

Here are the key points on anonymous functions:

- It can be stored in a variable or passed as an argument.
- It uses the function keyword.
- It has its own `this` context.

Arrow functions can be said to be the new and simplified way of writing functions in general that was introduced since ES6. An arrow function has a shorter syntax for writing functions using the arrow character `=>`. It has lexical `this` binding. That phrase is worth unpacking, because you will meet it often and it is rarely explained. ‘Lexical’ simply means ‘to do with the written text of your code’. A lexical rule is one settled by WHERE something appears on the page, rather than by what happens while the program is running. So ‘lexical `this` binding’ means that an arrow function’s `this` is fixed by where the arrow function was written in your source code, and nothing afterwards can change it. A regular function is quite different: its `this` is worked out at the moment it is called, from whatever it was called on. The same regular function handed to two different objects will report a different `this` for each of them, whereas an arrow function will report the same one no matter who calls it. We will see this in action shortly, in the section on the `this` keyword. You will also meet the same word in ‘lexical scope’, where it means the same thing again: which

variables a piece of code can see is decided by where that code sits in the source, not by who calls it.

The left side of the arrow has the argument(s) being passed to the function, while the right side of the arrow constitutes the return value. Let us simplify the above `greet()` anonymous function by converting it into an arrow function:

```
const greet = (name) => `Hello, ${name}!`;

// Output: "Hello, Alice!"
console.log(greet("Alice"));
```

Here are the key points about arrow functions:

- More concise than traditional functions.
- Does not use the function keyword
- Do not have their own this, but rather inherit it from the surrounding scope (lexical).
- Cannot be used as a constructor (new keyword won't work).
- Implicit return when using a single expression.
- The left side of the arrow has the argument(s) being passed to the function, while the right side of the arrow constitutes the return value.

```
(arguments) => value;
```

Note that there is no return keyword in that line. With a single expression the return is implied, and writing `(arguments) => return value` is a `SyntaxError`. If you do want to write return out in full, you need the curly braces around it:

```
(arguments) => { return value; }
```

Convert an anonymous function into an arrow function

An anonymous function can always be converted into an arrow function. Let's convert the above anonymous function into an arrow function. To do so, simply replace `'function()'` with `'() =>'`:

```
let greet = function() {
  return "Hello";
}

//make the change (this is the same greet, rewritten,
//not a second one)
let greet = () => {
  return "Hello";
}
```

We can shorten the syntax even further. If all we are returning in the function is one single expression, then we do not need the opening and closing curly braces, nor do we even need the return keyword. Here is what the function can be reduced to:

```
let greet = () => "Hello";
```

You therefore see how short and concise our code can be as a result. Always remember that whatever comes after the arrow (`=>`) is automatically implied to be the return value of the function. What comes on the left side of the arrow will be any parameters—if applicable—with or without parentheses.

Quick object literals from function arguments

You sometimes have a function in your code that simply needs to return an object literal from arguments passed to the function. If you do not know what object literals are, quickly hop over to Chapter 12 (Object Oriented Programming) and read about objects and object literals before hopping back here to continue. Here is an example of such a function. Since we are on the topic of functions, just for the purpose of better understanding of the differences, I will write three versions of the same function; one using the traditional function syntax, one in anonymous function syntax, and one in the arrow function syntax. The differences should be very subtle, and you should be able to distinguish between them by now:

(These three are alternatives to each other, not three things to write out one after the other. Pick whichever you prefer.)

```
// traditional function
function createObject (make, model, year) {
  return {
    make: make,
    model: model,
    year: year
  };
};

// anonymous function version
const createObject = function(make, model, year) {
  return {
    make: make,
    model: model,
    year: year
  };
};
```

```
// arrow function version
const createObject = (make, model, year) => {
  return {
    make: make,
    model: model,
    year: year
  };
};

console.log(
  createObject('Toyota', 'Rav4', '2025')
);
```

This function return value will be the object:

```
{make: 'Toyota', model: 'Rav4', year: '2025'}
```

Notice that the keys of the object literal returned by the function exactly match the parameters of the function. This seems like a repetition of the same names, first in the function argument, and then again in the keys of the object literal being returned, as seen in `make`, `model`, and `year` in the `createObject()` function above. With the coming of arrow functions in ES6 (ECMAScript 2015), and as part of the code simplification effort, came the provision of a simplified way for this exact same situation. Basically, if you are creating a simple function that returns an object literal, and you know the object it will return is going to, or should have keys that exactly match the names of the arguments you are passing to the function, then you can really simplify the function return code in one line by placing the parameter names in curly braces separated by commas. Here is how:

```
const createObject =
  (make, model, year) =>
    ({make, model, year});

console.log(
  createObject('Toyota', 'Rav4', '2025')
);
```

One detail there is easy to miss and worth pausing on. The object literal is wrapped in its own pair of round brackets:

```
({make, model, year})
```

Those brackets are not decoration. Without them, JavaScript reads the curly braces as the function's body rather than as an object you want back, and the function quietly returns undefined. The round brackets are what tell it "this is a value, not a block".

The return value of this simplified code is still the same:

```
{make: 'Toyota', model: 'Rav4', year: '2025'}
```

Anonymous and arrow functions and the this keyword

Here is an example of how the `this` keyword works differently with both function types:

```
const obj = {
  value: 10,
  regularFunction: function() {
    console.log(this.value);
    // Works: 10
  },
  arrowFunction: () => {
    // Undefined (inherits from parent scope)
    console.log(this.value);
  }
};
obj.regularFunction();
obj.arrowFunction();
```

To modify `obj.arrowFunction()` so that it successfully accesses the `value` property, you need to use a regular function instead of an arrow function. Since arrow functions do not have their own `this`, they inherit it from the surrounding lexical scope, which in this case is the global scope (or undefined in strict mode). To make the `arrowFunction()` above work, here is a good fix that replaces the arrow function with a regular function:

```
const obj = {
  value: 10,
  regularFunction: function() {
    console.log(this.value);
    // Works: 10
  },

  // Changed from an arrow function to a regular one.
  // Note the name no longer really fits, but it is kept
  // here so you can see exactly what changed.
  arrowFunction: function() {
    // Works: 10
    console.log(this.value);
  }
};

obj.regularFunction(); // 10
obj.arrowFunction();   // 10 - now it works
```

Alternatively, if you still want to use an arrow function, one older trick is to capture (store) the ‘`this`’ word in a variable (often called `self`) inside a regular function, then use an arrow inside that function so that the arrow can reach that variable. Here is an example:

```
const obj = {
  value: 10,
  fixUsingLexicalScope: function() {
    const self = this; // Capture this
    const arrowFunction = () => {
      console.log(self.value);
      // Works: 10
    };
    arrowFunction();
  }
};

obj.fixUsingLexicalScope(); // 10
```

That pattern is worth recognising, because you will meet it in older code written before arrow functions existed. These days you do not actually need it. An arrow function written inside a regular method already inherits that method's `this`, so the `self` variable is doing no work:

```
const obj = {
  value: 10,
  fixWithoutSelf: function() {
    const arrowFunction = () => {
      console.log(this.value);
      // Works: 10, no self needed
    };
    arrowFunction();
  }
};

obj.fixWithoutSelf(); // 10
```

The thing to hold on to is that an arrow function takes its `this` from wherever it was written. Written directly on an object literal, that is the surrounding scope, not the object. Written inside a method, it is the method's `this`, which is what you usually want.

As a reminder, here is an anonymous function again. It does not have a name:

```
let greet = function() {
  return "Hello";
}
```

Passing arguments to an arrow function

Say we have a regular function as below:

```
let addThemUp = function(arg1, arg2) {
  return arg1 + arg2;
}
```



```
// Output: 7
console.log(addThemUp(4, 3));
```

To convert it into an arrow function and yet keep the arguments being passed to it, instead of replacing `function()` with `() =>` as before, we need to keep the arguments passed. The right way to do it is this:

```
let addThemUp = (arg1, arg2) => arg1 + arg2;
```

Some functions in JavaScript accept functions as arguments for processing collections of data, for example higher order functions like `map()`, `filter()`, and `reduce()` etc. Whenever a function accepts another function as an argument, that is a good time to use an arrow function.

If the arrow function only accepts a single argument, then you do not even need to put the argument in parentheses—though it will still work if you do. For example, this function

```
const square = function(num) {
  return num * num;
}
```

Will become:

```
const square = num => num * num;
```

Still on the subject of arguments being passed to arrow functions, there is a way arguments work when it comes to higher order functions that we need to understand too. First of all, let's define what a higher order function in JavaScript is. A higher-order function (HOF) is a function that either takes one or more functions as arguments or returns a function as its result. Simply put, higher-order functions work with other functions, treating them as first-class citizens (meaning functions can be assigned to variables, passed as arguments, and returned from other functions). Again, as I said, examples of such functions are `map()`, `filter()`, and `reduce()` etc. We have already come across these functions before so I am not here to explain how they work. Rather, I mention them to demonstrate how such functions handle the arguments that they receive in a unique way. These higher order functions all have something in common—they perform some kind of iteration on some data, while running the function on each one of them. I will take just one of them as an example; `map()`. Take a look at this example:

```
const data = [1, 2, 3, 4];
const doubled =
  data.map(num => num * 2);

console.log(doubled); // [2, 4, 6, 8]
```

The way `map()` works with its arguments, it uses an arrow function. It iterates over the given data, which in this case is a series of numbers in an array. Note that the value in its parentheses is an arrow function, and it looks like this:

```
num => num * 2
```

It is important to note that `num` on the left side of the arrow (`num =>`) is always the argument, and in this case, `num` represents a different number in the data array at each iteration, which it passes in to be processed by the function or expression on the right side of the arrow (`=> num * 2`).

Note also that the name `num` is entirely your choice. It can be anything you like and the code will still work, so long as the name on the left of the arrow matches the one used in the expression on the right of it (`=> num * 2`). Therefore, in the above example, either of the following lines would work the same:

```
num => num * 2  
dat => dat * 2
```

I said everything above to talk about this `num` value on the left side of the arrow, which is the value `map()` passes into the arrow function at each iteration. Because `num` is only one parameter, it is used without parentheses around it. However, if we were dealing with more than one, we would have had to place them in parentheses. `map()` passes only one value in its iteration, hence we end up with the arrow function looking like this, with no parentheses around the single parameter on the left side of the arrow:

```
num => num * 2;
```

Working with `reduce()` on the other hand, which takes a function as an argument and accumulates values into a single result, the arrow function it is given has two parameters, as seen in 'acc' and 'num' below

```
const numbers = [1, 2, 3, 4];  
const sum = numbers.reduce((acc, num) => acc + num, 0);  
  
console.log(sum); // 10
```

Hence its arrow function looks like this:

```
(acc, num) => acc + num
```

Note that we keep the two parameters within parentheses, because there is more than one of them. Take care not to read that trailing `, 0` as part of the arrow function. It is not. It is `reduce()`'s own second argument, the value the accumulator starts from, and it sits outside the arrow function entirely.

Immediately Invoked Function Expression (IIFE)

An IIFE (Immediately Invoked Function Expression) is a self-executing function that runs immediately after it is defined. It is wrapped in parentheses to make it an expression, and then followed by `()` to invoke it immediately.

Here is the basic syntax:

```
(function() {  
    console.log("I run immediately!");  
})();
```

```
// Output: "I run immediately!"
```

Why should you use an IIFE

We use it for the following reasons:

- It avoids polluting the global scope – variables inside the IIFE stay private.
- It executes immediately – No need to call it separately.
- It is useful for initialisation code – meaning, it would make a good place to run your application setup logic code just once.

Private Variables with IIFE

Let's see how you can have private variables that are only available within an IIFE.

```
const result = (function() {  
    let secret = "Hidden Data";  
    return secret;  
})();
```

```
// Output: "Hidden Data"  
console.log(result);
```

```
// Output: undefined (because it's private)  
console.log(typeof secret);
```

In this example, the variable `secret` is not accessible outside the IIFE.

IIFE with Parameters

You can pass arguments into an IIFE like a normal function:

```
(function(name) {  
    console.log(`Hello, ${name}!`);  
})("Alice");
```

```
// Output: "Hello, Alice!"
```

Arrow Function IIFE

With ES6, we can write IIFEs using arrow functions. We already know from studying arrow functions that you convert a regular function into an arrow one by replacing the “function” keyword with a pair of parentheses and an arrow like so “() =>”, and that if there are any parameters, they will go into the parentheses on the left side of the arrow.

```
(() => {  
  console.log("Arrow function IIFE");  
})();  
// Output: "Arrow function IIFE"
```

QUIZ — Functions

This page contains the Q & A (questions and answers) for this chapter — Chapter 7: Functions. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 11 to 14 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. In this code, which of `name` and `"Sam"` is the parameter, and which is the argument?

```
function greetUser(name) {  
  console.log("Hello, " + name + "!");  
}  
  
greetUser("Sam");
```

Clue: one of them is written when you define the function, the other when you call it.

2. Look at this function. It prints `"Hello John"` — but there is a bug in it. What is it, and how would you know?

```
let username = "John";  
  
function greeting(user_name) {  
  console.log("Hello " + username);  
}  
  
greeting(username);
```

Clue: try calling it with a different name and see what happens.

3. What is wrong with each of these two lines?

```
greeting;  
Console.log("Hello");
```

Clue: one is missing something small but essential. The other has a capital letter where it should not.

4. When would you write a function that returns something, and when would you write one that just does something? Give a one-line example of each.

Clue: think about whether you need an answer back that you can store and use.

5. What does this print, and what is the name of the error?

```
function updateStatus() {  
  var testLocalVar = "wild";  
}  
  
updateStatus();  
console.log(testLocalVar);
```

Clue: the variable exists, but not where you are standing when you ask for it.

6. Rewrite this anonymous function as an arrow function, in two stages: first keeping the curly braces, then in its shortest possible form.

```
let greet = function() {  
  return "Hello";  
}
```

Clue: in the shortest form, two things disappear — the braces and one keyword.

7. Why is this a `SyntaxError`, and what are the two correct ways to write it?

```
const double = (num) => return num * 2;
```

Clue: with a single expression, one of those words is doing a job that is already being done for you.

8. What does each of these print, and why are they different?

```
const obj = {  
  value: 10,  
  regularFunction: function() { console.log(this.value); },  
  arrowFunction: () => { console.log(this.value); }  
};  
  
obj.regularFunction();  
obj.arrowFunction();
```

Clue: one of these two kinds of function brings its own `this`. The other borrows one from wherever it was written.

9. This function is meant to return an object, but it returns `undefined`. Why, and what is the one-character-each-side fix?

```
const createObject = (make, model) => {make, model};
```

Clue: JavaScript cannot tell whether you meant those braces as an object or as the function's body — so it guesses, and it guesses the other one.

10. What is an IIFE, and what is the main reason for using one?

Clue: the first word of the abbreviation tells you when it runs; the benefit has to do with keeping things to yourself.

11. EXERCISE. Write a function called `addNumbers` that takes two numbers and returns their sum. Call it, store the result in a variable, and print a sentence using that result.

Clue: you will need the `return` keyword, and the `+` sign will do two different jobs in this exercise.

12. EXERCISE. Write a function that can add up any quantity of numbers — two, five, twenty — without you having to say in advance how many there will be. Call it twice with different amounts of numbers.

Clue: three dots in the parameter list will gather them all into an array for you, and then an array method from Chapter 3 will add them up.

13. EXERCISE. Write a function called `setMember` that takes a name and a type, where type defaults to "member" if nothing is passed for it. Prove that the default works by calling it both ways.

Clue: you give the default right there in the parameter list, with an equals sign.

14. EXERCISE. Take this array and use `map()` with an arrow function to produce a new array where every number has been tripled. Print both arrays to show the original is untouched.

```
const data = [1, 2, 3, 4];
```

Clue: `map()` hands each element to your function one at a time and collects up whatever you return.

ANSWERS

1. name is the parameter. "Sam" is the argument.

The parameter is the name you write inside the parentheses when you **define** the function. It is a placeholder, standing in for whatever will be handed over later.

The argument is the actual value you pass when you **call** it.

A way to keep them straight: parameters are Placeholders, arguments are the Actual values.

2. The bug is that the function never uses its parameter.

The parameter is called `user_name`, but the body reads `username` — which is the variable declared outside the function. So the function ignores whatever you hand it and always prints the outer value.

You would find it the moment you called it with anything else:

```
// still prints "Hello John"
greeting("Somebody Else");
```

The fix is to make the body use the parameter it was given:

```
function greeting(userName) {
  console.log("Hello " + userName);
}
```

Note also that `user_name` should be `userName`. JavaScript convention is camel-Case, as covered in the naming rules earlier in this chapter.

3.
 - `greeting;` is missing its parentheses. Without them you are only mentioning the function, not calling it. Nothing happens. It must be `greeting();`, and the parentheses are required even when you are passing nothing.
 - `Console.log` has a capital C. JavaScript is case-sensitive, so this gives you `ReferenceError: Console is not defined`. It is `console.log`.
4. Write a function that **returns** something when you need an answer back that you can store and use further on:

```
function addNumbers(a, b) {
  return a + b;
}
```

```
// sum is 10
let sum = addNumbers(5, 5);
```

Write a function that just **does** something when there is no answer to hand back — it changes something, or displays something, and that is the whole job:

```
function updateStatus() {
  appStatus = false;
}
```

```
// nothing to capture
updateStatus();
```

The giveaway is whether it makes sense to put the call on the right-hand side of an equals sign.

5. It prints nothing, and throws:

`ReferenceError: testLocalVar is not defined`

`testLocalVar` was declared with `var` inside the function, which makes it function-scoped. It exists only while the function is running, and only inside it. From outside, the name means nothing at all.

Note that this is a `ReferenceError`, not `"undefined"`. `undefined` is a value a variable can hold; a `ReferenceError` is JavaScript telling you the name does not exist here.

6. With the braces kept:

```
let greet = () => {  
  return "Hello";  
}
```

And in its shortest form:

```
let greet = () => "Hello";
```

In the short form both the curly braces and the `return` keyword disappear.

Whatever sits after the arrow is automatically treated as the value to return.

7. It is a `SyntaxError` because with a single expression the `return` is already implied.

Writing `return` as well is asking for it twice, and JavaScript rejects it with

```
SyntaxError: Unexpected token 'return'.
```

The two correct forms are:

```
// implicit return, no braces  
const double = (num) => num * 2;  
  
// explicit return, with braces  
const double = (num) => { return num * 2; };
```

Pick one or the other. You cannot half-do it.

8. It prints:

```
10 undefined
```

`regularFunction` is an ordinary function, and when you call it as `obj.regularFunction()`, its `this` refers to the object it was called on. So `this.value` is 10.

`arrowFunction` is an arrow function, and arrow functions do not get their own `this`. They take it from wherever they were written. This one was written directly on the object literal, and the scope surrounding that is not the object — it is the enclosing scope, where there is no `value` property. So `this.value` is `undefined`.

The rule worth memorising: **an arrow function takes its `this` from where it was written, not from what it was called on.**

Note that an arrow written

inside

a regular method behaves differently, and usually the way you want, because the method's `this` is what it inherits:

```
const obj = {
  value: 10,
  method: function() {
    const inner = () => console.log(this.value);
    inner();    // 10
  }
};
```

9. It returns undefined because JavaScript reads those curly braces as the **function's body**, not as an object you want back. Inside a body, `{make, model}` is just two statements that do nothing, and a function that never returns anything gives you undefined.

The fix is a pair of round brackets around the object:

```
const createObject = (make, model) => ({make, model});
```

Those brackets tell JavaScript "this is a value, not a block". It is a small thing that causes a lot of confusion the first time, because nothing goes wrong loudly — there is no error, you simply get undefined.

10. An IIFE is an Immediately Invoked Function Expression: a function that runs the moment it is defined, rather than waiting to be called. It is wrapped in parentheses to make it an expression, and then followed by `()` to invoke it:

```
(function() {
  console.log("I run immediately!");
})();
```

The main reason for using one is to **keep things private**. Anything declared inside stays inside, so it never touches the global scope and cannot clash with names elsewhere in your program:

```
const result = (function() {
  let secret = "Hidden Data";
  return secret;
})();

console.log(result);
// "Hidden Data"
console.log(typeof secret);
// "undefined" - it never escaped
```

It is also handy for setup code that should run exactly once.

11. Here is one way:

```
function addNumbers(numOne, numTwo) {
  return numOne + numTwo;
}
```

```
let sum = addNumbers(5, 5);

console.log('The sum of the two numbers is: ' + sum);
```

Output:

The sum of the two numbers is: 10

Notice the two jobs the + sign does here. Inside the function it adds two numbers together. In the console.log line it joins a piece of text to a value, which is called concatenation. Same character, different behaviour depending on what sits either side of it.

12. Use a rest parameter:

```
function addAll(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}

console.log(addAll(1, 2, 3, 4));
// 10
console.log(addAll(5, 10));
// 15
```

The three dots gather however many arguments were passed into a real array called numbers. Because it is a genuine array, you can use array methods on it — here reduce(), from Chapter 3, to add them all up starting from 0.

A function that accepts a varying number of arguments like this is called a variadic function.

```
13. function setMember(name, type = "member") {
  console.log(name + " has type: " + type);
}

// Dolph has type: member
setMember("Dolph");
// Dolph has type: admin
setMember("Dolph", "admin");
```

The = "member" in the parameter list is the default. It is used only when nothing is passed for that parameter. Pass something and yours wins.

Defaults must come after the parameters that have none, otherwise you would have no way of skipping them.

```
14. const data = [1, 2, 3, 4];

    const tripled = data.map(num => num * 3);

    console.log(data);
    // [1, 2, 3, 4]
    console.log(tripled);
    // [3, 6, 9, 12]
```

`map()` hands each element to your arrow function one at a time and collects up whatever you return, giving you a brand new array. The original is left exactly as it was, which is why printing both shows data unchanged.

Because there is only one parameter, `num` needs no parentheses around it. The name is yours to choose – `n => n * 3` would work just as well, so long as both sides of the arrow agree.

Chapter 8 — COOKIES

- How cookies work
- Security considerations
- Managing cookies
 - Setting a cookie
 - Retrieving a cookie
 - Get a specific cookie by name
 - Delete a cookie by name
- A cookie consent solution

A cookie is a small piece of data stored in a user's browser that helps websites remember information between visits. It plays a key role in frontend-server communication, allowing websites to track user sessions, store preferences, and manage authentication. Cookies are commonly used for:

- User authentication — keeping users logged in across pages.
- Session management — tracking shopping cart items or user actions.
- Personalisation — storing user preferences like themes or language settings.

How cookies work

When a user visits a website, the server or client-side JavaScript can set a cookie in the browser. On subsequent requests, the browser automatically includes the cookie, enabling the server to recognise returning users.

Security considerations

- Same-Origin Policy. Cookies are only accessible to the domain that set them.
- The HttpOnly flag. This stops a cookie from being read by JavaScript at all, which protects it from scripts running on the page.
- The Secure flag. This is a different job: it makes the browser send the cookie only over HTTPS, never over plain HTTP.
- Expiration & Storage Limits – Cookies have expiration times and are limited in size (~4KB).

While cookies are useful for storing small amounts of data, they are not suitable for storing sensitive information, as they can be accessed by client-side scripts unless secured properly.

Managing cookies

Let us look at how to work with cookies. Cookies are written and read differently from how `localStorage` and `sessionStorage` are written to and read from the browser (we come to those in Chapter 13, Databases and Storage). Here is how to create a cookie and its value.

Setting a cookie

```
document.cookie =  
  "username=JohnDoe; expires=Fri, 31 Dec 2027 23:59:59 GMT; path=/";
```

What this does is - Sets a cookie named "username" with the value "JohnDoe". - The cookie expires on 31 Dec 2027 (after this, it will be deleted automatically). - The `path=/` makes the cookie accessible across the entire site.

However, instead of writing a long string like "Fri, 31 Dec 2025 23:59:59 GMT", you can generate it dynamically using JavaScript's `Date` object. Here is an example:

```
let expiryDate = new Date();  
  
// make it expire in 7 days  
expiryDate.setDate(expiryDate.getDate() + 7);  
  
document.cookie =  
  `username=JohnDoe; expires=${expiryDate.toUTCString()}; path=/`;
```

Take care to keep the `${ ... }` together on one line. If the `$` and the `{` get separated, JavaScript stops seeing it as a slot to fill in and just treats the whole thing as ordinary text, so your cookie ends up with a literal `${expiryDate...}` in it rather than the date.

This code does the following:

- gets the current date.
- Adds 7 days to the expiry date.
- converts it to a proper UTC string format using `.toUTCString()`.
- Sets the cookie with a clear expiration time.

This method is easier and more flexible than writing the date manually.

Retrieving a cookie

To retrieve all cookies as a single string, do it like so:

```
console.log(document.cookie);
```

The output will be something like this:

```
"username=JohnDoe; theme=dark; loggedIn=true"
```

Each cookie is separated from the next by a semicolon and a space ("; ").

Get a specific cookie by name

The way to do it is to retrieve all the cookies in one string, as in the above example, then loop through the result, whilst maybe checking for the keys or values of each against the value you want to match with. Once you find the match, end the loop and there you have your cookie by the name/value you searched for. Say we need to find a cookie with a key named “username”, here is how to do it:

```
function getCookie(name) {  
    // Split cookies into an array  
    let cookies = document.cookie.split("; ");  
  
    for (let cookie of cookies) {  
        // Split key-value pair  
        let [key, value] = cookie.split("=");  
  
        // Return the matching cookie value  
        if (key === name) return value;  
    }  
  
    // Return null if not found  
    return null;  
}  
  
// Output: JohnDoe  
console.log(getCookie("username"));
```

So, what the code does is:

- Splits the document.cookie string into an array of key-value pairs.
- Loops through the array to find the matching key.
- Returns the cookie's value if found, otherwise returns null.

Delete a cookie by name

To delete a cookie, you simply set its expiration date to a past date. This makes the browser automatically remove it. For example:

```
document.cookie =  
  "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

This code does the following:

- Sets the cookie's value to an empty string ("").
- Sets the expires attribute to a past date (Jan 1, 1970), effectively deleting it.
- Uses path=/ to ensure it deletes the cookie from all paths on the site.

It would be nice to create a re-usable function to delete any cookie by name. We can make it even better and get it to first of all check whether the cookie exists before deleting it. Here it is:

```
function deleteCookie(name) {  
  if (document.cookie.split("; ").some(cookie =>  
    cookie.startsWith(name + "="))) {  
    document.cookie =  
      `${name}=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/`;  
  
    console.log(`Cookie "${name}" deleted.`);  
  } else {  
    console.log(`Cookie "${name}" not found.`);  
  }  
}
```

Here is how to use it:

```
// Deletes 'username' if it exists  
deleteCookie("username");  
  
// Outputs: Cookie "nonExistingCookie" not found.  
deleteCookie("nonExistingCookie");
```

What it does is: -Retrieves all cookies as a string (document.cookie). -Splits them into an array using ";" as the separator. -Checks if any cookie starts with name=. -Deletes it only if it exists.

A cookie consent solution

Let's put the whole chapter to work with a small, real example: a cookie consent popup, the kind almost every website now has to show. It is worth building because it uses cookies for exactly what they are for - remembering one small fact

about a visitor between visits. We will also write it so that you can switch between the three ways of storing data on the visitor's machine, since the popup does not much care which one you use. Those other two, `localStorage` and `sessionStorage`, are covered in Chapter 13 (Databases and Storage).

First the markup for the popup itself:

```
<div id="consent-popup" class="consent-hidden">
  <p>
    Be aware that we use cookies to improve your
    experience, and nothing more
    <a href="#">Link to your Terms and Conditions
    or Data Policy here</a>.

    <a href="#" id="accept-cookie-use"
      class="btn btn-primary rounded-pill">
      Okay
    </a>
  </p>
</div>
```

And the little bit of CSS that hides it. Without this the popup would simply always be on show, since all our JavaScript does is add and remove that one class:

```
.consent-hidden {
  display: none;
}
```

Now the JavaScript. Cookies are written and read differently from `localStorage` and `sessionStorage`, so the first thing we do is wrap them in an object with `getItem()` and `setItem()` methods. That way the rest of the code can treat all three the same way:

```
const cookieStorage = {
  getItem: (key) => {
    // Get the whole cookie string and turn it
    // into an object of key-value pairs
    const cookies = document.cookie
      .split(';')
      .map(cookie => cookie.split('='))
      .reduce(
        // The {} at the end is the starting
        // value, so we build up an object
        (acc, [name, value]) =>
          ({ ...acc, [name.trim()]: value }),
        {}
      );

    return cookies[key];
  }
};
```

```

setItem: (key, value) => {
  document.cookie = `${key}=${value}; path=/`;
}
};

```

Take care with that last line. There must be no spaces around the equals sign. Writing `${key} = ${value}` would create a cookie whose name ends in a space and whose value begins with one, which is not what you asked for and leads to some baffling afternoons.

With that in place, the rest reads plainly:

```

const storageType = cookieStorage;
const consentPropertyName = 'jdc_consent';

// Show the popup only if we have
// not already been told "Okay"
const shouldShowPopup = () => !storageType.getItem(consentPropertyName);

const saveToStorage = () =>
  storageType.setItem(consentPropertyName, true);

const consentPopup =
  document.getElementById('consent-popup');
const consentAcceptBtn =
  document.getElementById('accept-cookie-use');

// When the accept button is clicked
const acceptFunc = event => {
  saveToStorage();
  consentPopup.classList.add('consent-hidden');
};

consentAcceptBtn.addEventListener('click', acceptFunc);

if (shouldShowPopup()) {
  // Delay the popup by 2 seconds so it does not
  // appear before the page has settled
  setTimeout(() => {
    consentPopup.classList.remove('consent-hidden');
  }, 2000);
}

```

If you would rather store the visitor's answer in `localStorage` or `sessionStorage` instead of a cookie, you only have to change one line:

```

// survives the browser closing
const storageType = localStorage;
// forgotten when the browser closes
const storageType = sessionStorage;
// our own wrapper, above
const storageType = cookieStorage;

```

That is the benefit of having wrapped the cookie code to look like the other two. Everything below that line stays exactly as it is.

QUIZ — Cookies

This page contains the Q & A (questions and answers) for this chapter — Chapter 8: Cookies. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 6 to 8 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What is a cookie, and name two things websites commonly use them for.

Clue: the key word is "remember" — something has to survive from one visit to the next.

2. The `HttpOnly` flag and the `Secure` flag are often mentioned in the same breath, but they do two different jobs. What does each one do?

Clue: one is about who can read the cookie. The other is about how it travels.

3. You have set three cookies. What does this print, and what separates one cookie from the next in the result?

```
console.log(document.cookie);
```

Clue: you get them all back as one single piece of text, not as a list.

4. How do you delete a cookie? There is no `deleteCookie()` built into JavaScript, so what is the trick?

Clue: you do not remove it so much as convince the browser it is already past its bedtime.

5. What is wrong with this line, and what would the cookie actually end up containing?

```
document.cookie = `username=JohnDoe; expires=${
    expiryDate.toUTCString()}; path=/`;
```

Clue: look very closely at the two characters that should be sitting next to each other.

6. EXERCISE. Set a cookie called `theme` with the value `dark`, which expires 7 days from now, and is available across the whole site.

Clue: work the date out with the Date object rather than typing one in by hand — start from today and add 7 to it.

7. EXERCISE. Write a function called `getCookie(name)` that returns the value of a single cookie by its name, or null if there is no such cookie.

Clue: `document.cookie` hands you one long string, so you will need to split it twice — once to separate the cookies from each other, and once to separate each name from its value.

8. EXERCISE. Write a function called `deleteCookie(name)` that deletes a cookie, but only after checking it exists. Print a message either way.

Clue: `some()` will tell you whether any item in an array passes a test, and `startsWith()` will tell you whether a piece of text begins with something.

ANSWERS

1. A cookie is a small piece of data stored in the user's browser, which lets a website remember information between visits or between pages.

Common uses include:

- **User authentication** — keeping someone logged in as they move around the site.
- **Session management** — remembering what is in a shopping cart.
- **Personalisation** — storing preferences such as a theme or a language choice.

The browser sends the cookie back automatically with each request, which is how the server recognises a returning visitor.

2. They protect against quite different things:

- **HttpOnly** stops the cookie from being read by JavaScript at all. Even `document.cookie` will not show it. This protects it from any script running on the page.
- **Secure** does not affect JavaScript. It tells the browser to send the cookie only over HTTPS, never over plain HTTP, so it cannot be read in transit.

One is about *who can read it on the page*, the other about *how it travels across the network*. You would typically want both on anything sensitive.

3. It prints all the cookies for that site as one single string, something like:

```
username=JohnDoe; theme=dark; loggedIn=true
```

Each cookie is separated from the next by a **semicolon and a space** (`;`).

Note what you do not get: there is no array, no object, and no way to ask for one cookie directly. That is exactly why question 7 exists — if you want a single cookie you have to pull the string apart yourself.

4. You set its expiry date to a date in the past. The browser then removes it for you:

```
document.cookie =  
  "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path="/;
```

The date used here, 1 January 1970, is a common choice because it is the earliest date many systems recognise, but any past date works.

Two details matter. Set the value to an empty string, and give the same `path=` the cookie was created with — a cookie set on one path will not be deleted by an instruction aimed at another.

5. The `$` and the `{` have been separated by a line break.

In a template literal, `${` is a single unit meaning "work this bit out and drop the result in here". Split it and JavaScript stops recognising it, treats the whole thing as ordinary text, and your cookie ends up containing the literal characters:

```
username=JohnDoe; expires=$  
{expiryDate.toUTCString()}; path=
```

rather than an actual date. Nothing errors — it just quietly does the wrong thing, which is the hardest kind of bug to spot.

The fix is to keep `${ ... }` together on one line:

```
document.cookie =  
  `username=JohnDoe; expires=${expiryDate.toUTCString()}; path=
```

6. `let expiryDate = new Date();`

```
// make it expire in 7 days  
expiryDate.setDate(expiryDate.getDate() + 7);
```

```
document.cookie =  
  `theme=dark; expires=${expiryDate.toUTCString()}; path=
```

Working the date out this way is far better than typing one in. A hand-written date is correct on the day you write it and quietly wrong forever afterwards.

`getDate()` gives you the day of the month, and `setDate()` puts a new one back. Adding 7 rolls into the next month or year on its own, so you do not have to think about it. `toUTCString()` then formats it the way cookies expect.

`path=` makes the cookie available across the whole site rather than just the page that set it.

```

7. function getCookie(name) {
    // Split cookies into an array
    let cookies = document.cookie.split("; ");

    for (let cookie of cookies) {
        // Split key-value pair
        let [key, value] = cookie.split("=");

        // Return the matching cookie value
        if (key === name) return value;
    }

    // Return null if not found
    return null;
}

// Output: JohnDoe
console.log(getCookie("username"));

```

The first split breaks the one long string into separate cookies. The second breaks each cookie into its name and its value.

The line `let [key, value] = cookie.split("=")` is called destructuring: it takes the two items the split produced and puts them straight into two variables in one go. There is more on it in Chapter 12.

Returning null when nothing matches is a deliberate choice. It gives the caller a clear "there was no such cookie" rather than leaving them with undefined.

```

8. function deleteCookie(name) {
    if (document.cookie.split("; ").some(cookie =>
        cookie.startsWith(name + "="))) {

        document.cookie =
            `${name}=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/`;

        console.log(`Cookie "${name}" deleted.`);
    } else {
        console.log(`Cookie "${name}" not found.`);
    }
}

// Cookie "username" deleted.
deleteCookie("username");
// Cookie "nonExistingCookie" not found.
deleteCookie("nonExistingCookie");

```

`some()` runs the test on each cookie and stops the moment one passes, handing back true or false. We check with `startsWith(name + "=")` rather than just the name, so that looking for "user" does not accidentally match a cookie called "username".

Note that the whole cookie string is written on one line. A template literal is allowed to run across several lines, but if you let it, the line break and the indentation become part of the text — and a cookie with a newline in the middle of its expiry date will not work.

Chapter 9 — STRINGS

- Escaping nested quotes
- Single vs double quotes
- Escape sequence characters
 - Examples of usage
- Concatenating strings
- Template literals and string interpolation
- String properties and functions
 - length
 - charAt()
- A string in computer programming refers to the data structure that is a piece of text. A string is denoted by wrapping it within a pair of opening and closing quotation marks or backticks. In JavaScript, you can create a string by wrapping the text within either Double quotes, single quotes, or backticks. For example:

```
let firstName = "John";  
let surname = 'Doe';  
let nickname = `Johnny`;
```

Escaping nested quotes

You cannot nest a string inside another using the same type of quotes. This will cause a conflict and you will get an error. For example this is wrong:

```
const sentence = "He said hello to "her"";
```

To prevent the error, you should use a different type of quote for the sub string. For example, this will work.

```
const sentence = "He said hello to 'her'";
```

The only way you can nest strings using the same type of quotes is if you escape the nested string. There is something in JavaScript called the escape character, and it is simply a back slash (). The way to use it is to place it right before the opening quote that you know will cause a conflict (because that quote type is already in use) and also just before the closing quote. Here is an example that will work:

```
const sentence = "He said hello to \"her\"";
```

The escape character `()` tells the JavaScript parser not to treat the character that follows it as part of the syntax, but to take it as an ordinary character belonging to the text. When a string contains quotes of one type, using the opposite type for the string helps avoid escaping.

Many coding styles and linters (like ESLint) recommend sticking to one quote style for consistency. When working with inline JavaScript in HTML, single quotes are often used for JavaScript strings to avoid conflicts with the quotes often used in HTML attributes. For example:

```
<button onclick="alert('Hello!')">Click me</button>
```

Single vs double quotes

At the end of the day, the type of quotes you use comes down to choice, and which ever you use will not affect performance or functionality. Both single and double quotes represent strings and behave identically in JavaScript.

Escape sequence characters

These are characters that you can use in your program to escape characters in different scenarios as shown below. They are basically a combination of the escape character `()` and one or more characters that represent a specific character or behaviour. They are used for formatting or representing special characters within strings.

Escape sequence	What it does
<code>\n</code>	a newline character
<code>\t</code>	a tab
<code>\'</code>	escape a single quote
<code>\"</code>	escape a double quote
<code>\\</code>	escape a backslash, when you want the backslash itself to appear in the text rather than act as an escape character
<code>\r</code>	carriage return
<code>\b</code>	a backspace
<code>\uXXXX</code>	escape a unicode character

Examples of usage

```
\n Create a new line eg
```

```
const text = "Hello\nWorld!";
console.log(text);
```

The output: Hello World!

```
\t Create a horizontal tab by tabbing in
const tabbed = "Column1\tColumn2";
console.log(tabbed);
```

the output: Column1 Column2

```
\' Escape quotes (single/double quotes)
const sentence = 'It\'s a beautiful day!';
console.log(sentence);
```

the output: It's a beautiful day!

```
\\ Escape a backslash to display it as a literal character
const backslash = "This is a backslash: \\";
console.log(backslash);
```

The output: This is a backslash: \

\r Insert a carriage return (rarely used on its own, often combined with \n for compatibility).

```
const carriageReturn = "First part\rSecond part";
console.log(carriageReturn);
```

The output may depend on platform; overwrites
"First part" on some systems.

```
\b Insert a backspace (removes one character to the left; rarely used).
const backspace = "AB\bC";
console.log(backspace);
```

The output: AC

```
\uXXXX Escape a unicode character
const heart = '\u2764';
console.log(heart);
```

The output: ♥ (a black heart symbol)

The \u tells JavaScript that the next four characters are a hexadecimal code identifying one character in the Unicode set. 2764 happens to be the heart. This is how you write characters your keyboard cannot type.

Concatenating strings

This refers to how you can programmatically join two or more strings together. Different languages do it in different ways. In JavaScript, we use the + character

which is also known as the concatenation operator, to do this. The syntax is this; say you are combining two strings, you start by opening and closing the quotes wrapping the first string, then type in the concatenation operator, followed by another opening and closing quotes containing the second string.

```
"String one" + "string two";
```

Here is an example:

```
let stringOne = "string one and ";
let stringTwo = "string two";
let oneString = stringOne + stringTwo;

console.log(oneString);
console.log("This is another example of concatenating "
  + "strings without variables");
```

Outputs: string one and string two

This is another example of concatenating strings without variables

You can also concatenate strings using the += operator. However, though the += operator is mostly used for adding up numbers (see notes under operators), when used with strings, it simply appends a new string to the end of another. Here is an example:

```
let oneString = "string one and ";
oneString += "string two";

console.log(oneString);
console.log(oneString + " make up one long string.");
```

Outputs: string one and string two

string one and string two make up one long string.

Notice how it is possible to combine literal strings, variables containing strings, or two variables containing strings, as seen in the example above:

```
var oneString = stringOne + stringTwo;
```

Or oneString + " make up one long string"

I leave a space at the end of the first string and the start of the following string in order to have some space between the last word of the first string and first word of the following string when the two strings are joined together and displayed on screen. So you see that, whether the strings are stored in variables or not, it works in the same way.

Template literals and string interpolation

Template literals are a modern way of handling strings in JavaScript, introduced in ES6 (ECMAScript 2015). They allow us to create multi-line strings, embed expressions, and make string formatting more readable and convenient. Unlike normal strings which use single quotes (') or double quotes ("), template literals use backticks (`). Here is a basic syntax of template literals:

```
const message = `Hello, world!`;
```

```
// Output: Hello, world!  
console.log(message);
```

String interpolation is the process of embedding variables or expressions inside a string without needing manual concatenation which we have seen above using the concatenation operator (+). In JavaScript this is done using template literals. Template literals enable string interpolation using the `${}` syntax inside backticks. Here is an example of template literal with string Interpolation:

```
const name = "Alice";  
const age = 25;
```

```
const greeting = `My name is ${name} and I am ${age} years old.`;
```

```
// Output: My name is Alice and I am 25 years old.  
console.log(greeting);
```

Keep that one on a single line. A template literal is allowed to run across several lines, but if you let it, the line break and the indentation become part of the text - which is exactly what the next example puts to good use, but is rarely what you want in the middle of a sentence.

Without template literals, we would have to use string concatenation, like so:

```
const greeting = "My name is " + name  
  + " and I am " + age + " years old.";
```

You can see how messy that can be. String interpolation therefore makes it much cleaner, readable and less error prone. Here are the benefits of template literals:

- Makes it so easy to embed variables and expressions inside strings (using `${}`)
- Within a template literal, you can write multi-line Strings with no need for `\n` or
- (to join strings) for new lines, or any other escape sequence characters for formatting like space, tabbing etc. Within a template literal, format your text as you would have it, eg adding space, tabbing to indent, hitting enter for new lines,

adding quotes where you need them etc, and the text will be rendered formatted in exactly that same way. This makes the handling of large blocks of text easier.

- Freely use quotes within your string without having to worry about escaping quotes in order to prevent quote type conflicts.

Let's look at an example on how to write multi-line strings in a template literal:

```
const multiline = `This is line 1.
  This is line 2.
  This is line 3.`;
```

```
// Output:
// This is line 1.
//           This is line 2.
//           This is line 3.
console.log(multiline);
```

You can see how the multiple (separate) lines are displayed without you having had to write any `\n` character in the code. Just press enter to create new lines when you write the code. Look closely at that output though. Lines 2 and 3 come out indented, because the tabs you typed in front of them inside the backticks are part of the text too. A template literal keeps everything exactly as you laid it out, spaces and all. That is its strength when you want it, and a trap when you do not.

Let's look at another example on how to embed expressions within a template literal. So, just like when adding variables (string interpolation), you can do calculations (expressions) inside a template literal by using the `${}` syntax. Here is how:

```
const a = 5, b = 10;

// Output: The sum of 5 and 10 is 15.
console.log(`The sum of ${a} and ${b} is ${a + b}.`);
```

String properties and functions

There are many built-in functions and properties offered by JavaScript to help you work with strings. We will look at a few examples:

- `length` is a property used for measuring the length of a string in characters
- `charAt(index)` used to get the character at a specific position (index) in a string.

length

```
let fullName = "John Doe";
let stringLength = fullName.length;
console.log(stringLength);
```

The output is: 8

This is because length counts the blank space between the two parts of the name.

charAt()

```
const str = "Hello, World!";
console.log(str.charAt(0)); // Outputs "H"
console.log(str.charAt(7)); // Outputs: "W"
console.log(str.charAt(20));
// Outputs: "" (empty string, index out of range)
```

Another quick way to get the character at an index is to use the bracket notation. Here is how to do it:

```
let sport = "Boxing";
let firstLetterOfSport = sport[0];
console.log(firstLetterOfSport);
```

Outputs: B

Note that when using the bracket notation, the counting starts from 0.

You can even place an expression within the brackets to work out the desired index to get a character of the string from. For example, to get the last character of the string, you can do this:

```
let sport = "Boxing";
let lastLetterOfSport = sport[sport.length - 1];
console.log(lastLetterOfSport);
```

Outputs: g

In the same way, to get the last but one character of a string, just increase the number you are deducting, eg:

```
let sport = "Boxing";
let lastButOneLetterOfSport = sport[sport.length - 2];
console.log(lastButOneLetterOfSport);
```

Outputs: n

QUIZ — Strings

This page contains the Q & A (questions and answers) for this chapter — Chapter 9: Strings. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 8 to 10 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Name the three ways you can wrap a piece of text to make it a string in JavaScript.

Clue: two of them are the ordinary quote marks. The third is the slanted one, usually found next to the number 1 on a keyboard.

2. Why does this line fail, and give two different ways to fix it?

```
const sentence = "He said hello to "her"";
```

Clue: JavaScript reads along until it meets the next quote of the same kind, and then thinks the string has finished.

3. What do each of these escape sequences do?

```
\n \t \ "
```

Clue: the first two are about layout, the second two are about characters that would otherwise be taken as syntax.

4. What is string concatenation, and which character does it?

Clue: it is the same character you would use to add two numbers, doing a different job.

5. What does this print? Look carefully at the line break.

```
const name = "Alice";

const greeting = `My name is ${name}
  and I am 25 years old.`;

console.log(greeting);
```

Clue: a template literal keeps everything exactly as you laid it out.

6. What is the length of "John Doe", and why?

Clue: count everything between the quotes, not just the letters.

7. Given `const sport = "Boxing";`, what do each of these give you?

`sport[0]` `sport[sport.length - 1]` `sport[sport.length - 2]`

Clue: counting starts at 0, which is why the last one is at length minus 1 rather than length.

8. EXERCISE. Create a string containing a sentence that has both a single quote and a double quote inside it — for example, He said "it's fine". Do it twice: once by choosing your outer quotes carefully, and once using escape characters.

Clue: backticks make the first version easy, because neither kind of quote conflicts with them.

9. EXERCISE. You have two variables, a first name and a surname. Produce the sentence "Hello, John Doe!" twice — once using concatenation, and once using a template literal.

Clue: watch where the spaces go. It is the most common thing to get wrong with concatenation.

10. EXERCISE. Write a piece of code that prints the last character of any string, without knowing in advance how long the string is. Test it on two strings of different lengths.

Clue: the property from question 6 will tell you how long it is, and question 7 shows you what to do with that number.

ANSWERS

1. Double quotes, single quotes, and backticks:

```
let firstName = "John";
let surname = 'Doe';
let nickname = `Johnny`;
```

The first two behave identically. The third, the backtick, creates a template literal, which can do things the other two cannot — we come to those in question 5.

2. It fails because JavaScript reads along the line until it meets the next double quote, and decides the string ended there. So it sees the string "He said hello to ", then runs into the word her sitting outside any string, and does not know what to do with it.

Two ways to fix it:

Use a different kind of quote inside:

```
const sentence = "He said hello to 'her'";
```

Or escape the inner quotes with a backslash:

```
const sentence = "He said hello to \"her\"";
```

The backslash tells the parser not to treat the character after it as part of the syntax, but to take it as an ordinary character belonging to the text.

Of the two, choosing a different quote is usually the tidier option. Escaping is there for when you have no choice.

3.

- `\n` — a newline. Everything after it starts on a fresh line.
- `\t` — a tab. Useful for lining things up in columns.
- `\\` — a single literal backslash. You need two, because one on its own would be read as the start of an escape sequence.
- `\"` — a literal double quote, one that does not end the string.

4. Concatenation is joining two or more pieces of text together into one. In JavaScript it is done with the `+` character:

```
"String one" + " string two";
```

That is the same `+` you would use for arithmetic. Given numbers it adds them; given strings it joins them end to end.

There is also `+=`, which appends to a string you already have:

```
let oneString = "string one and ";  
oneString += "string two";  
// oneString is now "string one and string two"
```

5. It prints:

My name is Alice and I am 25 years old.

on two lines, not one — with the indentation included.

A template literal keeps whatever you put inside it exactly as you laid it out. Because the text was written across two lines with a tab in front of the second, the line break and the tab both become part of the string.

That is a real strength when you want it, which is how multi-line strings work:

```
const multiline = `This is line 1.  
This is line 2.`;
```

But in the middle of a sentence it is almost never what you meant. Keep an ordinary sentence on one line.

6. Eight.

```
let fullName = "John Doe";
console.log(fullName.length); // 8
```

J-o-h-n is four, D-o-e is three, and the space between them counts as a character too, which makes eight. `length` counts every character, not just the letters.

7.

<code>sport[0]</code>	// "B" - the first character
<code>sport[sport.length - 1]</code>	// "g" - the last character
<code>sport[sport.length - 2]</code>	// "n" - the last but one

"Boxing" has six characters, so `length` is 6. But counting starts at 0, so the positions run 0 to 5. That is why the last character sits at `length - 1` and not at `length` — asking for `sport[6]` would give you undefined, because there is nothing there.

8. Two ways:

```
// 1. Choose outer quotes that
// do not clash. Backticks work
// for both kinds at once.
const line1 = `He said "it's fine"`;

// 2. Or escape whichever ones
// clash with your outer quotes
const line2 = "He said \"it's fine\"";

console.log(line1);
// He said "it's fine"
console.log(line2);
// He said "it's fine"
```

Both print the same thing. The first is easier to read, which is a good reason to prefer it. Notice that in the second version the single quote in "it's" needs no escaping at all, because the string is wrapped in double quotes and so a single quote poses no threat to it.

```
9. const firstName = "John";
    const surname = "Doe";

    // with concatenation
    console.log("Hello, " + firstName + " " + surname + "!");

    // with a template literal
    console.log(`Hello, ${firstName} ${surname}!`);
```

Both print:

Hello, John Doe!

The spaces are the fiddly part of the concatenated version. There is one after the comma, inside the first piece of text, and another between the two names that has to be added as a piece of text all of its own. Miss it and you get "Hello, JohnDoe!".

The template literal version has no such problem: the spaces are simply where you typed them, which is much of the reason template literals are worth using.

```
10. function lastCharacter(text) {  
    return text[text.length - 1];  
}  
  
console.log(lastCharacter("Boxing"));  
// g  
console.log(lastCharacter("Football"));  
// l
```

Because we work the position out from `text.length` rather than typing a number in, this works on a string of any length without being told how long it is.

You could also use `charAt()` for the same job:

```
return text.charAt(text.length - 1);
```

The difference between the two shows up only when you ask for a position that does not exist: brackets give you `undefined`, whereas `charAt()` gives you an empty string.

Chapter 10 – DATA TYPES

- Intro
- Why data types differ across languages
- Similarities across languages
- Understanding primitive and reference types
 - Primitive types
 - Why the name “primitive”?
 - Why the Name “Reference”?
 - Why the name object literal
 - Reference types
- Understanding strong and weak typing
 1. Strongly typed languages
 2. Weakly typed languages
- Understanding static and dynamic typing
 1. Static typing
 2. Dynamic typing
- The best way to study data types
- Explicit casting
- Types of data in JavaScript
 - Primitive
 - Non-primitive
 - Symbols explained
 - Symbol Creation
 - Symbols as Object Keys
 - Symbols and Iteration
 - Built-in Symbols (aka well-known Symbols)
 - Undefined vs null values
 - Convert the datatype of a variable
 - Built-in utility functions for type checking
- Booleans
 - The different interpretations of true or false

In computer programming, a data type is a classification identifying one of various types of data, such as floating-point, integer, or Boolean, that determines the possible values for that type; the operations that can be done on values of that type; the meaning of the data; and the way values of that type can be stored. Programming languages allow you to also convert data from one type to another. If this is not initially clear to you, do not worry, you are not alone. I have got you covered on this—it will all become clear in a minute.

The concept of data types can be confusing for new programmers. When I started out, the sources of information I consulted, which were mostly books and online materials did not seem to correlate. The data types (primitives and reference types) in Java, seemed to be different from data types in python, and those in Javascript also seemed different. The more i tried to piece the data together to make sense, the more some questions came to mind, which i found so many developers were wondering about too. The first question was whether the concept of data types is programming-language-agnostic, or if they are different for each language. Secondly, i wanted to know the best way to study data types. Do you have to study them for each programming language, or is it possible to master the topic in a way that applies correctly to all languages? I finally got the answers. Let's talk about what i found. The concept of data types exists in all programming languages, but how they are implemented and categorized can differ between languages. Broadly, data types define what kind of data a variable can hold, but the specific details of types—especially primitive types—can vary across languages.

Why data types differ across languages

- 1. Design Philosophy: Different languages have different goals. For example, Java is a statically typed language, meaning data types are explicitly declared and checked at compile-time. Python, on the other hand, is dynamically typed, meaning data types are determined at runtime.
- 2. Memory Management: Some languages, like Java, provide primitive types (e.g., int, float) that are more memory-efficient compared to reference types. Python treats everything as an object, which changes how it handles data types.
- 3. Typing System: JavaScript has weak typing (types are coerced), Python is dynamically typed, and Java is strongly typed with clear distinctions between primitive types and reference types.

Similarities across Languages

Even though the specifics differ, the high-level concept of data types is similar across languages:

- 1. Primitive Types: These usually include basic data types like integers, floats (or doubles), booleans, and characters. Some languages extend this with additional types (e.g., `long` in Java or `bigint` in JavaScript).
- 2. Reference Types: These refer to more complex data structures like arrays, objects, lists, or dictionaries. While the syntax and underlying implementation differ, the idea of reference types is shared across most languages.

Understanding primitive and reference types

Sure! Let's break down primitive types and reference types in simple terms.

a) Primitive Types Primitive types are the basic building blocks of data in a programming language. They are called 'primitive' because they are the most simple, low-level data types, and they are not made up of other types.

Key Features: - They directly store the value. - They are generally fixed-size and take up a predictable amount of memory. - They are faster to access and manipulate because they are stored in a simple form. - They are immutable, meaning once you assign a value to them, it can't change (in languages like Java; though in Python, immutability applies to some, but not all).

Why the Name "Primitive"?

They are called primitive because they are the basic, simple types that can't be broken down further. This because they are stored directly in memory, and not as references in memory as is the case with reference types.

Examples of Primitive Types: - Integer types - Numbers without a decimal (e.g., `int`, `short`, `long`, `byte` in Java). - Floating-point types: Numbers with decimals (e.g., `float`, `double`). - Character type: Single characters (e.g., `char` in Java). - Boolean type: Represents true/false values (e.g., `boolean` in Java). - Byte: A data type that holds a small integer value (e.g., in Java, `byte` can store values from -128 to 127).

In Python and JavaScript, these are similar, but types like `'int'` or `'float'` are automatically managed and are a

bit more flexible. For example:

- Python: ``int`, `float`, `bool``
- JavaScript: ``number`, `boolean``

Here are the primitive types in JavaScript: string, number, boolean, null, undefined, bigint, symbol

b) Reference Types Reference types are more complex data types that store a reference (or address) to the actual data rather than the data itself. They are often made up of multiple primitive types or other reference types.

Key Features:

- They store the location (reference) of the data, not the actual data.
- They can be larger and more complex because they hold more than one value.
- They are often mutable, meaning the data they reference can be changed.
- They are slower to access because you first need to look up where the data is stored in memory.

Why the Name “Reference”?

They are called reference types because instead of directly storing the value, they store a reference (like an address) to where the actual value or object is located in memory.

Examples of Reference Types:

- Arrays: Collections of items of the same type (e.g., `int[]` in Java, `list` in Python).
- Objects: Complex types made of properties (e.g., custom objects in Java, dictionaries or class instances in Python, object literals in JavaScript, classes in JavaScript etc).
- Strings: In some languages like Java, `String` is a reference type, because it is an object, not a primitive.
- Lists: In Python, lists are reference types because they store a reference to where the list items are held in memory.
- Functions: In JavaScript, functions are treated as reference types because they are objects.

Here are the reference types in JavaScript: Object, Array, Function, Class, Date, RegExp, etc.

Object literals `{}` are reference types because they store values by reference, not directly in memory. Classes (which are just special objects) are also reference types because they use constructors to create instances stored by reference.

Here are the key differences between primitive and reference types

Primitive types: Directly store values, like a box that holds a number.

Reference types: Store the location of the data, like an index card that points to where the actual data is stored.

Why does the distinction matter? When you work with ‘primitive types’, you are manipulating the actual value. With reference types, you are manipulating the

nce (pointer) to the data. This difference affects how variables behave, especially when passing them to functions or copying them.

Why the name object literal

Let's talk about why the name 'object literal' is used in JavaScript and not just 'object'. The term "object literal" is used because it refers to a specific way of defining an object using a literal notation—that is, directly writing out the key-value pairs within curly braces {}. An "object" is a general term that refers to any instance of the Object type which is the parent of all objects in JavaScript. An "object literal" is a specific way of creating an object using curly braces {} with properties inside. For example:

```
const person = {  
  name: "Alice",  
  age: 25  
};
```

Here, person is an object, and it was created using the object literal syntax.

The term "literal" in programming means a fixed, direct value that is written in code, rather than being created dynamically or through constructors.

```
const car = {  
  brand: "Toyota",  
  model: "Camry"  
};
```

The above object is created directly using {}.

```
const car = new Object();  
car.brand = "Toyota";  
car.model = "Camry";
```

This above object is created dynamically using new Object(), then properties are added later.

Why use object literals?

- More concise (faster to write than new Object()).
- It's easier to read and understand
- More commonly used in JavaScript than object constructors.

Here is a list of other "Literals" in JavaScript. The word "literal" is used in JavaScript for other direct-value creations too:

- Array Literal → const arr = [1, 2, 3];
- String Literal → const str = "hello";
- Number Literal → const num = 42;

- Boolean Literal → `const bool = true;`

Understanding strong and weak typing

The concepts of ‘strongly typed’ and ‘weakly typed’ refer to how strictly a programming language enforces the rules around data types.

1) Strongly typed languages

In a strongly typed language, the type of a variable is enforced, meaning you cannot freely mix and match different data types without following specific rules or explicitly converting them.

Key Points:

- You have to be careful with how you use variables.
- Data types are strictly checked, and there is no type coercion (the language doesn’t automatically convert one type to another if it doesn’t make sense).
- If you want to use data in a way that changes its type, you need to explicitly convert it.

Examples

Java:

```
int num = 5;
String text = "Hello";

// Trying to combine them without
// converting would throw an error

// Error: incompatible types
String result = num + text;

// You must convert 'num' to a string
// Now it works: "5Hello"
String result =
    String.valueOf(num) + text;
```

So, in Java, you can’t mix an integer and a string without first converting the integer to a string. The language forces you to respect the types you’re working with.

Python:

```
age = 25
name = "John"

# This will cause an error in Python
# because you cannot concatenate a
```

```
# string with an integer directly
# Error: TypeError
result = name + age

# You need to explicitly convert 'age'
# to a string
# Now it works: "John25"
result = name + str(age)
```

2) Weakly typed languages

In a weakly typed language, the language is more flexible and will try to convert data types automatically when needed, even if it doesn't always make perfect sense. You would sometimes hear people talk of loosely-typed languages. When they do, they are generally talking about weakly typed languages. Both terms are often used interchangeably

Key Points:

- Data types are not strictly enforced.
- The language might automatically convert one type to another (a process called type coercion).
- You can mix types more freely, but it might lead to unexpected behaviour or results.

Examples:

JavaScript

```
let num = 5;
let text = "Hello";

// JavaScript will automatically
// convert the number to a string
// and combine them
let result = num + text; // "5Hello"

// Another example:
// the value of sum will be "510",
// not 15
let sum = "5" + 10;
```

So, in JavaScript, when you try to combine a number with a string, the number is automatically converted to a string, and the result is `"5Hello"`. This automatic conversion is known as type coercion and is a hallmark of weak typing.

Recap:

- Strongly Typed languages require you to be explicit about data types and perform conversions yourself. Errors occur if you try to mix types improperly (e.g., combining a string with an integer).
- Weakly Typed languages are more relaxed and try to convert types automatically, which can be convenient but might sometimes lead to unexpected behaviour (e.g., `"5" + 10` results in `"510"` instead of `15`).

Understanding static and dynamic typing

The difference between static typing and dynamic typing is about when the type of a variable is determined and whether you need to declare it explicitly.

1) Static Typing

In statically typed languages, you must declare the type of a variable when writing the code, and the type is checked at compile time (before the program runs).

Key Points:

- The type is known and fixed when the code is written.
- The compiler checks for type errors before the program runs, preventing some bugs.
- You cannot change the type of a variable after it's declared.

Example Java:

```
// You must declare the type as 'int'
int age = 25;

// Trying to assign a different type later
// will cause an error
// Error: incompatible types
age = "twenty-five";
```

In Java, the type of 'age' is declared as 'int', and the compiler won't let you assign a string to it later.

2) Dynamic Typing

In dynamically typed languages, you do not need to declare the type of a variable, and the type is determined at runtime (while the program is running).

Key Points:

- The type is figured out as the program runs.

- You can assign a value of any type to a variable, and change its type later if needed.
- The flexibility comes at the cost of potential runtime errors.

Example Python

```
# No need to declare the type
age = 25

# You can change the type of 'age' at
# any point
# Works fine; now it's a string
age = "twenty-five"
```

In Python, you don't declare the type of 'age' upfront. You can assign an integer, and later assign a string to the same variable without any issues —until the program runs and an error might occur.

Recap:

- In Static Typing the variable type is declared and fixed when you write the code (before running the program). Example: Java, C.
- In Dynamic Typing the variable type is determined at runtime, and it can change as the program runs. Example: Python, JavaScript.

The best way to study data types

You should do this in two steps; mastering the general core concepts that apply to all languages, then learn the individual language implementations. This way, you'll build a strong foundation and can easily switch between languages by learning their specific syntax and behaviour around data types. Let us break down the two steps.

- 1. Understand the language-agnostic core concepts. This involves properly understanding: a) Primitive vs. Reference Types: Grasp the idea of primitives (basic, low-level types) vs. reference types (complex, stored by reference). b) Strong vs. Weak Typing: Understand the difference between strongly typed (Java, C++) and weakly typed (JavaScript) languages. See the in-depth explanation on strong vs weak typing later below. c) Static vs. Dynamic Typing: Get comfortable with the idea of static typing (e.g., Java, where types are checked at compile time) vs. dynamic typing (e.g., Python, where types are checked at runtime). See the in-depth explanation on static and dynamic typing later below.

- 2. Study language-specific implementations. Once you understand the core concepts, focus on how individual languages implement these concepts. This will help you adapt to the specifics of any language while keeping the big picture in mind. When learning a language, make sure you:

a) Study its primitive types and understand how it handles more complex data structures. b) Learn about its memory management and how it treats data types internally (e.g., Java uses objects for everything except primitives, while Python treats everything as an object).

We have pretty much covered the first step in the notes above, where we mastered the core concepts by theoretically examining all the facets of data types. This involved learning about primitive vs reference types, strong vs weak types, and static vs dynamic types which are programming-language agnostic. We just have step 2 to cover. You will do step 2 on your own whenever you pick up a new language to learn. You just have to make sure you learn of how it implements primitive types as well as reference types-which are more complex data structures.

Explicit casting

JavaScript is a loosely typed language, meaning it does not require you to explicitly declare the datatypes of all variables, function/method parameters or objects that you use. What it does is to automatically deduce the type you intend to have from the values you use. For example, if you assign an int to a variable, like so:

```
let num = 25;
```

JS then automatically treats that number variable as an int, and if you later re-assign num to a string eg:

```
num = 'Welcome';
```

It internally converts the data type of num to a string. This behaviour of second-guessing (implying) the datatype you desire and automatically converting (casting) the datatype of the output for you is known as ‘implicit casting’. Casting is the process of converting a variable from one data type to another. If it is done by the programming language, it’s implicit, if done by you the programmer, then it is explicit. There are times when you may want to explicitly cast a variable output to a data type because the implicit casting will not be what you precisely want.

- In JavaScript, explicit casting (or type conversion) can be done in several ways, depending on the types you’re converting between. Here are some common

methods:

- cast to string
- cast to a number
- cast to a boolean
- cast to an object
- cast to an array

1) Casting to String

For this we use either the `String()` function or the `toString()` method. For example:

String() function

```
let num = 123;

// convert to "123"
let str = String(num);
```

toString() method

```
let num = 123;

// convert to "123"
let str = num.toString();
```

2) Casting to a number

There are 3 ways to do this:

a) Using the `Number()` function:

```
let str = "123";

// convert to 123
let num = Number(str);
```

b) Using unary plus (+) operator

```
let str = "123";

// converts to 123
let num = +str;
```

c) Using `parseInt()` (for integers) or `parseFloat()` (for floating-point numbers)

```
let str = "123.45";

// convert to 123.45
let num = parseFloat(str);
```

3) Casting to a boolean

There are 2 ways to cast to a boolean;
using the `Boolean()` function or using double negation. For example:

a) Using the `Boolean()` function

```
let value = 1;

// converts to true
let bool = Boolean(value);
```

b) Using double negation (`!!`)

```
let value = 1;

// converts to true
let bool = !!value;
```

4) Casting to an object

This is done by wrapping a primitive in its object equivalent.

```
let num = 123;

// converts to Number {123}
let obj = Object(num);
```

5) Casting to an array

This is done in 2 ways. You can either do it using `Array.from()` (for iterable or array-like functions), or you can use `split()` for strings.

a) Using `Array.from()`

```
let str = "hello";

// converts to ["h", "e", "l", "l", "o"]
let arr = Array.from(str);
```


b) Using split()

```
let str = "hello";

// converts to ["h", "e", "l", "l", "o"]
let arr = str.split('');
```

Types of data in JavaScript

JavaScript broadly categorises data types into primitive types and non-primitive types.

Primitive (7)

These are immutable (meaning they what they are in value and are not references, so they cannot be changed), and represent single values. Their values are literally what they appear to be. In JavaScript there are 5 primitive data types, plus two types Symbols and BigInt added in later specifications of JavaScript. Here is an abbreviation I came up with-use it or find your own way to remember them: SNBUNSB (SN BUN SB).

- i) String (literal value like 'hello world')
- ii) Number (represents both integers and floating-point numbers, like 42, or 3.14)
- iii) Boolean (represents true or false values)
- iv) Undefined (represents an uninitialised variable or missing value)
- v) Null (represents the intentional absence of a value)
- vi) Symbol (introduced in ES6: and represents a unique identifier)
- vii) BigInt (introduced in ES11: Represents large integers beyond the range of Number)

I will like to introduce to you at this point, a special numeric value worth knowing because it is very useful in handling numbers in JavaScript. It is NaN. NaN is a special numeric value that means 'Not a Number'. It is part of the Number type in JavaScript. However, it is not a data type, but rather a special numeric value within the Number data type. It represents the result of an invalid or undefined mathematical operation. For example:

```
console.log(0 / 0);           // NaN
console.log(Math.sqrt(-1));   // NaN
console.log(Number("abc"));   // NaN
```

I will explain how it works when we come to validating values to see if their type is a number.

Non-primitive (1)

These are objects, which can store multiple values or complex entities. Object: The base for many structures, including the following which we know are all objects in JavaScript:

- Arrays
- Functions
- Classes
- Other custom objects eg Interfaces

They all have different sizes depending on the type of data they contain.

Symbols explained

Symbols in JavaScript are unique and immutable identifiers introduced in ES6. They are often used to create unique keys for object properties, ensuring no naming conflicts, even in scenarios like extending or customising objects. Here are some examples to help you understand Symbols:

Symbol Creation

```
// Creating symbols
// 'description' is optional and for debugging only
const sym1 = Symbol('description');
const sym2 = Symbol('description');

// Symbols are unique this will return false
console.log(sym1 === sym2);
```

Symbols as Object Keys

```
const symKey = Symbol('uniqueKey');

const myObject = {
  [symKey]: 'value',
};

// Output: 'value'
console.log(myObject[symKey]);

// The symbol key doesn't clash with string keys
myObject['uniqueKey'] = 'another value';

// Output: 'value'
console.log(myObject[symKey]);
```

```
// Output: 'another value'
console.log(myObject['uniqueKey']);
```

Symbols and Iteration

Symbols are not enumerable, meaning they don't appear in for...in loops or Object.keys(). However, you can explicitly access them using Object.getOwnPropertySymbols.

```
const sym1 = Symbol('key1');
const sym2 = Symbol('key2');

const obj = {
  [sym1]: 'value1',
  [sym2]: 'value2',
};

// [] (symbols are not enumerable)
console.log(Object.keys(obj));

// [Symbol(key1), Symbol(key2)]
console.log(Object.getOwnPropertySymbols(obj));
```

Built-in Symbols (aka well-known Symbols)

JavaScript has several built-in Symbols known as "well-known symbols." These are used to customise or override default behaviours in objects.

Example 1: Symbol.iterator

Used to define custom iteration behaviour for an object.

```
const iterable = {
  values: [1, 2, 3],
  [Symbol.iterator]() {
    let index = 0;
    return {
      next: () => ({
        value: this.values[index++],
        done: index > this.values.length,
      }),
    };
  },
};

// Outputs: 1, 2, 3 }
for (const value of iterable) { console.log(value);
```

Example 2: Symbol.toPrimitive

Controls how an object converts to a primitive value.

```
const obj = {
  [Symbol.toPrimitive](hint) {
    if (hint === 'string') return 'Object as string';
    if (hint === 'number') return 42; return null;
  },
};

// Outputs 'Object as string'
console.log(`${obj}`);

// outputs 42
console.log(+obj);
```

The Symbol type provides a way to ensure unique, non-clashing keys and customise object behaviours in JavaScript. They're particularly useful in large-scale applications and libraries. It is one of JavaScript's primitive data types, just like string, number, or boolean. But unlike the others, it's a more advanced feature that most beginner (and even many experienced) developers rarely use in day-to-day code. It was introduced mainly for building safer, more robust libraries or frameworks — where developers want to create unique identifiers that won't accidentally clash with other property names. While it's useful in specific cases (like creating private-like object keys or customizing how objects behave in certain operations), you can go a long way in JavaScript without ever needing to use it. Still, it's good to know that it exists — just in case you see it in someone else's code or get curious later on. Here is a simple example of using a Symbol as a hidden property key:

```
// Create a Symbol to use as a hidden key
const secretKey = Symbol('secret');

const user = {
  name: 'Alice',
  age: 30,
  [secretKey]: 'This is a hidden value'
};

// Accessing normal properties
console.log(user.name);    // Alice
console.log(user.age);     // 30

// Trying to list all properties
console.log(Object.keys(user));
```

Output: ['name', 'age'] — no sign of the Symbol key

```
// But the hidden value is still there
console.log(user[secretKey]);
```

Output: This is a hidden value

Let me explain the above example. `Symbol('secret')` in JavaScript is how you create a unique identifier. Each time you call `Symbol('secret')`, the value it returns will be different. So:

- we use `Symbol('secret')` to create a unique value which we want to use as the value of a property which we want to create on our object called `secretKey`.
- The great thing is; when you run `Object.keys(user)`, the symbol-keyed property is not included — it's like a hidden property.
- But if you know the `Symbol`, you can still access the value using `user[secretKey]`.

A `Symbol` is useful for the following reasons;

- It helps avoid property name collisions (two pieces of code accidentally using the same key).
- It's great for adding "internal" values to objects that other code shouldn't mess with.
- Used often in libraries or framework internals — for example, JavaScript itself uses well-known symbols like `Symbol.iterator`.

Undefined vs null values

Both `null` and `undefined` mean "there is nothing here", which is why they are so often confused. The difference is in who put the nothing there. `undefined` is JavaScript's way of saying "nobody has given this a value yet". You get it when you declare a variable without assigning to it, when you read a property that does not exist, or when a function returns nothing at all. `null` is a value you set deliberately. It is a programmer saying "this is empty, and I meant it to be". JavaScript will never hand you `null` on its own.

```
let notSetYet;
// undefined - nobody assigned anything
let deliberatelyEmpty = null;
// null - we chose this

console.log(notSetYet);
// undefined
console.log(deliberatelyEmpty); // null
```

A useful way to hold on to it: undefined is the absence of a value, null is the presence of an empty one. Now to how they compare, which is where the three results below often surprise people:

```
// false - loose in value, but not the same type
console.log(null === undefined);

// true - loosely equal, because == treats them as
// two ways of saying "nothing"
console.log(null == undefined);

// "object" - see the note below
console.log(typeof null);
```

That last one is not a mistake in the book. `typeof null` really does return "object", even though null is not an object at all. It is a bug that has been in JavaScript since the very first version, and it can never be fixed now because too much existing code depends on it. Just remember it, and test for null with `=== null` rather than with `typeof`.

Convert the datatype of a variable

One way to convert a value from one type to another is JavaScript's `parseInt()` function, which turns a string into a whole number. Here is how you would use it:

```
let num = "5";
alert('The type of '+num+' is '+typeof num);

num = parseInt(num);
alert('Now the type of '+num+' is '+typeof num);
```

The first alert popup will display: "The type of 5 is string" The second alert popup will display: "Now the type of 5 is number".

Two things to know about `parseInt()` before you reach for it. It gives you whole numbers only, so `parseInt("3.9")` is 3, not 3.9 - use `Number()` if you want the decimals. And if the text is not a number at all, `parseInt("abc")` gives you NaN rather than an error. `parseInt()` is only one of several ways to convert between types. The full set, covering strings, numbers, booleans, objects and arrays, is in the Explicit casting section earlier in this chapter.

You can also display the contents of a variable containing an object or array in JavaScript. You just have to convert that data into a string so you can display it on screen or write it to the console. Here is how you do it:

```
let testData = [5, 10, 20, 25];

let customer = {
```

```

name: 'Tom Sawyer',
  age: 10,
  brother: 'Sid',
  aunt: 'Polly'
};

```

```

// convert the testData array into
// a string using JSON.stringify()
console.log("The testData array contains: " + JSON.stringify(testData));
// this will output:

```

The testData array contains: [5,10,20,25]

```

// convert the customer object into
// a string using JSON.stringify()
console.log("The customer object: " + JSON.stringify(customer));
// It will output:

```

The customer object: {"name":"Tom Sawyer","age":10,"brother":"Sid","aunt":"Polly"}

Built-in utility functions for type checking

JavaScript offers you some very useful built-in helper functions and operators that you can use to check the data type of any value. For example, you will be able to tell in code if a value you are dealing with is a number or not. If you are already thinking that this will be very handy when validating form input values to make sure they conform specific types, you are absolutely spot on. Values validation is one of the most popular uses of these type utility functions and operators. Understanding these will help to make you an efficient JavaScript programmer. Here's a clean, alphabetical list of JavaScript's built-in utility functions for type checking, along with short explanations and simple code examples for each.

- 1. `Array.isArray()` Checks if a value is an array. For example:

```

console.log(Array.isArray([1, 2, 3]));
console.log(Array.isArray("hello"));

// let's pass it a variable
let someText = "This is some text";
console.log(Array.isArray(someText));

```

Output: true // [1, 2, 3] is an array false // text is not array false // text is not array

- 2. `instanceof` Checks if an object is an instance of a particular class or constructor. For example

```
class Animal {}
const dog = new Animal();

console.log(dog instanceof Animal);
console.log(dog instanceof Object);
```

Output: true true // Object is the parent of all JavaScript objects

- 3. isNaN() Checks if a value is Not-a-Number. It coerces the value to a number first. For example

```
console.log(isNaN("hello"));
console.log(isNaN(42));
```

Output: true // → "hello" becomes NaN false // 42 is a number

I need to add some clarification with this one. When I say that isNaN() first of all coerces its value to a number, I mean, in the example:

```
isNaN("hello");
```

JavaScript tries to convert the string "hello" into a number before checking if it is NaN. NaN is a special numeric value that means 'Not a Number'. It is part of the Number type in JavaScript. However, do not mistake it for a data type, for it is not a data type. Rather, it is a special numeric value within the Number data type. It represents the result of an invalid or undefined mathematical operation. For example:

```
console.log(0 / 0); // NaN
console.log(Math.sqrt(-1)); // NaN
console.log(Number("abc")); // NaN
```

Here's what happens behind the scenes:

```
Number("hello"); // becomes NaN
```

Since "hello" is not a valid number, JavaScript converts it to NaN. Then checks if its a NaN:

```
isNaN("hello") → isNaN(NaN) → true
```

That's why

```
console.log(isNaN("hello"));
// returns true
```

It is important to understand this distinction because this behavior can be confusing. Let's take this other example

```
isNaN("123");
```

Will return false-because "123" becomes the number 123 which is a number, so it is not considered to be a NaN. So, it is the conversion it does that can really cause the confusion. You and I know that "123" is not a number (NaN), and so it

should return true, but it gets converted and therefore returns false—meaning it is a number. Let's look at one more example:

```
let nan = NaN;
let number = 123;
let numberString = "123";
let helloString = "hello";

console.log(isNaN(nan));
console.log(isNaN(number));
console.log(isNaN(numberString));
console.log(isNaN(helloString));
```

Returns: true false false true // coercion to NaN first

The trick lies in remembering that it always converts its value to a NaN first, before checking its value.

Use `isNaN()` when you're okay with JavaScript trying to convert the value to a number first.

There is a better way to check if a value is literally a NaN or not, and it does that without this conversion. This is by using `Number.isNaN()`, which I talk about next.

- 4. `Number.isNaN()` A stricter version of `isNaN()`. Only returns true for the actual NaN value. If the value was not already a NaN-literally, it will return false. For example

```
let nan = NaN;
let number = 123;
let numberString = "123";
let helloString = "hello";

console.log(Number.isNaN(nan));
console.log(Number.isNaN(number));
console.log(Number.isNaN(numberString));
console.log(Number.isNaN(helloString));
```

Output: true false false false // no coercion

Use `Number.isNaN()` when you want a strict, reliable check — only returns true for the actual NaN value.

- 5. `typeof`

This is used to check the data type of a value. It works well with primitives and objects. It returns a string showing the type of the value. For example:

```
typeof "";           // returns "string"
typeof "John";       // returns "string"
typeof 3;            // returns "number"
typeof true;         // returns "boolean"
typeof false;        // returns "boolean"
```

Take care with `typeof` and arithmetic. It binds more tightly than `+`, so this does not do what it looks like:

```
typeof 2 + 2;  
// "number2", not "number"
```

That is `(typeof 2) + 2`, which is the string "number" joined to the number 2. Put brackets round the sum if that is what you meant:

```
typeof (2 + 2);    // "number"  
console.log(typeof "hello");    // "string"  
console.log(typeof 42);         // "number"  
console.log(typeof true);  
// "boolean"  
console.log(typeof {});        // "object"  
console.log(typeof null);  
// "object" ← (weird quirk!)  
console.log(typeof undefined);  
// "undefined"
```

You might ask the question: Why does this return a object:

```
console.log(typeof null);
```

It is one of JavaScript's most infamous quirks! This happens because of a bug in JavaScript's original implementation that has been around since the very beginning (1995), and it has never been fixed for backward compatibility reasons. But don't worry—even though it says 'object', null just means 'nothing here' and it's actually not an object at all

Here's what happened:

- In the early version of JavaScript, all values were represented as types using a tag system under the hood.
- The type tag for objects was 0.
- The value null also got the type tag 0 by mistake.
- So when `typeof` checks the internal type tag of null, it wrongly reports it as "object".

So what data type is null really meant to be:

- null is a primitive type.
- It means "no value", or "empty value".
- It is not an object.

So, how can you accurately check for a null type? If you want to accurately check for null, use strict comparison instead. For example:

```
if (value === null) {
  console.log("It is really null!");
}
```

-6) `value === null` This uses the JavaScript identical operator (`===`) which compares the values as well as the data types of two values. Here it shows how you can use it to check if a value is exactly null. For example:

```
let x = null;
console.log(x === null); // true
```

-7) `value === undefined` This uses the JavaScript identical operator (`===`) which compares the values as well as the data types of two values. Here it shows how you can use it to check if a value is exactly undefined. This is the way to check if a variable is undefined (or is defined but has no value yet). For example:

```
let y;

console.log(y === undefined); // true
```

BOOLEANS

A boolean is a data type with exactly two possible values: `true` and `false`. They are written in lowercase, and only in lowercase.

Be careful here if you have come to JavaScript from another language. Some languages, PHP among them, also accept `TRUE` and `FALSE` in capitals as the same thing. JavaScript does not. Written in capitals they are not booleans at all, just names JavaScript has never heard of, and using one gives you a `ReferenceError`:

```
let isValid = true;

if (isValid) {
  console.log("Valid!");
}

// Error: ReferenceError: TRUE is not defined
let alsoValid = TRUE;
```

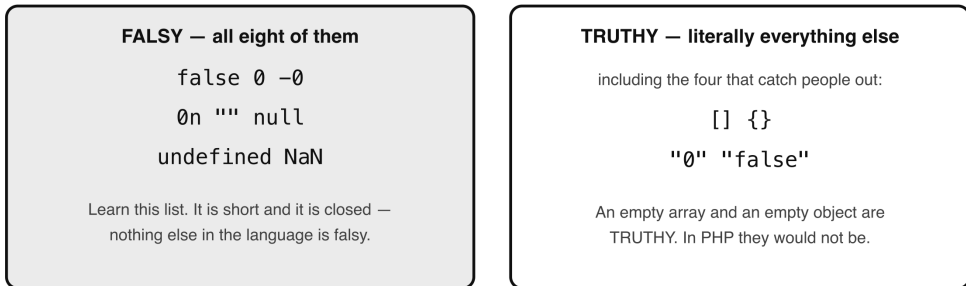
So there is only one form to remember, which is one less thing to think about.

The different interpretations of true or false

JavaScript will happily accept things other than `true` and `false` wherever it expects a yes-or-no answer, such as inside an `if` statement. When it meets a value that is not a boolean in one of those places, it works out whether to treat it as `true`

or false. Values that come out as false are called falsy, and everything else is truthy. There are exactly eight falsy values in JavaScript, and it is worth learning the list,

The eight falsy values — and everything else



So to ask "does this array have anything in it?", never write
`if (myArray)` — that is always true. Ask `if (myArray.length > 0)` instead.

- Figure 10.1 — The eight falsy values, and everything else*

because everything not on it is truthy:

- i) false the boolean itself
- ii) 0 and -0 zero, either sign
- iii) 0n zero as a BigInt
- iv) "" an empty string
- v) null a deliberate "no value"
- vi) undefined a value never set
- vii) NaN "Not a Number"

Everything else is truthy. That includes some things people often expect to be falsy:

-an empty array, `[]` -an empty object, `{}` -the string `"0"`, and even the string `"false"`

That first one catches people out constantly, and it is worth pausing on, because in several other languages an empty array IS falsy. Not here:

```
if ([]) {  
    console.log("This really does run.");  
}
```

// Output: This really does run.

So if you want to know whether an array is empty, testing it directly will not tell you. You have to ask about its length.

There are times when you want to be sure not just that something is falsy, but exactly what it is. Knowing a value is falsy does not tell you whether it is null, an

empty string or zero, and those often need handling differently. For that, use the strict equality operator (===) from Chapter 5, which compares the type as well as the value. Here is how to tell each case apart.

- 1. null null represents a deliberate absence of a value.

```
let value = null;

if (value === null) {
  console.log("The variable is null.");
}
```

-Distinguishing: use strict equality (===). Do not use == here, because null == undefined is true, and you would not be able to tell the two apart.

- 2. undefined undefined means no value was ever put there, as opposed to null, which means someone deliberately put "nothing" there. There is more on the difference earlier in this chapter.

```
let value;

if (value === undefined) {
  console.log("The variable is undefined.");
}
```

- 3. An empty string ("") A string with no characters in it.

```
let value = "";

if (value === "") {
  console.log("The variable is an empty string.");
}
```

- Distinguishing: strict equality works perfectly well for strings, because a string is a primitive and is compared by its value.
- 4. An empty array ([]) This one needs a different approach, and here is why. An array is a reference type, as we saw in Chapter 3. Two arrays are never strictly equal to each other, even when both are empty, because they are two different arrays sitting in two different places in memory:

```
console.log([] === []); // false, always
```

So a test like if (value === []) can never be true, no matter what value holds. Ask about the length instead, having first checked that it really is an array:

```
let value = [];

if (Array.isArray(value) && value.length === 0) {
  console.log("The variable is an empty array.");
}
```

- 5. Zero (0) The number zero.

```
let value = 0;

if (value === 0) {
  console.log("The variable is zero.");
}
```

-Distinguishing: strict equality again, and here it matters more than usual. With loose equality, `0 == ""` and `0 == false` are both true, so `==` would tell you a value is zero when it is actually an empty string.

The relationship between data types and the Built-in Object of JavaScript

JavaScript has a built-in object named `Object` which is the parent of all JavaScript objects. This means That all objects automatically inherit from it, and this is Is very useful because these objects can then make use of the powerful utility methods and properties of `Object`. So the best way to describe the relationship between `Object` and other non-primitive objects in JavaScript is as follows:

- The object data type represents all non-primitive values, while `Object` is a constructor and utility provider for working with objects.
- All objects (including arrays, functions, etc.) ultimately inherit from `Object` via the prototype chain. We will learn more about `Object` and how to use it in the Object Oriented section.

QUIZ — Data Types

This page contains the Q & A (questions and answers) for this chapter — Chapter 10: Data Types. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 11 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What is the difference between null and undefined? Which one does JavaScript give you on its own, and which one do you have to ask for?

Clue: both mean "nothing here". The difference is who put the nothing there.

2. JavaScript has exactly eight falsy values. Name as many as you can, then say which of these three is NOT among them:

`[]` `{}` `""`

Clue: two of those three surprise people, because in several other languages they would be falsy.

3. What does this print, and why might it catch you out?

```
if ([]) {  
  console.log("Yes");  
} else {  
  console.log("No");  
}
```

Clue: see question 2.

4. A friend writes this to check whether an array is empty, and it never works. Why not, and what should they write instead?

```
if (myData === []) {  
  console.log("It is empty");  
}
```

Clue: think back to Chapter 3, and what it means for an array to be a reference type.

5. What does `typeof null` return, and is that correct?

Clue: the answer is famously wrong, and has been since 1995.

6. What does this print? Look very carefully at the order things happen in.

```
console.log(typeof 2 + 2);
```

Clue: `typeof` grabs hold of what is next to it before the `+` gets a look in.

7. What is the difference between `==` and `===` when comparing `null` and `undefined`?

Clue: one of them cares about the type, the other does not.

7. You have a value and you want to know whether it is a number. Name a built-in way to check, and say what it would give you for the string `"5"`.

Clue: the operator from questions 5 and 6.

8. EXERCISE. Write code that declares one variable left unset and one deliberately set to `null`. Print both, and print the result of comparing them with `==` and with `===`.

Clue: you should get `undefined`, `null`, `true` and `false`, in that order.

9. EXERCISE. Write a function called `isEmptyArray(value)` that returns `true` only when the value really is an array with nothing in it. Test it with an empty array, an array with items, an empty string, and `null`.

Clue: you need two checks joined with `&&`: that it is an array at all, and that its length is zero.

11. EXERCISE. Take an array and an object, convert each into a string, and print them.

Clue: there is one built-in method that does this for both, and it has `JSON` in the name.

ANSWERS

1. Both mean "there is nothing here". The difference is in who put the nothing there.

- **undefined** is what JavaScript gives you on its own. You get it from a variable that was declared but never assigned, from a property that does not exist, and from a function that returns nothing.
- **null** is one you set deliberately. It is a programmer saying "this is empty, and I meant it". JavaScript will never hand you `null` by itself.

```
let notSetYet;  
// undefined - nobody assigned anything  
let deliberatelyEmpty = null;
```



```
// null – we chose this
```

A short way to hold on to it: undefined is the absence of a value, null is the presence of an empty one.

2. The eight falsy values are:

```
false 0 -0 0n "" null undefined NaN
```

Of the three given, only "" is falsy. Both [] and {} are truthy.

That surprises a lot of people, and it is worth knowing why: in several other languages, PHP among them, an empty array counts as false. In JavaScript it does not. Anything that is not on the list of eight above is truthy, and an empty array is an object, so it is truthy.

3. It prints:

Yes

Because [] is truthy, the if branch runs. If you have come from a language where an empty array is falsy, you would expect "No", and this is exactly the sort of thing that produces a bug you stare at for an hour.

If what you meant was "does this array have anything in it", ask about its length:

```
if (myArray.length > 0) { ... }
```

4. It never works because two arrays are never strictly equal to one another, even when both are empty:

```
console.log([] === []);  
// false, always
```

An array is a reference type, as we saw in Chapter 3. The name does not hold the array itself, only a pointer to where it lives. So myData === [] is comparing a pointer to your array against a pointer to a brand new empty array that was just created on that line. They are two different arrays, so they are never the same one.

What to write instead:

```
if (Array.isArray(myData) && myData.length === 0) {  
    console.log("It is empty");  
}
```

Two checks: that it is an array at all, and that it has nothing in it.

5. It returns "object".

```
console.log(typeof null);  
// "object"
```

And no, that is not correct. `null` is not an object. It is a bug that has been in JavaScript since the very first version in 1995, and it can never be fixed now, because far too much existing code relies on the current behaviour.

So do not use `typeof` to test for `null`. Use strict equality:

```
if (value === null) { ... }
```

6. It prints:

```
number2
```

not `"number"`. `typeof` binds more tightly than `+`, so JavaScript reads it as `(typeof 2) + 2` — that is, the string `"number"` joined to the number 2, which gives the string `"number2"`.

If you meant to check the type of the sum, put brackets round it:

```
console.log(typeof (2 + 2));  
// "number"
```

7. `null == undefined` // true
 `null === undefined` // false

The loose operator `==` treats `null` and `undefined` as two ways of saying the same thing, so it reports them as equal. The strict operator `===` compares the type as well, and they are different types, so it reports them as different.

This matters when you are testing for one specifically. If you write `if (value == null)` you are also catching `undefined`, which may or may not be what you wanted. Use `===` when you need to tell them apart.

8. The `typeof` operator:

```
console.log(typeof 5);  
// "number"  
console.log(typeof "5");  
// "string"
```

For the string `"5"` it gives you **`"string"`**, not `"number"`. The quotes are what matter, not what is inside them. A number that arrived from a form field or a URL will be a string until you convert it — which is why the conversion section in this chapter exists.

```

9. let notSetYet;
   let deliberatelyEmpty = null;

   console.log(notSetYet);
   // undefined
   console.log(deliberatelyEmpty);
   // null
   console.log(notSetYet == deliberatelyEmpty);
   // true
   console.log(notSetYet === deliberatelyEmpty);
   // false

```

The last two lines are the whole point. Loosely, JavaScript considers them the same sort of nothing. Strictly, it does not, because their types differ.

```

10. function isEmptyArray(value) {
      return Array.isArray(value) && value.length === 0;
    }

    console.log(isEmptyArray([]));
    // true
    console.log(isEmptyArray([1, 2, 3]));
    // false
    console.log(isEmptyArray(""));
    // false
    console.log(isEmptyArray(null));
    // false

```

The `Array.isArray()` check has to come first, and not only for tidiness. An empty string also has a length of 0, so without it `isEmptyArray("")` would wrongly say true. And null has no length property at all, so reading `.length` on it would throw a `TypeError` — the `&&` stops before it ever gets that far.

```

11. let testData = [5, 10, 20, 25];

    let customer = {
      name: 'Tom Sawyer',
      age: 10,
      brother: 'Sid',
      aunt: 'Polly'
    };

    console.log("The testData array contains: " +
      JSON.stringify(testData));
    console.log("The customer object: " + JSON.stringify(customer));

```

Output:

```

The testData array contains: [5,10,20,25]
The customer object: {"name":"Tom
Sawyer","age":10,"brother":"Sid","aunt":"Polly"}

```

`JSON.stringify()` handles both arrays and objects. Without it, joining an object to a string gives you the unhelpful "[object Object]", which is a message most JavaScript programmers have seen more often than they would like.

Chapter 11 — DATA STRUCTURES

Intro Data Structures and Data Types List of data structures

- Arrays
- Linked lists
- Stacks
- Queues
- Tuples
- Dictionaries (Maps/HashMaps)
- Sets
- Structs
- Trees (Binary Trees, AVL Trees, etc)
- Collections

The topic of data structures is like an extended study of data types. When we studied data types; we learned about the two groups that make them: primitive and reference types. Data structures are closely related to reference types in programming because they often deal with collections of data and more complex data management, as opposed to the simple, single-value nature of primitive types.

Data Structures and Data Types

We know that where programming languages are similar when it comes to handling data types is in the fact that they all have the core concepts of primitive and reference types. They only differ in the way they implement these concepts. Before we introduce data structures, let us briefly remind ourselves of what primitive types are again. Primitive types are the simplest, indivisible data types that store single values. Examples are integers, booleans, and characters. They don't involve any structure or organization beyond the basic storage of a single piece of data. There is the other group known as reference types, which are much closer to data structures. Let us see why. Data structures are ways of organizing and storing data so that they can be accessed and modified efficiently. Since data structures often involve collections of data (e.g., lists, trees, hash tables), they need to store references to multiple pieces of data, rather than the actual values themselves. This is why data structures are typically implemented using reference types. Next, let us look at how data structures use reference types. Reference types hold the address

(or reference) to the actual data in memory. When you work with data structures like arrays, lists, or dictionaries, you're dealing with references to elements, not the actual values themselves. Mutable or Immutable. Many data structures allow you to change (mutate) the data they contain. This is possible because reference types can point to new or modified data without changing the variable itself. For instance, in a list, you can update individual elements without changing the whole structure. Also, with classes, you can change the property of an object without changing that data on the class where the object came from. Some examples of data structures are: - Arrays/Lists: They are collections of items stored in contiguous memory. Each element in the array is accessed through its reference (i.e., the index points to the location of the data). - Dictionaries/Maps: They store key-value pairs where each value is referenced by a key. The actual values and keys are reference types. - Trees: Trees are hierarchical structures where each node references its child nodes, creating a complex network of relationships between the data. - Stacks/Queues: These are collections of data organised in a specific order (e.g., Last In, First Out for stacks). Each item in the structure is referenced, making it easy to manage more complex data flows.

So, while primitive types represent single, indivisible data points (e.g., an integer or a character), data structures are complex ways of organizing multiple pieces of data and are built using reference types, because they need to handle multiple values, often of varying or unknown sizes, efficiently. In essence, data structures rely on reference types to manage and manipulate collections of data, while primitive types serve as the fundamental building blocks for individual data elements.

List of data structures

With that said, we are ready to study the different data structures. There are some things to keep in mind. Some data structures exist in certain languages but do not exist in others. For example, C and Go have structs, while JavaScript has none, and Python has a real tuple type where JavaScript has to make do with an array. Some data structures come built into a programming language, while in other languages, you would have to create them yourself in code. When studying each data structure, make sure with each, you are studying how the individual language you are learning handles that in memory, and that you are understanding the weaknesses and strengths of the data structure. This will help you know which one to use in which scenario when solving problems in programming. We will proceed with the list of data structures by listing each one and examining everything about it from which language uses it, how is it implemented, what kind of problems it can solve, what limitations it has etc. We will look at only the most popular structures. With this guidance, you will be able to pick any data structure from any new lang-

uage you are learning and master them, if we have not covered them here already. All the code examples here are in JavaScript. For each structure I will say plainly whether JavaScript has it built in or not. Where it does not, the code is there to show you how the structure behaves, built out of the pieces JavaScript does give you, rather than something the language hands you ready made. I will also say where each one actually turns up in real JavaScript work, because knowing a structure exists is only half of it; knowing when you would reach for it is the other half.

ARRAYS

Arrays are a collection of elements, stored one after another and reached by an index. They provide quick access to any element by its position, and are a foundational structure in most programming languages.

In JavaScript: built in. Arrays are part of the language and you have been using them since Chapter 3. Implementation: elements held in order and reached by index, counting from 0. Problems it solves: efficient storage and access of a list of values. Limitations: in many languages an array has a fixed size. JavaScript arrays grow and shrink freely, which is convenient, but it also means an array can hold a mixture of types, which other languages would not allow.

```
// Creating an array
let arr = [1, 2, 3, 4];

arr.push(5);           // add to the end
console.log(arr[0]); // 1
console.log(arr.length); // 5
```

Used in JavaScript for: almost everything. Lists of items, results from an API, collections of DOM elements, and as the building block for several of the structures below.

LINKED LISTS

Linked lists are made of nodes, where each node holds a value and a pointer to the next node. They allow easy insertion and removal anywhere in the list, but you have to walk through them from the start to find anything.

In JavaScript: NOT built in. There is no linked list type, so you build one yourself out of objects or classes. The code below is a demonstration of how the structure works, not something the language hands you. Implementation: a series of nodes, each pointing to the next. Problems it solves: efficient insertion and deletion, espec-

ially in the middle. Limitations: slower access than an array, because you must traverse the nodes to reach one.

```
// A node, built by hand
class Node {
  constructor(data) {
    this.data = data;
    this.next = null;
  }
}

// Joining three of them into a chain
let first = new Node(10);
first.next = new Node(20);
first.next.next = new Node(30);

// Walking the chain from the start
let current = first;

while (current !== null) {
  console.log(current.data);
  current = current.next;
}

// Output: 10, 20, 30, each on its own line
```

Used in JavaScript for: very little, in practice. An ordinary array does the same job with less effort, so you will rarely build one. It is worth understanding because it comes up in technical interviews, and because the idea of one thing pointing to the next turns up everywhere, including in the tree structure further down.

STACKS

Stacks are LIFO (Last In, First Out) structures, meaning the last item added is the first one removed. Think of a stack of plates: you take from the top.

In JavaScript: no separate stack type, but you do not need one. An ordinary array already behaves as a stack, because `push()` adds to the end and `pop()` takes from the end. Implementation: usually an array, sometimes a linked list. Problems it solves: undo operations, managing function calls, and backtracking. Limitations: restricted access, since you can only reach the top item.

```
// A stack, using a plain array
let stack = [];

stack.push(1);      // push
stack.push(2);
```



```
console.log(stack.pop());  
// 2 - last in, first out  
console.log(stack);      // [1]
```

Used in JavaScript for: undo and redo features, "go back" navigation, and checking that brackets or tags are properly nested. JavaScript itself uses one internally, the call stack, which keeps track of which function called which - it is the thing you see listed when an error is thrown.

QUEUES

Queues are FIFO (First In, First Out) structures, meaning the first item added is the first one removed. Think of a queue at a till: first come, first served.

In JavaScript: no separate queue type, but an array does the job. `push()` adds to the back and `shift()` takes from the front. Implementation: usually an array or a linked list. Problems it solves: handling tasks in the order they arrived, such as print jobs or scheduled work. Limitations: restricted access, since you can only reach the front and the back. Also, `shift()` has to renumber every remaining element, so on a very large array it is slower than it looks.

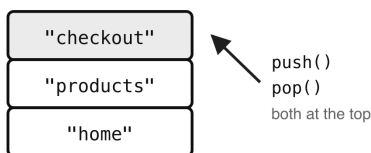
```
// A queue, using a plain array  
let queue = [];  
  
queue.push("first");    // enqueue  
queue.push("second");  
console.log(queue.shift());  
// "first" - first in, first out  
console.log(queue);  
// ["second"]
```

Used in JavaScript for: anything that must happen in order - a list of jobs waiting to run, messages waiting to be sent, or walking through a tree level by level. JavaScript's own event loop works on a queue, which is why things you schedule run in the order you scheduled them.

Stack and queue: same array, different end

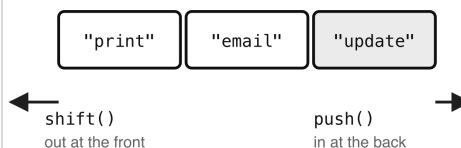
STACK — Last In, First Out

a pile of plates: you take from the top



QUEUE — First In, First Out

people at a till: first come, first served



In JavaScript both are just an array

```
stack.push(item); stack.pop();
```

— add and remove at the same end

```
queue.push(item); queue.shift();
```

— add at one end, remove from the other

The only difference is `pop()` versus `shift()`.

- Figure 11.1 — Stack and queue: same array, different end*

TUPLES

Tuples are ordered collections of a fixed size, often holding values of different types, which cannot be changed once created.

In JavaScript: NOT built in. There is no tuple type, so an array is used instead. The code below is a demonstration of the idea rather than a real tuple, because nothing stops you changing an array afterwards - unless you freeze it. Implementation: a small, fixed group of related values. Problems it solves: returning more than one value from a function, or representing a fixed record. Limitations: in languages that have real tuples they cannot be modified at all. JavaScript only gets close to that with `Object.freeze()`.

```
// An array standing in for a tuple
let tup = [1, "apple", 3.14];

// Pulling the values back out, using
// destructuring from Chapter 12
let [count, fruit, price] = tup;
console.log(fruit); // "apple"

// Freezing it, to get closer to a
// real tuple
const frozen = Object.freeze([1, "apple"]);
// silently ignored
```

```
frozen[0] = 99;
console.log(frozen[0]);
// still 1
```

Used in JavaScript for: returning several values from one function. You will meet this constantly in modern JavaScript, where a function hands back a small array and the caller unpacks it in one line.

Dictionaries (Maps/HashMaps)

Dictionaries, also called maps or hash maps, store data as key-value pairs. They offer fast lookup, insertion and deletion by key.

In JavaScript: built in, and in two forms. A plain object gives you key-value pairs with string keys, and the Map type gives you the same thing with keys of any type at all. Implementation: typically a hash table under the surface. Problems it solves: fast retrieval of a value when you know its key. Useful for lookups, caching and indexing. Limitations: keys must be unique. With a plain object, keys are always text, so a number key quietly becomes a string.

```
// As a plain object
let person = { name: "Alice", age: 30 };
console.log(person.name);
// "Alice"

// As a Map, which allows any type of key
let scores = new Map();
scores.set("alice", 10);
scores.set(42, "the answer");

console.log(scores.get("alice"));
// 10
console.log(scores.size);
// 2
```

Used in JavaScript for: settings and configuration, counting how often something appears, caching results so they are not worked out twice, and any time you want to look something up by name rather than by position. Reach for a Map when your keys are not strings, or when you need to know how many entries there are.

Sets

Sets are collections of unique values. Adding something twice has no effect the second time.

In JavaScript: built in, as the Set type. Implementation: typically backed by a hash table. Problems it solves: checking quickly whether something is present, and removing duplicates. Limitations: values must be unique, and you cannot reach an item by index the way you can in an array.

```
// A Set
let mySet = new Set([1, 2, 3, 4]);

// already there, ignored
mySet.add(4);
console.log(mySet.size);           // 4
console.log(mySet.has(3));
// true

// The neatest use: removing duplicates
// from an array
let withDuplicates = [1, 2, 2, 3, 3, 3];
let unique = [...new Set(withDuplicates)];
console.log(unique); // [1, 2, 3]
```

Used in JavaScript for: stripping duplicates out of a list, which is the one-line trick shown above, and keeping track of things you have already seen or already processed.

Structs

Structs are user-defined types that group several related pieces of data together into one record.

In JavaScript: NOT built in. There is no struct keyword. JavaScript uses an object literal, or a class when you want the same shape made repeatedly. The code below is therefore the JavaScript equivalent of a struct rather than a struct itself. Implementation: a group of named fields held together as one value. Problems it solves: keeping related data together as a single record instead of loose separate variables. Limitations: in languages like C and Go a struct holds only data, with no methods. A JavaScript object has no such restriction, which makes it more flexible but also less strict.

```
// The JavaScript equivalent: an object
// literal
let person = {
  name: "Alice",
  age: 30
};

// Or a class, when you need to make many
// of the same shape
```

```

class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

let alice = new Person("Alice", 30);
console.log(alice.name);
// "Alice"

```

Used in JavaScript for: any time you have several facts about one thing and want to keep them together - a user, a product, a setting. This is so ordinary in JavaScript that you will do it constantly without ever calling it a struct.

Trees (Binary Trees, AVL Trees, etc.)

Trees are hierarchical structures that store data in nodes, where each node has child nodes. They suit anything shaped like a family tree or a folder structure.

In JavaScript: NOT built in as a type, but you meet trees constantly all the same. You build your own out of objects or classes, as below. Implementation: nodes that point to child nodes. Problems it solves: efficient searching and sorting of hierarchical data. Limitations: a tree can become lopsided, which makes searching it slower.

```

// A binary tree node, built by hand
class Node {
  constructor(data) {
    this.data = data;
    this.left = null;
    this.right = null;
  }
}

// Example usage
let root = new Node(10);
root.left = new Node(5);
root.right = new Node(20);

console.log(root.left.data); // 5

```

Used in JavaScript for: more than you might expect. The DOM - the structure of the web page itself, which we come to in Chapter 15 - is a tree, where every element has a parent and may have children. Any nested JSON you get back from an API is a tree too. So although you will rarely build one from scratch, you will spend a great deal of time walking through them.

In the example above, we created a simple binary tree with a root node and two children, left and right. This basic structure can be extended to support more complex tree operations such as traversal, insertion and deletion.

COLLECTIONS

Collections are a broad category rather than a single structure. The word covers any structure that groups multiple values into one thing and offers ways to add, remove and modify them.

In JavaScript: built in. Array, Map, Set, WeakMap and WeakSet are all collections. You have met the first three already in this chapter. Implementation: each collection type stores its data differently, with its own strengths. Problems it solves: storing, retrieving and manipulating several values in a structured way. Limitations: each type has its own. A Set holds only unique values, an array is ordered but slower to search, a Map keeps insertion order but takes more memory than a plain object.

```
// The three collections you will use most
// Array
let arr = [1, 2, 3];
arr.push(4);

let myMap = new Map();    // Map
myMap.set("key", "value");

// Set
let mySet = new Set([1, 2, 3]);
mySet.add(4);

console.log(arr.length, myMap.size, mySet.size);
// Output: 4 1 4
```

Used in JavaScript for: choosing the right one for the job. Use an array for an ordered list, a Set when every value must be unique, and a Map when you want to look things up by a key that is not a string.

QUIZ — Data Structures

This page contains the Q & A (questions and answers) for this chapter — Chapter 11: Data Structures. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 8 to 11 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Of the structures in this chapter, which ones does JavaScript give you built in, and which do you have to build yourself?

Clue: three are handed to you ready made. The rest you assemble out of what the language already has.

2. What is the difference between a stack and a queue? Which array methods give you each one?

Clue: a stack is a pile of plates. A queue is people waiting at a till.

3. JavaScript has no tuple type. What is used instead, and what is the one thing you lose by doing it that way?

Clue: the missing quality is in the definition of a tuple.

4. You have a plain object and a Map, both storing key-value pairs. Name two things a Map can do that a plain object cannot.

Clue: one is about what you are allowed to use as a key. The other is about counting.

5. Write the one-line trick for removing duplicates from an array, and explain how it works.

Clue: it uses a structure from this chapter plus the three dots from Chapter 3.

6. JavaScript has no struct. What do you use in its place, and when would you pick a class over an object literal?

Clue: one is for a single record. The other is for making many of the same shape.

7. The chapter says you will rarely build a tree yourself, but will spend a lot of time walking through them. Why?

Clue: look at the page you are reading this on, and at what an API sends back.

8. EXERCISE. Build a stack that holds the pages someone has visited, so that a "back" button could return to the previous one. Push three pages, then go back twice.

Clue: an ordinary array already is a stack.

9. EXERCISE. Build a queue of three jobs waiting to be processed, and process them in the order they arrived.

Clue: add to one end, take from the other.

10. EXERCISE. Build a linked list of three nodes by hand and print every value in it, without knowing in advance how many nodes there are.

Clue: start at the first node and keep following `.next` until there is no next.

11. EXERCISE. Given a list of names with repeats in it, produce a list of the unique names, and then say how many unique names there are.

Clue: one structure from this chapter does both jobs.

ANSWERS

1. Built in:

- **Arrays** — the ones you have used since Chapter 3
- **Maps and plain objects** — for key-value pairs
- **Sets** — for collections of unique values

Built by you, out of what JavaScript already has:

- **Stacks and queues** — an ordinary array already behaves as both
- **Tuples** — an array stands in for one
- **Structs** — an object literal or a class stands in
- **Linked lists and trees** — assembled from objects or classes

That is a useful thing to notice in itself: JavaScript gives you a small number of flexible structures and expects you to build the rest from them, where a language like C or Go hands you more specialised types.

2. A **stack** is Last In, First Out. The last thing you added is the first thing you get back, like taking a plate off the top of a pile.

A **queue** is First In, First Out. The first thing you added is the first thing you get back, like people served in the order they joined the line.

Both use an array:

```
// stack – add and remove at the same end
stack.push(item);
stack.pop();

// queue – add at one end, remove from the other
queue.push(item);
queue.shift();
```

The only difference is `pop()` versus `shift()`.

3. An **array** is used instead. What you lose is **immutability**.

A real tuple cannot be changed once created. A JavaScript array can be changed by anyone at any time, so nothing stops the values being altered afterwards.

The nearest you can get is to freeze it:

```
const frozen = Object.freeze([1, "apple"]);
// silently ignored
frozen[0] = 99;
console.log(frozen[0]);
// still 1
```

Note that the attempt fails quietly rather than throwing an error, which is worth knowing.

4. A Map can:

- **Use any type as a key.** A plain object turns every key into text, so `person[42]` and `person["42"]` are the same key. A Map keeps them separate, and will happily use a number, an object or even a function as a key.
- **Tell you how many entries it has**, through `.size`. A plain object has no equivalent — you have to count its keys with `Object.keys(obj).length`, which you met in Chapter 3.

```
let scores = new Map();
scores.set("alice", 10);
scores.set(42, "the answer");

console.log(scores.get("alice"));
// 10
console.log(scores.size);
// 2
```

5.

```
let withDuplicates = [1, 2, 2, 3, 3, 3];
let unique = [...new Set(withDuplicates)];

console.log(unique); // [1, 2, 3]
```

It works in two steps. `new Set(withDuplicates)` builds a Set from the array, and because a Set only ever holds unique values, the repeats are dropped as it is built. Then the three dots — the spread operator from Chapter 3 — expand that Set back out into a new array.

Without the spread you would be left holding a Set rather than an array, which is fine if that is what you wanted, but usually you want an array back.

6. You use an **object literal**, or a **class**.

```
// one record
let person = { name: "Alice", age: 30 };

// many records of the same shape
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

let alice = new Person("Alice", 30);
```

Pick an object literal when you need one particular record. Pick a class when you will be making many things of the same shape, so that the shape is written down once instead of being repeated every time.

7. Because two of the things you work with constantly in JavaScript **are already trees**.

- The **DOM** — the structure of the web page itself, which Chapter 15 covers — is a tree. Every element has a parent, and may have children.
- **Nested JSON** coming back from an API is a tree too. An object holding objects holding arrays is a tree by another name.

So the skill worth having is not building trees, but moving around inside one and finding what you need in it.

8. `let history = [];`

```
history.push("home");
history.push("products");
history.push("checkout");

console.log(history.pop());
// "checkout" – go back
console.log(history.pop());
// "products" – go back again
console.log(history);
// ["home"]
```

A stack is exactly right here, because "back" always means the most recent page, which is the last one added. That really is how the back button works.

```
9. let jobs = [];  
  
    jobs.push("print report");  
    jobs.push("send email");  
    jobs.push("update record");  
  
    while (jobs.length > 0) {  
        console.log("Processing: " + jobs.shift());  
    }
```

Output:

Processing: print report Processing: send email Processing: update record

They come out in the order they went in, because `shift()` takes from the front while `push()` adds to the back. Swap `shift()` for `pop()` and you would process them backwards, which for a job queue would be quite wrong.

```
10. class Node {  
    constructor(data) {  
        this.data = data;  
        this.next = null;  
    }  
}  
  
let first = new Node(10);  
first.next = new Node(20);  
first.next.next = new Node(30);  
  
let current = first;  
  
while (current !== null) {  
    console.log(current.data);  
    current = current.next;  
}
```

Output: 10, 20 and 30, each on its own line.

The loop is the important part. We hold on to where we are in `current`, print it, then move `current` along to whatever it points at next. When we reach a node whose `next` is null, there is nowhere further to go and the loop stops. That is why we never needed to know how many nodes there were.

```
11. const names = ["Ada", "Grace", "Ada", "Alan", "Grace", "Ada"];

    const unique = [...new Set(names)];

    console.log(unique);
    // ["Ada", "Grace", "Alan"]
    console.log(unique.length);    // 3
```

A Set does both jobs at once. Building one from the array removes the repeats, and you can either spread it back into an array as above, or ask the Set itself how many it holds:

```
console.log(new Set(names).size);
// 3
```

Notice the order is preserved: Ada, Grace, Alan appear in the order they were first seen. A Set will not hold duplicates, but it does remember the order things arrived in.

Chapter 12 — OBJECT ORIENTED PROGRAMMING (OOP)

- Introduction
- Relationship between the object data type and Object
 - How to create an object
 - The Object Constructor
 - Object.prototype
 - Static Methods of Object
 - Understanding how to use the prototype property
 - The traditional two ways of creating an object and their prototypes
 - object initialisers (aka object literals)
 - constructor functions
 - The new way to create objects and the prototype
 - The Object.create() method
 - Objects and their prototypes
 - The constructor property
 - Handy properties and methods of Objects
 - The prototype property
 - The constructor property
 - isPrototypeOf() and the instanceof operator
 - getPrototypeOf()
 - The **proto** property
 - The getOwnPropertyNames()
 - The hasOwnProperty() method (non-inherited)
 - Check all properties of an object (including inherited)
 - Creating a custom method on an object
 - The call(), apply() and bind() methods of Function
 - Passing arguments when using call() and apply()
 - The bind() function
- OBJECTS IN DEPTH
 - Object literals

- JavaScript objects vs object literals
- Object literals vs constructor function
- Changing object literal values on the fly
- Restricting modification on objects
- The difference between a JavaScript object and JSON
- Converting a JavaScript object to JSON
- Converting JSON to a JavaScript object
- Destructuring of objects
- Simplified object literal creation from function arguments
- Viewing the contents of an object
- Classes
 - Class inheritance
 - difference between a child class and an instance
 - Call parent constructor from child class
 - Check if an object is an instance of a class
 - Function Constructors, object literals and inheritance
 - Function Constructors (Old Way of Creating Classes)
 - Object literals
 - Class setters and getters
- Sharing data between files
 - The difference between import and require
 - Exporting multiple functions or variables
 - Using a wildcard to import all from a file
 - Export fallback with export default
- Other advanced OOP concepts
 - Static methods
 - Abstract classes
 - Interfaces
 - Dependency Injection (DI)
 - Private methods and properties (Encapsulation)

Introduction

Object Oriented Programming (OOP) refers to the ability of a programming language to handle the building blocks of an application as objects and properties. This offers a much more organised, readable, testable, and scalable approach to building software applications. These benefits it offers and more, will be evident as the OOP topic is broken down in detail.

Relationship between the object data type and Object

I mentioned before under the topic of data types that JavaScript has a built-in object named `Object` that is the parent of all objects in your code. Well, the built-in `Object` in JavaScript is a constructor function, and its prototype—`Object.prototype`—is what every object in JavaScript ultimately derives from. It provides the foundation for the object data type by offering essential methods and properties that every object can inherit. Here's how the built-in `Object` relates to the object data type: First of all as a reminder; any value in JavaScript that is not a primitive (like string, number, boolean, undefined, null, `BigInt`, or `symbol`) is an object. The relationship between an object data type and `Object` can best be described in the following ways:

- The object data type represents all non-primitive values, while `Object` is a constructor and utility provider for working with objects.
- All objects (including arrays, functions, etc.) ultimately inherit from `Object` via the prototype chain.

Here is an example of the prototype chain:

```
// outputs true
const obj = {};
console.log(obj.__proto__ ===
  Object.prototype);

// outputs null (end of chain)
console.log(Object.prototype.__proto__);
```

The built-in `Object` is both a constructor and a toolkit for working with the object data type. It provides shared functionality, enabling the creation, manipulation, and behaviour customisation of all JavaScript objects.

How to create an object

Objects can be created in various ways

- Object literals: { key: 'value' }
- Using constructors like new Object(), new Array(), or new Map().
- Through classes, functions, or prototype inheritance.

The Object Constructor

The built-in Object is a constructor that allows creating objects explicitly. It also provides static methods for working with objects. To prove to you that all objects inherit from the built-in JavaScript Object, here is an example:

```
const obj1 = {}; // Object literal

// Using Object constructor
const obj2 = new Object();

// outputs true
console.log(obj1 instanceof Object);

// outputs true
console.log(obj2 instanceof Object);
```

You can clearly see that in spite of both obj1 and obj2 being created in different ways—one as an object literal and the other via the Object() constructor—both still inherit from the JavaScript Object.

Object.prototype

All objects inherit from Object.prototype unless explicitly created otherwise. Object.prototype contains methods like:

- toString()
- hasOwnProperty()
- isPrototypeOf()
- valueOf()

Here are some examples of how you can use these functions with your custom Objects:

```
const myObject = { name: 'John' };

// will output true
console.log(myObject.hasOwnProperty('name'));

// will output "[object Object]"
console.log(myObject.toString());
```


Static Methods of Object

The Object constructor includes static methods for object manipulation.

Common Static Methods:

- `Object.keys()`: Returns an array of keys in the object.
- `Object.values()`: Returns an array of values in the object.
- `Object.entries()`: Returns an array of key-value pairs
- `Object.assign()`: Copies properties from one or more source objects to a target object, or to clone an object.
- `Object.create()`: Creates a new object with a specified prototype.

Here are examples of how these static methods can be used:

```
const obj = { a: 1, b: 2 };

console.log(Object.keys(obj));
// ['a', 'b']
console.log(Object.values(obj)); // [1, 2]
console.log(Object.entries(obj));
// [['a', 1], ['b', 2]]
```

Object.assign() can be used in a couple of different ways

```
// use Object.assign() for copying
// properties between objects

const target = { a: 1 };
const source1 = { b: 2 };
const source2 = { c: 3 };

Object.assign(target, source1, source2);
console.log(target);
// { a: 1, b: 2, c: 3 }

// use Object.assign() to clone an object
const original = { name: 'John', age: 30 };

const clone = Object.assign({}, original);
console.log(clone);
// { name: 'John', age: 30 }
console.log(clone === original);
// false (new reference)

// use Object.assign() to add default properties
const defaults = { name: 'Anonymous', age: 18 };
```

```
const user = { name: 'Alice' };

const completeUser = Object.assign({}, defaults, user);
console.log(completeUser);
// { name: 'Alice', age: 18 }
```

`Object.create()` is also a powerful method and can be used in different ways for creating a new object with a specified prototype.

// Example 1: Creating an Object with a Prototype

```
const prototypeObject = {
  greet() {
    console.log('Hello, ' + this.name);
  },
};

const newObject = Object.create(prototypeObject);
newObject.name = 'John';
newObject.greet(); // "Hello, John"
```

// Example 2: Creating a Null-Prototyped Object

```
const nullPrototypeObject = Object.create(null);
nullPrototypeObject.name = 'John';

// outputs "John"
console.log(nullPrototypeObject.name);

// outputs undefined (no inherited properties)
console.log(nullPrototypeObject.toString);
```

The reason `console.log(nullPrototypeObject.toString);` returns undefined is that `nullPrototypeObject` is created with `Object.create(null)`. If you had created the object using `Object.create()`, it would have created an object with a prototype—meaning the object would inherit properties and methods from `Object.prototype` like `toString()`, `hasOwnProperty()`, etc. In this case it didn't, which is fine if you wanted a prototype-less object. Accessing `toString` returns undefined because the object has no prototype—meaning it does not inherit `toString` from `Object.prototype`. Accessing `name` works because it's a direct property added to the object.

When can you create a prototype-less object? - when you want a truly plain object without extra methods (e.g., for a dictionary or a map-like structure). - When you need to avoid accidental property conflicts with built-in methods.

// Example 3: Initialising Properties

```
const personPrototype = {
```

```

return `${this.name} is ${this.age} years old.`;
  },
};

const person = Object.create(personPrototype, {
  name: { value: 'Alice', writable: true },
  age: { value: 25, writable: true },
});

console.log(person.describe());
// "Alice is 25 years old."

```

Understanding how to use the prototype property

Javascript objects work differently from the typical objects in languages like Java or the C family of languages. They use a unique mechanism known as prototypes. Every object in JavaScript carries an internal link to the object it is inheriting from. This is what JavaScript uses to keep track of what object is inheriting from what. This is also referred to as the prototype chain. The prototype property of an object contains its immediate parent (object it is inheriting from). The prototype property of that parent object in turn contains a reference to its own parent object and so forth until it gets to the top most object of the prototype chain, whose own link contains null to indicate that it does not inherit from any object. In JavaScript this top dog of objects is `Object.prototype`. Again, this prototype-based inheritance mechanism is unique to the JavaScript programming language. The term prototype refers to the object an object is created from. It is the mold, the boilerplate, the blueprint if you like. Hopefully you get the idea. To see the prototype of an object, check the value of the **proto** property like so: `personObject2.proto` assuming that there is a `personObject2` you had created. Alternatively, and more safely, you can check it using the `getPrototypeOf()` method of `Object`, which has been available since ES5 in 2009, like so `Object.getPrototypeOf(new Person())`; So we have established that every object carries a link to its parent, and that at the top of the prototype chain sits `Object.prototype`. But there is a second thing in JavaScript also called "prototype", and it must not be confused with the first. It is specifically available ONLY to functions, so that they can be used to create objects in their capacity as constructor functions. This is the single biggest source of confusion in JavaScript objects, so let us settle it now, before we go any further. There are two different things wearing the same name:

- `.prototype` — a real, visible property that ONLY functions have. It holds the object that instances made from that function will inherit from.
- **proto** — the internal link that EVERY object has, pointing at the object it actually inherits from. (Its proper name in the specification is `[[Prototype]]`). You read it

with `Object.getPrototypeOf()`, and modern browser consoles now show it as `[[Prototype]]` rather than **proto**.)

They are not two kinds of the same thing. They are the two ends of one relationship, and this single line of code shows how they meet:

```
function Person(firstName) {
  this.firstName = firstName;
}

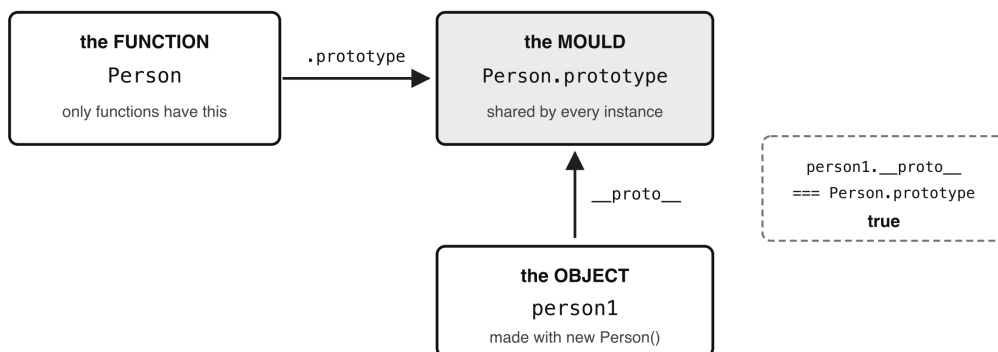
let person1 = new Person('Tom');

// The link on the INSTANCE points
// at the property on the FUNCTION
console.log(person1.__proto__ === Person.prototype);
// true

// And the instance itself has
// no .prototype property at all
console.log(person1.prototype);
// undefined
```

Read those last two lines together and the whole thing falls into place. `Person.prototype` is the mould. `person1.prototype` is the arrow pointing back at the mould it was pressed from. A function owns a mould; an object remembers which mould it came from. Objects do not have a `.prototype` property, and functions do not need a **proto** link to do their job as constructors. For the rest of this chapter, whenever you see the word prototype, ask yourself which of the two is meant. It will nearly always be obvious from whether it is written after a function name (`Person.prototype`) or fetched from an object (`person1.prototype`).

Two different things are called "prototype"



A function owns a mould. An object remembers which mould it came from.

- Figure 12.1 — Two different things are called "prototype"

Such functions are special objects that you create as a developer so that other objects can inherit from. They are known as constructor functions. This distinction is very important to be understood as many developers get it confused. Very uniquely to JavaScript, the syntax of a regular function is very similar to a constructor function which is basically a class. The best way to get a grasp of this is to recognise that JavaScript is a unique language of its own, in terms of classes and objects, and agree with yourself not to try to compare it with any other language. Once you do that, you will find it becomes easier to put things into context and understand. Don't worry, by the end of this chapter you will have mastered the creation of objects in JavaScript. Even in JavaScript, regular functions and objects, as identical as they look, have their distinctions if only you are observant. I will proceed to show you now. To make it easier for you to see the visual differences, I will demonstrate with an object and a function below.

```
function person(firstName, lastName)
{
    return firstName+' '+lastName;
}

function Person(firstName, lastName)
{
    this.firstname = firstName;
    this.lastname = lastName;
}
```

As you can see, regular functions and objects are created in exactly the same way, or so it seems. However, you can still make clear distinctions between the two. First of all, both of these are functions — that is worth saying plainly, because in JavaScript a constructor is a function, not a separate kind of thing. What differs is how you intend to CALL them. The second one is meant to be called with new, which makes it a constructor function; the first is meant to be called normally. You can see that Person sets properties on itself in which it stores the arguments passed to it, so that they can then be used later, while person() just returns a value and keeps nothing. Person uses the this keyword to refer to those properties. Be careful with that last clue, though. It is a hint, not a rule. Ordinary functions can use this too — its value simply depends on how the function was called rather than on what the function is. So seeing this suggests you are probably looking at a constructor or a method, but it does not prove it. Secondly, the . (dot) operator is how you reach members (properties or methods) of an object, so seeing it is another hint. Again it is not proof: JavaScript lets you use the dot on primitives too, wrapping them in a temporary object for you. That is why "hello".length and (5).toFixed(2) both work, even though neither a string nor a number is an object. Lastly, to make a distinction between functions and objects, developers have a

convention of always starting the name of the object with an uppercase letter. This is just a convention and you will certainly still see a lot of constructor function objects spelled beginning in lowercase, but it is a useful practice to follow that convention to make code readable, both for yourself, and your fellow developers. So, going back to the two prototypes we separated earlier: the `.prototype` property belongs only to functions, while the **proto** link belongs to every object. Once you create a constructor function, a `.prototype` property becomes available on it. It holds an object that is very nearly empty — it starts with just one property, `constructor`, pointing back at the function itself, which is where the `constructor` property we look at later comes from. Everything else is yours to add on the fly, and whatever you add becomes available to every inheriting instance. The powerful thing about prototypes and constructor functions is that you can create an object (a constructor function) and have other objects extend from it, then you can use the `prototype` property to add members to your constructor on the fly which will automatically be accessible to all the other inheriting objects. Here is an example:

```
function Person(firstName, lastName)
{
    this.firstname = firstName;
    this.lastname = lastName;

    this.showFirstName = function()
    {
        alert('First name is ' +this.firstname);
    }

    this.showLastName = function()
    {
        alert('Last name is ' +this.lastname);
    }
}

let person1 = new Person('Tom', 'Bands');
let person2 = new Person('John', 'Jones');

person1.showLastName();
person2.showLastName();

// let's add a new method showFullName()
Person.prototype.showFullName = function()
{
    alert('Full name is ' +this.firstname+' '+this.lastname);
}

person1.showFullName();
person2.showFullName();
```

The above code works, and displays the following: Last name is Bands Last name is Jones

Full name is Tom Bands Full name is John Jones

You see how by having one object you can have multiple instances inherit from it, as well as add functionality to the blueprint (the boilerplate, the template, the prototype), which will make all other inheriting instances automatically get access to the newly added feature. Think of a huge tech giant like BMW producing a version of the car say i3. There are going to be millions of customers worldwide purchasing units of this vehicle, and so they cannot afford to be recreating the exact same car type from scratch a copy every time someone orders for a piece. That will be counter-productive and very expensive. Their work will also be prone to errors, as an artist or engineer being human, cannot guarantee that some product or part they made following certain steps can be accurately identically replicated. Rather, what they do is automate the production of the first prototype (aka blueprint, boilerplate, template) from which thousands can be remade effortlessly. They have to be able to vary some properties of the specimen, like the colour, size etc to fulfil the custom demand of specific clients. To achieve this; they will have a way to change certain parameters being passed to whatever automation system is being used to churn out those vehicles. The parameters (arguments) passed will make sure the variable aspects not related to the core functionality of the cars are taken care of; like, the colour, the type of rims for the wheels, etc etc In the same way, when we generate new instances of objects from our blueprint Person constructor function, we just make sure it is a different person every time by passing in arguments of firstName and lastName to the constructor e.g.

```
let person1 = new Person('Tom', 'Bands');
```

Note that when creating a new instance from your object, you have to use the keyword new

The traditional two ways of creating an object and their prototypes

There are three main ways of creating an object in modern JavaScript, but before we go into the new way of creating objects, we will first of all, talk about the two traditional ways to do so. As we demonstrate the creation of objects, we will explain how the concept of prototypes underpins it all. This is because in JavaScript, you can not fully understand object creation and instantiation if you do not understand the concept of prototypes and how it relates to object creation. Here are two traditional ways to create objects, and make no mistake about the term traditional or old; for they are still meant for everyday use and work perfectly fine.

i) Initialiser objects also known as object literals.

ii) Constructor objects sometimes referred to as object templates or blueprints.

i) Initialiser objects Initialiser objects are the simplest form of object creation. The syntax is simple and you basically create a block and assign it to a variable.

```
let obj = {}
```

In this object you can assign members. Initialiser objects are useful when you are manipulating data somewhere in code and you need to have some quick data access object. It is almost a temporary way of storing data that you need to use there and then, and which you are sure you would not need anywhere else in your code. This means in such a case, there is no need to create a constructor object. It is a very handy way to create an object on the fly for direct use. This is also often referred to as an object literal. The syntax goes like this:

```
let person = {  
  name: 'Steve',  
  age: 10  
}
```

If you think it looks exactly like an associative array, then you are absolutely right. That is what it is. You can add more values to it as you go along like so:

```
person.surname = 'Johnson';
```

You can access the values like so:

```
document.body.innerHTML = 'His surname is: ' + person.surname;
```

This prints out 'His surname is Johnson' The built-in Object sits at the top of JavaScript's object world, and its prototype, Object.prototype, is what all inbuilt JavaScript objects as well as the ones you create ultimately inherit from — unless you choose to create an object that does not inherit from it—which is possible, as we saw above. The prototype of an object literal is Object.prototype. Let us prove this. Consider the following code:

```
let piero = {};  
console.log(piero);
```

If you check your console, you will see {} and if you click on the arrow next to it, you will see the property of the object called **proto** that denotes the prototype (parent) of all objects, and its value in this case will be Object:

proto: Object

You can have an object literal that contains properties and methods, and any of its properties may in turn contain an object (sub object). You would set or access these sub objects using a loop. You can create an array of objects on the fly, and in

a loop, access the properties of each one of them. For example, consider the following code:

```
let collection = [];
let colours = ['red', 'orange', 'blue'];
for(let i = 0; i < colours.length; i++)
{
    let temporal = {}
    temporal.name = colours[i];
    collection.push(temporal);
}

// note the += on each line. A
// plain = would REPLACE what is
// already there, so only the last line would ever be seen
document.body.innerHTML += 'HERE IS THE collection: '+collection+'<br>';
document.body.innerHTML += 'FIRST OBJECT IN collection:
'+collection[0].name+'<br>';
document.body.innerHTML += 'SECOND OBJECT IN collection:
'+collection[1].name+'<br>';
document.body.innerHTML += 'THIRD OBJECT IN collection:
'+collection[2].name;
```

The output will be: HERE IS THE collection: [object Object],[object Object],[object Object] FIRST OBJECT IN collection: red SECOND OBJECT IN collection: orange THIRD OBJECT IN collection: blue

So we have three objects in the collection array, and each of these objects has a name property whose value is the colour that this object represents. Creating object literals like this is the equivalent of creating an empty object using the new Object() method in JavaScript. The prototype of this new object created using new Object() is Object.prototype, exactly the same as for object literals.

ii) Constructor functions Constructor objects as we have seen above are like a boilerplate from which you expect to generate other unique but related objects. You do it when you expect to need multiple objects; possibly in several different places in your application. You create an instance of a constructor function using the keyword new and the class name like so:

```
let student = new Student();
```

The new keyword makes sense because you are creating a new instance (copy) of the boilerplate object. So, without any further explanations (we already explained them well above), let me demonstrate with an example of how you would use a constructor object to manage the customised display of the portrait of four different members of the same family; a girl, her brother, a mum and a dad.

```

<html>
<body>
<div id='girl'></div>
<div id='boy'></div>
<div id='mum'></div>
<div id='dad'></div>

<script type='text/javascript'>

function PersonDiv(backgroundColor, id)
{
    this.backgroundColor = backgroundColor;
    this.id = id;
    this.placeImage = function() {
        let div = document.getElementById(this.id);
        div.style.backgroundColor = this.backgroundColor;

        if (this.id == 'girl')
        {
            let img = "<img src='girlImg.png' />";
            div.innerHTML = img;
        }

        if (this.id == 'boy')
        {
            let img = "<img src='boyImg.png' />";
            div.innerHTML = img;
        }

        if (this.id == 'mum')
        {
            let img = "<img src='mumImg.png' />";
            div.innerHTML = img;
        }

        if (this.id == 'dad')
        {
            let img = "<img src='dadImg.png' />";
            div.innerHTML = img;
        }

    }
}

let girl = new PersonDiv('pink', 'girl');
girl.placeImage();

let boy = new PersonDiv('blue', 'boy');
boy.placeImage();

```

```

let dad = new PersonDiv('green', 'dad');
dad.placeImage();

</script>
</body>
</html>

```

In this example, we instantiate objects from the constructor object `PersonDiv()` and depending on if it is a girl, we grab the div from the DOM having the ID of 'girl' and we give the div a background colour of pink, and we place the image of the girl in it. If it is a boy, we grab the div from the DOM with the ID of 'boy' and give it a blue background color and so on for the 'mum' and 'dad' divs as well. Once more, we use the new keyword to create objects from constructor functions, but it would also work on JavaScript's built-in constructor objects like `Object`, `Array`, `String`, `Date`, `RegExp` etc. You would instantiate objects from these like so:

```
new Object(), new Array(), new Date(), new RegExp() etc
```

When it comes to prototypes and objects created from a constructor function, it is important to get the relationship exactly right, because it is easy to state it one step out. The object you get back from `new Person()` does NOT have the `Person` function as its prototype. It has `Person.prototype` — the object hanging off the function, which is the mould we talked about earlier. So for `let person1 = new Person('Tom')`, the chain runs like this:

```
person1 → Person.prototype → Object.prototype → null
```

You can walk it yourself and watch it end:

```

console.log(Object.getPrototypeOf(person1) === Person.prototype);
// true

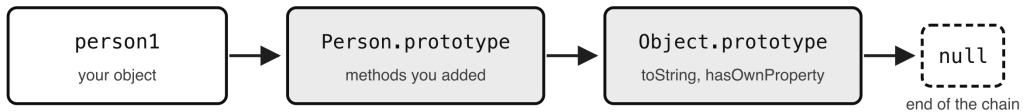
console.log(Object.getPrototypeOf(Person.prototype) === Object.prototype);
// true

console.log(Object.getPrototypeOf(Object.prototype));
// null – the end of the chain

```

Notice that an object has exactly ONE prototype, never two. When you hear that a `Date` "inherits from both `Date.prototype` and `Object.prototype`", what is really meant is that both of them are in its chain — `Date.prototype` directly, and `Object.prototype` one step further up. That distinction matters, because inheritance in JavaScript is a single line of links, not a bundle of parents. This linking or association of objects, as we already know, is what we call the prototype chain, and it is how JavaScript answers every property lookup: it checks the object itself, then its prototype, then its prototype's prototype, until it either finds what you asked for or reaches null.

The prototype chain: a single line of links



Ask for a property and JavaScript walks left to right until it finds it, or reaches null.

Each object has exactly ONE prototype — never two. Both boxes above are in person1's chain, but only Person.prototype is its prototype.

- Figure 12.2 — The prototype chain: a single line of links*

iii) The new way to create objects and the prototype There is a more modern way to create an object in JavaScript. Since ES5, released in 2009, you can create an object using the `create()` method of the JavaScript Object passing it the name of the object's prototype (the object you want to create an instance of). Here is how you would do it:

```
let girl = Object.create(Person);
```

Here we are creating an object `girl` from another object called `Person`. The `Person` object will be the prototype of the `girl` object. If you need to pass arguments into the constructor of the prototype object you want to create an instance of, you have to pass an object as a second argument to the `create()` method. Each key in this object is the variable name, and the value of the key should be an object whose key is the keyword 'value' and its value the value of that variable. This probably sounds more complicated than it actually is so let me demonstrate with a simple example. Imagine we have the following constructor object called `Person` and it looks like this:

```
function Person(fname, lname) {  
    this.firstName = fname;  
    this.lastName = lname;  
}
```

This class or object declares a constructor that needs you to pass in `fname` and `lname` arguments when you instantiate it, which will be used for the `firstName` and `lastName` properties of the new person to be created. Do not forget to use the `new` keyword as you instantiate the `Person` class (create an object from it), as we have seen already. Here's how to do it:

```
let girl = new Person('Joan', 'Langley');
```

Then

```
document.body.innerHTML =  
    girl.firstName+' '+girl.lastName;
```

will display Joan Langley

Let's see how you would use the create() method to create objects from the Person constructor object. In doing that, we will pass in the arguments that the constructor object needs to set itself up.

```
let girl = Object.create(Object.create(Person, { firstName: {value:  
'Josephine'}, lastName: { value: 'Kindjo'} }));  
let boy = Object.create(Object.create(Person, { firstName: {value: 'Jim'},  
lastName: { value: 'Kindjo'} }));  
let mum = Object.create(Object.create(Person, { firstName: {value:  
'Natasha'}, lastName: { value: 'Kindjo'} }));  
let dad = Object.create(Object.create(Person, { firstName: {value:  
'William'}, lastName: { value: 'Kindjo'} }));  
  
let family = [];  
family.push(girl);  
family.push(boy);  
family.push(mum);  
family.push(dad);  
  
for (i = 0; i < family.length; i++)  
{  
    document.body.innerHTML = family[i].firstName + ' ' + family[i].lastName  
    + '<br>';  
}
```

This will print out on screen the following:

Josephine Kindjo Jim Kindjo Natasha Kindjo William Kindjo

Let me explain the above code; we start by instantiating the Person constructor object four different times, passing into its constructor each time a different person-a family member. Each person has a different firstname and last name. Then to prove that it worked and has created four separate objects; we place them all into all an array and then loop though this array printing out their first and last names on screen.

Obects and their prototypes We have already discussed how the concept of prototypes is unique to JavaScript. Once more, a prototype is the object from which another is created.

```
let gusto = Object.create(Person);  
let juanita = Object.create(gusto);  
  
console.log(juanita);
```

The prototype of `juanita` is a blank constructor function which contains another constructor function by the name of `Person`. The first blank constructor function represents `gusto`, and the second one is the `Person` constructor function that is the parent of them all. So `juanita` is two levels down the prototype chain. The following command reveals the topmost constructor function:

```
console.log(juanita.prototype.constructor);
```

It prints out the whole `Person` object like so:

```
function Person(fname, lname) {  
    this.firstName = fname;  
    this.lastName = lname;  
}
```

The `Object.create()` method therefore has to be passed an argument which is the object to be used as the prototype of the new object to be created. But you can also create a blank or empty object whose prototype is the JavaScript Object, just as is the case when creating an object using `{}`, or `new Object()` which do not need to be passed any previously created object. To do so, just pass to `Object.create()` the `Object.prototype`. For example:

```
let chi = Object.create(Object.prototype);  
console.log(chi);
```

The prototype of `chi` is `Object()`, JavaScript's topmost parent object. Therefore, as a reminder again; creating an object in any of the following ways will result in a new object with JavaScript's `Object` as their prototype:

```
let obj = {};  
let obj = new Object();  
let obj = Object.create(Object.prototype);
```

On the other hand, it is also possible to create an object without a prototype. Here is how:

```
let obj = Object.create();  
let obj = Object.create(null);
```

So, if you just do `Object.create()` without passing it the argument of JavaScript's parent object, you will end up with a really empty object that is void of all the basic built-in properties and methods that come with JavaScript's constructor objects out of the box. Such an object can also be referred to as a prototype-less object, and it will probably not be of much use to you.

Handy properties and methods of Objects

This in other words refers to the properties and methods of the JavaScript parent of all objects, `Object`. All the methods and properties below can be tested by referencing or calling them on `Object`, like so:

`Object.propertyName` OR `Object.methodName`

Take for instance, our good old `Person` class again.

```
function Person(fname, lname) {
    this.firstName = fname;
    this.lastName = lname;
}

let gusto = Object.create(Person);
let juanita = Object.create(gusto);
```

-1) The prototype property Here is how you can use the prototype property on the fly to add members to an object and have all instances automatically have access to them. Because properties most likely will be different for each instance-which is really the point and power of classes; it is recommended to use the prototype property only to add methods, not properties. Depending on how the object instances are created from the constructor function-either using the `new` keyword, or using `Object.create()`, you would add methods to the prototype in slightly different ways. Pay attention to this. Here is an example:

If you created instances from the prototype using the `new` keyword, for example:

```
let aunt = new Person('Audrey', 'Kindjo');
```

Then you would just add your new method on to the prototype property of the `Person` object (constructor function) as follows:

```
Person.prototype.showFullName = function()
{
    return this.firstName+' '+this.lastName;
}
```

Then you can have any of your `Person` instances use the new methods like this:

```
alert('The full name of the aunt is '+aunt.showFullName());
```

But if you created instances from the prototype using the `Object.create()` method, for example:

```
let sister = Object.create(Object.create(Person,
    {
        firstName: {value: 'Janet'},
```

```
lastName: { value: 'Kindjo'}
    }));
```

Then you would need to add your new method onto the prototype property of the constructor property of the Person object (constructor function) as follows:

```
Person.constructor.prototype.showFullName = function()
{
    return this.firstName+' '+this.lastName;
}

alert('The full name of the sister is '+sister.showFullName());
```

This is because, unlike when using the new keyword in creating an instance of a constructor object where the new object's prototype is the prototype of the constructor function, when creating an instance using the Object.create(), a constructor property is assigned to the blueprint constructor function, and it is this constructor property that will be used as the prototype of objects created from it. This brings us to the next property, the constructor property.

-2) The constructor property To rephrase the last statement above; objects created from a constructor function using the Object.create() method have the constructor property of their parent constructor function as their prototype. Let us demonstrate this.

```
let gusto = Object.create(Person);
```

Here is one way we can get information on the full structure of the prototype of an object created from a constructor function:

```
console.log(gusto);
```

This displays in the log the Person object. This reveals that the prototype of gusto is a constructor function by the name of Person. You can tell because the value of the **proto** property is a Person function. You can further verify this by doing this:

```
console.log(gusto.prototype);
```

The result in the console will be a constructor object. You can rig down further to find out what this constructor object is by running the following: console.log(gusto.prototype.constructor); The result in console will be

```
function Person(fname, lname) {
    this.firstName = fname;
    this.lastName = lname;
}
console.log(gusto.prototype.constructor.name);
```

This will write the object name Person to the console.

By observing that, I am sure you would agree with me that the prototype property of a constructor function is therefore not always a reliable way of determining an objects' prototype. This is obviously because, as we have seen above, the prototype of an object is different depending on the blueprint (parent object) it is created from. By the way, just like objects created with the `Object.create()` inherit a constructor property which will contain the prototype of the new object being created, object literals also have a constructor property. However, while the `constructor.prototype` of an object literal will always point to the correct prototype of the object literal, it may not always do so for objects created using the `Object.create()` method. This is because each time `Object.create()` is used, it may be used to create an object from a constructor function, or from one of JavaScript's parent objects, which means that in some instances, to verify the prototype of an object, you have to check on the prototype property, while in others, you have to check on the `constructor.prototype` property. The most efficient way to determine the true prototype of an object, or determine the prototype chain it belongs to is to use the `isPrototypeOf()` method.

- 3. `isPrototypeOf()` and the `instanceof` operator It is used to verify if an object is the prototype of another object. Here is the syntax:
`objectParent.isPrototypeOf(objectName)`

```
let obj1 = {x:1}

//check if the prototype of obj1 is Object()
// the result is true
Object.prototype.isPrototypeOf(obj1);

//further create an object from obj1
let obj2 = Object.create(obj1);

// check if the prototype of obj2 is obj1
// the result is true
obj1.isPrototypeOf(obj2);
```

Note that `isPrototypeOf()` works in exactly the same way as the `instanceof` operator, but is even more effective. It is more effective in that it recognises grand children of objects whereas `instanceof` will fail if an object is not a direct child of another. For example:

```
var superProto = {
  // some super properties
}

var subProto = Object.create(superProto);
subProto.someProp = 5;
```

```

console.log(superProto.isPrototypeOf(sub));
// true
console.log(sub instanceof superProto);
// TypeError

```

The phrase constructor function is used to refer to the name of the class that many objects are to be created (constructed) from. Once more, take for example our Person class inheritance example above:

```

function Person(fname, lname) {
    this.firstName = fname;
    this.lastName = lname;
}
let gusto = Object.create(Person);
let juanita = Object.create(gusto);

//this returns true
console.log('Person is the prototype of gusto: '
    + Person.isPrototypeOf(gusto));

//this returns true
console.log('Person is the prototype of juanita: '
    + Person.isPrototypeOf(juanita));

//this returns true
console.log('Object.prototype is the prototype of juanita: '
    + Object.prototype.isPrototypeOf(juanita));

//this returns false
console.log('juanita is the prototype of gusto: '
    + juanita.isPrototypeOf(gusto));

```

- 4. `getPrototypeOf()`

It is used to get the prototype of an object. Since ES5 you can check an object's prototype using the `getPrototypeOf()` on `Object`. Prior to ES5, there was no equivalent way to do that, though you could determine the prototype of an object by checking the prototype property of its constructor property as we have seen already:

```
objectName.constructor.prototype
```

Just to recap again; that was possible because all objects created with the new keyword, or from constructor functions inherit a constructor property which points to the constructor function that is their prototype. Object literals also have a constructor property whose prototype property points to the JavaScript Object. Here is the syntax e.g.

```
Object.getPrototypeOf(objectName)
```

```
//get the prototype – it reveals the Person object  
console.log(Object.getPrototypeOf(gusto));
```

- 5. The **proto** property

You can also access the prototype property of an object via the old **proto** property that is still supported by most browsers like so:

```
let chi = Object.create(Object.prototype);  
console.log(chi.__proto__);
```

This code displays all the properties and methods that the object `chi` has inherited from the JavaScript's `Object`. By the way, again, this is how you should instantiate an object whose prototype you want to be `Object` object. You pass to `Object.create()` the keyword `Object.prototype`, hence

```
let chi = Object.create(Object.prototype);
```

- 6. The `getOwnPropertyNames()` method

This is a method that displays the properties of an object. Use it like so:

```
let employee = new Person('Jude', "Chairman");  
console.log('The props of employee are:  
' + Object.getOwnPropertyNames(employee));
```

This writes to the console 'The props of employee are: firstName,lastName'

- 7. The `hasOwnProperty()` method (non-inherited)

This is a built-in method that comes from `Object.prototype`, and it's used to check whether an object has a specific own (non-inherited) property. It's used like this:

```
object.hasOwnProperty("propertyName")
```

Here is an example:

```
const person = {  
  name: "Alice",  
  age: 25  
};  
  
// will return true  
console.log(person.hasOwnProperty("name"));  
  
// returns false (because toString  
// is inherited from prototype)  
console.log(person.hasOwnProperty("toString"));
```

- 8. Check all properties of an object (including inherited)

If you want to check if a property exists anywhere in the object (including its prototype chain), use the `in` operator:

```
// returns true – inherited
console.log("toString" in person);
```

Creating a custom method on an object

A method is a function that is a member of an object, as opposed to a stand-alone function in a program. There are two ways to create a method. One way is an older way using the function keyword, while another, simpler new way allows you to do so without using the function keyword. Here is the older way:

```
const myObject = {
  property1 : 1,

  objFunc : function(arg) {
    this.property1 = arg;
  }
}

myObject.objFunc('TestValue');
console.log(myObject.property1);
```

The result of this is that the text 'TestValue' will be written to the console.

The following is the new way introduced with JavaScript ES6 (ECMAScript 2015). Notice it eliminates the use of the colon to mark the member as a property of the class, and also the function keyword:

```
const myObject = {
  property1 : 1,

  objFunc (arg) {
    this.property1 = arg;
  }
}

myObject.objFunc('TestValue');
console.log(myObject.property1);
```

The result of this is exactly the same as in the previous example above- 'TestValue' will be written to the console.

The call(), apply() and bind() methods of Function

In JavaScript, functions are objects, as we have learned so far. This means every function inherits from `Function.prototype`, and the methods living there are therefore available on every function you write. With that said, there are three very important functions you should know; and these are the `call()`, `apply()`, and `bind()` functions. They are special because they serve a specific purpose. Understanding how they work will take your JavaScript mastery to a deeper level. So, the three methods `call()`, `apply()` and `bind()` are methods of the function object. In other words, these are methods that JavaScript has provided you with to use on functions. You can therefore call them on any function (both built-in ones and your own) and it will work. These functions solve a very pertinent problem when it comes to functions, and that is the problem of resolving the context in which a function is called. When it comes to objects, context refers to the current object on which a function is being run at any point in time. This current object is represented by the `this` keyword.

In the flow of your program, your objects will often interact with one another and you will invariably find yourself needing to call functions or methods of other objects outside of the object in which you are operating for various reasons. One reason could be to extend the functionality of the object you are in by making it do something it cannot do, or utilise (re-use) a feature already offered by another function rather than rewriting it all over again. It is generally known in the programming world that code should be re-used wherever possible in order to avoid code duplication.

With that said; when calling one object from within another object, or in other words; when a method of an object is being passed to another function as a callback (which essentially means calling another/external function inside a function), the `this` keyword is lost. This is because the function has been separated from the object it belongs to, so JavaScript no longer has anything to point the `'this'` keyword at. What happens then depends on the mode you are running in. If your JavaScript is in strict mode, you will get an error of undefined, and in non-strict mode, the value of `'this'` will default back to the global object. Either way, `'this'` will not represent the object you wanted, and the usual symptom is a value coming back with the error undefined, or a "Cannot read properties of undefined". The three methods `call()`, `apply()` and `bind()` make it possible for you to tell JavaScript which object has the context, or better put; which of the objects should be referenced by `'this'`.

Let us start by looking at `call()` and `apply()` as they are used in exactly the same way apart from a slight difference in their syntax. To call a function of another object

B from inside another object A as if the method of B is a method of object A, you can use the `call()` method. To do so, while within object A, you call the function of the external class B, followed by the keyword `.call()`, and pass the object you want to be referenced by 'this' as the first argument to the `call()` function. Here is an example of two objects Car and MotorBike:

```
function Car(name, fuel)
{
    this.name = name;
    this.fuel = fuel;

    this.getName = function()
    {
        return this.name;
    }

    this.getType = function()
    {
        return this.name+'-'+this.fuel;
    }

    this.multiply = function(first, second)
    {
        return (first * second);
    }
}

function Motorbike(name, fuel)
{
    this.name = name;
    this.fuel = fuel;

    this.getName = function()
    {
        return this.name;
    }
}
```

Both objects have the same properties name and fuel, though their values will be different. They both also have in common a method called `getName()` which returns the value of their name property.

However, the Car object has two methods that the Motorbike object does not have, and these are `getType()`, and the `multiply()`. The `getType()` method returns the name and fuel type of a car in a nice format which is the name property joined to a hyphen and joined to its fuel property.

```
    this.getType = function()
```

```
{
    return this.name+'-'+this.fuel;
}
```

The other method `multiply()` does not make sense in the context of this example of automobiles, but I included it as an example of how you would call a method of an object that takes arguments from another object.

```
this.multiply = function(first, second)

{
    return (first * second);
}
```

Now let's see how you can use `call()` and `apply()` to call the method of another (external) class, and even pass arguments to the method.

```
let car = new Car('Mercedes', 'petrol');

let bike = new Motorbike('Harley Davidson', 'petrol');

alert(car.getName());
```

This will display a popup that says 'Mercedes'

```
alert(bike.getName());
```

This will display a popup that says 'Harley Davidson'

Now pay attention, and see how we can call a method of a class, and call `call()` or `apply()` on that method, passing the class we want to be referenced by 'this' within that method we are calling as the argument. This may sound confusing at first, but it really is not once you get it. Understanding it is simple once you pay attention to it. It is the kind of knowledge that separates an expert from a novice JavaScript learner. It's best to see it in action, so let's call the `getName()` method of the Bike function (object), and use `call` to make the `getName()` method of Bike refer to the Car class instead.

```
let car = new Car('Mercedes', 'petrol');
let bike = new Motorbike('Harley Davidson', 'petrol');

alert(bike.getName.call(car));
```

If you run this code, instead of the popup saying 'Harley Davidson' as you would expect because the Motorbike's `getName()` method returns its own name, it will say:

'Mercedes'

I hope you can now see why. We didn't just do `bike.getName()` but we used `call()` on it too like this: `bike.getName.call(car)`. That `car` argument passed to `call()` is key.

It is what tells `getName()` that 'this' (which is what it uses internally to reference its own name property) should be the Car object, not itself

```
this.getName = function()
{
    return this.name;
}
```

Let's do the same thing by calling the `getName()` of the Car function, but this time using `apply()`, which does the same thing as `call()`:

```
alert(car.getName.apply(bike));
```

Now you already know what the `alert()` popup will display when you run this code. Instead of displaying 'Mercedes' which `car.getName()` does, it will display:

```
'Harley Davidson'
```

which is the name property of the Bike object (function). We passed bike to `apply()` to tell the `getName()` method of the car object that 'this' refers to the bike object and not itself.

Similarly;

```
alert(car.getType());
```

will display a popup that says:

```
'Mercedes-petrol'
```

while this:

```
alert(car.getType.call(bike));
```

will display a popup that says:

```
'Harley Davidson-petrol'
```

This is like magic, don't you agree? It's exciting stuff. We basically call `getType()` on the car object, but because of `call()`, it's instead the `getType()` of the bike object that ends up being called behind the scenes, and it's all because we slapped `call()` on the call chain, and passed it which object we wanted to become 'this'. I think you've got it now, so let's proceed.

Passing arguments when using `call()` and `apply()`

What if the method of the external class you are calling needs arguments to be passed in? These two functions do exactly the same thing, with the only difference found in how their arguments are passed. Let's see how you would pass arguments to the method you are calling on the other object, if it requires arguments. We will

use the `multiply()` method on the `Car` object which accepts two arguments for its parameters `first` and `second`. Here is the method again:

```
function Car(name, fuel)
{
    ...
    this.multiply = function(first, second)
    {
        return (first * second);
    }
}
```

Let's do a quick recap before we dive in. So, we already know that if we call `multiply()` on the `bike` object, it would not work, because `multiply()` is a method defined on the `Car` object and not the `Motorbike` object. So this will throw an error in the console saying `'bike.multiply is not a function'`:

```
alert(bike.multiply(2,2));
```

while this will run correctly with no errors, and display 4, since `multiply()` returns the product of its two parameters:

```
alert(car.multiply.call(bike, 2, 2));
```

Notice how you can pass in as many arguments you want, from the second argument of `call()` onwards, separated by commas if there are multiple. These arguments are destined for the method that will be called, whatever it will be. If you are wondering why the above code did not throw an error, then you are observant. we used `call()` on `car.multiply`, like so:

```
car.multiply.call(...)
```

which means the JavaScript parser/interpreter should have tried to call a `multiply()` method on the `bike` object, and not the `car` object-especially as we used `call()` and passed it the context of `bike` (`call(bike, 2, 2)`).

Here is the catch, and why it did not error; it never called, or attempted to call `multiply()` on the `bike` object. This is because, if you look at the `multiply()` method of `car` again, you will see that there is no use of the `'this'` keyword anywhere inside it.

```
function Car(name, fuel)
{
    ...
    this.multiply = function(first, second)
    {
        return (first * second);
    }
}
```

In that case `call()` always hands over the arguments you give it to the method that will be called - whatever it will be; which in this case is `multiply()` of `car`. To be precise about what happens: `call()` does set `'this'` to `bike`, exactly as you asked it to. It is `multiply()` that never looks at `'this'`, so the setting makes no difference to the result. The function that runs is still the one you reached for, `car's multiply()`.

So, we now know that if an object's method does not use the `'this'` keyword inside of itself, there is no need to call it using `call()`, `apply()` or `bind()`. We will learn about `bind()` in a second. Now that is out of the way, let us do the same thing, only this time we will actually call a `multiply()` method on the `Motorbike` object so that it is actually an external method call, and also, this time we will also use `apply()` instead of `call()`, so that we see that they are all the same, except for a syntax difference when arguments need to be passed in.

Create two new methods; one `multiply()` on `Motorbike`, and one `resolve()` on the `Car` object. The `multiply()` on `Motorbike` will mirror the `multiply()` already in `Car`. However, let's make the `Motorbike` object's own `multiply()` do a division of its two arguments internally and return the results to make it clearly different from the multiplication result of `Car.multiply()`. Here is the final code of `Car` with its new `resolve()` method:

```
function Car(name, fuel)
{
    this.name = name;
    this.fuel = fuel;

    this.getName = function()
    {
        return this.name;
    }

    this.getType = function()
    {
        return this.name+'-'+this.fuel;
    }

    this.multiply = function(first, second)
    {
        return (first * second);
    }

    this.resolve = function(first, second)
    {
        return this.multiply(first, second);
        //return (first * second);
    }
}
```

Here is the full Motorbike object with its new multiply() method added:

```
function Motorbike(name, fuel)
{
    this.name = name;
    this.fuel = fuel;

    this.getName = function()
    {
        return this.name;
    }

    this.multiply = function(first, second)
    {
        return (first / second);
    }
}
```

Notice how we have structured the code here. The multiply did not use 'this' as we know already, but we have made sure its new resolve() method uses 'this' to reference its own multiply() method. Then the new multiply() of the Motorbike class mirrors that of car in name, so that we can get the resolve() method to call either one based on the context we give it, and we will be able to tell from the output which method of which class was evoked. Here is a demonstration of how it works in code. Obviously, we need to start by building the objects, so let's do that:

```
let car = new Car('Mercedes', 'petrol');
let bike = new Motorbike('Harley Davidson', 'petrol');
```

Next, let's first of all call the resolve() method of Car:

```
alert(car.resolve(2, 2));
```

The output is 4, and it is straight forward; Car's resolve() makes a direct call to its multiply()

```
alert(car.resolve.call(bike, 2, 2));
```

Here the output in the alert popup is 1, which is correct. Motorbike's multiply() is run because we used call(bike) and gave it the context of bike.

```
alert(car.resolve.apply(bike, [2, 2])); // with array
```

Here the output in the alert popup is 1 which is also correct. Motorbike's multiply() is run since we used apply(bike) and gave it the context of the bike object.

Notice the difference between how we pass arguments to call() and how we pass it to apply(). Both of them take the first argument to be the context of 'this'. While call() takes multiple arguments destined for the method to be called as comma-

uments as its second parameter. This is the only difference between them, otherwise they all work the same way.

The bind() function

Now that we have mastered the use of call() and apply(), let us look at the bind() function which, though also used to set the context ('this' keyword) of a function call, is also used in a slightly different way from the other two. The main difference is that unlike call() and apply(), which run the function immediately and give you the result, bind() prepares and returns a new function with the context ('this' keyword) already set, that you can call whenever you need it. Take the following example:

```
let person = {
  name: 'John Doe',
  getName: function() {
    console.log(this.name);
  }
};

setTimeout(person.getName, 1000);
```

The output of this code will be:

undefined

So, person.getName() returns 'undefined' instead of 'John Doe'.

That's because setTimeout() received the function person.getName outside of (separately from) the person object. The getName() method has been passed as a callback function to the setTimeout() function, so the 'this' reference which is used in getName() will not be the same inside setTimeout(). While inside setTimeout(), getName() is not inside its original object, so 'this' will then refer to the global Object instead. Actually, 'this' will be the global Object if you are running JavaScript in non-strict mode, but in strict mode, it will be undefined. Either way, the value of 'this' as you expected it (to reference the person object) is lost. Hence when setTimeout() runs, and person.getName() gets invoked, the name is not found on the global object, or it comes back as undefined, either way, it fails. There are two ways to fix this issue;

Either you wrap the call to person.getName() inside another anonymous function, or use the bind() method.

1) Using an anonymous function

```
setTimeout(function () {
  person.getName();
}, 1000);
```

This will work because the anonymous function will get the person object from the outer scope and then call the method `getName()` on it.

2. Using the `bind()` method

You first of all prepare or bind the method `getName()` outside of the person object before you proceed to use it outside of the person object. Once you bind it (with the `bind()` function), `bind()` will automatically set the context for you, and you can then safely use it. For example:

```
let p = person.getName.bind(person);
setTimeout(p, 1000);
```

What if just like we did with `call()` and `apply()`, you want to use `bind()` to make an object have some extra functionality by calling a method from another object which it does not have. This is also easy to do, though it's done slightly differently from the way `call()` and `apply()` do it. The following example is how you can make the `Motorbike` object borrow and make use of the `getType()` and the `multiply()` methods of the `Car` object.

```
let car = new Car('Mercedes', 'petrol');
let bike = new Motorbike('Harley Davidson', 'petrol');
let getType = car.getType.bind(bike);
alert(getType());
```

This will display a popup that says 'Harley Davidson-petrol'

Notice that unlike with the `call()` and `apply()` methods, when binding, we are kind of like creating a stand-alone function separate from the object it belongs to, which we can just call as a standalone whenever we need it, and it will automatically already know what context, and what arguments to use. For example when we just called `getType()` above and did not have to do anything else.

Let's create such a standalone function from the `resolve()` method of the `Car` object, then bind it to the `Motorbike` object so that it's the `Motorbike` method that will be called instead of a method of `Car`, just as we have seen being done using `call()` and `apply()` above. Let's name this function `divide`:

```
let car = new Car('Mercedes', 'petrol');
let bike = new Motorbike('Harley Davidson', 'petrol');

let divide = car.resolve.bind(bike, 6, 3);
alert(divide());
```

This will display a popup with 2

That is the output of dividing 6 by 3, which is what the `multiply()` method in `Motorbike` object does, if you remember-it actually divides unlike the `multiply()` of `Car` that multiplies. That is proof that `bike.multiply()` was called and not `car.multiply()`

Notice that just like with the `call()` function, if the method being called with `bind` needs arguments, the arguments are passed to `bind()` after the first argument as comma-separated values.

In a nutshell, the `bind()` function allows an object to borrow a method from another object without making a copy of that method. This is known as function borrowing in JavaScript, and it's very powerful because it broadens the capabilities of your classes and objects while promoting code reuse.

Losing "this", and giving it back

The only difference is whether the owner travels with the function.

WITHOUT its owner



WITH its owner



Three ways to send the owner along

wrap	<code>setTimeout(function () { person.getName(); }, 1000)</code>	
bind	<code>person.getName.bind(person)</code>	— hands back a function to run later
call	<code>person.getName.call(person)</code>	— runs it straight away (apply takes an array)

Figure 12.3 — Losing "this", and giving it back

OBJECTS IN DEPTH

Object literals

This is a simple version of an object in JavaScript, which actually has the same syntax as an associative array. It is written as a pair of opening and closing curly braces with its properties written as key-value pairs. Here is an example:

```
let obj = {
  msg: "Hello",
  title: "Welcome",
  footer: "Good bye"
};
```

The above object is called `obj`, and it has three properties; `msg`, `title`, and `footer`.

The keys are on the left while their values are on the right. Both keys and values are separated by a colon. The values can vary in data type. Each key-value entry pair is separated by a comma though the last item does not have to have a trailing comma.

JavaScript objects vs object literals

In JavaScript, objects created using a specific syntax are often referred to as object literals because they are defined using literal notation directly within the code. This contrasts with creating objects using constructors or class-based approaches.

Why the term “object literal”?

The term “literal” in programming refers to representing a value directly in code, rather than creating it dynamically or through a function or constructor. When you define an object directly by specifying its key-value pairs in braces {}, you are using object literal syntax. Here is an example of an object literal:

```
let person = {  
  name: 'John',  
  age: 30  
};
```

In the example above, the object is created directly using the {} syntax, making it a literal expression. There is no need to invoke a constructor or function to build or replicate an instance of this object. Here are the key characteristics of object literals:

- i) Direct declaration. The object is declared in-line, which is efficient and simple.
- ii) Object literals are highly readable and concise, allowing for quick creation of key-value pairs.
- iii) No Constructor Needed: You don't have to use a new Object() syntax (which was more common in older JavaScript).

Object literals vs constructor function

In contrast, here's how an object could be created using a constructor function:

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
let person = new Person('John', 30);
```

In this example, you use a constructor function to create an object. The object is created dynamically using the 'new' keyword, unlike the object literal where the object is created directly. Here are some reasons why you would use object literals:

- i) Convenience. They provide a way to quickly create objects when you know the properties in advance.
- ii) Efficiency. They reduce the need for additional code to create and initialise objects.
- iii) Clarity. Object literals are easy to read and understand, especially for simple objects.

In a nutshell, JavaScript objects are referred to as object literals when they are defined directly in code using the `{}` syntax. This is because the object is created “literally” in the code, as opposed to being instantiated through a function or class.

Changing object literal values on the fly

Note that just like with constructor functions, you can alter the values of properties in object literals dynamically (on the fly) in JavaScript. Once an object is created, you can change the values of existing properties, add new properties, or delete properties at any time. You can change the value of a property by referencing it using either dot notation or bracket notation. For example:

```
let person = {  
  name: 'John',  
  age: 30  
};
```

Change the value of the 'name' property like this:

```
// Dot notation  
person.name = 'Jane';
```

Change the value of the 'age' property:

```
// Bracket notation  
person['age'] = 35;
```

You can also add new properties to an object at any time, even after the object has been defined. For example, add a ‘country’ property to contain the home country of the person:

```
person.country = "Canada";
```

You can also delete properties from an object using the delete operator. Here’s how:

```
delete person.age;
```


Restricting modification on objects

You are able to make changes to, add and remove properties and their values from objects as you please in JavaScript. This is because in JavaScript objects are mutable, meaning you can modify their structure and content after creation. This is a fundamental characteristic of JavaScript objects, making them flexible and allowing dynamic changes during runtime. However, there are cases where you may want to restrict this mutability. For that, JavaScript provides two methods on its built-in `Object`; `freeze()` and `seal()`. `Object.freeze()` prevents any modifications to the object (no adding, deleting, or changing properties). Here is how to use it:

```
let car = { brand: 'Toyota', model:
  'Corolla' };
Object.freeze(car);

// This will not work, as the object is
// frozen
car.model = 'Camry';
```

Be careful with the words "will not work" there, because what happens next surprises people. In ordinary, non-strict code the assignment fails SILENTLY. There is no error and nothing in the console — the line simply has no effect, and `car.model` is still 'Corolla' afterwards. In strict mode the very same line throws a `TypeError` instead. So if you are ever puzzled that an assignment seems to be ignored, a frozen object is one thing worth checking. Freezing also blocks adding and deleting, not just changing, which is what makes it a freeze rather than a lock on existing values.

`Object.seal()` is the gentler of the two. It allows you to change existing properties but prevents adding or deleting them:

```
let user = { name: 'Ada' };
Object.seal(user);

user.name = 'Grace'; // allowed
// blocked, stays undefined
user.age = 36;
// blocked, name survives
delete user.name;
```

The difference between a JavaScript object and JSON

It is simple, JSON which stands for JavaScript Object Notation is just the syntax for a file format called `json`. It is similar to how you have `xml` and `yml` files in programming, which both have their content written in `xml` and `yaml` syntax respectively. JSON is a lightweight data format used for transmitting data between syst-

ems (typically between a server and a web client eg in APIs). JSON is often used to store and exchange data, and it follows a strict syntax that is a subset of JavaScript object syntax, but it is purely textual. It has no way to represent a function, and any property whose value is a function or undefined is simply dropped when you convert. A Date is a special case worth knowing: it does survive the conversion, but only as text, so it comes back from `JSON.parse()` as a string rather than as a Date object. JSON is embraced so widely by JavaScript that it is common to write content in JSON and pass it around your application, or share it between different applications. Note though that JSON is a data FORMAT, not a data type — we will come back to that distinction at the end of this section, because it is the root of most of the confusion around the word. Where there can be a confusion is that people fail to understand the distinction between a json object and an object (a regular JavaScript object). This is because a lot of people refer to them interchangeably when speaking. However, they are not the same, even though their syntaxes are kind of similar. JavaScript objects are used to store and manipulate data within JavaScript programs. Let's look at how you can distinguish between the two types from their syntaxes. In the json object syntax, all its property names (keys) must always be in double quotes. The values of the properties must only be made of primitive data types (numbers, strings, booleans, null) or arrays, or other JSON objects. No other data types are allowed. For example:

```
{
  "animal": "Dog",
  "color": "brown",
  "num_legs": 4
}
```

Multiple JSON objects are enclosed within square brackets like any other array. For example:

```
[
  {
    "animal": "Dog",
    "color": "brown",
    "num_legs": 4
  },
  {
    "animal": "Cat",
    "color": "black",
    "num_legs": 4
  }
]
```

The items are terminated with commas, but the last item does not need to be followed by a comma. Also, multiple objects are separated by commas within the square brackets. The syntax of an object has its keys treated just like variables

(with no quotes around them), while its values can vary in data types eg strings, arrays, other objects, functions etc. For example:

```
let employee = {  
  id: 157,  
  name: 'Gus',  
  email: 'gus@gmail.com'  
}
```

Similarly to a json object, the items in a JavaScript object are terminated with commas, but the last item does not need to be followed by a comma.

To conclude, JSON (JavaScript Object Notation) is not a data type in JavaScript. Instead, it's a data format used to structure and exchange data. However, JSON is based on two fundamental data structures in JavaScript:

- Objects (key-value pairs)
- Arrays (ordered lists)

Since JSON data is stored as text (a string) and can be converted to and from JavaScript objects using `JSON.parse()` and `JSON.stringify()`, it is best categorised as a data format rather than a built-in data type or structure. So, while people sometimes loosely say "JSON object," the correct term is just JSON (for the string format) or JavaScript object (after parsing). Again, this is because JSON (JavaScript Object Notation) is a data format, not an actual JavaScript object. It looks like a JavaScript object, but it's just a string representation of data that follows a specific structure. However, when you parse JSON using `JSON.parse()`, it becomes a JavaScript object. Similarly, when you stringify a JavaScript object using `JSON.stringify()`, it turns into a JSON-formatted string.

Converting a JavaScript object to JSON

Use `JSON.stringify()`. Here is how you do it:

```
let person = { name: 'John', age: 30 };  
  
let jsonPerson = JSON.stringify(person);  
  
console.log(jsonPerson);
```

This will print out the following JSON object to the console:

```
{"name":"John","age":30}
```

Converting JSON to a JavaScript object

Use the `JSON.parse()`. Here is how to do it:

```
let jsonPerson =
  '{"name":"John","age":30}';

let person = JSON.parse(jsonPerson);

console.log(person.name);
```

This will print out to the console: John which is the value of the name property of the person JavaScript object.

Destructuring of objects

In simple terms, destructuring in JavaScript is a way to quickly take values out of an object or array and assign them to variables. Imagine you have a box (an object) with different items (properties) inside, like a name, a message and a footer. Normally, you would get each item out of the box one by one, assigning them individually to variables like so:

```
let obj = { msg: "Hello", title: "Welcome",
  footer: "Goodbye" };

let message = obj.msg;
let title = obj.title;
let footer = obj.footer;
```

Instead of assigning all three values to the three variables in three separate lines of code as above, destructuring makes it easier. It makes it possible to automatically create all three variables with their corresponding values from the object in one line of code. Simply do it like so:

```
let { msg, title, footer } = obj;
```

Now msg, title and footer will be separate variables containing values from obj. It's like unpacking the box in one step. When doing destructuring, it is advisable to give the variables default values. This is because sometimes the object you are destructuring (your box) might be missing some items. For example, what if the box does not have a "title". Destructuring with defaults lets you say, "if this item is missing, use this value instead". So let's give our example above some default values:

```
let {
  msg = "No message",
  title = "No title",
  footer = "No footer"
} = obj;
```

This way, if the box does not have a title, the variable title will automatically be set to "No title" instead of undefined. Take careful note of the equals sign there. It is

es everywhere else in an object. But inside destructuring a colon means something quite different — it RENAMES the variable:

```
// put the value of obj.msg into a
// variable called greeting
let { msg: greeting } = obj;
```

So a colon renames, and an equals sign supplies a fallback. You can use both together when you need to, and they read in that order — rename first, then default:

```
let { msg: greeting = "No message" } = obj;
```

Simplified object literal creation from function arguments

I talked about this under the topic of functions, under the heading “Quick object literals from function arguments”, but since this is a topic on objects, I thought I should quickly mention it again. Since you are learning how to create objects, you should remember how that links up with functions and how they work together. The idea is that you sometimes have a function whose only job is to return an object literal from the arguments passed to the function. For example:

```
// arrow function version
const createObject = (make, model, year) => {
  return {
    make: make,
    model: model,
    year: year
  };
};

console.log(
  createObject('Toyota', 'Rav4', '2025')
);
```

This function return value will be the object:

```
{make: 'Toyota', model: 'Rav4', year: '2025'}
```

Notice that the keys of the object literal returned by the function exactly match the arguments of the function. This is a repetition of the same names, first in the function argument, and then again in the keys of the object literal being returned. JavaScript provides a shorthand for exactly this situation: when a key and the variable supplying its value have the same name, you may write the name once. So { make: make } can simply be written { make }. This is called object shorthand, and although you will most often meet it alongside arrow functions, it is not an arrow-function feature — it works just as well inside an ordinary function. Combined with an arrow function’s one-line body, it lets you write the whole thing like this:

```
const createObject =
  (make, model, year) =>
    ({make, model, year});

console.log(
  createObject('Toyota', 'Rav4', '2025')
);
```

The return value of this simplified code is still the same:

```
{make: 'Toyota', model: 'Rav4', year: '2025'}
```

Viewing the contents of an object

When you want to read an object in the console and its contents are nested, printing it directly can be awkward. `JSON.stringify()` takes two optional extra arguments that turn it into a neat, indented listing:

```
console.log(JSON.stringify(YOUROBJECT,
  null, 4));
```

The second argument (`null` here) is a filter you will rarely need, and the third is how many spaces to indent each level by. Using 4 gives you a readable, tree-shaped view of the whole object. One thing to remember from the section above: anything JSON cannot represent will be missing from that listing. If your object holds methods, they will not appear, because functions are dropped on the way in.

Classes

Now that we have learned all about objects, it's time to understand what classes are in programming. You might be surprised by how simple the concept is once you understand objects! That's why I introduced objects first. In JavaScript, a class is essentially a blueprint for creating objects. It defines a group of objects that share the same properties and methods. Each object created from a class is called an instance of that class. Before ES6, JavaScript did not have a built-in class system. Instead, objects were created using constructor functions, and inheritance was handled through prototypes. If an object inherits from a prototype object, that object is considered an instance of the prototype. If multiple objects inherit from the same prototype, they are considered instances of the same class. In simple terms, a class is a group.

The Old Way: Constructor Functions (Pre-ES6) Before ES6, classes were simulated using constructor functions, and it still works. Here's an example:

```
function Obj(arg) {
  this.arg = arg;
}
```

```
let myObj = new Obj('car');
```

```
// Output: car  
console.log(myObj.arg);
```

This code instantiates the `Obj` class (using the `new` keyword) and writes to the console the value of the property `'arg'` which in this case is `"car"`. Here is how the above code works:

- The function `Obj` acts as a constructor.
- The `new` keyword creates an instance of `Obj`, assigning `"car"` to `arg`.
- This is how JavaScript developers created reusable object structures before ES6 introduced the class syntax.

The New Way: ES6 Classes With ES6, JavaScript introduced the `class` keyword, which provides a more structured and readable way to define classes:

```
class Obj {  
  constructor(arg) {  
    this.arg = arg;  
  }  
}
```

```
let myObj = new Obj('car');
```

```
// Output: car  
console.log(myObj.arg);
```

This code logs the value of the `arg` property of the class, which in this case will be `'car'`. Here are the key differences between the class approach and constructor functions:

- Classes use the `class` keyword instead of a function.
- Classes add a `constructor()` method to initialise the object.
- Functions, as we know, have the `prototype` property and the methods that come with it. With a class you no longer write to that property by hand — you simply define methods inside the class body and JavaScript puts them on the `prototype` for you. So the `prototype` has not gone away; the class syntax is just doing that work on your behalf. You can see it for yourself:

```
class Dog { speak() {} }  
  
// true – the method really is on the prototype  
console.log(  
  Dog.prototype.hasOwnProperty("speak")  
);
```

This ES6 approach improves readability and makes JavaScript more consistent with class-based languages like Java and Python. Use ES6 classes whenever possible—they're clearer and more structured. Constructor functions still work but are now considered outdated for defining classes. Both methods (constructors functions and classes) still rely on prototypes under the hood, but ES6 classes provide a more intuitive way to work with objects.

Class inheritance

JavaScript supports class inheritance using the `extends` keyword. This is a core feature of object-oriented programming in modern JavaScript (ES6 and later). The `extends` keyword allows one class (a child or subclass) to inherit from another class (a parent or superclass). The child class gets access to all properties and methods of the parent class, and can also define its own. Here is an example of extending a class in action:

```
// Parent class
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

// Child class
class Dog extends Animal {
  speak() {
    console.log(`${this.name} barks.`);
  }
}

const rex = new Dog("Rex");
rex.speak(); // Rex barks.
```

The child class can override parent methods, as `Dog` does with `speak()`. You can still call the parent method from within the child class using `super.methodName()`. For example, if you wanted to, you can always call the parent's method, for example, to call the `speak()` method of the `Animal` (parent class), instead of create its own, from within the `Dog` (child) class, you can do it like so:

```
class Dog extends Animal {
  speak() {
    super.speak();
  }
}
```


Inside a subclass constructor, you must call `super()` before you use the `this` keyword. And be careful with what this means here: it refers to the current INSTANCE — the particular object being built — not to the class itself. That distinction matters enough that the next section is devoted to it. The above example will work, but when one class extends another using the `extends` keyword, the child class inherits all the methods and properties of the parent. But to properly initialise the parent class, you need to use the `super()` function inside the child's constructor, and it must run before you touch `this`. In practice people put it on the first line, and that is the habit to keep. Strictly speaking JavaScript only requires that it comes before any use of `this` — reaching for `this` first throws a `ReferenceError`. Let's understand what `super()` is, and what it does. `super()` is used to call the constructor of the parent class. It must be called before using the `this` keyword inside a subclass constructor. Overriding is when the child class defines a method with the same name as another method on the parent class in order to change the behaviour of the method to suit its own needs.

What is the benefit of inheritance: - Promotes code re-use, so you avoid repeating code - To make general classes (like `Animal`) and then specific ones (like `Dog`, `Cat`, `Bird`) - It helps you build a class hierarchy that models real-world relationships

Difference between a child class and an instance

The difference between a child class and an instance is a specific version of the broader idea: the difference between a class and an object. A class is like a blueprint from which other objects or even other classes can be formed. A child class is a class that inherits properties and methods from another class (called the parent class). It is created using the `extends` keyword like so:

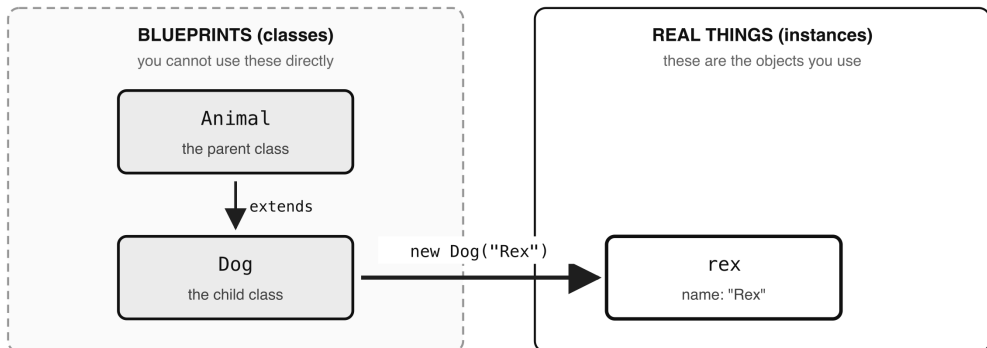
```
class Dog extends Animal {  
    ...  
}
```

A child class is still a class — just one that inherits from another (parent) class. On the other hand, an instance is a specific object created from a class-like a real-world version of the blueprint. Instances are created using the `new` keyword like this:

```
const rex = new Dog("Rex");
```

An instance is still an object — it's the actual usable thing created from any class (parent or child). Therefore, the class that the instance is created from can be either a base class or a child class—it does not matter.

A child class is still a blueprint. An instance is a real thing.



What isinstance says

<code>rex isinstance Dog</code>	true	— it was made from Dog
<code>rex isinstance Animal</code>	true	— Dog extends Animal, so rex is one too
<code>Dog isinstance Animal</code>	false	— Dog is a blueprint, not a thing made from one

Figure 12.4 — A child class is still a blueprint; an instance is a real thing

Call parent constructor from child class

When one class extends another using the `extends` keyword, the child class inherits all the methods and properties of the parent. But to properly initialize the parent class, you need to use the

`super()` function

inside the child's constructor, before any use of this. Here is an example:

```
// Child class
class Dog extends Animal {
    constructor(name, breed) {
        // pass argument(s) needed by parent constructor
        // Call parent constructor
        super(name);
        this.breed = breed;
    }
}
```

You can also call any parent method from within the child class using the following syntax:

`super.methodName()`

This can be useful if this child class has a method with the same name, but wants to call that of the parent.

Check if an object is an instance of a class

In programming, there will be times when you will need to know if an object is an instance of a class, for example for validation reasons. JavaScript offers you a simple way to check that, and that is by using the 'instanceof' keyword. The syntax goes like this:

```
objectName instanceof ClassName
```

Let's put that to action and check if rex is an instance of the Animal class. Here is the exact same code we used above—just so we can take a closer look:

```
// Parent class
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

// Child class
class Dog extends Animal {
  speak() {
    console.log(`${this.name} barks.`);
  }
}

const rex = new Dog("Rex");
rex.speak(); // Rex barks.

console.log("rex is an instance of Animal: ", rex instanceof Animal);
```

The output of the console.log() above will be:

```
rex is an instance of Animal: true
```

This is because although rex is an instance of Dog, and Dog is a child class of Animal, that still makes rex an instance of the Animal class too. This code: console.log("rex is an instance of Dog: ", rex instanceof Dog);

will return: rex is an instance of Dog: true and that is because rex is an instance of Dog as well, in fact, it was formed from the Dog class.

I will tell you what is not an instance of the Animal class—Dog. This code:

```
console.log("Dog is an instance of Animal: ", Dog instanceof Animal);
```

will return: Dog is an instance of Animal: false and that is because Dog is a child class, but not an instance/object of Animal. Both of them are classes, or blueprints from which other objects/instances would be created.

Function Constructors, object literals and inheritance

In JavaScript, we know the old way of creating a class was via Function, so it evolved from function-based “classes” and object literals to the modern class syntax. The following questions which we have answered for regular classes can also be asked of the old-alternative ways to create classes;

-Can you instantiate or extend an object literal or Function? -If so, can you call the parent property or constructor of a parent from a child object literal or Function?

Let's answer them below.

Function Constructors (Old Way of Creating classes)

Yes, function constructors can be instantiated using the new keyword. Here is how:

```
function Animal(name) {
    this.name = name;
}

Animal.prototype.speak = function () {
    console.log(`${this.name} makes a sound`);
};

// Instantiating
const a = new Animal("Lion");

// "Lion makes a sound"
a.speak();
```

You can also extend them using prototypal inheritance. Here is how:

```
function Dog(name) {
    // Call parent constructor
    Animal.call(this, name);
}

// Inherit methods
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

const d = new Dog("Rex");
```

```
// Will output "Rex makes a sound"
d.speak();
```

As you can see, you can call the parent constructor (`Animal.call(this, ...)`) and access parent methods via the prototype chain.

Object literals

Object literals cannot be instantiated with `new` (they are not constructor functions). But you can extend them using `Object.create()`:

```
const animal = {
  speak() {
    console.log(`${this.name} makes a sound`);
  }
};

// Inherit from object literal
const dog = Object.create(animal);
dog.name = "Rex";
dog.speak(); // "Rex makes a sound"
```

No constructor is called here — just delegation via the prototype chain.

In conclusion: - You cannot "instantiate" an object literal. - You can extend it. - There's no constructor to "call" — only properties/methods to inherit.

Class setters and getters

With classes, just like with object literals or constructor functions, you often want to pass in a value to store (set) on one of the properties, and you need a way to retrieve (get) that value when you need it. The methods in a class which you create to do so are also referred to as setter and getter methods, or in short, setters and getters, for setting and getting the property value, respectively. In our `Obj` example above, we are passing in a value via the constructor to be set (store) on the `arg` property. We did not use a setter method to do so, and even when we retrieved the value of the `arg` property, we did not use a getter method either, we just got the value directly from the object like so: `myObj.arg`

```
let myObj = new Obj('car');

// Output: car
console.log(myObj.arg);
```

However, to harness the power of classes, that is not always an ideal approach. There are times when you want your class to provide dedicated methods to set (store) and get (retrieve) the values of its properties, so that any code outside of the

important because by only having access to the value of a property through a getter method, the getter method can be a central place for example, to do the necessary validation and check if the caller has access to that value, or format the data in a desirable format before returning it etc. Getters and setters are therefore a very big part of class encapsulation, which is a term used in OOP to refer to how classes manage to keep the integrity and security of the data they hold. A class is well encapsulated when it provides getters and setters so you do not have direct access to its members (properties or methods), and, or when it defines good visibility for its members in terms of whether a member is accessible only within the defining class, or by its children as well, or if they are publicly accessible. Let's modify our Obj class by adding a getter and a setter method.

```
class Obj {
  // set an initial value for arg
  constructor (arg) {
    this._arg = arg
  }

  // getter
  get arg() {
    return this._arg;
  }

  // setter
  set arg(newArg) {
    this._arg = newArg;
  }
}

// instantiate the class
let myObj = new Obj('car');

// check value of _arg
console.log(myObj.arg);

// update value of _arg
myObj.arg = 'truck';

// check value of _arg
console.log(myObj.arg);
```

The above code will write to the console the following two values; one which is the initial value of `_arg` when the class is created, and the other which the new value when it is updated.

car truck

The constructor serves to set an initial default value for the `_arg` property when the class is created. Naming the property with an underscore prefix (`_arg`) is genera-

lly how you mark a property in a JavaScript class as private. A private member (method or property) means that member is not supposed to be accessed from outside the class that defines (owns) that property. In Object Oriented Programming in general, specifying the accessibility of the member of a class is known as visibility. Types of class member visibility are private, public. Notice that the getter is declared using the 'get' keyword (get arg()) while a setter method is declared using the 'set' keyword (set arg()). Also notice that when you call the setter of getter, you do not use the parenthesis, but rather, you reference it like you would do a property eg:

```
// call the setter to set the value
myObj.arg = 'truck';

// call the getter to get the value
console.log(myObj.arg);
```

Sharing data between files

The difference between import and require

In the past, developers would use the require() function to pull data from an external file into a file where they need to use it. The modern and better way is to use import and export. This allows you to export code from one file and import it into another file. It also allows you to only import certain functions and variables from the code in the exporting file. Here is how:

```
// file1.js
export const capitalizeWord = str => str.toUpperCase();

// file2.js
import { capitalizeWord } from "./file1";

const upper = capitalizeWord("jump");
console.log(upper);
```

This will write to the console: JUMP

In this example, the file we want to import the capitalizeWord from is file1.js, so within file1.js we write the function capitalizeWord and place 'export' in front of it to export it. This function contains an arrow function that accepts an argument (str) and returns it all capitalised using the built-in JavaScript string function toUpperCase().

The file where we want to import this function is file2.js, so in file2.js we use the 'import' keyword to pull in file1.js. Notice that you have to enclose the variable or function you want to import within curly braces (like so { capitalizeWord }), then use

the 'from' keyword to specify the path of the target file. In this case we are assuming that both files are within the same directory.

Note also that when you specify the file, you leave out the '.js' extension, so this will do just fine:

```
import { capitalizeWord } from "./file1";
```

Exporting multiple functions or variables

You can export multiple variables and functions from one file to another. Here is an example of our file1.js file above, modified to export multiple separate values:

```
// file1.js
// single function
export const capitalizeWord = str => str.toUpperCase();

// separate variables
export const foo = "bar";
export const bar = "foo";

let name = "John Bands";
let age = 40;

export {name, age};

// file2.js
import { capitalizeWord, foo, bar, name, age } from "./file1";
```

Here are the points to take note of:

- As can be seen, when exporting multiple values, wrap them all, comma-separated within curly braces like so:

```
export {name, age};
```
- You can place the 'export' keyword to export individual variables or functions, as we do with capitalizeWord, and foo and bar above, but you can also declare them all without the 'export' keyword, and then export them all at once by wrapping them, comma-separated within curly braces like so:

```
export {capitalizeWord, foo, bar};
```
- Here is a point that trips people up, and it is worth slowing down for. The curly braces in export { ... } are NOT a block of code. They are simply a LIST of names you are handing out. That is why you never put declarations inside them. This does not work:


```
export {
  let name = "John Bands";
  let age = 40
}
```

And nor does wrapping the declarations in a real block and exporting afterwards, which looks reasonable but fails for a different reason:

```
{
  let name = "John Bands";
  let age = 40
}
```

```
// SyntaxError: Export 'age' is not defined in module
export {name, age};
```

The reason is scope, which we met back in Chapter 2. A pair of braces creates a block, and `let` and `const` are block-scoped, so `name` and `age` exist only inside those braces and are gone by the time the `export` line runs. There is nothing left to export. The fix is simply not to use a block at all. Declare the values at the top level of the file, then list them:

```
let name = "John Bands";
let age = 40;
```

```
export {name, age};
```

- Notice how in the importing file, you import the multiple variables and functions in the same way, except you simply add all the multiple values to the curly braces separated by commas eg:

```
import { capitalizeWord, foo, bar, name, age } from "./file1";
```

Using a wildcard to import all from a file

Take our example from “Exporting multiple functions or variables” above where we are exporting multiple values, there is another simplified way to pull them in, in the file where they are to be used (`file2.js`). So instead of writing the variables/functions all out in the curly braces at the ‘import’ statement like so:

```
import { capitalizeWord, foo, bar, name, age } from "./file1";
```

Simply do this:

```
import * as fileOneValues from "./file1";
```

Call the `fileOneValues` anything you want. That will be the variable within this `file2.js` which will hold all values exported from `file1.js`. To use those values within

this file2.js, just reference them from fileOneValues as its properties like so: fileOneValues.foo Or fileOneValues.bar etc

For example, let us do our capitalising example again:

```
const upper = fileOneValues.capitalizeWord("jump");
console.log(upper);
```

This will write JUMP to the console.

Export fallback with export default

The exports we have looked at above are named exports. There is also a concept known as export defaults. This is a fallback export, and this is used when you only want to export one value from a file.

```
// file1.js
function subtract(a, b) {
  return a - b;
}

export default function add(a, b) {
  return a + b;
}
```

The add() function is the only thing that is going to be exported from the file1.js file.

To import the default export values, here is how. The example below will import the default function 'add()' that was exported from file1.js into file2.js

```
// file2.js
import add from "./file1";
```

You do not need to wrap the imported value within curly braces, the curly braces are meant only for named exports.

Other advanced OOP concepts

Once you learn JavaScript, you start to compare it to other programming languages, and then the concept of classes suddenly feels weird and different in JavaScript. You look at other server-side languages like Java, PHP, Python, C# etc and you start to wonder: does JavaScript have concepts like static methods or properties? What about abstract classes, interfaces, dependency injection? Do not worry, you are not alone in this. JavaScript is a language not like the others, not least because its turf is on the front end side of the web. By the way, Node.js has you covered on all of that. It has all those concepts since it is the solution for running JavaScript on the server-side. However, our focus in this book is on vanilla

JavaScript, which means core or pure JavaScript with no external frameworks or libraries.

Throwing these questions back to JavaScript, there is good news. JavaScript does have equivalents (or similar concepts) to things like abstract classes, interfaces, dependency injection, and static methods, but they work a bit differently compared to the other programming languages. We will now proceed to list these concepts one after the other, and show in code how they can be implemented just as effectively in JavaScript.

Static Methods

What Are Static Methods? A static method belongs to the class itself, not to instances of the class. Such a method is useful for utility functions or helper methods. Here is an example:

```
class MathHelper {
  static add(a, b) {
    return a + b;
  }

  subtract(a, b) {
    return a - b;
  }
}

// Output: 8
console.log(MathHelper.add(5, 3));

// create an INSTANCE of MathHelper
let mathObject = new MathHelper();

// Output: 10
console.log(mathObject.subtract(15, 5));

// output: 7
console.log(mathObject.subtract(9, 2));

// Output: TypeError: mathObject.add is not a function
console.log(mathObject.add(3, 3));
```

The above example class `MathHelper` has two methods, one static `'add()'` and one non-static `'subtract()'`. The rule to hold on to is this: a static method lives on the CLASS, not on the objects made from it. That is why calling `add()` on the class itself works:

```
MathHelper.add(5, 3); // 8
```

But when we make an object from that class and try the same thing:

```
// this is an INSTANCE, not a class
let mathObject = new MathHelper();

console.log(mathObject.add(3, 3));
```

We get the error: `TypeError: mathObject.add is not a function`. Note that the instance can use the other, non-static method `subtract()` perfectly well:

```
// output: 7
console.log(mathObject.subtract(9, 2));
```

And it works just fine. Be careful not to read this as "statics cannot be inherited", because they can. If you make a genuine CHILD CLASS with `extends`, that child gets the static method too:

```
class ChildHelper extends MathHelper {}

// 8 - the child class inherits the static
console.log(ChildHelper.add(5, 3));

// but its instances still cannot see it
console.log(new ChildHelper().add);
// undefined
```

So the dividing line is not parent versus child. It is class versus instance. Statics travel down the class line, and never appear on the objects made from it.

Abstract Classes

In other programming languages like PHP, an abstract class is a class that cannot be instantiated and is meant to be extended by subclasses. JavaScript doesn't have built-in abstract classes, but you can simulate them. To implement a JavaScript Equivalent of an abstract class, you should use a base class. Here is how to do it:

- Detect any attempt at instantiation of the class from its constructor and throw an error. This will prevent direct instantiation.
- Use inheritance (with the 'extends') to force subclassing.

Here is an example:

```
class Animal {
  constructor() {
    if (new.target === Animal) {
      throw new Error(
        "Cannot instantiate an abstract class!"
      );
    }
  }
}
```

```

makeSound() {
    throw new Error(
        "Method 'makeSound()' must be implemented in a subclass"
    );
}
}

class Dog extends Animal {
    makeSound() {
        return "Woof!";
    }
}

let myDog = new Dog();
console.log(myDog.makeSound());
// Output: Woof!

// Error: Cannot instantiate an abstract class!
let myAnimal = new Animal();

```

In the example above, the parent abstract class is `Animal`, and `myDog` inherits from it (using the 'extends' keyword). If you try to instantiate the `Dog` class, it works fine. It is okay to create an instance of the child class. However, when you try to create an instance (instantiate) the `Animal` (parent) class like so:

```
let myAnimal = new Animal();
```

You get the error: `Error: Cannot instantiate an abstract class!`

This is because of this code in the constructor of the `Animal` class that forbids the creation of instances of the class:

```

if (new.target === Animal) {
    throw new Error(
        "Cannot instantiate an abstract class!"
    );
}

```

This behaviour makes that class behave exactly like an abstract class of any other programming language.

Interfaces

Languages like `Java` or `TypeScript` have interfaces, which define a contract that classes must follow. A `JavaScript` superset like `TypeScript` does support interfaces, though `JavaScript` itself does not have built-in interfaces. However, in `JavaScript` you can simulate interfaces using objects or decorators. Here is an example simulating an Interface with a Mixin.

```

const Logger = {
  log(message) {
    console.log(` [LOG]: ${message}`);
  }
};

class User {
  constructor(name) {
    this.name = name;
  }
}

// Applying the interface (mixin)
Object.assign(User.prototype, Logger);

let user = new User("Alice");

// Output: [LOG]: User created!
user.log("User created!");

```

Here, Logger acts like an interface. `Object.assign()` is basically used to add `log()` to User dynamically, so after instantiating User, we can then call `log()` on the user child class like so:

```

user.log("User created!");

```

Dependency Injection (DI)

What is Dependency Injection? In Java or Angular, DI is a design pattern where dependencies (like services) are injected instead of being hardcoded inside classes. JavaScript doesn't have built-in DI, but you can implement it manually.

Manual Dependency Injection

```

class Database {
  connect() {
    console.log("Connected to the database.");
  }
}

class UserService {
  constructor(database) {
    // Injected dependency
    this.database = database;
  }

  getUser() {
    this.database.connect();
    return { name: "Alice" };
  }
}

```

```
// Create dependency
let db = new Database();

// Inject dependency
let userService = new UserService(db);

// Output: { name: "Alice" }
console.log(userService.getUser());
```

Notice that it's crucial for DI that the database object is passed into `UserService` instead of being created inside it. This makes `UserService` more flexible and testable.

Private methods and properties (Encapsulation)

Before ES2020, JavaScript did not have private methods and properties. It only had public members, but now it has private members which you can create using the `#` syntax. Up until then, developers simulated private properties by prefixing names of properties they wanted to use as private with an underscore like so: `_property`. This was however just a convention as that did not really make those properties private. Here is an example of how they did it:

```
class Person {
  constructor(name) {
    // Just a naming convention
    this._name = name;
  }
}

let pers = new Person("Alice");

// Output: Alice (Still accessible!)
console.log(pers._name);
```

So, as you see, the issue with this approach is that it does not prevent access—it's just a developer convention to indicate "please don't touch this property."

There is another way developers implemented private properties, and it ended up achieving a true private property, however it is a complex approach. This was by using closures and `WeakMaps`. Here is an example:

```
const Person = (function () {
  let privateData = new WeakMap();

  return class {
    constructor(name) {
      privateData.set(this, { name });
    }
  }
})();
```

```

    return privateData.get(this).name;
  }
};
})();

let pers = new Person("Alice");
console.log(pers.getName());
// Output: Alice
console.log(pers.name);
// Undefined (truly private)

```

Why did this work? The property was stored in a `WeakMap`, which is only accessible inside the class. The downside of this approach was, it was complex and harder to maintain.

With the coming of ES2020, both private properties and methods were introduced using `#`:

```

class Person {
  #privateAge = 30; // Private property

  #getSecret() { // Private method
    return "This is a secret!";
  }

  revealSecret() {
    return this.#getSecret();
  }
}

let pers = new Person();

// Output: This is a secret!
console.log(pers.revealSecret());

```

Notice that the instance `pers` can use the ordinary methods of the class, like `revealSecret()`, and it works just fine. That method is allowed to reach `#getSecret()` because it is written inside the class. But reaching for a private field from outside the class does not work:

```

// SyntaxError: Private field '#privateAge' must be
// declared in an enclosing class
console.log(pers.#privateAge);

```

Look closely at what kind of error that is, because it is stricter than you might expect. It is a `SyntaxError`, not a runtime error. JavaScript refuses at the moment it reads your file, before a single line has run. So this is not a case of the value coming back as undefined, or of an error being thrown when that line is reached — the whole script fails to start. That is what makes `#` properties genuinely private, and different from the older conventions. Naming a property `_age` to signal "please do

not touch this" relies on everyone being polite. A # field is enforced by the language itself.

In conclusion of this OOP topic in JavaScript, I will say that JavaScript does support OOP concepts but in a more flexible and prototype-based way. ES6 classes make JavaScript feel more like traditional OOP, but under the hood, it's still prototype-based. For real interfaces and dependency injection, TypeScript or frameworks like Angular provide more structured solutions.

QUIZ — OOP

This page contains the Q & A (questions and answers) for this chapter — Chapter 12: OOP. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

This chapter carries more ideas than any other in the book, so the quiz is a long one. Try every question before you look below. Each one carries a clue. Questions 12 to 16 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Two different things in JavaScript are called "prototype". What are they, and which kind of thing has each one?

Clue: one is a property you can see and only one kind of value has it. The other is a link that everything has.

2. Given `function Person() {}` and `let p = new Person();` — what is `p`'s prototype? Write down the full chain from `p` to the end.

Clue: it is not `Person` itself, and the chain always finishes at the same value.

3. What does this print, and why?

```
console.log(({}).prototype);
```

Clue: see question 1. This is the quickest way to prove the point.

4. A method is passed to `setTimeout()` and suddenly `this` is not what you expected. Explain why, and name three ways to fix it.

Clue: the function arrives without the object it belongs to.

5. What is the difference between `call()`, `apply()` and `bind()`?

Clue: two of them run the function immediately. Of those two, only one thing separates them.

6. `Object.freeze()` stops an object being changed. What happens if you try to change one anyway?

Clue: the answer depends on strict mode, and one of the two possibilities is nastier than the other.

7. In destructuring, what is the difference between these two lines?

```
let { msg: greeting } = obj;  
let { msg = greeting } = obj;
```

Clue: one is about a name, the other is about a missing value.

8. `rex` is created from `class Dog extends Animal`. What do these three give you?

`rex instanceof Dog` `rex instanceof Animal` `Dog instanceof Animal`

Clue: two of them agree and the third is the interesting one.

9. A static method is defined on a class. Can an instance of the class use it? Can a child class use it?

Clue: the answers are different, and that is the whole point of the question.

10. Why does this fail, and what is the fix?

```
{
  let name = "John Bands";
  let age = 40
}

export {name, age};
```

Clue: think about scope, and about what the braces in `export { ... }` actually mean.

11. What kind of error do you get from reading a `#private` field outside its class, and why is that stricter than it first appears?

Clue: it happens earlier than you would think.

12. EXERCISE. Write a `Person` constructor function taking a name, add a `greet()` method to its prototype after the fact, then create two people and prove they both have it.

Clue: add the method to the mould, not to each object.

13. EXERCISE. Prove in code that a method added to `Person.prototype` is not an own property of the instance.

Clue: one method name from `Object.prototype` answers this directly.

14. EXERCISE. Write `Animal` with a `speak()` method, and `Dog extends Animal` whose constructor also takes a breed. Call the parent constructor properly and override `speak()`.

Clue: one keyword does both jobs, in two different forms.

15. EXERCISE. Take `{ name: 'Ada', age: 36 }`, convert it to JSON, print it, convert it back, and print the name.

Clue: two methods, both starting with JSON.

16. EXERCISE. Destructure `{ msg: 'Hello' }` into three variables — `msg`, `title` and `footer` — where the two missing ones fall back to sensible defaults.

Clue: question 7 tells you which character you need.

ANSWERS

1. `.prototype` — a real, visible property that **only functions have**. It holds the object that instances made from that function will inherit from.
- `__proto__` — the internal link that **every object has**, pointing at the object it actually inherits from. Its proper name in the specification is `[[Prototype]]`, and modern browser consoles display it that way.

They are not two kinds of the same thing. They are two ends of one relationship:

```
person1.__proto__ === Person.prototype
// true
```

A function owns a mould. An object remembers which mould it came from.

2. `p`'s prototype is `Person.prototype` — not `Person` itself. The full chain:

`p` → `Person.prototype` → `Object.prototype` → `null`

You can walk it:

```
Object.getPrototypeOf(p) === Person.prototype;
// true
Object.getPrototypeOf(Person.prototype) === Object.prototype;
// true
Object.getPrototypeOf(Object.prototype);
// null
```

Every chain ends at `null`. That is how JavaScript knows to stop looking.

3. It prints `undefined`.

Ordinary objects do not have a `.prototype` property at all — only functions do. If you want an object's prototype you have to ask for it the other way:

```
// Object.prototype
Object.getPrototypeOf({});
```

Getting this the wrong way round is the single most common confusion in JavaScript objects, which is why it is worth being able to prove it in one line.

4. Because the method was **separated from the object it belongs to**. `setTimeout()` receives the function on its own and calls it with no owner, so `this` is `null`.

longer `person`. In strict mode it becomes `undefined`; in non-strict mode it falls back to the global object. Either way, `this.name` is not what you wanted.

The three fixes all amount to the same idea — send the owner along with the function:

```
// 1. wrap it, so the owner is
// named again at call time
setTimeout(function () { person.getName(); }, 1000);

// 2. bind it, which returns a
// NEW function with this fixed
let f = person.getName.bind(person);
setTimeout(f, 1000);

// 3. call or apply it, which run it straight away
person.getName.call(person);
```

5. **`call()` and `apply()` run the function immediately.** The only difference between those two is how you hand over the arguments:

```
// a list of arguments
car.multiply.call(bike, 2, 2);
// an array of arguments
car.multiply.apply(bike, [2, 2]);
```

`bind()` does not run anything. It returns a *new* function with `this` already fixed, for you to call whenever you like:

```
let multiply = car.multiply.bind(bike, 2, 2);
multiply(); // 4 - runs now
```

A way to remember it: `call` and `apply` are verbs you do to the function now; `bind` is a preparation for later.

6. **In non-strict code it fails silently.** No error, nothing in the console — the line simply has no effect:

```
let car = { brand: 'Toyota', model: 'Corolla' };
Object.freeze(car);
car.model = 'Camry';
console.log(car.model);
// still 'Corolla'
```

In strict mode the same line throws a `TypeError`.

The silent version is the nastier of the two, because there is nothing to tell you what happened. If an assignment seems to be ignored for no reason, a frozen object is worth checking.

Note that freezing also blocks *adding* and *deleting*, not just changing. `Object.seal()` is the gentler one: existing properties can still be changed, but

nothing may be added or removed.

```
7. // RENAME
   let { msg: greeting } = obj;
   // DEFAULT
   let { msg = greeting } = obj;
```

The **colon renames**: take the value of `obj.msg` and put it in a variable called `greeting`.

The **equals sign supplies a fallback**: put `obj.msg` in a variable called `msg`, but if `obj.msg` is missing, use `greeting` instead.

This trips people up because everywhere

else

in an object a colon separates a key from its value. Inside destructuring it does not. You can use both together, and they read in that order — rename first, then default:

```
let { msg: greeting = "No message" } = obj;
```

8.

rex instanceof Dog	// true
rex instanceof Animal	// true
Dog instanceof Animal	// false

The first two are true because `rex` really was made from `Dog`, and `Dog` extends `Animal`, so `Animal.prototype` is in `rex`'s chain as well.

The third is the interesting one. `Dog` is a **class** — a blueprint. It was never made *from* `Animal` with `new`; it merely extends it. `instanceof` asks "was this thing built from that blueprint?", and a blueprint is not a thing built from a blueprint.

That is the difference between a child class and an instance in one line.

9. An instance cannot use it. A child class can.

```
class MathHelper {
  static add(a, b) { return a + b; }
}

MathHelper.add(5, 3);
// 8 - on the class, fine

let obj = new MathHelper();
// TypeError: obj.add is not a function
obj.add(3, 3);
```

```

class ChildHelper extends MathHelper {}
ChildHelper.add(5, 3);
// 8 - the child class inherits it
new ChildHelper().add;
// undefined - but its instances do not

```

So the dividing line is not parent versus child. It is **class versus instance**.

Statics travel down the class line and never appear on the objects made from those classes.

10. It fails with **SyntaxError: Export 'age' is not defined in module**.

Two things are going on, and the second is the important one.

First, the braces in `export { ... }` are **not a block of code**. They are a *list of names* you are handing out. That is why declarations never go inside them.

Second — and this is why the code above fails — a pair of braces creates a **block**, and `let` and `const` are block-scoped. So `name` and `age` exist only inside those braces and are gone by the time the `export` line runs. There is nothing left to export.

The fix is to drop the block entirely:

```

let name = "John Bands";
let age = 40;

export {name, age};

```

11. You get a **SyntaxError**:

SyntaxError: Private field '#privateAge' must be declared in an enclosing class

It is stricter than it looks because a `SyntaxError` happens when JavaScript *reads* your file, before a single line has run. So this is not a value coming back as `undefined`, and not an error thrown when execution reaches that line — **the whole script fails to start**.

That is what makes `#` fields genuinely private. The old convention of naming something `_age` to signal "please do not touch" relies on everyone being polite. A `#` field is enforced by the language.

```

12. function Person(name) {
    this.name = name;
}

// add the method to the MOULD, after the fact
Person.prototype.greet = function () {
    return "Hello, " + this.name;
};

let p1 = new Person("Tom");
let p2 = new Person("John");

console.log(p1.greet());
// "Hello, Tom"
console.log(p2.greet());
// "Hello, John"

```

Both objects get the method even though it was added after they were created. That is the power of the prototype: they do not each hold a copy, they all look up the same one.

```

13. console.log(p1.greet);
    // function - it can use it
    console.log(p1.hasOwnProperty("greet"));
    // false - but it does not own it
    console.log(Person.prototype.hasOwnProperty("greet"));
    // true - the mould owns it

```

`hasOwnProperty()` asks only about the object itself, ignoring everything up the chain. So this is the tool for telling "this object has it" apart from "this object can reach it".

The same is true of class methods, which is worth knowing: `class Dog { speak() {} }` puts `speak` on `Dog.prototype`, not on each dog.


```

14. class Animal {
    constructor(name) {
        this.name = name;
    }

    speak() {
        return this.name + " makes a noise.";
    }
}

class Dog extends Animal {
    constructor(name, breed) {
        // pass what the parent constructor needs
        super(name);
        this.breed = breed;
    }

    speak() {
        return this.name + " barks.";
    }
}

let rex = new Dog("Rex", "Alsatian");

console.log(rex.speak());
// "Rex barks."
console.log(rex.breed);
// "Alsatian"

```

The one keyword doing two jobs is `super`. As `super(...)` it calls the parent's **constructor**; as `super.speak()` it calls the parent's **method**.

`super(name)` must run before you touch `this` — reaching for `this` first throws a `ReferenceError`. In practice, put it on the first line.

```

15. let person = { name: 'Ada', age: 36 };

    let json = JSON.stringify(person);
    console.log(json);
    // {"name":"Ada","age":36}

    let back = JSON.parse(json);
    console.log(back.name);
    // Ada

```

Note what JSON gives you back is a **new** object, not the original one. And remember what does not survive the trip: functions and `undefined` values are dropped entirely, and a `Date` comes back as a string rather than as a `Date`.

```
16. let obj = { msg: 'Hello' };

    let {
      msg = "No message",
      title = "No title",
      footer = "No footer"
    } = obj;

    console.log(msg);
    // "Hello" - present, so kept
    console.log(title);
    // "No title" - missing, so defaulted
    console.log(footer);
    // "No footer" - missing, so defaulted
```

The equals sign is what you need, as question 7 established. Reach for a colon here and you get `SyntaxError: Invalid destructuring assignment target`, because a colon expects a variable name on its right, not a value.

Chapter 13 – DATABASES AND STORAGE

- Storage for data persistence e.g `localStorage`. Also, everything about database access in terms of SQL queries, database configuration, and database design and management.
- Learn the SQL (Structured Query Language) which is a global standard for communication with most database systems.
- Learn about the different types of database systems out there. For example, RDBS (Relational database systems) like MySQL, PostgreSQL, MariaDB, then learn about noSQL databases like Mongo DB
- Find out about what APIs your chosen programming language provides to use in connecting with your chosen database solution.

Data Persistence

- Introduction
- Persisting data with `LocalStorage`
- How to read data from `LocalStorage`
- How to save data to `LocalStorage`
- How to update data in `LocalStorage`
- How to delete data from `LocalStorage`
- Persisting with Session Storage
- Persisting with Cookies
- Databases
- What is a database
- JavaScript and databases
- Relational vs NoSQL Databases
- Common database stacks you may encounter
- Why JavaScript doesn't talk directly to databases
- Popular databases for JavaScript developers

Introduction Here we will be looking at the ways in which data can be persisted so that it does not just go away once we close our application. In JavaScript, there are many technologies and tools to achieve this: for example, `localStorage`, `sessionStorage`, Cookies, and database systems (relational/non-relational). Though, databases are quite easy to understand in terms of saving data and reading from them, you will need something like a server for example Node.js to handle those operations. That is beyond the scope of this book, hence we will not go into that. In this section, we will be focusing on three storage mechanisms; `localStorage`, `sessionStorage` and Cookies. Once you understand how to write to, and read from a storage system like `localStorage`, then working with a NoSQL database like MongoDB will be very easy to pick up as they both use json objects to store data. It

is worth noting that among the three storage types we will look at, cookies work differently from `localStorage` and `sessionStorage`. While `localStorage` and `sessionStorage` are client-side storage mechanisms used to store data locally, cookies are different because they are mainly used for exchanging small pieces of data between the client and server (like authentication tokens or user preferences). However, since cookies also persist data, they are worth mentioning here. Here's a clear and structured comparison of cookies, `localStorage`, and `sessionStorage` in bullet points:

Cookies:

- Purpose: Primarily used for server-client communication (e.g., authentication, session tracking, user preferences).
- Storage Limit: About 4KB per cookie.
- Persistence: Can have an expiration date, or be deleted when the browser closes (session cookies).
- Access: Available to both JavaScript and the server (automatically sent with every HTTP request).
- Security: Can be vulnerable to cross-site scripting (XSS) attacks if not handled properly. Can be marked as `HttpOnly` (inaccessible to JavaScript) or `Secure` (sent only over HTTPS).
- Use Case: Storing user session tokens, authentication details, and preferences that need to be shared with the server.

`localStorage`:

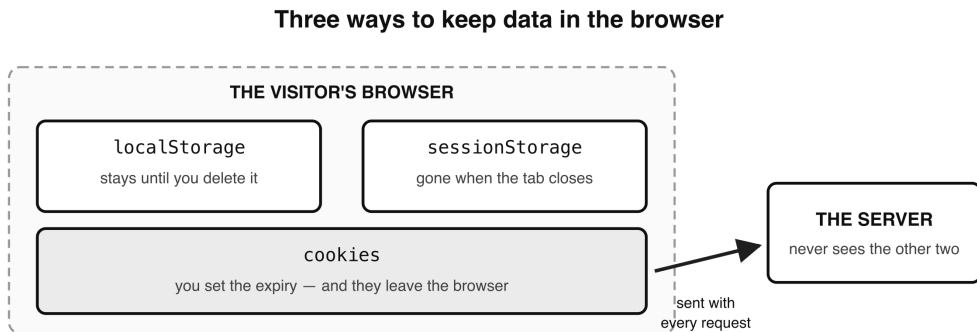
- Purpose: Stores data in the browser for long-term use.
- Storage Limit: Around 5-10MB per domain.
- Persistence: Does not expire (remains even after the browser is closed and reopened).
- Access: Only available to JavaScript (not sent to the server).
- Security: More secure than cookies in terms of preventing session hijacking (but still vulnerable to XSS if misused).
- Use Case: Storing user preferences, theme settings, or cached data that should persist across sessions.

`sessionStorage`:

- Purpose: Similar to `localStorage`, but only for a single session.
- Storage Limit: Around 5-10MB per domain.
- Persistence: Only lasts until the browser or tab is closed.

- Access: Only available to JavaScript (not sent to the server).
- Security: Like localStorage, it is vulnerable to XSS attacks.
- Use Case: Storing temporary data like form inputs, or session-based preferences that don't need to persist after the user leaves the page.

So, the take-away from this is this: if you need to store data permanently, use localStorage, if you only need it for a single session, use sessionStorage, and if you need the server to access the data, use cookies.



Choosing between them

Need it after today?	localStorage	Server needs it?	cookies
Only for this tab?	sessionStorage	Sensitive? None of them.	

- Figure 13.1 — Three ways to keep data in the browser*

Persisting data with localStorage

For purposes of demonstration, we will assume that we are storing the data from a TodoList application in the LocalStorage. The specific reference of the data in LocalStorage is `nlm.todoList`. The data is in the following format: `[ {id: 1, name: "todo list value 1", date: "20-8-2020"}, {id: 2, name: "todo list value 2", date: "20-8-2020"}, {id: 3, name: "todo list value 3", date: "20-8-2020"} ]`

We will proceed to learn how we can go about reading from this data and how to write to it. We will also look at how to update specific records in that data, and how to delete specific records in that data.

```
const LOCAL_STORAGE_TODO_KEY = 'nlm.todoList';

//an array to hold all current todoList items
let stuffTodo = [];
```

How to read data from LocalStorage

In this reading demonstration, we will loop through all the `todoList` items grabbing their id, and their name (value), then format that into a string that can be disp-

laid on screen.

```
/*-----DISPLAY OF TODOLIST ITEMS-----*/
// manage displaying of existing
// todoList items to the user.
// You get the data from localStorage
// if there are previously
// saved todoList items, grab them
if (JSON.parse(localStorage.getItem(
  LOCAL_STORAGE_TODO_KEY)) !== null) {
  JSON.parse(localStorage.getItem(
    LOCAL_STORAGE_TODO_KEY
  )).map(todoFromStorage => {
    const li = document.createElement('li');
    let todoString = "";
    todoString = `
      <div>
        <span class='todo-item'>${todoFromStorage.name}
      </span>
      <button name='deleteButton'
        id="deleteButton${todoFromStorage.id}"
        class='deleteButton'>Delete
      </button>

      <button name='editButton' id='${todoFromStorage.id}'
        class='editButton'>Edit
      </button>

      <input type='checkbox' name='checkButton'
        class='checkButton' />

      <div class='editDiv'
        id='editFormDiv${todoFromStorage.id}'>
        <form class='editForm' onSubmit="saveEdit(event)">
      <input type='text'
        class='form-control'
        name='editField' />

      <a onClick='cancelEdit(event)'
        class='btn btn-danger-sm cancelEdit'>x</a>

      <input type='submit'
        class='form-control btn btn-primary-sm editValue editInput'
        value='Save'>
      </form>
    </div>
  </div>`;

    li.innerHTML = todoString;
    Object.assign(li, {
      'draggable': 'true',
```

```

ul.appendChild(li);

});
}

```

How to save data to LocalStorage

```

function addTodo(e)
{
    //get the todoList item submitted by user
    const somethingTodo = input.value;
    if (somethingTodo == '')
    {
        alertError.children[0].innerHTML =
        "Please type in something!";
        alertError.style.display = 'block';
        setTimeout(() => {
            alertError.children[0].innerHTML = "";
            alertError.style.display = 'none';
        }, 3000)
    }
    else
    {
        // get the ID to be used for
        // the ingoing todoList Item
        let id = getId();

        //create an object to hold the todoList item
        let somethingTodoObj = {
            id: id,
            name: somethingTodo,
            date: day+'-'+month+'-'+year
        };

        // clear the item just submitted from the input field
        // input.value = "";
        // Check in the local storage
        // for todoList items
        // already saved there. if
        // there's nothing in storage,
        // use what user has submitted
        if (JSON.parse(localStorage.getItem(
        LOCAL_STORAGE_TODO_KEY)) === null)
        {
            stuffTodo.push(somethingTodoObj);

            //then save it in storage
            localStorage.setItem(
            LOCAL_STORAGE_TODO_KEY,
            JSON.stringify(stuffTodo)
        );
    }
}

```

```

// gets the updated list
    window.location.reload();
  }
  else
  {
    // there are items already saved in storage, so
    // grab them n add to the array to be saved (this is
    // // coz when saving anything to localStorage, prev
    // data is overridden)
    JSON.parse(localStorage.getItem(
      LOCAL_STORAGE_TODO_KEY)).map(
      todoFromStorage => {
        // push in all items
        // from local storage
        // into our array as well as the new
        // todoList item the user just
        // submitted
        stuffTodo.push(todoFromStorage);
      }
    );

    // push in the newly submitted todoList item as well
    // stuffTodo.push(somethingTodoObj);
    // save them all to the local storage again
    localStorage.setItem(
      LOCAL_STORAGE_TODO_KEY,
      JSON.stringify(stuffTodo)
    );

    // refresh page so the landing page part of the
    // code gets the updated list
    window.location.reload();
  }
}
e.preventDefault();
}

function getId()
{
  if ((JSON.parse(localStorage.getItem(
    LOCAL_STORAGE_TODO_KEY)) === null) ||
    (JSON.parse(localStorage.getItem(
      LOCAL_STORAGE_TODO_KEY)).length == 0))
  {
    return 1;
  }
  else
  {
    // JSON.parse() returns an
    // array, so get the ID of the
    // last element and increment it by 1

```



```

LOCAL_STORAGE_TODO_KEY))
    [JSON.parse(localStorage.getItem(
        LOCAL_STORAGE_TODO_KEY
    )).length - 1].id);

let idFigure = IdNum + 1;
return idFigure;
}
}

```

How to update data in LocalStorage

```

function saveEdit(e)
{
    //grab the edit form's parent div
    let editFormDiv = e.target.parentNode;
    let todoId = parseInt((editFormDiv.id).split('editFormDiv')
        [1]);

    let savedValue = e.target.children[0].value;

    if (savedValue !== '')
    {
        let itemsLocal = [];

        // grab all existing todoList items from storage except
        // the one being edited
        JSON.parse(localStorage.getItem(
            LOCAL_STORAGE_TODO_KEY)).map(
            todoFromStorage => {

                // get the matching todoList item by ID and
                // update its text
                if (todoFromStorage.id == todoId) {
                    todoFromStorage.name = savedValue;
                    itemsLocal.push(todoFromStorage);
                }
                else
                {
                    itemsLocal.push(todoFromStorage);
                }
            }
        );

        // reset localStorage todoList data
        localStorage.setItem(LOCAL_STORAGE_TODO_KEY,
            JSON.stringify(itemsLocal));

        let todoParent = e.target.parentNode.parentNode;

        // get the the first element inside the parent li item,
        // which is the span containing the todoList item value
        // and replace its value with the new one
        let todoItem = todoParent.children[0];
    }
}

```

```

// hide the form div again
    editFormDiv.style.display = 'none';
  }
  else
  {
    alert('You did not enter anything!');
  }
  e.preventDefault();
}

```

How to delete data from LocalStorage

This deletion of data from localStorage is done by the method `removeItem()`. You pass it the key of the storage item/data you wish to delete. Here is the syntax:

```
localStorage.removeItem("keyName");
```

This will delete only the item associated with "keyName", leaving other stored data intact. But if you want to clear everything from the localStorage, then use the `clear()` function instead. Here is its usage syntax:

```
localStorage.clear();
```

This completely wipes everything stored in localStorage.

Here is an example of removing an item from our example todo list.

```

function deleteTodo(e)
{
    // get the todoList item ID from the delete button that
    // was clicked
    let delId = (e.target.id).split('deleteButton')[1];
    if (JSON.parse(localStorage.getItem(
    LOCAL_STORAGE_TODO_KEY)) !== null)
    {
        let itemsLocal = [];
        JSON.parse(localStorage.getItem(
        LOCAL_STORAGE_TODO_KEY)).map(
        todoFromStorage => {
            if (todoFromStorage.id !== delId) {
                itemsLocal.push(todoFromStorage);
            }
        });

        //reset localStorage todoList data or delete it if it's
        // empty
        if (itemsLocal.length !== 0) {
            localStorage.setItem(
            LOCAL_STORAGE_TODO_KEY,
            JSON.stringify(itemsLocal));
        }
    }
}

```

```

{
    localStorage.removeItem(
        LOCAL_STORAGE_TODO_KEY);
    }
}

// get rid of the <li> elem in the browser
let item = e.target.parentNode.parentNode;

item.addEventListener('transitionend', () => {
    item.remove();

    // check if the todoitems are
    // less than two and hide the
// clear all button
    if (ul.children.length < 2)
    {
        hideClearBtn();
    }
});

item.classList.add('todo-list-item-fall');
}

```

Persisting with Session Storage

sessionStorage is stored and retrieved in exactly the same way as local storage, as in, they both have write and read methods of the same names. The only diff is that session storage variables stored on the client computer get wiped out when the user closes their browser. Here is an example:

```

const myData = 'has_visited_my_business';

// retrieve data
const has_visited = () => !
    sessionStorage.getItem(myData);

// store data
const saveToStorage = () =>
    sessionStorage.setItem(myData, true);

```

To remove an item from sessionStorage, do it like so:

```
sessionStorage.removeItem("keyName");
```

This will delete only the item associated with "keyName", leaving other stored data intact. Alternatively, to clear everything from the sessionStorage, do it like so:

```
sessionStorage.clear();
```

Notice how sessionStorage has the same getItem(), setItem(), removeItem() and clear() methods as localStorage has, for retrieving, storing, and deleting data, res-

pectively. Use that to check for data you have stored on the user's computer while they are browsing your web page. This data is a string and a value. Websites use that data creatively; for example you can use it to detect certain user behaviour on your web page and tailor the data displayed to the user's behaviour or perceived needs.

Persisting with Cookies

Cookies are the third way of keeping data on the visitor's machine, and the oldest of the three. Unlike `localStorage` and `sessionStorage`, a cookie is sent back to the server automatically with every request, which is what makes it useful for things like keeping somebody logged in. You will find the three compared side by side at the start of this chapter. Cookies are written and read quite differently from `localStorage` and `sessionStorage`, and they have enough to them - expiry dates, paths, the `Secure` and `HttpOnly` flags - to deserve a chapter of their own. That is Chapter 8 (Cookies), which covers setting a cookie, reading one back, finding a single cookie by name, deleting one, and a worked cookie consent popup that can be switched between all three storage types with a single line. So rather than repeat it here, treat Chapter 8 as the full reference for cookies, and this chapter as the place that shows you where they sit among the other ways of persisting data.

DATABASES

So far in this chapter, you've learned how to store data on the frontend — using tools like Local Storage, Session Storage, and Cookies. These are great for small-scale persistence, like keeping a user logged in, saving theme preferences, or temporarily storing form data. But what if your app grows? What if you want to store and manage data that multiple users can access, even after the browser closes or the device restarts? That's where databases come in.

What is a database

A database (or DB) is a system for storing, organising, and retrieving data—usually on a server, not just inside a browser. Think of it like a filing cabinet for your web app, where you can keep things like:

- User accounts
- Product listings
- Orders and invoices
- Messages and comments

JavaScript and databases

JavaScript applications can use databases, but not directly, because as I said, databases run on servers, not in a browser. To work with a real database, there has to be server-side code (written with something like Node.js, PHP or Go). JavaScript can send requests to this backend using APIs, and that backend will handle communication with the database and return results to the frontend. For example:

```
db.query("SELECT * FROM products WHERE category = 'books'");
```

In the example above, the string inside the `query()` method is called an SQL query string. It's a command sent to the database that tells it what data to fetch.

"SELECT * FROM products WHERE category = 'books'"

SQL (Structured Query Language) is very readable and beginner-friendly — especially for common operations like selecting, inserting, updating, or deleting data. As you can see, the query reads like a regular English phrase. Here's what that query means:

- SELECT means "get data."
- ◦ means "get all columns (fields)."
- FROM products means we're pulling from the products table.
- WHERE category = 'books' filters the results to only items in the "books" category.

So we are telling the database that we want it to search (query) the products table (which is like a storage shelf or cabinet), and get for us all books that are in the 'books' category. The SELECT part of the query translates to 'get for me'. The asterisk (*) character is a wildcard which translates to ALL—meaning we are telling the database to search through the entire collection of products. The WHERE clause is the part that specifies the condition of the query—in this case the condition is to limit the products returned to those from the 'books' category.

Relational vs NoSQL Databases

Relational databases store data in tables (like spreadsheets). Each row is a record, and each column represents a field. Common relational databases include:

- MySQL
- PostgreSQL

For example, in a products table, each row might be a product, and each column could store things like name, price, and category. In contrast, NoSQL databases

like MongoDB and Firebase store data as documents — often in JSON-like format. These documents are grouped in collections rather than tables. This is often faster, more flexible, and works well with JavaScript because:

- JS uses objects,
- NoSQL uses JSON—which looks like JS objects!

This similarity makes MongoDB especially popular in JavaScript ecosystems.

Common database stacks you may encounter

In web development, you'll often hear about stacks, which are acronyms formed from the combinations of technologies used to build full apps. Here are examples of development stacks:

- LAMP: Linux + Apache + MySQL + PHP. MAMP is the same set on macOS, and WAMP on Windows.
- MERN: MongoDB + Express.js + React + Node.js.
- MEAN: MongoDB + Express.js + Angular + Node.js.

Each stack uses a specific type of database. For example, MERN and MEAN use MongoDB, a NoSQL database.

Why JavaScript doesn't talk directly to databases

For security reasons, frontend JavaScript doesn't connect directly to databases. Instead, it sends HTTP requests to a backend API, and the backend:

- Receives the request,
- Talks to the database,
- Returns the results to the frontend.

Frameworks like React, Vue, and Angular do this behind the scenes using APIs.

Popular databases for JavaScript developers

There are many database systems out there. Here are a few that are popular in the JavaScript world:

Type	Name	Description
NoSQL	MongoDB	Stores data as flexible JSON-like documents. Great for JavaScript because the data format matches JS objects.
SQL	MySQL	Traditional relational database. Great for apps with structured, interrelated data.
SQL	PostgreSQL	Another powerful relational DB. Supports complex queries and data relationships.
NoSQL	Firebase	A cloud-hosted NoSQL database from Google. Great for real-time apps and mobile/web projects.
Hybrid	SQLite	A lightweight, file-based DB often used in mobile and desktop apps.
Key-Value	Redis	Often used for caching or temporary storage, not as a primary DB. Fast and lightweight.

Don't worry if databases seem advanced right now — this is just your first step into the world of data. This book focuses on frontend storage like Local Storage and Cookies, while databases are a larger, separate topic. However, as you grow in your development journey, you'll almost certainly encounter databases — whether you're building apps, APIs, or full-stack platforms. If you explore backend development (like Node.js), you'll eventually learn how to:

- Connect to databases,
- Save and retrieve data,
- Build scalable, data-driven applications.

This section gives you a taste of what's ahead — so when you come across databases later, they won't feel like complete strangers!

QUIZ — Databases and Storage

This page contains the Q & A (questions and answers) for this chapter — Chapter 13: Databases and Storage. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 8 to 10 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Name the three ways of storing data in the visitor's browser covered in this chapter, and give the one thing that most distinguishes each from the other two.

Clue: one survives everything, one dies with the tab, and one gets posted to the server whether you like it or not.

2. You want to remember a visitor's chosen theme, dark or light, so it is still there when they come back next week. Which of the three would you use, and why not the other two?

Clue: next week is a long time. And the server does not need to know.

3. Which of the three is sent to the server with every request, and why is that both its greatest strength and its main cost?

Clue: think about what has to travel with every single image, script and page your site loads.

4. `localStorage` and `sessionStorage` both offer four methods with the same names. Name them and say what each does.

Clue: `get`, `set`, `remove`, and one that clears the lot.

5. Why can JavaScript running in a browser not talk directly to a database, and what has to sit in between?

Clue: the answer is about security, and about where databases actually live.

6. Read this SQL query and say in plain English what it asks for:

```
SELECT * FROM products WHERE category = 'books'
```

Clue: three of the four words do most of the work — one picks, one says where from, and one narrows it down.

7. None of the three storage types is a safe place for sensitive data. Why not?

Clue: think about whose computer the data is sitting on.

8. EXERCISE. Save a visitor's theme preference to `localStorage`, read it back, and print it. Then delete it and show that it has gone.

Clue: you need three of the four methods from question 4.

9. EXERCISE. `localStorage` can only hold text. Save an array of three to-do objects, then read it back as a real array and print the second one's name.

Clue: two methods with JSON in the name — one on the way in, one on the way out.

10. EXERCISE. Write a small helper called `remember(key, value)` that saves to `localStorage`, and `recall(key)` that reads it back, returning null if nothing was stored under that key.

Clue: `getItem` already returns null when it finds nothing, which does half the job for you.

ANSWERS

1. **localStorage** — stays until you delete it. Closing the browser makes no difference.

sessionStorage — wiped the moment the tab is closed.

Cookies — you set the expiry yourself, and they are sent to the server automatically with every request.

The clearest way to separate them is by how long the data lives and who can see it. `localStorage` and `sessionStorage` never leave the browser. Cookies do.

2. **localStorage.**

Not `sessionStorage`, because that is emptied as soon as the tab closes, so it would be gone long before next week.

Not a cookie, because the server has no need to know which theme they picked. Sending that choice to the server with every single request would be waste, and cookies are limited to about 4KB in any case.

```
localStorage.setItem("theme", "dark");
```

3. **Cookies.**

The strength is that the server sees them without you doing anything. That is exactly what you want for a login token: the server can tell who is asking on every request, without the page having to attach it each time.

The cost is that "every request" means

every

request — every image, every stylesheet, every script, not just page loads. A large cookie is therefore paid for many times over on every visit. That is part of why the 4KB limit exists.

4.

<code>setItem(key, value)</code>	// store something
<code>getItem(key)</code>	// read it back, or null if there is nothing
<code>removeItem(key)</code>	// delete one item
<code>clear()</code>	// delete everything for this site

Both storage types offer all four with identical names, which is why you can swap one for the other by changing a single word.

5. Because **databases live on servers, not in browsers**, and because letting the browser connect directly would be a serious security hole. Anything the browser can do, a visitor can do — they could read the connection details out of your JavaScript and then help themselves to the whole database.

What sits in between is **server-side code**, written with something like Node.js, PHP or Go. The browser sends an HTTP request to that backend, the backend talks to the database, and it sends the results back.

So the chain is: browser → API → server code → database, and back again.

6. "Get me every column, from the products table, for the rows whose category is 'books'."

Word by word:

- `SELECT` means "get data"
- `*` is a wildcard meaning "all columns"
- `FROM products` says which table to look in
- `WHERE category = 'books'` narrows it to just the rows that match

SQL is deliberately readable, which is one of the reasons it has lasted since the 1970s.

7. Because all three sit **on the visitor's own computer**, in their own browser. The visitor can open the developer tools and read every one of them in a few seconds. So can any script running on the page, which is what makes cross-site scripting attacks so damaging.

Never put passwords, card details or anything private in any of the three. Store a token that the server can check instead, and keep the sensitive data on the server.

```
8. // save it
   localStorage.setItem("theme", "dark");

   // read it back
   console.log(localStorage.getItem("theme"));
   // "dark"

   // delete it
   localStorage.removeItem("theme");

   console.log(localStorage.getItem("theme"));
   // null
```

Notice that reading a key which is not there gives you `null`, not an error and not undefined. That is useful, because it means you can check for it directly:

```
   if (localStorage.getItem("theme") === null) {
       // nothing saved yet, use the default
   }

9. const todos = [
    { id: 1, name: "Buy milk" },
    { id: 2, name: "Walk the dog" },
    { id: 3, name: "Write chapter 13" }
];

// On the way in: turn the array into text
localStorage.setItem("todos", JSON.stringify(todos));

// On the way out: turn the text back into an array
const stored = JSON.parse(localStorage.getItem("todos"));

console.log(stored[1].name);
// "Walk the dog"
```

This is the single most important thing to know about `localStorage`: **it stores text and nothing else**. Hand it an array or an object without `JSON.stringify()` and it will store the string `"[object Object]"`, which is of no use to anybody.

`JSON.stringify()` on the way in, `JSON.parse()` on the way out. Always both.

```

10. function remember(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
}

function recall(key) {
    const stored = localStorage.getItem(key);

    if (stored === null) {
        return null;
    }

    return JSON.parse(stored);
}

remember("user", { name: "Alice", theme: "dark" });

console.log(recall("user").name);
// "Alice"
console.log(recall("nothingHere"));
// null

```

The null check matters. `JSON.parse(null)` happens to return null without complaining, so this one would survive without the check — but `JSON.parse()` throws a `SyntaxError` on most other rubbish it is given, so checking first is the habit worth having.

Using `JSON.stringify()` on everything, even plain text, keeps the pair symmetrical: whatever you put in through `remember()` comes back out of `recall()` as the same type it went in as.

Chapter 14 — DATES AND TIME

- Date and time in JavaScript
- Create a date
- Formatting Dates
- Working with Time Zones
 - Understanding UTC
 - Handling UTC in JavaScript
 - Converting local time to UTC
 - A trap worth knowing about
 - What `Date.UTC()` is actually for
 - Convert UTC to local time
- Working with Timers: `setTimeout()` and `setInterval()`
 - `setTimeout()`
 - `setInterval()`
 - Real-life `setInterval` use case for Dates
 - Stopping a timer
 - Performing animations with the timer functions
 - Make a ball move across the screen
 - Create a slideshow of images
- A datepicker library in vanilla JS & Bootstrap

This has to do with how dates and time are managed by your programming language, including timezone configurations and formatting of the date and time values displayed to users.

Date and time in JavaScript

JavaScript provides a built-in `Date` object for handling dates and times. The `Date` object allows you to create, manipulate, and format dates, making it essential for tasks like scheduling events, logging timestamps, or working with time zones. JavaScript dates are based on milliseconds since January 1, 1970 (UTC) (the Unix epoch). The `Date` object works with both local time and UTC (Coordinated

Universal Time). Unlike some other languages, JavaScript's Date object is mutable, meaning its values can change after creation.

Create a date

You can create a date in several ways, here are some examples:

```
// Current date & time
let now = new Date();

// Mar 16, 2024 (Months are 0-indexed)
let specificDate = new Date(2024, 2, 16);

// ISO 8601 format (UTC)
let fromString = new
  Date("2024-03-16T12:00:00Z");

// Using milliseconds
let fromTimestamp = new
  Date(1710590400000);
```

Formatting dates

JavaScript provides basic date formatting with various methods of the Date object. The way you use them is to first of all create a date—which will be an instance of the Date object, then call any of the formatting methods on it to format the date. Here are the Date formatting methods in action:

```
let now = new Date();

// Make a string of the date eg: "Sat Mar 16
// 2024"
console.log(now.toString());

// show the date in terms of time eg:
// "12:34:56 GMT+0200"
console.log(now.toTimeString());

// convert to an ISO string eg:
// "2024-03-16T10:34:56.789Z" (UTC)
console.log(now.toISOString());

// Formats based on user's locale
console.log(now.toLocaleString());
```

However, for advanced formatting, Intl.DateTimeFormat is recommended. For example:

```

let now = new Date();

let formatter =
  new Intl.DateTimeFormat(
    "en-US",
    {
      weekday: "long",
      year: "numeric",
      month: "long",
      day: "numeric"
    }
  );

// this will print to the console something
// like this: "Saturday, March 16, 2024"
console.log(formatter.format(now));

```

As you can see, the way the `DateTimeFormat()` method of the `Intl` object works, is as follows:

- the first argument is the locale, which is a language-and-region code rather than a place (en-US is English as used in the USA)
- the second argument is an object literal in which you specify how you would like the various components of your date to be displayed. The components of a date include the day, weekday (different from day), the month, the year etc. In this case, we specify that we want the weekday to be long meaning, we want it to be spelled out fully (so Sunday, not Sun), we also specify that we want the day and the year to be numeric—which are standard anyway, and then we also indicate that we want the month to be long, meaning we want it spelled out fully as in, for example March instead of Mar.

Working with Time Zones

Understanding UTC

UTC (Coordinated Universal Time) is the primary time standard used worldwide to keep clocks and time zones synchronized. It is not affected by daylight savings time or local time zones. When you create a `Date` object in JavaScript using `new Date()`, it automatically adjusts the date and time to match your computer's local time zone. But sometimes, especially in international applications, you need a consistent, universal reference for time that does not change based on the user's location. That's where UTC comes in. Let us look at an example.

```

let now = new Date();

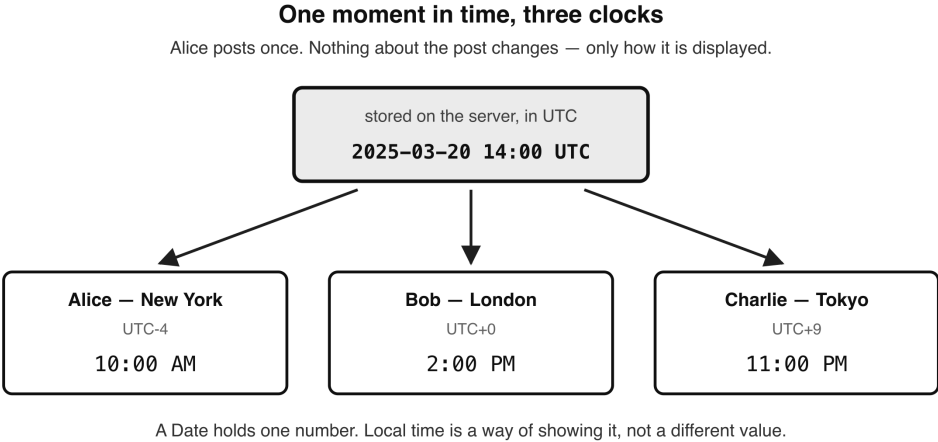
console.log(
  "Local Time:", now.getFullYear(), now.getHours()
);

console.log("UTC Time:", now.getUTCFullYear(),
  now.getUTCHours()
);

```

What happens here is that `getFullYear()` and `getHours()` return values in your local time zone while `getUTCFullYear()` and `getUTCHours()` return values in Coordinated Universal Time (UTC). So, for example, if you're in New York (UTC-4) and your local time is 10:00 AM, then `now.getHours()` will produce 10, while `now.getUTCHours()` will produce 14 (14:00). This is because UTC is 4 hours ahead of New York time. This may seem confusing at first. On international apps, it is still important for the application to display dates based on specific user times, depending on where in the world they are located. Let's use Facebook as a case study. When a user makes a post, all their friends (contacts) around the world still need to see what time the post was made relative to their local times, which will vary for the different contacts depending on where they are. So if UTC means that all users worldwide will have the same time, that surely cannot be right, because it's not practical. Yes it is definitely not practical, and users should indeed be shown the time that matches their local time zones. However, on computer servers behind the scenes, if everyone stored and processed dates in different time zones, it would create inconsistencies across different users and systems. For example, imagine that a user Alice in New York (UTC-4) makes a post at 10:00 AM her time. At the exact same moment, Bob in London (UTC+0) sees it on his feed. If Facebook only stored local times exactly as it is in their databases, then it would save Alice's post time as 10:00 AM in their database, and the problem is that all users all over the world will see 10:00 am which will be wrong. That wouldn't make sense to Bob, because 10:00 AM New York time would have been 2:00 PM in London. This tells you that some conversion is needed when it comes to times. There needs to be a mechanism that saves data with some type of reference that will let the system know what the local time is for Alice in New York, and be able to work out and display the right local date and time to any user, anywhere on the globe, based on the local time where the post was made. This is where UTC comes in. UTC solves this problem by acting as a universal reference point. Think of UTC as the one true time that the servers agree on, with every user's local time being that same moment dressed up for where they happen to be. Facebook (or any global platform) in code by convention would store all timestamps in UTC, then convert them when displaying times to each user. So, for our example above, Facebook would store the timestamp of

Alice’s post in UTC: 2025-03-20 14:00 UTC. When Bob views the post in London (UTC+0), it will display 2:00 PM. When Alice views it in New York (UTC-4), it displays 10:00 AM. When Charlie in Tokyo (UTC+9) sees the same post, it will show 11:00 PM. This way, the same timestamp (UTC) is correctly adjusted for every user.



• Figure 14.1 — One moment in time, three clocks*

Let’s talk about some cases where UTC is useful. When it comes to timestamps and databases, UTC ensures that time records are consistent and don't get mixed up by daylight saving or different time zones. This is particularly useful for global applications that need to store user actions (posts, messages, payments, etc.). This is why most database systems like PostgreSQL and MySQL etc all store timestamps in UTC by default when using the `TIMESTAMP` or `DATETIME` data types. This is done to ensure consistency across different time zones. When you insert a timestamp without specifying a time zone, the database usually assumes it's in UTC. If you store a timestamp with a time zone (e.g., `TIMESTAMPTZ` in PostgreSQL), the database converts it to UTC internally but can return it in the local time zone when queried. When querying that data from such backend database systems, applications (like a JavaScript frontend eg via APIs) can convert the UTC timestamps to the user's local time zone for display. Why do databases use UTC? Here are some key reasons:

- Consistency—a globally standardised time avoids confusion when multiple users across different time zones access the same data.
- Avoids daylight saving issues—since UTC does not change with daylight saving time, it prevents errors in time calculations.
- Easier calculations—time differences are easier to compute without worrying about time zone offsets. Another system where timezone conversion occurs is with airline tickets. When booking a flight from New York to London, for

example, the airline must show your departure time to you in New York time and your arrival time in London time, otherwise it will not make sense. Behind the scenes however, what the airline's system is actually doing is storing everything in UTC and converting it for every passenger. During debugging and logging of computer system faults, using the right date and time is crucial too. If an application logs errors or user activity, UTC ensures all events are in the same reference frame, making debugging easier. Live events like sports and webinars with participants from different time zones taking part need to get the timing right too. A global live event (e.g., FIFA World Cup) must be scheduled in UTC.

Handling UTC in JavaScript

When we use the `Date` object in JavaScript, the date is displayed to us in the user's own time zone, but it can be read back in UTC whenever we need it, using methods like `getUTCFullYear()`. So, when you create a `Date` object in JavaScript like this:

```
let now = new Date();
console.log(now);
```

It automatically shows you the date and time in the user's local time zone. But if you need to read that same moment as UTC, you can use methods like:

```
now.getUTCFullYear(); // UTC year
// UTC month (0-based, Jan = 0)
now.getUTCMonth();
// UTC day of the month
now.getUTCDate();
now.getUTCHours();    // UTC hours
now.getUTCMinutes();  // UTC minutes
now.getUTCSeconds();  // UTC seconds
```

These methods return the same moment in time but adjusted to UTC instead of the user's local time.

Converting local time to UTC

Here is the thing that catches almost everybody out when they first meet the `Date` object: there is nothing to convert. A `Date` does not store a local time. Inside itself it holds a single number—the count of milliseconds since January 1, 1970 UTC, which we met at the start of this chapter. That number is exactly the same number whether the computer running your code is in New York, London or Tokyo. What changes from one country to the next is not the date, but how it is displayed. So the moment you write this:

```
let localTime = new Date();
```

you already have a UTC time. You do not convert it to UTC—you just ask it to show itself as UTC whenever you want that:

```
let localTime = new Date();

// one instant, printed two different ways
console.log("Local Time:", localTime.toString());
console.log("UTC Time:", localTime.toUTCString());
```

The output for a user in New York (UTC-4) will look like this:

Local Time: Thu Mar 20 2025 12:00:00 GMT-0400 (Eastern Daylight Time) UTC
Time: Thu, 20 Mar 2025 16:00:00 GMT

Notice how UTC is 4 hours ahead of New York time. But notice something more important than that: we only called `new Date()` once. There is a single moment in time here, and we printed it twice. Reading individual pieces off the date works the same way. That is what the `getUTC...` methods under the ‘Handling UTC in JavaScript’ heading above are for:

```
let now = new Date();

now.getHours();
// 12 – the hour where the user is
now.getUTCHours();
// 16 – the same instant, read as UTC
```

One date. Two ways of reading it. No conversion anywhere.

A trap worth knowing about

Sooner or later you will run into code that tries to convert to UTC like this. It looks perfectly sensible, and it is wrong:

```
// this does NOT do what it appears to do
let utcTime = new Date(
  Date.UTC(
    localTime.getFullYear(),
    localTime.getMonth(),
    localTime.getDate(),
    localTime.getHours(),
    localTime.getMinutes(),
    localTime.getSeconds()
  )
);
```

Read slowly through what that actually does. `getHours()` hands you the hour on the user’s own clock—12, for somebody in New York at midday. `Date.UTC()` then

takes that 12 and treats it as though it had been 12 o'clock in UTC all along. What comes back is not your moment converted. It is a different moment altogether, four hours earlier than the one you started with. Print it and you get:

Thu, 20 Mar 2025 12:00:00 GMT

not the 16:00 you were expecting. The numbers on the clock face were copied across, and the time zone they belonged to was quietly thrown away.

What `Date.UTC()` is actually for

`Date.UTC()` does have a real job. It is simply not converting. Its job is the opposite direction—building a date out of components that you already know are in UTC. Compare these two lines:

```
// "the 20th of March 2025 at
// 16:00, wherever this code runs"
let a = new Date(2025, 2, 20, 16, 0, 0);

// "the 20th of March 2025 at 16:00 UTC, everywhere"
let b = new Date(Date.UTC(2025, 2, 20, 16, 0, 0));
```

The first one depends on where it runs. Run it in New York and it means 20:00 UTC. Run the very same line in Tokyo and it means 07:00 UTC—a different moment entirely, from identical code. The second one means one single moment everywhere on earth, no matter where it runs. What `Date.UTC()` gives you is the milliseconds for those components read as UTC, and handing that number to `new Date()` turns it back into a `Date` object. This is exactly what you want when a date arrives from somewhere that has already told you it is in UTC—a database record, or a response from an API—and you need to rebuild a `Date` from its parts. The rule to hold on to is this: a `Date` is a moment, and a moment is always stored in UTC. Local time is a way of displaying that moment, not a different kind of value.

Convert UTC to local time

Let's say we have a UTC timestamp and need to show it in the user's local time:

```
// UTC time
let utcDate = new Date("2025-03-20T16:00:00Z");

console.log("UTC Time:", utcDate.toUTCString());

// converts to the user's local time
console.log("Local Time:", utcDate.toString());
```

Here is the output for a user, who is for example, in New York (UTC-4):

UTC Time: Thu, 20 Mar 2025 16:00:00 GMT

Local Time: Thu Mar 20 2025 12:00:00 GMT-0400 (Eastern Daylight Time)

So basically, to convert a UTC time to local time we just need to call the `.toString()` method on the `Date` object. JavaScript automatically adjusts the time to match the user's system time zone.

Working with Timers: `setTimeout()` and `setInterval()`

When working with time in JavaScript, it's not just about getting the current time or formatting a date—sometimes you want to run actions after a specific time delay, or repeatedly at timed intervals. That's exactly what `setTimeout()` and `setInterval()` allow you to do.

`setTimeout()`

This function lets you run code once after a specified delay (in milliseconds). Think of it like saying: "Do this, but wait a little bit first."

```
// Waits 2 seconds (2000 milliseconds)
// and runs this code once
setTimeout(() => {
  console.log("Hello after 2 seconds");
}, 2000);
```

Such functionality would be useful for the following:

- Showing a splash screen and then hiding it after a few seconds
- Delaying a tooltip
- Performing cleanup tasks after an action
- Staggering animations

`setInterval()`

This one keeps repeating the action after each time interval—until you tell it to stop. Basically, it will run code repeatedly at intervals of the time period in milliseconds, as specified by the second parameter. Here is its syntax:

```
setInterval(callback, ms);

// Logs "Ping!" every second
const intervalId = setInterval(() => {
  console.log("Ping!");
}, 1000);
```

You can program it to stop the cycle by using `clearInterval(intervalId)`. Here are some scenarios where such a helper function will be useful:

- Clocks or countdown timers
- Polling a server every few seconds
- Repeated animations (like a blinking element, or bouncy ball)
- Updating the UI on a regular schedule

Real-life setInterval use case for Dates

Let's say you want to build a live digital clock. This is a perfect job for `setInterval()`.

```
<p id="clock"></p>

<script>
  function updateClock() {
    const now = new Date();
    const time = now.toLocaleTimeString();
    document.getElementById("clock").textContent = time;
  }

  // Call immediately to avoid delay
  updateClock();

  // Update every second
  setInterval(updateClock, 1000);
</script>
```

This updates the clock on your webpage every second, giving your users a real-time view of the current time.

Stopping a timer

Both `setTimeout()` and `setInterval()` return an ID that you can use to cancel them:

```
const timeoutId = setTimeout(() => {
  console.log("This will NOT show");
}, 5000);

// cancels the above timeout
clearTimeout(timeoutId);

const intervalId = setInterval(() => {
  console.log("Still running...");
}, 1000);

setTimeout(() => {
```

```
// stops the repeated action after 5 seconds
clearInterval(intervalId);
}, 5000);
```

Both `setTimeout()` and `setInterval()` use time in milliseconds (not seconds), so: *
1000 ms = 1 second * 5000 ms = 5 seconds * 60000 ms = 1 minute

Performing animations with the timer functions

In the introduction to the timers—`setTimeout()` and `setInterval()`—I mentioned that they are very useful for creating animations and repeated actions that are time-based. Whether you want to code a quick image slideshow to impress your family and friends, or build the next Pacman game, these two functions have got what you need. As always, I have to let you see that in action with some simple, but practical examples.

Make a ball move across the screen

We are going to create a circle on screen which will be nothing other than a div that we will style to appear as a circle. Next, with JavaScript, we will use X and Y position coordinates—which you always have to deal with whenever you want to deal with objects moving on the screen—to make the ball start from the left edge of the screen and move towards the right edge. Once the ball arrives at the right edge of the screen, it will start moving in the opposite direction back towards the left of the screen. When it hits the left edge, it will switch direction again and keep repeating that cycle. What makes the ball move is simply the value of the `left` property of the ball's (div) style that we will keep increasing at intervals in JavaScript. The `left` style property of an element is essentially the distance the element sits from the left edge of whatever contains it — for our ball, which is positioned absolutely, that is the browser window itself. By increasing that value, the element will keep moving towards the right of the screen. The `right` property of the style of an element is the direct opposite of that. JavaScript is also used to detect when the object hits the edge of your browser window, so we can make it switch direction.

CSS code

Here is the code to style the ball

```
#ball {
  width: 50px;
  height: 50px;
  background-color: red;
  border-radius: 50%;
  position: absolute;
```

```
top: 100px;
  left: 0;
}
```

HTML code

Create the div element that will be the ball

```
<div id="ball"></div>
```

JavaScript code

```
const ball = document.getElementById("ball");
// starting left position
let position = 0;
let direction = 1;
// 1 means right, -1 means left
const speed = 5; // pixels per frame

const intervalId = setInterval(() => {
  position += speed * direction;
  ball.style.left = position + "px";

  // detect edges
  if (
    position + ball.offsetWidth >= window.innerWidth ||
    position <= 0
  )
  {
    // reverse direction
    direction *= -1;
  }

  // run every 20 milliseconds (~50 frames per second)
}, 20);
```

Let me explain this code. The magic really happens here:

```
if (
  position + ball.offsetWidth >= window.innerWidth ||
  position <= 0
)
{
  // reverse direction
  direction *= -1;
}
```

This is because it checks whether the ball has reached the edge of the screen, and if it has, it makes the ball turn around (move in the opposite direction). This is a conditional statement that does two checks. Let me start by describing what all the parameters and variables are meant to do. Firstly, the 'position' variable is used to

set the current horizontal position (in pixels) of the ball from the left side of the browser window. We use 0 because we want it to start from the left edge of the window.

'ball.offsetWidth' gives the width of the ball, including borders. We need this because the ball takes up space — we don't want its right edge to go past the right edge of the screen.

'window.innerWidth' is the width of the visible part of the browser window (the viewport).

Two conditions are as follows:

```
position + ball.offsetWidth >= window.innerWidth
```

This checks if the right edge of the ball has reached (or passed) the right edge of the screen.

If position = 750, and ball.offsetWidth = 50, then $750 + 50 = 800$ — and if the window is also 800px wide, the ball is touching the right edge.

The second condition is:

```
position <= 0
```

This checks if the left edge of the ball has reached (or passed) the left edge of the screen. This happens when position is 0 or less — meaning the ball is fully against the left wall.

If this returns true, then we reverse the direction:

```
direction *= -1;
```

Which is short for:

```
direction = direction * -1;
```

So if direction was 1 (moving right), now it becomes -1 (move left). If it was -1, now it becomes 1.

The movement is then done in multiples of the speed (pixels per frame), which is 5.

```
position += speed * direction;  
ball.style.left = position + "px";
```

Ultimately, the movement happens because we are increasingly setting the ball.style.left value. That on its own will keep an object moving, because each time we do it, the value is applied as an increment to its current position which will be changing each time we increment it. The repetition is made possible because that code is wrapped inside a setInterval() function:

```
const intervalId = setInterval(() => {
    position += speed * direction;
    ball.style.left = position + "px";
    // ...
}, 20);
```

Create a slideshow of images

This will use `setInterval` at intervals of 3 seconds (3000 milliseconds) to cycle through a collection of images that you have listed in an array, and also have the images locally in a folder 'images'.

HTML code

Create the `<img>` tag in your HTML code. Notice I have given it a width and height of 500 by 300, and that is relative to the images I have. Tip: to make it work nicely, make all the images have the same dimensions, so the slideshow will flow seamlessly.

```
<img id="slideshow" src="" width="500" height="300" />
```

JavaScript code

```
const images = [
    "/images/blurred-image.png",
    "/images/brightened-image.png",
    "/images/contrasted-image.png",
    "/images/grayscale-image.png",
    "/images/resized-image.png",
];

// start at the first image
let index = 0;
const imgElement = document.getElementById("slideshow");

// Set the first image
imgElement.src = images[index];

// Change image every 3 seconds (3000 milliseconds)
setInterval(() => {
    // move to next image
    index++;

    // if we've reached the end,
    // start again at the beginning
    if (index >= images.length) {
        index = 0;
    }
}, 3000);
```

```
// update image
  imgElement.src = images[index];
}, 3000);
```

This piece of code is pretty simple, but there are four key parts to it. They are as follows:

- We need to start by setting the src attribute of the image element to the first image in the array (the element at index 0):

```
imgElement.src = images[index];
```

This is why we had to initialise the value of index to 0.

- In the setInterval() function, you need to increment the value by 1. That is how a different image will be displayed each time from a different index of the array.

```
index++;
```

- If you update the index value, then you have to update the src reference of that index in the image element

```
imgElement.src = images[index];
```

- Finally, you must perform some kind of check to make sure that if the cycling through the images has come to the last image in the array, then it should restart from index 0. This is why we reset the value of index to 0 within this conditional that ascertains that the current index is either greater than or equal to the number of items in the images array.

```
if (index >= images.length) {
  index = 0;
}
```

In these animation examples, you have learned the following:

- Arrays and indexes
- DOM manipulation using .src
- Time-based actions with setInterval()
- Cycling (looping back to start)

A datepicker library in vanilla JS & Bootstrap

- Use this awesome JS library: <https://mymth.github.io/vanillajs-datepicker>
- You can do all sorts eg inline date pickers, date ranges etc. It's about copying and pasting the CDN links into your code in the JavaScript sections of your

HTML page, then pasting in the JavaScript code that calls the datepicker class. For a date range, I did something like this:

```
<input type="date" id="dateField" name="dateFields" />

const elem = document.getElementById("dateField");

const rangePicker = new DateRangePicker(
    elem,
    {
        format: "dd-mm-yyyy"
    }
);
```

QUIZ — Dates And Time

This page contains the Q & A (questions and answers) for this chapter — Chapter 14: Dates And Time. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 12 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What date does this create, and why is it not the 3rd of March?

```
let d = new Date(2024, 2, 16);
```

Clue: JavaScript counts months the way it counts array indexes.

2. A Date object is often described as holding "a local time". Why is that wrong, and what does it actually hold?

Clue: one number, counted from a fixed moment in 1970.

3. You have one Date. Name the method that shows it in the user's own time zone and the one that shows it in UTC.

Clue: one of them is the plain, ordinary one you would guess.

4. This code claims to convert a local time to UTC. It does not. What does it really do?

```
let utcTime = new Date(
  Date.UTC(
    localTime.getFullYear(),
    localTime.getMonth(),
    localTime.getDate(),
    localTime.getHours()
  )
);
```

Clue: think about what getHours() hands you, and what Date.UTC() then assumes about it.

5. So what IS Date.UTC() for? Give the one situation where you genuinely want it.

Clue: it builds rather than converts.

6. What is the difference between `setTimeout()` and `setInterval()`, and what unit do they both take their delay in?

Clue: one happens once. And the unit trips up everybody at least once.

7. Both timers hand you something back when you call them. What is it, and what is it for?

Clue: you cannot cancel something you cannot name.

8. Why do database systems like PostgreSQL and MySQL store timestamps in UTC rather than in local time?

Clue: think about two users in different countries looking at the same record.

9. EXERCISE. Create a date for the 25th of December 2025 at 9:30 in the morning, then print it as a readable date string.

Clue: remember what question 1 taught you about the month.

10. EXERCISE. Take one `Date` and print the same moment twice — once in local time and once in UTC — proving they are the same instant.

Clue: call `new Date()` only once. Compare `getTime()` on it against itself if you want the proof in numbers.

11. EXERCISE. Build a live digital clock that updates every second, then make it stop after ten seconds.

Clue: you need `setInterval` to start it and `setTimeout` to end it — and the ID from the first one.

12. EXERCISE. Write a countdown that starts at 5, prints each number one second apart, and prints "Liftoff!" when it reaches zero — stopping itself cleanly.

Clue: the timer has to cancel itself from inside its own callback.

ANSWERS

1. It creates the **16th of March 2024**, not the 3rd.

```
let d = new Date(2024, 2, 16);
console.log(d.toString());
// "Sat Mar 16 2024"
```

The months are **0-indexed**, exactly like array positions. So January is 0, February is 1, and March is 2. The day of the month, confusingly, is not — the 16 really does mean the 16th.

This is one of the oldest complaints about the `Date` object, and it catches everybody at least once. When you are reading somebody else's code and the month

looks wrong by one, it probably is not wrong at all.

2. A Date does not store a local time, and it does not store a time zone either.

What it holds is a **single number: the count of milliseconds since January 1, 1970 UTC**.

That number identifies one moment in the history of the universe. It is exactly the same number whether the computer running the code sits in New York, London or Tokyo.

The local time you see is produced only when the date is **displayed**. It is a way of dressing that number up for wherever the user happens to be, not a different value.

3.

```
// the user's own time zone
localTime.toString();
// the same instant, as UTC
localTime.toUTCString();
```

`toString()` is the plain one, and it is what you get automatically if you print a Date without calling anything. For a user in New York at midday:

Local Time: Thu Mar 20 2025 12:00:00 GMT-0400 (Eastern Daylight Time) UTC
Time: Thu, 20 Mar 2025 16:00:00 GMT

One date. Two ways of writing it down.

4. It **copies the numbers off the clock face and throws the time zone away**.

`getHours()` gives you the hour where the user is — 12, for somebody in New York at midday. `Date.UTC()` then takes that 12 and assumes it was 12 o'clock *in UTC* all along.

What you get back is not your moment converted. It is a **different moment altogether**, four hours earlier than the one you started with:

Thu, 20 Mar 2025 12:00:00 GMT // what it really gives Thu, 20 Mar 2025 16:00:00 GMT // what people expect

And it is worse than simply wrong, because it looks right. The date is right, the minutes are right, and only the hour is quietly off by the size of your time zone offset — which is zero if you happen to test it in London, so it can pass every test you run and then fail for your users.

The honest answer is that there was never anything to convert. Use `toUTCString()`.

5. `Date.UTC()` is for **building** a date out of components you already know are in UTC.

```
// "16:00, wherever this code happens to run"
let a = new Date(2025, 2, 20, 16, 0, 0);

// "16:00 UTC, everywhere on earth"
let b = new Date(Date.UTC(2025, 2, 20, 16, 0, 0));
```

The first line means different moments in different countries. Run it in New York and it is 20:00 UTC; run the identical line in Tokyo and it is 07:00 UTC. The second line means one single moment no matter where it runs.

So you want it when the parts of a date arrive from somewhere that has already told you they are UTC — a database row, or an API response — and you need to rebuild a `Date` from them.

6. **`setTimeout()`** runs your code **once**, after a delay. **`setInterval()`** runs it **over and over**, once per interval, until something stops it.

Both take their delay in **milliseconds**, not seconds:

```
1000 ms = 1 second
5000 ms = 5 seconds
60000 ms = 1 minute
```

Passing 5 when you meant 5 seconds gives you five
thousandths

of a second, which is effectively instant. It is a rite of passage.

7. They return a **timer ID** — a number identifying that particular timer.

```
const intervalId = setInterval(tick, 1000);
```

It exists so you can cancel the timer later, with `clearInterval(intervalId)` or `clearTimeout(timeoutId)`. Without holding on to the ID there is no way to refer to that timer again, so a `setInterval` you did not capture will keep running for as long as the page is open.

Get into the habit of storing it even when you think you will not need it.

8. Because UTC gives every record **one single reference point**, so the same moment means the same thing to everybody.

If a database stored each user's local time as it arrived, then Alice's 10:00 AM in New York and Bob's 10:00 AM in London would look identical in the table while being four hours apart in reality. Sorting would be wrong, comparing would be wrong, and working out which of two events happened first would be impossible.

Three reasons in particular:

- **Consistency** — one standard time avoids confusion across time zones.

- **No daylight saving problems** — UTC never shifts, so an hour never happens twice or goes missing.
- **Easier calculations** — differences between times are simple subtraction, with no offsets to juggle.

The display conversion happens at the very last moment, when the value is shown to a particular user.

```
9. // remember: month 11 is December, not November
   let christmas = new Date(2025, 11, 25, 9, 30, 0);

   console.log(christmas.toDateString());
   // "Thu Dec 25 2025"
   console.log(christmas.toString());
   // "Thu Dec 25 2025 09:30:00
   // GMT+0000 (Greenwich Mean Time)"
```

If you wrote `new Date(2025, 12, 25)` you would get the **25th of January 2026**, because month 12 has run off the end of the year and JavaScript rolls it over without complaining. No error, just a date a month later than you wanted.

```
10. const moment = new Date();

   console.log("Local Time:", moment.toString());
   console.log("UTC Time:", moment.toUTCString());

   // the proof: one object, one underlying number
   console.log("Milliseconds:", moment.getTime());
   console.log("Local hour:", moment.getHours());
   console.log("UTC hour:", moment.getUTCHours());
```

Run this anywhere outside the UTC+0 zone and the two hours will differ, while `getTime()` gives one number that never changed. That single number is the moment. The two printed times are just two ways of writing it down.

```
11. function updateClock() {
      const now = new Date();
      console.log(now.toLocaleTimeString());
    }

    // start it – and keep the ID
    // so we can stop it later
    updateClock();
    const clockId = setInterval(updateClock, 1000);

    // stop it after ten seconds
    setTimeout(() => {
      clearInterval(clockId);
      console.log("Clock stopped.");
    }, 10000);
```

Two things worth noticing. First, `updateClock()` is called directly once before the interval starts — without that, the display would sit blank for a whole second before the first tick. Second, the whole thing only works because `clockId` was stored; the `setTimeout` needs it to know which timer to cancel.

On a web page the only change is where the time goes:

```
document.getElementById("clock").textContent = now.toLocaleTimeString();
```

```
12. let count = 5;
```

```
const countdownId = setInterval(() => {
  if (count === 0) {
    console.log("Liftoff!");
    clearInterval(countdownId);
    return;
  }

  console.log(count);
  count--;
}, 1000);
```

Output, one line per second:

```
5 4 3 2 1 Liftoff!
```

The interesting part is that the timer cancels itself from inside its own callback, using the ID stored just outside it. That works because `const countdownId` is assigned before the first tick ever runs — a second passes before the callback is called for the first time.

The `return` matters too. Without it, execution would carry on to `console.log(count)` and print a stray 0 after "Liftoff!", because `clearInterval()` stops future ticks but does not abandon the one currently running.

Chapter 15 — DOM AND URL MANIPULATION

- Introduction to the DOM
- Understanding the DOM Tree (Parent-child relationships)
- The difference between Nodes and Elements
- Selecting DOM Elements (Getting elements to work with)
 - The `getElementsByName()` method
 - The `getElementById()` method
 - The `getElementsByClassName()` method
 - The `querySelector()` method
 - The `querySelectorAll()` method
 - Difference between `HTMLCollection`, `NodeList`, and `HTMLElement`
 - Document methods that return `HTMLCollection`
 - Document methods that return `NodeList`
 - Are `HTMLCollections` and `NodeLists` arrays?
 - Looping through `HTMLCollections` & `NodeLists`
 - `for...of` loop
 - the classic `for` loop
 - Using all array methods on `HTMLCollections` & `NodeLists`
 - The `Array.from()` method
 - The spread operator `[...]`
 - Selecting nested elements
 - Get the value or contents of an HTML element
- Traversing the DOM
 - Parent node
 - Child nodes
 - Sibling nodes
 - Example of DOM traversing
 - Scrolling & Focus
- Manipulating elements
 - Changing elements

- Adding/removing elements
- Creating New Elements
- Working with Attributes & Classes
- Handling styling
 - Common styling properties
- Working with positioning
 - Common positioning properties
- Working with events
- XPath and selecting DOM Elements
 - Basic usage of document.evaluate()
 - Use document.evaluate() to read an XML document
 - The XPathResult types
 - Retrieving the data based on XPathResult type
 - When to use XPath instead of CSS selectors
- The DOMParser
- Examples of DOM manipulation
 - Examples of selecting DOM elements
 - Select an element by its ID value
 - Select an element by its class
 - Create a todo list
 - Adding attributes to an element
 - Assign multiple attributes to an element in one go
 - Concatenate a variable with a string and display the result
 - How to dynamically insert content into an element
 - Implementing drag and drop
 - Dynamically create a table with data from an array
- The Window object
 - Relationship between window and document
 - Properties and methods
 - 1. Window Size & Position
 - 2. Alerts & User Interaction
 - Example alert()
 - Example confirm()

- Example prompt()
- 3. Reloading a window, and navigating to other windows
- 4. Manipulating the URL with the URL API
- Key components of a URL
- Parsing a URL
- Modifying URL components
 - Modifying query parameters
 - Understanding Blobs
- Generating links to assets created in memory
 - 5. Navigating Through Browser History
 - Go back or forward in browser history
 - Go multiple steps in history
 - 6. Opening a New Child Window
 - Opening a New Window
 - Security issues with multiple tab navigation

Introduction to the DOM

- The JavaScript programming language really shines in this domain.

The DOM is an abbreviation for Document Object Model. It refers to the concept that JavaScript sees everything on an HTML document as an object. These objects are known as elements and like any object in programming, they have properties. A property is basically a part of what makes an object, for example, a door is a property of a car, just like its engine, or a tyre. A person or user object will have the following properties: username, name, age, address, etc. So JavaScript regards a web page as an object (the document object), and all the elements on this document object are also objects themselves having their own properties. For example; a web document has a form element, an unordered list or ordered list, a div element etc. The div element (object) has properties like style, class, id etc.

We went in depth into, and learned all about objects and how they work, back in Chapter 12 (Object Oriented Programming). Here we will look at how to use JavaScript to dynamically manipulate the DOM, which means we will learn how to create HTML elements, display/hide them on a web page (document), modify them, delete them, traverse (move) through them etc. If this sounds like fun, it's because it really is. The properties of HTML elements are all, but not limited to their attributes. For example, the following is an image HTML tag and it has three properties: src

(which points to the path of the image to be displayed), and then the width and height of the image.

```
<img src="" width="100" height="200">
```

As you can probably already imagine, since attributes are the properties of HTML elements, as we learn about manipulating DOM elements, we will therefore be working a lot with attributes.

When you first get introduced to JavaScript, the thing that excites most programmers—which is usually the biggest selling point of JavaScript, is DOM manipulation. That was for me as well. We immediately want to get right into selecting DOM elements, traversing the DOM, etc but most of the time, it seems like the tools that JavaScript has to achieve these things are just so many and haphazard. Different learning sources talk of nodes, child nodes, parent nodes, sibling nodes, then there is the ubiquitous `HTMLElement` object, however, no one seems to be talking about how they are all related to each other or when to use which, and which are the most efficient in any given scenario. If that is you, then you are not alone. I struggled with that too. Personally, I always like to have the full list presented to me in a structured way. The fact is; the DOM can feel like a chaotic mix of tools, but there's actually a structure to it. If it seems complicated to most, it's only because it's not being presented in a structured way. Once that structure becomes clear, it will then become easy for you to master it all. Here's a structured breakdown of the key DOM manipulation tools and how they relate to one another.

Understanding the DOM Tree (Parent-child relationships)

The DOM is a hierarchical tree where every element is a node. Nodes can be elements, text, or comments.

Key node relationships:

- Parent-child relationships: An element inside another is its child.
- Sibling relationships: Elements with the same parent are siblings.
- Ancestor-descendant relationships: Any element higher in the tree is an ancestor.

The difference between Nodes and Elements

A node is any item in the DOM (elements, text, comments).

An element (`HTMLElement`) is a specific type of node that represents an actual HTML tag.

It is important to be able to tell when you're dealing with a Node and when you are dealing with an Element. When we say some properties or methods of the `HTMLElement` object (like `children`) ignore non-element nodes, we mean they only return actual elements and ignore text nodes and comment nodes. What actually are text nodes and comment nodes? Text in your HTML can exist inside elements (like inside a `<p>` tag) or directly in the DOM as a separate text node. The same applies to comments. It is the ones that are not within HTML elements (tags) that are not considered as element nodes. Here is a demonstration:

```
<div id="parent">
Some text <!-- This is a text node -->
<p>Paragraph text</p> <!-- The <p> is
an element, but its text inside is a
text node -->

    <!-- This is a comment node -->
</div>
```

- "Some text" → A text node (not inside a `<p>`, so it's separate in the DOM).
- `<!-- This is a comment node -->` → A comment node (ignored by properties like `children`).
- `<p>` → A real element, but its inner text is still inside a text node.

Here is the kicker; a `<p>` tag is an element, so it will always be included in `.children`. However, the text inside a `<p>` is a text node, not an element, so `children` will count (select) the `<p>` tag, but not the text within it. The `childNodes` property, on the other hand, does accept text and comments, and so it will return the text within the `<p>` tag. Here is a demonstration:

```
<p id="para">Hello World!</p>

let p = document.getElementById("para");

// Logs an empty HTMLCollection (no element
// children!)
console.log(p.children);

// Logs NodeList(1): ["Hello World!"]
// (a text node)
console.log(p.childNodes);
```

Why does `p.children` return empty? Because the `p` tag has no element children, just a text node.

Every element is a node. Not every node is an element.

Your markup:

```
<div id="parent">
Text Node
<p>Paragraph 1</p>
<p>Paragraph 2</p>
</div>
```

What the DOM actually holds inside that div:

[0] #text	the line break + "Text Node"
[1] <p>	an ELEMENT
[2] #text	the line break between them
[3] <p>	an ELEMENT
[4] #text	the line break before </div>

Three of the five are whitespace you never typed on purpose.

parent.childNodes	5	— everything, including the text nodes
parent.children	2	— elements only

Want elements only? Use children, firstElementChild, nextElementSibling.

Figure 15.1 — Every element is a node; not every node is an element

Here are some properties of nodes and elements that you **MUST** know:

- **nodeType** – Used to differentiate between node types. This can be very handy in determining what type of element you are dealing with. For example, **nodeType 1** (element node), **nodeType 3** (text node), etc.
- **nodeName** – Returns the tag name (DIV, SPAN, etc.).
- **innerText** vs. **textContent** – Both get and set the text inside an element, but they are not the same thing, and the difference matters. See below.

That last pair deserves a few more words, because the two look interchangeable and are not. Say you have this on your page:

```
<div id="myDiv">
  Visible <span style="display:none">hidden</span> text
</div>
```

textContent gives you every piece of text that is in there, whether the visitor can see it or not:

```
let div = document.getElementById("myDiv");

// "Visible hidden text"
console.log(div.textContent);
```

innerText gives you only what is actually rendered on screen. The span is styled `display:none`, so it is not shown, so **innerText** leaves it out:


```
// "Visible text"
console.log(div.innerText);
```

So the short version is this: `textContent` is what is in the HTML, and `innerText` is what the visitor can see. There are two more differences worth knowing. `innerText` collapses runs of spaces and line breaks the way the browser does when it draws the page, while `textContent` hands back the whitespace exactly as it sits in your markup. And because `innerText` has to know what is visible, the browser may have to work out the page layout before it can answer, which makes it slower than `textContent`. For most jobs `textContent` is the one you want. It is faster, and it does not surprise you by changing its answer when somebody edits the CSS. Reach for `innerText` only when you genuinely mean "the text as the visitor sees it".

Selecting DOM Elements (Getting elements to work with)

I will start by introducing to you the methods of the document object. These methods target and return references to elements on the web document (page) so you can manipulate them. They belong to the document object, so you obviously have to call them on the document object. For example:

```
document.methodName(idOrClassOfElementYouWant);
```

Selecting elements on a web page is probably the most frequent task you will have to be carrying out as a JavaScript programmer. If you have to do anything with elements on a web page—which is what JavaScript does, then you have to learn how to select these elements before you can do anything with them. This skill alone will reveal the power of JavaScript to you and give you the confidence to harness the power of this versatile scripting language. There are five methods to learn here. Three of them are the older ones, each built to answer one particular question, and two are newer and more flexible. Here they are with the job each one does:

The three older methods -`document.getElementById("id")` Gets ONE element, by its `id` attribute.

```
-document.getElementsByClassName("class")
  Gets ALL elements carrying that class.
```

-`document.getElementsByTagName("tag")` Gets ALL elements of that tag, such as every `<p>`.

The two newer methods

```
-document.querySelector("selector")
  Gets the FIRST element matching a CSS selector.
```

```
-document.querySelectorAll("selector")
  Gets ALL elements matching a CSS selector.
```

Read down that list and you will notice something about the naming that is worth holding on to: where the method name says Element, singular, you get back one element. Where it says Elements, plural, you get back a collection. `querySelector` and `querySelectorAll` follow the same rule in different words — All in the name means more than one. The two newer methods can do everything the three older ones can, because a CSS selector can describe an id (`"#myId"`), a class (`".myClass"`) or a tag (`"p"`). That does not make the older three obsolete, and you will meet all five in real code, which is why we cover them all.

These methods will select and return a single or multiple `HTMLElement` object(s). Actually, to be specific, what it returns when you select a paragraph (p tag), is an `HTMLParagraphElement`, but this `HTMLParagraphElement` inherits from the `HTMLElement` object. The fact is, in the Document Object Model (DOM) every HTML element is represented by a specific JavaScript object type, usually named something like: `HTML<TagName>Element`, and they all inherit from the `HTMLElement` object. Before we proceed to learn how to select elements; here's a list of the most commonly used HTML elements and their corresponding DOM interface names (JavaScript object types): This is not the exhaustive list, but rather a few, just so you get the idea.

HTML tag	DOM Object Type
<code><div></code>	<code>HTMLDivElement</code>
<code><p></code>	<code>HTMLParagraphElement</code>
<code><h1></code> , <code><h3></code> etc	<code>HTMLHeadingElement</code>
<code><a></code> (link)	<code>HTMLAnchorElement</code>
<code></code>	<code>HTMLImageElement</code>
<code><input></code>	<code>HTMLInputElement</code>
<code><textarea></code>	<code>HTMLTextAreaElement</code>
<code><form></code>	<code>HTMLFormElement</code>
<code><table></code>	<code>HTMLTableElement</code>
<code></code> , <code></code>	<code>HTMLUListElement</code> , <code>HTMLOListElement</code>
<code><body></code>	<code>HTMLBodyElement</code>

All I need you to know right now is that all HTML elements have their specific object types, like the `<p>` tag has `HTMLParagraphElement`. But these object types all inherit from another JavaScript object, `HTMLElement`, so they get everything it has.

(Careful with the word “child” here—in this chapter it means a child node on the page, which is a different idea altogether.) Therefore all the properties and methods of the `HTMLElement` object are available to them. That is why after selecting an element, we can reference properties on it like `.innerHTML`, `.style`, `.classList`, to manipulate it. Let’s now look at how to select elements on a web page.

The `getElementsByTagName()` method

It is used like this: `document.getElementsByTagName("tagName");` It returns a live collection of elements selected by their HTML tag name. For example, you can decide to select all the `<p>` tag elements on a web page.

The `getElementById()` method

It is used like this: `document.getElementById("id");` This method of the document object grabs an element by the value of its `id` attribute. It selects that single element and returns it as an `HTMLElement` object. Pass it the value of the `id` attribute of the element you want to select, as a string. For example:

```
<p id='myText'>Some text</p>

let myText =
  document.getElementById("myText");
```

The `getElementsByClassName()` method

It is used like this: `document.getElementsByClassName("className");` Just like the `document.getElementById()` grabs an element by its `ID`, `getElementsByClassName()` method as its name suggests, selects elements by the class name that you give to it. Note that the `Elements` in the method name `getElementsByClassName()` is spelled with an ‘s’. This makes sense because a class attribute is usually assigned to multiple elements anyway.

Call this method on the document object passing to it the class you want to target on the web page. Note that this obviously means you will end up with multiple elements, since unlike the `id` attributes which have to be unique, the same class attribute value can, and is usually assigned to multiple HTML elements. These (multiple) items of the matching class are returned as a live collection (`object HTMLCollection`). Live collection means it automatically updates when the DOM changes. For example:

```
<p class='myText'>Some text 1</p>
  <p class='myText'>Some text 2</p>
  <p class='myText'>Some text 3</p>
```

```
let myText = document
  .getElementsByClassName("myText");
```

This returns a collection of (multiple) elements that have the specified class. This collection returned is an `HTMLCollection`. You can then access the elements using the bracket notation and the specific index of the element you want in the same way you would do with any array using the indexes like (`[0]`, `[1]`, etc.) or you may even convert it to an array in order to be able to use array methods on it. Example:

```
let myText = document
  .getElementsByClassName("myText");

// Logs the first element in the collection
console.log(myText[0]);

// Logs the text inside that first element
console.log(myText[0].textContent);
```

The output of this will be:

Some text 1

The `getElementByClassName()` is not the only method of the document object that you can use to select an element by its class. There are also the `document.querySelector()` and the `document.querySelectorAll()` which are relatively newer to JavaScript and more elegant, especially because you can pass to them the exact same selector syntax as you use in CSS to select elements. We will talk about them next.

The `querySelector()` method

It is used like this: `document.querySelector("cssSelector");` It returns the first matching element. Pass it the id, class, or tag name of the target element in a string. Tip: it accepts a CSS selector as a string, and this means that it takes the exact same selector used in CSS to target an HTML element. So, for ids, you have to prefix the id name in the string with a hash character for example:

`"#id"`

and for a class, you prefix it with a dot (`.`). For example:

`".className"`

Here is how you would use it to select an element by its id:

```
<h1 id="myHeading">The heading</h1>
```

```
let elemById =
  document.querySelector("#myHeading");
```

Here is how you would use it to select an element using its class:

```
<p class='myText'>Some text 1</p>

let item =
  document.querySelector(".myText");
```

Note very carefully—and this is the difference between the `querySelector()` and the `getElementsByClassName()` methods of the `HTMLElement` object, that unlike `getElementsByClassName()` which returns an `HTMLCollection` of multiple elements that have that class, `querySelector()` returns ONLY a `Single HTMLElement`. This single element is the first matching element in the entire document or null if no match is found. For example:

```
<p class='myText'>Some text 1</p>
  <p class='myText'>Some text 2</p>
  <p class='myText'>Some text 3</p>
<ul id='myUl'>
  <li>
    <p class='myText'>Text within a list</p>
  </li>
</ul>
```

```
let myText =
  document.querySelector(".myText");

// This logs the first matching element
// with the 'myText' class, which is the
// first p tag with the content "Some text 1"
console.log(myText);
```

If you want to select all the elements matching that class, in the same way that `getElementsByClassName()` does, you have to use the `querySelectorAll()`. Let's talk about that next.

The `querySelectorAll()` method

It is used like this: `document.querySelectorAll(cssSelector)`; It returns a static `NodeList` of all matching elements. This means multiple elements are returned. What it returns is a `NodeList` unlike the `getElementsByClassName()` method that returns an `HTMLCollection`. Here is an example:

```
let items = document.querySelectorAll(".myText");
console.log(items);
```

This code logs a `NodeList` to the console.

Difference between HTMLCollection, NodeList, and HTMLElement

The first two of these return multiple elements, as opposed to a single element. The HTMLCollection is sometimes referred to as a live HTMLCollection. It's said to be live because it is tracked by JavaScript and is updated whenever any of the corresponding elements on the web page changes. A static NodeList unlike an HTMLCollection does not update automatically. Also, unlike an HTMLCollection, a NodeList has the advantage that you can directly call an array method like `.forEach()` on it to loop through its data. Let's talk about the HTMLCollection and NodeList return types. Let's learn about which methods of the document object return which, and look at how they (a HTMLCollection and a NodeList) are different from an HTMLElement object (single element).

Document methods that return HTMLCollection

The following are the DOM methods that return an HTMLCollection. There are only two methods.

- `getElementsByClassName()`
- `getElementsByTagName()`

- `document.getElementsByClassName("className")`. It will work just the same if you call it within a specific element—basically using that element as the root or base within which to search for a match, as opposed to using the document object. For example; the first example code will return an HTMLCollection of all elements having a class value of "item". The second example will return an HTMLCollection of all elements with the class "divItems" that are found within the div that has the id of "myDiv", only:

```
let items = document.getElementsByClassName("item");
```

```
let div = document.getElementById("myDiv");  
let spans = div.getElementsByClassName("divItems");
```

* `document.getElementsByTagName("tagName")`. Returns all elements with the given tag name. It will work just the same if you call it within a specific element—basically using that element as the root to search from, as opposed to using the document object. For example; the first line of code below will return an HTMLCollection of all `<p>` tags in the current web document. The next example code will return an HTMLCollection of all span tags within the div having the id of "myDiv", only:

```
let paragraphs = document.getElementsByTagName("p");  
let div = document.getElementById("myDiv");  
let spans = div.getElementsByTagName("span");
```

Document methods that return NodeList

A `NodeList` is simply a list of `Node` objects, which may include elements, text nodes, or comments—though usually it's just elements in most common use cases. So, a `NodeList` contains not just legal HTML elements (`HTMLElement` objects), but also `childNodes` that can be text that are within HTML tags (elements) but are not the elements themselves, as well as comments in your code like this:

```
<!-- comment in HTML code here -->
```

While a `NodeList` looks similar to `HTMLCollection`, it is not live and has slightly different behaviour. The one you will reach for most often by far—and you guessed it correctly—is the

- `querySelectorAll()` method

For example 1:

```
let nodes = document.querySelectorAll(".item");
```

Will select and return a `NodeList` of all elements on the current web page that have (match) the class of `".item"`.

For example 2:

```
let items = document.querySelectorAll(".myText");
```

```
// Logs a NodeList of all p tags with the class ".myText"  
console.log(items);
```

```
// Works directly!  
items.forEach(item =>  
  console.log(item.textContent));
```

This will output: Some text 1 Some text 2 Some text 3 Text within a list

We have seen now that three great methods are key in the world of `HTMLCollection` and `NodeList`:

- `getElementsByClassName()` (`HTMLCollection`)
- `getElementsByTagName()` (`HTMLCollection`)
- `querySelectorAll()` (`NodeList`)

I had to show you which DOM methods return an `HTMLCollection`, and which returns a `NodeList`. Now that you know, let's get down to the main question we are trying to answer; what is the relationship between an `HTMLCollection`, a `NodeList`, and an `HTMLElement`? In other words, how are they all related. Let me break it

- `HTMLCollection` is a live collection of only `HTMLElement` objects.
- `NodeList` is a list of `Node` objects, which may include HTML elements, but also other things like text nodes, or comments etc.

Both are taken when the page has two matching paragraphs. Then a third is added.

And what you can do with each

- Figure 15.2 — Live HTMLCollection vs static NodeList*

The answer to this is no. This is a very important question, and my answer here should set you apart because it typically takes a lot of developers a while to figure it out. Both entities just so happen to have been endowed with some array-like properties/functions, but that does not make them Arrays. Here are the properties they share with arrays:

HTMLCollections and NodeLists are not real arrays. HTMLCollections only have a .length property and index access but have no array methods at all. NodeLists

have a `.length` property too, have index access, and the `.forEach()` method—but no other array methods like `.map()` or `.filter()`. If you want to be able to perform array-like operations on them, like loops, and much more, there are ways to do so. Let's talk about that next.

Looping through HTMLCollections & NodeLists

Because when we talk of HTMLCollections and NodeLists, we are speaking of multiple items—with 'multiple' being the operative word, it follows that we need a way to be able to traverse through these items in order to make any use of them. Without being true arrays, they have their own ways to be looped over. Both HTMLCollection and NodeList work with the `for...of` loop that applies to Arrays, and both work with the traditional `for` loop used in Arrays as well. So for looping alone, the two behave the same. What separates them is `.forEach()`, which a NodeList has and an HTMLCollection does not—which is exactly what the table above tells you. Let's see how to use these two array-like loops. The `for...of` loop works directly on either of them, and is therefore the easiest choice. Here is an example:

HTML code

```
<p class='myText'>Some text 1</p>
<p class='myText'>Some text 2</p>
<p class='myText'>Some text 3</p>
<ul id='myUl'>
  <li>
    <p class='myText'>Text within a list</p>
  </li>
</ul>
```

JavaScript code

```
const items = document.getElementsByTagName("p");

for (let item of items) {
  console.log(item.textContent);
}
```

The output is:

Some text 1 Some text 2 Some text 3 Text within a list

Notice that even the `<p>` tag nested within the `<li>` tag of a `<ul>` element was fetched.

You can use the classic `for` loop familiar from arrays to loop through both HTMLCollections and NodeLists. It works because the `for` loop works with anything

that has the `.length` property, and `HTMLCollections` and `NodeLists` have it. Here is an example using the same HTML markup as above:

JavaScript code

```
const items = document.getElementsByClassName("myText");

for (let i = 0; i < items.length; i++) {
  console.log(items[i].textContent);
}
```

The output is same as above:

Some text 1 Some text 2 Some text 3 Text within a list

It is great to be able to loop through your collection of data. Now you have the power to really make use of the data you select from the DOM. What if we could go one step further, and have the full power that we have with arrays on `HTMLCollections` and `NodeLists`? There is a way.

Using all array methods on `HTMLCollections` & `NodeLists`

You can have the full power that we have on arrays on `HTMLCollections` and `NodeLists`. The way to achieve that is easier than you think. It is: ...drumroll... you guessed it right, to convert the collection into an array. Yes, it's as easy as that. There are two ways to convert an `HTMLCollection` or `NodeList` into a true array, and it works the same for both of them. These two ways are:

- Use the `Array.from()` method
- Use the spread operator ...

Let's see how that works.

The `Array.from()` method

This uses the `.from()` method of the JavaScript Array object. The following example will grab and return an `HTMLCollection` of all `<p>` tags on the current web page, then use the `Array.from()` to convert it into a true array. You can then call or apply any array method or property of your choice to manipulate the array.

```
const items = Array.from(document.getElementsByTagName("p"));

//this logs an array of all the p tags
console.log(items);

items.forEach(item => {
```

```
// this logs the contents of all the <p> tags
  console.log(item.textContent);
});
```

In this example, we use the array `.forEach()` to loop through the data after converting the `HTMLCollection` to an array. But it is up to you. Having the data as an array allows you to harness the full power of the built-in JavaScript Array object on your data.

The spread operator ...

Let's look at the other way of converting a collection into an array using the spread operator. To get a good understanding of what the Spread operator is, refer back to Chapter 3 (Arrays), in the section where I talked about Rest parameters and the Spread operator. This time let's convert a `NodeList` just for balance, but remember that both the `Array.from()` approach and the spread operator will work regardless of if we are dealing with an `HTMLCollection` or a `NodeList`.

```
const nodeList = [...document.querySelectorAll("p")];

// this line logs an array
console.log(nodeList);

nodeList.forEach(item => {
  console.log(item.textContent);
});
```

Here we use the `querySelectorAll()` method of the document object to select all `<p>` tags on the current web page. We already know the `querySelectorAll()` method returns a `NodeList`—which is why I am using it. When we call this method, we use the Spread operator to capture what it returns into an array like so:

```
[ ...methodCall ]
```

The square brackets are what create the array, as we already know. That is it. Now that we have an array, the rest of the code is us using the `forEach()` method of all arrays to loop through the data.

The output of the `console.log(item.textContent);` line logs this to the console: Some text 1 Some text 2 Some text 3 Text within a list

Selecting nested elements

To select the `p` tag containing the text: “Text within a list” which is within a `ul` list, you need to query the DOM beginning from within the parent of the `<p>` element, which in this case is a `li` element, instead of using `document` as the parent—which will search the whole document. So, to do so, start by selecting the list element

which is the parent, and use that as the base of your selection instead of document. For example, let us retrieve the text within the `<p>` tag that is nested within the unordered list (`<ul>`) tag.

```
let myUl = document.querySelector("#myUl");
let textInList = myUl.querySelector(".myText");
console.log(textInList.textContent);
```

This will output the text within that `<p>` tag

Text within a list

Get the value or contents of an HTML element

When we select HTML elements, it could be for a variety of reasons, to change their styling, for example their colour, background position, position on the page, move them—as in the case of drag-and-drop, etc. But very often, we want to retrieve their value or content. Generally, if we are dealing with a form element, like a form input, or textarea field, we would mostly refer to its content as value, but if we are dealing with something like a `<p>` tag, or a `div`, we would refer to its content as, just content. Let me show you some examples on how to retrieve values/contents of elements. Imagine we have the following markup in our HTML page:

```
<h1 id='myHeading'>Heading text</h1>

<p class='myText'>Some text 1</p>
<p class='myText'>Some text 2</p>
<p class='myText'>Some text 3</p>

<ul id='myUl'>
  <li>
    <p class='myText'>Text within a list</p>
  </li>
</ul>
```

Let's extract the content of the `<h1>` tag

```
let myHeading = document.querySelector("#myHeading");
console.log(myHeading.textContent);
```

The output will be: Heading text

Let's extract the content of the second `<p>` tag on the page

```
let myText = document.querySelectorAll(".myText");
console.log(myText[1].textContent);
```

The output will be: Some text 2

Let's get the content of the `<p>` tag that is inside the `<li>` tag that is inside the `<ul>` tag. As you can see, there are many nested (sub) elements, but I wanted to

show you different ways to handle that from the way I showed you above under selecting nested elements. Here you will see the power of the new `querySelector()` and how you can pass it a series of elements to search through in hierarchy, just like in CSS.

```
let myUl = document.querySelector("#myUl li p");
console.log(myUl.textContent);
```

The output as always is: Text within a list

Let's try that same example again, but this time using the `querySelectorAll()` which returns a `NodeList` that you can directly run array methods on or extract values by index from it, just like you would do with a real array. Notice that the selector is exactly the same as before—all that changes is the method, and that we now have to reach for the first match with `[0]`:

```
let myUl = document.querySelectorAll("#myUl li p");
console.log(myUl[0].textContent);
```

The output again, is: Text within a list

Let me wrap this DOM selection section with another example on how to extract the value from a form input field this time. Let's say you have the following input field on your HTML page:

```
<input type="text" id="nameInput" value="Gustav" />
```

I have given the field an id of "nameInput" and a value of "Gustav". Normally, form elements should be within a form element (`<form>` tag) so it would also have other properties like a button which all browsers know how to respond to by submitting the form when it is clicked. Another thing a real form element has is an optional HTTP method in the opening form tag which is meant for the headers of the request being sent to a server by the browser. The header values will help the browser determine how the data from that form is to be transmitted over to a server. So I have not placed this input field inside a form tag because we are just testing here. The value which I gave is also for testing. In a real scenario, that value will only be there once the user has typed something into that input element, and submitted. The general practice is to place what we call an event listener on the form's submit button, and define that the event we want to listen for is a click event. Our code due to the event listener placed on that button will then be listening for that event to occur (fire). Once the event fires (occurs), we will then grab the value entered into that field because it is always sent through the event object by the browser. We will learn all about event handling and form handling in later chapters. For now, just understand that our "nameInput" field above is just for testing, so there is no form element, and no submit button with an event-listener on it, waiting

for the user to submit it after completing the form. This is why I manually give the field a value so we can test extracting it after selecting the field. The following is the code to select the input field and get its value. It works by first of all selecting the target field using the `.querySelector()` method that you are already familiar with, then we get the value from the `.value` property of the DOM object. Here it is:

```
let nameInput = document.querySelector("#nameInput");
console.log(nameInput.value);
```

The output is: Gustav

In this section, we have learned about how to select the content of an `h1`, a `p` tag, a `ul/li` and one form input element. I will save extracting values from form fields for Chapter 19 where I go in depth into form manipulation. This is because there are better ways to extract values from form fields than the approach in this example. You will learn all about that in Chapter 19. You will come out of that chapter as a pro form handler.

Traversing the DOM

These are properties and methods provided for traversing DOM elements. You would call or refer to them on the HTML element object, for example after selecting it. For example, the following code will display the value of the `id` property (attribute) of the parent element of the text box stored in `textElement`:

```
let textElement =
  document.getElementById("idOfTextField");
console.log(textElement.parentNode.id);
```

Notice how to use these properties or methods, you have to reference it on the HTML element itself using the dot operator (`.`) like so `HTMLInputElement.propertyName`.

Parent node

- `parentNode` It gets the parent of an element. This parent node (can be an element, document, or even null).
- `parentElement` This is the same as `.parentNode`, but guarantees an `HTMLElement` or null. The downside of using `parentElement` is that it depends on the exact structure. If you ever add another wrapper/ parent element, it will break (fail to be accurate).
- `closest()` `closest()` only looks for element nodes (`HTMLElement`), so it ignores text nodes and comment nodes automatically. It starts with the element you called it on—if that one matches, it is what you get back—and then keeps movi-

ng up the DOM until it finds a match or reaches <html>. You have to pass it a CSS selector as a string, written exactly as you would write it in a stylesheet: "#id" for an id, ".className" for a class, or just the tag name. For example,

.closest(".editFormDiv") will search up the DOM to find the nearest ancestor with the class name of class .editFormDiv. It is more precise than parentElement simply because you can be specific by passing it the exact id, class name or tag name to fetch from the base element (the element you call it on).

Child nodes

- **children** It gets all child HTML elements. It fetches only actual HTML elements, and excludes text and comment nodes. It will then contain a collection of HTMLElement objects represented like so if you do console.log() of it to view its contents: [object HTMLCollection]. This is like an array of child elements (but not exactly an array). To then get (select) a specific child inside that collection, you can reference it on the result using the bracket notation and the index—just as you would do with arrays, like so:

```
let parentElement =
  document.getElementById("elemId");
let children = parentElement.children;

// Get the first child element
console.log(children[0]);
```

- **childNodes** Returns all child nodes (elements, text, comments).
- **firstChild** Returns the first child node (can be text, comment, or element).
- **lastChild** Returns the last child node (can be text, comment, or element).
- **firstElementChild** / **lastElementChild** Gets the first/last child element. It only gets actual HTML elements and ignores text and comment nodes.

Sibling nodes

- **nextSibling** Returns the next node (could be text, comment, or element).
- **previousSibling** Returns the previous node (could be text, comment, or element).
- **nextElementSibling** / **previousElementSibling** It gets the next/previous sibling. It only gets actual HTML elements.

Note: childNodes, firstChild, lastChild, nextSibling, and previousSibling include text nodes and comment nodes, which often leads to unexpected results. Prefer

children, firstElementChild, lastElementChild, nextElementSibling and previousElementSibling, unless you need non-element nodes.

So, if you want only elements, use:

- parentElement
- closest()
- children
- firstElementChild
- lastElementChild
- nextElementSibling
- previousElementSibling

And if you need all nodes (elements, text, comments), use:

- childNodes
- firstChild
- lastChild
- nextSibling
- previousSibling

Example of DOM traversing

Here is a simple example to demonstrate DOM traversal using the above HTMLElement properties:

```
<div id="parent">
  Text Node
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</div>

let parent =
  document.getElementById("parent");

console.log(parent.childNodes);
```

This outputs a NodeList of FIVE nodes, not three: [#text, (the line break and "Text Node")

```
<p>Paragraph 1</p>,
#text,                (the line break between them)
<p>Paragraph 2</p>,
#text                (the line break before </div>)
]
```


That is worth staring at for a moment, because it is exactly the trap this section is warning you about. Every line break and run of spaces between your tags becomes a text node of its own. Three of the five things here are whitespace you never typed on purpose.

```
console.log(parent.children);
```

This outputs:

```
[
  <p>Paragraph 1</p>,
  <p>Paragraph 2</p>
]
```

```
console.log(parent.firstChild);
```

This outputs: `#text` (because of the space and line breaks)

```
console.log(parent.firstChild);
```

This outputs: `<p> Paragraph 1 </p>`

Scrolling & Focus

- `scrollIntoView(alignWithTop)` It scrolls to the element.
- `focus()` Moves focus to the element.
- `blur()` Removes focus from the element.

Manipulating elements

We are now getting into the fun part. These are methods and properties to manipulate DOM elements. These are `HTMLElement` object properties and methods, so remember that you will reference or call them on those elements using the dot operator (.) after selecting them from the DOM like so: `textElement.property`

Changing elements

In JavaScript, there are special properties you can use to change the content (what people see) inside elements on a web page. These properties belong to what's called an `HTMLElement`, which is just a fancy name for any element you see on a web page—like a `<div>`, `<p>`, `<h1>`, and so on.

The way this is done in JavaScript is to select the HTML element, and then reference the document object property on it as you have seen before. You can also select the element first, store it in a variable for example a variable called `elem`, then you would reference the property you want to change on that variable—which now

represents the HTML element, using this syntax: `elem.propertyName = "new value"`;

Let's explore the four main properties you'll use most often.

- `textContent`
- `innerHTML`
- `outerHTML`
- `innerText`
- `textContent` It changes or gets the plain text inside an element. It does not treat any content as HTML, so if you write HTML tags, they will show up as text. Use it like this:

```
let elem = document.getElementById("myHeading");
elem.textContent = "<h1>Hello world</h1>";
```

This will select an element on the web page with the id of "myHeading", then set or place the text "<h1>Hello world</h1>" in that element, which can be for example a div. I should point out here that you should not be confused by the <h1> tags in the value. The text of the targeted element will not now contain an h1 element, but rather just the literal text "<h1>Hello world</h1>". Again, this is because `textContent` treats everything as plain text. Use `textContent` when you only want to work with text and don't need to insert any HTML. It's also safer (see XSS section below).

- `innerHTML` It places or gets the HTML content inside an element that contains HTML code. If you write tags, unlike content assigned to `textContent` which the browser only reads as text, the browser will treat them as real HTML. Use it like this:

```
let elem = document.getElementById("pTag");
elem.innerHTML = "Hello <b>World</b>";
```

This will select an element on the web page with an id of "pTag" and set the following text in it that contains HTML code:

```
"Hello <b>World</b>"
```

That element will now display in the browser: Hello World with the word "World" in bold. Use `innerHTML` when you want to add content that includes HTML, like links, bold text, or line breaks.

- `outerHTML` It gets or sets the entire element itself, including its tags and everything inside it. Basically, it overrides the element you use it on, as opposed to only touching its content. The result is a complete replacement of the old elem-

ent with the value you pass in. As for the syntax, it's the same as the examples above. Similarly to `innerHTML`, it handles HTML and not just text—as you could probably guess from the HTML in its name.

Use `outerHTML` if you want to replace the entire element, not just its content.

- `innerText` It is similar to `textContent`, but it respects what is visible on the screen. If something is hidden using CSS (like `display: none`), `innerText` won't return it, but `textContent` will. The syntax, is exactly the same as with `textContent`.

Use `innerText` if you care about what the user actually sees, not just what's in the HTML.

Both the `textContent` and `innerHTML` properties insert content into an element on a web page, but it is preferable to use `textContent` over `innerHTML` because `innerHTML` opens a security risk known as XSS (Cross-Site Scripting). This is when someone inserts malicious HTML or JavaScript into your page. For example:

```
elem.innerHTML = "<img src='x' onerror='alert(\"Hacked!\")'>";
```

This could cause popups, data leaks, or worse. If you're inserting user input (like from a form or a database), always use `textContent` to avoid these dangers. It will treat everything as plain text—so there will be no surprises. Let me talk about why `<img src='x' onerror='alert("Hacked!")'>` is dangerous. This kind of code is a classic example of XSS (Cross-Site Scripting), and here's how it works:

-The `src='x'` is invalid—it points to an image that doesn't exist. -Because the image fails to load, the browser triggers the `onerror` event. -The `onerror="alert('Hacked!')"` attribute tells the browser: “When this error happens, run this JavaScript code.” The result: a pop-up appears with the message “Hacked!”.

That's just an example. Now imagine you have a form on your website where visitors from the public can type in their comments, and your website collects and displays them on a web page. This is typical for a blog or social media web page. Imagine a hacker inputs the following into the website comment form:

```

```

If the code of your website blindly uses `.innerHTML` to display those user comments on your webpage like this:

```
commentBox.innerHTML = userComment;
```

the code that hacker entered will be executed since `.innerHTML` knows how to read HTML. This is bad. That script will run in your browser, and possibly steal session cookies, login tokens, private data etc. That will be a security nightmare, espec-

ially for websites with accounts, admin panels, or user-sensitive data. Here is how to prevent it and be safe:

- Always use `.textContent` when inserting user data.
- Sanitise inputs submitted by users before you use them in your application (especially if you must use `.innerHTML`). Sanitising means validating or checking the content to ensure it meets a certain standard, or contains no suspicious or unwanted characters.
- Use security libraries or frameworks that auto-sanitise content (like React, which escapes HTML by default).

Adding/removing elements

- `remove()` Call it on an `HTMLElement`, and you do not have to pass it any argument. It removes the element it is called on.
- `appendChild(newElement)` – Adds a child to an element. The new element will then become the last child (at the end) of that element.
- `insertBefore(newElement, referenceElement)` – Inserts the `newElement` as a child in the element in front of the `referenceElement` which is another sibling in that same element.
- `removeChild(childElement)` – Removes a child element `childElement` from the element.
- `replaceChild(newElement, oldElement)` – Replaces an element `oldElement` with a `newElement` in an element.
- `insertAdjacentHTML(position, html)` Inserts HTML at a position ("beforebegin", "afterbegin", "beforeend", "afterend").
- `cloneNode(deep)` Clones an element (true for deep clone).

Creating New Elements

JavaScript provides you an awesome method on the document object that you can use to dynamically create any HTML element of your choice. This method is `createElement()`. Pass as an argument to it a string of the HTML tag name you wish to create. You can then add attributes to the newly created element and place it anywhere on the HTML document. Note that you have to call this method on the document object (using the dot operator as usual), since the method belongs to the document object. In this simple example, we will see how you can create a new `div` element, add some text content into it, and insert it into the body of an HTML web page.

```
let newDiv =
    document.createElement("div");

newDiv.textContent = "New DIV Element is here";
document.body.appendChild(newDiv);
```

Let's modify this example slightly by adding a border to the created div, and adding some border color, and background color, to blue. Let us also make the text within the div be in a `<p>` tag, and make it bold and white. This should demonstrate to you how you can add attributes and even styling to the elements you create dynamically. Here is the code for that:

```
let newDiv = document.createElement("div");

// Add HTML content
newDiv.innerHTML = "<p>New DIV Element is here</p>";

// Add styling directly via the style property
newDiv.style.border = "2px solid blue";
newDiv.style.borderRadius = "5px";
newDiv.style.backgroundColor = "dodgerblue";
newDiv.style.color = "white";
newDiv.style.fontWeight = "bold";
newDiv.style.padding = "10px";
newDiv.style.marginTop = "10px";

// Append to the body
document.body.appendChild(newDiv);
```

Notice that we changed how we assign the text value to the div (in `newDiv`). We first of all wrap the text in a pair of `<p>` tags. Then instead of using the `.textContent` we used earlier—which would display only text, we use `.innerHTML` so that the string which now contains HTML (in the `<p>` tag) will be parsed as HTML and displayed on the web document as such. To add styling, you have to reference the style properties on the style property of the HTML element, which in this case is `newDiv`. I should point out that when using JavaScript like this to add styling to elements, the style properties are not written in the same way as in CSS. In JavaScript most of the names are the same, however, camel-casing is used. Here is a small example of CSS properties and the equivalent names of the JavaScript HTML element's style properties—just to give you a hint:

CSS property	JavaScript style property
background-color	backgroundColor
border-radius	borderRadius
font-weight	fontWeight
margin-top	marginTop

We finally make use of the `.appendChild()` on the body of the web document, which is a method of the `HTMLElement` object (for all elements) to add (append) the new div as the last element inside the body.

```
document.body.appendChild(newDiv);
```

Let us look at one more example of dynamically creating HTML using JavaScript. Let us create a table this time, and append it as the last element inside the same div we created above. It should be a very simple table of maybe two columns and two rows, it should have white borders for visibility on the blue background of the div. Let's make the table contain users, and let's give the table headings the values of first name and surname. Here is the code:

```
// 1. Create the parent div
let newDiv = document.createElement("div");

// Add HTML content
newDiv.innerHTML = "<p>New DIV Element is here</p>";

// Add styling directly via the style property
newDiv.style.border = "2px solid blue";
newDiv.style.borderRadius = "5px";
newDiv.style.backgroundColor = "dodgerblue";
newDiv.style.color = "white";
newDiv.style.fontWeight = "bold";
newDiv.style.padding = "10px";
newDiv.style.marginTop = "10px";

// Append to the body
document.body.appendChild(newDiv);

// 2. Create the table
let table = document.createElement("table");
table.style.border = "1px solid white";
table.style.borderCollapse = "collapse";
table.style.marginTop = "10px";

// 3. Create the table header row
let headerRow = document.createElement("tr");

let th1 = document.createElement("th");
```

```

th1.textContent = "First Name";
th1.style.border = "1px solid white";
th1.style.padding = "5px";

let th2 = document.createElement("th");
th2.textContent = "Surname";
th2.style.border = "1px solid white";
th2.style.padding = "5px";

headerRow.appendChild(th1);
headerRow.appendChild(th2);
table.appendChild(headerRow);

// 4. Add a couple of data rows
let row1 = document.createElement("tr");
let row1Col1 = document.createElement("td");
row1Col1.textContent = "Alice";
row1Col1.style.border = "1px solid white";
row1Col1.style.padding = "5px";

let row1Col2 = document.createElement("td");
row1Col2.textContent = "Johnson";
row1Col2.style.border = "1px solid white";
row1Col2.style.padding = "5px";

row1.appendChild(row1Col1);
row1.appendChild(row1Col2);
table.appendChild(row1);

let row2 = document.createElement("tr");
let row2Col1 = document.createElement("td");
row2Col1.textContent = "Bob";
row2Col1.style.border = "1px solid white";
row2Col1.style.padding = "5px";

let row2Col2 = document.createElement("td");
row2Col2.textContent = "Smith";
row2Col2.style.border = "1px solid white";
row2Col2.style.padding = "5px";

row2.appendChild(row2Col1);
row2.appendChild(row2Col2);
table.appendChild(row2);

// 5. Append the table to the existing newDiv
newDiv.appendChild(table);

```

This code may seem long, but it is really easy to understand once you take a close look at it. It's only long because some steps are repetitive, for example:

- We start by creating the exact same div we created previously so we can also inject the new table we are creating into it.

- Then we create the "tr" element for the table's headings
- Then we create a pair of "th" elements for both the First Name and the Surname headings.
- Then we create a pair of "tr" elements to contain the table's data rows (records). Each "tr" is followed by a creation of the "table data (td)" cell, and then the use of .textContent to add the content for that table cell.

Let us count the times we use document.createElement() in the example:

```

1. document.createElement("div")
2. document.createElement("table")
// row for table headings
3. document.createElement("tr")
// First Name heading
4. document.createElement("th")
// Surname heading
5. document.createElement("th")
6. document.createElement("tr") // row 1
// row 1 First name
7. document.createElement("td")
// row 1 Surname
8. document.createElement("td")
9. document.createElement("tr") // row 2
// row 2 First name
10. document.createElement("td")
// row 2 Surname
11. document.createElement("td")

```

As you can see, creation of elements should be done chronologically, in the order in which they come in the DOM, and appended to the parent element. You do not have to create a closing tag for the element. Once you create an element with the .createElement() method, it knows to close the element's tag when it is done creating and adding any attributes and data to it. Hopefully, this exercise has taught you how to create a table element and structure it row-by-row and cell-by-cell, how to apply styles to each part of the table dynamically, and how to nest elements by inserting a table inside a div. This example visually reinforces how DOM manipulation allows you to build complex HTML structures with JavaScript. Further on in this chapter, under the section "Examples of DOM manipulation" we will expand on this table creation exercise. We will use an array of data, create a table in a similar fashion, then loop through the array, injecting the data as cell values in the new table. At the end, we place the table inside the div on the web page.

Working with Attributes & Classes

As you can imagine, because attributes are properties of HTML elements, the properties and methods provided by JavaScript for working with them are part of the `HTMLElement` object. You should therefore reference or call them on an HTML element after selecting it.

Attribute properties

- `id` It gets/sets the `id` attribute
- `className` It gets/sets the class as a string
- `classList` Provides methods to manipulate classes
- `title` Gets/sets the tooltip text
- `hidden` Hides/shows the element
- `style` Used to access inline CSS styles.

Attribute methods

- `setAttribute("attributeName", "value")` This sets an attribute with the name `attributeName`, and assigns it the value `“value”`. It updates the element’s `attributeName` attribute with the value `“value”` if it already exists, or creates a new one. Both arguments are required. For HTML5 attributes that do not need a value (such as `‘disabled’` or `‘required’`) you still have to pass a second argument—give it an empty string, `“”`. Leaving it out altogether throws a `TypeError`. For example:

```
document.getElementsByTagName("div")
[0].setAttribute("class", "active");
```

This will grab all `div` elements on a page, and set the class value of the first `div` (`[0]`) to `“active”`.

While we are on creating attributes for an element, another way is to use `Object.assign()`. Here is how to do it:

```
let li = document.querySelector("#li");

Object.assign(li, {
  'active': 'true',
  className: 'list-item draggable'
});
```

This code selects a list element that has an `id` of `“li”`, and then assigns it a few attributes. It assigns an attribute named `‘active’` and gives it a value of `‘true’`, and

two classes; 'list-item' and 'draggable'.

- `getAttribute("attributeName")` This retrieves (gets) the value of the attribute that goes by the name of `attributeName`. Here's an example:

```
document.getElementsByTagName("h1")
[0].getAttribute("class");
```

This will grab all `h1` elements on a page, and retrieve the value of the class attribute of the first one (`[0]`).

- `hasAttribute("attr")` It checks if an attribute exists. Here it checks if the attribute by the name of 'attr' exists on the element.
- `removeAttribute("attr")` – Removes an attribute. Here it removes the attribute 'attr' from the element.
- `classList.add("className")` Adds a class. This adds a new class by the name of `className` to the element.
- `classList.remove("className")` This removes a class. It removes the class by the name of `className` from the element.
- `classList.contains("class")` Used to check if a class exists.
- `classList.toggle("class")` Toggles a class. This toggles between opposite states going by the class name.

Handling styling

JavaScript provides direct access to the styles of HTML elements, which allows you to dynamically update the look and feel of a page. While most of the visual appearance is handled through CSS, JavaScript can manipulate these styles in real time — like changing colours when a user clicks a button, hiding/showing sections, or adjusting sizes for animation effects.

Common styling properties:

- `style` Accesses and sets inline CSS styles directly on an element. For example:

```
element.style.color = "red";
element.style.backgroundColor = "yellow";
```
- `offsetWidth / .offsetHeight` Returns the visible width/height of an element including padding and borders, but excluding margins. It is useful for layout calculations.
- `clientWidth / .clientHeight` Returns the width/height of the element including padding but excluding borders, scrollbars, and margins.

- `scrollWidth` / `.scrollHeight` Returns the full width/height of the element's content, including the parts that are hidden and require scrolling. They are good for detecting overflow content.
- `getComputedStyle(element)` Note that this one belongs to window, not to the element—you pass the element to it rather than calling it on the element. It allows you to get the final computed CSS styles for an element (including those from external stylesheets). For example:

```
const styles = getComputedStyle(element);
console.log(styles.marginTop);
```

- `.classList` Lets you add, remove, toggle, or check CSS classes dynamically. Here are some examples:

```
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("open");
```

Working with positioning

Positioning is essential when you need to move elements around, detect their location on the page, or create drag-and-drop interfaces. JavaScript offers several properties that help you calculate positions and adjust layouts dynamically.

Common positioning properties:

- `offsetParent` Returns the nearest ancestor element that has a positioning context (i.e., has a position value other than static). This is very useful in determining relative position.
- `offsetTop` / `offsetLeft` This works out the distance from the `offsetParent`. Basically, it gives you the distance in pixels from the top/left of the element to the top/left of its `offsetParent`.
- `scrollTop` / `scrollLeft` Returns the number of pixels that the content of an element has been scrolled vertically/horizontally. It can also be used to set a scroll position. For example:

```
element.scrollTop = 100;
```

- `.getBoundingClientRect()` Returns an object containing the position and size of the element relative to the viewport. This is great for detecting collisions or element visibility. For example:

```
const rect = element.getBoundingClientRect();
console.log(rect.top, rect.left);
```

- `window.pageXOffset` / `window.pageYOffset` These two belong to `window` rather than to an element. They return how far the document is scrolled horizontally or vertically from the top-left corner. These are useful for calculating absolute positions on the page.

Working with events

When users interact with a web page — by clicking a button, moving their mouse, typing into a form, or scrolling down — JavaScript needs a way to notice these actions and respond. This is where events come in. Events are signals that tell JavaScript, "Hey, something just happened on the page!". Since the DOM is the structure of everything the user sees and interacts with, events are a natural part of DOM management. By listening for events and reacting to them, your JavaScript code can make web pages interactive and dynamic. For now, it's enough to know that JavaScript provides simple tools to:

- Listen for specific types of events (like clicks or keypresses),
- Run code when an event happens,
- And manipulate the DOM in response.

We will explore events in much greater detail. Here, we just want to get comfortable with the idea that handling events is a key part of working with the DOM.

The following is a small example of how you can use JavaScript to listen for a user's click on a button and respond with an action.

```
<button id="myButton">Click me!</button>

<script>
  document.getElementById("myButton").addEventListener("click",
  function() {
    alert("Button was clicked!");
  });
</script>
```

In this example, we tell JavaScript: "When the user clicks on the button, run this function that shows a message." This shows the basic idea of event handling — listening for an action, and responding to it by changing something on the page (or doing anything else you want). We'll dive much deeper into how events work, the different types of events, and powerful ways to control them in the dedicated Events chapter, Chapter 24.

XPath and selecting DOM Elements

XPath stands for XML Path Language. It's a powerful query language designed to navigate and select nodes in an XML or HTML document. While JavaScript developers often use CSS selectors to get elements (like `getElementById()`, `querySelector()` or `querySelectorAll()`), XPath gives you more precision and flexibility, especially when you need to traverse deeply nested or irregular structures. If you want one sentence to hold on to, it is this: consider XPath to be to XML data what SQL (Structured Query Language) is to relational databases. It lets you find specific elements, attributes, or text inside a structured document using path-like expressions. It supports filtering, in the same way a WHERE clause does in SQL, and it comes with string, number and boolean functions such as `contains()` and `count()` that are not far off the JavaScript methods you already know. It has been around since 1999, and people who assume it is obsolete are mistaken. It is true that plenty of developers go a whole career without touching it. But it is still supported in every modern browser, and it is still what you reach for in some very specific corners, which we will come to.

Syntax examples of XPath (`//tag`, `/html/body`, etc.)

XPath expressions use a path-like syntax to describe elements in a document. Here are a few examples:

```
// p Selects all <p> elements in the document, no matter
*
    where they are.

* /html/body/div Selects a specific <div> that is a direct child of the
    <body> and <html> elements.

// div[@class='box'] Selects all
// <div> elements with the class box.
*

// ul/li[2] Selects the second
// <li> element inside every <ul>.
*

// *[@id='main'] You probably guessed
// this one; yes, it selects every
// element on this web document that
// has the id of "main" (id="main").
```

Here is a fuller list to give you an idea of the range. Play and experiment with these, and check the online documentation for more complex queries and

explanations:

Select all <book> nodes	//book
Attribute access (select id attributes)	//@id
All <book> tags whose id attribute is "b1"	//book[@id="b1"]
Filter books by price greater than 10	//book[price > 10]
Wildcards (select all nodes)	//*
Axes (advanced) - navigate relationships	//book/child::title
Find all <h1> tags	//h1
Find links whose href value is "#foo"	//a[@href="#foo"]
All tags containing the text "Hello"	//span[text()='Hello']/..
A <p> tag containing the text "warning"	//p[contains(text(), "warning")]
All <title> tags that are children of <book>	//book/title
Books by Orwell costing less than 10	//book[author="Orwell" and price < 10]

These examples show that XPath is more expressive than CSS in certain cases—like selecting based on text content or element position. Mastering this skill will set you apart as a JavaScript developer, and you will be able to work in any niche environment where expertise in handling complex XML documents is required. As you can see, it goes beyond the basics of the other CSS query selectors. Knowing XPath will make you able to take any complex HTML or XML document and make sense of its data.

Introducing document.evaluate()

The `document.evaluate()` method is how JavaScript interacts with XPath. It allows you to run an XPath expression against the DOM and get matching results back. This method was created specifically to work with XPath in HTML and XML documents, and it's part of the DOM Level 3 XPath specification. XPath works on both HTML and XML because their structures are very similar—both are markup languages with nested elements. You might not see the `.evaluate()` method mentioned in many beginner books or online tutorials. That's because XPath is a bit of a niche topic. Most JavaScript developers go their entire careers without ever needing it—especially if they only work with websites and don't deal with XML. However, XPath becomes useful in environments where advanced XML processing is done, such as in data-heavy applications or when working with APIs that return XML. I'm including it in this book so you're not caught off guard if you ever enc-

ounter it. My goal is to make you a well-rounded and prepared JavaScript programmer—even in the rare cases.

DID YOU KNOW? XPath was actually designed before CSS selectors became popular! While we now use things like `document.querySelector()` to grab elements easily, XPath was the go-to method for finding elements in XML documents—especially in older systems. Even today, tools like browser dev tools and web scraping software still support XPath because of how powerful it is for navigating complex document structures.

Basic Usage of `document.evaluate()`

Here's how the `document.evaluate()` method works:

```
const xpath = "//p";

const result = document.evaluate(
  // The XPath expression
  xpath,
  // Context node (usually `document`)
  document,
  // Namespace resolver (null for HTML)
  null,
  // Result type
  XPathResult.ORDERED_NODE_SNAPSHOT_TYPE,
  // Initial result (null for new)
  null
);

// Loop through and log matched nodes
for (let i = 0; i < result.snapshotLength; i++) {
  console.log(result.snapshotItem(i).textContent);
}
```

The `'document'` argument—which is the second argument passed to `.evaluate()`, should always have the value of `document` if you are trying to read an HTML document, but if you are reading an XML document, its value should be the formatted XML document. The `XPathResult` (fourth) argument is the format you wish the results from XPath to be in. You would pass in the relevant property of the `XPathResult` object that represents the type of result you need. In this case we use the `.ORDERED_NODE_SNAPSHOT_TYPE` which is a very common choice as it returns a list of the matched (selected) nodes nicely in an array, which is then easy for you to loop through and use. The type or format of the result returned will determine how you retrieve and work with the data. The names of these properties are written as constants, and are referenced on the `XPathResult` object when used like so:

XPathResult.propertyName

Let's see it in action reading data from documents.

Use document.evaluate() read HTML document

Here's a simple and clear example of using XPath to extract data from a regular HTML document (not XML), using document.evaluate(). Imagine you have the following HTML structure containing data about a collection of books that you wish to extract and use:

```
<body>
  <h1>Book List</h1>
  <ul>
    <li><span class="title">JavaScript: The Good Parts</span></li>
    <li><span class="title">Eloquent JavaScript</span></li>
    <li><span class="title">You Don't Know JS</span></li>
  </ul>

  <script>
    // JavaScript code will go here
  </script>
</body>
```

Let's see how you would use the document.evaluate() method to run an XPath query on the HTML document to extract all the book titles that are within `<span>` tags on the web page. Here is the JavaScript code to do that:

```
// XPath expression to find all
// span elements with class "title"
const xpath = "//span[@class='title']";

// Run XPath query on the current HTML document
const result = document.evaluate(
  // the XPath expression
  xpath,
  // context node – default for HTML document
  document,
  // namespace resolver (not needed for HTML)
  null,
  XPathResult.ORDERED_NODE_SNAPSHOT_TYPE,
  null
);

// Loop through the results and print the book titles
for (let i = 0; i < result.snapshotLength; i++) {
  const node = result.snapshotItem(i);
  console.log(node.textContent);
}
```

The output will be:

JavaScript: The Good Parts Eloquent JavaScript You Don't Know JS

Here is how it works: -This example selects every `<span>` element with the class title using an XPath expression. -The expression `//span[@class='title']` means: “Find all `<span>` tags anywhere in the document that have a class of title.” -We pass this XPath string into `document.evaluate()`, which gives us back a list of matching elements. -We then loop through that list and use `.textContent` to get the readable text.

Use `document.evaluate()` to read an XML document

When using `document.evaluate()`, we often work directly on an HTML document. But if you want to use `document.evaluate()` on an XML string, you need to first parse the string into a usable XML format (document) using a tool like `DOMParser`, and then run XPath on that parsed document. Here's a simple, working example:

Imagine you have some XML data on books in your library to read and manipulate. The first thing you want to do is to store it in a variable as a string:

```
// Step 1 – Store XML string in a variable
const xmlString = `
  <library>
    <book>
      <title>JavaScript: The Good Parts</title>
      <author>Douglas Crockford</author>
    </book>
    <book>
      <title>Eloquent JavaScript</title>
      <author>Marijn Haverbeke</author>
    </book>
  </library>`;

// Step 2: Parse string into an XMLDocument with DOMParser
const parser = new DOMParser();
const xmlDoc = parser.parseFromString(xmlString, "text/xml");

// Step 3: Use document.evaluate() to extract titles
const xpath = "//book/title";

const result = xmlDoc.evaluate(
  xpath,          // XPath expression
  xmlDoc,         // context node (root of the XML)
  null,           // namespace resolver (not needed here)
  XPathResult.ORDERED_NODE_SNAPSHOT_TYPE, // result type
  null            // result (optional, reuse old result)
);

// Step 4: Loop through results
```

```
for (let i = 0; i < result.snapshotLength; i++) {  
    console.log(result.snapshotItem(i).textContent);  
}
```

The output of this code will be all the book titles in the document:

JavaScript: The Good Parts Eloquent JavaScript

Here is what happens:

- DOMParser turns a string into a real XML Document.
- `evaluate()` runs the XPath on that document.
- `XPathResult.ORDERED_NODE_SNAPSHOT_TYPE` lets you get multiple nodes in order.
- You use a for loop through the result and extract the data you need using `snapshotItem(i)`.

Remember that in this example we have used an XML string stored in a variable `xmlString`, but in practical examples, that data could be coming from the result of reading the data from a local file, or an AJAX made `fetch()` or `XMLHttpRequest` request etc. The returned data will be received in by your code and stored in a variable, and further processed with the `DOMParser()` and `.evaluate()` in exactly the same way as in this example.

The XPathResult types

The possible `XPathResult` (properties) types are 10 in number, and here is a list of them:

Property name	Code	Description
ANY_TYPE	0	
NUMBER_TYPE	1	Returns a number result, e.g., from count()
STRING_TYPE	2	Returns a string result
BOOLEAN_TYPE	3	Returns a boolean (true/false)
UNORDERED_NODE_ITERATOR_TYPE	4	Iterator: returns nodes one by one (unordered)
ORDERED_NODE_ITERATOR_TYPE	5	Iterator: returns nodes one by one (in document order)
UNORDERED_NODE_SNAPSHOT_TYPE	6	Returns a static list of nodes in no guaranteed order
ORDERED_NODE_SNAPSHOT_TYPE	7	Returns a static list of nodes in order
ANY_UNORDERED_NODE_TYPE	8	Returns any one matching node (not guaranteed to be first)
FIRST_ORDERED_NODE_TYPE	9	Returns the first node (in the order in which it occurs on the document)

Each result type constant (like `XPathResult.STRING_TYPE`) is actually a number behind the scenes, and that is what is represented by the code numbers 0-9 which are returned with the result. When handling the returned result in code you can use either the name or the raw number, but the name is much clearer and the number is not recommended for beginners. I just listed the codes here for you to understand it, but you should not have to worry about them. If you really ever need to check what type was returned, you can do so by using the `XPathResult.ANY_TYPE` property. It is the one that does not commit you to a type up front: you ask for `ANY_TYPE`, and then read back which type you actually got. Here is how to check for it:

```
const result = document.evaluate(
  xpath,
  document,
  null,
  XPathResult.ANY_TYPE,
  null
);
```

Then retrieve the result with the `resultType` property like so:

```
console.log(result.resultType);
```

The output will log a number like:

1, 2, 9, etc.

This is useful only if you request `ANY_TYPE`, and then inspect the actual result type dynamically.

Retrieving the data based on XPathResult type

The response data format will depend on the XPathResult type you specified in `.evaluate()`. This will also determine how you handle the result to retrieve the data it contains. The result object happens to have properties to help you access the data based on the different types that may be returned. For example, retrieve the data like so:

```
const result = document.evaluate(...);
const data = result.propertyName;
```

Here is a list of these access properties of the result object.

Property name	Result access property
ANY_TYPE	You must detect type yourself
NUMBER_TYPE	.numberValue
STRING_TYPE	.stringValue
BOOLEAN_TYPE	.booleanValue
UNORDERED_NODE_ITERATOR_TYPE	.iterateNext()
ORDERED_NODE_ITERATOR_TYPE	.iterateNext()
UNORDERED_NODE_SNAPSHOT_TYPE	.snapshotItem(index)
ORDERED_NODE_SNAPSHOT_TYPE	.snapshotItem(index)
ANY_UNORDERED_NODE_TYPE	.singleNodeValue
FIRST_ORDERED_NODE_TYPE	.singleNodeValue

Let's see some examples of retrieving the returned data in different ways based on the XPathResult type. I intend to leave you with these many examples so you have a deep understanding of these concepts. The key is knowing the nature (type) of the results you want, and passing the XPathResult argument the right property, as well as knowing which property of the result object to use to retrieve the data based on that kind of data which you are expecting. We will look at how to get the following kinds of results from an XPath query return value.

1. Getting a Number (e.g., Count)

2. Getting Text (String)
3. Boolean Check
4. Iterating Nodes (Ordered)
5. Snapshot List of Nodes (Like an Array)
6. Just the First Node

For all the examples, we will be querying this HTML structure, which is the same one we used in the examples above:

```
<body>
  <h1>Book List</h1>
  <ul>
    <li><span class="title">JavaScript: The Good Parts</span></li>
    <li><span class="title">Eloquent JavaScript</span></li>
    <li><span class="title">You Don't Know JS</span></li>
  </ul>

  <script>
    // JavaScript code will go here
  </script>
</body>
```

For each example, I will start by stating the following two things:

- What type I expect
- How I will retrieve the data returned

1. Getting a Number (e.g., Count)

- | | |
|---------------------|-------------------------|
| - Type we want: | XPathResult.NUMBER_TYPE |
| - Result retrieval: | .numberValue |

```
// get number of list items
const xpath = "count(//li)";

// Run XPath query on the current HTML document
const result = document.evaluate(
  xpath,
  document,
  null,
  XPathResult.NUMBER_TYPE,
  null
);

// retrieve the data
console.log("Number of results is: "+result.numberValue);
```

This example uses an XPath expression to search for and count the number of `<li>` elements on the current web page. We use `XPathResult.NUMBER_TYPE` to specify that we expect a number— which makes sense because we are running a `count()` query. Once we get back the result, we retrieve it using the `.numberValue` property of the result object. This is because we are expecting a number, of course.

Because there are three `<li>` items in our HTML example above, the output is the following logged to the console:

Number of results is: 3

2. Getting Text (String)

- Type we want: `XPathResult.STRING_TYPE`
- Result retrieval: `.stringValue`

```
const xpath = '(/span[@class="title"])[1]/text()';

// Run XPath query on the current HTML document
const result = document.evaluate(
  xpath,
  document,
  null,
  XPathResult.STRING_TYPE,
  null
);

// retrieve the data
console.log("The first title is: "+result.stringValue);
```

This example uses an XPath expression to search through all the `<span>` elements on the current web page that have a class of `'title'`. It then gets only the first one of what is matched (`[1]`), and returns its content (using `text()`). We use `XPathResult.STRING_TYPE` to specify that we expect a string— which makes sense because we are running a `text()` query. Once we get back the result, we retrieve it using the `.stringValue` property of the result object. This is because we are expecting a string, of course.

The aim here is to select only the first title, the output is the following logged to the console:

JavaScript: The Good Parts

3. Boolean Check

- Type we want: `XPathResult.BOOLEAN_TYPE`
- Result retrieval: `.booleanValue`

```

const xpath =
    "boolean(//span[@class='title'] [2]/text()='Eloquent
JavaScript')";

// Run XPath query on the current HTML document
const result = document.evaluate(
    xpath,
    document,
    null,
    XPathResult.BOOLEAN_TYPE,
    null
);

// retrieve the data
console.log(result.booleanValue);

```

This example uses an XPath expression to search for the `<span>` elements with a class of `title`, and see whether the second one of them (`[2]`) has `text()` equal to the string `'Eloquent JavaScript'`. We use `XPathResult.BOOLEAN_TYPE` to specify that we expect a boolean response—which makes sense because we are running a `boolean()` query. Once we get back the result, we retrieve it using the `.booleanValue` property of the result object. This is because we are expecting a boolean, of course.

Because the second `<span>` in our HTML does have the text `'Eloquent JavaScript'` in it, the output is the following logged to the console: `true` Otherwise it would have returned: `false`

4. Iterating Nodes (Ordered)

- Type we want: `ORDERED_NODE_ITERATOR_TYPE`
- Result retrieval: `.iterateNext()`

```

const xpath = "//li";

const result = document.evaluate(
    xpath,
    document,
    null,
    XPathResult.ORDERED_NODE_ITERATOR_TYPE,
    null
);

// loop through data
let node = result.iterateNext();
while (node) {
    console.log("Node name: "+node.nodeName);
}

```

```
// e.g. LI
    node = result.iterateNext();
  }
}
```

This example uses an XPath expression to search for all nodes that match the name `li` on the current web page. We use `XPathResult.ORDERED_NODE_ITERATOR_TYPE` to specify that we expect an iterable value—since we expect more than one node returned. Once we get back the result, we loop through it using the `.iterateNext()` method of the result object. This is because having indicated that we want the type `ORDERED_NODE_ITERATOR_TYPE`, we know it returns nodes one by one, so we have to call the `.iterateNext()` method in a loop to keep getting the next value till all the matched elements are cycled over.

Because there are three `<li>` items in our HTML example above, the output is the following logged to the console:

```
Node name: LI Node name: LI Node name: LI
```

5. Snapshot List of Nodes (Like an Array)

```
- Type we want:
XPathResult.ORDERED_NODE_SNAPSHOT_TYPE
- Result retrieval:      .snapshotItem(i)
```

```
const xpath = "//li";

// Run XPath query on the current HTML document
const result = document.evaluate(
  xpath,
  document,
  null,
  XPathResult.ORDERED_NODE_SNAPSHOT_TYPE,
  null
);

// Loop through the result
for (let i = 0; i < result.snapshotLength; i++) {
  console.log(result.snapshotItem(i).textContent);
}
```

This example uses an XPath expression to search for and match all `li` elements on the current web page. We use `XPathResult.ORDERED_NODE_SNAPSHOT_TYPE` to specify that we want an array-like list. Once we get back the result, we loop through it and access the `li` items using the `.snapshotItem(i)` with the index.

Because there are three `<li>` items in our HTML example above, the output is the following logged to the console:

JavaScript: The Good Parts Eloquent JavaScript You Don't Know JS

6. Just the First Node

- Type we want:
`XPathResult.FIRST_ORDERED_NODE_TYPE`
- Result retrieval: `.singleNodeValue`

```
const xpath = "//li";

// Run XPath query on the current HTML document
const result = document.evaluate(
  xpath,
  document,
  null,
  XPathResult.FIRST_ORDERED_NODE_TYPE,
  null
);

// Get only the first node returned (<li>)
console.log(result.singleNodeValue.textContent);
```

This example uses an XPath expression to search for and match all `li` elements on the current web page, just like the one above that returns an array-like list. We use `XPathResult.FIRST_ORDERED_NODE_TYPE` to specify that we want only the first match. Once we get back the result, we retrieve it from the result using the `.singleNodeValue` property of the result object. This is the property meant for single node values, and because this node is simply an HTML element as we know it, we are free to go ahead and use any JavaScript document object method/property on it, and we do so here. We get its text using the document object's `.textContent` property.

The output therefore, is the text of the first `<li>` node being logged to the console:

JavaScript: The Good Parts

The six examples above demonstrate to you, not only the different ways to do XPath queries on an HTML, but also show you how passing a different `XPathResult` type to `document.evaluate()` means that a different format of data will be returned, and a different approach is needed to retrieve the values. We now know how powerful XPath can be. But when do we use XPath, and when are CSS selectors sufficient? Let's talk about that next.

When to use XPath instead of CSS selectors

Use XPath when:

- You need advanced selection logic that would be hard to do with CSS selectors. For example, if you need to select elements by their position (like the 2nd or last child). But for everyday tasks, modern JavaScript developers more commonly use `querySelectorAll()`.
- You want to filter based on attributes, text content, or hierarchical relationships.
- You are working with XML documents, where CSS selectors don't apply.

Use CSS selectors when:

- You need simple, fast, and readable queries (e.g. `.class`, `#id`, `div > p`).
- Your goal is quick DOM access for styling or manipulation.

There is one more way to tell them apart that is worth knowing, because it explains why XPath can express things CSS cannot. XPath is declarative: you describe what you want and it works out how to get it. The DOM methods are imperative: you have to code how to get there, step by step. These are the situations where XPath genuinely earns its place:

- Web scraping, where it is often faster than traversing by hand.
- RSS and Atom feeds, which are still XML-based.
- Legacy enterprise systems, in banking and healthcare especially.
- Browser dev tools - Chrome and Firefox both support `$x()` in the console.
- XML-heavy systems such as SOAP APIs, RSS and SVG manipulation.
- Complex document queries that CSS selectors simply cannot express.

My advice is to master CSS selectors first, because they cover about 95% of what you will ever need. Then learn XPath for when you are dealing with XML, or with HTML complicated enough that `querySelectorAll()` runs out of road. XPath and `document.evaluate()` are advanced but powerful tools in your DOM toolkit. For most everyday tasks, CSS selectors are fine—but XPath shines when your selection needs are more complex or precise. It is a powerful but niche skill: add it to your toolkit after the core DOM methods, not before them. You will get the chance to see some more practical examples of how to use XPath to process XML data in Chapter 18 (File Management), where we read XML out of real files.

The DOMParser

In this section above on the `document.evaluate()` method, you would remember I mentioned that in order to read (parse) data in XML, we must make sure the data is converted from a string into an actual XML document before we can use it in `document.evaluate()`. The data is usually a string in XML format which you could have obtained through any of the following ways:

- You might have created this XML string yourself in code
- Your code may read this data from an XML file and store it in a string. See how to read files in JavaScript in File Management (Chapter 18).
- Your code might have received this data as a response after making an AJAX or API request to read a file or get data that can be from a remote server.

Whatever the case may be, you would typically end up having this data as a string stored in a variable. I need you to understand one thing here. Just because a string is formatted like HTML or formatted like XML does not make it a real HTML or XML document. The `.evaluate()` method of the document object only works with real documents not strings. Therefore, in order to manipulate this data with JavaScript in the same way you would manipulate a real HTML or XML document, you have to first of all convert that string into an HTML document or an XML document before you hand it over to be used in `document.evaluate()`. That's when the DOMParser comes in. DOMParser is a built-in JavaScript object that allows you to convert strings of XML or HTML into actual document objects that your code can interact with using the DOM (Document Object Model). Think of it this way: if you have a block of XML or HTML stored as a string, DOMParser helps you turn that string into something JavaScript can "walk through", read from, or manipulate — just like the document object. DOMParser is part of the DOM Living Standard, maintained by the WHATWG (Web Hypertext Application Technology Working Group) — the same group that maintains standards for HTML and the DOM. It is widely supported across all modern browsers (Chrome, Firefox, Safari and Edge among them). It's safe to use in nearly any project. DOMParser has two jobs:

- Converts XML strings to XML documents
- Converts HTML strings to HTML documents

Again, this is essential when working with data from APIs, file uploads, or manually loaded XML, because XPath and DOM methods (like `document.evaluate()`) only work on real DOM objects, not raw strings. Another JavaScript tool which is not directly related, but works well together with the DOMParser is the FileReader.

FileReader reads files (like XML or HTML) from the user's device and gives you the data as a string. We will learn all about FileReader under File Management in Chapter 18. DOMParser takes that string and turns it into a DOM document you can work with. Let's look at an example of the DOMParser in action, which should be a familiar one to you by now. Here, I will create an XML string and store it in a variable (xmlString):

```
const xmlString = `
  <books>
    <book>
      <title>JavaScript: The Good Parts</title>
    </book>
  </books>
`;

const parser = new DOMParser();
const xmlDoc = parser.parseFromString(xmlString, "text/xml");

const result = xmlDoc.evaluate(
  "//book/title/text()",
  xmlDoc,
  null,
  XPathResult.STRING_TYPE,
  null
);

console.log(result.stringValue);
```

The output will be: JavaScript: The Good Parts

Examples of DOM manipulation

Select an element by its ID value

This is achieved using the JavaScript document object's getElementById() method of which all HTML elements are children. You therefore need to call getElementById() on the document object, and then pass as its argument the value of the ID attribute of the target child element. For example, the following ul element in HTML which has an id attribute with the value of ul:

```
<ul id="ul">
</ul>
```

Select this element in JavaScript like so:

```
let ul = document.getElementById('ul');
```

Select an element by its class

To select an element by its class attribute value you should use the `getElementsByClassName()` method of the document object, and provide it with the name of the class you want to target on the web page. Note that this obviously means you will end up with multiple elements, returned as an `HTMLCollection`—not an array, as we established earlier in this chapter—since unlike id attributes, the same class is usually used on multiple HTML elements.

```
const items =
  document.getElementsByClassName("list-item");
OR
const items = document.querySelector(".list-item");
OR
const list = document.querySelectorAll(
  ".list-item"
);
```

Just like the `document.getElementById()` method of the document object grabs an element by its ID, the `getElementsByClassName()` method belongs to the document object and as its name says, it selects elements by their class names. Mind the 's' in `getElementsByClassName()`. This makes sense because a class is usually assigned to multiple elements anyway. The above alternative ways can all be used to accomplish the same thing, but the last two are more elegant and relatively newer to JavaScript. However, if you are trying to grab all the elements of a certain class in order to manipulate them as an array and do some cool stuff with them, then `querySelector()` is not quite suitable, because it returns only the first matching element and nothing else. Therefore it is often better to use `querySelectorAll()` instead, when you need to select more than one item (with these elements having the same class).

Create a todo list

This will be a very practical demonstration of how you can use the properties and methods that the `HTMLElement` object offers to manipulate HTML elements. A todo list is simply list (li) elements that you manage by dynamically creating and injecting them into the DOM, and also manage the values they contain. It will involve three files, the style sheet (in `index.css`), the HTML code (in `index.html`), and the JavaScript code (in `index.js`). Here is how it works:

- you will start with a blank page (no list item) and a button to add a new item
- you can click on the button to add a new todo (list) item and a text field will appear for you to enter some text for the content of the list item

- if you try to save the entry with no text entered, you will get a validation error, a popup text that disappears after 2 seconds.
- you can add multiple list items
- you can edit the text on an item
- when editing, an edit box (div) will appear with the current text pre-populated in an edit text field.
- while editing, if you submit the edit form with no text entered, you will get a validation error, a popup text that disappears after 2 seconds.
- while editing, if you change your mind, you can cancel the edit, and this will clear any edit text you had started typing in, and hide the edit box.
- delete any list item by clicking on its 'delete' button

index.css

```
body {
    font-family: Arial, Helvetica, sans-serif;
    background: #27292c;
    color: white;
    text-align: center;
}

h1 {
    margin-top: 40px;
    display: inline;
}

#addItemButton {
    position: relative;
    top: -0.2em;
    width: 5em;
    height: 2.3em;
    background-color: dodgerblue;
    color: white;
    border-radius: 5px;
    margin-left: 1rem;
}

#addItemDiv {
    display: none;
}

.itemDiv {
    height: auto;
    border: solid 1px dodgerblue;
}
```

```

color: green;
  font-size: 1rem;
  font-weight: bold;
  padding: 1rem 0rem 0.8rem 0rem;
}

#notify {

  display: none;
  color: red;
  font-size: 2em;
}

.cancelAddItem, .cancelEdit, .deleteButton {
  background-color: red;
  color: white;
}

.cancelEdit {
  width: 2em;
  height: 1.5em;
}

.editButton {
  background-color: orange;
  color: white;
  border-radius: 5px;
}

.editButton, .deleteButton {
  float: right;
  margin-right: 1rem;
  margin-top: -0.5em;
}

.editFormDiv {
  display: none;
}

#gusUl li {

  list-style: none;
}

.btn-primary-sm {
  background-color: dodgerblue;
  color: white;
  border-radius: 5px;
}

.headingUnderline {
  background-color: white;
}

```

index.html

```
<!doctype html>
<html>
  <head>
    <title>Todo list</title>
    <link rel="stylesheet" href="/index.css">
  </head>
  <body>
    <h1>Amazing todo list</h1> <button id='addItemButton'>Add</button>
    <hr class='headingUnderline' />
    <ul id="gusOneUl">

      </ul>

      <div id="gusDiv">
        <span id='notify'></span>
        <div id='addItemDiv'>
          <form id='addItemForm'>
            <input type='text' class='form-control'
id='addItemField' />
            <button class='cancelAddItem'>X</button>
            <input type='submit' class='form-control btn btn-
primary-sm' value='Add'>
          </form>
        </div>

        <ul id="gusUl">

          </ul>
        </div>

        <script type="module" src="/index.js"></script>
      </body>
</html>
```

index.js

```
let todoString = '';
todoString = `
  <div class='itemDiv'>
    <span class='todo-item'></span>
    <button class='deleteButton'>Delete</button>
    <button class='editButton'>Edit</button>
    <div class='editFormDiv'>
      <form class='editForm'>
        <input type='text' class='form-control editField' />
        <button class='cancelEdit'>X</button>
        <input type='submit' class='form-control btn btn-primary-sm'
value='Save'>
```



```

</form>
    </div>
  </div>`;

// -----
// ADD EVENT LISTENERS
// -----
let addItemButton = document.querySelector("#addItemButton");
addItemButton.addEventListener("click", openAddItemDiv);

let addItemForm = document.querySelector("#addItemForm");
addItemForm.addEventListener("submit", addTodoItem);

let cancelAddItemButton = document.querySelector(".cancelAddItem");
cancelAddItemButton.addEventListener("click", cancelNewItem);

// -----

// -----
// THE FUNCTIONS
// -----
function addTodoItem(e) {
  e.preventDefault();

  // get the value of the new todo item
  let newTodoItem = document.querySelector("#addItemField").value;
  if (newTodoItem !== "") {

    const li = document.createElement('li');
    li.innerHTML = todoString;

    // get the ul tag to place our new li in
    let myUl = document.getElementById("gusUl");
    myUl.appendChild(li);

    let targetTodoTextSpan = li.firstElementChild.firstElementChild;
    targetTodoTextSpan.textContent = newTodoItem;

    // hide the new item form again
    let addItemField = document.querySelector("#addItemField");
    addItemField.value = "";

    let addItemDiv = document.querySelector("#addItemDiv");
    addItemDiv.style.display = "none";
  }
  //-----
  // ADD EVENT LISTENERS TO BUTTONS
  // (editButton, cancelEdit, saveEdit)
  //-----

  // Find the .editButton inside

```

```

// Add click event listener to the found editButton
editButton.addEventListener("click", openEdit);

    let cancelEditButton =
li.firstElementChild.lastElementChild.querySelector(".cancelEdit");
    cancelEditButton.addEventListener("click", cancelEdit);

    let editForm =
li.firstElementChild.lastElementChild.querySelector(".editForm");
    editForm.addEventListener("submit", saveEdit);

    // add event to deleteButton
    let deleteButton =
li.firstElementChild.querySelector(".deleteButton");
    deleteButton.addEventListener("click", deleteTodo);

} else {
    notify();
}

}

function openAddItemDiv(e) {
    e.preventDefault();
    let addItemDiv = document.querySelector("#addItemDiv");
    addItemDiv.style.display = "block";
}

function cancelNewItem(e) {
    e.preventDefault();
    // clear any text from the field
    e.target.previousElementSibling.value = "";

    // hide the add item form
    let addItemDiv = document.querySelector("#addItemDiv");
    addItemDiv.style.display = "none";
}

function openEdit(e) {
    e.preventDefault();

    let editFormDiv = e.target.nextElementSibling;
    editFormDiv.style.display = "block";

    // it will be nice to pre-fill
    // the edit form with old value
    let todoTargetTextSpan = e.target.parentElement.firstElementChild;
    let oldText = todoTargetTextSpan.textContent;
    editFormDiv.firstElementChild.firstElementChild.value = oldText;
}

```

```

e.preventDefault();
  // clear any input text from edit field
  e.target.previousElementSibling.value = "";

  // Find the closest ancestor with
  // class .editFormDiv and hide it
  let editFormDiv = e.target.closest(".editFormDiv");
  if (editFormDiv) {
    editFormDiv.style.display = "none";
  }
}

function saveEdit(e) {
  e.preventDefault();
  // get the todo text target span
  let todoTargetTextSpan =
e.target.parentElement.parentElement.firstChild;

  // Get new text from input field
  let newText = e.target.querySelector(".editField").value;

  if (newText !== "") {
    todoTargetTextSpan.textContent = newText;

    // clear edit field & hide it
    e.target.querySelector(".editField").value = "";

    let editFormDiv = e.target.parentElement;
    editFormDiv.style.display = "none";

  } else {
    notify();
  }
}

function notify() {
  let warning = document.querySelector("#notify");
  warning.textContent = "Please enter a value";
  warning.style.display = "block";

  // Hide the warning after 2 seconds
  setTimeout(() => {
    warning.textContent = "";
    // Hide the message
    warning.style.display = "none";
  }, 2000);
}

function deleteTodo(e) {
  e.preventDefault();

```

```
//remove the current li element
e.target.closest("li").remove();
}
```

Adding attributes to an element

There are a few ways to assign attributes to HTML elements dynamically. The following are three ways to add a class or any attribute to an element:

```
const li = document.createElement('li');

li.className = 'list-item';
or
li.setAttribute('class', 'list-item');
or
li.classList.add('list-item');
or
Object.assign(
  li,
  {
    'id': 'truest',
    className: 'listGus-item something'
  }
);
```

In the first example, we use the `className` property of the JavaScript Element object which this list (`li`) element is an instance of to give it a class attribute. You just have to assign it the name of the class you want the element to have, and that is what we have done here. In the second example, we use the `setAttribute()` method of the JavaScript Element object which this list element (`li`) is an instance of, to give it a class attribute. It takes two arguments; the name of the attribute—in this case `'class'`, and the value you want it to have—in this case `'list-item'`. Alternatively, we can use the `classList` member of the JavaScript Element object, again, of which this list (`li`) element is an instance. This `classList` member happens to be an object itself, and you have to call its `add()` method to actually assign the class. You pass in the name of the class you want the element to have. The last alternative is also a powerful way to dynamically assign attributes to an element in JavaScript. We use the `assign()` method of the `Object` object, which we know is the parent of all objects in your JavaScript code. The `assign()` method takes two arguments in our example above. The first argument is the target element (it has to know which element to assign the attributes to), and the second is an object literal in which you can assign as many attributes as you want to the element. In this example, we assign an `'id'` attribute and give it a value of `'truest'`, though of course the value can be anything you want. We also assign some classes to the element. Pay close attention to how multiple classes are passed. They go in as one single string with a space between them, exactly as you would write them in your HTML. Do not be tempted to put

them in an array and separate them with commas: `className` is a string, so an array of `["list-item", "draggable"]` is quietly turned into the text `"list-item,draggable"`, which the browser reads as one strangely-named class rather than two. One string, spaces between the names.

Assign multiple attributes to an element in one go

Any of the above ways to assign attributes to an element is fine, but there are times when you would have multiple attributes to an element, and there is a beautiful way to do it in one block. This is done by the `assign()` method of the base JavaScript Object Object. You simply pass the target element as the first argument to `assign()` and an object as its second argument containing all the attributes you want the element to have and their values. Here is an example:

```
Object.assign(li, {
  'draggable': 'true',
  className: 'list-item draggable'
});
```

There are three things you should note here. The first is that `Object.assign()` needs the element itself as its first argument—leave it out and you have assigned the attributes to nothing at all. The second is how `className` must be used instead of `class` to assign a class to an element when using this approach. The third is the way you pass multiple classes to the same element: as one string, with the class names separated by spaces.

Concatenate a variable with a string and display the result

It can be challenging to display a string that contains a variable and have the variable parsed and its value displayed. The trick is to do two things; a) first, create the string using back ticks which will make your spacing and indentation respected by the browser. b) Secondly, start with a dollar sign, followed by a pair of curly braces around the variable. The following example is how you should do it:

```
let todoString = "";
todoString = `
<div>
<span class='todo-item'>${todoFromStorage.name}</span>
<button name='deleteButton' id="deleteButton${todoFromStorage.id}"
class='deleteButton'>Delete</button>
<button name='editButton' id='${todoFromStorage.id}'
class='editButton'>Edit</button>
<input type='checkbox' name='checkButton' class='checkButton' />
<div class='editDiv' id='editFormDiv${todoFromStorage.id}'>
  <form class='editForm' onSubmit="saveEdit(event)"><input type='text'
class='form-control' name='editField' />
  <a onClick='cancelEdit(event)' class='btn btn-danger-sm
```

```
cancelEdit'>x</a>
  <input type='submit' class='form-control btn btn-primary-sm editValue
editInput' value='Save'>
</form>
</div>
</div>`;
```

How to dynamically insert content into an element

There are two ways to do this; using either the `textContent` or the `innerHTML` properties of the JavaScript `HTMLElement` object of which your web page elements are instances. This means that you can just refer to any of these properties of your element, and assign whatever content you want to them as a string. Here is an example:

index.html

```
<!doctype html>
<html>
  <head>
    <title>Injecting elements into the DOM</title>
  </head>
  <body>
    <ul id="gusUl">

    </ul>

    <script type="module" src="/index.js"></script>
  </body>
</html>
```

index.js

```
let todoString = "";
  todoString = `
<div>
<h1>The new li element is here</h1>
<span class='todo-item'>Test item</span>
<button name='deleteButton' id="deleteButton"
class='deleteButton'>Delete</button>
<button name='editButton' class='editButton'>Edit</button>
</div>`;

const li = document.createElement('li');
li.innerHTML = todoString;
/* OR
li.textContent = todoString;
*/
```

```
// get the ul tag to place our new li in
let myUl = document.getElementById("gusUl");
myUl.appendChild(li);
```

Understand the difference between using `textContent` and `innerHTML` to insert content into an element. The difference lies in their names. Use `innerHTML` when you want to render some HTML tags, as in this case a list item. If you use `textContent`, the `li` item will not be parsed as the browser would read your HTML code, and rather it will insert and display the following raw text as it is:

```
<div> <h1>The new li element is here</h1> <span
  class='todo-item'>Test item</span>
  <button name='deleteButton' id="deleteButton"
    class='deleteButton'>Delete</button>
  <button name='editButton' class='editButton'>
    Edit</button>
</div>
```

Implementing drag and drop

We are going to see how to drag and drop an element. We will see how to drag an element from one container to another. Let's dive straight into the code, and I will explain how it works at the end.

The HTML code

```
<!DOCTYPE html>
<html>
  <head>
    <title>The JavaScript Blueprint</title>
    <link rel="stylesheet" href="index.css">
  </head>
  <body>
    <h1>Drag and drop</h1>

    <div class="empty">
      <div class='fill' draggable="true">
        <img
          src='/images/urban.jpg'
          width="205" height="305"/>
        </div>
      </div>
      <div class='empty'></div>
      <div class='empty'></div>

      <script type="module" src="index.js"></script>
    </body>
  </html>
```

The CSS code

```

.hovered {
    background: #f4f4f4;
    outline: 3px dashed limegreen;
}

/* When the drag starts */
.hold {
    border: 4px solid #ccc;
    outline: 3px dashed limegreen;
    cursor: grabbing;
}

/* While dragging, hide image at the original position */
.invisible {
    display: none;
}

.fill {
    position: absolute;
    cursor: grab;
    padding: 5px;
}

.fill img {
    pointer-events: none;
}

/* Drop targets */
.empty {
    display: inline-block;
    width: 220px;
    height: 320px;
    margin: 10px;
    border: 3px solid salmon;
    border-radius: 5px;
    background-color: dodgerblue;
    position: relative;
    transition: all 0.3s ease;
}

```

The JS code

```

const fill = document.querySelector('.fill');
const empties = document.querySelectorAll('.empty');

// Add drag event listeners to the draggable item
fill.addEventListener('dragstart', dragStart);
fill.addEventListener('dragend', dragEnd);

//drag functions

```



```

console.log('We started dragging');
    this.classList.add('hold');
    setTimeout(() => this.classList.add('invisible'), 0);
}

function dragEnd()
{
    // reset to original class
    this.className = 'fill';
}

//loop through & add drag events to each empty box
empties.forEach(empty => {
    empty.addEventListener('dragover', dragOver);
    empty.addEventListener('dragenter', dragEnter);
    empty.addEventListener('dragleave', dragLeave);
    empty.addEventListener('drop', dragDrop);
});

function dragOver(e)
{
    //prevent default so that the drop event doesn't fire
    //when we drop the element
    e.preventDefault();
}

function dragEnter(e)
{
    e.preventDefault();

    // show dashed green border
    this.classList.add('hovered');
}

function dragLeave()
{
    // remove green border
    this.classList.remove('hovered');
}

function dragDrop()
{
    // remove green border
    this.classList.remove('hovered');

    // move the image
    this.append(fill);
}

```

Here is an explanation of the code, and how drag and drop generally works. Imagine you're picking up a sticker and moving it to another page in a sticker book. That's exactly what you're doing with drag and drop on a web page. Here is what happens behind the scenes, step-by-step as you drag and drop your item:

- First, we have three divs on our web page, to which we all assign the class attribute of 'empty'. In the first div, we place another inner div to hold an image, and this is the div we will allow the user to drag from this parent 'empty' div to any of the other divs. Notice that the image we are using in this inner div is pulled from this path: `/images/urban.jpg`, so in your local project files ensure you have a directory named 'images' in which you have an image named 'urban.jpg', or change this file name if your image is named something else.
- Next, we give that inner (image-holding) div its own unique class attribute so that we can target it individually from JavaScript or CSS. For this, we give it the class value 'fill'.
- Draggable Item: You choose one thing to be draggable by setting `draggable="true"` on it (in this case, the image inside the div given the class of `.fill`).
- Then, in order to make this inner div draggable, we need to give it an attribute called 'draggable', and its value needs to be true. This is a standard, HTML 5 requirement, and it makes this (inner) div draggable without us having to write any code for that. However, having done all this, we have the basis setup, and the dragging will not actually move that element to anywhere. Even if you tried, it will appear to move but then withdraw right back to where it was once you let go of it. To make the dragging actually happen, as in allow the user to drag the item-which in this case is an image, and drop it elsewhere on the DOM, we have to write the JavaScript code to make that happen. Let us proceed and see how that is done.
- Starting the Drag: When the drag starts, we highlight it (using the `.hold` class) and hide it (using `.invisible`) so it looks like it's moving.
- Target Zones: The `.empty` boxes are drop zones. They are set up to react when something is dragged over them.
- The drag events: There are four drag events you need to manage. These are `dragover`, `dragenter`, `dragleave` and `drop`. Let's talk about how they work.
 - `dragenter` and `dragover` are triggered when you drag the image over a box.
 - `dragleave` is triggered when you leave the box without dropping.
 - `drop` is triggered when you let go of the item inside a box.

- Visual Feedback: As you drag, we add a dashed green border to show where the image is hovering over. Once you drop the image, we move image into that new (drop) box.

This makes your web page interactive-no page reloads, just drag, drop, and go.

Dynamically create a table with data from an array

HTML code

```
<!DOCTYPE html>
<html>
  <body>

    <script type="module" src="index.js"></script>
  </body>
</html>
```

JavaScript code

```
// the data array
let clients = [
  {
    name: "John",
    country: "England",
  },
  {
    name: "David",
    country: "USA"
  },
  {
    name: "Gus",
    country: "Canada"
  }
];

// 1. Create the div
let newDiv = document.createElement("div");
newDiv.textContent = "New DIV Element is here";
newDiv.style.border = "2px solid blue";
newDiv.style.backgroundColor = "blue";
newDiv.style.color = "white";
newDiv.style.fontWeight = "bold";
newDiv.style.padding = "10px";
newDiv.style.marginTop = "10px";
document.body.appendChild(newDiv);

// 2. Create the table
let table = document.createElement("table");
table.style.border = "1px solid white";
```

```

table.style.borderCollapse = "collapse";
table.style.marginTop = "10px";

// 3. Create the table header row
let headerRow = document.createElement("tr");

let th1 = document.createElement("th");
th1.textContent = "Name";
th1.style.border = "1px solid white";
th1.style.padding = "5px";

let th2 = document.createElement("th");
th2.textContent = "Country";
th2.style.border = "1px solid white";
th2.style.padding = "5px";

headerRow.appendChild(th1);
headerRow.appendChild(th2);
table.appendChild(headerRow);

// Do the loop to get the data
clients.forEach((person) => {
  // 4. Add a couple of data rows
  let row = document.createElement("tr");

  let rowCol1 = document.createElement("td");
  rowCol1.textContent = person.name;
  rowCol1.style.border = "1px solid white";
  rowCol1.style.padding = "5px";

  let rowCol2 = document.createElement("td");
  rowCol2.textContent = person.country;
  rowCol2.style.border = "1px solid white";
  rowCol2.style.padding = "5px";

  row.appendChild(rowCol1);
  row.appendChild(rowCol2);
  table.appendChild(row);
});

// 5. Append the table to the existing newDiv
newDiv.appendChild(table);

```

Explanation of the Code

- Create the Data: A clients array is defined, where each object contains a person's name and country.
- Step 1: Create the div container
 - A div element is created using `document.createElement("div")`.

- Text is added using `.textContent`.
- Styling is applied directly using JavaScript to add a blue background, white bold text, padding, margin, and a visible border.
- The div is added to the page with `document.body.appendChild(newDiv)`.
- Step 2: Create the table element
 - A table is created and styled to have white borders and collapsed borders for a cleaner look.
 - A top margin is added so it doesn't stick to the text above.
- Step 3: Add the table headers
 - A header row (`<tr>`) is created.
 - Two header cells (`<th>`) for "Name" and "Country" are added with white borders and padding.
 - The header row is appended to the table.
- Step 4: Loop through the clients array
 - Using `.forEach()`, each client object is processed.
 - For each person, a new row (`<tr>`) is created.
 - Two data cells (`<td>`) are filled with that person's name and country.
 - Each cell is styled with white borders and padding.
 - The row is then appended to the table.
- Step 5: Add the table to the div
 - Finally, the completed table is appended inside the previously created div using `newDiv.appendChild(table)`.

This example demonstrates the following:

- How to create and style elements with JavaScript
- How to dynamically loop through data to populate a table

THE WINDOW OBJECT

The window object is the global object in JavaScript that represents the browser window or tab in which your script is running. It provides access to, and control over the browser environment, including the document, history, storage, and various methods for controlling the window itself. It helps manage pop-ups, navigation, and storage. Every global variable or function declared in JavaScript automatically becomes a property or method of the window object. Read this last sentence again, it's so important to keep that in mind. Most window properties and methods

are standardised and work across modern browsers. However, always check for browser compatibility when using lesser-known methods.

Relationship between window and document

Here are some differences between the window object and the document object.

- The document object is a property of window: So, the document object, which represents the current webpage (DOM) can also be accessed in code like so:

`window.document...`

- While document deals with HTML content, window is responsible for broader browser-level functionalities like alerts, navigation, and storage.
- Methods like `querySelector()` and `getElementById()` belong to document, not window.

Properties and methods

Here are some common and useful properties and methods of window:

1) Window Size & Position

`window.innerWidth / window.innerHeight` They get the viewport size

`window.scrollX / window.scrollY` They get the scroll position

`window.scrollTo(x, y)` Helps you scroll to a specific position

Here is an example:

```
console.log(window.innerWidth, window.innerHeight);
window.scrollTo(0, 500);
```

2) Alerts & User Interaction

`window.alert("message")` This displays an alert box

`window.confirm("message")` Returns true or false

`window.prompt(message, default)` The `prompt()` method displays a dialog box that asks for user input and returns the entered value. So, we can say it is used to capture user input.

Example `alert()`:

```
let message = "This is an alert?";

// displays an alert popup
```

```
window.alert(message);
```

Example confirm():

```
let message = "Are you sure?";
let response = window.confirm(message);

if (response == true)
{
    console.log('You said yes');
} else {
    console.log('You said no');
}
```

This will display a popup with the message "Are you sure?", and two buttons, one a 'Cancel', and another 'Ok'. The variable response that confirm() is assigned to will have a value of false if you clicked on Cancel, or a value of true if you clicked on Ok. You can then use a conditional expression to check for this value as in the example above, and take whatever action you wish your code to take.

Example prompt():

```
let message = "What is your name?";
let name = window.prompt(message, 'John Doe');

if (name == "John Doe")
{
    let confirm = window.confirm(
        "Is your surname really Doe?"
    );

    if (confirm == true) {
        console.log("Thanks for confirming");
    } else {
        console.log("That was a mistake then");
    }
}
else if (name != null && name !== "")
{
    console.log("Your name is: "+name);
}
else
{
    console.log("You did not enter a name!");
}
```

This demonstrates a prompt() statement amongst other things like multiple nested if statements and a confirm statement. Here is how it works:

- window.prompt() accepts two arguments: a) The heading text or label of the popup's input field b) the default value that will be pre-entered into the field, to be submitted as the value if the user types nothing in.

- Hopefully, you can understand what the rest of the code does. Basically, by assigning the value of the `prompt()` to a variable 'name', it then displays a confirm popup asking the user to confirm if their surname is really 'Doe'. If the user confirms, then the script terminates with a console message of "Thanks for confirming". On the other hand, if for some reason the user had cleared the first `prompt()` default message and ended up submitting the prompt with no input, the script will terminate with a console message of "You did not enter a name". If the user actually submitted a value via the prompt, the script will exit with a console message of "Your name is theNameTheyEntered".

3) Reloading a window, and navigating to other windows

In other words, you could also say refreshing a web page, and redirecting to another browser URL (Uniform Resource Locator). For these, the window object has a property called 'location' which has two useful members of its own; a property named 'href' and a method named 'reload()'. The href property refers to the path of the current web page, also known as the browser URL. This is what you will find in the browser's search bar. It looks something like this: "<http://my-website.com/index.html>". To get or know the path of a web page, so you can create a link to it, for example, here is how to get it dynamically:

```
let url = window.location.href;
console.log("The URL of your web page is: "+url);
```

This will write the following to your console:

The URL of your web page is: <http://my-website.com/index.html>

To reload or refresh a web page, use the `reload()` method. Here is how to do it:

```
window.location.reload();
```

When it comes to navigating/redirecting to other browser windows (same as browser tabs), JavaScript cannot directly switch between existing browser tabs due to security and privacy concerns. However, you can do the following:

- open a new tab using `window.open()`.
- communicate between windows using `window.postMessage()`.
- redirect a tab using `window.location.href`.

You can navigate to another page in multiple ways. Let's see some ways:

Using the `location.href` property

You can redirect the browser from the current browser path to another web page by assigning a new web page path as a string to the href property of `window.location`.

ion like so:

```
window.location.href = "http://example.com/about.html";
```

This will change your web page to `'http://example.com/about.html'`.

Using the `location.assign()` method

You can also redirect your browser using the `assign()` method of the `location` property. It does the same job as assigning to `href`, and the two are interchangeable—some developers simply prefer a method call to a property assignment, and a method is easier to stub out in tests. Just pass it the string of the new URL as its argument. Here is an example:

```
window.location.assign("https://example.com");
```

Using the `location.replace()` method

This replaces the current page in such a way that there is no history of the previous page—hence you will see no back button in the browser (to go to previous pages). This can have its own uses, in situations where you intentionally do not want the visitor or user accessing any previous view of your application. Just pass it the new URL string as an argument. Here is how to do it:

```
window.location.replace("https://example.com");
```

4) Manipulating the URL with the URL API

We will start here by defining what a URL is, and what an API is. In JavaScript and general web development, URL stands for Uniform Resource Locator. It is a string that specifies the address of a resource on the internet. This resource can be a web page, an image, or an API. That is why you can copy a link of an image on the internet and share it on your web page or social media platform, and visitors who click it can be taken to wherever that link is on the internet to view it. That long text that represents the image link is essentially the path to the website where the image is stored, and it usually points to the website name (domain), and it can even contain the folder name where that file is stored on the server that hosts that domain (website). An API (Application Programming Interface) is like a waiter in a restaurant. Imagine you are at a table looking at a menu. You choose what you want to eat, but you don't go to the kitchen and cook it yourself. Instead, you tell the waiter, and they bring your order from the kitchen to you. An API works the same way in coding. The application (A in API) is the kitchen and the food. As the user (visitor), you do not need to know or see the code used to make the food ready. Rather, it is made easy so that you indirectly run the program through an int-

erface (PI in API) made up of a menu and waiter. An API helps different programs or websites talk to each other. You (the user) make a request, the API takes that request to the system (like a website or a database), and then it brings back what you asked for-like delivering your food. In practice, an API is usually a software application to provide some service that someone has written, which comes with some sort of documentation on how to use it. See this documentation as that restaurant menu, where the developers of the API have made it easy for you by telling you the various methods on the API to call to achieve any of the services it provides, and the type and format of the result you will get back. The various methods to call will be executed on variations of the URL string that lead to the domain on which the API is hosted. Depending on the modification of the URL, each request will trigger a different action on the server, and thus, a different kind of response. A request endpoint could look something like this:

<http://my-api.com/shoes> <http://my-api.com/clothes>

The above example URLs all lead to the same domain but all have a different endpoint, one to shoes, and the other to clothes. In API speak, a URL is referred to as an endpoint. There is a lot more to making API requests besides using an endpoint URL, for example, you have to specify the request method by passing in the right method in the request header. This will tell the receiving (endpoint) server the action you want taken on the endpoint. The request methods in a request header are universal and consist of the following: GET, POST, PUT, PATCH and DELETE for fetching, submitting, replacing, updating and deleting resources, respectively, on the endpoint. Understanding APIs is a whole other topic of its own, and beyond the scope of this book. However, I just needed you now, to understand how important a URL is to web development.

Working with URLs is part of working with web applications. Visiting a web page involves typing a URL string into the search bar of a browser. APIs need URLs as target paths, also known as endpoints, to send requests to. These requests can either be to fetch data from, or send data to these endpoints. That is why URLs are not only used to visit web pages (through the browser), but are used in JavaScript's AJAX calls using `fetch()` or `axios`. They are also used in routing by frontend frameworks like `React.js`, `Vue.js`, and `Angular.js` etc to manage navigation. URLs are also very widely used in handling validation to prevent attacks like phishing to various applications. This makes sense because, a browser URL path is ultimately the access point (door) to your application. With all this being said, there is therefore the clear need for developers to be able to manipulate browser URL strings, whatever their purpose may be. Let us talk about the tools JavaScript has in its tool kit for you to achieve this. The URL API is part of the modern JavaScript improvements. It

was introduced in HTML5 and became widely supported in modern browsers around 2014. It provides the URL constructor (`new URL()`) to parse, manipulate, and construct URLs easily. This API is now a standard part of JavaScript and works across major browsers. The API was developed as part of the WHATWG (Web Hypertext Application Technology Working Group) standards, which also maintains the HTML Living Standard. WHATWG is a collaboration between major browser vendors like Google, Mozilla, Apple, and Microsoft, aiming to improve web technologies. Its main purpose was to replace older, less efficient ways of handling URLs, such as `window.location` string parsing. A URL string consists of several parts that a developer might need to work with individually. These include the following list:

- Protocol (Scheme) – Specifies the protocol used (e.g., `http`, `https`, `ftp`).
- Host (Domain) – The domain name or IP address (e.g., `example.com`).
- Port – The port number (e.g., `:8080` in `https:// example.com:8080`).
- Pathname – The specific path to a resource (e.g., `/about/ us`).
- Query String – Contains key-value pairs for parameters (e.g., `? search=books&page=2`).
- Fragment (Hash) – A section identifier for navigation within a page (e.g., `#contact`).

Each of these parts can be useful in various scenarios, such as routing, fetching data, and manipulating URLs dynamically. As a web developer, you will often run into circumstances when you wish to dynamically extract values from the browser's URL path, or modify it in order to redirect the user to another part of your application, or show them a specific type of content. We have already seen that you can easily get the URL path using the `window.location.href` property. However, to manipulate it and extract values from it or modify parts of it is not as easy. In the old way, once that value has been retrieved using `window.location...`, a developer would have to manually go about splitting the string in various ways to try and work from the URL string the part that represents the domain, or the protocol, or the query string etc. This can be very tricky and prone to mistakes. For this reason, doing it this old way is not recommended, and the URL API is the way to go about it. Let's talk about how the URL API makes working with URLs a breeze.

Key components of a URL

A URL can be broken down into parts, which JavaScript can parse and manipulate using the built-in URL API. Let us see this API in action by demonstrating using an example URL string, to see how it can detect the various components accurately.

```
const url = new URL(
  "https://www.example.com:8080/path/to/page?query=123#section"
);

console.log("The URL is: "+url);
console.log("Protocol: "+url.protocol);
console.log("Hostname: "+url.hostname);
console.log("Port: "+url.port);
console.log("Path: "+url.pathname);
console.log("Query string: "+url.search);
console.log("Hash/fragment: "+url.hash);
```

When you place this code in your JavaScript and run it in the browser, you will get the following result written to your console:

```
// www.example.com:8080/path/to/page?
The URL is: https:
query=123#section
```

Protocol: https: Hostname: www.example.com Port: 8080 Path: /path/to/page
Query string: ?query=123 Hash/fragment: #section

You find that the URL object successfully extracts the value of all the various components of the long and complex URL string. This is better than you could ever manage by extracting it yourself, manually. Let's pick out a few lessons about how to identify URL string components from here. We can see the path refers to the section after the hostname and the port number. It is always separated from them by a forward slash ('/path/to/page'). Note that a query string is the part that comes in your URL after a '?' character, and its syntax is always 'key=value'. In this example URL, the query string is 'query', and its value is '123'. A fragment is always the value that follows a hash (#) character, which is 'section' in this case.

Parsing a URL

Let us see another example of the URL API in action. Let us use it to quickly extract the value of a specific query string.

```
const url = new URL(
  "https://www.example.com/search?q=JavaScript"
);

console.log(url.searchParams.get('q'));
```

The result of this code is the following being written to the console:

```
JavaScript
```

We learn here that once you instantiate a URL object, you get a property for handling query strings, and that property is `searchParams`. To then read the value of

any query string in that URL, we simply pass that query string's name to the `get()` method of `searchParams` like so:

```
url.searchParams.get("queryStringName");
```

Modifying URL components

We can also modify URL strings on the fly, for example, let us look at how we can add a path to a URL which did not have a path before, and let's assign a query string to the URL, then view the result of the modified string.

```
const newUrl = new URL("https://api.example.com");
newUrl.pathname = '/users';
newUrl.searchParams.set('id', '123');

console.log(newUrl.href);
```

The result of this code is the following being written to the console:

```
https://api.example.com/users?id=123
```

We learn from this that to set the value of any query string, we need to pass the desired query string name as a string to the first argument of the `set()` method of `searchParams`, and its value as the second argument to `set()`, like so:

```
newUrl.searchParams.set('id', '123');
```

Modifying query parameters

Here is how you can dynamically modify the URL of the current web page, and refresh the page so it has the new query string parameters. As you can probably already guess, this will involve using the `searchParams.set()` method and the `window.location` properties to modify the query string, and reload the web page, respectively. Here is how:

```
const url = new URL(window.location.href);
url.searchParams.set('sort', 'asc');

window.location.href = url.href;
```

This will reload the current web page, and it will now have a query string of `'?sort=asc'` added to it.

Understanding Blobs

Let's talk about what Blobs mean in JavaScript. It is a concept which is worth understanding, as we will come across it when we come to deal with the management of files, and the handling of various resources in memory. If it does not make

sense now, do not worry, it will do when we come to look at it in action. Imagine you're working with a container full of stuff—this could be text, images, or even video data. In JavaScript, this kind of container is called a Blob, which stands for "Binary Large Object". Let's forget its fancy name for a second and focus on what it really does. Think of a blob as a lunchbox. Let's say you just modified a version of a text file in your browser. Now you want to let the user download it. You can't just hand them the text—you need to package it up neatly. That's where a Blob comes in.

A Blob is like a lunchbox where you put your file contents (a sandwich, a juice carton and an apple). Once it's packed, the browser knows how to hand it over to the user as a downloadable file. In more technical terms, but still simple; a Blob lets you store and treat raw data (text, images, anything!) like a file. You can create it in JavaScript like so:

```
const blob = new Blob([data], { type: "text/plain" });
```

Then, you can generate a download link using:

```
URL.createObjectURL(blob)
```

Generating links to assets created in memory

Sometimes you will have assets like files or images that were dynamically created in memory—meaning they are not physically stored in your file system. JavaScript, as we will come to learn in Chapter 18 (File Management), is restricted from accessing your computer's file system for security reasons. When you create files therefore—which you definitely can in JavaScript—they will be stored as objects in memory. Being in memory, they are temporary, so JavaScript has provided a way for you to make such objects downloadable if the user wants to keep them. The URL API has a method named `createObjectURL()` which is used to create a URL that represents a Blob or File object, allowing you to generate a temporary URL that can be used, for example via a button or link, to reference the object from the browser. Here is how `createObjectURL()` works:

- Takes a Blob/File as input: You pass a 'Blob', 'File', or 'MediaSource' object to '`URL.createObjectURL()`'
- `URL.createObjectURL()` generates a unique URL: and returns a DOMString (a URL) that points to the object in memory.
- Temporary lifetime: The URL is valid only while the document (window/browser tab) that created it is open. It should be revoked when no longer needed to free memory. The URL API has a method for this purpose known as `revokeObjectU-`

RL()). You pass it the URL link to the object you created. Do this in code when you expect the user to have completed the downloading of the object (eg file/blob/media etc). So, always remember to use URL.revokeObjectURL() after using URL.createObjectURL() in order to prevent memory leaks.

Let us look at a very common use case for the createObjectURL() method, which is to generate a downloadable file from text content. We will see more examples like this in Chapter 18 (File Management), but I need you to see the createObjectURL() method in action here so you can understand its practical use.

```
// Create a Blob from text content
const text = "Hello, world! This is a downloadable file.";
const blob = new Blob([text], { type: 'text/plain' });

// Generate a URL for the Blob
const url = URL.createObjectURL(blob);

// Create a download link
const a = document.createElement('a');
a.href = url;

// Set the filename of your choice
a.download = 'example.txt';
a.textContent = 'Download File';

// Append the link to the DOM (optional)
document.body.appendChild(a);
```

Elsewhere in your code, make sure to clean up the URL link created in memory when it is no longer needed by revoking the URL when done. This will prevent memory leaks. A memory leak in programming occurs when a computer program fails to release memory it no longer needs, causing unnecessary memory consumption over time. This can slow down or crash the system as available memory gets exhausted. If you created a link or button in the browser to reference the URL (link) to the object as we did in the example above, then the best way to clear that from memory by implementing the revoke action (after the file is downloaded) is to add an event listener to that same button or link. Let's add one for our example:

```
a.addEventListener('click', () => {
  setTimeout(() =>
    URL.revokeObjectURL(url), 300);
});
```

Here, we are saying; once the button is clicked, we use the setTimeout() function to wait 300 milliseconds—that is, three tenths of a second—before getting rid of the object's URL from memory. The idea is that this is enough of a gap for the browser to have started the download. You get to see the setTimeout() function

again, which we talked about in Chapter 14 (Dates and Time). As a reminder, it accepts two arguments, a function which contains the action it needs to take, and the second argument is time in milliseconds. This time is the amount of time it needs to wait before executing the function/action. Hopefully you can see how useful the `createObjectURL()` method of the URL API can be. Again, some of its use cases are: -Generating downloadable files dynamically. -Displaying images/videos from user-uploaded files (``). -Streaming media.

5) Navigating Through Browser History

JavaScript gives access to the browser's session history. For this it uses the `window.history` property. Let's just dive straight into code examples as it should all be self-explanatory to you by now.

Go back or forward in browser history

Go back to a previous page with `history.back()`, or forwards with the `history.forward()` method.

```
// Go back one page
window.history.back();

// Go forward one page
window.history.forward();
```

Go multiple steps in history

There is a way for you to dynamically allow the user or visitor to jump (even multiple pages) to a specific page, going back a step number in history. This can have its own uses, for example, in a scenario where a user is completing a multi-page form, and you want the user to be able to click on a button to return to a specific earlier step in the form. Anyway, for that, you can use the `go()` method of the history property, which accepts a number as its argument. That number is the number of steps in history to go, where a positive number means forward in history, while a negative number will take the user that many pages back in their browsing history. Here is how to do it:

```
// Go two pages back
window.history.go(-2);

// Go two pages forward
window.history.go(2);
```

6) Opening a New Child Window

You can open a new browser window (or tab) from a parent window.

Opening a New Window

The following code will open up a new browser window (tab) in the specified dimensions (width and height).

```
let childWindow = window.open("https://example.com", "_blank",  
    "width=600,height=400");
```

When testing this, your web page might be set to block popups, but it will usually tell you if it has blocked it. All you then have to do is click on the warning in browser (usually near the search bar), and choose not to block popups from your web page.

Security issues with multiple tab navigation

I must say that relying on `window.open()` for inserting content dynamically is not a good idea. There are quite a few reasons for this. Pop-up blocking: modern browsers often block `window.open()` calls unless triggered by direct user interaction (like a button click) on the page. Another issue comes from browser Security Restrictions. Some browsers prevent writing to a newly opened window immediately. There is also an issue with third-party browser extensions. Some of them may block pop-ups. Some browsers restrict `document.write()` on new windows, especially if `noopener` or `noreferrer` attributes are used. If the new window has a different domain, it may also be blocked by some browsers due to cross-origin restrictions. Most modern browsers block pop-ups unless triggered directly by a user interaction (e.g., clicking a button). Even if the window opens, some browsers prevent JavaScript from modifying its content. If `window.open()` loads a new page from a different domain, modifying document will fail due to cross-origin security policies. There is also some browser-specific behaviour. For example, some browsers (especially Chrome) treat new tabs/windows differently and restrict what can be done with them. This enforcement of security limitations by browsers with different levels of strictness, makes the feature really inconsistent and thus unreliable for real-world applications.

QUIZ — DOM and URL Manipulation

This page contains the Q & A (questions and answers) for this chapter — Chapter 15: DOM and URL Manipulation. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

This is the longest chapter in the book, so it has the longest quiz. Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 12 to 16 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What is the difference between a node and an element? Given this markup, how many nodes does `parent.childNodes` return, and how many does `parent.children` return?

```
<div id="parent">
  Text Node
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</div>
```

Clue: count the line breaks, not just the tags. One answer is larger than most people expect.

2. Name the three document methods that return more than one element, and say which type each one hands back.

Clue: two return the same type as each other. The odd one out is the newest of the three.

3. What is the difference between a "live" `HTMLCollection` and a "static" `NodeList`?

Clue: one of them notices when the page changes underneath it.

4. True or false: you can use a `for...of` loop on an `HTMLCollection`. And what is the one array method a `NodeList` has that an `HTMLCollection` does not?

Clue: this catches people out because the two answers point in opposite directions.

5. Neither an `HTMLCollection` nor a `NodeList` is a real array. Give the two ways of turning one into a real array.

Clue: one is a method on the Array object. The other is three dots.

6. What is the difference between `textContent` and `innerHTML`, and why is one of them a security risk?

Clue: the risk has a three-letter name and involves an image that fails to load.

7. `innerText` and `textContent` both give you the text inside an element. What makes them different?

Clue: one of them cares what the CSS is doing.

8. You are inside a click handler and need to find the nearest ancestor with the class `.editFormDiv`. Which method, and what exactly do you pass it?

Clue: it is a CSS selector, so it needs its punctuation.

9. What is wrong with each of these two lines?

```
element.setAttribute("disabled");  
Object.assign(li, { className: ['list-item', 'draggable'] });
```

Clue: one throws immediately. The other silently gives you one class where you wanted two.

10. `document.evaluate()` takes five arguments. What is the fourth one for, and why does it change how you read your results?

Clue: asking for a number and asking for a list of nodes cannot possibly be read back the same way.

11. What is the difference between the `window` object and the `document` object? Give one thing that belongs to each.

Clue: one is the browser, one is the page. And one of them owns the other.

12. EXERCISE. Given three paragraphs with the class `note`, select them all and print the text of each — using a real array method, not a plain loop.

Clue: you will need question 5 to get there.

13. EXERCISE. Create a `<div>`, give it the classes `card` and `wide` in one go, put a `<p>Hello</p>` inside it, and add it to the page.

Clue: remember what question 9 taught you about multiple classes.

14. EXERCISE. Using XPath, count the `<li>` elements on a page and print the number. Then use XPath again to get the text of the first `<li>`.

Clue: two different XPathResult types, and two different properties to read the answer from.

15. EXERCISE. Take the URL `https://shop.example.com:8080/cart?item=42#summary` and print its protocol, hostname, port, pathname, the value of the `item` query parameter, and its hash.

Clue: one built-in constructor gives you all six.

16. EXERCISE. Take the current page's URL, add a query parameter `sort=asc` to it, and print the resulting full URL — without reloading the page.

Clue: build a URL object from it, set the parameter, then read `.href` back.

ANSWERS

1. A **node** is any item in the DOM — that includes elements, but also text and comments. An **element** is the specific kind of node that represents an actual HTML tag.

- `parent.childNodes` returns **5** nodes.
- `parent.children` returns **2** elements.

The five are:

<code>[0]</code>	<code>#text</code>	the line break and "Text Node"
<code>[1]</code>	<code><p></code>	Paragraph 1
<code>[2]</code>	<code>#text</code>	the line break between the two paragraphs
<code>[3]</code>	<code><p></code>	Paragraph 2
<code>[4]</code>	<code>#text</code>	the line break before <code></div></code>

Three of the five are whitespace you never typed on purpose. Every line break and run of spaces between your tags becomes a text node of its own, which is exactly why `childNodes`, `firstChild`, `lastChild`, `nextSibling` and `previousSibling` so often surprise people.

When you want elements and only elements, use `children`, `firstElementChild`, `lastElementChild`, `nextElementSibling` and `previousElementSibling`.

2. `document.getElementsByClassName()` → `HTMLCollection`
`document.getElementsByTagName()` → `HTMLCollection`
`document.querySelectorAll()` → `NodeList`

The first two return a **live** `HTMLCollection`. `querySelectorAll()` is the newest of the three and returns a **static** `NodeList`.

Note that `querySelectorAll()` is not the only source of a `NodeList` — `childNodes` gives you one too, and that one is live.

3. A **live** collection is tracked by the browser and updates itself when the page changes. Add a matching element to the DOM after you took the collection, and it appears in the collection without you doing anything.

A **static** `NodeList` is a snapshot. It shows what matched at the moment you asked, and it never changes afterwards, no matter what happens to the page.

This matters more than it sounds. Looping over a live collection while adding elements that match it is a classic way to write an infinite loop.

4. **True** — `for...of` works on an `HTMLCollection`, and on a `NodeList`. Both are iterable.

The array method a `NodeList` has and an `HTMLCollection` does not is

`.forEach()`.

That is the pairing worth memorising, because the two facts pull in opposite directions:

`for...of` `HTMLCollection`: yes `NodeList`: yes `.forEach()` `HTMLCollection`: NO
`NodeList`: yes

Neither of them has `.map()`, `.filter()` or any of the rest.

5.

```
// 1. Array.from()
const items = Array.from(document.getElementsByTagName("p"));

// 2. the spread operator
const nodes = [...document.querySelectorAll("p")];
```

Both work on either type. Once you have a real array you get the whole Array object — `.map()`, `.filter()`, `.reduce()` and everything else.

Note that it is the square brackets in the second one that create the array; the three dots just spread the collection out inside them.

6. `textContent` treats everything you give it as **plain text**. Hand it `"<h1>Hello</h1>"` and the page shows those angle brackets literally.
`innerHTML` treats what you give it as **real HTML**. Hand it `"Hello<b>World</b>"` and you get World in bold.

The security risk is **XSS (Cross-Site Scripting)**, and it belongs to `innerHTML`:

```
elem.innerHTML = "<img src='x' onerror='alert(`\`Hacked!\`')'>";
```

The `src='x'` points at an image that does not exist, so loading it fails, so the browser fires the `onerror` handler — and runs whatever JavaScript is sitting in it. Now imagine that string came from a comment box on your blog, and instead of an alert it reads the visitor's session cookie.

The rule: **when you are inserting anything a user typed, use `textContent`**. If you genuinely must use `innerHTML`, sanitise the input first.

7. `textContent` gives you all the text that is in the HTML, whether the visitor can see it or not. `innerText` gives you only what is actually rendered on screen.

```
<div id="myDiv">Visible <span style="display:none">hidden</span>
text</div>
```

```
div.textContent // "Visible hidden text" div.innerText // "Visible text"
```

Two more differences. `innerText` collapses runs of spaces and line breaks the way the browser does when it draws the page, while `textContent` hands the whitespace back exactly as it sits in your markup. And because `innerText` has to know what is visible, the browser may need to work out the page layout before it can answer, which makes it slower.

Default to `textContent`. It is faster, and its answer does not change when somebody edits the CSS.

8. `closest()`:

```
let editFormDiv = e.target.closest(".editFormDiv");
```

You pass it a **CSS selector**, written exactly as you would write it in a stylesheet — so `".editFormDiv"` with the dot for a class, `"#myId"` with the hash for an id, or a bare tag name. Passing `"editFormDiv"` with no dot would look for a `<editFormDiv>` tag and find nothing.

Two things worth knowing about it. It **starts with the element you called it on** — if that one matches, that is what you get back — and only then works upwards. And it returns `null` if it reaches the top without finding anything, which is why the chapter's example checks before using the result.

9. **Line 1 throws immediately.**

```
element.setAttribute("disabled");
// TypeError: 2 arguments
// required, but only 1 present
```

`setAttribute` always needs both. For attributes that do not take a value, like `disabled` or `required`, you pass an empty string:

```
// <input disabled="">
element.setAttribute("disabled", "");
```

Line 2 fails silently, which is worse. `className` is a *string*, so handing it an array makes JavaScript convert that array to text — and arrays convert by joining with commas:

```
className: ['list-item', 'draggable'] -> "list-item,draggable"
```

That is **one** class with a strange name, not two classes. No error, nothing in the console, and your CSS simply never applies. What you want is one string with spaces:

```
Object.assign(li, { className: 'list-item draggable' });  
// classList -> ['list-item', 'draggable']
```

10. The fourth argument is the **XPathResult type**. It tells `document.evaluate()` what shape of answer you want — a number, a string, a boolean, an iterator of nodes, or a snapshot list of nodes.

It changes how you read the results because each type puts the answer in a different place:

NUMBER_TYPE	-> .numberValue
STRING_TYPE	-> .stringValue
BOOLEAN_TYPE	-> .booleanValue
ORDERED_NODE_ITERATOR_TYPE	-> .iterateNext()
ORDERED_NODE_SNAPSHOT_TYPE	-> .snapshotItem(i), with .snapshotLength
FIRST_ORDERED_NODE_TYPE	-> .singleNodeValue

There are ten types in all, numbered 0 to 9. `ANY_TYPE` is 0, and it is the one that does not commit you up front — you ask for it, then read `.resultType` to find out what you actually got.

Ask for a number and try to read `.snapshotItem(0)` and you will get nothing useful, so the type you request and the property you read must always be chosen together.

11. `window` is the **browser window or tab**. `document` is the **page loaded inside it**.

And `document` is a property of `window`, so `window.document` and `document` are the same object.

- Belonging to `window`: `alert()`, `confirm()`, `prompt()`, `innerWidth`, `location`, `history`, `open()`, `scrollTo()`, `getComputedStyle()`, `pageXOffset`.
- Belonging to `document`: `getElementById()`, `querySelector()`, `querySelectorAll()`, `createElement()`, `body`, `evaluate()`.

A useful way to keep it straight: if it is about the page's *content*, it is on `document`. If it is about the *browser* around that content — its size, its address bar, its history — it is on `window`.

One more thing worth remembering: every global variable or function you declare becomes a property of `window`.

```
12. // three ways in, all equivalent
    const notes = [...document.querySelectorAll(".note")];

    notes.forEach(note => console.log(note.textContent));
```

Output, for three paragraphs reading "First note", "Second note", "Third note":

First note Second note Third note

`querySelectorAll()` would actually let you call `.forEach()` directly, because `NodeLists` have it. But spreading it into a real array first is the habit worth having, because it works whichever of the three selection methods you used, and it gives you `.map()` and `.filter()` as well.

```
13. const div = document.createElement("div");

    // one string, spaces between the class names
    div.className = "card wide";

    const p = document.createElement("p");
    p.textContent = "Hello";

    div.appendChild(p);
    document.body.appendChild(div);

    console.log(div.outerHTML);
    // <div class="card wide"><p>Hello</p></div>
```

Two alternatives for the classes, both fine:

```
div.classList.add("card", "wide");
div.setAttribute("class", "card wide");
```

Note the order of the last two lines. We build the whole thing first and append it to the page once, at the end. Appending an element and then filling it works too, but doing it this way means the browser only has to redraw the page a single time.


```

14. // 1. count the list items – we want a NUMBER back
    const countResult = document.evaluate(
        "count(//li)",
        document,
        null,
        XPathResult.NUMBER_TYPE,
        null
    );
    console.log("Number of items: " + countResult.numberValue);

    // 2. the text of the first
    // one – we want a STRING back
    const textResult = document.evaluate(
        "(//li)[1]/text()",
        document,
        null,
        XPathResult.STRING_TYPE,
        null
    );
    console.log("First item: " + textResult.stringValue);

```

For a page with three list items reading "one", "two" and "three":

Number of items: 3 First item: one

Notice that the two calls differ in exactly two places: the type asked for in the fourth argument, and the property read from the result. That is the whole idea of question 10.

Notice too that XPath counts from **1**, not 0 — `(//li)[1]` is the first one. It is one of the few places in this book where you will not be counting from zero.

```

15. const url = new URL(
    "https://shop.example.com:8080/cart?item=42#summary"
);

console.log("Protocol: " + url.protocol);
console.log("Hostname: " + url.hostname);
console.log("Port: " + url.port);
console.log("Path: " + url.pathname);
console.log("Item: " + url.searchParams.get("item"));
console.log("Hash: " + url.hash);

```

Output:

Protocol: https: Hostname: shop.example.com Port: 8080 Path: /cart Item: 42
Hash: #summary

Two details that catch people. `protocol` includes the colon — `"https:"`, not `"https"`. And `hash` includes the `#`, just as `search` would include the `?`.

Doing this by splitting the string yourself is possible, and it is how everybody did it before the URL API existed. It is also fiddly and easy to get wrong, which is precisely why the URL API was created.

```
16. const url = new URL(window.location.href);  
  
    url.searchParams.set("sort", "asc");  
  
    console.log(url.href);
```

If the page was at `https://example.com/products`, this prints:

```
// example.com/products?sort=asc  
https:
```

Remember which method does what: `get()` **reads** a parameter, `set()` **writes** one. And `set()` replaces the parameter if it is already there rather than adding a second copy of it, which is usually what you want.

To actually navigate to the new URL you would then assign it:

```
window.location.href = url.href;
```

That reloads the page at the new address. Leave that line out, as we did above, and you have simply built the URL string without going anywhere — which is often all you need, for instance when you are putting it into a link.

Chapter 16 — THE CANVAS ELEMENT

- The Canvas element—for drawing on the web
 - Draw a rectangle
 - Draw a circle
 - Printing text on the canvas
 - Create a drawing app
 - The paintbox setup
 - Detecting colour selection
 - Eraser functionality
 - Positioning, animation and collision detection
 - Positioning
 - Animation
 - Collision detection
 - Detecting the collision of two shapes
 - A word about the shape we are really testing
 - The precise version, if you need it

The Canvas element—for drawing on the web

The `<canvas>` element is a special part of HTML5 that lets you draw graphics using JavaScript — right in your browser! Think of it like a blank sheet of paper (or canvas!) where you can use code to draw lines, shapes, images, animations, and even build simple games. The `<canvas>` element was introduced with HTML5, around 2009–2010, as a way to allow developers to draw and animate things directly in the browser without using Flash or plugins. It became a big part of making the web more interactive and powerful. Before canvas, making interactive graphics (like games, charts, or animations) was tricky and usually needed extra software. Now, with canvas and JavaScript, you can draw anything directly on the page, using only your browser. The canvas element is supported by all modern web browsers and is used in the following ways:

- Games
- Charting tools (like graphs)
- Animations

- Image editors
- Drawing apps
- Data visualisations

In this section, we will build a drawing application, with which you can draw things on the canvas, and use different colours for the strokes, just like the coloured pencils of a sketchbook—only you will be doing this on the web, by holding and dragging your mouse. In Chapter 20 where we look at working with Images, I will also show you how to use the HTML5 Canvas API to create an image editing application. This is what a canvas element looks like on an HTML page:

```
<canvas id="myCanvas" width="300" height="150"
  style="border:1px solid black;">
</canvas>
```

How does it relate to other DOM elements? Just like other elements (like `<div>`, `<p>`, or `<img>`), canvas is part of the DOM. You can do the following:

- create it with `document.createElement("canvas")`
- add it to the page with `appendChild()`
- get it with `getElementById()`
- style it with CSS
- control it with JavaScript

But unlike an `<img>`, canvas doesn't have content on its own—you draw everything using a special drawing tool called the canvas context, which comes as part of the Canvas itself. This is why in all Canvas operations in code, you will notice they all begin with a setup of the context, followed by the process of building stuff upon that context. Let's see some examples of the Canvas in action. I think the best way for you to get introduced to the canvas is if I show you practical examples in code, and then explain what the various properties and methods do.

Draw a rectangle

HTML code

```
<canvas
  id="myCanvas"
  width="300"
  height="150"
  style="border:1px solid black;">
</canvas>
```

JavaScript code

```
let canvas = document.getElementById("myCanvas");

// Create the context – this gives us the drawing tool
let ctx = canvas.getContext("2d");

// Set fill color to blue
ctx.fillStyle = "blue";

// Draw a rectangle (x, y, width, height)
ctx.fillRect(50, 40, 200, 60);
```

Explanation:

- The line `getContext("2d")` is how you access the 2D drawing tools of canvas.
- The `.fillStyle` property of the context sets the colour that any subsequent filling will use—rather like choosing which pot of paint your brush is about to go into.
- The `.fillRect()` method of the context, actually draws the filled rectangle. The 4 arguments it takes are very clear from the names; the `x` and `y` coordinates where you want the element to be placed in the canvas. The next 2 are for the width and height you want for the rectangle. The rectangle created in this example will be one 200 pixels wide and 60 pixels high.

Draw a circle

HTML code

```
<canvas
  id="myCanvas"
  width="300"
  height="150"
  style="border:1px solid black;">
</canvas>
```

JavaScript code

```
let canvas = document.getElementById("myCanvas");
let ctx = canvas.getContext("2d");
ctx.beginPath();

// x, y, radius, start angle, end angle
ctx.arc(150, 75, 40, 0, 2 * Math.PI);
ctx.fillStyle = "red";
ctx.fill();
```

This creates a nice red circle against a white background, and it looks like the national flag of Japan. There is a lot more involved here, like the `.beginPath()`, `.arc()` and `.fill()` methods of the canvas context, so let me break it down for you. Understanding how `ctx.beginPath()`, `ctx.arc()`, and `ctx.fill()` work together is key to mastering the canvas API.

Explanation:

- `ctx.beginPath()` It starts a new path. This tells the canvas that you're about to start drawing a new shape. It matters because, without `beginPath()`, your new shape might accidentally get connected to the previous one (if you drew something earlier). It ensures shapes are independent. This is why we call it before the other methods.
- `ctx.arc(150, 75, 40, 0, 2 * Math.PI)` It defines a circle by drawing an arc:
 - 150 and 75 represent the x and y coordinates, respectively, of the center of the circle.
 - The third argument (40) is its radius, which is how far the circle reaches out from its centre.
 - The last two arguments (start angle and end angle), where 0 is the standard starting angle (in radians) while $2 * \text{Math.PI}$ is the standard ending angle (where a full circle = 360° or 2π radians). It is important to note that `ctx.arc()` only defines the path of the circle. It doesn't actually draw or color it yet. It just traces the shape's outline (like drawing with a pencil).
- The `.fillStyle` property is what declares that the circle will be red in color when it is filled. Note that it does not do the filling (colouring) itself.
- The `.fill()` method of the context, actually draws the circle. In doing so, it works with `.arc()` and `.fillStyle` to draw the shape following the path you just defined with `arc`, while filling (colouring) it with the colour defined by the `.fillStyle` property. It actually paints the inside of the shape (like coloring it with a brush).

In this example we draw a circle with a filled in color. If you wanted a stroked circle, which means a drawn circle outline with no colour filling, you can achieve that by using the combination of the following two properties:

- `ctx.strokeStyle` e.g. `ctx.strokeStyle = "red";`
- `ctx.stroke()` instead of:
- `ctx.fillStyle`
- `ctx.fill()`

Test it out with the following code, and you will get a circle outline:

```
let canvas = document.getElementById("myCanvas");
let ctx = canvas.getContext("2d");
ctx.beginPath();
ctx.arc(150, 75, 40, 0, 2 * Math.PI);
ctx.strokeStyle = "red";
ctx.stroke();
```

Printing text on the canvas

HTML code

```
<canvas
  id="myCanvas"
  width="300"
  height="150"
  style="border:1px solid black;">
</canvas>
```

JavaScript code

```
let canvas = document.getElementById("myCanvas");
let ctx = canvas.getContext("2d");
ctx.font = "20px Comic Sans MS";
ctx.fillStyle = "blue";
ctx.fillText("Hello Canvas!", 70, 50);
```

Explanation:

- The `ctx.font` line will set the font type for the text you will write
- The `.fillStyle` property will be the color of the text.
- The `.fillText()` is the equivalent of `fill()` or `stroke()`. So, instead of using a combination of `.fillStyle` and then `fill` to draw the element, when writing text, we use a combination of:
 - `.fillStyle` - for the text color, and
 - `.fillText()` to write the text.

Note that you pass in the text to be written to `.fillText()` as a string, and then tell it the x and y coordinates for where you wish to position the text on the canvas.

```
ctx.fillText("Hello Canvas!", 70, 50);
```

The x coordinate is 70, and the y coordinate is 50.

Create a drawing app

This one is going to be more complex than the three examples above, but it really is not hard to understand. I will try to explain what every line of code does, and it

should all become clear to you.

HTML code

```
<canvas
  id="paintCanvas"
  width="500"
  height="400"
  style="border:1px solid black;">
</canvas>
<br>
<div id="colorIndicator"
  style="width:20px;
  height:20px;
  border-radius:50%;
  background-color:black;
  display:inline-block;
  border:0.5px solid black;">
</div>
<p id="indicatorName" style="display:inline-block;">Pen</p>
<div id="eraser"
  style="background-color:dodgerblue;font-weight:bold;
  color:white;width:50px;height:30px;
  padding:6px 1px 0px 1px;border-radius:4px;">
  Eraser
</div>
<br>
<canvas
  id="paintBox"
  width="500"
  height="100"
  style="border:1px solid black;">
</canvas>
```

JavaScript code

```
let colorIndicator = document.getElementById("colorIndicator");
let indicatorName = document.getElementById("indicatorName");
const canvas = document.getElementById("paintCanvas");
const ctx = canvas.getContext("2d");
let paintColor = "black";
let penSize = 4;
let canvasBackground = "white";

// Painting state
let painting = false;

// Start drawing
canvas.addEventListener("mousedown", (e) => {
  painting = true;
```



```

const x = e.offsetX;
const y = e.offsetY;

// Set the pen up before we put it down on the paper
ctx.lineWidth = penSize;
ctx.lineCap = "round";
ctx.strokeStyle = paintColor;

// Start a fresh path, and place
// the pen on that exact spot
ctx.beginPath();
ctx.moveTo(x, y);

// Now draw a line from that spot
// to itself. That sounds like a
// pointless thing to do, but because
// lineCap is "round" it paints
// one round dot. This is what makes
// a plain click leave a mark.
ctx.lineTo(x, y);
ctx.stroke();
});

// Stop drawing
canvas.addEventListener("mouseup", () => {
  painting = false;

  // reset path to avoid drawing a line when mouse is moved
  // without holding click
  ctx.beginPath();
});

// Draw on canvas
canvas.addEventListener("mousemove", (e) => {
  if (!painting) return;

  // Get mouse coordinates relative to canvas
  const x = e.offsetX;
  const y = e.offsetY;

  // make it thicker when it's an eraser
  ctx.lineWidth = penSize;
  ctx.lineCap = "round";
  ctx.strokeStyle = paintColor;

  // Draw line to current position
  ctx.lineTo(x, y);
  ctx.stroke(); // Render it
  // Begin a new path so lines don't all connect
  ctx.beginPath();
  // Move the pen to the current mouse position

```

```

// Paint box
let paintBox = document.getElementById("paintBox");
let paintBox_ctx = paintBox.getContext("2d");

// Track the colors and their positions
const paint_colors = [
  { x: 130, y: 55, radius: 20, color: "black" },
  { x: 180, y: 55, radius: 20, color: "red" },
  { x: 230, y: 55, radius: 20, color: "blue" },
  { x: 280, y: 55, radius: 20, color: "dodgerblue" },
  { x: 330, y: 55, radius: 20, color: "green" },
  { x: 380, y: 55, radius: 20, color: "yellow" },
  { x: 430, y: 55, radius: 20, color: "grey" },
];

// Loop through paint colors & draw circles for them
paint_colors.forEach((paint) => {
  paintBox_ctx.beginPath();

  // x, y, radius, start angle, end angle
  paintBox_ctx.arc(paint.x, paint.y, paint.radius, 0, 2 * Math.PI);
  paintBox_ctx.fillStyle = paint.color;
  paintBox_ctx.fill();
});

// Listen for clicks on paintBox canvas
paintBox.addEventListener("click", (e) => {
  // We know user would not click in paintBox if erasing
  indicatorName.textContent = "Pen";
  penSize = 4;
  const rect = paintBox.getBoundingClientRect();
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  // Loop through paint colors, check
  // which one the user clicked
  paint_colors.forEach((paint) => {
    const dx = x - paint.x;
    const dy = y - paint.y;
    const distance = Math.sqrt(dx * dx + dy * dy);

    if (distance <= paint.radius) {
      paintColor = paint.color;

      // indicate the active pen color
      colorIndicator.style.backgroundColor = paint.color;
    }
  });
});

// show user the active color
let eraser = document.getElementById("eraser");

```

```

eraser.addEventListener("click", (e) => {
    // make the pen size thicker
    penSize = 20;
    paintColor = canvasBackground;

    // change indicated pen color to the eraser
    // color—which is canvas background-color
    colorIndicator.style.backgroundColor = canvasBackground;
    indicatorName.textContent = "Eraser";
});

```

Explanation: This is an application where you can draw with your mouse, and see the coloured strokes on your screen as the mouse moves. You can change the color of the pen by clicking on your desired colour circle in the palette (smaller canvas) next to the main drawing canvas. In the color canvas which we named paint-Box, you can choose from the colours: black, red, blue, dodgerblue, green, yellow, and grey. You can easily add a new colour to the application by adding another color object to the `paint_colors` array, and it will just work:

```

const paint_colors = [
    // ...the existing colours...
    { x: 480, y: 55, radius: 20, color: "purple" },
];

```

Add the new entry inside the array where it is declared. Do not try to reassign `paint_colors` afterwards—it is a `const`, so that would throw "Assignment to constant variable".

Keep in mind that the value of the `x` property of each colour is incremented by 50 pixels with each new color. This is to make sure that along the `x`-axis—which is to say horizontally—the color circles are lined up nicely next to each other without any overlapping. The following is a step-by-step explanation of how it all works:

- We start by setting up the variables

```

let colorIndicator = document.getElementById("colorIndicator");
let indicatorName =
    document.getElementById("indicatorName");

```

These lines grab the HTML elements that show which color is selected and display whether the user is using the "Pen" or "Eraser".

-Next, this grabs the `<canvas>` element and prepares its 2D drawing surface (`ctx`) so we can draw on it.

```

const canvas = document.getElementById("paintCanvas");
const ctx = canvas.getContext("2d");

```

- Then we set some default variables that we will reuse like pen color (black), size (4 pixels), and the canvas background (white).

- Next we deal with the starting and stopping of drawing. A flag to track whether the user is currently drawing:

```
let painting = false;
```

When the mouse is pressed down on the canvas, start drawing:

```
canvas.addEventListener("mousedown", (e) => {
  painting = true;

  const x = e.offsetX;
  const y = e.offsetY;

  ctx.lineWidth = penSize;
  ctx.lineCap = "round";
  ctx.strokeStyle = paintColor;

  ctx.beginPath();
  ctx.moveTo(x, y);
  ctx.lineTo(x, y);
  ctx.stroke();
});
```

There is more going on here than you might expect from "start drawing", so let us take it slowly. The `beginPath()` and `moveTo()` put the pen down at the exact spot the mouse was pressed. Leave them out and the line only starts from the SECOND mouse move, which quietly loses the first few pixels of every stroke. The `lineTo(x, y)` and `stroke()` then draw a line from that spot to itself. A line from a point to the same point has no length at all, so you would think it draws nothing. But we set `lineCap` to "round", and a round cap on a zero-length line is simply a circle. So we get one round dot, the width of the pen, exactly where the user pressed. Without it, clicking once without moving the mouse leaves no mark at all, which is not what anyone expects from a drawing app - try dotting the letter i and you will see the problem.

If drawing a line from a point to itself feels like too much of a trick, you can paint that dot directly instead. This is the more advanced way of writing the same four lines, using the `arc()` method you met when we drew a circle earlier:

```
ctx.fillStyle = paintColor;
ctx.beginPath();
ctx.arc(x, y, penSize / 2, 0, 2 * Math.PI);
ctx.fill();
```

Both produce the same dot. Use whichever reads more clearly to you

- though remember that the `arc()` version fills, so it needs `fillStyle` rather than `strokeStyle`.

When the mouse is released, stop drawing. `ctx.beginPath()` clears the current drawing path to avoid unwanted lines:

```
canvas.addEventListener("mouseup", () => {
  painting = false;
  ctx.beginPath();
});
```

If the mouse moves but the button isn't pressed, skip the rest of the function:

```
canvas.addEventListener("mousemove", (e) => {
  if (!painting) return;
  // // ...
```

Get the mouse position inside the canvas:

```
const x = e.offsetX;
const y = e.offsetY;
```

Set the stroke size, round the line ends, and choose the stroke color:

```
ctx.lineWidth = penSize;
ctx.lineCap = "round";
ctx.strokeStyle = paintColor;
```

This is how to draw a line to the new point and start a new path from there. This avoids connecting all paths:

```
ctx.lineTo(x, y);
ctx.stroke();
ctx.beginPath();
ctx.moveTo(x, y);
```

The paintbox setup

```
let paintBox = document.getElementById("paintBox");
let paintBox_ctx = paintBox.getContext("2d");
```

Access the smaller canvas for color selection and its 2D drawing surface.

Define an array of colors with positions and sizes for each color circle.

```
const paint_colors = [
  { x: 130, y: 55, radius: 20, color: "black" },
  ...
];
```

Loop through the colors and draw colored circles on the paintBox canvas:

```
paint_colors.forEach((paint) => {
  paintBox_ctx.beginPath();
  paintBox_ctx.arc(paint.x, paint.y, paint.radius, 0, 2 *
    Math.PI);
```

```

paintBox_ctx.fillStyle = paint.color;
paintBox_ctx.fill();
});

```

Detecting colour selection

When user clicks the color canvas, set the pen mode and reset the pen size to normal:

```

paintBox.addEventListener("click", (e) => {
  indicatorName.textContent = "Pen";
  penSize = 4;

```

Next, we have this line:

```

const rect = paintBox.getBoundingClientRect();
const x = e.clientX - rect.left;
const y = e.clientY - rect.top;

```

Let's talk about the `getBoundingClientRect()` method. It gives the exact position of the canvas on the page. We subtract it from the click coordinates to get the mouse position inside the paint box canvas.

Next, we calculate how far the mouse click was from each color circle's center using the Pythagorean theorem:

```

paint_colors.forEach((paint) => {
  const dx = x - paint.x;
  const dy = y - paint.y;
  const distance = Math.sqrt(dx * dx + dy * dy);

```

If the click was within the confines (radius) of the color circle, update the drawing color and color indicator:

```

if (distance <= paint.radius) {
  paintColor = paint.color;
  colorIndicator.style.backgroundColor = paint.color;
}

```

Eraser functionality

There are two common ways to add an eraser to your canvas drawing app, and both are effective. One way is to draw with a white background, or whatever the background color of your canvas is. This is the simplest way to mimic an eraser: Just change the `ctx.strokeStyle` to "white" (or the canvas background color), and keep drawing. It looks like you're erasing, but you're actually painting over the drawing. Here is an example code:

```

eraser.addEventListener("click", () => {
  // Use background color
  ctx.strokeStyle = "white";
});

```

If you're using a brush with a line width, make the eraser slightly thicker. This is the approach we took for the eraser in our example above. Here is how we did it: - we set an event listener for a click on the eraser div element, then increased the brush size (`penSize = 20`), set the painting color to the same color as the background of our canvas (`canvasBackground`), and finally we changed the text beside the eraser div from “Pen” to “Eraser” like this:

```

indicatorName.textContent = "Eraser";

```

just so the user is made aware that they are now in erasing mode. The example eraser code is as follows:

```

let eraser = document.getElementById("eraser");
eraser.addEventListener("click", (e) => {
  penSize = 20;
  paintColor = canvasBackground;
  colorIndicator.style.backgroundColor = canvasBackground;
  indicatorName.textContent = "Eraser";
});

```

Here is how it works: - It's not a separate tool—it's just a pen with a thicker size that draws using the background color (white), so it appears to "erase". - This method is simple and fast, especially for programmers. There's no need to manually clear pixels.

There is another way to implement an eraser in such a drawing app as ours, and I will like to show you here just so you know the two ways. This second way is to use the `clearRect()` method to clear part of the canvas. Using `ctx.clearRect(x, y, width, height)`, you can implement a pixel-level eraser (like Photoshop) that is more precise. To do so, you would have to track the mouse like you would do when drawing, except that instead of drawing lines, you would be clearing small squares. This is what the code may look like:

```

if (isErasing)
{
  const rect = canvas.getBoundingClientRect();
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  // Erase a small square under the cursor
  ctx.clearRect(x - 5, y - 5, 10, 10);
}

```

If we are to modify the above drawing app to use this, the best way is to make it possible to detect when the user is drawing and when they are erasing. You could set a variable like `isErasing = true` when you activate the eraser tool.

Here is the modified code to implement pixel erasing using `clearRect()`:

```
let colorIndicator = document.getElementById("colorIndicator");
let indicatorName = document.getElementById("indicatorName");
const canvas = document.getElementById("paintCanvas");
const ctx = canvas.getContext("2d");
let paintColor = "black";
let penSize = 4;
let canvasBackground = "white";

// Two separate pieces of state. This
// is the important bit: whether
// the button is held down is a
// different question from which tool
// is selected, so we keep them
// in two different variables.
// is the mouse button being held?
let mouseDown = false;
// which tool are we using?
let isErasing = false;

// Start drawing or erasing
canvas.addEventListener("mousedown", (e) => {
  mouseDown = true;

  const x = e.offsetX;
  const y = e.offsetY;

  if (isErasing) {
    // A click on its own should rub out that spot
    ctx.clearRect(x - penSize / 2, y - penSize / 2, penSize,
penSize);
  }
  else {
    // Set the pen up, put it down,
    // and mark the spot so that a
    // click on its own still leaves a dot
    ctx.lineWidth = penSize;
    ctx.lineCap = "round";
    ctx.strokeStyle = paintColor;

    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x, y);
    ctx.stroke();
  }
}
```



```

// Stop drawing or erasing
canvas.addEventListener("mouseup", () => {
    mouseDown = false;

    // reset path to avoid drawing
    // a line when mouse is moved
    // without holding click
    ctx.beginPath();
});

// If the pointer leaves the canvas
// we stop too, otherwise letting go
// outside the canvas would leave mouseDown stuck on true
canvas.addEventListener("mouseleave", () => {
    mouseDown = false;
    ctx.beginPath();
});

// Draw or erase on canvas
canvas.addEventListener("mousemove", (e) => {
    // nothing to do unless the button is being held
    if (!mouseDown) return;

    // Get mouse coordinates relative to canvas
    const x = e.offsetX;
    const y = e.offsetY;

    if (isErasing) {
        // Erase a square under the
        // cursor, the size of the pen
        ctx.clearRect(
            x - penSize / 2, y - penSize / 2, penSize, penSize
        );
    }
    else {
        ctx.lineWidth = penSize;
        ctx.lineCap = "round";
        ctx.strokeStyle = paintColor;

        // Draw line to current position
        ctx.lineTo(x, y);
        ctx.stroke(); // Render it

        // Begin a new path so lines don't all connect
        ctx.beginPath();

        // Move the pen to the current mouse position
        ctx.moveTo(x, y);
    }
});

```

```

let paintBox_ctx = paintBox.getContext("2d");

// Track the colors and their positions
const paint_colors = [
  { x: 130, y: 55, radius: 20, color: "black" },
  { x: 180, y: 55, radius: 20, color: "red" },
  { x: 230, y: 55, radius: 20, color: "blue" },
  { x: 280, y: 55, radius: 20, color: "dodgerblue" },
  { x: 330, y: 55, radius: 20, color: "green" },
  { x: 380, y: 55, radius: 20, color: "yellow" },
  { x: 430, y: 55, radius: 20, color: "grey" },
];

// Loop through paint colors & draw circles for them
paint_colors.forEach((paint) => {
  paintBox_ctx.beginPath();

  // x, y, radius, start angle, end angle
  paintBox_ctx.arc(paint.x, paint.y, paint.radius, 0, 2 * Math.PI);
  paintBox_ctx.fillStyle = paint.color;
  paintBox_ctx.fill();
});

// Listen for clicks on paintBox canvas
paintBox.addEventListener("click", (e) => {
  // We know user would not click
  // in paintBox if erasing
  isErasing = false;
  indicatorName.textContent = "Pen";
  penSize = 4;
  const rect = paintBox.getBoundingClientRect();
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  // Loop through paint colors, check
  // which one the user clicked
  paint_colors.forEach((paint) => {
    const dx = x - paint.x;
    const dy = y - paint.y;
    const distance = Math.sqrt(dx * dx + dy * dy);

    if (distance <= paint.radius) {
      paintColor = paint.color;

      // indicate the active pen color
      colorIndicator.style.backgroundColor = paint.color;
    }
  });
});

// show user the active color
let eraser = document.getElementById("eraser");

```

```

eraser.addEventListener("click", (e) => {
    // switch tool only - whether the
    // button is down is not our business
    isErasing = true;

    // make the eraser square bigger than the pen
    penSize = 20;

    // change indicated pen color to the eraser
    // color—which is canvas background-color
    colorIndicator.style.backgroundColor = canvasBackground;
    indicatorName.textContent = "Eraser";

});

```

Hopefully you have learned a lot about the HTML5 canvas from this chapter. Visit the documentation online, experiment and play with it to see its full power.

Positioning, animation and collision detection

Finally, to round up our learning of the canvas, I will now show you how to create an animation using the simple example of a moving ball, that will demonstrate collision detection. The animation moves a ball on screen from left to right and top to bottom on a keydown event of your keyboard's arrow keys. When the ball reaches the edge of the canvas, it can go no further, and can only move in another direction. That is thanks to collision detection. The code is able to detect (determine) that it has run into an obstacle. Also we will place an item (a 'rock') in the middle of the canvas that the ball can only go round but cannot run through. That's another example of collision detection. Neither of these two things—animation and collision detection—is possible if we are not able to determine the position of elements on the web page. We will start by learning all about detecting the position of an object using what are known as x and y axes or coordinates. Once we are able to do that, we can make the object move, and are then able to tell if they run into (collide with) any other object on the screen. We know there has been a collision if the position of our object overlaps with that of another. I hope hereby to teach you how to detect boundaries, collisions, keyboard key presses, and how to make objects move. These are great skills to have as a JavaScript programmer. You would get the fundamental skills needed to start building your own animations, games—whether it is to write racing games where you detect which car came first based on their positions against the finish line, or a keyboard-driven object that skips over moving obstacles (the flappy bird game comes to mind), or a pacman game, or a shooting game where you detect bullets hitting the enemy to deduct points or lives from them, you will be limited only by your own imagination. The possibilities are endless. Let's look at the example of the moving ball on a canvas. Note that the ball is

drawn using the `arc()` method of the canvas, while the rock is a rectangle drawn on the canvas using the `fillRect()` method. Let's dive into the code:

HTML code

```
<canvas
    id="gameCanvas"
    width="500"
    height="400"
    style="border:1px solid black;">
</canvas>
```

JavaScript code

```
const canvas = document.getElementById("gameCanvas");
const ctx = canvas.getContext("2d");

// Ball object
const ball = {
  x: 50,
  y: 50,
  radius: 15,
  color: "dodgerblue",

  // Moves 10 pixels per key press
  speed: 10
};

// Rock object
const rock = {
  x: 220,
  y: 170,
  width: 60,
  height: 60,
  color: "gray"
};

// Draw the ball
function drawBall() {
  ctx.beginPath();
  ctx.arc(ball.x, ball.y, ball.radius, 0, Math.PI * 2);
  ctx.fillStyle = ball.color;
  ctx.fill();
  ctx.closePath();
}

// Draw the rock
function drawRock() {
  ctx.fillStyle = rock.color;
  ctx.fillRect(rock.x, rock.y, rock.width, rock.height);
}
```

```

// Check if ball will hit the wall or rock
function canMove(dx, dy) {
  const nextX = ball.x + dx;
  const nextY = ball.y + dy;

  // Check canvas boundaries
  if (nextX - ball.radius < 0 || nextX + ball.radius > canvas.width) return false;
  if (nextY - ball.radius < 0 || nextY + ball.radius > canvas.height) return false;

  // Check rock collision
  const ballLeft = nextX - ball.radius;
  const ballRight = nextX + ball.radius;
  const ballTop = nextY - ball.radius;
  const ballBottom = nextY + ball.radius;

  const rockLeft = rock.x;
  const rockRight = rock.x + rock.width;
  const rockTop = rock.y;
  const rockBottom = rock.y + rock.height;

  const hitRock =
    ballRight > rockLeft &&
    ballLeft < rockRight &&
    ballBottom > rockTop &&
    ballTop < rockBottom;

  // Only move if no collision
  return !hitRock;
}

// Handle arrow key movement
document.addEventListener("keydown", (e) => {
  let dx = 0;
  let dy = 0;

  if (e.key === "ArrowUp") dy = -ball.speed;
  if (e.key === "ArrowDown") dy = ball.speed;
  if (e.key === "ArrowLeft") dx = -ball.speed;
  if (e.key === "ArrowRight") dx = ball.speed;

  // Redraw canvas with updated position
  if (canMove(dx, dy)) {
    ball.x += dx;
    ball.y += dy;
    draw();
  }
});

// Redraw everything
function draw() {
  // Clear the canvas

```

```

ctx.clearRect(0, 0, canvas.width, canvas.height);
// Draw the obstacle
drawRock();

// Draw the ball
drawBall();
}

// Initial draw
draw();

```

Explanation: Since the ball only moves per keypress, we just call `draw()` once after movement. `canMove(dx, dy)`: This helper checks both walls and rock to make sure the ball won't go out of bounds or crash into the rock. `ball.speed`: Set to 10 so every arrow key press nudges the ball 10 pixels. Collision Detection: happens once on each key press.

Positioning

In computer graphics, every object (like our ball or rock) has a position on a 2D grid. This grid is made up of two invisible lines:

- The x-axis goes left to right (horizontal),
- The y-axis goes top to bottom (vertical).

The top-left corner of the canvas is (0, 0). As:

- You increase the x-value, you move right.
- You decrease the x-value, you move left.
- You increase the y-value, you move down.
- You decrease the y-value, you move up.

In the above example, we have the following code initialising the value to be used as the value of the x and y coordinates.

x: 50, y: 50,

These values are set as properties of the ball object:

```

const ball = {
  x: 50,
  y: 50,
  radius: 15,
  color: "dodgerblue",

  // Moves 10 pixels per key press
  speed: 10
};

```

These are later used when creating the ball on the canvas using the `ctx.arc()` method in `drawBall()` function. Having `x`, `y` coordinates of 50, 50 each means the ball starts 50 pixels from the left and 50 pixels from the top of the canvas. In order to detect the position of an object, we use the `x` and `y` coordinates, like we do for the ball in this example using `ball.x` and `ball.y`.

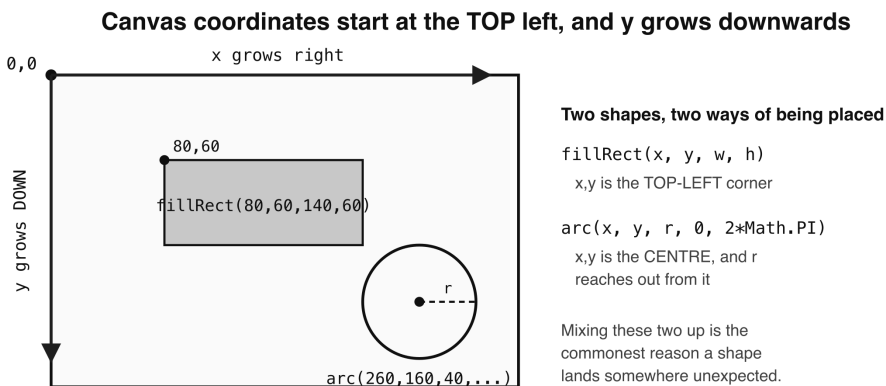
Let's say for example, you wish to detect the exact spot where the user clicks on a canvas, you can do so by using mouse events (like click) and then get the `offsetX` and `offsetY` of the event. Here is an example:

```
canvas.addEventListener("click", (e) => {  
    console.log("Click coordinates are: ", e.offsetX, e.offsetY);  
});
```

Save that and if you click in the canvas, you will see the message written to the console:

Click coordinates are: 51 49

There you have it; the `x` and `y` coordinates of the exact spot in the canvas element on which you clicked.



This is upside down compared to the graphs you drew at school, where `y` grows upwards.

- Figure 16.1 — Canvas coordinates start at the top left*

Animation

Animation in JavaScript means updating the position or style of something over time, so it looks like it's moving. In our example code above, we place an event listener on the press of a key on your keyboard, then we proceed to specify an action to be taken depending on which arrow key was clicked. We only check for the case of arrow keys because those are the keys we are interested in.

```
document.addEventListener("keydown", (e) => {
  // // ...
  if (e.key === "ArrowUp") dy = -ball.speed;
    if (e.key === "ArrowDown") dy = ball.speed;
    if (e.key === "ArrowLeft") dx = -ball.speed;
    if (e.key === "ArrowRight") dx = ball.speed;
  // // ...
});
```

We move the ball a little each time the user presses an arrow key. This is how the ball movement happens:

- The initial position of the ball as mentioned above, is set to x coordinate 50, and y coordinate 50

```
const ball = {
  x: 50,
  y: 50,
  // // ...
  speed: 10
};
```

- The speed property of the ball object is set to 10 (for 10 pixels) which is the steps we have decided that the ball will move by at each key press.
- Whenever the ArrowUp key is pressed, we decrement the value of the y coordinate by the value of speed. We already know that along a y axis (top to bottom), decreasing the value will move the object upwards and vice versa. Equally, if the ArrowDown key is pressed, we increase the value of the y coordinate by the value of the speed property. This should make the ball move downwards as we know. That updated value—whether less or more—is stored in a variable dy.

```
if (e.key === "ArrowUp") dy = -ball.speed;
if (e.key === "ArrowDown") dy = ball.speed;
```

- The exact opposite is true of the ArrowLeft and ArrowRight keys. Whenever the ArrowLeft key is pressed, we decrement the value of the x coordinate by the value of speed. We already know that along a x axis (left to right), decreasing the value will move the object to the left and vice versa. Equally, if the ArrowRight key is pressed, we increase the value of the x coordinate by the value of the speed property. This should make the ball move to the right, as we know. That updated value—whether less or more—is stored in a variable dx.

```
if (e.key === "ArrowLeft") dx = -ball.speed;
if (e.key === "ArrowRight") dx = ball.speed;
```

- Finally, we update the position of the ball object by setting its x and y, property values with the new values stored in the dx and dy variables:


```

// Redraw canvas with updated position
if (canMove(dx, dy)) {
    ball.x += dx;
    ball.y += dy;
    draw();
}

```

The position itself is updated right there in the key handler. The `draw()` function is then called to clear the canvas and redraw everything in its new place.

Another common way to animate is using the built-in function `requestAnimationFrame()`. That method is used when we want to make something move continuously or very smoothly—like a character that keeps walking or a bouncing ball. Here is the syntax of using `requestAnimationFrame()` function:

```

function animate() {
    // Move something
    draw(); // Draw new position
    // Loop forever
    requestAnimationFrame(animate);
}

animate();

```

This method is a bit more advanced, but you'll definitely use it later as your skills grow.

Collision detection

Collision detection means checking if two objects bump into each other. In our example code above, the ball has two objects we want to detect collision against. They are as follows:

- We must detect when the ball reaches the edge of the canvas and prevent it from going off that limit.
- We must detect when the ball runs against the rock in the centre of the canvas and stop, so that the user can move around it.

We create a custom function to handle the collision detection for us. This is the `canMove()` function, and it looks like this:

```

function canMove(dx, dy) {
    const nextX = ball.x + dx;
    const nextY = ball.y + dy;
    // // ...
}

```

We will be passing to `canMove()` the updated `x` and `y` position every time the user presses a key to move the ball. We call `canMove()` first before drawing (re-creating) the new canvas with the new ball position. In doing so, we are checking if there is no obstacle in that new position before we let the ball move to it. That is why we call `canMove()` within the event listener of every key press:

```
document.addEventListener("keydown", (e) => {
  // // ...
  if (canMove(dx, dy)) {
    ball.x += dx;
    ball.y += dy;
    draw();
  }
});
```

The following are the steps to do the collision detection which we do in `canMove()`:

- First, we work out what the ball's `x` and `y` coordinates are about to be, and store them in the variables `nextX` and `nextY` which represent the `x` and `y` coordinates, respectively.

```
function canMove(dx, dy) {
  const nextX = ball.x + dx;
  const nextY = ball.y + dy;
  // // ...
}
```

- Next we do some canvas edge detection to make sure the ball doesn't move beyond the edges. To achieve this, we do some simple math. Let's establish some parameters so that you would understand perfectly.
- On the `x` (horizontal) axis
 - the position of the left wall of the canvas is 0 (`x = 0`)
 - the position of the right wall of the canvas is the width of the canvas (`canvas.width`)
- On the `y` (vertical) axis
 - the position of the top wall of the canvas is 0 (`y = 0`)
 - the position of the bottom wall of the canvas is the height of the canvas (`canvas.height`)
- The radius of the circular ball is the distance between the centre of the ball and the edge of the ball. We already know the radius of our ball because we have it as the `radius` property of the ball object.

```
const ball = {
  // // ...
  radius: 15,
```

```
// // ...
};
```

In fact, we even used that radius value when creating the ball in the first place.

```
function drawBall() {
    // // ...
    ctx.arc(ball.x, ball.y, ball.radius, 0, Math.PI * 2);
    // // ...
}
```

The x and y coordinates define one spot in the middle of the circle, while the radius creates a circle around that spot where the distance between that spot and the outer edges of the circle represent the radius.

- To work out if the ball has touched any of the inner walls of the canvas, you must find out the position of the outer edges of the ball, because it takes into consideration the radius.
- To calculate the position of the outer edge of the circle, it depends on if we are talking about the left edge, or the top edge, or the right edge, or the bottom edge. This is easy to work out, just stay with me. For left and right edges, we are looking at the horizontal line, or x axis, while for the top and bottom edges, we are talking about the vertical or y axis. We already know that on the x axis, to move left, you reduce the x value, and to move right, you increase the x value, just as on the y axis you have to reduce the y value to move upwards, and increase it to move downwards. To work out the position of the outer edges of the ball, do this:

```
- left edge:      x - radius
- right edge:     x + radius
- top edge:       y - radius
- bottom edge:    y + radius
```

- With all that in mind, to know if the left edge of the circle has touched the left wall of the canvas, just work out if the position of the left edge is equal to 0.

```
x - radius == 0
```

If it is, then the ball is exactly at the edge, touching the left canvas (x axis) wall. If it is less than 0, then the ball has gone beyond the left edge. To know if the right edge has touched the right inner wall of the canvas, the calculation is exactly the opposite. We simply check if the value of the ball's right edge is equal to the canvas.width.

```
x + radius == canvas.width
```

If it is greater, then we know the ball has gone beyond the right edge of the canvas. The logic we apply in our example code is to check if both left and right edges

of the ball have not gone beyond the canvas edges. That's why instead of using `x - radius == 0` and `x + radius == canvas.width`, we use `x - radius < 0` (less than) and `x + radius > canvas.width` for the left canvas edge and right canvas edge, respectively. Either of those being true means the ball has gone too far. That check happens in the function `canMove()`, and read its return value carefully, because it is the opposite way round from what you might expect. `canMove()` returns `FALSE` when the ball has hit something, and `TRUE` when the way is clear. Its name is the clue: we are asking "can the ball move?", not "has it crashed?". That is why we only allow the ball to move when `canMove()` returns true.

```
if (nextX - ball.radius < 0 || nextX + ball.radius >
    canvas.width) return false;
```

Here `nextX` is the new value of the x position after the user hits to move the ball on the x axis. This is getting the position of the left edge of the ball since we are subtracting the radius (`ball.radius`) from that x value. We then check if it is less than 0. We run the same check for the right canvas wall. The pipe characters (`||`) separating the two if conditions makes sure we run both checks in one if statement. Similarly, we also check the top and bottom edges in `canMove()`. Again, touching either edge makes the function return false, and we only allow the ball to move when it returns true.

```
if (nextY - ball.radius < 0 || nextY + ball.radius >
    canvas.height) return false;
```

Here `nextY` is the new value of the y position after the user hits to move the ball on the y axis. This is getting the position of the top edge of the ball since we are subtracting the radius (`ball.radius`) from that y value. We then check if it is less than 0. We run the same check for the bottom canvas wall. The pipe characters (`||`) separating the two if conditions makes sure we run both checks in one if statement.

Detecting the collision of two shapes

When it comes to detecting collision of our ball against the rock on the canvas, the approach is exactly the same. To detect the collision between any two objects, you must start by knowing the position of their outer edges—bear in mind that the math for achieving this will depend on the shape of each object. So, in this example, we have a rectangle (the rock), and a circle (the ball).

- We start by calculating the edges of the ball:

```
const ballLeft = nextX - ball.radius;
const ballRight = nextX + ball.radius;
const ballTop = nextY - ball.radius;
const ballBottom = nextY + ball.radius;
```

- Next, we work out the edges of the rock. Take note of how this is calculated differently from how it's done with a circle. For example, the left edge is simply its x position, while its top edge is its y position.

```
const rockLeft = rock.x;
const rockRight = rock.x + rock.width;
const rockTop = rock.y;
const rockBottom = rock.y + rock.height;
```

Finally, we check if the edges of these objects ever overlap. The following piece of code in our example takes care of that:

```
const hitRock = ballRight > rockLeft &&
  ballLeft < rockRight &&
  ballBottom > rockTop &&
  ballTop < rockBottom;
```

If ever all four conditions are true, it means the ball is inside or touching the rock! So, we return false, else we return true:

```
return !hitRock;
```

Because this check is inside the `canMove()` function, it means: only allow the move if the ball doesn't hit the rock.

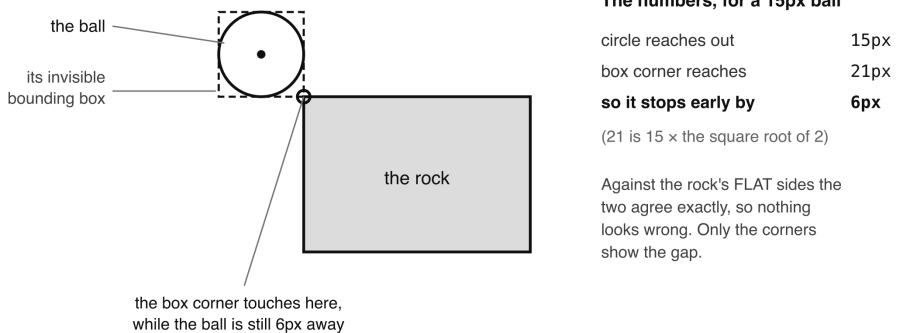
A word about the shape we are really testing

There is an honest admission to make about those four comparisons, and it is worth making because you will meet this trade-off in every game you ever write. Look again at what we compared. We took the ball's left, right, top and bottom edges and treated them as the four sides of a box. But the ball is not a box. It is a circle. What we have actually been testing all along is whether an invisible square drawn around the ball overlaps the rock. This technique has a name. It is called AABB collision detection, which stands for Axis-Aligned Bounding Box — "bounding box" because we draw the smallest box that contains the shape, and "axis-aligned" because that box is always square to the screen and never tilted. Most of the time you cannot tell the difference. Push the ball straight at the flat side of the rock and the box edge and the circle edge are in exactly the same place, so it stops precisely where you would expect. The difference only shows itself at the rock's corners. Approach one diagonally and the corner of the invisible box reaches the rock before the ball itself does, so the ball stops with a small gap, as though it had bumped into thin air. How big is that gap? The corner of the box sits further from the centre than the edge of the circle does — the radius multiplied by the square root of 2, which for our 15 pixel ball is about 21 pixels rather than 15. So in the very worst case, coming in at exactly 45 degrees, the ball stops around 6 pixels early.

On a 500 pixel canvas that is small enough that most people never notice. So why use it? Because it is four simple comparisons. No square roots,

no multiplication, nothing expensive. When you have a hundred objects on screen and you are checking every one of them against every other, sixty times a second, that cheapness is the whole game. Real game engines use AABB as a first, fast pass, and only do the precise, expensive maths on the few pairs that the boxes say might be touching.

Why the ball sometimes stops just short of the rock



We accept it because four comparisons are far cheaper than square roots — real game engines do the same.

- Figure 16.2 — Why the ball sometimes stops just short of the rock*

The precise version, if you need it

If you did want the true circle-against-rectangle test, here is how it works. Find the point on the rectangle that is closest to the centre of the circle, then measure the distance from the circle's centre to that point. If that distance is smaller than the radius, they really are touching:

```
function circleHitsRock(cx, cy) {  
  // Find the closest point on the  
  // rock to the ball's centre.  
  // Math.max and Math.min together  
  // "clamp" the ball's centre so  
  // that it never falls outside the rock's edges.  
  const closestX = Math.max(  
    rock.x, Math.min(cx, rock.x + rock.width)  
  );  
  const closestY = Math.max(  
    rock.y, Math.min(cy, rock.y + rock.height)  
  );  
}
```

```

const distanceX = cx - closestX;
const distanceY = cy - closestY;
const distance = Math.sqrt(
  distanceX * distanceX + distanceY * distanceY
);

// Touching only if that distance
// is less than the radius
return distance < ball.radius;
}

```

You could drop that straight into `canMove()` in place of the `hitRock` calculation. For our one rock it makes no practical difference, and the box version is easier to read, which is why the chapter uses it. But now you know both, and you know why you would reach for each.

You’ve just taken your first steps into the world of graphics and animation using the HTML5 `<canvas>`! From moving a ball around the screen to detecting collisions with obstacles, you now understand how powerful simple shapes, positioning, and logic can be. But this is just the beginning. The canvas API can do so much more — from drawing images and text to creating full-blown games. Explore the official Canvas documentation on MDN (https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API) to see what’s possible. And if you’re feeling adventurous, try building a maze, a simple pong game, or a mini obstacle course — anything that challenges you to move objects, detect hits, and control motion.

QUIZ — The Canvas Element

This page contains the Q & A (questions and answers) for this chapter — Chapter 16: The Canvas Element. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 13 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What makes a `<canvas>` different from an `<img>` or a `<div>`, and what is the very first thing you must do in JavaScript before you can draw on one?

Clue: the element itself is empty. You have to ask it for something before anything can happen.

2. What is the difference between these two pairs, and when would you use each?
`ctx.fillStyle + ctx.fill()` `ctx.strokeStyle + ctx.stroke()`

Clue: one colours the inside, the other draws the outline.

3. Read this line and say what each of the five arguments does:

```
ctx.arc(150, 75, 40, 0, 2 * Math.PI);
```

Clue: two are a position, one is a size, and two are angles measured in something other than degrees.

4. After calling `ctx.arc(...)`, is anything visible on the canvas yet? Why or why not?

Clue: think of `arc()` as tracing with a pencil that has no lead in it.

5. A reader copies this from a tutorial and gets a black circle instead of a red one. What went wrong?

```
ctx.beginPath();  
ctx.arc(150, 75, 40, 0, 2 * Math.PI);  
ctx.strokeStyle = "red";  
ctx.stroke();
```

Clue: look very closely at the third line. Nothing will appear in the console to help you.

6. In the drawing app, why is `ctx.beginPath()` called on mouseup, and what would happen without it?
Clue: think about what the pen does between the end of one stroke and the start of the next.
7. The corrected drawing app keeps two separate variables, `mouseDown` and `isErasing`. Why is it important that these are two variables and not one?
Clue: they answer two completely different questions.
8. What does `canMove()` return when the ball has hit the rock — `true` or `false`? And how does the function's name help you remember?
Clue: read the name as a question and answer it out loud.
9. EXERCISE. Draw a green rectangle 100 wide and 50 tall, starting 20 pixels from the left and 30 pixels from the top of the canvas.
Clue: one property and one method, and the four numbers go in a particular order.
10. EXERCISE. Draw a circle outline (no fill) with a radius of 30, in the middle of a canvas that is 200 by 200.
Clue: the middle of a 200 by 200 canvas is not 200, 200.
11. EXERCISE. Write the words "Hello Canvas!" onto a canvas in 20 pixel blue text, 70 pixels from the left and 50 pixels from the top.
Clue: three lines, and one of them is the text equivalent of `fill()`.
12. EXERCISE. Given a ball with `x`, `y` and `radius`, and a canvas, write a check that returns true only if the ball is completely inside the canvas.
Clue: four comparisons, one per wall, and every one of them needs the radius.
13. EXERCISE. The ball is at (250, 200) with a radius of 15. The rock sits at (220, 170) and is 60 by 60. Work out, by hand, whether the bounding-box check says they are touching.
Clue: write down the four edges of each, then compare them in pairs.

ANSWERS

1. A `<canvas>` has **no content of its own**. A `<div>` can hold text and an `<img>` shows a picture you gave it, but a canvas starts as a blank rectangle and stays blank until JavaScript draws on it.

The first thing you must do is ask it for its **drawing context**:

```
let canvas = document.getElementById("myCanvas");
let ctx = canvas.getContext("2d");
```

The context is the actual drawing tool — every method in this chapter is called on `ctx`, not on the canvas element. That is why every canvas program in this chapter starts with those two lines.

In every other respect the canvas is an ordinary DOM element: you can create it with `createElement()`, find it with `getElementById()`, style it with CSS and append it to the page.

2. **fillStyle** and **fill()** colour the **inside** of a shape — a solid circle, a solid rectangle.
- **strokeStyle** and **stroke()** draw the **outline** only, leaving the middle empty. They pair up, and mixing them is a common mistake — setting **fillStyle** and then calling **stroke()** gets you an outline in the default black, because **stroke()** never looks at **fillStyle**.

```
// solid red circle
ctx.fillStyle = "red";
ctx.fill();

// red outline, hollow middle
ctx.strokeStyle = "red";
ctx.stroke();
```

Note that neither of the two

style

properties draws anything by itself. They only say what colour the next fill or stroke will use.

3. `ctx.arc(150, 75, 40, 0, 2 * Math.PI);`

|
|
|
|
|
x

|
|
|
|
y

|
|
|
radius

|
end angle
start angle

- **150** and **75** are the x and y coordinates of the **centre** of the circle.
- **40** is the **radius** — how far the circle reaches out from that centre.
- **0** is the starting angle and **2 * Math.PI** is the ending angle.

The angles are in **radians**, not degrees. A full circle is 360 degrees, which is `2 * Math.PI` radians, so going from 0 to `2 * Math.PI` draws the whole way round. If you only went to `Math.PI` you would get half a circle.

4. No, nothing is visible yet.

`arc()` only defines a **path**. It traces the shape's outline without putting any colour down — like drawing with a pencil that has no lead in it. The canvas now knows the shape you mean, but has not painted it.

Something appears only when you call `fill()` or `stroke()`. That is the pattern for every shape in this chapter:

```
// start a new shape
ctx.beginPath();
// describe it
ctx.arc(...);
// choose the colour
ctx.fillStyle = "red";
// NOW it appears
ctx.fill();
```

`fillRect()` is the exception that proves the rule — it describes *and* paints in one go, which is why the rectangle example is shorter than the circle one.

5. The third line says **StrokeStyle** with a capital S. The property is `strokeStyle`, with a small s.

JavaScript is case-sensitive, so `ctx.StrokeStyle = "red"` does not set the stroke colour. It simply creates a brand new property called `StrokeStyle` on the context object and puts `"red"` in it, where nothing will ever look at it. The canvas carries on using its default stroke colour, which is black.

What makes this one nasty is that **nothing goes wrong**. There is no error, no warning, no red text in the console. You just get a black circle and no explanation. If a canvas colour is being ignored, checking the capitalisation of your property name is a good first move.

6. Because it **lifts the pen off the paper**.

The canvas keeps a current path as you draw. If you finish a stroke and do not clear that path, then the next time the user presses down somewhere else and moves, the canvas draws a line joining the end of the old stroke to the start of the new one.

So without it, every separate stroke would be connected by a straight line running across your drawing — as though you never lifted the pencil.

```
canvas.addEventListener("mouseup", () => {
  painting = false;
  // lift the pen
  ctx.beginPath();
});
```

7. Because they answer **two completely different questions**:

- `mouseDown` — *is the button being held right now?*
- `isErasing` — *which tool has the user chosen?*

These change independently. You can be erasing with the button up, or drawing with the button down, or any other combination. Trying to store both in one variable, or deriving one from the other, produces exactly the bug that made this fix necessary: releasing the mouse did not stop the pen, and simply moving the mouse across the canvas erased whatever it passed over.

The general lesson is worth more than the canvas detail: **when two things can vary independently, they need two variables**. If you find yourself writing `a = !b ? true : false`, stop and ask whether `a` and `b` are really the same fact.

8. `canMove()` returns **false** when the ball has hit something, and **true** when the way is clear.

The name is the memory aid. Read it as a question — "*can the ball move?*" — and the answer is yes or no. It is not asking "*has it crashed?*", which would be the other way round.

That is why the calling code reads the way it does:

```
if (canMove(dx, dy)) {
    ball.x += dx;
    ball.y += dy;
    draw();
}
```

Move only if we are allowed to. Naming a function so that its name reads as the question it answers is a small habit that saves a lot of confusion later.

9.

```
let canvas = document.getElementById("myCanvas");
let ctx = canvas.getContext("2d");

ctx.fillStyle = "green";
ctx.fillRect(20, 30, 100, 50);
```

The four numbers go in the order **x, y, width, height** — so 20 across, 30 down, then 100 wide and 50 tall. Getting them in the wrong order is easy and the shape simply appears somewhere unexpected, with no error.

Note that `fillRect()` needs no `beginPath()` and no `fill()`. It is a shortcut that describes and paints in one call.

```
10. let canvas = document.getElementById("myCanvas");
    let ctx = canvas.getContext("2d");

    ctx.beginPath();
    ctx.arc(100, 100, 30, 0, 2 * Math.PI);
    ctx.strokeStyle = "red";
    ctx.stroke();
```

The middle of a 200 by 200 canvas is **(100, 100)** — half the width and half the height, not the width and height themselves. Since `arc()` takes the coordinates of the *centre*, that is exactly what it wants.

Use `strokeStyle` and `stroke()` because we want an outline. Swap them for `fillStyle` and `fill()` and you get a solid disc instead.

```
11. let canvas = document.getElementById("myCanvas");
    let ctx = canvas.getContext("2d");

    ctx.font = "20px Arial";
    ctx.fillStyle = "blue";
    ctx.fillText("Hello Canvas!", 70, 50);
```

`fillText()` is the text equivalent of `fill()` — it is what actually puts the text on the canvas. So the pattern matches the shapes: `fillStyle` chooses the colour, and the `fill` method does the painting.

The `font` property takes the same kind of value you would write in CSS, size first and then the typeface.

```

12. function isInsideCanvas() {
    // the left edge of the ball must
    // not be past the left wall
    if (ball.x - ball.radius < 0) {
        return false;
    }

    // the right edge must not
    // be past the right wall
    if (ball.x + ball.radius > canvas.width) {
        return false;
    }

    // the top edge must not be past the top wall
    if (ball.y - ball.radius < 0) {
        return false;
    }

    // the bottom edge must not
    // be past the bottom wall
    if (ball.y + ball.radius > canvas.height) {
        return false;
    }

    // nothing was out of bounds, so we are inside
    return true;
}

```

The radius is the whole point. `ball.x` is the **centre**, so comparing it to the walls on its own would let half the ball disappear off the edge before anything complained. Subtract the radius to get the left and top edges, add it to get the right and bottom.

The same four checks written more compactly, once you are comfortable with them:

```

return ball.x - ball.radius >= 0 &&
    ball.x + ball.radius <= canvas.width &&
    ball.y - ball.radius >= 0 &&
    ball.y + ball.radius <= canvas.height;

```

13. Yes — the bounding-box check says they are touching.

Work out the eight edges:

ball at (250, 200), radius 15	rock at (220, 170), 60 by 60
ballLeft = 250 - 15 = 235	rockLeft = 220
ballRight = 250 + 15 = 265	rockRight = 220 + 60 = 280
ballTop = 200 - 15 = 185	rockTop = 170
ballBottom = 200 + 15 = 215	rockBottom = 170 + 60 = 230

Now the four comparisons:

```
ballRight > rockLeft    265 > 220    true
ballLeft  < rockRight   235 < 280    true
ballBottom > rockTop    215 > 170    true
ballTop   < rockBottom  185 < 230    true
```

All four are true, so `hitRock` is true, so `canMove()` returns false and the ball is not allowed to move there.

That makes sense if you picture it: the ball's centre at (250, 200) is inside the rock, which spans 220 to 280 across and 170 to 230 down. The ball is sitting right on top of it.

Remember that all four must be true. If even one is false the shapes have missed each other, because a gap on any single side is enough to keep them apart.

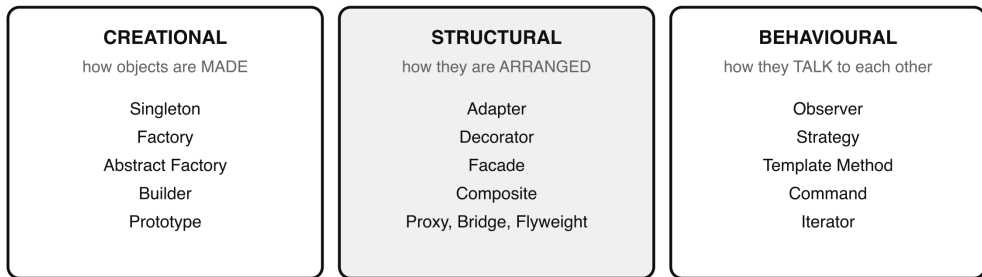
Chapter 17 – DESIGN PATTERNS

Design patterns are reusable solutions to common software design problems. They provide a way to solve issues in your code's structure while promoting maintainability and scalability. Design patterns are categorised into three groups:

1. Creational Patterns These are concerned with object creation.
 - SINGLETON
 - FACTORY
 - ABSTRACT FACTORY
 - BUILDER
 - PROTOTYPE
2. Structural Patterns These patterns focus on the composition (structure) of classes or objects.
 - ADAPTER PATTERN
 - DECORATOR PATTERN
 - FACADE PATTERN
 - COMPOSITE PATTERN
 - PROXY PATTERN
3. Behavioural Patterns These patterns deal with object interaction and responsibility distribution.
 - OBSERVER PATTERN
 - STRATEGY PATTERN
 - TEMPLATE METHOD PATTERN
 - COMMAND PATTERN
 - ITERATOR PATTERN

The three families of design pattern

Birth, arrangement, conversation.



You already use one of these without knowing its name: `addEventListener` is the Observer pattern.

- Figure 17.1 — The three families of design pattern*

These design patterns are global and programming-language agnostic, however, each programming language has its own way of implementing them. In the explanations of all these design patterns below, I provide examples in JavaScript. If you program in another language, you just have to learn how each of the patterns are implemented in that language. I can assure you that the approach is generally always the same, with syntactical differences of course.

1) Creational Design Patterns

Creational design patterns deal with object creation mechanisms, trying to create objects in a manner suitable to the situation. These patterns provide flexibility in how objects are instantiated and constructed. By using these creational patterns, you can better manage object creation processes in your applications, ensuring efficiency, flexibility, and maintainability. The following are the patterns in this group:

- SINGLETON
- FACTORY
- ABSTRACT FACTORY
- BUILDER
- PROTOTYPE

SINGLETON PATTERN

The singleton pattern ensures that a class has only one instance and provides a global point of access to it. It's commonly used for shared resources like databases, configuration, or logging. The following is an example of a singleton implementation in JavaScript to make a database connection.

JavaScript example

```
class DatabaseConnection {
  constructor() {
    if (DatabaseConnection.instance) {
      return DatabaseConnection.instance;
    }

    // Simulated database connection
    this.connection = this.#connectToDatabase();
    DatabaseConnection.instance = this;
  }

  // Simulate a private connection method
  #connectToDatabase() {
    console.log("Establishing new database connection...");
    return { connected: true, db: "my_database" };
  }

  getConnection() {
    return this.connection;
  }
}

// Usage
const db1 = new DatabaseConnection();
const db2 = new DatabaseConnection();

console.log(db1.getConnection());
console.log(db1 === db2);
```

The output will be as follows:

```
Establishing new database connection...
{ connected: true, db: 'my_database' }
true
```

Key Points:

- Normally, to implement a singleton, you would create a private constructor in whichever programming language you are using. A private constructor will ensure that no external code can directly create an instance of the class. Only the class itself can create an instance, and in our example, that happens in the constructor. Notice how, in that constructor we check if a database connection has already been created and return the existing one if it is. A new instance is only created if no earlier connection exists. In JavaScript classes however, private constructors are not supported like they are in PHP, for example. But we can still achieve a Singleton behaviour by storing the instance in a static property and returning it if it already exists. This is the whole idea behind a singleton—

multiple calls to create a database connection will all return the same instance. This ensures that only one database connection is used throughout the application, which can improve performance and prevent multiple connections from being opened unnecessarily, which could lead to inefficiency and resource exhaustion. This is why this pattern is commonly used in situations like managing a single database connection in a web application.

- The `connectToDatabase()` method is declared private using the `#` prefix. Because the `connectToDatabase()` is a private method, it cannot be called directly, and that is the intention. That is why it is called by the class itself, from the constructor, when you instantiate the class in order to make the database connection.
- The `connectToDatabase()` method (which is private) is where we make the database connection. Normally, in a real application, this is where you will have the code on your server to make and return the connection to your database server. But since working on a server is beyond the scope of this book, I have mimicked that process by logging some text to the console saying we are making a connection with the database. Here is what it looks like:

```
#connectToDatabase() {  
  
    console.log("Establishing new database connection...");  
    return { connected: true, db: "my_database" };  
}
```

-Finally, note that we have logged the result of the expression that checks if the `db1` and `db2` instances are the same instance to see if the Singleton implementation worked.

```
console.log(db1 === db2);
```

The result of this is true.

FACTORY PATTERN

The factory pattern provides a way to create objects without specifying the exact class. It delegates the object creation process (logic) to subclasses or another object.

JavaScript example:

```
class Dog {  
    speak() {  
        return "Woof";  
    }  
}
```

```

class Cat {
    speak() {
        return "Meow";
    }
}

class AnimalFactory {
    static createAnimal(type) {
        if (type === "dog") {
            return new Dog();
        }
        else if (type === "cat") {
            return new Cat();
        }
        throw new Error("Unknown animal type");
    }
}

```

Using the factory class

```

const animal = AnimalFactory.createAnimal("dog");
console.log(animal.speak()); // Woof

```

ABSTRACT FACTORY PATTERN

The Abstract Factory Pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes. It allows the client to create objects that are part of a specific family, where the exact class of each object is determined by the factory. It is ideal for UI libraries or platform-dependent logic.

Example Scenario:

Let's say you're building an application that can work with two types of user interfaces: Windows and Mac. Each UI has its own specific elements (like buttons, checkboxes), and the abstract factory will help you create the appropriate family of UI components based on the environment.

JavaScript Code Example

```

// Abstract Products
class Button {
    render() {}
}
class Checkbox {
    check() {}
}

// Concrete Products for Windows

```

```

class WindowsButton extends Button {
    render() {
        console.log("Rendering Windows Button");
    }
}

class WindowsCheckbox extends Checkbox {
    check() {
        console.log("Checking Windows Checkbox");
    }
}

// Concrete Products for Mac
class MacButton extends Button {
    render() {
        console.log("Rendering Mac Button");
    }
}

class MacCheckbox extends Checkbox {
    check() {
        console.log("Checking Mac Checkbox");
    }
}

// Abstract Factory
class GUIFactory {
    createButton() {}
    createCheckbox() {}
}

// Concrete Factories
class WindowsFactory extends GUIFactory {
    createButton() {
        return new WindowsButton();
    }

    createCheckbox() {
        return new WindowsCheckbox();
    }
}

class MacFactory extends GUIFactory {
    createButton() {
        return new MacButton();
    }

    createCheckbox() {
        return new MacCheckbox();
    }
}

```

```
// Client
function renderUI(factory) {
  const button = factory.createButton();
  const checkbox = factory.createCheckbox();

  button.render();
  checkbox.check();
}

// Example of using the abstract
renderUI(new WindowsFactory());
renderUI(new MacFactory());
```

Key points

- Abstract Factory pattern allows creating families of related objects.
- You can easily switch between different families (e.g., Windows vs Mac).

BUILDER PATTERN

The Builder Pattern is used to create complex objects step by step. It separates the construction of a complex object from its representation, allowing different representations of the object to be created using the same process.

Example Scenario:

Consider building a 'House' with various customisable features like walls, doors, windows, and roof. Each house may differ in terms of these features, but the construction process is the same.

JavaScript Code Example:

```
// Product
class House {
  constructor() {
    this.walls = "";
    this.doors = "";
    this.windows = "";
    this.roof = "";
  }

  show() {
    console.log(
      `House with ${this.walls} walls, ${this.doors} doors, `
      + `${this.windows} windows, and a ${this.roof} roof`
    );
  }
}
```

```

// Builder Interface
class HouseBuilder {
    buildWalls() {}
    buildDoors() {}
    buildWindows() {}
    buildRoof() {}
    getHouse() {}
}

// Concrete Builder
class WoodenHouseBuilder extends HouseBuilder {
    constructor() {
        super();
        this.house = new House();
    }

    buildWalls() {
        this.house.walls = "Wooden";
    }

    buildDoors() {
        this.house.doors = "Wooden";
    }

    buildWindows() {
        this.house.windows = "Wooden";
    }

    buildRoof() {
        this.house.roof = "Wooden";
    }

    getHouse() {
        return this.house;
    }
}

// Director
class ConstructionEngineer {
    constructor(builder) {
        this.builder = builder;
    }

    constructHouse() {
        this.builder.buildWalls();
        this.builder.buildDoors();
        this.builder.buildWindows();
        this.builder.buildRoof();
        return this.builder.getHouse();
    }
}

// Usage

```

```
const builder = new WoodenHouseBuilder();
const engineer = new ConstructionEngineer(builder);
const house = engineer.constructHouse();
house.show();
```

The output of this code will be:

House with Wooden walls, Wooden doors, Wooden windows, and a Wooden roof

Key points:

- The Builder Pattern focuses on step-by-step construction of complex objects.
- The same building process can create different results (e.g., WoodenHouse or BrickHouse).

PROTOTYPE PATTERN

The Prototype Pattern involves creating new objects by copying an existing object (the prototype). This is useful when the cost of creating a new object is expensive, and you can avoid it by cloning an existing object. This pattern basically clones existing objects instead of creating new ones from scratch.

Example Scenario:

Imagine you are creating a 'Document Editor' where you need to create copies of documents. The prototype pattern allows you to create new documents by cloning existing ones instead of creating them from scratch.

JavaScript Code Example:

```
class TextDocument {
  constructor(content) {
    this.content = content;
  }

  clone() {
    return new TextDocument(this.content);
  }

  showContent() {
    console.log("Document content: " + this.content);
  }
}

// Usage
const originalDocument = new TextDocument("Original Content");
const clonedDocument = originalDocument.clone();
```



```
// Display the contents of both the
// original and the cloned document

// The output: Document content: Original Content
originalDocument.showContent();

// The output: Document content: Original Content
clonedDocument.showContent();
```

Key points:

- Prototype Pattern allows creating new objects by copying an existing object.
- It is useful for cases where object creation is expensive, and copying is more efficient.

2) STRUCTURAL DESIGN PATTERNS

Structural design patterns focus on how objects and classes are composed (structured). They focus on how objects and classes are composed to form larger structures, making code easier to manage and scale. They help ensure that if one part of a system changes, the entire structure doesn't need to change. These structural patterns help organise and manage the relationships between classes and objects in your applications, providing flexibility, reusability, and better organisation. The following design patterns fall under this group:

- ADAPTER PATTERN
- DECORATOR PATTERN
- FACADE PATTERN
- COMPOSITE PATTERN
- PROXY PATTERN
- BRIDGE PATTERN
- FLYWEIGHT PATTERN

ADAPTER PATTERN

The Adapter Pattern allows objects with incompatible interfaces to work together. It acts as a bridge between two interfaces that otherwise couldn't interact directly. This is useful when you want to integrate a class with a different interface into your system.

Example Scenario:

Imagine you have a Media Player that only supports playing MP3 files, but you want to add support for playing MP4 files. The adapter pattern helps convert the

MP4 interface to be compatible with the MP3 player.

JavaScript code example

```
// Old API
class OldPrinter {
  printText(text) {
    console.log(`Old Printer: ${text}`);
  }
}

// New expected interface
class NewPrinter {
  print(text) {
    console.log(`New Printer: ${text}`);
  }
}

// Adapter
class PrinterAdapter {
  constructor(oldPrinter) {
    this.oldPrinter = oldPrinter;
  }

  print(text) {
    // adapts the method
    this.oldPrinter.printText(text);
  }
}

// Usage
const oldPrinter = new OldPrinter();
const adaptedPrinter = new PrinterAdapter(oldPrinter);

// works like the new printer
adaptedPrinter.print("Hello World!");
```

Imagine you bought a new phone charger, but your wall socket is old and doesn't match the charger plug. What do you do? Use a plug adapter! It lets the new charger connect to the old socket. In this pattern, PrinterAdapter plays the role of the plug adapter. It makes the old printer work with new code that expects a different method (print instead of printText).

Key points

- The Adapter Pattern enables objects with incompatible interfaces to work together.
- You can introduce new functionality to an existing system without changing its structure.

DECORATOR PATTERN

The decorator pattern allows behaviour to be added to individual objects, either statically or dynamically, without affecting the behaviour of other objects from the same class. Basically, you can use this pattern to add behaviour to objects dynamically without changing their structure.

JavaScript example:

```
// Base
class Coffee {
  cost() {
    return 5;
  }
}

// Decorator
class MilkDecorator {
  constructor(coffee) {
    this.coffee = coffee;
  }

  cost() {
    return this.coffee.cost() + 1;
  }
}

class SugarDecorator {
  constructor(coffee) {
    this.coffee = coffee;
  }

  cost() {
    return this.coffee.cost() + 0.5;
  }
}

// Usage
let myCoffee = new Coffee();
myCoffee = new MilkDecorator(myCoffee);
myCoffee = new SugarDecorator(myCoffee);

console.log(`Total cost: ${myCoffee.cost()}`);
```

The result in the console will be something like this:

Total cost: \$6.5

Imagine you buy a basic coffee. Then you say, “Add milk.” Then, “Add sugar.” Each time, you’re upgrading the same coffee without changing the original cup. The Decorator pattern works exactly like that—wrapping extras around something, layer by layer, without touching its core.

Key points

- The purpose of the Decorator Pattern is to allow behaviour to be added to individual objects dynamically, without affecting the behaviour of other objects from the same class. This is done by “wrapping” the object with decorator classes that enhance or modify its functionality.
- The pattern is called a decorator because the decorator class is used to “decorate” or wrap the original class, adding additional features or responsibilities to the object being decorated. Each decorator class implements the same interface or inherits from the same parent class as the original object.
- Here is how the Code really Works: Component class Coffee:
 - This Coffee class defines the core functionality, which in this case is The method `cost()`
 - This sets the contract for both the base class (Coffee) and the decorators. Concrete Classes(like MilkDecorator or SugarDecorator):
 - These classes implement the basic behaviour of the Coffee class. In this case, it represents a simple coffee with a base cost. Decorator Classes (MilkDecorator and SugarDecorator):
 - These decorator classes implement the same Coffee class but enhance the functionality of the base class (Coffee). They add their own behaviour-in this case flavour (adding milk or sugar) while still calling the base class’s methods to maintain the existing functionality. For example, MilkDecorator adds the cost of milk to the base cost of coffee. Similarly, SugarDecorator adds the cost.
- The decorator pattern also allows for dynamic and flexible behaviour addition. You can apply multiple decorators in sequence, as shown in the example where we first decorate the coffee with milk and then with sugar. Each decorator adds to the cost and description, building on top of the previous one.

Use Case:

The Decorator Pattern is useful when you want to add functionality to an object without modifying its code, especially when you need to apply different combinations of behaviour (e.g., coffee with milk, coffee with sugar, coffee with milk

ination, you can achieve this through decorators. This decorator pattern provides flexibility because you can “stack” multiple decorators in any order and combine them as needed without altering the underlying object.

FACADE PATTERN

The facade pattern provides a simplified interface to a complex subsystem. It hides the complexities of the system behind a unified, easy-to-understand interface.

JavaScript example:

```
class CPU {
    start() {
        console.log("CPU started");
    }
}

class Memory {
    load() {
        console.log("Memory loaded");
    }
}

class HardDrive {
    read() {
        console.log("Hard drive read");
    }
}

// Facade
class Computer {
    constructor() {
        this.cpu = new CPU();
        this.memory = new Memory();
        this.hardDrive = new HardDrive();
    }

    start() {
        this.cpu.start();
        this.memory.load();
        this.hardDrive.read();
    }
}

// Usage
const myComputer = new Computer();
myComputer.start();
```

The output of this code will be:

CPU started Memory loaded Hard drive read

Key points:

Starting a computer involves many parts—CPU, memory, hard drive—all working together. But you don’t push ten buttons to turn it on. You press one power button. The Facade pattern is like that power button—it hides the messy details behind a single, simple interface.

- The Facade Pattern provides a simplified interface to a complex system or a set of classes. It hides the complexity of the subsystems and offers a unified, easier-to-use interface for the client.
- It is called a Facade because it acts as the “front” or “face” of a set of subsystems. Instead of interacting directly with complex subsystems (like CPU, Memory, and HardDrive in this example), the client interacts with a single class (the ComputerFacade) that manages the communication with the subsystems behind the scenes.
- Here is how the Code Works: Subsystem Classes:
 - The CPU, Memory, and HardDrive are the individual subsystems that perform specific tasks.
 - Each class has its own methods (start(), load(), read(), etc.) that are specific to its responsibility. Facade Class (Computer):
 - This class aggregates (brings together) the subsystems (CPU, Memory, and HardDrive) and exposes a simple start() method to the client.
 - The start() method internally coordinates the necessary calls to the subsystems, such as freezing the CPU, loading data into memory, reading from the hard drive, and finally executing instructions.
 - The client only needs to call start(), without worrying about the individual steps required to boot the computer.

Use Cases:

- This can be used anywhere to simplify complex systems. When you have a system composed of several intricate components (such as hardware or APIs), a facade can simplify interactions by consolidating them into a single, easy-to-use interface. For example, in this case, starting a computer requires multiple steps, but the facade hides all that complexity.
- It can be used to Reducing Tight Coupling. The client code is only coupled with the facade (ComputerFacade), not with the individual subsystems (CPU, Memory, HardDrive). This makes the code easier to maintain and modify.

- It can also be used to Improve Code Readability. Facade patterns are often used to improve code readability. If the client had to call each subsystem's methods directly, the code would be more complex and harder to read.
- Another great benefit of the facade pattern allows for a clean, organised separation between the client and the complex internals of a system. It also allows for easier maintenance because changes to the subsystem do not affect the client, as long as the facade's interface remains consistent.
- You can see how the Facade Pattern simplifies interactions with a complex system by creating a single entry point for the client to use.

COMPOSITE PATTERN

The Composite Pattern is used to treat individual objects and compositions of objects uniformly. This pattern allows you to build a tree structure where individual objects and groups of objects are handled the same way.

Example Scenario:

A company might have employees who can be regular staff members or managers who supervise other employees. The composite pattern helps manage this hierarchy by treating both employees and managers uniformly.

JavaScript Example:

```
// Leaf
class File {
  constructor(name) {
    this.name = name;
  }

  display(indent = '') {
    console.log(`${indent}- File: ${this.name}`);
  }
}

// Composite
class Folder {
  constructor(name) {
    this.name = name;
    this.children = [];
  }

  add(item) {
    this.children.push(item);
  }
}
```

```

    console.log(`${indent}+ Folder: ${this.name}`);
    this.children.forEach(child => child.display(indent + ' '));
  }

}

// Usage
const root = new Folder('Root');
const file1 = new File('file1.txt');
const file2 = new File('file2.txt');

const subFolder = new Folder('SubFolder');
subFolder.add(new File('file3.txt'));

root.add(file1);
root.add(subFolder);
root.add(file2);

root.display();

```

The output will look like so:

- Folder: Root
- File: file1.txt
- Folder: SubFolder
- File: file3.txt
- File: file2.txt

Notice the indentation. Each level down adds two more spaces, because `display()` passes `indent + ' '` to its children. That indentation is the whole point: it is what turns a flat list of lines into a picture of the hierarchy.

Think of your computer's folders and files. A folder can contain both files and other folders. Whether it's a file or a folder, you can click and view it the same way. The Composite pattern lets you treat both single items (files) and groups (folders with files) the same, simplifying how you interact with them.

Key points

- The Composite Pattern allows you to treat individual objects and groups of objects the same way.
- It is useful for representing hierarchical structures like employees, files, or GUI components.

PROXY PATTERN

The Proxy Pattern provides a placeholder or surrogate for another object to control access to another object. It's used to add an extra level of control before accessing the actual object, such as in cases of lazy loading, access control, or logging.

Example Scenario:

Imagine you have a large image that takes time to load. Instead of loading it directly, you can use a proxy that loads the image only when it's needed.

JavaScript Code Example:

```
// Real object
class RealImage {
  constructor(filename) {
    this.filename = filename;
    this.load();
  }

  load() {
    console.log(`Loading ${this.filename}`);
  }

  display() {
    console.log(`Displaying ${this.filename}`);
  }
}

// Proxy
class ProxyImage {
  constructor(filename) {
    this.filename = filename;
    this.realImage = null;
  }

  display() {
    if (!this.realImage) {
      this.realImage = new RealImage(this.filename);
    }

    this.realImage.display();
  }
}

// Usage
const image = new ProxyImage('cat.png');
```

```
image.display(); // Loads and displays
// Only displays, doesn't load again
image.display();
```

The output will be:

Loading cat.png Displaying cat.png Displaying cat.png

Look carefully at those three lines, because they are the whole point. "Loading" appears only once, on the first call. The second call to `display()` finds the real image already made and skips straight to displaying it. That is the proxy doing its job: the expensive work happens once, and only when it is first needed.

Imagine opening a large image file. The first time, it takes time to load. But after that, it opens instantly. The Proxy pattern works like a smart assistant that only loads the heavy image when truly needed—saving time and resources.

Key points:

- The Proxy Pattern provides a surrogate to control access to another object.
- It's useful for lazy initialisation, access control, or logging.

BRIDGE AND FLYWEIGHT PATTERNS

These remaining two structural patterns are less common than the others and you therefore may not find it listed in some books or documents. This is because Some beginner-level articles or tutorials only introduce the most common patterns. Besides, patterns like Bridge and especially Flyweight are used less often or are harder to grasp, so they sometimes get left out of beginner resources. Also, some blog authors may occasionally incorrectly group patterns or exclude a few based on personal interpretation. Nonetheless, I will teach you about them here.

BRIDGE PATTERN

The bridge pattern separates an abstraction from its implementation so both can change independently.

JavaScript example

```
// Implementation
class DrawingAPI1 {
  drawCircle(x, y, radius) {
    console.log(`API1 drawing circle at (${x}, ${y}) with radius
    ${radius}`);
  }
}
```

```

drawCircle(x, y, radius) {
    console.log(`API2 drawing circle at (${x}, ${y}) with radius
${radius}`);
}
}

// Abstraction
class Circle {
    constructor(x, y, radius, drawingAPI) {
        this.x = x;
        this.y = y;
        this.radius = radius;
        this.drawingAPI = drawingAPI;
    }

    draw() {
        this.drawingAPI.drawCircle(this.x, this.y, this.radius);
    }
}

// Usage
const circle1 = new Circle(5, 10, 15, new DrawingAPI1());
const circle2 = new Circle(2, 4, 8, new DrawingAPI2());

circle1.draw();
circle2.draw();

```

The output in the console here will be:

API1 drawing circle at (5, 10) with radius 15 API2 drawing circle at (2, 4) with radius 8

Key points:

Let's say you're a designer, and you have two types of drawing tools: a pencil and a marker. You can draw the same circle with either tool. The Bridge pattern lets you separate what you draw (a circle) from how you draw it (pencil or marker). This means you can swap tools anytime without changing the actual process of drawing (the drawing logic).

FLYWEIGHT PATTERN

This pattern helps reduce memory usage by sharing common data between similar objects.

JavaScript example

```

// Shared flyweight
class TreeType {
    constructor(name, color) {

```

```

    this.name = name;
    this.color = color;
  }

  draw(x, y) {
    console.log(`Drawing ${this.name} tree at (${x}, ${y}) in
    ${this.color}`);
  }
}

// Factory
class TreeFactory {
  constructor() {
    this.treeTypes = {};
  }

  getTreeType(name, color) {
    const key = name + color;
    if (!this.treeTypes[key]) {
      this.treeTypes[key] = new TreeType(name, color);
    }

    return this.treeTypes[key];
  }
}

// Usage
const factory = new TreeFactory();

const tree1 = factory.getTreeType('Oak', 'Green');
tree1.draw(10, 20);

const tree2 = factory.getTreeType('Oak', 'Green');
tree2.draw(15, 25);

console.log(tree1 === tree2); // true

```

The output in the console here will be:

Drawing Oak tree at (10, 20) in Green Drawing Oak tree at (15, 25) in Green true

Key points:

Imagine a video game with a forest of thousands of trees. If each tree has its own copy of “Oak, Green,” memory will run out fast. The Flyweight pattern makes all identical trees share the same blueprint and only store what’s unique (like position). This saves tons of memory.

3) BEHAVIORAL DESIGN PATTERNS

Behavioural design patterns focus on how objects communicate and interact with each other. They define the way in which classes and objects collaborate. The following design patterns fall under this group:

- OBSERVER PATTERN
- STRATEGY PATTERN
- TEMPLATE METHOD PATTERN
- COMMAND PATTERN
- ITERATOR PATTERN

OBSERVER PATTERN

The observer pattern is used when there is one subject and multiple observers that depend on the subject's state. Whenever the subject changes its state, it notifies all its observers.

JavaScript example:

```
// Subject class
class Subject {
  constructor() {
    this.observers = [];
  }

  addObserver(observer) {
    this.observers.push(observer);
  }

  notify() {
    this.observers.forEach(observer => observer.update());
  }
}

// Observer class
class Observer {
  update() {
    console.log("Observer notified");
  }
}

// Usage
const subject = new Subject();
const observer = new Observer();
```

```
subject.addObserver(observer);
subject.notify();
```

The output in the console will say:

Observer notified

Key points:

This is how this works:

- The Subject is like a news channel. It allows multiple subscribers (Observers) to register with it.
- When the Subject has an update (something changes), it calls `notify()`, and all subscribers get notified through their own `update()` method.
- This pattern is great for scenarios like chat apps (new messages notify all listeners) or live price updates.

STRATEGY PATTERN

The strategy pattern allows you to define a family of algorithms, encapsulate each one, and make them interchangeable. It lets the algorithm vary independently from the clients that use it. In other words, the clients use the algorithm relevant to them through the family interface. This pattern needs

- an interface to define the algorithm family
- one or more strategy classes to implement the different algorithms in the family, each in its own way
- one client-interfacing class (often referred to as context) to bring the two (interface and strategies) together, which clients will use to vary their strategies seamlessly.

JavaScript example:

```
// Strategy Interface: Just a shared shape in JS
class PaymentStrategy {
  pay(amount) {
    throw new Error("This method should be overridden");
  }
}

// Concrete strategy: PayPal
class PayPalStrategy extends PaymentStrategy {
  pay(amount) {
    return `Paid ${amount} using PayPal`;
  }
}
```

```
// Concrete strategy: Credit Card
class CreditCardStrategy extends PaymentStrategy {
    pay(amount) {
        return `Paid ${amount} using Credit Card`;
    }
}

// Context class
class PaymentContext {
    constructor(strategy) {
        this.strategy = strategy;
    }

    executePayment(amount) {
        return this.strategy.pay(amount);
    }
}

// Client usage
const paypalContext = new PaymentContext(new PayPalStrategy());
console.log(paypalContext.executePayment(100));
// Paid $100 using PayPal

const cardContext = new PaymentContext(new CreditCardStrategy());
console.log(cardContext.executePayment(200));
// Paid $200 using Credit Card

// The output in the console: Paid $100 using PayPal Paid $200 using Credit Card
```

Key points and explanation

- You create different payment strategies (like PayPal or Credit Card).
- The PaymentContext is like a wallet that lets you swap how you want to pay.
- This avoids messy if-else code and makes it easy to add new payment methods without touching the core logic.
- The purpose of the Strategy Pattern is to define a family of algorithms (or strategies) that can be used interchangeably. Instead of hardcoding specific behaviour into a class, different strategies are encapsulated in separate classes, allowing the behaviour to be selected at runtime.
- Here is how it works:
 - Strategy Interface (PaymentStrategy) This interface defines the common method pay(amount) that all concrete strategies must implement. It ensures that all payment methods share the same contract.
 - Concrete Strategies (PayPalStrategy, CreditCardStrategy): These classes implement the PaymentStrategy interface and provide the specific behaviour

for how the payment is processed. For example, `PayPalStrategy` handles payments using PayPal, while `CreditCardStrategy` handles payments using a credit card.

- **Context Class (`PaymentContext`):** The context class (`PaymentContext`) is responsible for interacting with the chosen strategy. It accepts a `PaymentStrategy` object as a parameter and uses it to process the payment without knowing the details of how the payment is handled. This decouples the client code from the specific strategies.

Use Cases:

- **Dynamic Behaviour Selection.** When you need to switch between different algorithms or behaviours at runtime, the strategy pattern is useful. For example, choosing different payment methods (like PayPal or credit card) based on the user's preference.
- **Avoiding Conditional Logic.** Instead of using complex if-else or switch statements to determine the behaviour, the strategy pattern encapsulates these behaviours into separate classes, making the code more maintainable and flexible.
- **The big benefit of the Strategy Pattern** is that it promotes the open/closed principle (the O in the SOLID principles). It is open for extension in that new strategies can be added without changing the existing code in the `PaymentContext`. The pattern allows for flexibility by letting you swap out behaviour dynamically while keeping the code structure clean and modular.

TEMPLATE METHOD PATTERN

The Template Method Pattern defines the skeleton of an algorithm in a base class, while allowing subclasses to override specific steps of the algorithm without changing its structure. It is called the Template Method because it provides a template for the overall process, with some steps left open for customisation by subclasses.

JavaScript Example:

```
// Base class
class MealPreparation {
  prepareMeal() {
    this.boilWater();
    // this will be different in each subclass
    this.cook();
    this.serve();
  }
}
```



```

console.log("Boiling water");
}

    serve() {
        console.log("Serving the meal");
    }

    cook() {
        throw new Error("You must override the cook method");
    }
}

// Concrete class: Pasta
class PastaMeal extends MealPreparation {
    cook() {
        console.log("Cooking pasta");
    }
}

// Concrete class: Rice
class RiceMeal extends MealPreparation {
    cook() {
        console.log("Cooking rice");
    }
}

// Usage
const pasta = new PastaMeal();
pasta.prepareMeal();
// Prints three lines – see the full output below

const rice = new RiceMeal();
rice.prepareMeal();
// Prints three lines – see the full output below

```

The output in the console will be:

Boiling water Cooking pasta Serving the meal Boiling water Cooking rice Serving the meal

Key points

- The purpose of the template design pattern is that it allows you define the framework of an algorithm in a base class, leaving the details of specific steps to be implemented by subclasses.
- Here is How It Works:
 - The Template Method is a predefined “recipe” (a sequence of steps).

- Some steps are shared (boiling, serving), while others (cooking) are left open so that each type of meal (Pasta or Rice) can define them.
- It ensures a consistent structure while allowing flexibility in specific steps.

Template Method (prepareMeal): This method is defined in the base class (Meal-Preparation) where the algorithm's structure is defined. Here, some steps (like boil-Water and serve) are common, while others (like cook) are left abstract for sub-classes to implement.

There are concrete Classes (PastaMeal, RiceMeal): These classes implement the step (cook) in their own way, allowing flexibility while still following the overall process defined by the base class.

Use Case:

The Template Method Pattern is useful when multiple classes share a similar process but require customisation for specific steps. In the example, both pasta and rice meals follow the same process but differ in the cooking step.

COMMAND PATTERN

The Command Pattern turns a request into an object, allowing the parameterisation of clients with queues, requests, or logs. It is called Command because each object represents an operation to be executed, stored, or undone.

JavaScript Example:

```
// Command interface
class Command {
  execute() {
    throw new Error("Execute method should be implemented");
  }
}

// Receiver class
class Light {
  turnOn() {
    console.log("Light is ON");
  }

  turnOff() {
    console.log("Light is OFF");
  }
}

// Concrete command: turn light on
```

```

super();
    this.light = light;
}

execute() {
    this.light.turnOn();
}
}

// Concrete command: turn light off
class LightOffCommand extends Command {
    constructor(light) {
        super();
        this.light = light;
    }

    execute() {
        this.light.turnOff();
    }
}

// Invoker class
class RemoteControl {
    setCommand(command) {
        this.command = command;
    }

    pressButton() {
        this.command.execute();
    }
}

// Client code
const light = new Light();
const remote = new RemoteControl();

remote.setCommand(new LightOnCommand(light));
remote.pressButton();
// Output: Light is ON

remote.setCommand(new LightOffCommand(light));
remote.pressButton();
// Output: Light is OFF

```

The output of this code in the console will be:

Light is ON
Light is OFF

Key points

Purpose of the Command Pattern: This pattern encapsulates requests as objects, allowing you to parameterise methods, delay execution, and queue operat-

ions. It decouples the invoker (client) from the object that performs the actual work (receiver).

How It Works:

- Think of the command as a wrapper around an action.
- Each Command holds a receiver (like Light) and tells it what to do (turn on or off).
- The RemoteControl is just a trigger—it doesn't know how the light works.
- This pattern is great when you want to schedule, queue, or undo operations.
- Command Interface:
 - The Command interface defines a method (execute()) that will be implemented by different commands. Concrete Commands
 - LightOnCommand,
 - LightOffCommand):

These command classes accept a receiver class Light. This makes sense because their command action is all about light. Through their execute() methods, these classes indirectly implement the specific actions (turnOn, turnOff) by delegating the work to the Light receiver class which is the class having these turnOn() and turnOff() methods. Which of them is called will depend on the command interface—so it will be turnOn() or turnOff() for LightOnCommand and LightOffCommand respectively.

Invoker (RemoteControl): The invoker class stores a command and executes it when the client presses a button. The invoker doesn't know the details of what the command does, it simply executes the execute() method on the command. It is called the invoker because the execution of the command starts from it. It all starts from its pressButton() method. It then runs the execute() method on the command which it had already stored in its 'command' property.

Use Case:

The Command Pattern is useful for implementing undo/redo functionality, executing commands in sequence, or logging operations for future execution. In the example, a remote control can switch between different commands (turning the light on or off) without knowing how each command works internally.

ITERATOR PATTERN

The Iterator Pattern provides a way to access the elements of a collection (like an array or list) sequentially without exposing the underlying structure. It is called Iterator because it “iterates” over a collection one element at a time.

JavaScript Example:

```
// Collection class
class BookCollection {
  constructor() {
    this.books = [];
  }

  addBook(book) {
    this.books.push(book);
  }

  getIterator() {
    return new BookIterator(this.books);
  }
}

// Iterator class
class BookIterator {
  constructor(books) {
    this.books = books;
    this.index = 0;
  }

  hasNext() {
    return this.index < this.books.length;
  }

  next() {
    return this.books[this.index++];
  }
}

// Client usage
const collection = new BookCollection();
collection.addBook("Design Patterns");
collection.addBook("Clean Code");

const iterator = collection.getIterator();

while (iterator.hasNext()) {
  console.log(iterator.next());
}
```

The output in the console will be:

Design Patterns Clean Code

Key points

Purpose of the Iterator Pattern: This pattern allows clients to traverse through the elements of a collection without needing to know the underlying structure of the collection. It provides a standardised way to access and iterate over data.

How It Works:

- This pattern helps you walk through a collection one item at a time without knowing how that collection is built internally.
- The BookIterator takes the book list and lets us use hasNext() and next() to access each book.
- It's like a movie queue: instead of grabbing all at once, you go through them in order.

Collection Interface (Collection):

- This defines a method (getIterator()) to return an iterator for the collection.
- Concrete collection classes (like BookCollection) implement this method.

Iterator Class (BookIterator):

- The BookIterator class is a class that stands on its own and is used by the BookCollection classes (via their getIterator() methods to which they will pass their array of items-be it books or anything else). It defines methods like hasNext() and next() to access elements in the collection one by one.

Client Code:

The client doesn't need to know how the BookCollection stores its books. It just uses the iterator to access the books sequentially using hasNext() and next().

Use Case:

The Iterator Pattern is useful when you need to traverse a collection without exposing its internal details. It is especially helpful when working with custom data structures or complex collections. In the example, the BookIterator provides a simple way to loop through a collection of books without directly accessing the array.

QUIZ — Design Patterns

This page contains the Q & A (questions and answers) for this chapter — Chapter 17: Design Patterns. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 12 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. The patterns in this chapter fall into three groups. Name them, and say in a few words what each group is concerned with.

Clue: one group is about making things, one about arranging things, and one about how things talk to each other.

2. What problem does the Singleton pattern solve, and why can JavaScript not implement it the way most other languages do?

Clue: the usual approach relies on a kind of constructor JavaScript does not have.

3. What is the `build()` method for in the Builder pattern, and what is wrong with creating an object like this?

```
const user = new User("John", 25, true, false, "admin");
```

Clue: read that line and try to say out loud what the third and fourth arguments mean.

4. What does the Factory pattern give you that calling `new Dog()` directly does not?

Clue: think about who has to know the name of the class.

4. In the Proxy example, `image.display()` is called twice. What appears in the console, and why is that the whole point of the pattern?

Clue: count how many times the word "Loading" appears.

5. What is the difference between the Facade pattern and the Adapter pattern? Both sit in front of something else.

Clue: one hides complexity you do not want to deal with. The other translates between two things that do not fit together.

6. The Observer pattern is described as things "subscribing" to a subject. Where have you already met this idea in JavaScript, probably without knowing its name?

Clue: you have been attaching one to a button since Chapter 1.

7. What does the Strategy pattern let you change, and when would you reach for it instead of a long if...else chain?

Clue: the word "strategy" is doing real work here — think of several different ways to accomplish the same task.

8. EXERCISE. Write a Singleton called Logger, so that no matter how many times you create one, you always get the same object back. Prove it with ===.

Clue: store the instance on the class itself, and hand it back if it already exists.

9. EXERCISE. Write a small Factory that returns a Circle, a Square, or throws an error for anything else. Each shape should have an area() method.

Clue: a static method on the factory class is enough. It does not need to be instantiated.

11. EXERCISE. Using the Builder pattern, build a Pizza step by step — size, then two toppings — and finish with build(). Make the calls chain together on one line.

Clue: each setter has to return something for the next call to attach to.

12. EXERCISE. Write a Subject that keeps a list of observers and notifies them all. Add two observers that print different messages, then notify.

Clue: the subject holds an array; notifying means looping over it and calling the same method on each.

ANSWERS

1.

- **Creational patterns** — concerned with how objects are made. Singleton, Factory, Abstract Factory, Builder, Prototype.
- **Structural patterns** — concerned with how objects are put together and arranged. Adapter, Decorator, Facade, Composite, Proxy, Bridge, Flyweight.
- **Behavioural patterns** — concerned with how objects communicate and share responsibilities. Observer, Strategy, Template Method, Command, Iterator.

A rough way to remember it: creational is about birth, structural is about arrangement, behavioural is about conversation.

2. The Singleton makes sure a class has only **one** instance, and gives everyone the same one. It suits shared resources such as a database connection, an application's configuration, or a logger — things you want exactly one of.

Most languages implement it with a **private constructor**, so that no outside code can call `new` at all, and a static method hands out the single instance. JavaScript has no private constructors, so that route is closed.

What JavaScript does instead is check inside the constructor whether an instance already exists, and if so return that one:

```
class DatabaseConnection {
  constructor() {
    if (DatabaseConnection.instance) {
      return DatabaseConnection.instance;
    }

    DatabaseConnection.instance = this;
  }
}
```

Returning something from a constructor is unusual, but perfectly legal, and it is what makes the pattern work here.

3. `build()` is the finishing step of the Builder pattern. It hands back the completed object, and it is the natural place to check the object is valid before letting it out.

What is wrong with that `new User(...)` line is that it is unreadable:

```
const user = new User("John", 25, true, false, "admin");
```

What is `true`? What is `false`? You cannot tell without opening the `User` class and counting the parameters. And the moment somebody swaps two of them by mistake, nothing complains — you simply get a user who is not an admin when they should be.

The Builder pattern fixes it by naming each step:

```
class UserBuilder {
  constructor(name) {
    this.user = { name };
  }

  setAge(age) {
    this.user.age = age;
    return this;
  }
}
```

```

    setAdmin(isAdmin) {
        this.user.isAdmin = isAdmin;
        return this;
    }

    build() {
        return this.user;
    }
}

const user = new UserBuilder("John")
    .setAge(25)
    .setAdmin(true)
    .build();

```

Now every value says what it is. The `return this` in each setter is what allows the calls to be chained one after another — each one hands the builder back so the next can be called on it.

4. It means the code asking for the object **does not need to know the class name**.

```
const animal = AnimalFactory.createAnimal("dog");
```

The caller passes a string and gets back something that can `speak()`. It never mentions `Dog` at all. So if you later rename the class, split it in two, or decide that "dog" should return a `Puppy` on Tuesdays, you change the factory and nothing else.

With `new Dog()` scattered through your program, every one of those places has to change.

5. The console shows:

Loading cat.png Displaying cat.png Displaying cat.png

The word "Loading" appears **once**, even though `display()` was called twice. That is the whole point. The proxy holds off creating the real, expensive image until it is genuinely needed, and once it exists it is reused. The second call skips straight to displaying.

This is why the pattern is useful for large images, heavy network calls, or anything else you would rather not do until you must — and never twice.

6. Both sit in front of something else, but for different reasons.

- **Facade** hides complexity. Behind it are several parts that all have to be used together, and the facade gives you one simple way in. The chapter's example is starting a computer: CPU, memory and hard drive all have to start, but you press one button.

- **Adapter** translates. It exists because two things that need to work together have incompatible shapes — an old interface and a new one, say — so the adapter converts between them.

Put simply: a facade **simplifies** something that already works; an adapter **converts** something that otherwise would not fit.

7. In **event listeners**.

```
button.addEventListener("click", handleClick);
```

That is the Observer pattern exactly. The button is the subject, your function is the observer, and `addEventListener` is the subscription. When the click happens, everything that subscribed gets told.

You have been using it since Chapter 1 without needing the name. That is worth noticing: patterns are mostly names for things good programmers already do.

8. The Strategy pattern lets you swap out **how** something is done, while what is being done stays the same. In the chapter's example, paying is always paying — but it might happen by PayPal or by credit card.

You would reach for it instead of a long `if...else` chain when:

- the list of options is likely to grow, since adding a strategy means adding a class rather than editing a chain everybody depends on
- you want to choose the approach while the program is running
- the branches are big enough that the chain becomes hard to read

For two or three short branches an `if...else` is perfectly fine. The pattern earns its place when there are many, or when they keep changing.

```

9. class Logger {
    constructor() {
        if (Logger.instance) {
            return Logger.instance;
        }

        this.messages = [];
        Logger.instance = this;
    }

    log(message) {
        this.messages.push(message);
        console.log(message);
    }
}

const logger1 = new Logger();
const logger2 = new Logger();

logger1.log("First message");
logger2.log("Second message");

console.log(logger1 === logger2);
// true
console.log(logger1.messages.length);
// 2

```

The last two lines are the proof. They are the same object, so the message logged through `logger2` is sitting in `logger1`'s list as well.

```

10. class Circle {
    constructor(radius) {
        this.radius = radius;
    }

    area() {
        return Math.PI * this.radius * this.radius;
    }
}

class Square {
    constructor(side) {
        this.side = side;
    }

    area() {
        return this.side * this.side;
    }
}

class ShapeFactory {
    static createShape(type, size) {
        if (type === "circle") {
            return new Circle(size);
        } else if (type === "square") {
            return new Square(size);
        }

        throw new Error("Unknown shape type");
    }
}

const c = ShapeFactory.createShape("circle", 2);
const s = ShapeFactory.createShape("square", 3);

console.log(c.area().toFixed(2));
// 12.57
console.log(s.area());
// 9

```

`static` means the method belongs to the class itself, so you call it as `ShapeFactory.createShape(...)` without ever making a `ShapeFactory` object. There would be no point making one, since the factory holds no data of its own.

```

11. class PizzaBuilder {
    constructor() {
        this.pizza = { toppings: [] };
    }

    setSize(size) {
        this.pizza.size = size;
        return this;
    }

    addTopping(topping) {
        this.pizza.toppings.push(topping);
        return this;
    }

    build() {
        return this.pizza;
    }
}

const pizza = new PizzaBuilder()
    .setSize("large")
    .addTopping("cheese")
    .addTopping("mushrooms")
    .build();

console.log(pizza);
// { toppings: [ 'cheese',
// 'mushrooms' ], size: 'large' }

```

The chaining works because every setter ends with `return this`. Each call hands the builder straight back, so the next call has something to attach to. Take one `return this` out and the chain breaks at that point with "Cannot read properties of undefined".

```

12. class Subject {
    constructor() {
        this.observers = [];
    }

    addObserver(observer) {
        this.observers.push(observer);
    }

    notify() {
        this.observers.forEach(observer => observer.update());
    }
}

class EmailObserver {
    update() {
        console.log("Email sent");
    }
}

class LogObserver {
    update() {
        console.log("Written to the log");
    }
}

const subject = new Subject();

subject.addObserver(new EmailObserver());
subject.addObserver(new LogObserver());

subject.notify();

```

Output:

Email sent Written to the log

Notice that the subject knows nothing about what its observers actually do. It only knows they each have an `update()` method. That is what makes the pattern useful: you can add a third observer tomorrow without touching the Subject class at all.

Chapter 18 — FILE MANAGEMENT

- The File Reader API
- Use-cases for the File Reader API
- Key methods
- Upload a text file and display its contents
- Drag and drop a file, modify and download it
- Generate CSV from a JavaScript Array
- Read data from a CSV file and inject into the DOM
- Reading a file's raw binary data with `readAsArrayBuffer()`
- Validating File Signatures (a.k.a. Magic Numbers)
- Sending binary file content over WebSocket
 - What happens on the WebSockets server side
 - Enabling CORS in a Node.js + Express Server
 - WebSockets vs video streaming
- Previewing a PDF file
- Reading XML files with JavaScript
 - Using the `FileReader` and `DOMParser`
 - Using the `Fetch API` and `DOMParser`
 - Using the `XMLHttpRequest` object
 - JavaScript XML handling with `XPath`
 - A quick reminder of what `XPath` is
 - `XPath` in action
 - Read from a local or remote XML file
 - Reading data from an XML string
- Limitations of the `FileReader` API

The managing of files is a very important part of every software application. This topic will demonstrate how your programming language is able to manage files (create, view, modify, delete, and transmit them through local/remote networks). When we talk about file management, we're usually referring to the ability to read, write, or manipulate files—things like text documents, images, or data files—either on a local machine or in a web application. In Chapter 20 (Images) we will talk about how JavaScript can interact with images in the browser, but in this chapter,

we will talk about what JavaScript can do with text files or data files. Just keep in mind that the techniques you will learn here will be very similar to the ones you will use when handling images as you will see. JavaScript does not have full access to the file system of your computer. Its access is read-only. This is for security reasons. It can also access files (read-only) on your local machine only in two ways; either via a file upload input field:

```
<input type="file">
```

or by using a drag and drop feature. Nonetheless, JavaScript can still do quite a lot when it comes to file handling as we will see. Just to give you a hint; being given only read access to a file does not mean those contents cannot be read and used on a web page, or used in another generated file for download. This is the approach we will be using when handling both files and images, as you will see. Here are some of the things you can do with files in JavaScript:

- Read local files that users upload via an `<input type="file">` element or by dragging and dropping them into the browser window.
- Preview file contents, such as displaying an image or reading the text content of a file.
- Upload files to a server using AJAX, for example via the Fetch API.
- Let users download modified files, such as resized images or edited text, by generating downloadable links.

Drag and drop is a popular, user-friendly way to allow users to upload or interact with files in the browser. While it's not file management in the strictest sense, it pairs beautifully with file handling features, and we'll explore it in this chapter as well. In this chapter we'll walk through:

- Reading and displaying text files.
- Using drag and drop to accept files.
- Saving or exporting edited content (like creating a downloadable text file).
- Understanding how browser limitations affect what we can do with files. Later, if you ever move on to backend JavaScript with Node.js, you'll unlock full file system access. But for now, we'll explore everything the browser side has to offer. Let's dive in.

The File Reader API

The tool which JavaScript uses to manage files is the File Reader API. This API which comes built into JavaScript, allows web applications to read the contents of

files selected by users via a file type input field, or drag-and-drop. However, it does not provide full filesystem access due to security restrictions. It is fully supported by all modern browsers. Let's start by talking about the shortcomings of the API. The File Reader API has a few limitations. Here is a list of them:

- The API cannot arbitrarily read or write files on the user's filesystem, at least not without user interaction (again, eg via clicking to upload a file, or drag-and-drop). This means it only works with files that the user has explicitly selected to upload via a file input field, or using drag-and-drop. Outside of these two ways, it has no access to files on a local machine.
- The API only has read-only access to these files. This means it is not able to modify or delete the files from disk.
- It doesn't support file streaming or heavy-duty file I/O the way server-side languages like Node.js, Python, or PHP can do.
- It is not meant for very large files. Reading very large files may cause performance issues.

For more capabilities with files, there is also a newer File System Access API which allows more advanced file operations (read/write). However, it needs the computer owner's permission each time, which makes the process very manual, and its support is still uneven. It works in the Chromium browsers — Chrome, Edge and Opera — while Firefox does not support it at all and Safari supports only part of it. That is enough of a gap to keep it out of anything you want working everywhere.

Use-cases for the File Reader API

Despite its limitations, the API is still very powerful. It is supported by all browsers, and is thus very reliable. Here is a list of the awesome things you can do with the API.

- It works well when used to upload and preview files before saving them for later use, or uploading them to a server.
- Convert images and media files into base64 format.
- Read binary data or raw bytes using `ArrayBuffers`.
- You can use it to process CSV, text, JSON, XML files etc in the browser.

The steps to edit a file are as follows. After uploading a file and capturing the uploaded file using the File Reader API, to edit it, you create a textarea input field dynamically, next, you place the content (eg text) of the file inside of this textarea,

then append the textarea to the DOM. In all of this, you are not able to save any changes you have made to the original file—as that will mean overriding the original file in its location in your computer’s filesystem—which, as we know, JavaScript is unable to do. To work around this restriction on file modification so that you can save the changes resulting from your processing of a file in the browser, you should make your changes downloadable, as a file. This way the user can download it to their local machine. To do this in JavaScript, after reading the file with the File Reader API’s `readAsText()` method for example, you would use the URL object’s `createObjectURL()` method (`URL.createObjectURL()`) to create a link to the processed file, and make that link a download trigger for the file to be saved to the downloads directory of the user’s machine. Note that if you were working with the canvas API, after reading the image file with the API’s `readAsDataURL()`, you would convert it into a download link using the `toDataURL()` method of the canvas object (`canvas.toDataURL()`). I will show all of this in action using some demo examples. By creating this downloadable version of the edited file, the original file stays unmodified. The only way JavaScript would have been able to modify the original file by saving changes to it is if you used the new File System Access API, which allows updating a file in its original file location. Using the File System Access API is not recommended since it requires user permission and is currently only supported in Chromium browsers.

Key methods

The following are the methods the File Reader API provides to read files with. The method names, reflect the formats in which a file will be read.

- `readAsText()`. This will read the contents of a file as plain text.
- `readAsDataURL()` will read the contents of a file as a Base64-encoded string. This is very useful for images. We will see this in action when dealing with images.
- `readAsArrayBuffer()` will read the contents of a file as binary data
- `readAsBinaryString()` is a legacy method. Browsers still support it, but it is deprecated and you should not reach for it in new code — use `readAsArrayBuffer()` instead.

I think it’s time to jump into some examples. Don’t you just love examples? The following examples are carefully chosen to give you enough information on various file handling operations, that you will gain the confidence to do much more with files with JavaScript. We will implement them all in one HTML page, so first, let’s style the page and define the HTML elements:

The HTML (eg index.html)

```
<!DOCTYPE html>
<html>
  <head>
    <title>The JavaScript Blueprint</title>
    <link rel="stylesheet" href="index.css">
  </head>
  <body>
    <h1>File Management Lab</h1>

    <!-- Example 1: Upload and display text -->
    <section>
      <h2>Upload a Text File and Display Its Content</h2>
      <input type="file" id="fileInput" />
      <pre id="output">Your file content will appear here...
</pre>
    </section>

    <!-- Example 2: Drag & Drop file, modify, download -->
    <section>
      <h2>Drag & Drop a Text File, Modify and Download</h2>
      <div id="dropZone">Drop a .txt file here</div>
      <button id="downloadBtn" style="display:none;">
        Download Modified File</button>
      <pre id="modifiedContent">
        Modified file content will show here...
      </pre>
    </section>

    <!-- Example 3: Generate CSV from array -->
    <section>
      <h2>Generate and Download a CSV File from Array</h2>
      <button id="generateCSV">Download CSV</button>
      <p>This button downloads a CSV file with sample data like
        names and ages.</p>
    </section>

    <script type="module" src="index.js"></script>
  </body>
</html>
```

The CSS (eg index.css)

```
body {
  font-family: Arial, sans-serif;
  padding: 20px;
  background: #f7f7f7;
}

h2 {
```

```

border-bottom: 2px solid #ccc;
padding-bottom: 4px;
}

section {
background: #fff;
border-radius: 10px;
padding: 20px;
margin-bottom: 30px;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.05);
}

pre {
background: #f0f0f0;
padding: 10px;
border-radius: 5px;
white-space: pre-wrap;
max-height: 200px;
overflow-y: auto;
}

#dropZone {
border: 2px dashed #888;
padding: 20px;
text-align: center;
color: #444;
margin-top: 10px;
}

button {
margin-top: 10px;
padding: 8px 15px;
background: #007bff;
color: #fff;
border: none;
border-radius: 5px;
cursor: pointer;
}

button:hover {
background: #0056b3;
}

```

Upload a text file and display its contents

In this example, we will use the first `<section>`... element in the HTML code above. I will show this section element again here just for context and clarity.

```

<section>
  <h2>Upload a Text File and Display Its Content</h2>
  <input type="file" id="fileInput" />

```

```
<pre id="output">Your file content will appear here...</pre>
</section>
```

First we allow the user to upload a file using the file input field with the id attribute of “fileInput”. As soon as the file is uploaded, we will extract its contents and insert them as the value of the `<pre>` tag with the id attribute of “output”.

Create a text file to test with. Give it any name eg test.txt. Type into this text file the following text, just so we can test the ability to read its text content using JavaScript.

test.txt

This is a test, this is a test, first line This is a test, this is a test, this is a test This is a test, this is a test, this is a test This is a test, this is a test, last line

Next, place the following code in your JavaScript file eg index.js which should be in the same directory as the index.html file of your project.

```
// ----- 1) Upload a text file
// and display its contents -----
const fileInput = document.getElementById('fileInput');
const output = document.getElementById('output');

fileInput.addEventListener('change', function () {
    const file = this.files[0];
    if (file && file.type === 'text/plain') {
        const reader = new FileReader();
        reader.onload = function (e) {
            output.textContent = e.target.result;
        };

        reader.readAsText(file);
    } else {
        output.textContent = 'Please upload a plain text (.txt) file.';
    }
});
```

Here is the full explanation of the code:

-We use an `<input type="file">` to let the user select a file from their device. Here is how we detect a file upload event on a file input field:

```
fileInput.addEventListener('change', function () {
    // any code in this block will
    // run when a file is uploaded
});
```

- Once that upload event fires (occurs), we capture the uploaded file from the `.files` property of the input field element object. Remember that this is the stand-

ard way to retrieve uploaded files. Refer back to Chapter 19 (Forms and Email), explained there in depth.

- Optionally, we apply some validation to ensure that the file uploaded is of the type we are expecting (text). If not, we display a message telling the user that only a text file is accepted.

```
if (file && file.type === 'text/plain') {  
    // ...  
}
```

- The `FileReader` object is used to read the file as plain text with `readAsText(file)`. This is an asynchronous call, and it is the thing that triggers the action to read the uploaded file, which when complete will trigger this `.onload` event here:

```
reader.onload = function (e) {  
    output.textContent = e.target.result;  
};
```

Note that the code within this block is only run when the file read has been completed. It is therefore very necessary to wrap that code within that event (`onload()`) block, so that being an asynchronous call, it only runs when the file is successfully read. Once it is read, the code within that block processes the contents of the file. In this case, we extract the file's contents— which is stored in `e.target.result`, and assign it to the value of the

`<pre>` tag in the DOM. -When reading is complete, the file content is displayed inside a `<pre>` block using `textContent`. - This is a good demonstration of how to load and show file content without uploading to a server—all done in the browser. ! [Figure 18.1 — Reading a file takes time, so the work happens in a callback] (images/ch18-fig-01-filereader-flow.png) - Figure 18.1 — Reading a file takes time, so the work happens in a callback* ## Drag and drop a file, modify and download it This demo lets a user drag a text file, then the contents of the file will be read, modified by appending a line of text to it, and bundled into a downloadable separate file which can then be downloaded by the user. To download the modified (new) file, the user has to click on the 'Download Modified File' button and the file will be downloaded to their machine. The uploaded file has to be a text file, otherwise it will not work, as we have a check (validation) in place making sure it only does what it is meant to do if the dragged-and-dropped file is of type 'text/plain' (a text file). The validation looks like this: `` if (file && file.type === 'text/plain') { // ... } `` As seen in the example HTML code above, here is the section element that holds the HTML code for this exercise: `<section> <h2>Drag & Drop a Text File, Modify and Download</h2> <div id="dropZone">Drop a .txt file here</div> <button id="downloadBtn" style="display:none;"> Download Modified File</button> <pre id="modifiedContent"> Modified file content will show here... </pre>`

</section>

Here is the full JavaScript code:

```
// ----- 2) Drag and drop a file,
// modify and download it -----
const dropZone = document.getElementById('dropZone');
const modifiedContent = document.getElementById('modifiedContent');
const downloadBtn = document.getElementById('downloadBtn');
let modifiedText = '';

dropZone.addEventListener('dragover', (e) => {
    e.preventDefault();
    dropZone.style.borderColor = 'green';
});

dropZone.addEventListener('dragleave', () => {
    dropZone.style.borderColor = '#888';
});

dropZone.addEventListener('drop', (e) => {
    e.preventDefault();
    dropZone.style.borderColor = '#888';
    const file = e.dataTransfer.files[0];

    if (file && file.type === 'text/plain') {
        const reader = new FileReader();
        reader.onload = function (e) {
            modifiedText = e.target.result + '\n\n--- File
Modified! ---';
            modifiedContent.textContent = modifiedText;
            downloadBtn.style.display = 'inline-block';
        };

        reader.readAsText(file);
    }
    else
    {
        alert("Please choose a text file!");
        return false;
    }
});

downloadBtn.addEventListener('click', () => {
    const blob = new Blob([modifiedText], { type: 'text/plain' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'modified-file.txt';
    a.click();
});
```



```
// start the download before we
    // throw the URL away - revoking
    // on the next line can cancel it
    setTimeout(() => URL.revokeObjectURL(url), 1000);
});
```

Here is an explanation of the code:

- The div with the id attribute of 'dropZone' is where the user is expected to drop the dragged files. So to handle that, we start by selecting this div and storing it in a variable dropZone.
- We also select the <pre> element of the id attribute 'modifiedContent' and the button of the id 'downloadBtn' and store them in variables of the same names. The 'modifiedContent' div is the spot where we intend to insert the content read from the file, while the 'downloadBtn' is the button the user clicks to download the contents of the file.
- Next, we go ahead and set a couple of event listeners on the 'dropZone' div to handle the drag and drop, and this will teach you how dragging and dropping works in JavaScript. For it to work, we need exactly three event listeners, and they are:
 - 'dragover'
 - 'dragleave'
 - 'drop'

This is how we add the event listeners:

```
dropZone.addEventListener('dragover', (e) => {
    // ...
});

dropZone.addEventListener('dragleave', () => {
    // ...
});

dropZone.addEventListener('drop', (e) => {
    // ...
});
```

I will now explain what we do in the event blocks, which each contain the code that will run when the corresponding events fire. You will find that it is so easy to understand, especially because the names of the events actually mean what they are meant to do. So, let's start with when the 'dragover' event fires. This will happen when the user has dragged an item over the border of the 'dropZone' div. This should happen before the item is released (dropped). What we do in the code block to react to this event is simple, we set the color of the border to green. This is just

to mark the zone to give the user a visual guide of the boundaries in which they are expected to drop the item.

In the code block of the 'dragleave' event listener, it is simple as well. This event happens when the user moves with their mouse out of the dropZone area whilst still dragging the item. By this, they are essentially leaving the zone, hence the name of the event is 'dragleave'. At this point, they have not let go of the item yet. What the code does here is revert the border colour of the dropZone to what it was before it was turned to green. Again, this is to give the user a visual guide of the drop zone which they have now left.

The 'drop' event happens when the user lets go of (drops) the item. It is in this code block that most of the work happens. Here we proceed to get the file, and this time we are getting the dragged-and-dropped file instead of a file input.

When a file is dragged into the #dropZone area, we use JavaScript to prevent the default browser behaviour and read the file. Next, we revert the border colour of the dropZone from green to what it was before.

- Next, we get the file, but notice carefully an important difference with where we are getting the file from. In other examples when we retrieved an uploaded file, we got it from the files property of the file input element, but with drag and drop, it is different. This time we access the dropped file from the files property of the 'dataTransfer' property of the current event (drop event). We do it like so:

```
e.dataTransfer.files[0];
```

- The rest of the code works as most using the File Reader API will work. The file's content is read using FileReader, just like before. Basically, we read the contents of the file by calling the readAsText() method on the reader and passing it the file reference.

```
reader.readAsText(file);
```

This method call will asynchronously read the file and trigger the onload event on the reader when it is done

```
reader.onload = function (e) {  
    // ...  
}
```

Once this onload event fires, we know the file has been read, so in its block is a function that responds to that. In this function, we grab the content of the file, which is in e.target.result and store it in a variable 'modifiedText'.

- As a basic modification, we append some custom text: '\n\n--- File Modified! ---' to it using the concatenation operator (+) and assign that modified text content

(in `modifiedText`) to the `<pre>` element identified by the `id` attribute: `modifiedContent`.

- The user can then download the modified text by clicking on the “Download Modified File” button. This is because we have a click event listener on that button to trigger the download.

```
downloadBtn.addEventListener('click', () => {  
  // ...  
})
```

Once the download button is clicked, the code in the function block passed as the second argument of the click event listener looks like this:

```
downloadBtn.addEventListener('click', () => {  
  const blob = new Blob(  
    [modifiedText], { type: 'text/plain' }  
  );  
  const url = URL.createObjectURL(blob);  
  const a = document.createElement('a');  
  a.href = url;  
  a.download = 'modified-file.txt';  
  a.click();  
  
  // give the browser a moment to  
  // start the download before we  
  // throw the URL away - revoking  
  // on the next line can cancel it  
  setTimeout(() => URL.revokeObjectURL(url), 1000);  
});
```

First, to prepare the contents to be converted into a downloadable file in the browser, we create a blob object from the contents.

```
const blob = new Blob([modifiedText], { type: 'text/plain' });
```

The constructor of the Blob object takes two arguments, the content for the download file, and information about what type of file it is. We pass the file type information as an object literal like so:

```
{ type: 'text/plain' }
```

This second argument is very important, because it needs to know what type of file it is creating. In this case, we want a text file. We then use the `createObjectURL()` method of the URL API to create a link to the file Blob has created in memory.

- Normally, you can use an existing button in your browser, or create one in code (dynamically) and insert it into the browser for users to use to download the created file via a click event. In this example however, we do it differently. Because

it is a drag-and-drop file which we wish to immediately download, we have no link or button in the browser for the user to click. There is no need for that. What we do is dynamically create an anchor (link) element, and trigger a click event which will activate the download action by visiting the link defined in the href attribute of the anchor element. Here is the code:

```
const a = document.createElement('a');
a.href = url;
a.download = 'modified-file.txt';
a.click();
```

The first line creates the anchor tag. Next, we assign the URL reference of the generated file in memory to the href attribute of the anchor element. We specify our desired name for the file to be downloaded as 'modified-file.txt'. Lastly, we trigger a click event by calling the click() method on the anchor element.

```
a.click();
```

This is amazing, to see that in JavaScript, you can dynamically simulate, so-to-speak, a click on a link or button, or any element for that matter; all in memory without that element ever existing in the browser.

- At the end of the code in the click event listener, after the file is supposed to have been downloaded, we close with a call to the revokeObjectURL() method of the URL API. This will discard the temporal URL reference to the file downloaded file that was created by URL.createObjectURL(). This is important to prevent memory leaks.
- Alternatively, instead of modifying the content of the read file in code, we could also easily allow the user to do that themselves in the browser by using a textarea input element instead of a <pre> element which is read-only. This means, the original content of the file, once read, could be inserted as the value of a textarea field on the web page. Textarea fields as we know, allow user input unless specifically made read-only by adding a disabled attribute to them. The user can then add or modify the content, and in JavaScript we would retrieve that content from the textarea field from its value property, to be used as the content of the generated download file.

The example has shown you how JavaScript can read, manipulate, and regenerate a file in the browser using a Blob. We have seen how a Blob is the key to generating a downloadable file from random or loose content in the browser. We talked about the concept of Blobs in Chapter 15 (DOM/ Frontend UI), under the sub-heading “Manipulating the URL with the URL API”, but now we got to see it in action. Once more, a BLOB (Binary Large Object) is like a container in which you wrap up stuff that might not have a defined size which you want to transmit eg by uploading

to a remote server, allowing it to be downloaded etc. You put it in this container to make it easier for JavaScript to manage them together as one object. This BLOB or container could contain text, images, or even video data. That is essentially what a BLOB is. It lets you store and treat raw data (text, images, anything!) like a file. You create it in JavaScript using the built-in Blob class like so:

```
const blob = new Blob([data], { type: "text/plain" })
```

Notice how you pass as the first argument, the data you want to put in the BLOB in an array, then pass in a second argument which is information about the type of the file you wish to create. In the code snippet above, we want to create a text file. Next, you can generate a download link for the file to be created using `URL.createObjectURL(blob)`. For example:

```
const url = URL.createObjectURL(blob);
```

Generate CSV from a JavaScript Array

This demo takes a JavaScript array of arrays and turns it into a CSV format.

```
const data = [
  ['Name', 'Age', 'Country'],
  ['Alice', '25', 'USA'],
  ['Bob', '30', 'UK'],
  ['Charlie', '28', 'Canada']
];

document.getElementById('generateCSV')
  .addEventListener('click', () => {
    const csvContent = data.map(row => row.join(',')).join('\n');
    const blob = new Blob([csvContent], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'data.csv';
    a.click();

    // give the browser a moment to
    // start the download before we
    // throw the URL away - revoking
    // on the next line can cancel it
    setTimeout(() => URL.revokeObjectURL(url), 1000);
  });
```

Here is how this code works. We wrap it all in an event listener, which listens for a click on the 'Download CSV' button. This makes sense because nothing needs to be done otherwise. Once that click event fires (occurs) we proceed by going

through all the elements in the data array, and joining each row with commas (,), ending each line (row) with newline character (\n) to form proper CSV text.

This ends up with each element (row) in the data array being written as a line of text in the CSV file to be created.

A Blob is created to hold the text as a downloadable file. Using `URL.createObjectURL()`, a temporary download link is created, and a file is downloaded when the button is clicked.

This example is meant to teach you how to generate files in the browser from scratch—useful for exporting user data or reports.

Read data from a CSV file and inject into the DOM

This code lets a user select the CSV file, reads its content using `FileReader`, and displays it as an HTML table. The example is as follows:

a) Create a CSV file in the root of your local project folder. Name the file anything you want, like ‘employees’. Add this dummy people data in it:

Name,Age,Email,Country Alice,25,alice@example.com,USA Bob,30,bob@example.com,UK Charlie,28,charlie@example.com,Canada

b) Have a stylesheet e.g `index.css` with this code

```
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
  padding: 8px;
}
```

c) Have an HTML file (e.g. `index.html`) with this code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>The JavaScript Blueprint</title>
    <link rel="stylesheet" href="index.css" type="text/css">
  </head>
  <body>

    <h1>Read CSV File and Display as Table</h1>
    <input type="file" id="csvFile" accept=".csv">
    <br><br>
    <div id="tableContainer"></div>

    <script type="module" src="index.js"></script>
  </body>
</html>
```

d) Have a JavaScript file (e.g. index.js) with this code in it:

```
document.getElementById('csvFile').addEventListener('change',
function(event) {
    const file = event.target.files[0];
    if (!file) return;

    const reader = new FileReader();
    reader.onload = function(e) {
        const text = e.target.result;
        displayCSVAsTable(text);
    };
    reader.readAsText(file);
});

function displayCSVAsTable(csvText) {
    const rows = csvText.trim().split('\n');
    const table = document.createElement('table');

    rows.forEach((row, rowIndex) => {
        const tr = document.createElement('tr');
        const cells = row.split(',');

        cells.forEach(cell => {
            const td = rowIndex === 0 ? document.createElement('th') :
document.createElement('td');
            td.textContent = cell;
            tr.appendChild(td);
        });

        table.appendChild(tr);
    });

    // Clear old tables
    document.getElementById('tableContainer').innerHTML = '';
    document.getElementById('tableContainer').appendChild(table);
}
```

You should be very familiar with this FileReader code flow by now. Here is the brief run-down of it:

- A user clicks to upload a file.
- `FileReader.readAsText()` reads the file content as plain text.
- The CSV data is split by every new line (`\n`) to get data for the rows, and put in an array, then the text in each row (line) is split by commas (`,`) to extract the data for the individual columns.
- A dynamic HTML table is created based on the file contents.

Reading a file's raw binary data with `readAsArrayBuffer()`

We have already used two methods for turning file data into something downloadable, and it is worth being clear about where each one lives, because neither belongs to the File Reader API. `createObjectURL()` is a method of the URL object and is suitable for handling blobs, while `toDataURL()` is a method of a canvas element and is suitable for canvas images. The File Reader API's own methods are the `readAs...` family we listed earlier, and there is one of those we have not talked about yet: `readAsArrayBuffer()`. The `readAsArrayBuffer()` method of the FileReader API is used to read a file's raw binary data. It reads the contents of a file (or blob) and returns it as an `ArrayBuffer`, which is a low-level representation of binary data. It can be used in different scenarios. Here are scenarios when you may need to use the `readAsArrayBuffer()` method:

a) When you're working with binary files, such as:

- Images
- Videos
- PDFs
- Custom binary formats
- Audio files (e.g., .mp3, .wav)

b) When you want to pass binary data to another API, like:

- WebSockets (to send binary data in real time)
- `fetch()/XMLHttpRequest` for uploading
- `Blob()` constructor for creating new binary blobs
- Encryption/Decryption libraries

c) When you want to manipulate or inspect the binary contents using `DataView`, `Uint8Array`, etc.

All this information is about being able to read into a file down to the binary level. But if you are like me, it does not mean much. You prefer to see it in practice, and that's the best way to learn. Let's look at an example of `readAsArrayBuffer()` being used.

Validating File Signatures (a.k.a. Magic Numbers)

Imagine you want to ensure that an uploaded file is really a JPEG or PNG, and not just renamed with a .jpg or .png extension. This is important because people can rename malicious files to trick file uploaders. Reading the first few bytes of the file (its magic number) can help detect fakes. This example therefore, is a validation solution. The following example code will detect if a file is truly a PNG or JPEG:

```
<input type="file" id="fileInput" />

<script>
    document.getElementById("fileInput")
    .addEventListener(
        "change", function () {
            const file = this.files[0];
            const reader = new FileReader();

            reader.onload = function (e) {
                const arrayBuffer = e.target.result;
                const bytes = new Uint8Array(arrayBuffer);

                // Check the PNG magic number.
                // Its full signature is
                // eight bytes, but the first four identify it.
                const isPng = bytes[0] === 0x89 &&
                    bytes[1] === 0x50 &&
                    bytes[2] === 0x4E &&
                    bytes[3] === 0x47;

                // Check JPEG magic number (first 2 bytes)
                const isJpeg = bytes[0] === 0xFF && bytes[1] === 0xD8;

                if (isPng) {
                    alert("This is a valid PNG image.");
                } else if (isJpeg) {
                    alert("This is a valid JPEG image.");
                } else {
                    alert("This file is NOT a valid PNG or JPEG.");
                }
            };

            reader.readAsArrayBuffer(file);
        });
</script>
```

Practically, this is useful in the following ways:

- Security: It helps prevent file spoofing (e.g. .exe renamed as .jpg).
- Reliability: It confirms file type without relying on file extensions or MIME types.

Hopefully, this example teaches you how to programmatically inspect the raw binary data of a file. This is a great skill to have as a developer learning to handle media or binary formats.

Sending binary file content over WebSocket

We will talk some more about WebSockets in Chapter 22 (Extensions), but here is what it is. A WebSocket is a two-way communication channel between the browser and the server. Unlike regular HTTP requests (which are one-way), WebSockets stay open, allowing both sides to send and receive data continuously. This makes WebSockets great for things like chat apps, multiplayer games, live dashboards, and — yes — even real-time file sharing. If you are building a collaborative file-sharing app or a peer-to-peer (computer-to-computer) media uploader, and you want to send a file chunk by chunk over a WebSocket—maybe to avoid HTTP overhead or allow real-time transmission. You can do that using the `readAs-ArrayBuffer()` method of the File Reader API. Why is it better to send a large file in chunks rather than uploading a whole file at once? Normally, when uploading a file to a remote server, you would use a regular

```
<input type="file">
```

input element, and AJAX, via the `fetch` API or `XMLHttpRequest`, to send it to the remote server. Then you would do the following:

- Select the entire file
- Send it all at once to the server
- Wait until it's 100% uploaded to do anything with it

This works just fine for small files. But for large files (like videos, audio, or big documents), sending the entire file at once can cause problems. Here's why chunking can be useful—especially over a WebSocket:

- **Real-time Streaming or Processing** This will be great for video platforms or live audio transcription. This means that instead of waiting for the whole file, the server can start decoding and showing the video as soon as the first chunks arrive. That's how media platforms like Netflix and YouTube let you start watching before the entire video is downloaded.
- **Avoiding HTTP Overhead** HTTP overhead refers to the many bits of information that need to be handled during each HTTP request, such as a file upload request. A file upload, as all HTTP requests, involves sending request headers back and forth between your computer and the server, with repeated connections being opened and closed for each request-response, and extra steps take-

n per file being uploaded. Using a WebSocket on the other hand has the following advantages:

- Opens one persistent connection
- Has less overhead
- Sends data as-is with minimal packaging

This results in faster communication, especially for apps that do a lot of back-and-forth (like collaborative tools).

- Using a WebSocket to send chunks provides more fine-grained control over the file upload process, such as giving you the ability to pause, resume the upload, or retry if something goes wrong. These can be useful if you are building a file upload manager app, or if you are working with slow or unstable networks which can stall mid way in the upload process. By sending chunks one at a time:
 - If one chunk fails, you can retry just that chunk
 - You can pause and resume the upload without starting over. Without using a WebSocket, you will be trying to upload the full file at once, which means if anything breaks, the user has to re-upload the entire file again
- Using a WebSocket to send chunks is ideal when building real-time peer-to-peer (P2P) file transfer apps. Examples of such apps can be Bluetooth/WiFi direct transfers, file-sharing without a central server. With such applications, sending chunks lets two users exchange data bit by bit, showing progress or reacting in real time—like a chat app for files.

When working with large files like videos, images, or even documents, JavaScript gives you a powerful tool called `readAsArrayBuffer()` from the `FileReader` API. This method reads a file and gives you its raw binary data, which is perfect for sending through a WebSocket connection. But what does that really mean? And how is it different from something like YouTube, where videos just seem to play instantly without being fully downloaded first? Let's break this down step-by-step. The following is an example of sending a Video File in Binary Chunks over WebSocket. We want to send a big file (like a video) from the browser to a server using a WebSocket bit by bit (called chunks), instead of sending it all at once. This allows better control and smoother progress, especially with large files.

HTML code

```
<input type="file" id="videoUploader">
```

JavaScript code

```
const socket = new WebSocket("ws://localhost:3000");

socket.addEventListener("open", () => {

    document.getElementById("videoUploader")
    .addEventListener("change", (e) => {
        const file = e.target.files[0];
        const chunkSize = 64 * 1024;
        // 64KB per chunk
        let offset = 0;

        const reader = new FileReader();

        reader.onload = function(event) {
            if (socket.readyState === WebSocket.OPEN) {
                // Send binary data
                socket.send(event.target.result);
                offset += chunkSize;

                if (offset < file.size) {
                    readNextChunk();
                } else {
                    console.log("File upload complete.");
                }
            }
        };

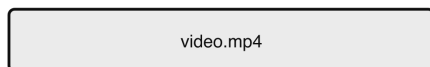
        function readNextChunk() {
            const slice = file.slice(offset, offset + chunkSize);
            reader.readAsArrayBuffer(slice);
        }

        readNextChunk();
    });
});
```

This code reads the file chunk by chunk and sends each piece over the WebSocket connection as binary data.

Sending a large file in chunks

One 200,000-byte file



sliced at 65,536 bytes each



The loop: slice from offset, read it, send it, advance offset, repeat until offset reaches file.size.

Why bother? If the connection drops you resend one chunk, not the whole file. You can pause and resume. And the server can start work on the first chunks while the rest are still on their way.

- Figure 18.2 — Sending a large file in chunks* Here is an in-depth explanation of how it works:

We start by creating a WebSocket connection to the server running at localhost (your computer) on port 3000. This sets up a 2-way chat pipe between your browser and the server.

```
const socket = new WebSocket("ws://localhost:3000");
```

Next, we wait until the WebSocket connection is fully open—like waiting for a phone call to connect before talking.

```
socket.addEventListener("open", () => {  
  // ...  
});
```

We listen for when the user selects a file (like a video) using a file input (`<input type="file" id="videoUploader">`). When they do, the code inside this function runs.

```
document.getElementById("videoUploader")  
  .addEventListener("change", (e) => {  
    // ...  
  });
```

We get the first file they selected, and store it in a variable called `file`.

```
const file = e.target.files[0];
```

We decide to slice the file into pieces (chunks) that are 64 kilobytes (KB) each. That's a reasonable size—not too big, not too small.

```
const chunkSize = 64 * 1024;  
// 64KB per chunk
```

We start from the beginning of the file. As we send chunks, this number keeps track of where we are.

```
let offset = 0;
```

We grab the `FileReader`—the specialised tool for reading files in our browser.

```
const reader = new FileReader();
```

Next, we set up what happens every time the `FileReader` finishes reading a chunk. When it's done, the code inside here runs.

```
reader.onload = function(event) {  
  // ...  
}
```

We make sure the `WebSocket` connection is still open before trying to send data.

```
if (socket.readyState === WebSocket.OPEN) {  
  // ...  
}
```

We send the chunk of binary data (from the file) over the `WebSocket` to the server.

```
socket.send(event.target.result);
```

We move our pointer forward by the size of the chunk. So we're ready to send the next piece.

```
offset += chunkSize;
```

If we haven't reached the end of the file yet, we call `readNextChunk()` to read and send the next piece. This is what creates the loop effect, so that the chunks of the file are being read until the very last one. If we have reached the end of the file, we say the upload is done by writing the text "File upload complete" to the console.

```
if (offset < file.size) {  
  readNextChunk();  
} else {  
  console.log("File upload complete.");  
}
```

Here is the custom function `readNextChunk()` which will keep reading the file until all the chunks are read. It does so by slicing the next piece of the file—from where we left off (`offset`) to the next chunk, each time it is called. While doing so, we tell the `FileReader` to read each of the slices as an `ArrayBuffer` (a binary format we can send).

We start the whole process by reading the first chunk. This kicks off the reading and sending loop.

```
readNextChunk();
```

What happens on the WebSockets server side

In the above example, we start with this code to create a WebSocket connection to the server running at localhost (your computer) on port 3000. This sets up a 2-way chat pipe between your browser and the server.

```
const socket = new WebSocket("ws://localhost:3000");
```

But what does this actually mean, and what is the meaning of "ws://localhost:3000"?

To actually upload a file using WebSocket (or even handle any server logic), you need to run a real backend server, such as:

- Node.js with ws, express, or socket.io
- PHP (though PHP typically uses HTTP, not WebSocket)
- Python, Go, or any backend tech that supports WebSockets

This means first of all that you must have a WebSocket server up and running, whether locally or remotely on the internet. This is needed for this or any WebSocket connection application to work. A WebSocket server is a server that supports WebSockets, as in the examples given above, which are servers running on Node.js, Python, Go etc. The string "ws://localhost:3000" passed to `WebSocket()` above means your WebSocket server is running on localhost (your local machine) and listening for WebSocket connections on port 3000.

localhost - means your own computer (not the internet).

3000 - refers to the port number where the WebSocket server should be listening for this to work. Change that port number to the port number your WebSocket server is listening.

If you are using localhost (your machine), as is in this example, it is very common to use a Node.js backend—because Node has full support for WebSockets, file handling, and custom server logic. The above example assumes you are doing that—running a Node.js server locally.

Do not confuse that with the Live Server in VS Code which I guided you to set up so you can test your JavaScript code. The Live Server in VS Code is a static file server, which means it only serves HTML, CSS, and JavaScript. It does not support WebSocket connections out of the box. It also does not handle server-side logic like receiving file uploads or processing binary data. So you cannot upload a file to Live Server the way the WebSocket example shows.

To test this feature locally; let us write code to set up an example Node.js server. Though Node.js is beyond the scope of this book, I will give the example code on the server-side just because it is important for you to see the full communication process that makes this feature work. Since we are on the topic of file management, it only makes sense for you to learn what happens on the backend, when you send a file over via an upload. I choose a Node.js server because it is the easiest to set up on your local machine, and is a very common and popular solution for JavaScript development. In your already existing web project, create a JavaScript file server.js – for the Node.js backend code

HTML code

We will use the same HTML code as in the above example that just has the file input field.

JavaScript code

We will maintain the same example JavaScript code above that captures the uploaded file and makes a WebSocket request to the Node.js server

Install Node.js, and the WebSocket library

- Install Node.js if you don't have it installed on your computer already. Node.js is a tool that lets you run JavaScript outside the browser-on your computer like a regular program. It also comes with a super useful tool called npm (Node Package Manager), which helps you install extra features and tools. Follow these steps to download Node.js:
 - Go to the official Node.js website: <https://nodejs.org>
 - Choose the LTS version and click to download it.
 - Next, open the downloaded file and go through the installer steps: Just keep clicking Next until you see Finish. Keep all default settings unless you know what you're doing.
 - Check If It Worked. After installation, you can check if Node.js was installed correctly:
 - On Windows: Press Windows + R, type cmd, and press Enter
 - On Mac: Open the Terminal app
 - On Linux: Use your terminal or shell
 - Type the following and press Enter:

```
node -v
```


You should see something like:

v20.12.2

That means Node.js is installed. Then check for npm:

npm -v

You should see a version number for that too. Now that Node.js is installed, you can run JavaScript files in your terminal using node, create backend servers, and use npm to install tools like express, ws, nodemon and more.

- Install the WebSocket library like so: Open a terminal, then navigate to your web project folder and run:

npm init -y npm install ws

server.js

This is the file that will contain the Node.js server-side code, that will open a WebSocket port, listen for in-coming requests, then receive and save the uploaded file. Here is the code for it:

```
const fs = require('fs');
const path = require('path');
const WebSocket = require('ws');

const wss = new WebSocket.Server({ port: 3000 });

console.log(
  "WebSocket server is running on ws://localhost:3000"
);

wss.on('connection', (ws) => {
  console.log("New client connected");

  const uploadPath = path.join(
    __dirname, 'uploaded_video.mp4');
  const writeStream = fs.createWriteStream(uploadPath);

  ws.on('message', (data) => {
    // Write each chunk of binary data to the file
    writeStream.write(data);
  });

  ws.on('close', () => {
    writeStream.end();
    console.log("Upload complete, connection closed");
  });
});
```

```
writeStream.end();
    });
});
```

To start the server, in your terminal and while in your web project directory where the 'server.js' file is, run the following code:

```
node server.js
```

Next, test the upload now by uploading a video or any large file.

After the upload, you should see a new file "uploaded_video.mp4" created in your folder, which contains the file/video you uploaded. Let us discuss a little about how the Node.js code (server-side) in server.js works.

- The "message" keyword on the line: `ws.on('message', (data) => { ... });` is not a custom word. It must be exactly as it is given. This is because "message" is a standard, built-in WebSocket event. The fact is, in Node.js (and in the browser too), WebSockets have a few standard event names you must use exactly — they are not customisable. Here is a breakdown of those event names:

Event name	Meaning
"message"	A new message (data) was received
"open"	The connection has been successfully opened
"close"	The connection has closed
"error"	Something went wrong

So when you write:

```
ws.on('message', (data) => {
    // handle incoming data
});
```

You're telling the WebSocket server to run this code whenever a message comes in from the client. If you changed "message" to something else like "fileUpload", it won't work — the server won't recognise it.

- The "close" event in the line `ws.on('close', ...)` can happen on the server when the client disconnects, and can also happen on the client when the server disconnects. It's a two-way event—either side can trigger it, and the other side will also get notified. For example:
 - If the user closes their browser tab, the client disconnects, and the server receives a "close" event.
 - If the server shuts down or manually calls `ws.close()`, then the client will receive the "close" event too.

-Think of WebSocket as a phone call. * message = when someone speaks. * close = when someone hangs up. * error = when the line has a problem. These events let you respond to those moments.

WebSockets with Socket.IO

What if you wish to make a WebSocket request to Socket.IO instead of your local machine, what would this line look like?

```
const socket = new WebSocket("ws://localhost:3000");
```

If you're switching from a vanilla WebSocket server (like the one you wrote using Node.js and the ws package) to Socket.IO, the connection process changes slightly — because Socket.IO isn't just a plain WebSocket. It uses its own protocol on top of WebSockets (with fallbacks like long polling), and it requires a dedicated Socket.IO client library on the frontend. Instead of:

```
const socket = new WebSocket("ws://localhost:3000");
```

There are two (2) ways you could do it. Here they are:

- 1. You can reference the remote Socket.io client via their Content Delivery Network (CDN) to create a client without having the code on your local machine. Do it like so:

HTML code (to load the Socket.IO client via CDN)

```
<script src="https://cdn.socket.io/4.7.2/socket.io.min.js">
</script>
```

JavaScript code

```
// Or io("http://localhost:3000")
const socket = io("https://your-server.com");
```

The value (string) you pass to `io()` will be location of your WebSocket server, which can be on your local machine (eg `server.js`), in which case you would pass in `"http://localhost:3000"`. Note that this is different from `"ws://localhost:3000"`.

- 2. Alternatively, you can use the Node Package Manager (NPM) tool which you should have installed on your machine together with Node.js—the same one you should have used to install the WebSocket (ws) library. Use NPM to install the socket.io client onto your local machine like so, via your terminal:

```
npm install socket.io-client
```

Then in your JavaScript code

```
import { io } from "socket.io-client";

// or io("http://localhost:3000")
const socket = io("https://your-server.com");
```

Again, the value (string) you pass to `io()` will be the location of your WebSocket server, which can be on your local machine (eg `server.js`), in which case you would pass in `"http://localhost:3000"`. Note that this is different from `"ws://localhost:3000"`.

Conclusion

In conclusion, when you are talking to a standard WebSocket server, you create the client end of the connection like so:

```
new WebSocket("ws://...")
```

Whereas, for communicating with a Socket.IO server, you create that client like so:

```
io("http://...")
```

Note that both of those lines run in the BROWSER and create a client. The server is the separate program you started with `node server.js`.

If your server is on another machine or hosted online, create a WebSocket client like so:

```
// or any deployed server with socket.io
const socket = io("https://your-cool-app.vercel.app");
```

Make sure CORS is enabled on your server. CORS stands for Cross-Origin Resource Sharing. It's a security rule built into web browsers that controls which websites are allowed to talk to each other. For example: If your webpage is running on <http://mywebsite.com>, and it tries to get data from <http://anotherwebsite.com>, the browser will block it unless the other website says, "Hey, it's okay for mywebsite.com to access me." So, if you're using WebSockets or APIs across different sites or ports, you'll need to enable CORS on the server so the browser allows the connection. This is done just on the server side.

Enabling CORS in a Node.js + Express Server

I know I said Node.js is beyond the scope of this book. But I will like to provide you with a sample server-side CORS setup just for Node.js. My aim is to provide

you with handy solutions to most challenges you are likely to face, especially something like CORS which is a vital skill to have. Luckily, setting up CORS is super easy, especially if you're using Node.js with Express. You can do it by following these steps:

- Install the cors package via npm

npm install cors

- Use it in your server file (server.js in our example). Here's an example server (server.js) with CORS enabled:

```
const express = require('express');
const http = require('http');
const WebSocket = require('ws');
const cors = require('cors');

const app = express();
// Allow all origins by default
app.use(cors());

const server = http.createServer(app);
const wss = new WebSocket.Server({ server });

wss.on('connection', (ws) => {
  console.log('A client connected via WebSocket.');
```

```
  ws.on('message', (message) => {
    console.log('Received:', message);
    // You can save chunks here
  });

  ws.on('close', () => {
    console.log('Client disconnected.');
```

```
  });
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
```

```
});
```

Notice how we import the cors package and then use it like so:

```
const app = express();
app.use(cors());
```

If you want to allow only a specific origin, just pass that origin (domain) as an object literal with a key that says 'origin' to cors() like so:

```
app.use(cors({
  origin: 'http://your-frontend-domain.com'
}));
```

Here is a bonus tip: If the browser ever blocks a request and says something like “CORS policy error”, it’s usually because the server hasn’t allowed your frontend to access it. Just use the cors package and you’re good to go. You may have noticed that when we set up the WebSocket server earlier, we never had a line like this:

```
server.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

This is because, earlier, we set up only a WebSocket server and not an HTTP server. So we just did:

```
const wss = new WebSocket.Server({ port: 3000 });
```

We didn’t use `http.createServer()` or Express. That style runs WebSocket-only, without a regular HTTP server. That’s why there was no need for a `server.listen()` in that case—the WebSocket server handled its own listening.

This time around, when adding CORS, we needed an HTTP server for that, and therefore we absolutely needed this line:

```
server.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

That line starts the HTTP server, which is required because we attached the WebSocket server to it like this:

```
const server = http.createServer(app);  
const wss = new WebSocket.Server({ server });
```

Without the `server.listen(...)` line, no server will be started.

WebSockets vs video streaming

When sending large files, for example video files via a WebSocket, that is not the same thing as real video streaming like Netflix or YouTube. Once uploaded to the (remote) server, the job of WebSocket is done. Though you can start processing or displaying the file on the remote server as it arrives, how you manage this playback process is a separate thing. You would need to build your own video player logic to play the data in chunks. Real streaming services offered by platforms like Netflix or YouTube use smart streaming protocols like HLS (HTTP Live Streaming) and MPEG-DASH (Dynamic Adaptive Streaming over HTTP). These streaming protocols offer the following:

- Chop the video into small, downloadable segments to pass down to the video player via the `<video>` HTML element of a web page.

- Allow the video player on a web page to request for only the needed segments
- Enable adaptive quality (e.g., switch from 1080p to 720p if internet is slow)

This makes video smooth, fast, and user-friendly — which WebSocket by itself doesn't provide. So, in conclusion, WebSockets can be used to stream media, but not in the traditional sense. It is not optimised for that. While you can stream binary chunks of a video or audio file over WebSocket, you'll need to build your own play-back logic. Real video streaming uses smarter protocols like HLS or MPEG-DASH, which streaming services like YouTube and Netflix use. Here is when you can use WebSockets for media:

- When you are building your own media uploader or experimental video player
- When building a peer-to-peer (P2P) file sharing app
- When building real-time collaborative apps (like drawing tools)
- When sending snapshots from a camera feed in real-time

Here is when not to use WebSockets with media:

- Not suitable for full video streaming to public users
- Not for building a Netflix/YouTube clone (use HLS or MPEG-DASH instead).

Sending files using WebSockets gives you more power and control — you decide how the file is read, uploaded, and processed. This is great for learning how data flows in real time. But remember, if you want your users to press play and instantly enjoy videos, it's better to stick with streaming protocols built for that job. Tip: You can still use what you've learned here to build cool tools — like live editors, custom dashboards, or unique file-transfer systems!

Previewing a PDF file

This example allows the user to select a .pdf file to upload, after which it reads it using `readAsArrayBuffer()`, then creates a temporary URL so the PDF file can be viewed directly in the browser.

HTML code

```
<input type="file" id="pdfUploader" accept="application/pdf" />
<br><br>
<iframe id="pdfPreview" width="100%" height="500px"></iframe>
```

JavaScript code

```
document.getElementById("pdfUploader")
.addEventListener("change", function (e) {
    const file = e.target.files[0];

    if (file && file.type === "application/pdf") {
        const reader = new FileReader();

        reader.onload = function (event) {
            const arrayBuffer = event.target.result;

            // Create a Blob from the ArrayBuffer
            const blob = new Blob(
                [arrayBuffer], { type: "application/pdf" }
            );

            // Create a temporary URL for the blob
            const url = URL.createObjectURL(blob);

            // Set the URL in an <iframe> to preview it
            document.getElementById("pdfPreview").src = url;
        };

        reader.readAsArrayBuffer(file);
    } else {
        alert("Please upload a valid PDF file.");
    }
});
```

This code should be familiar to you by now. But here is a run down of what it does:

- The user selects a file to upload We detect the file and ensure it's a PDF.
- FileReader reads it readAsArrayBuffer() loads the raw binary contents.
- A blob is created We turn the buffer into a Blob with MIME type application/pdf
- An object URL is created A temporary blob URL is made with URL.createObjectURL()
- The PDF file is previewed We insert that URL into an <iframe> so it shows up in-browser.

Reading XML files with JavaScript

JavaScript provides several powerful ways to read and work with XML files, both from local sources (such as user-uploaded files) and remote servers (such as web APIs or hosted XML documents). XML, which stands for eXtensible Markup

Language, is a structured format used for storing and exchanging data. It's commonly used in many older APIs and in configurations, making it important for JavaScript developers to know how to handle it effectively. There are four common approaches in JavaScript to read and work with XML files, each suited for a different use case. In this section, we'll explore these methods one by one with hands-on examples. But first, let's understand what each approach does and when you'd use it:

a) Using the `FileReader` and `DOMParser` This method is ideal when you want to read local XML files, such as when a user selects a file from their computer using an `<input type="file">` element. JavaScript's `FileReader` object is used to read the contents of the file, and the `DOMParser` object then converts the raw XML string into a document that can be navigated like a regular HTML page. Once parsed, you can use standard DOM methods (like `getElementsByTagName()` or `querySelector()`) to extract and display data from the XML file.

Workflow hint: local file → `FileReader` → `DOMParser` → DOM methods

b) Using the Fetch API and `DOMParser` This modern and clean method is perfect for reading remote XML files available over the internet. You start by using the `fetch()` function to request the XML file from a URL. Once the response is received as plain text, you pass it into `DOMParser` to convert it into an XML document. Just like before, you can then explore the data using DOM methods.

Workflow hint: `fetch()` remote file → `DOMParser` → DOM methods

c) Using `XMLHttpRequest` This is the older way to read XML from a remote source, but it still works and is widely used in legacy systems. The `XMLHttpRequest` object can directly receive an XML document response. In many cases, you can work with this `responseXML` directly. If you're dealing with a plain text response instead, you can still use `DOMParser` to turn it into a DOM document and proceed with normal XML handling using DOM methods.

Workflow hint: `XMLHttpRequest` → (optional) `DOMParser` → DOM methods

d) JavaScript XML Handling with XPath When you need precise control over what part of the XML to extract—especially from deeply nested structures—XPath becomes extremely useful. XPath lets you write expressions to “query” specific data from an XML document. You can use it with any of the above methods, once you have an XML document in hand. JavaScript provides the `document.evaluate()` function to apply XPath queries, allowing you to pull out just the data you need, even from complex XML files.

Workflow hint: (any method to get XML) → `DOMParser` if needed → `document.evaluate()`

Here are some example scenarios when you may need to deal with an XML file:

- Reading config files in XML format.
- Parsing RSS feeds (which are XML-based).
- Processing API responses in XML (though JSON is more common today).

Once XML data has been extracted and parsed (converted) into a document; among the DOM methods used to read the data, the following are common:

- Get elements by tag using `.getElementsByTagName()` for example:

```
xmlDoc.getElementsByTagName("book");
```

- Query with CSS selectors using `.querySelector()` for example select an `<author>` tag like this:

```
xmlDoc.querySelector("author");
```

- Extract text content using `.textContent` for example:

```
element.textContent;
```

- Get attributes using `getAttribute()` for example select an element with an id attribute value of "id":

```
element.getAttribute("id");
```

Each of the approaches listed above for reading XML files has its own strengths and use cases. As you go through the upcoming examples, you'll see how flexible JavaScript is when it comes to handling XML—whether it's reading a local file from your computer or pulling data from a web service. Let's dive deeper into the above-mentioned four approaches JavaScript takes to read and process XML files.

Using the FileReader and DOMParser

The DOMParser API converts an XML string into a traversable DOM object, similar to the HTML DOM whereby it then becomes very easy to read the XML document using the various methods provided by the HTML document object—which you are already familiar with. I introduced the DOMParser in Chapter 15 (DOM and URL Manipulation). Let's dive right into the steps of reading an XML file. This approach involves using a combination of FileReader and the DOMParser. FileReader will do the reading of the file, and pass its contents to DOMParser as a string. This combination is best suited for reading local files. The workflow is as follows:

- You select an XML file via a file input field (`<input type="file">`) on a web page.
- Next, you read the file using the FileReader, and typically store its text (string) in a variable.

- Next, you parse (convert) the text into an XML DOM object using the `DOMParser`
- Finally, extract the data you need from the file using DOM methods (`getElementsByTagName`, `querySelector`, etc.).

Let us look at an example. Using your IDE (e.g VS Code), create a local XML file to use for testing the following examples with. Name the file for example 'book-s.xml' and paste the following XML code in it:

```
<library>
  <book>
    <title>JavaScript: The Good Parts</title>
    <author>Douglas Crockford</author>
  </book>
  <book>
    <title>Eloquent JavaScript</title>
    <author>Marijn Haverbeke</author>
  </book>
</library>
```

Next, place this code in your HTML document.

```
<input type="file" id="xmlFileInput" accept=".xml" />
<pre id="output"></pre>
```

It contains the file input field to allow you select the XML file to upload into the program to process it. Notice the `accept=""` attribute which has a value of `'xml'` so it accepts XML files. The `<pre>` tag will be the target element where the parsed XML data will be displayed for you to see.

Here is the code to place in your JavaScript file:

```
document.getElementById('xmlFileInput')
  .addEventListener('change', (e) => {
    const file = e.target.files[0];
    const reader = new FileReader();

    reader.onload = (event) => {
      const xmlString = event.target.result;
      const parser = new DOMParser();
      const xmlDoc = parser.parseFromString(
        xmlString, "text/xml"
      );

      // Extract data (example: get all <book> tags)
      const books = xmlDoc.getElementsByTagName("book");
      let output = "";

      for (let book of books) {
```

```

.textContent;

    output += `Title: ${title}, Author: ${author}\n`;
  }

  document.getElementById("output").textContent = output;
};

reader.readAsText(file);
});

```

As you can see, it uses the `FileReader` in combination with the `DOMParser`. The working of the code should already be familiar to you right now. But let me drop you some hints on what it does, just in case. This code reads an XML file selected by the user, extracts information from it (in this case, book titles and authors), and displays that information on the screen.

- First it waits for the User to Select a File

```

document.getElementById('xmlFileInput')
  .addEventListener('change', (e) => {
    // ...

```

This line listens for when the user chooses a file from an `<input type="file" id="xmlFileInput">`. When that happens, the code inside the function runs. The event of 'change' is what fires in JavaScript whenever a file upload HTML field is used to select a file.

- Next, it gets the File That Was Chosen

```
const file = e.target.files[0];
```

This grabs the first file the user selected. Even if they were allowed to choose many files—in our example, they are not, we're only grabbing the first one here. This is because we only fetch the file at the first index in the files array (`files[0]`).

- Next, we set up a `FileReader`

```
const reader = new FileReader();
```

This creates a `FileReader`—the special browser object that can read files from your computer, which we have seen in action already, from previous examples.

- Next, we wait for the `FileReader` to be done reading the file

```

reader.onload = (event) => {
  const xmlString = event.target.result;

```

and then capture and store the result in a variable `xmlString`

- Next, we parse the XML string into useful data:

```
const parser = new DOMParser();
const xmlDoc = parser.parseFromString(
  xmlString, "text/xml"
);
```

- The DOMParser turns the raw text from the file into a real XML Document, which we can now work with just like any other HTML DOM element.

-Next we get all the <book> elements

```
const books = xmlDoc.getElementsByTagName("book");
```

This grabs all the <book> elements inside the XML. It returns something like an array, so we can loop through it. -Loop Through the Books and Extract Info

```
let output = "";
for (let book of books) {
  const title = book.querySelector("title")
    .textContent;

  const author = book.querySelector("author")
    .textContent;
  output += `Title: ${title}, Author: ${author}\n`;
}
```

- Each book is like an object with children.
- .querySelector("title") finds the <title> inside the <book>, and .textContent grabs just the text inside it.
- We build a string of all the titles and authors.
- Then display the output

```
document.getElementById("output").textContent = output;
```

-Thus, finally, showing all that data inside the DOM <pre> element with the ID output.

Using the DOMParser is the most modern and recommended way to parse XML files.

Using the Fetch API and DOMParser

The Fetch API of JavaScript as we have come to see so far, is suitable for fetching remote files through a network, through making AJAX requests. So, if the XML file you want to read from, is hosted on a server and not on your local machine, fetch has got your back. It lets you fetch and parse it directly. This approach involves a combination of fetch() and the DOMParser. The data is fetched by fetch(), and then passed to DOMParser as a string, which reads (extracts) the XML data

you need. The following is an example code to make the AJAX request using `fetch`, process the XML data, and use it as you please. In this case, I will write the data to the console just for demonstration. Again, this combination is best suited for reading files from a remote server, although it will still work if the file is in your local file system. Just pass the file path or remote path as an argument to `fetch()`. The workflow is as follows:

- You make a `fetch()` request to get the data from the remote server and get that data back as a text string.
- Next, you parse (convert) the text into an XML DOM object using the `DOMParser`
- Finally, extract the data you need from the file using DOM methods (`getElementsByTagName`, `querySelector`, etc.)

Let's see an example. Place the following code in your JavaScript file:

```
fetch('books.xml')
  .then(response => response.text())
  .then(xmlString => {
    const parser = new DOMParser();
    const xmlDoc = parser.parseFromString(
      xmlString, "text/xml"
    );

    // Process XML data here
    console.log(xmlDoc.querySelector("title")
      .textContent);
  })
  .catch(error => console.error("Error loading XML:", error));
```

In a real-life scenario, instead of the string `'books.xml'` that I have passed to `fetch()` above, you would pass in the real URL path of the API endpoint (target web address or URL) where you are expecting to get a response in XML format from. Using `fetch` is the best way to deal with remote XML files.

Using the XMLHttpRequest object

The `XMLHttpRequest` object is a built-in JavaScript object that helps your web page ask for data from a server without refreshing the page. That is the definition of an AJAX (Asynchronous JavaScript and XML) request. When I come to talk about Extensions and APIs in Chapter 22, you will learn all about AJAX and how it works. You will see how this same `XMLHttpRequest` object is very effective in that. However, it is not only good at making requests (local and remote) and receiving the response back in text format. As its name suggests, it is designed to handle XML files out of the box. In fact, it was the old standard way of reading and proc-

essing XML files. It was very effective and powerful then, and it still works in all browsers today. Though it may seem outdated because it's no longer commonly used, it is still relevant and being used in legacy systems. Unlike when reading XML files using `FileReader` and `fetch()` where you have to use pass the data over to `DOMParser` to be read, when using the `XMLHttpRequest`, you do not need the `DOMParser`. This is because, the `XMLHttpRequest` object has its own tools to parse the XML data. If the remote server responds to the AJAX request with valid XML (with the `Content-Type` header correctly set e.g. `text/xml` or `application/xml`), the `responseXML` property of `XMLHttpRequest` automatically parses the XML data into a readable DOM object (DOM)-which is what the `DOMParser` would normally do, so you do not need it. This means you can directly query the data stored in `responseXML` using DOM methods like `getElementsByTagName()`, `querySelector()`, etc. The `DOMParser` would only be needed if you are working with raw XML text-like a string or a non-XML HTTP response like `responseText`. In this case, you would get the response data as text from the `responseText` property, and can use the `DOMParser` to parse it into a DOM object.

Again, this combination is best suited for reading files from a remote server, although it will still work if the file is in your local file system. Just pass the file path or remote path as the second argument to the `open()` method of the `XMLHttpRequest` object. The workflow is as follows:

- You make a `XMLHttpRequest` request to get the data from the remote server and get that data back optionally as a text string (using the `responseText` property) or an already parsed XML DOM object (using the `responseXML` property).
- Next, you parse (convert) the text into an XML DOM object using the `DOMParser` if it is string format (gotten using the `responseText` property). If however the data is an already parsed XML DOM object, you do not need to use the `DOMParser`.
- Finally, extract the data you need from the file using DOM methods (`getElementsByTagName`, `querySelector`, etc.)

Let's see an example of reading a local XML file 'books.xml', which is in our project folder, parsing it and displaying its data in the console.

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "books.xml", true);
xhr.onreadystatechange = function() {
  if (xhr.readyState === 4 && xhr.status === 200) {
    // Already parsed as XML
    const xmlDoc = xhr.responseXML;
    const books = xmlDoc.getElementsByTagName("book");
```

```

// to array & map through
// the data
const bookArray = Array.from(books)
  .map(book => {
    const title = book.querySelector("title")
      .textContent;

    const author = book.querySelector("author")
      .textContent;
    return { title, author };
  });

console.table(bookArray);
}
};
xhr.send();

```

Let's understand the above code.

```
const xhr = new XMLHttpRequest();
```

This creates a new request object called `xhr` (short for "XML HTTP Request"). It will help you send a request to get data from a file (in this case, an XML file). Remember to test this by placing the XML file `books.xml` from our previous `fetch()` example in the root folder of your project—the same location as this JavaScript code's file.

Next, we call the `open()` method of the `XMLHttpRequest` like so:

```
xhr.open("GET", "books.xml", true);
```

This is the line that makes the request for the file. It prepares the request by saying:

- "GET": You want to get some data (like reading a file)
- "books.xml": This is the path and file you're asking for.
- `true`: This means the request is asynchronous, so your page doesn't freeze while waiting for the server to respond. This is essentially what makes this request an asynchronous (AJAX) one. If you used `false`, the browser would pause everything until the request finishes—which is not desirable at all.

In the following line, this is how we check if anything has changed in the state of the request, and run a function:

```
xhr.onreadystatechange = function() { ... }
```

Requests go through 5 ready states (0-4), and these are stored in the `readyState` property of the `XMLHttpRequest` object. I will explain all the `readyState` property values and what they mean in Chapter 22 under the topic of AJAX using

XMLHttpRequest. As you can see in this example, the one we need to care about is the state that says it's done, and that is when the value of the `readyState` property is 4.

```
if (xhr.readyState === 4 && xhr.status === 200)
```

Actually, we check for two things: a) if the value of the `readyState` property is 4, and b) if the value of the `status` property is 200

`xhr.readyState === 4` means the request is done and we got a response. `xhr.status === 200` means the server said 'OK', and everything went well.

Only when both are true do we process the response. Next, we receive the XML data returned and store it in a variable `xmlDoc`.

```
const xmlDoc = xhr.responseXML;
```

The `responseXML` property of the `XMLHttpRequest` object gives you the parsed XML data as a DOM object, so you can use methods of the `HTMLElement` object like `.getElementsByTagName()` and `.querySelector()` etc, just as you would do with any other HTML element. This, you would agree with me, is amazing.

Other than the `responseXML` property, there are other useful properties of the `XMLHttpRequest` object designed for you to work with other data formats and handle the whole request process efficiently. I will provide you with all the properties when I go in depth into AJAX requests in Chapter 22.

Let's take a look at what we do with the result in our example. Remember as pointed out above that at this point, we already have an `HTMLElement`-like object made possible by `responseXML` and stored in `xmlDoc`. The next thing we do therefore is to grab all the book (in `<book>`) tags from the XML data like so:

```
const books = xmlDoc.getElementsByTagName("book");
```

`getElementsByTagName()` returns an `HTMLCollection` object, and so that is now what `books` is.

Subsequently, I am displaying the XML data on books in the console, and notice that this time I use `console.table()` instead of the usual `console.log()` we have been using. I will provide you with the other methods of the `console` object later in Chapter 21 (Error, Debugging and Testing). For now, just know that `console.table()` takes an array, and displays the data in a clean table format in the console.

This means we need to convert the `HTMLCollection` `books` into an array to pass to `console.table()`. That is exactly what we do in this line:

```
const bookArray = Array.from(books)...
```

If you remember in Chapter 15 when we learned about the collection of elements `HTMLCollection` and `NodeList`, we looked at how they can be converted to an array under the section "Using all array methods on `HTMLCollections` & `NodeLists`". If you wish to refresh your mind on how and why we need to convert an `HTMLCollection` into an array, revisit that section and then come back. Just to remind you once more, there are two ways to do the conversion of both `HTMLCollections` and `NodeLists`. These are by either using the `Array.from()` method or the spread operator. So, there you have it, a good lesson on how to convert an `HTMLCollection` into an array. Being able to convert data in programming from one data type to another is a valuable skill. Because `bookArray` is now an array, we can call any array method on it—and in this case, we use the `map()` method. Using `map()`, we map through `bookArray` to extract the data of each individual book, which in this case is two pieces of the data; title and author. To understand how the `map()` function works, refer back to Arrays in Chapter 3 where I listed and explained the methods of the array object in JavaScript. The code doing the conversion from an `HTMLCollection` to an array, and looping through it using `map()` looks like this:

```
const bookArray = Array.from(books).map(book => {
  const title = book.querySelector("title").textContent;
  const author = book.querySelector("author").textContent;

  return { title, author };
});
```

Basically, `.map(...)` – loops through each `<book>` and creates a new object. Within that loop, we use `.querySelector()` to find the `<title>` and `<author>` tags inside each of those books. We use `.textContent` to grab the actual text inside the tag. Each object is therefore returned with title and author keys in it.

Because this all happens inside `map()`—which works in a loop fashion, the data of 'title' and 'author' which we extract from each book is being repeatedly sent back to be pushed into the `bookArray` variable. The sending back of the extracted data happens because of the `return` statement inside the `map()` function:

```
return { title, author };
```

The `bookArray` then ends up as an array containing multiple book objects like this:

```
[
  { title: "JavaScript: The Good Parts", author: "Douglas Crockford" },
  { title: "Eloquent JavaScript", author: "Marijn Haverbeke" }
]
```

That is it, we finally display the array in the console using `console.table()`;

```
console.table(bookArray);
```

JavaScript XML handling with XPath

A quick reminder of what XPath is

XPath (XML Path Language) is a query language for selecting nodes out of an XML document. Consider it to be to XML data what SQL is to a relational database: you describe the thing you want, and it goes and finds it for you. We covered XPath properly in Chapter 15, under the heading “XPath and selecting DOM Elements”. That is where you will find what `document.evaluate()` does, the full list of valid query expressions, the `XPathResult` types and how each one changes the way you read your results back, and when XPath is worth reaching for instead of `querySelectorAll()`. If any of that is hazy, go back and read it now, because the rest of this section assumes it. What this chapter adds is the part that belongs to file management: how you get the XML into the browser in the first place. Take particular note that how you acquire the XML document does not matter to XPath. Whether you get it using `FileReader`, the Fetch API or `XMLHttpRequest` is not the point of focus here, because that is not what XPath is. XPath is what you run on the data once you have it, when you hand it over to `document.evaluate()`.

XPath in action

Let’s look at some examples of handling XML files using XPath.

Read from a local or remote XML file

Create a file in your local file system, in the same directory as this JavaScript file. Name the file `books.xml` and paste the following code in it:

```
<library>
  <book>
    <title>JavaScript: The Good Parts</title>
    <author>Douglas Crockford</author>
  </book>
  <book>
    <title>Eloquent JavaScript</title>
    <author>Marijn Haverbeke</author>
  </book>
</library>
```

As you can see, this is XML structured data about books. The data contains two books, and you can tell because it contains two `<book>` tags. Within each book data is the title (`<title>`) and the author (`<author>`) of the book. We are going to use XPath to read the data about the books from this XML file, and log the title of

each to the console. Here is the code, and I will explain how it works in the code comments as well as below:

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'books.xml');
xhr.responseType = 'document';
xhr.onload = () => {
    // This is an XML Document
    const xmlDoc = xhr.responseXML;
    // Use XPath on xmlDoc here (document.evaluate())

    const result = document.evaluate(
        // XPath expression
        "//*[book/title]",
        // Context node (the parsed XML)
        xmlDoc,
        // No custom namespace resolver
        null,
        XPathResult.ANY_TYPE, // Type of result
        // No previous result to reuse
        null
    );

    let node = result.iterateNext();
    while (node) {
        // This gives you the <title> text
        console.log("Title:", node.textContent);
        node = result.iterateNext();
    }
};
xhr.send();
```

Output:

Title: JavaScript: The Good Parts Title: Eloquent JavaScript

Explanation: We use an XMLHttpRequest object to make a request to grab the file books.xml. The request is a GET request, as seen in the first argument of xhr.open(). We pass the name of the file we need (books.xml) as the second argument to it. If the request was to a remote server, the second argument to open() would have been a long URL string of the server path.

```
open('GET', 'books.xml');
```

We indicate that we expect a document back:

```
xhr.responseType = 'document';
```

This means that when the data comes back, it would already be parsed into an XML document. That is why we access the response via the responseXML property of XMLHttpRequest like so:

```
const xmlDoc = xhr.responseXML;
```

Also, because the data has been converted into a document, it is ready for DOM methods to be used on it. This is why we are able to pass it to the `document.evaluate()` method without needing to first of all convert it using `DOMParser`. For this example, we fetched the file using `XMLHttpRequest`, but the file could have just as well have been read using `FileReader`, or the `Fetch API`, and it would work in the same way. Here is the same file `books.xml` being read using the `Fetch API`:

Example 2: Using XPath on XML data from `fetch()` request

```
fetch('books.xml')
  .then(r => r.text())
  .then(xmlStr => {
    const xmlDoc = new DOMParser().parseFromString(
      xmlStr, 'text/xml');
    // Use XPath here

    const result = document.evaluate(
      // XPath expression
      "//book/title",
      // Context node (the parsed XML)
      xmlDoc,
      // No custom namespace resolver
      null,
      // Type of result
      XPathResult.ANY_TYPE,
      // No previous result to reuse
      null
    );

    let node = result.iterateNext();
    while (node) {
      // This gives you the <title> text
      console.log("Title:", node.textContent);
      node = result.iterateNext();
    }
  });
```

Reading data from an XML string

The two examples above both read their data out of an actual file named `books.xml` sitting in your project folder. The first pulled it in with an `XMLHttpRequest` object, the second with a `fetch()` request. But what if you did not have to read the data from a file at all? What if you already had it in your script, stored in a variable as a string? Then there is no request to make, and neither `XMLHttpRequest` nor `fetch()` comes into it. You hand the string to `DOMParser` to turn it into a real XML document, and run `document.evaluate()` on that. That is exactly the example worked

through in Chapter 15, under “Use `document.evaluate()` to read an XML document”, using this same library of books. Rather than repeat it here, go and read it there — and note while you are looking at it that `DOMParser` is doing the job `XMLHttpRequest` did for us above. `XMLHttpRequest` parses the XML for you internally when you set `responseType` to `'document'`; `fetch()` and a plain string do not, which is why those two need `DOMParser` and the `XMLHttpRequest` version does not.

Limitations of the FileReader API

In this chapter, you’ve seen how JavaScript’s `FileReader` API allows us to work with local files — reading text files, images, and even CSV data — all within the browser. It’s a powerful tool for building interactive applications like text editors, previewers, or data visualizers, all without needing a backend. However, it’s important to understand the limitations of the `FileReader` API:

- It has read-only access. It can read files, but cannot write or modify them on disk. This is a built-in security feature to protect users' local files from unauthorized changes.
- It is user-driven. It only works after the user manually selects a file- there’s no way for JavaScript to access files silently or automatically.
- It has no file system control. It cannot create, delete, or list files or directories.

For more advanced file handling, such as saving files, editing them, or accessing directories directly, you’ll need to move beyond the browser environment. To build more powerful desktop-like applications with full file system access, you can explore tools like:

- `Node.js` – A backend JavaScript runtime that gives you full control over files using modules like `fs`.
- `Electron` – A framework that lets you build cross-platform desktop apps using HTML, CSS, and JavaScript, with access to the entire operating system.
- `Progressive Web Apps (PWAs)` – For limited file writing capabilities using newer browser APIs like the `File System Access API` (still evolving and not supported everywhere as of the time of this writing).

By understanding both the strengths and the constraints of the browser environment, you’ll be better equipped to choose the right tools and platforms when building real-world applications that work with files.

QUIZ — File Management

This page contains the Q & A (questions and answers) for this chapter — Chapter 18: File Management. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 10 to 14 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. JavaScript in the browser cannot open any file it likes from your hard drive.
What are the only two ways a file can get in, and why is it restricted like this?
Clue: in both cases the visitor has to do something deliberate.
2. Name the three `FileReader` methods you would actually use, and say what each gives you back.
Clue: their names all start the same way, and the ending tells you the format.
3. Why must the code that uses a file's contents live inside `reader.onload` rather than on the line after `reader.readAsText(file)` ?
Clue: reading a file is not instant.
4. You have read a text file and changed it. Why can you not simply save it back over the original, and what do you do instead?
Clue: the answer to the first half is the same reason as question 1.
5. Getting a file from an `<input type="file">` and getting one from a drag-and-drop are almost the same, but not quite. What is the one difference?
Clue: both end in `.files[0]`. It is what comes before that differs.
6. What is a "magic number" in a file, and what problem does checking one solve?
Clue: renaming `virus.exe` to `holiday.jpg` takes about two seconds.
7. The chapter gives four ways to read XML. Match each to the situation it suits best:
`FileReader + DOMParser fetch()` + `DOMParser XMLHttpRequest XPath` with `document.evaluate()`
Clue: one is for local files, one is the modern remote way, one is the old remote way, and one is not about fetching at all.

8. XMLHttpRequest has both a `responseText` and a `responseXML` property. When would you use each, and which one saves you a step?
- Clue: one of them has already done some work for you.
9. Why send a large file over a WebSocket in chunks rather than all at once? Give two reasons.
- Clue: think about what happens when the connection drops at 95%.
10. EXERCISE. Write the code to let a user pick a `.txt` file and show its contents in a `<pre id="output">`, rejecting anything that is not a plain text file.
- Clue: one event, one `FileReader` method, and one check on `file.type`.
11. EXERCISE. Take the array below, turn it into CSV text, and print it.
- ```
[['Name', 'Age'], ['Alice', '25'], ['Bob', '30']]
```
- Clue: two joins, with different separators.
12. EXERCISE. Given CSV text, split it into rows and cells and print the second row's cells as an array.
- Clue: the reverse of question 11, and `trim()` first saves you an empty last row.
13. EXERCISE. Given this XML in a string, print every book's title and author.
- ```
<library>
<book><title>Book One</title><author>Ann</author></book>
<book><title>Book Two</title><author>Ben</author></book>
</library>
```
- Clue: parse it first, then treat it exactly like HTML.
14. EXERCISE. Take some text, wrap it in a Blob, and build a link that downloads it as `notes.txt`.
- Clue: three steps — Blob, URL, anchor — and one thing to remember to clean up.

ANSWERS

1. The two ways in are:
- the visitor **selects a file** with an `<input type="file">` element
 - the visitor **drags and drops** a file onto the page

In both cases the

visitor

chose the file. That is the whole point. If a web page could read any file on your machine just by running JavaScript, every site you visited could quietly help itself to your documents, your photos and your saved passwords. So the browser only ever hands over files a person has deliberately offered.

And even then, access is **read-only**. JavaScript cannot modify or delete the file on disk.

2.

<code>readAsText(file)</code>	gives you the file as plain text
<code>readAsDataURL(file)</code>	gives you a Base64-encoded string
<code>readAsArrayBuffer(file)</code>	gives you the raw binary data

`readAsText()` is for text, CSV, XML and JSON. `readAsDataURL()` is what you use for images, because the result can go straight into an ``. `readAsArrayBuffer()` is for when you need the actual bytes — checking a file signature, or sending binary over a WebSocket.

There is a fourth, `readAsBinaryString()`, but it is deprecated. Use `readAsArrayBuffer()` instead.

3. Because reading a file is **asynchronous**. `readAsText()` does not return the contents — it only *starts* the reading. The line after it runs immediately, long before the file has finished being read, so at that moment there is nothing to use.

```
const reader = new FileReader();

reader.onload = function (e) {
  // this runs LATER, when the read has finished
  console.log(e.target.result);
};

// this just starts it off
reader.readAsText(file);
```

`onload` is the callback that fires when the read completes, and `e.target.result` is where the contents arrive. Put your code anywhere else and you will be working with `undefined`.

4. You cannot save over the original because JavaScript's access to the file is **read-only**, for the security reason in question 1. Overwriting a file in place would mean writing to your visitor's disk, which the browser will not allow.

What you do instead is **offer the changed version as a download**:

```
const blob = new Blob([modifiedText], { type: 'text/plain' });
const url = URL.createObjectURL(blob);
```

```
const a = document.createElement('a');
a.href = url;
a.download = 'modified-file.txt';
a.click();
```

The original file is untouched; the visitor gets a new one. There is a newer File System Access API that

can

write in place, but it needs permission every time and only works in the Chromium browsers, so it is not something to rely on.

5. The difference is **where you find the file**:

```
// from a file input
const file = e.target.files[0];

// from a drag and drop
const file = e.dataTransfer.files[0];
```

A dropped file arrives on the event's `dataTransfer` property rather than on an input element. Everything after that — the `FileReader`, the `onload`, `readAsText()` — is identical.

One other thing drag and drop needs: `e.preventDefault()` in the `dragover` and `drop` handlers. Without it the browser does its own default thing, which is usually to navigate away and open the file itself.

6. A **magic number** (or file signature) is the first few bytes of a file, which identify what the file really is. A PNG always begins `89 50 4E 47`; a JPEG always begins `FF D8`.

The problem it solves is that **a file extension is just part of the name**, and anyone can change it. Renaming `virus.exe` to `holiday.jpg` takes seconds and fools any check that only looks at the extension — or even at the MIME type, which the browser largely infers from that same extension.

Reading the actual bytes tells you the truth:

```
const bytes = new Uint8Array(arrayBuffer);
const isPng = bytes[0] === 0x89 && bytes[1] === 0x50 &&
  bytes[2] === 0x4E && bytes[3] === 0x47;
```

You need `readAsArrayBuffer()` for this, because you are looking at raw bytes rather than text.

7.

- **FileReader + DOMParser** — for a **local** XML file the visitor selected.
- **fetch() + DOMParser** — the **modern** way to read a **remote** XML file.

- **XMLHttpRequest** — the **older** way to read a remote one. Still works everywhere, and you will meet it in legacy code.
- **XPath with document.evaluate()** — not a way of *fetching* XML at all. It is a way of
 - querying* XML you already have, and it works with whichever of the other three brought you the document.

The first three answer "how do I get the XML?"; the fourth answers "how do I find the bit I want inside it?".

8.

- **responseText** gives you the response as a plain string. You then have to parse it yourself with `DOMParser` before you can use DOM methods on it.
- **responseXML** gives you an **already-parsed XML document**. No `DOMParser` needed.

So `responseXML` saves you a step — that is the one to reach for when you know the response is XML. Use `responseText` when the response is something else, or when you want the raw string for another purpose.

For `responseXML` to be populated, the server must send the response with an XML content type, or you must ask for one:

```
xhr.responseType = 'document';
```

9. Any two of these:

- **Recovery.** If the connection drops, you retry only the failed chunk. Sending the whole file in one go means starting again from zero — painful at 95% of a large video.
- **Pause and resume.** Because you track your position with an offset, you can stop and pick up where you left off.
- **Real-time processing.** The server can begin working on the first chunks while the rest are still arriving, rather than waiting for the entire file.
- **Less overhead.** A WebSocket opens one persistent connection, instead of the repeated headers and connection setup of separate HTTP requests.

The mechanism is `file.slice(offset, offset + chunkSize)`, reading each slice with `readAsArrayBuffer()` and moving the offset along until you reach `file.size`.

```

10. const fileInput = document.getElementById('fileInput');
    const output = document.getElementById('output');

    fileInput.addEventListener('change', function () {
        const file = this.files[0];

        if (file && file.type === 'text/plain') {
            const reader = new FileReader();

            reader.onload = function (e) {
                output.textContent = e.target.result;
            };

            reader.readAsText(file);
        } else {
            output.textContent = 'Please upload a plain text (.txt) file.';
        }
    });

```

The `file &&` part of the check matters: if the visitor opens the file picker and then cancels, `files[0]` is `undefined`, and asking `undefined.type` would throw.

```

11. const data = [['Name', 'Age'], ['Alice', '25'], ['Bob', '30']];

    const csv = data.map(row => row.join(',')).join('\n');

    console.log(csv);

```

Output:

```
Name,Age Alice,25 Bob,30
```

Two joins doing two different jobs: the inner one puts commas **between the cells of a row**, and the outer one puts newlines **between the rows**.

```

12. const csv = `Name,Age
    Alice,25
    Bob,30`;

    const rows = csv.trim().split('\n');
    const cells = rows[1].split(',');

    console.log(cells);
    // ['Alice', '25']

```

`trim()` first, because a CSV file usually ends with a newline. Without it, `split('\n')` hands you an empty string as a final row, and you end up building a blank table row from nothing.

```

13. const xmlString = `<library>
    <book><title>Book One</title><author>Ann</author></book>
    <book><title>Book Two</title><author>Ben</author></book>
  </library>`;

const parser = new DOMParser();
const xmlDoc = parser.parseFromString(xmlString, "text/xml");

const books = xmlDoc.getElementsByTagName("book");

for (let book of books) {
  const title = book.querySelector("title").textContent;
  const author = book.querySelector("author").textContent;
  console.log(`Title: ${title}, Author: ${author}`);
}

```

Output:

Title: Book One, Author: Ann Title: Book Two, Author: Ben

The point worth taking away is that once `DOMParser` has done its work, the XML behaves **exactly like HTML** — `getElementsByTagName()`, `querySelector()` and `.textContent` all work just as they did in Chapter 15.

```

14. const text = "These are my notes.";

// 1. wrap the text in a Blob
const blob = new Blob([text], { type: 'text/plain' });

// 2. make a temporary URL that points at it
const url = URL.createObjectURL(blob);

// 3. build a link and click it
const a = document.createElement('a');
a.href = url;
a.download = 'notes.txt';
a.click();

// 4. and clean up – but give the
// download a moment to start first
setTimeout(() => URL.revokeObjectURL(url), 1000);

```

That last step is the one people forget. A blob URL holds the data in memory until you revoke it, so leaving them lying about is a memory leak.

Note the small delay. Revoking on the very next line after `a.click()` can cancel the download before the browser has begun fetching the blob.

Chapter 19 — FORMS AND EMAIL

- The three ways to handle form data in JavaScript
 - i) Extracting the value from form fields
 - ii) Using the elements property of HTMLFormElement
 - iii) Using the FormData object
 - The FormData object and files
 - Mastering FormData and form fields
 - Multiple checkboxes with the same name
 - Multiple file uploads
 - Disabled form fields
 - Conclusion on form handling
- Sending emails
 - Sending Emails with EmailJS
 - Sending Emails with Node.js and Nodemailer
 - App Passwords
 - Why use an App Password

This demonstrates how your programming language allows you to manage web forms and user-submitted input. Forms are a very important part of web development. With forms, you can convert an otherwise boring static website/application into an interactive application which responds to user input and can respond with some data depending on the value or values of the input. With forms you can create an application that conducts a user survey, collect data from users to register them for an event whereby their preferences are taken into consideration, you can order for a product, service or book a reservation etc. With forms, your web application can also collect data from a user and submit them to a server or another remote external application and listen for, and return the response. JavaScript really shines in this domain. There are three ways to handle form data in JavaScript. Let's start by writing an HTML form to use in demonstration:

```
<form id="myForm"> <input type="text" id="username" name="username"
value="JohnDoe">

<input type="email"
id="email"
name="email"
```

```
value="john@example.com">

<input type="file" name="myFile" />

<button type="submit">Submit</button>

</form>
```

i) Extracting the value from form fields

With this approach, you simply use any of the available element selector properties of the `HTMLElement` object to select a form field and then grab its value. Here is an example, here is how you extract the values of the username and the email fields of the above form by selecting them based on their id attributes:

```
let username =
    document.getElementById("username").value;

let email =
    document.getElementById("email").value;
```

This works, but it is not flexible because it requires you to manually select each field and get its value. It could be tedious and hard to maintain for large forms with many fields, especially if some of those form fields may be dynamically added. If you have a file field to process, for example:

```
<input type="file" id="myFile" name="myFile" />
```

You would select the field as normal, like so:

```
let fileInput = document.getElementById("myFile");
let file = fileInput.files[0];
```

Note very carefully that even though you select the field and it is stored in the variable `fileInput`, you still need to access the `.files` property of that element to get the uploaded file. The `.files` property will give you a `File` object of the uploaded file. Do not make the common mistake of using the `.value` property, which is meant for retrieving the values of other input fields. On a file field it just gives you a path string—and browsers deliberately fake that path, for privacy reasons. Here is a bonus tip for you; the `.files.length` property eg:

```
fileInput.files.length
```

will tell you how many files were selected.

If your file field supports multiple uploads—meaning it has the ‘multiple’ attribute eg:

```
<input type="file" id="myFiles" name="myFiles" multiple />
```

You will retrieve the values via the `.files` property of the field all the same:

```
let files = document.getElementById("myFiles").files;
```

It is worth being precise about what comes back, because this catches people out: `.files` ALWAYS gives you a `FileList`, whether the field allows one file or many. A `FileList` is a list even when it holds a single item. What changes with the multiple attribute is only how many items can be in it. The `File` object itself is what you get when you reach into that list — `files[0]`.

ii) Using the `elements` property of `HTMLFormElement`

JavaScript also provides an `elements` property to be used for event handling when working with forms. It's a property of the `HTMLFormElement` interface that allows you to easily access every single form control (input, select, textarea fields, etc.) inside a `<form>` element. Therefore note that the `e.target.elements` property is only available to be referenced on `<form>` elements. That is because the 'elements' property belongs to the `HTMLFormElement` object and not the `HTMLElement` object. `HTMLFormElement` is the interface a `<form>` element is an instance of, which is why the property lives there and not on `HTMLElement`. Uniquely, access to fields is obtained by using the name attributes of form elements (like so: `form.elements.name`), and not the id, class attributes or tag names like the selector properties of the `HTMLElement` object.

When an event (like submit) is triggered on a form, `e.target` refers to the `<form>` element. The `.elements` property on that form gives you a collection of all input fields inside it.

Let's look at a demonstration of how to use the `elements` property to process our `myForm` example above. We will add an event listener to our `myForm` form, and write a closure (anonymous function) to immediately handle the form submission upon the event occurring.

```
document.getElementById("myForm")
  .addEventListener("submit", function(e) {

  // Prevent page refresh
    e.preventDefault();

  // The <form> element
    let form = e.target;

  // Collection of form fields
    let formElements = form.elements;

  // Logs all form elements
```



```
let username = formElements.username.value;
    let email = formElements.email.value;
});
```

This is how `.elements` works:

- `elements` returns an `HTMLFormControlsCollection` (which is like an array but not exactly).
- You can access inputs using their name attributes (`form.elements.name`).
- You can also access inputs using numeric indices (`form.elements[0]`).
- Button elements are also included in `elements`.

Unlike selecting elements individually using selectors like `getElementById()` or `querySelector()` to retrieve their values, using `.elements` property is more efficient. Also, when dynamically working with forms where the number of inputs might change, using `.elements` is the way to go. If the form field is a file type eg:

```
<input type="file" name="profilePic" />
```

It is the same as above (retrieving from value directly). You would select the field, then use the `.files` property on it to retrieve the uploaded file. Here's an example:

```
let form = document.getElementById("myForm");
let fileInput = form.elements["profilePic"];
// Just like before
let file = fileInput.files[0];
```

Here are some points to bear in mind:

- `form.elements["fieldName"]` works like accessing a property from an object-with `elements` being the property of the form object.
- You still need to use `.files` to get the uploaded file(s)
- Again, do not make the common mistake of going for the `.value` property, as that will only give you the path string, which is not the actual file.

Using the FormData object

The `FormData` object provided by JavaScript makes working with forms very easy, and it provides various methods and properties to use to deal with the handling of submitted forms. Here is the syntax, where you will instantiate the object using the new keyword and passing to its constructor your form element:

```
myForm = new FormData(form);
```

Here is an example of how you would use it to retrieve the values of our `myForm` example above. We will first of all set an event listener on the form, to detect a submit event. This way, once the form is submitted, our custom function will be called

to process the form submission. Pay careful attention to see how we get the values of the form fields submitted, first, we create a FormData object instance by passing to its (FormData object) constructor the object that triggered the 'submit' event-which is our form. Here is where the object instance is created:

```
let formData = new FormData(e.target);
```

The e.target is our form, because it is the element on which the submit event was triggered. To then retrieve values from any of the form fields, it's easy-we call the get() method of the form object, passing it the name attribute of our form field as a string. Here is the syntax:

```
let myField1 = formData.get("myField1Name");
let myField2 = formData.get("myField2Name");
```

You do this for as many fields on your form as you wish to retrieve values for.

```
document.getElementById("myForm")
    .addEventListener("submit", myFunction);

// function to handle/process the submission
function myFunction(e) {
    // Prevent page refresh
    e.preventDefault();

    let formData = new FormData(e.target);

    let username =
        formData.get("username");

    let email =
        formData.get("email");

    let file = formData.get("myFile");
    if (file && file.name) {
        console.log("File name:", file.name);
        console.log("File type:", file.type);
        console.log("File size:", file.size, "bytes");

        const reader = new FileReader();
        reader.onload = function (event) {
            console.log("File content:", event.target.result);
        };

        // or use readAsDataURL(file) for images
        reader.readAsText(file);
    } else {
        console.log("No file selected.");
    }
}
```

Notice how we have used the `get()` method of the `FormData` object to retrieve the values submitted by the user of our form. The event listened for was a ‘submit’ event, which is the first argument passed to the `addEventListener()` method of the `HTMLElement` object, which is the method used in listening for events. The second argument to `addEventListener()` is your desired action (the thing you want done) in response to that event occurring. This is usually a named function in your code which will automatically be run, or a closure (anonymous function) which is run in the same way. See Chapter 24 (Events Handling) to learn more about how events work in JavaScript. Notice also that inside `myFunction`, we invoke the `FormData()` object, passing it a reference (selection) of our form—which was selected by its id like so: `document.getElementById("myForm")` before adding the submit event listener to it.

```
document.getElementById("myForm")
  .addEventListener("submit", myFunction);
```

The reference passed to `FormData()` is written as `e.target` which always refers to the element that triggered the event (the element the event occurred on). `FormData()` will then know how to wrap itself around that form element and apply all its methods on it.

iv)The `FormData` object and files

The `FormData` object handles file inputs too. It is built for working with all kinds of form fields—including file fields, and it treats them intelligently. Notice in our example above, we have a file field with the name attribute of “myFile”. We retrieve its value after the form submission like so:

```
let file = formData.get("myFile");
```

The `formData.get("myFile")` will return a `File` object (or null if no file was selected). This is why it is a good idea when handling a form submission to check if the file field has a value before doing anything with it. This way, you do nothing if no file was uploaded. That is why we have this check right after the retrieving of the file value above:

```
if (file && file.name) {
  // a file was uploaded, you can retrieve
  // its properties and, or, process
  // it here...
} else {
  console.log("No file selected.");
}
```

This `File` object is not just a string or file name—it is a real JavaScript object that includes the following properties:

- file.name - the file name
- file.size - file size in bytes
- file.type - MIME type (like "image/png" or "text/plain")
- And you can even pass it directly to a FileReader to read its contents.

Let me explain how to pass a retrieved uploaded file to the File Reader API. The trick lies in these lines of code, as above:

```
let file = formData.get("myFile");
// ...
// ...

const reader = new FileReader();
reader.onload = function (event) {
    console.log("File content:", event.target.result);
};

// or use readAsDataURL(file) for images
reader.readAsText(file);
```

The code on this line:

```
reader.onload ...
```

Is run asynchronously. This means that it will not be run immediately, and in fact, it will be triggered by this line after it:

```
reader.readAsText(file);
```

I know it may at first seem confusing to see that the line after (below) the reader.onload ... is run before it. But it's true, and the key lies in understanding that the line is not run, until it is triggered by this code:

```
reader.readAsText(file);
```

Basically; we call the readAsText() method on the File Reader API passing it the reference to the uploaded file—which was retrieved and stored in the file variable. The readAsText() method is what triggers the reading of the uploaded file. Because the reading of the file may take a while, we have to wrap the code to process that file in an onload event block like so:

```
reader.onload = function (event) {
    console.log("File content:", event.target.result);
};
```

The event.target.result contains the contents of the file that has been read. Bear in mind that readAsText() does what it says, it is meant for reading and returning data from a file in text format. If we were trying to read an image, we should have

used another method of the File Reader API dedicated for that; `readAsDataURL(file)`.

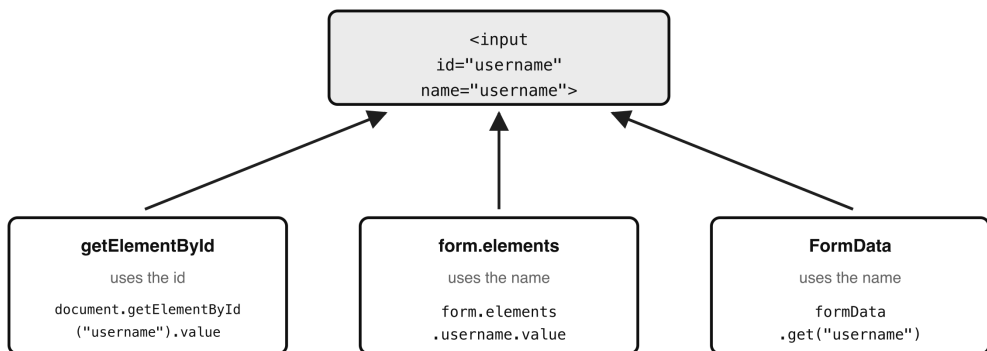
In summary,

- the `formData.get("fieldName")` syntax works for text inputs, checkboxes, selects, and file inputs.
- For file fields, the returned value is a `File` object—not a string. You can read from the file using `FileReader`, or send it via `fetch()` or AJAX without any extra steps.

Mastering FormData and form fields

Now that you have a full understanding of the working of the `FormData` object, I should go a bit deeper and show you some potential pitfalls. For the most part, the `FormData.get("fieldName")` syntax works for retrieving values submitted through nearly all types of form fields. But there are a few subtle points and exceptions worth understanding. First of all, you would have noticed that unlike the usual targeting of the `id` attribute, `FormData` uses the `name` attribute instead. Take note of that, because as simple as it is, it can lead to confusion sometimes. This also means that if a form element has no `name`, it will be ignored by `FormData`. Be on the lookout for that.

Three routes to the same field — and which attribute each uses



A field with no `NAME` is invisible to the last two — it will not appear in `FormData` at all, even though it looks perfectly normal on the page. So: `id` for finding one element, `name` for submitting.

- Figure 19.1 — Three routes to the same field, and which attribute each uses*

Multiple checkboxes with the same name

If you have several checkboxes like this:

```
<input type="checkbox" name="fruits" value="apple" />
<input type="checkbox" name="fruits" value="banana" />
```

`formData.get("fruits")` will return only the first checked value. To get all selected values, use its `getAll()` method — note the capital A, because JavaScript is case-sensitive and `getall()` does not exist:

```
// will return ['apple', 'banana']
formData.getAll("fruits");
```

Multiple File Uploads

If your form's file field allows multiple files—meaning its file field has the `multiple` attribute; `formData.get("photos")` will get just the first selected file. Here is an example of a file field that accepts multiple file uploads. Notice it has the 'multiple' attribute.

```
<input type="file" name="photos" multiple />
```

To get all the files, use its `getAll()` method like so:

```
// will return [File, File, File...]
formData.getAll("photos");
```

Using `getAll()` gives you an ordinary `ARRAY` of `File` objects, rather than the single `File` object that `get()` returns. That is worth remembering, because an array means you can go straight to `.map()`, `.filter()` and the rest without converting anything first.

Disabled form fields

Disabled fields are ignored. Any input with the `disabled` attribute is not included in `FormData` at all. For example:

```
<input type="text" name="age" value="30" disabled />
```

This field will not appear in `formData`.

Conclusion on form handling

Each one of the three approaches of form handling can handle files just as well as the other. The `.files` property is where the actual `File` objects live—regardless of how you get the field (ID, elements, or `FormData`). For file uploads, `FormData` is the go-to if you're sending the file to a server. But if you're working with files inside the browser only (like previews or editing), `.files[0]` is often more direct.

Sending Emails

JavaScript doesn't have built-in capabilities to send emails directly from the browser. This is mainly for security reasons — allowing anyone to send emails from

the browser would be a major spam and privacy risk. However, there are two popular and safe ways to send emails using JavaScript:

Sending Emails with EmailJS (no backend needed)

EmailJS is a service that lets you send emails directly from JavaScript — without needing a server. It connects your frontend to email services like Gmail, Outlook, etc. All you need to get started with it is the following:

- A free account on EmailJS
- A public API key
- An email template set up in your EmailJS dashboard
- Your service ID and template ID

Let's see the steps to set it up.

a) Sign up at emailjs.com and log in. b) From the EmailJS dashboard, Add an email service (like Gmail). c) Create a new email template, and add dynamic fields like `{{name}}`, `{{message}}`, etc. This is done within the EmailJS app d) Get your service ID, template ID, and public API key from the EmailJS dashboard. e) Include the EmailJS SDK in your HTML for example as a CDN:

```
<script src="https://cdn.emailjs.com/dist/email.min.js"></script>
<script>
  // Replace with your actual public API key
  emailjs.init('YOUR_PUBLIC_KEY');
</script>
```

f) If you are triggering the email sending after a form is submitted (most popular approach), create the HTML form:

```
<form id="contact-form">
  <input type="text" name="user_name"
    placeholder="Your name" required>

  <input type="email" name="user_email"
    placeholder="Your email" required>

  <textarea name="message"
    placeholder="Your message" required></textarea>

  <button type="submit">Send</button>
</form>
```

g) Create an event listener to listen for a submit event on the form, and send the email. This JavaScript code is in your JavaScript file eg `index.js`:

```

document.getElementById('contact-form')
.addEventListener('submit', function(e) {
  e.preventDefault();
  emailjs.sendForm(
    'YOUR_SERVICE_ID', 'YOUR_TEMPLATE_ID', this
  )
  .then(function(response) {
    alert('Email sent successfully!');
  }, function(error) {
    alert('Failed to send email. Error: ' + error.text);
  });
});

```

That's it. This is great for contact forms, feedback tools, or sending small user-generated messages — all without setting up a backend server.

Sending Emails with Node.js and Nodemailer (backend needed)

If you're working with a Node.js server, you can send emails using the popular nodemailer library. This is more flexible and powerful, especially for real-world apps that require authentication, attachments, or automated messages. Here are the steps to follow:

a) Assuming you already have Node.js installed, go ahead and initialise Node.js in your application if you have not already. Do it by navigating in your Terminal application into your project folder, and running the following command:

```
npm init -y
```

b) Install Nodemailer into your project by running this command:

```
npm install nodemailer
```

c) Create a send-email.js file which will contain the Node.js Nodemailer code:

```

// send-email.js
const nodemailer = require('nodemailer');

// Step 1: Create a transporter
// (using Gmail in this example)
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'yourgmail@gmail.com',
    // Use App Password if 2FA is on
    pass: 'yourgmailpassword'
  }
});

```



```

from: 'yourgmail@gmail.com',
  to: 'recipient@example.com',
  subject: 'Hello from Node.js!',
  text: 'This email was sent using Nodemailer and Node.js.'
});

// Step 3: Send the email
transporter.sendMail(mailOptions, function(error, info) {
  if (error) {
    console.log('Error:', error);
  } else {
    console.log('Email sent:', info.response);
  }
});

```

d) Run the file containing the Nodemailer code so it sends the email. Run it by running this command in the Terminal:

```
node send-email.js
```

Nodemailer lets you attach files, send HTML emails, and use templates. It is very powerful. In conclusion, JavaScript in the browser can't send emails directly — but thanks to tools like EmailJS (for frontend apps) and Nodemailer (for backend apps), you can still build powerful email features in your applications. If you're just learning vanilla JavaScript, EmailJS is a great beginner-friendly option. When you later learn Node.js, Nodemailer becomes a valuable tool for server-side email handling.

App Passwords

You will have noticed the comment in the Nodemailer code above saying to use an App Password if 2FA is on. Let us break down what that means, because it is no longer optional for most people. An App Password is a special password you generate from your Google Account that allows a specific app (like your Node.js script using Nodemailer) to access your Gmail account without using your main Google password. This is especially useful — and often required — when:

- Your Google account has 2-Step Verification (2FA) turned on (which is highly recommended).
- You want to connect a less secure app or script (like a Node.js app) to Gmail safely.

You may still find older tutorials telling you to switch on a setting called "Less Secure Apps". Do not go looking for it — Google withdrew it for most accounts, and App Passwords are the replacement. Here is a step-by-step guide on how to generate an App Password for Gmail. First of all, you must have 2-Step Verification enabled on your Google account before generating App Passwords.

a) Go to your Google Account settings—here:

Visit: <https://myaccount.google.com>

b) Turn on 2-Step Verification (if you haven't already)

- Click Security in the left sidebar.
- Under “Signing in to Google”, click 2-Step Verification.
- Follow the steps to enable it.

c) Generate an App Password

- Once 2-Step Verification is enabled, go back to the Security section.
- Under “Signing in to Google”, you’ll now see a link: App passwords.
- Click it (you may need to log in again).

d) Create a New App Password

- Under “Select app”, choose Mail.
- Under “Select device”, choose Other and type something like NodeMailer App.
- Click Generate. You’ll now see a 16-character password that looks something like this:

abcd efgh ijkl mnop

e) Next, use this password in Nodemailer. Replace your regular Gmail password in your Nodemailer code with the new App Password:

```
// 'pass' is the App Password you just generated
auth: {
  user: 'yourgmail@gmail.com',
  pass: 'abcd efgh ijkl mnop'
}
```

You can now send emails securely from your Node.js app without giving out your real Google password.

Why use an App Password

A huge reason is security. Your real password stays private. It’s safe to use—you can revoke or delete the App Password at any time. Compatibility is also a good factor. App Password works with apps that can’t handle Google’s full sign-in flow (like scripts, terminal apps, etc.).

QUIZ — Forms and Email

This page contains the Q & A (questions and answers) for this chapter — Chapter 19: Forms and Email. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 8 to 11 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Name the three ways of getting data out of a form, and say when each is the right choice.
Clue: one picks fields off one at a time, one gets the whole set, and one is built for sending to a server.
2. Two of the three approaches find fields by one attribute, and the third uses a different one. Which is which, and what happens to a field that lacks it?
Clue: the odd one out is `FormData`.
3. Why does `e.preventDefault()` appear at the top of every form-submit handler in this chapter?
Clue: without it you never get to see your own code work.
4. You have `<input type="file" id="myFile">`. What is wrong with reading `fileInput.value`, and what should you use instead?
Clue: what you get back is a path, and not even a real one.
5. Does `.files` give you a `File` or a `FileList`? Does the answer change when the field has the `multiple` attribute?
Clue: this is a trick question, and the trick is in the second half.
6. A form has two checkboxes with the same `name`. What does `formData.get()` return, and what should you use instead? What type comes back?
Clue: watch the capital letter in the method name.
7. A field on your form has the `disabled` attribute. What does `formData.get()` give you for it, and why does that matter?
Clue: it is not an empty string.

8. EXERCISE. Write a submit handler that stops the page reloading and prints the values of a `username` and an `email` field, using `FormData`.
- Clue: the form is `e.target`.
9. EXERCISE. Given a form with three checkboxes named `fruits`, of which two are checked, print all the checked values as an array.
- Clue: question 6 gives you the method.
10. EXERCISE. Using the `elements` property, get a form's `username` field and print its value — then print the same value using `getElementById`, to show they agree.
- Clue: `elements` is keyed by the name attribute, `getElementById` by the id.
11. EXERCISE. Write the check that runs only when a file really was chosen, and prints its name, type and size.
- Clue: `get()` on an empty file field does not return `undefined`.

ANSWERS

1.

- **Picking fields off individually** with `getElementById()` or `querySelector()`, then reading `.value`. Fine for a small form with two or three fields. Tedious and fragile for anything bigger, and awkward if fields are added dynamically.
- **The `elements` property** of the form. `form.elements` gives you every control in the form in one collection, so you are not writing a selector per field. Good when the form is large or its fields change.
- **The `FormData` object**. Built for the job of gathering everything up, and the one to reach for when you are sending the data to a server, because `fetch()` will take a `FormData` object directly.

All three can handle file fields equally well.

2. `getElementById()` uses the **id** attribute. `form.elements` and `FormData` both use the **name** attribute.

`form.elements.username` // by name `formData.get("username")` // by name

A field with **no name** attribute is ignored by `FormData` entirely — it simply will not appear. That is a quiet source of "why is my value missing?", because the field looks perfectly normal on the page and may well have an id.

So: id for CSS and for grabbing one element; name for anything that submits.

3. Because submitting a form **reloads the page by default**. That is the browser's original behaviour, from long before JavaScript: gather the fields, send them off, load whatever comes back.

If you let that happen, your page reloads the instant the user clicks Submit and your handler never gets to finish. `e.preventDefault()` tells the browser "I am handling this myself, don't do your usual thing."

```
form.addEventListener("submit", function (e) {
    // stop the reload first
    e.preventDefault();
    // ...now do the work
});
```

4. `fileInput.value` gives you a **path string**, not the file. And it is not even a real path — browsers deliberately fake it (you will typically see something like `C:\fakepath\holiday.jpg`) so that a web page cannot learn how your folders are laid out.

What you want is the `.files` property:

```
let fileInput = document.getElementById("myFile");
let file = fileInput.files[0];
```

That gives you a real `File` object, with `.name`, `.type` and `.size`, which you can hand straight to a `FileReader`.

5. `.files` **always** gives you a `FileList` — and no, the `multiple` attribute does not change that.

```
// both of these are FileList
document.getElementById("one").files
document.getElementById("many").files
```

A `FileList` is a list even when it is holding a single item. What `multiple` changes is only how many items the list is allowed to contain.

The `File` object is what you get by reaching into the list:

```
// a File
let file = fileInput.files[0];
// how many were chosen
let count = fileInput.files.length;
```

6. `formData.get("fruits")` returns **only the first checked value**. To get them all, use `getAll()`:

```
formData.getAll("fruits");
// ['apple', 'banana']
```

Two things to note. The capital **A** matters — `getall()` does not exist, and JavaScript is case-sensitive, so you would get `TypeError: formData.getall`

is not a function.

And what comes back is an **ordinary array**, not a special collection. That is convenient: you can go straight to `.map()`, `.filter()` or `.length` without converting anything.

The same applies to a file field with `multiple` — `getAll("photos")` gives you an array of `File` objects.

7. It gives you `null`.

```
<input type="text" name="age" value="30" disabled />

formData.get("age"); // null
```

A disabled field is not included in `FormData` at all, so asking for it is like asking for a field that was never there.

It matters because `null` is not an empty string. If your code does `formData.get("age").trim()` you get `TypeError: Cannot read properties of null` — and the cause is a `disabled` attribute somewhere in your HTML, which is not where most people look first.

```
8. document.getElementById("myForm")
   .addEventListener("submit", function (e) {
       e.preventDefault();

       const formData = new FormData(e.target);

       const username = formData.get("username");
       const email = formData.get("email");

       console.log("Username:", username);
       console.log("Email:", email);
   });
```

`e.target` is the form itself, because the form is the element the submit event happened on. That is why it can be passed straight to `new FormData(...)` without selecting the form again.

```
9. const form = document.getElementById("myForm");
   const formData = new FormData(form);

   const chosen = formData.getAll("fruits");

   console.log(chosen);
   // ['apple', 'cherry']
   console.log(chosen.length); // 2
```

Only the **checked** boxes appear. Unchecked checkboxes are not submitted at all — which is another way of saying they behave rather like disabled fields, in that

asking for them gives you nothing back rather than `false`.

```
10. const form = document.getElementById("myForm");

    // by name, through the form's elements collection
    const byName = form.elements.username.value;

    // by id, the direct way
    const byId = document.getElementById("username").value;

    console.log(byName);
    // "JohnDoe"
    console.log(byId);
    // "JohnDoe"
    console.log(byName === byId);
    // true
```

Both reach the same input; they just arrive by different routes. `form.elements` is keyed by the **name** attribute, `getElementById` by the **id**. In the chapter's example form the field happens to have both, and they happen to match — but there is no rule that says they must, and mixing them up is a common source of confusion.

You can also reach fields by position: `form.elements[0]`.

```
11. const file = formData.get("myFile");

    if (file && file.name) {
        console.log("File name:", file.name);
        console.log("File type:", file.type);
        console.log("File size:", file.size, "bytes");
    } else {
        console.log("No file selected.");
    }
```

The check is worth understanding rather than copying. When no file has been chosen, `formData.get()` on a file field does not give you `undefined` — it gives you an **empty File object**, whose `name` is an empty string. An empty File is still an object, and every object is truthy, so `if (file)` on its own would pass and you would go on to read a file that is not there.

Testing `file.name` as well is what makes the check reliable.

Chapter 20 —IMAGES

- JavaScript and images
- What JavaScript can do with images
- Image manipulation
 - Image preview before upload
 - Add a grayscale filter Effect to an image
 - Toggle a grayscale filter Effect on an image (on/off)
 - Permanently grayscale an image using Canvas
 - Resizing an image using Canvas
 - Rotate an image in 2d
 - Add feature to download the rotated image
 - Adding image filters
 - Increase the brightness of an image
 - Adding contrast to an image
 - Add a blur filter
 - Bonus tip - Combining filters

JavaScript and images

JavaScript can be used to handle images — and it's actually quite versatile in that area. It's capable of managing many image-related tasks, especially when combined with browser features or server-side tools. That said, how you use it depends on what you're building. For small web projects and learning vanilla JavaScript, you'll mostly use it to change images on a webpage or handle basic uploads. But if you go deeper into building real-world apps — like photo editors, e-commerce platforms, or social networks — JavaScript has the tools to do even more. You've already learned how to preview images using the DOM, load files with the `FileReader` API, and respond to file input events in earlier chapters. But what if we want to go beyond simply displaying images? What if we want to edit them — crop, apply filters, draw over them, or change their content in more advanced ways? That's where the power of the HTML5 `<canvas>` element comes in. You were introduced to `<canvas>` in Chapter 16 (The Canvas Element), where we used it for drawing shapes and making interactive graphics. In this chapter, we'll take it

further and explore how the canvas can act as a powerful tool for image processing. The canvas allows us to:

- Draw images onto it using `drawImage()`
- Access and manipulate pixel data with `getImageData()` and `putImageData()`
- Apply transformations, filters, overlays, and more

In short, while the DOM and `FileReader` can help load and preview images, it's the canvas that enables us to edit and process them directly in the browser.

What JavaScript can do with images

On the frontend (in the browser), you can use JavaScript to:

- Preview uploaded images before sending them to a server
 - Resize or crop images using the `<canvas>` element
 - Switch images dynamically (e.g., for image sliders or galleries)
 - Apply filters or effects (like converting to black-and-white)
 - Delete images from the webpage view (e.g., in a gallery)
- On the backend (e.g., using Node.js), JavaScript can also:
- Upload images to a server or cloud storage
 - Resize or compress images automatically (with libraries like Sharp)
 - Rename or delete image files from disk
 - Convert image formats (e.g., PNG to JPEG)
- For backend image handling, you'll need to explore the documentation of the backend platform you're using — such as Node.js, Express, or cloud services like AWS S3.

What you'll learn in this chapter is all practical-based. I will go ahead and give you practical, real-world image handling examples using vanilla JavaScript. All the examples run entirely in the browser — no backend required. You'll even build a basic image editor that uses the canvas to transform images right on the page. Let's dive straight in.

Image manipulation

Image preview before upload

Add this code to your HTML file (eg `index.html`)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My JavaScript Project</title>
</head>
<body>

    <h1>The JavaScript Blueprint</h1>

    <input type="file" id="imageInput" />
    <img
        id="preview"
        src=""
        alt="Image preview will appear here"
        style="max-width: 300px;
        display: block; margin-top: 10px;" />

    <script src="index.js"></script>
</body>
</html>

```

Add this code to your JavaScript file (eg index.js)

```

const imageInput = document.getElementById('imageInput');
const preview = document.getElementById('preview');

imageInput.addEventListener('change', function () {
    const file = this.files[0];

    if (file) {
        preview.src = URL.createObjectURL(file);
    }
});

```

In this code as you can see, there is an `<img>` tag in the HTML code, which will be used to preview the image. There is also a file upload input field where you will upload a file that you select from your computer. The uploaded file is then grabbed and dynamically inserted into the image tag for you to preview it before submission. This is a simple but very practical example of a feature you can add to enhance the user experience of your application.

Add a grayscale filter Effect to an image

Make sure you have an `images/` directory in the same folder as this `index.html` file, with an image in it named `'urban.jpg'`. That is the image used as the `src` attribute of the `<img>` tag in this page, and the one we are practising adding a grayscale filter to.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My JavaScript Project</title>
</head>
<body>

  <h1>The JavaScript Blueprint</h1>

  
    <br />

    <button id="grayScaleBtn">Make Grayscale</button>

    <script src="index.js"></script>
</body>
</html>

```

Add this code to your JavaScript file (eg index.js)

```

const btn = document.getElementById('grayScaleBtn');
btn.addEventListener('click', applyFilter);

function applyFilter() {
  const img = document.getElementById('myImage');

  // give the image filter a grayscale effect
  img.style.filter = 'grayscale(100%)';
}

```

Toggle a grayscale filter Effect on an image (on/off)

This effect is so cool, but we cannot reverse it. Let's make it possible to add and reverse the grayscale effect on the image. Make sure you have an `images/` directory in the same folder as this `index.html` file, with an image in it named `'urban.jpg'`, exactly as in the previous example. Here is the code to make that happen:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My JavaScript Project</title>
</head>
<body>

```

```


<br />

  <button id="grayScaleBtn">Make Grayscale</button>

  <script src="index.js"></script>
</body>
</html>

```

Add this code to your JavaScript file (eg index.js)

```

const btn = document.getElementById('grayScaleBtn');
btn.addEventListener('click', toggleFilter);

function toggleFilter() {
  const img = document.getElementById('myImage');

  /* This will still work, but let's use a ternary operator which is
  cleaner
  if (img.style.filter == 'grayscale(100%)')
  {
    img.style.filter = 'grayscale(0%)'
  } else {
    img.style.filter = 'grayscale(100%)'
  }

  - /

  img.style.filter = img.style.filter == 'grayscale(100%)' ?
    'grayscale(0%)' : 'grayscale(100%)';

}

```

Notice the modifications of the previous code: - Now when the ‘Make Grayscale’ button is clicked, we check the previous value of the `img.style.filter`, and if it was `'grayscale(100%)'`, we update it to `'grayscale(0%)'`. It would work just the same if we set it to `'none'` instead. Otherwise we update it to `'grayscale(100%)'`. - We re-named the function from `applyFilter()` to `toggleFilter()` so it reflects what it really does now—which is to turn the grayscale filter on the image on or off. - Notice also how, though using an if statement inside `index.js` will still work just the same, using a ternary operator to do the logic makes for less and cleaner (readable) code. This exercise is a great example for interactive photo editors, or for learning about CSS filters through JavaScript.

Permanently grayscale an image using Canvas

The grayscale effect using CSS filters (like in the previous example) only affects how the image looks in the browser. It doesn't actually modify the image data itself. So, that effect isn't permanent and can't be saved as-is. However, as I mentioned above, it is a great example for interactive photo editors, or for learning about CSS filters through JavaScript.

A CSS filter changes the view. A canvas changes the pixels.

CSS FILTER — temporary



CANVAS — permanent



Note the shape of it: you never modify the original. You read it, rebuild it on a canvas, and save that as a separate file. JavaScript cannot write over a file on the visitor's disk, so the "edited image" is always a new one sitting beside the old.

Figure 20.1 — A CSS filter changes the view; a canvas changes the pixels

But if you want to make the grayscale change—or any change to an image file—permanent, so that the user can download or upload the edited image, you have to use the `<canvas>` element. This is because, the canvas API's context which is the thing the image is built on, and all the changes you make on it are saved, makes the changes persist. You just have to then make the image downloadable as a new image file. Always remember that because of JavaScript's restriction on local files, what essentially happens is this; you read an (original) image, re-create it as a new image on the canvas, then download the new image as a new, separate file. The original file remains unmodified. Back to our greyscale example, here is what the `<canvas>` element will do:

- Draw the image to the canvas,
- Apply grayscale pixel by pixel,
- Export the result as a new image (base64 or blob),
- Let the user download or upload it.

Your `index.html` code, should look like this:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My JavaScript Project</title>
</head>
<body>

    <h1>The JavaScript Blueprint</h1>

    <input type="file" id="upload" />

    <a id="downloadLink"
        style="display: none; margin-top: 10px;">Download Image
    </a>

    <canvas id="canvas"
        style="display: block; margin-top: 10px;"></canvas>

    <script src="index.js"></script>
</body>
</html>

```

Add the following code to your JavaScript file (eg index.js)

```

const upload = document.getElementById('upload');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
const downloadLink = document.getElementById('downloadLink');

upload.addEventListener('change', function () {
    const file = this.files[0];
    const reader = new FileReader();

    reader.onload = function (event) {
        const img = new Image();
        img.onload = function () {
            canvas.width = img.width;
            canvas.height = img.height;
            ctx.drawImage(img, 0, 0);

            // Apply grayscale pixel by pixel
            const imageData = ctx.getImageData(
                0, 0, canvas.width, canvas.height
            );

            const data = imageData.data;

            for (let i = 0; i < data.length; i += 4) {
                const r = data[i];
                const g = data[i + 1];
                const b = data[i + 2];

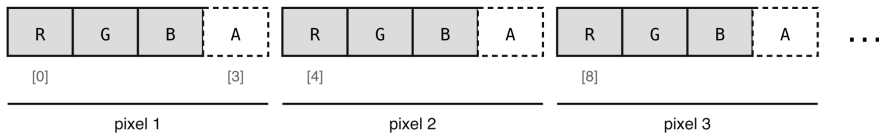
```

```
// Set all to avg for gray
    data[i] = data[i + 1] = data[i + 2] = avg;
  }

  ctx.putImageData(imageData, 0, 0);
```

Why the pixel loop counts in fours

imageData.data is one long flat list — four numbers per pixel.



```
for (let i = 0; i < data.length; i += 4) {
  const avg = (data[i] + data[i+1] + data[i+2]) / 3;
  data[i] = data[i+1] = data[i+2] = avg;
}
```

$i += 4$ steps one whole pixel at a time. `data[i+3]` — the dashed A boxes — is never written, because alpha is transparency, not colour. Average it in and the image would fade as well as grey.

- Figure 20.2 — Why the pixel loop counts in fours*

```
// Make it downloadable
const finalImage = canvas.toDataURL('image/png');
downloadLink.href = finalImage;
downloadLink.download = 'grayscale-image.png';
downloadLink.textContent = 'Download Grayscale Image';
downloadLink.style.display = 'inline-block';
};

img.src = event.target.result;
};

reader.readAsDataURL(file);
});
```

This is a very cool feature which can be used in a simple photo editing app so that a user can change an image to grey, and then save it on their computer, and, or upload it to a server.

Resizing an image using Canvas

This will let you shrink an image in the browser before uploading or displaying it. Let's add some styling to make the web page look nice and professional. Create a CSS file eg `index.css` in the same folder as your `index.html` file. Place this code in it:

```

body {
    font-family: Arial, sans-serif;
    padding: 30px;
    max-width: 600px;
    margin: auto;
    background: #f9f9f9;
    color: #333;
}

h1 {
    margin-top: 40px;
    color: blue;
}

#openWindowBtn {
    background-color: cornflowerblue;
}

input[type="file"] {
    display: block;
    margin-bottom: 20px;
}

label {
    font-weight: bold;
}

#scale {
    width: 100%;
    margin-top: 5px;
}

canvas {
    display: block;
    margin-top: 20px;
    border: 1px solid #ccc;
    border-radius: 8px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.05);
    max-width: 100%;
}

button {
    margin-top: 20px;
    padding: 10px 20px;
    background-color: #007acc;
    color: white;
    border: none;
}

```



```

    cursor: pointer;
    transition: background-color 0.3s ease;
  }

  button:hover {
    background-color: #005fa3;
  }

  #scaleValue {
    font-weight: normal;
    color: #007acc;
  }
}

```

Your index.html code, should look like this:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My JavaScript Project</title>
  <link rel="stylesheet" href="index.css">
</head>
<body>

  <h1>The JavaScript Blueprint</h1>

  <input type="file" id="upload" accept="image/*" />
  <br /><br />

  <button id="download">Download Resized Image</button>
  <br /><br />

  <label for="scale">Resize (%):
  <span id="scaleValue">100</span>%</label>
  <input
    type="range"
    id="scale"
    min="0.1" max="2" step="0.1"
    value="1">

  <br /><br />

  <canvas id="canvas"></canvas>

  <script type="module" src="index.js"></script>
</body>
</html>

```

Add the following code to your JavaScript file (eg index.js). Note all the comments between the lines to explain everything the code does:

```

// First, grab references to important
// HTML elements by their ID
// File input
const upload = document.getElementById('upload');

// Canvas to display image
const canvas = document.getElementById('canvas');

// Get the drawing context from canvas
const ctx = canvas.getContext('2d');

// Slider for resizing
const scaleInput = document.getElementById('scale');

// Download button
const downloadBtn = document.getElementById('download');

// Where we show the scale value (like "1.0")
const scaleValue = document.getElementById('scaleValue');

// We'll store the image here once it's loaded
let img = new Image();

// When the user selects an image file...
upload.addEventListener('change', function () {

    // Get the selected/uploaded file.
    // Store it in a variable 'file'
    const file = this.files[0];

    // Create a FileReader to read the file
    const reader = new FileReader();

    // When the file has been read...
    reader.onload = function (event) {

        // Create an image object. It has no picture in it yet -
        // it is an empty frame waiting for a src
        img = new Image();

        // Setting src starts a download,
        // which takes time, so we
        // say in advance what should
        // happen once it has finished.
        // Nothing is drawn until this fires
        img.onload = function () {
            // By now the picture really
            // is loaded, so draw it
            drawImageAtScale(scaleInput.value);
        };
    };
};

```

```

// everything above in motion.
    // The file was read as base64
    // text, which an Image can
    // take as a source just like a URL
    img.src = event.target.result;
};

    // Start reading the image file as a data URL
    // We do not assign file to img
    // because the Image() object doesn't
// accept a file directly. Instead,
// you read the file as a base64 URL,
// then assign it to the src attribute
// of img (the newly drawn image
// object)
    reader.readAsDataURL(file);
    downloadBtn.style.display = 'inline-block';
});

// This function draws the image on
// the canvas, resized based on the
// slider
function drawImageAtScale(scale) {
    // Convert slider value (string) to
    // a number so we can use that to
    // calculate percentages
    const scaleNum = parseFloat(scale);

    // Set canvas width based on scale
    canvas.width = img.width * scaleNum;

    // Set canvas height based on scale
    canvas.height = img.height * scaleNum;

    // Clear previous image
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // Draw the image at new size
    ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
}

// When the slider is moved...
scaleInput.addEventListener('input', function () {
    // Show the scale value next to
    // the slider. The slider holds a
    // MULTIPLIER (1 = full size),
    // but the label reads "%", so we
    // convert. Without this you
    // would see "Resize (%): 1.5%"
    // beside an image that had just grown by half.
    scaleValue.textContent = Math.round(this.value * 100);
});

```

```

drawImageAtScale(this.value);
});

// When the download button is clicked...
downloadBtn.addEventListener('click', function () {
    // Create a temporary link
    const link = document.createElement('a');

    // Set the filename for the download
    link.download = 'resized-image.png';

    // Convert canvas image to a downloadable URL
    link.href = canvas.toDataURL();

    // Trigger the download dynamically
    link.click();
});

```

If it seems as if the code is complicated, do not be daunted by the length of it, it really isn't hard. Here is some more explanation on how it all works:

- You create a `FileReader` and tell it what to do when it's done reading the file. Here is the code, but it does not run straight away:

```

reader.onload = function (event) {
    // We'll do stuff after the file is read here
};

```

- You start the file-reading process:

```

// Asynchronous operation
reader.readAsDataURL(file);

```

- JavaScript doesn't block or wait. It moves on and comes back later when the file is fully read.
 - When it's finally read, the `reader.onload` function gets triggered with the result. Inside this same function:

```

reader.onload = function (event) {
    ...
}

```

is where you should then set:

```

img.src = event.target.result;

```

* In summary; here are the steps:

- `reader.readAsDataURL(file)` triggers the reading.
- Only after the reading is done does `reader.onload` run.
- Inside that callback, we assign that file value to `img.src`.

The `img.src = event.target.result`; doesn't run immediately — it only runs after the `FileReader` has finished loading the file and fired the `onload` event. The key in grasping this process lies in understanding that the event that is triggered when a file is uploaded has a `result` property which is the uploaded file itself. That is why when we listen in the code for that upload event like so:

```
reader.onload = function (event) {  
    // ...  
}
```

It is the event (file upload) object's `target.result` that we assign to the `src` attribute of `img`. We are hereby pointing the `src` attribute of the newly drawn (created) image to the location of the uploaded image.

Even though it looks like we are assigning the uploaded image to the `src` attribute of the new canvas image (`img.src = ...`) before we have read the data from the uploaded image-by calling `readAsDataURL(file)`, I want to assure you that the call to `readAsDataURL(file)` really happens first. Only after this call is done does the new image get its `src` attribute value. Here is where the trick is; JavaScript handles this whole process in a non-blocking (asynchronous) way using event listeners. This is a common theme you'll see in many real-world JS apps. This is called asynchronous programming. In the HTML code, where we have the range (slider) input field. The value of that field's `step` attribute controls how much we want to incrementally resize the image-like zooming in or out. Making its value 0.1 as we have done means that we will be able to use values like 0.5 or 1.5 so that the image is resized by a percentage (e.g., 50% smaller or 150% bigger). If we had used big numbers like 100 or 200, the image would have become huge (100 times bigger!), which your browser might not be able to display, making it look like nothing happened. So we stick to small numbers between 0.1 and 2 for safe and smooth resizing.

Rotate an image in 2d

With the Canvas you can rotate an image, and it's a nice thing to be able to do with your mages. We will look at a full working example with a button that allows you upload an image, and another button which rotates the image in increments of 45 degrees clockwise. When you are happy with the rotation, you can download the image as usual. The rotation done is in 2D. Canvas alone does not support real 3D transforms. It's strictly 2D. To give you a tip; in CSS, you can simulate a 3D flip (rotation) effect on an image using the `transition` and `transform` properties (just visually, not on canvas). For real 3D rendering in JavaScript, you would need WebGL or a 3D library like Three.js. Let's look at how to rotate an image in 2D with JavaScript.

HTML code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My JavaScript Project</title>
  <link rel="stylesheet" href="index.css">

</head>
<body>

  <h1>The JavaScript Blueprint</h1>

  <input type="file" id="upload">
    <button id="rotateBtn">Rotate Image</button>
    <button id="downloadImgBtn">Download Image</button>
    <canvas id="canvas"></canvas>

  <script type="module" src="index.js"></script>
</body>
</html>
```

index.js code

```
const upload = document.getElementById('upload');
const rotateBtn = document.getElementById('rotateBtn');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
const downloadBtn = document.getElementById('downloadImgBtn');

let img = new Image();
// Keep track of rotation in degrees
let rotation = 0;

// When an image is uploaded
upload.addEventListener('change', function () {
  const file = this.files[0];
  const reader = new FileReader();

  reader.onload = function (event) {
    img = new Image();
    img.onload = function () {
      rotation = 0; // Reset rotation
      drawRotatedImage();
    };
    img.src = event.target.result;
  };

  reader.readAsDataURL(file);
```

```

});

// When the rotate button is clicked
rotateBtn.addEventListener('click', function () {
// Rotate by 90 degrees each time
rotation = (rotation + 90) % 360;
drawRotatedImage();
});

// Function to draw the image at the current rotation
function drawRotatedImage() {
const angleInRadians = rotation * Math.PI / 180;

// Adjust canvas size based on
// rotation (to fit image properly)
if (rotation % 180 === 0) {
canvas.width = img.width;
canvas.height = img.height;
} else {
canvas.width = img.height;
canvas.height = img.width;
}

ctx.clearRect(0, 0, canvas.width, canvas.height);
ctx.save();

// Move the canvas "zero point" to the center
ctx.translate(canvas.width / 2, canvas.height / 2);

// Rotate the canvas
ctx.rotate(angleInRadians);

// Draw the image from the new
// origin, with image centered
ctx.drawImage(img, -img.width / 2, -img.height / 2);

// Go back to normal canvas settings
ctx.restore();
}

```

Use the same CSS code as all the other examples-in a file in the same directory as this index.js and index.html. Here are important points to understand about the working of the above code:

- `ctx.translate()` moves the canvas "zero point" to the center so we can rotate around the middle.
- `ctx.rotate()` expects radians, not degrees, so we use `degrees * Math.PI / 180`.
- We reverse the translate/rotate with `ctx.restore()` so future drawings are not affected.

By default, the canvas starts drawing from the top-left corner (0, 0). So if you rotate from there, the image will spin off the canvas or get cut off.

What does `ctx.translate(canvas.width / 2, canvas.height / 2)` mean? You're moving the canvas's drawing center to the middle — like saying: “Hey canvas, pretend the center of this image is now the (0, 0) point.” That way, when you rotate, the image spins around its own center, not from the corner. What does `ctx.drawImage(img, -img.width / 2, -img.height / 2)` mean? This draws the image starting halfway left and halfway up from the center. If you didn't do this, the image would rotate but still be drawn from the top-left — which would cause it to appear off-center or clipped. So in human terms: “First move to the center of the canvas. Then, rotate. Then place the image so that its own center lines up with the canvas center.”

This is a very common pattern when rotating things in canvas: `translate` → `rotate` → `draw` → `restore`.

Add feature to download the rotated image

You are able to download your rotated image easily just like we did with the example of the resized images because it was done on the canvas. Adding the ability to download the resized image will give you a nice feature which you can build and share with your family and friends to use. Achieving this is as simple as adding the following two pieces of code—a download button in your HTML code, and some code in your JavaScript file to listen for a click event on that download button, and download the already created image from the canvas. Here is the code:

html (eg index.html)

```
<button id="downloadImgBtn">Download Image</button>
```

JavaScript (eg index.js)

Add this code to the already existing code above for rotating an image:

```
const downloadBtn = document.getElementById('downloadImgBtn');

// ...

downloadBtn.addEventListener('click', function () {
  // Create a temporary link
  const link = document.createElement('a');

  // Set the filename for the download
```



```
// Convert canvas image to a downloadable URL
link.href = canvas.toDataURL();

// Trigger the download dynamically
link.click();
});
```

From the previous example, it should be clear to you how that download code works. If not just refer back to the explanations in the previous examples.

Adding image filters

Adding filters to images using the canvas is quite straight forward. You just have to add the relevant filter by calling its method and assigning its result to the context of the image canvas. Available filter methods are `brightness()`, `contrast()`, `blur()`. Without further ado, let's see them in action.

Increase the brightness of an image

As usual, we will now look at code that allows you to upload an image of your choice, adjust its brightness using a range slider, then download the desired result to your computer. To brighten an image, we just need to add the brightness filter to the canvas context. Let's dive right into the code:

HTML (eg index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>The JavaScript Blueprint</title>
  <link rel="stylesheet" href="index.css">
</head>
<body>

  <h1>Brighten your image</h1>

  <input type="file" id="upload" accept="image/*" />
  <br /><br />

  <button id="downloadImgBtn">Download Image</button>

  <input type="range" id="brightness" min="0" max="2"
    step="0.1" value="1">

  <canvas id="canvas"></canvas>
```

```
<script type="module" src="index.js"></script>
</body>
</html>
```

JavaScript code (eg index.js)

```
const brightnessSlider = document.getElementById('brightness');
const downloadBtn = document.getElementById('downloadImgBtn');
const upload = document.getElementById('upload');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let img = new Image();

upload.addEventListener('change', function () {
  const file = this.files[0];

  // Create a FileReader to read the file
  const reader = new FileReader();

  // When the file has been read...
  reader.onload = function (event) {

    // Create an empty image object
    img = new Image();

    img.onload = function () {
      canvas.width = img.width;
      canvas.height = img.height;
      draw();
    };

    // img.src = 'your-image.jpg';
    // // Or load from file input
    img.src = event.target.result;
  }
  reader.readAsDataURL(file);
});

brightnessSlider.addEventListener('input', draw);

function draw() {
  ctx.filter = `brightness(${brightnessSlider.value})`;
  ctx.drawImage(img, 0, 0);
}

downloadBtn.addEventListener('click', function () {
  // Create a temporary link
  const link = document.createElement('a');
```

```

link.download = 'brightened-image.png';

// Convert canvas image to a downloadable URL
link.href = canvas.toDataURL();

// Trigger the download dynamically
link.click();
});

```

The line of code that does the magic is the line in the `draw()` function where we call the `brightness()` filter function and assign its result to the `filter` property of the canvas context (`ctx.filter`). Here is the line:

```
ctx.filter = `brightness(${brightnessSlider.value})`;
```

Understand that this works because the canvas context which we create at the beginning of every canvas job is the thing on which the dynamically created image is built. Changing the filter on the context therefore automatically makes the change on your image that is drawn on it.

Adding contrast to an image

As with brightening up an image, let's look at a code example that allows you to upload an image, adjust its brightness using a range slider, then download the desired result to your computer. To add contrast to an image, we just need to add the contrast filter to the canvas context. Let's take a look:

HTML (eg `index.html`)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>The JavaScript Blueprint</title>
  <link rel="stylesheet" href="index.css">
</head>
<body>
  <h1>Add contrast to your image</h1>

  <input type="file" id="upload" accept="image/*" />
  <br /><br />

  <button id="downloadImgBtn">Download Image</button>

  <input
    type="range"
    id="contrast"

```

```
<canvas id="canvas"></canvas>
```

```
    <script type="module" src="index.js"></script>
</body>
</html>
```

I would recommend you set up your slider input field with the following: min="0" max="3" step="0.1" value="1". A max range of 3 will give you more contrast room to play with.

JavaScript code (eg index.js)

```
const contrastSlider = document.getElementById('contrast');
const downloadBtn = document.getElementById('downloadImgBtn');
const upload = document.getElementById('upload');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let img = new Image();
```

```
upload.addEventListener('change', function () {
  const file = this.files[0];
```

```
  // Create a FileReader to read the file
  const reader = new FileReader();
```

```
  // When the file has been read...
  reader.onload = function (event) {
```

```
    // Create an empty image object
    img = new Image();
```

```
    img.onload = function () {
      canvas.width = img.width;
      canvas.height = img.height;
      draw();
    };
  }
```

```
  // img.src = 'your-image.jpg';
  // // Or load from file input
  img.src = event.target.result;
```

```
  }
  reader.readAsDataURL(file);
});
```

```
contrastSlider.addEventListener('input', draw);
```

```
function draw() {
  ctx.filter = `contrast(${contrastSlider.value})`;
  ctx.drawImage(img, 0, 0);
}
```

```

}

downloadBtn.addEventListener('click', function () {
  // Create a temporary link
  const link = document.createElement('a');

  // Set the filename for the download
  link.download = 'contrasted-image.png';

  // Convert canvas image to a downloadable URL
  link.href = canvas.toDataURL();

  // Trigger the download dynamically
  link.click();
});

```

Add a blur filter

As with brightening or adding contrast to an image, the following code example will allow you to upload an image, adjust its blur level using a range slider, then download the desired result to your computer. To add a blur filter to an image, add the blur filter to the canvas context. Here is how:

HTML (eg index.html)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>The JavaScript Blueprint</title>
  <link rel="stylesheet" href="index.css">
</head>
<body>
  <h1>Add blur to your image</h1>

  <input type="file" id="upload" accept="image/*" />
  <br /><br />

  <button id="downloadImgBtn">Download Image</button>

  <input
    type="range"
    id="blur"
    min="0" max="10" step="1" value="0">

  <canvas id="canvas"></canvas>

```

```
<script type="module" src="index.js"></script>
</body>
</html>
```

I would recommend you set up your slider input field with the following: `min="0" max="10" step="1" value="0"`. The max of 10 gives you more room to fine-tune the blur filter effect.

JavaScript code (eg index.js)

```
const blurSlider = document.getElementById('blur');
const downloadBtn = document.getElementById('downloadImgBtn');
const upload = document.getElementById('upload');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let img = new Image();

upload.addEventListener('change', function () {
  const file = this.files[0];

  // Create a FileReader to read the file
  const reader = new FileReader();

  // When the file has been read...
  reader.onload = function (event) {

    // Create an empty image object
    img = new Image();

    img.onload = function () {
      canvas.width = img.width;
      canvas.height = img.height;
      draw();
    };

    // img.src = 'your-image.jpg';
    // // Or load from file input
    img.src = event.target.result;
  }
  reader.readAsDataURL(file);
});

blurSlider.addEventListener('input', draw);

function draw() {
  ctx.filter = `blur(${blurSlider.value}px)`;
  ctx.drawImage(img, 0, 0);
}
```

```
downloadBtn.addEventListener('click', function () {
  // Create a temporary link
  const link = document.createElement('a');

  // Set the filename for the download
  link.download = 'blurred-image.png';

  // Convert canvas image to a downloadable URL
  link.href = canvas.toDataURL();

  // Trigger the download dynamically
  link.click();
});
```

Notice that the `blur()` filter function accepts its value in pixels, hence in the `draw()` function above, we append 'px' to the value we pass to `blur()` like so:

```
ctx.filter = `blur(${blurSlider.value}px)`;
```

Bonus tip - Combining filters

You can combine filters like this:

```
ctx.filter = `brightness(1.2) contrast(1.5) blur(2px)`;
ctx.drawImage(img, 0, 0);
```

This will apply all three filters at once!. Let's just go ahead and create an application for that-editing an image using all three filters working together. The following code is a working solution of all three filters. As always, the user can upload an image, and after the upload, the image is previewed in a canvas, with three sliders available below to apply all three filters; brightness, contrast and blur, separately to the same image and see all changes applied live. When they are happy with the result, they can hit on the Download Image button to save the edited file to their machine. Here is the code:

CSS styling for the web page

```
body {
  font-family: Arial, sans-serif;
  padding: 20px;
  background: #f2f2f2;
  text-align: center;
}

#canvas {
  margin-top: 20px;
  border: 1px solid #ccc;
  max-width: 100%;
}
```

```

.controls {
  margin-top: 20px;
  display: flex;
  flex-direction: column;
  gap: 15px;
  max-width: 400px;
  margin-inline: auto;
}

.control-group {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

label {
  flex: 1;
  text-align: left;
}

input[type="range"] {
  flex: 2;
}

button {
  margin-top: 20px;
  padding: 10px 20px;
  background-color: #007acc;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 16px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: #005fa3;
}

```

HTML (eg index.html)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>The JavaScript Blueprint</title>
  <link rel="stylesheet" href="index.css">

</head>
<body>

```



```

<input type="file" id="upload" accept="image/*" />
  <br /><br />

  <button id="downloadImgBtn">Download Image</button>

  <canvas id="canvas"></canvas>

  <div class="controls">
    <div class="control-group">
      <label for="brightness">Brightness</label>
      <input type="range" id="brightness" min="0"
max="200"
value="100">
    </div>

    <div class="control-group">
      <label for="contrast">Contrast</label>
      <input type="range" id="contrast" min="0"
max="200"
value="100">
    </div>

    <div class="control-group">
      <label for="blur">Blur</label>
      <input type="range" id="blur" min="0" max="10"
value="0"
step="0.1">
    </div>
  </div>

  <script type="module" src="index.js"></script>
</body>
</html>

```

Note as usual, that the controls of the range slider input can be slightly different, depending on the filters. In this example, they are as follows:

- For brightness, min is 0, max is 200, value is 100
- For contrast, min is 0, max is 200, and value is 100
- For blur, the min is 0, max is 10, and value is 0.1

JavaScript code (eg index.js)

```

const downloadBtn = document.getElementById('downloadImgBtn');
const upload = document.getElementById('upload');
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

const brightnessInput = document.getElementById('brightness');
const contrastInput = document.getElementById('contrast');

```

```

const blurInput = document.getElementById('blur');

let img = new Image();

// When an image is uploaded
upload.addEventListener('change', function () {
  const file = this.files[0];
  const reader = new FileReader();

  reader.onload = function (e) {
    img = new Image();

    img.onload = function () {
      // Set canvas size
      canvas.width = img.width;
      canvas.height = img.height;

      // Initial draw
      applyFilters();
    };

    img.src = e.target.result;
  };

  reader.readAsDataURL(file);
});

// Attach filter sliders to re-draw function
brightnessInput.addEventListener('input', applyFilters);
contrastInput.addEventListener('input', applyFilters);
blurInput.addEventListener('input', applyFilters);

// Function to draw the image with current filter settings
function applyFilters() {
  const brightness = brightnessInput.value;
  const contrast = contrastInput.value;
  const blur = blurInput.value;

  // Set the CSS filter style before drawing
  ctx.filter = `
  brightness(${brightness}%)
  contrast(${contrast}%)
  blur(${blur}px)
  `;

  // Clear and redraw image
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
}

```

```
// Create a temporary link
const link = document.createElement('a');

// Set the filename for the download
link.download = 'blurred-image.png';

// Convert canvas image to a downloadable URL
link.href = canvas.toDataURL();

// Trigger the download dynamically
link.click();
});
```

In all these image manipulation demos, you have learned so much, including the following: - [] How to use the `<input type="range">` to create sliders - [] How to listen to slider events with `.addEventListener('input', ...)` - [] How to apply multiple filters at once using the `ctx.filter` property - [] How to work with `<canvas>` and `FileReader` - [] How to dynamically update image rendering based on user input

Do not worry much if you do not fully grasp the concepts explained in this image management chapter. Proceed with your learning of JavaScript in the other chapters, and it will click, and there will come a point when the dots begin to all connect. This is a promise. I was once that novice who did not have a clue. This book is not meant to be a one-time read anyway, but rather, a handy reference for whenever you want to refresh your mind on a topic. Even the most experienced programmers often have to look up references to refresh their knowledge. If it encourages you, the canvas is so rich and fun that it needs a whole separate book of its own. But what I have shown you, most JavaScript books will not go into. But because this book is all about showing you the potential of JavaScript as a programming language, it felt only natural that I should talk about image management.

QUIZ — Images

This page contains the Q & A (questions and answers) for this chapter — Chapter 20: Images. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 12 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. You apply a grayscale effect with `img.style.filter = 'grayscale(100%)'`. Why can the user not download the grey version of that image?

Clue: think about what you actually changed.

2. So what do you use instead when the change has to be permanent, and why does that work?

Clue: one HTML element, and it holds real pixels.

3. In the preview example, why is `URL.createObjectURL(file)` enough, with no `FileReader` in sight?

Clue: you are not reading the file, only pointing at it.

4. This line appears in the chapter. What does it do, and why must the code that uses the image live inside it?

```
img.onload = function () { ... };
```

Clue: setting `src` starts something that takes time.

5. Walk through the four steps of making a canvas edit downloadable.

Clue: draw, edit, export, link.

6. In the pixel loop below, why does the counter go up in fours, and why is `data[i + 3]` never touched?

```
for (let i = 0; i < data.length; i += 4) {  
  const avg = (data[i] + data[i+1] + data[i+2]) / 3;  
  data[i] = data[i+1] = data[i+2] = avg;  
}
```

Clue: each pixel is four numbers, and one of them is not a colour.

7. Two filters are set like this. One of them would silently do nothing if you copied the pattern of the other. Which, and why?

```
ctx.filter = `contrast(${slider.value})`;
ctx.filter = `blur(${slider.value}px)`;
```

Clue: look at what is on the end.

8. The chapter's grayscale button does this. Spot the bug.

```
btn.addEventListener('click', applFilter);

function applyFilter() { ... }
```

Clue: read the two names side by side, slowly.

9. EXERCISE. Write the toggle that switches an image between full grayscale and none each time a button is clicked, using a ternary.

Clue: read the current value, then set the opposite.

10. EXERCISE. Given a 2×1 canvas with one red pixel and one blue pixel, apply the averaging grayscale from question 6 and print the resulting pixel values.

Clue: 255 divided by 3.

11. EXERCISE. Write the download handler: turn the canvas into a data URL and trigger a download named `edited-image.png`.

Clue: three lines and a click.

12. EXERCISE. A resize slider holds a multiplier from 0.1 to 2, but its label reads "%". Write the line that displays it correctly.

Clue: 1 should read as 100.

ANSWERS

1. Because a **CSS filter only changes how the image is displayed**. It does not touch the image data at all. The browser paints the picture differently on screen, but the file behind it is exactly as it was.

So there is nothing new to download. If you saved that image you would get the original, in full colour.

That is not a failing — CSS filters are perfect when all you want is a visual effect, and they are cheap and instant. They are simply the wrong tool when the change has to be kept.

2. You use the `<canvas>` element.

A canvas holds actual pixels that you can read and write. When you draw an image onto it and then change those pixels, the change is real: it exists in the canvas, not just in how the browser is rendering something.

From there you can export the canvas as a new image with `toDataURL()` and offer it for download.

One thing to hold on to, because it explains the whole shape of this chapter: JavaScript cannot modify the original file on disk. What actually happens is that you **read** the original, **rebuild** it on a canvas, and **save that as a new file**. The original is never touched.

3. Because `URL.createObjectURL()` does not read the file's contents — it just creates a short temporary address that points at the file already sitting in the browser's memory.

```
preview.src = URL.createObjectURL(file);
```

An `<img>` only needs an address, so that is all you have to give it. No reading, no waiting, no callback.

You need a `FileReader` when you actually want the **contents** — the text of a file, or base64 data you are going to feed to an `Image` object before drawing it on a canvas.

One habit worth keeping: an object URL stays in memory until you release it with `URL.revokeObjectURL()`.

4. Setting `img.src` **starts a load**, and loading takes time — even for a base64 string. `img.onload` is the callback that runs once the picture is genuinely there.

That matters because until it fires, the image has no real width or height and nothing to draw. Code like this, placed after `img.src = ...`, would run too early:

```
img.src = event.target.result;
canvas.width = img.width;
// 0 - the image has not loaded yet
// draws nothing
ctx.drawImage(img, 0, 0);
```

So the order looks backwards but is not: you say what should happen *when* it loads, and only then give it a `src` to set the loading off.

```

5. // 1. draw the image onto the canvas
   ctx.drawImage(img, 0, 0);

   // 2. edit the pixels (or set
   // ctx.filter before drawing)
   const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
   // ...change imageData.data...
   ctx.putImageData(imageData, 0, 0);

   // 3. export the canvas as an image
   const finalImage = canvas.toDataURL('image/png');

   // 4. hand it to a link with a download attribute
   downloadLink.href = finalImage;
   downloadLink.download = 'grayscale-image.png';

```

`toDataURL()` gives you a base64 string beginning

`data:image/png;base64,...`, which a link can use as its `href` exactly like an ordinary URL.

6. **The counter goes up in fours because each pixel is four numbers**, laid out end to end in one long list:

[R, G, B, A, R, G, B, A, R, G, B, A, ...] pixel 1 pixel 2 pixel 3

So `data[i]` is red, `data[i+1]` green, `data[i+2]` blue, and `data[i+3]` is the **alpha** — how opaque that pixel is.

Alpha is left alone because it is not a colour. Averaging it in would make the picture grey

- and* partly transparent. By skipping it, the image keeps exactly the transparency it had.

7. **`blur()` is the one that would silently fail** if you wrote it like `contrast()`.

`contrast()` takes a plain number: `contrast(1)` means unchanged, `contrast(1.5)` means 150%. No unit needed.

`blur()` takes a **length**, so it needs a unit. `blur(5)` is not valid CSS, and an invalid filter string is ignored — the image is simply drawn with no blur at all, and nothing warns you. That is why the chapter appends `px`:

```
ctx.filter = `blur(${blurSlider.value}px)`;
```

A filter that quietly does nothing is harder to debug than one that throws, so this is worth remembering.

8. The listener names **`applFilter`** while the function is called **`applyFilter`** — the `y` is missing.

This does not fail on click. It fails immediately, when that line runs, because JavaScript looks for a variable called `applFilter` and there is no such thing:

ReferenceError: applFilter is not defined

Two names that differ by one letter are hard to spot, which is a good argument for letting your editor autocomplete function names rather than typing them again.

```
9. const btn = document.getElementById('grayScaleBtn');
   btn.addEventListener('click', toggleFilter);

   function toggleFilter() {
       const img = document.getElementById('myImage');

       img.style.filter =
           img.style.filter === 'grayscale(100%)'
               ? 'grayscale(0%)'
               : 'grayscale(100%)';
   }
```

Read the ternary as a sentence:

is it currently full grayscale? then set it to none, otherwise set it to full.

The first click works because the filter starts as an empty string, which is not `'grayscale(100%)'`, so it takes the second branch. `'none'` would do just as well as `'grayscale(0%)'`.

```
10. const canvas = document.createElement('canvas');
    canvas.width = 2;
    canvas.height = 1;
    const ctx = canvas.getContext('2d');

    ctx.fillStyle = 'rgb(255,0,0)';    ctx.fillRect(0, 0, 1, 1);
    ctx.fillStyle = 'rgb(0,0,255)';    ctx.fillRect(1, 0, 1, 1);

    const imageData = ctx.getImageData(0, 0, 2, 1);
    const data = imageData.data;

    for (let i = 0; i < data.length; i += 4) {
        const avg = (data[i] + data[i+1] + data[i+2]) / 3;
        data[i] = data[i+1] = data[i+2] = avg;
    }

    console.log(data[0], data[1], data[2]);
    // 85 85 85
    console.log(data[4], data[5], data[6]);
    // 85 85 85
```

Both pixels come out as **85**, because $255 \div 3$ is 85 whether the 255 was in the red slot or the blue one. Two very different colours land on the same grey — which

I tools convert to greyscale. They weight the channels, because our eyes are far more sensitive to green than to blue.

```
11. downloadBtn.addEventListener('click', function () {  
    const link = document.createElement('a');  
  
    link.download = 'edited-image.png';  
    link.href = canvas.toDataURL();  
  
    link.click();  
});
```

The link never has to be added to the page. Creating it, pointing it at the data URL and clicking it in code is enough to start a download.

Note that `download` is what makes the browser save the file rather than navigate to it, and it is also where the filename comes from.

```
12. scaleValue.textContent = Math.round(this.value * 100);
```

The slider holds a **multiplier** — `1` means full size, `0.5` means half, `2` means double. The label says `%`. Printing the raw value beside a percent sign gives you "Resize (%): 1%" next to an image at its original size, which is confusing.

Multiplying by 100 makes the two agree: `1` reads as `100`, `0.5` reads as `50`. `Math.round` tidies away the floating-point dust that a step of `0.1` can produce.

Chapter 21 — Error Debugging and Testing

- Displaying values on screen
 - `document.write()`
 - `innerHTML`
 - `alert()`
 - `confirm()`
 - `prompt()`
 - `console.log()`
- The browser developer tools
- Stopping code execution for debugging
- Using the Browser Tools Network Tab
- Throwing and handling exceptions
 - The throw statement
 - Custom exceptions
 - Asynchronous exception handling
 - Using `.catch()`
 - Using `async/await`
 - Multiple catch blocks
 - Conclusion and exception handling examples
- Testing

This discusses the tools made available by your programming language to find bugs in your code. It also talks about conventions and best practices for writing reliable, performant and fault-tolerant applications. This is also a section where performance pitfalls are identified and work-arounds given.

Displaying values on screen

All programming languages offer you built-in tools which could be properties or functions to use in displaying stuff on the screen. These are very handy as you can use them in:

- i) the working of your application to dynamically write text or display data on screen, for example in certain parts of your web page,
- ii) or you can also use them to inform/warn the user of something or even prompt the user to enter some information
- iii) or you can use them during development to print out data held in maybe a variable in order to view its contents so you know what type of data you are dealing with, or to help you debug a part of your web page that is not working properly.

JavaScript has its share of these properties/functions and here are the common ones:

```
-document.write()  
-innerHTML  
-alert()  
-confirm()  
-prompt()  
-console.log()
```

We will be using them a lot in the code examples in this book to show the results we are getting. You will find that I particularly make use of `console.log()` and `alert()` the most.

i) `document.write()`. To write text on screen, use the `write()` method on the document object:

```
document.write("String to write here");
```

However, using `document.write()` to write text to a web document is considered bad practice in modern JavaScript. This is because there is a good risk that `document.write()` can overwrite (erase) the entire page if it's called after the page has finished loading. What is recommended is the `innerHTML` property of HTML elements. For example, instead of doing:

```
document.write("My data here");
```

It is better to append the content like this:

```
document.body.innerHTML += "My data here";
```

Take note of the `+=` that is used instead of just `=`. This ensures that if you are looping for example, and wish to display several values on screen, each subsequent value being written will be displayed without completely wiping what was previously there. Basically, the `+=` makes sure that each new line is added to what's already there, instead of replacing it.

ii) `innerHTML` To display text (also known as a string) inside an element on your web page, you can use the very popular `innerHTML` property of document elements:

```
selectedElement.innerHTML = "String to display in that element";
```

As explained above, it is recommended over using `document.write()`. Using `document.body.innerHTML` lets you safely update the page content without disrupting the rest of your page. Why does `document.write()` do this:

- When the browser finishes loading the HTML, it closes the document stream.
- If `document.write()` is used after that, it reopens the stream, which deletes everything on the page and writes fresh content.
- So it disrupts the existing structure, styles, and scripts.

The better alternative is to use

```
document.body.innerHTML += 'something'
```

Because of the following points: - It appends content safely to the page. - It keeps the rest of your HTML intact. - It doesn't cause errors or wipe the screen.

iii) `alert()` To display a popup on screen to inform or warn the user of something and have them click OK to dismiss the popup, use the `alert()` function:

```
alert("String to display here");
```

iv) `confirm()` To display a popup that will require a user to confirm an action by clicking on 'Confirm' in order to proceed, or abort the action by clicking on 'Cancel', use the `confirm()` function. Ideally, you will capture (store) the user's response by assigning `confirm()` to a variable. The value of that variable will then contain `true` if they clicked on 'Confirm', or `false` if they clicked on 'Cancel'. Your program will then check for the value (of this variable) in order to know how to proceed. Create a confirm popup and store the user's response to this in a variable called `response` like so:

```
let response = confirm("Are you sure?");
```

Check for the value of `response` and act accordingly:

```
if (response) {  
    // the user clicked on Ok  
    alert('You have confirmed!');  
} else {  
    // the user clicked on Cancel  
    alert('You said no!');  
}
```

v) `prompt()` To retrieve some information from the user through a popup text box, use the good old `prompt()` function. Just like with the confirm popup, you can capture what the user enters by assigning the prompt to a variable like so:

```
let idNumber = prompt(
  "Write some text for the user like: What is your ID number"
);
```

You can optionally pass a second parameter as a string, which fills the text field in with a starting value, e.g.

```
prompt('Enter surname', 'Enter surname here');
```

The text 'Enter surname' will be shown to the user above the data input field, in the same way a label is displayed beside the input field of an HTML form. The second string, 'Enter surname here', is the DEFAULT VALUE of the field. Be careful not to think of it as a placeholder. A placeholder is grey hint text that vanishes the moment you type; a default is real text already sitting in the box, and if the user presses OK without touching it, that is exactly what you get back.

vi) The console object. Another means to display information and much more, to yourself as the developer is to write to the console using the console object and its family of functions. Here is a list of them: `-console.log()` `-console.error()` `-console.warn()` `-console.info()` `-console.debug()` `-console.table()` `-console.dir()` `-console.assert()` `-console.group()` and `console.groupEnd()` `-console.time()` and `console.timeEnd()`

console.log()

This is the most-commonly used method of the console object used to print something to the browser's console.

```
let username = "Alice";
console.log("Current user:", username);
```

It's purpose: general purpose Arguments: You can pass strings, numbers, variables, objects, arrays, and even multiple values separated by commas:

```
console.log("x:", x, "y:", y);
```

Use Case: Checking values during development, tracing flow, or understanding what's going on in your program.

Other console Methods and Their Uses

console.error()

Purpose: Displays an error message, usually in red text in the console. Example:

```
console.error("Something went wrong!");
```

Use Case: Reporting serious issues like failed API calls, missing files, or exceptions. Benefit: Makes it easy to visually identify problems in logs.

console.warn()

Purpose: Highlights a warning in yellow, without treating it like a full-blown error.

Example:

```
console.warn("This is just a warning.");
```

Use Case: When something is suspicious but not necessarily fatal (like deprecated features).

console.info()

Purpose: Prints information messages.

Example:

```
console.info("This is some useful info.");
```

Use Case: Not as commonly used, but can be useful for providing insights during debugging. Appears similar to `log()` in most browsers

console.debug()

Purpose: Specifically for debugging messages.

Example:

```
console.debug("This is a debug message.");
```

Use Case: Used when you want something that can be hidden in some browsers unless "Verbose" logging is turned on. Note: May not always be visible unless you expand the console log level settings.

console.table()

Purpose: Displays tabular data as a formatted table.

Example:

```
const users = [
  { name: "Alice", age: 25 },
  { name: "Bob", age: 30 },
];
```

```
console.table(users);
```

Use Case: When printing arrays of objects or data collections. Super useful!

console.dir()

Purpose: Displays an interactive list of the properties of a specified JavaScript object. Example:

```
console.dir(document.body);
```

Use Case: Useful for exploring DOM objects and nested structures.

console.assert()

Purpose: Only logs the message if the assertion fails. Example:

```
console.assert(2 + 2 === 5, "Math is broken!");
```

Use Case: Helpful for adding sanity checks during development.

console.group() and console.groupEnd()

Purpose: Groups console messages together.

Example:

```
console.group("User Details");  
console.log("Name: Alice");  
console.log("Age: 25");  
console.groupEnd();
```

Use Case: Organize output when logging a bunch of related data.

console.time() and console.timeEnd()

Purpose: Measures how long an operation takes.

Example:

```
console.time("loop");  
for (let i = 0; i < 100000; i++) {}  
console.timeEnd("loop");
```

Use Case: Performance testing.

The browser developer tools

It is very handy to use a debugging tool, and almost all browsers have one built into them that you can use. Here is how to visit the error console for different browsers:

- In Firefox, go to Tools > Error console

- In Opera, go to Tools > Advanced > Error console
- In Microsoft Internet Explorer, go to Tools > Internet Options > Advanced > then uncheck the Disable Script Debugging box, then check the box 'Display a Notification about Every Script Error.'
- In Google Chrome, if you click on the three dots (...) on the top-right corner of the window > then choose More Tools > Developer Tools, that will take you to the debugger console.
- Safari has no Error console enabled, but you can turn it on by: Safari > Preferences > Advanced > Show develop menu in Menu bar

Alternatively, you can use the Firebug Lite JavaScript module which is easier to use. To use it, place the following code in your HTML, right before the body tag:

```
// tinyurl.com/fblite'></script>
<script src='http:
```

The way the browser debuggers work is to display the line of the code and hint you on the cause whenever you run code that has some kind of error in the syntax. Firefox is the best for debugging JS as it reports a clearer message. There is also a plug-in in Firefox for JavaScript known as FireBug which is quite popular and very recommended to use. Learn about it and acquire it here:

(<http://getfirebug.com>)

Stopping code execution for debug

It is possible when running some code, to set points (aka breakpoints) in the code where you want the execution to stop. This is when you wish to stop the execution in separate sections to investigate at what point in the code something-usually an error, is occurring. This is usually made possible as a tool in IDEs (Integrated Development Environments). In IDE is basically an advanced text editor for writing code which has evolved over the years to add more features that go beyond just writing code (text). VS Code is one of such IDEs, and you can see how within VS Code you can do more than just type out code. You can create and run a simple server to test your code as we did with the 'Live Server' extension in VS Code. Within most IDEs including VS Code, you also have an integrated Terminal application which means you can navigate or connect eg via SSH to your server (local or remote) and run commands, all without leaving your code/text editor. This is how such modern code editors came to earn the name of Integrated Development Environment (IDE). Let's look at two ways in which you can place break points in your code.

a) Stopping Code Execution for Debugging in JavaScript You can pause JavaScript code in the browser which will be picked up by the break point tools in the browser DevTools. Just type 'debugger'; at the line you want execution to stop at.

Wherever you place that line, the browser will pause execution when it gets there. The DevTools in your browser have to be open for that to happen. For example:

```
function checkUser() {  
    let name = "Alice";  
    // Pause here when DevTools are open  
    debugger;  
    console.log("Name is", name);  
}
```

- Code stops at 'debugger' line when DevTools is open
- You have to be on the Sources tab in DevTools
- From the DevTools in your browser, you will then be able to step in and out of, or skip over functions.
- With each click on the to 'Step into function' will keep skipping to the next line.

This is great for temporarily inspecting variables without needing to manually set breakpoints in DevTools.

b) Using Breakpoints in Browser DevTools Without writing code to stop execution, you can achieve the same thing directly in DevTools of your browser. To do so, follow these steps:

- Open your browser's Developer Tools (usually with F12 or right-click > "Inspect").
- Go to the "Sources" tab.
- Open the JavaScript file you're working with (on the left side).
- As the file opens on the right, click on the actual line number on the left column of the open file, where you want to pause. You will see a blue arrow appear on that line number indicating that a breakpoint has been set at that point.
- Reload or run your code, and when the browser hits that line, it will pause execution, so you can:
 - Inspect variables
 - Step through the code
 - Watch how values change

- To turn off the breakpoints from the browser tools, hit the the 'Deactivate breakpoints' icon which is a crossed-out blue arrow. Script execution will then resume as normal.

Bonus tip: From DevTools, you can also step through code line-by-line using the “Step over”, “Step into”, and “Step out” buttons in the while your code execution is paused-just like a professional debugger

Using the Browser Tools Network Tab

The Console tab shows you what your code said. The Network tab shows you what your page asked for, and what came back. Beginners rarely open it, which is a pity, because it answers one question the Console cannot: did that file even arrive?

Open your Developer Tools (F12, or right-click > "Inspect"), choose the Network tab, then reload the page. A list appears, with one row for every single thing the page asked for - your HTML, your CSS, your JavaScript files, every image, and later on, every call your code makes to a server. The column to watch is Status.

a) Checking that a file really loaded A status of 200 means "here it is". A status of 404 means "there is nothing at that address", and that is nearly always a spelling mistake in a src or href, or a file whose name does not match what you typed.

This matters far more than it looks. When a JavaScript file fails to load, the browser does not stop and warn you in the middle of the page. It quietly runs none of it. Every button on the page dies at the same moment, and not one line of your code is at fault. It is easier still to hit this with a script marked type="module", because a module also refuses to run when you open the page straight from your hard drive rather than through a server such as Live Server.

So if you ever meet a page where nothing at all works, do not start reading your JavaScript. Open the Network tab and check that the script actually arrived. "Nothing works" almost always means "the file never loaded", not "my code is wrong".

b) Watching the requests your code sends out The Network tab is not only for files. Every request your code makes to a server while the page is running - to fetch data, to submit a form, or to upload a file - appears as a new row the moment it happens. Click any row and you can read what you sent and what came back.

That is what makes it indispensable once you start talking to other computers. When we call APIs with fetch() in Chapter 22 (Extensions - APIs & Libraries), upload files in Chapter 18 (File Management), or send form data in Chapter 19 (Forms and Email), the Network tab is where you see whether your request ever left, what the server replied, and why it refused. The status code answers that at a glance: 200

when all is well, 404 when the thing you asked for does not exist, and 500 when the server itself broke.

Bonus tip: a 404 is not a CORS error. New coders mix these two up constantly, and they are completely different problems. A 404 means the file was not found, so the address is wrong. A CORS error means the file was found perfectly well, but another website refused to share it with your page.

Telling them apart is refreshingly easy: a CORS error always contains the word "CORS". If that word is not in the message, it is not a CORS problem, and you should go hunting for a wrong filename instead. We look at CORS properly in Chapter 22.

Throwing and handling exceptions

In the real world, things go wrong all the time—and programming is no different. Files might not load, users might type unexpected things, and network requests may fail. Vanilla JavaScript (i.e. JavaScript without any external libraries) gives us a way to catch and handle errors using what is known as a try...catch statement. This involves the throwing and handling of exceptions using try, catch, finally. This mechanism allows you to manage runtime errors gracefully. This means that—and this is the whole essence of exceptions, when errors occur in your code, instead of your software stalling and breaking up, which will annoy or frustrate your users, you can ‘catch’ these errors, so that you can deal with the error in a better way. Dealing with the error in a better or graceful way could mean, informing other parts of your code that use that functionality (function or service), so that they can offer the user an alternative result, or it could mean informing the developer (you or your team) of the issue in the code, so it can be fixed as soon as possible. Instead of your whole program crashing when something goes wrong, try...catch lets you: write some code to do something. If something goes wrong, catch the error, and handle it without breaking everything. Here is the syntax of try...catch:

```
try {  
    // Code that might throw an error  
} catch (error)  
{  
    // Code to run if an error happens  
}  
finally  
{  
    // (Optional) Code that always runs, no matter what  
}
```

Here is how it works:

- **try block:** Place any code here that might cause an error. If an error occurs, JavaScript stops running this block and jumps to the catch block.
- **catch(error) block:** This block runs if an error happens. It receives an error object that contains information about what went wrong.
- **finally block (optional):** This block always runs—whether there was an error or not. It's good for clean-up tasks like closing connections, resetting UI elements, or clearing temporary data.

This is a very useful mechanism in programming because, as a developer, you try your best to anticipate things going wrong with every piece of code you write, but you can never predict it all. Typically, without using exceptions, you would have an idea of which areas in your program issues can potentially come from, and therefore try to handle those edge cases (error-prone sections or scenarios) by wrapping conditional (if...else) statements around them. Let's say, for example in your shopping application, when you wish to get an item from a products array, it makes sense to first of all make sure there is something in the array, before you try to grab it, otherwise you will get a standard programming error of trying to get something from an array that does not exist. You do the check by wrapping that array access in a conditional statement like so:

```
if (products) {
    let item = products[0];
} else {
    alert("Sorry, that item is out of stock!");
}
```

This works well for simple checks to prevent standard errors in your code. Exceptions are more powerful because they go deeper than that. There are errors that you either just may not be able to anticipate, or sections in your program that are mission-critical, meaning, your program will just not be able to work when such errors occur. Such an error, can be someone trying to access your banking software, and submitting a bank account number that does not match the password or date of birth they are using. For such errors, you want to definitely capture them when they occur and halt the script execution, then show the user a meaningful message, rather than letting them through. Another type of mission-critical error may be your bank's server is down, and rather than let the application just crash and blackout—which will confuse or even upset your customers/users, you will want to always check if the server is up and running before trying to access it. If it is then found to be down, you can inform the users in a friendly way to try again later, and then inform your technical department immediately to fix the issue. These are situations where exceptions in programming come in. They really get into the engineer-

itional statements. Your take-home key point here should be that conditional statements are for anticipating standard programming errors, while exceptions are for anticipating mission-critical errors in your application. All programming languages have exception handling built right into them, and they let you create your own custom exceptions as per the needs of your software application. Speaking of objects, if you are unsure about objects and classes, do not worry, we learned all about them in Object-oriented programming (OOP) back in Chapter 12, so feel free to hop back there for a quick refresher before carrying on here.

The way it works is, when creating a service or function, in parts where potential errors may occur (eg a mission-critical error), you will check for this situation so that if your code encounters such an error, it should use a throw statement to create an exception. Then, in other parts of your code that make use of that service or function that will potentially throw an exception, you will call such services or functions within a try...catch block. Basically, within the try block, you place code to access the service/function, then in the catch block, which is where any exception that would be thrown by that service/function will be captured (caught) and made available to you, you will deal with (handle) the exception. This means that your application will run smoothly without unexpectedly stalling and confusing your users. Any potential issues will be captured in the catch block, so that you can deal with them in a manner befitting of a well-thought-through, and user-friendly application. JavaScript uses two types of block constructs to capture and deal with (handle) the thrown exceptions-more on this shortly. Here is an example:

```
// create the service/function
function viewAccount(account, user_id)
{
    // user cannot view account that is not theirs
    if (account.user_id !== user_id)
    {
        // throw an Error object (JavaScript in-built or custom)
        throw new Error(
            "User " + user_id + " trying to access account not theirs"
        );

        /*
        OR

        // throw an object literal
        throw { code: 401, message: "Account not theirs" };

        OR

        // throw a simple string error (less common)
        throw "User " + user_id + " trying to access account not
theirs";
        */
    }
}
```

```

{
    // this is the correct user, show them account details
}

}

// handling exceptions (using try/catch/finally)
try {
    const result = viewAccount(account, user_id);
}
catch (error)
{
    // handle the error – respond accordingly
    console.error("An error occurred: ", error.message);
}
finally
{
    // Optional: runs regardless of success or failure
    console.log("The viewAccount() was called");
}

```

If the `viewAccount()` function determines the user to be the right owner of the account, the user gets their account details shown to them, if not an `Error` exception is thrown. The API code 401 refers to an unauthorised access attempt, which is usually due to invalid user credentials to access a resource. We will learn more about API response codes when we come to learn about APIs in Chapter 22 (Extensions). The `Error` exception is a JavaScript (in-built) exception. You pass to its constructor a string, which will be available to any code that catches this exception on its `message` property like so:

```
error.message
```

If no exception is thrown, then all is well, and the `viewAccount()` did not throw any exception. The code in the `try {}` block where this `viewAccount()` function is called will therefore work, while the code in the `catch {}` block will not be run. The `finally {}` block is optional, and you will rarely see it being used. But when it is used in code, the code in that block will always be run, regardless of whether an exception was thrown by the service or function. Use it therefore only when there is an action you want to take no matter the outcome of calling that function or service. Every programming language has built-in exceptions but you can write your own. Let's start by looking at some of the exceptions provided to you by JavaScript. JavaScript has the following built-in error constructors:

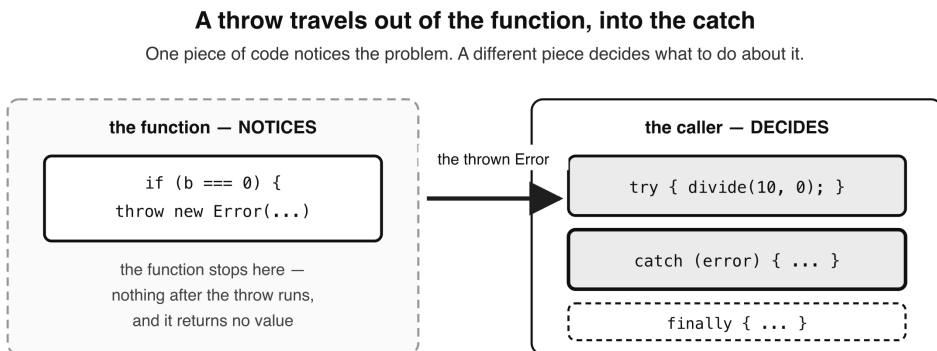
- `Error` (for generic errors — the one used in the example above)
- `SyntaxError` (for parsing errors)
- `TypeError` (for wrong type errors eg "undefined is not a function" which is an exception you'll get if you try to use a function that does not exist)

- `ReferenceError` (thrown each time you try to use an undefined variable)
- `RangeError` (invalid range, e.g. `toFixed(-1)`)

These built-in error types help categorise issues that can go wrong in an application. Here is how you can use one of these in-built exceptions. Let's take the `TypeError` exception for example:

```
try {
  // this will throw a TypeError exception
  null.someMethod();
}
catch (error)
{
  // handle the error – respond accordingly
  if (error instanceof TypeError)
  {
    console.error("TypeError caught");
  }
  else
  {
    console.error("Other error");
  }
}
```

Exceptions work in both synchronous and asynchronous (Promises, `async/await`) code.



`try` runs until something throws, then abandons the rest of its block.

`catch` receives the thrown value — use `error.message`, and `instanceof` to tell types apart.

`finally` runs EITHER WAY, which is why cleanup belongs there and not in the other two.

If nothing throws, `catch` is skipped entirely — it is not an "else".

Figure 21.1 — A throw travels out of the function, into the catch

The throw statement

A throw statement lives inside the function that detects the problem — not in the try block. The try block is where you CALL that function, and the catch block is where the thrown value arrives. Look back at the example above and you will see the throw sitting inside viewAccount(), while the try/catch sits around the call to it. What is thrown can be one of the following three data types:

- an Error object which can be a JavaScript in-built error object, or your custom error object).
- an object literal. For example: throw { code: 401, message: "Account not theirs" };
- a simple string error (less common) For example: throw "User " +user_id+ " trying to access account not theirs";

Custom exceptions

You can define custom errors by extending the Error exception of JavaScript. Here is how you would do it:

```
class CustomError extends Error {
  constructor(message) {
    super(message);
    this.name = "CustomError";
  }
}

try {
  throw new CustomError("Oops, something went wrong");
}
catch(error) {
  console.error(error.name, error.message);
}
```

This code will print: CustomError Oops, something went wrong

Asynchronous exception handling

When dealing with promises that you know can potentially throw exceptions, use .catch() or try...catch with async/await. Here is an example:

Using .catch()

```
someAsyncFunction()
  .then((result) =>
    console.log(result))
```



```
.catch((error) => )
    console.error("Failed:", error));
```

Let's explain what is happening here. Here, we are calling an asynchronous function that we know will throw an exception, which is why we have the `.catch()` block. If not, we wouldn't have needed the `catch()` block. We handle the potential error exception thrown within this catch block.

Using `async/await`

```
async function fetchData() {
    try {
        const data = await someAsyncFunction();
    }
    catch(error) {
        console.error("Fetch failed:", error);
    }
}
```

Multiple catch blocks

Unlike some other languages like Java or C#, JavaScript does not support multiple catch blocks. However, you can use `if...else` logic inside a single catch to handle different error types. Here is an example:

```
try {
    // code that may fail
} catch (error)
{
    if (error instanceof TypeError) {
        console.log("Type error!");
    } else if (error instanceof ReferenceError) {
        console.log("Reference error!");
    } else {
        console.log("Some other error:", error.message);
    }
}
```

Conclusion and exception handling examples

To really sum up the essence of exception handling in programming, here are the key points:

- You can call code that can potentially throw an exception in the try block. This is the most common use case. If a function or API may throw an error, you wrap it in try to catch it safely.

- You can throw your own exceptions in the try block. This is perfectly valid and actually a common practice. You can manually throw errors when you detect

valid input or unexpected conditions. In input validation, for example, throw new Error(...) allows you to interrupt normal flow and handle it consistently in the catch block. It's not just for external libraries—it's a way of managing bad situations in your own logic too. This gives you a uniform way to handle errors—whether they come from your code or built-in functions.

- In the catch block, you can catch the exception and handle it. This is its main job, and the whole point of the catch. You can still throw errors from within the catch block if you want to re-throw or escalate. Here are scenarios when you may throw an exception within a catch block:
 - When you want to re-throw the original error
 - When you wish to throw a different error
 - When you wish to escalate that error to a higher-level handler

Handling exceptions in programming offers the following benefits:

- It prevents app crashes
- It gives meaningful error messages
- It helps users understand what went wrong
- It offers clean-up opportunities via finally

I would like to leave you with some more examples so that you really master this rare skill.

Examples

Example 1: try...catch where no error is caught:

```
try {
  let user = JSON.parse('{ "name": "Jane" }');
  console.log(user.name);
} catch (e) {
  console.log("Something went wrong:", e.message);
}
```

Output: Jane

Comments: No error occurred, so the catch block is skipped.

Example 2: A syntax error is caught:

```
try {
  let user = JSON.parse('INVALID JSON STRING');
} catch (e) {
  console.log("Oops! Error:", e.message);
}
```

Output: Oops! Error: Unexpected token l in JSON at position 0

Comments: JavaScript throws an error when trying to parse bad JSON, but we handle it and avoid a crash.

Example 3: The use of finally:

```
try {
    throw new Error("Unexpected problem");
} catch (err) {
    console.log("Caught error:", err.message);
} finally {
    console.log("Cleanup happens here!");
}
```

Output: Caught error: Unexpected problem Cleanup happens here!

Comments: The finally block runs even though an error occurred.

Example 4: User input validation:

```
function processAge(age) {
    try {
        if (isNaN(age)) {
            throw new Error("Age must be a number");
        }

        if (age < 0) {
            throw new Error("Age can't be negative");
        }

        console.log("Valid age:", age);
    } catch (error) {
        console.log("Input Error:", error.message);
    } finally {
        console.log("Processing done.");
    }
}

// Let's call our processAge() function to test it
processAge("twelve");
processAge(-5);
processAge(30);
```

Output: Input Error: Age must be a number Input Error: Age can't be negative
Valid age: 30

Comments: Because we know we want to be throwing exceptions if validation errors are found, we wrap our whole set of validation checks within a try {} block so we can handle them in the catch block.

Example 5: Re-throwing and Handling at a Higher Level Let's look at an example where we:

- Catch an error in one place
- Re-throw it from the catch block
- Have it caught and handled by a higher-level handler.

```
// A function that may throw an error
function validateUsername(username) {
  try {
    if (!username) {
      throw new Error("Username is required");
    }

    if (username.length < 4) {
      throw new Error(
        "Username must be at least 4 characters"
      );
    }

    console.log("Username is valid!");
  } catch (error) {
    console.warn("Validation failed, re-throwing...");

    // Re-throw to be handled elsewhere
    throw error;
  }
}

// A higher-level handler that wraps the call
function handleUserInput() {
  try {
    // Simulate user input

    // Too short on purpose
    const input = "Tom";
    validateUsername(input);
  } catch (err) {
    console.error("Error handled at a higher level:");
    console.error("Message:", err.message);
  }
}

// Run the code
handleUserInput();
```

Comments:

- The structure of this pattern is like this:

```
handleUserInput()
├─ validateUsername("Tom")
│   └─ throw new Error(...)    // thrown
│       └─ catch & re-throw error
└─ catch (err)                  // final handler
```

- `validateUsername()` is a lower-level function responsible for checking user input.
- If the input is bad, it throws an error and catches it locally first, then re-throws it.
- `handleUserInput()` is a higher-level function. It's in charge of user interaction logic, so it wraps the call in `try...catch` and catches the re-thrown error. -This allows separation of responsibilities: * One function checks for correctness. * The other decides how to inform the user or log it.

This pattern is useful when a lower-level function detects an error but wants a higher-level function to decide how to deal with it.

TESTING

Testing is possible in vanilla JavaScript—without using external libraries. But in practice, it's usually manual or involves building minimal custom test setups. At its most basic, you can write simple if statements and `console.assert()` calls to check expected values:

```
function add(a, b) {
    return a + b;
}

// Manual test
if (add(2, 3) === 5) {
    console.log('Test passed');
} else {
    console.log('Test failed');
}

// This LOGS an error if the condition
// is false. Note that it does not
// throw - the lines after it still run
console.assert(add(1, 2) === 3, 'Test failed: add(1, 2) should equal 3');
```

Using `console.assert()` will log an error to the console and display the message string that you pass as its second argument (in this case above: 'Test failed: add(1, 2) should equal 3'), if the expression in its first argument evaluates to false. While manual testing like this is fine for beginners or toy projects, and for learning logic and debugging, it is not scalable for large or real-world apps. In professional JavaScript development, we use libraries like:

- Jest (very popular for both frontend and backend)
- Mocha + Chai
- Vitest (modern Jest alternative for Vite projects)

These tools give us:

- Test runners
- Better reporting (what passed/failed and why)
- Assertion libraries
- Mocking/stubbing abilities

As you begin building bigger apps, you'll want to ensure everything works as expected through automated tests. In professional projects, developers use tools like Jest or Mocha to write structured test cases. But even with just vanilla JavaScript, you can start by writing simple checks using `console.assert()` or `if` statements to test your code.

QUIZ — Error, Debugging and Testing

This page contains the Q & A (questions and answers) for this chapter — Chapter 21: Error, Debugging and Testing. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 12 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. Why is `document.write()` considered bad practice, and what should you use instead?

Clue: it depends entirely on

when

it runs.

2. `prompt()` takes a second argument. What is it, and why is calling it a "placeholder" misleading?

Clue: one of them disappears when you type. The other does not.

3. When would you reach for an exception rather than a plain `if` statement?

Clue: think about which errors you can see coming, and which you cannot.

4. Where does a `throw` statement actually live — in the `try` block or somewhere else?

Clue: look at which piece of code *detects* the problem, and which piece *decides what to do* about it.

5. Name the three kinds of thing you can throw, and say which one you should normally use.

Clue: one of them gives you a `.message` and a stack trace for free.

6. Match each built-in error type to what causes it:

TypeError ReferenceError RangeError SyntaxError

Clue: try `null.someMethod()`, an undeclared variable, and `(5).toFixed(-1)`.

7. What does the `finally` block do, and when is it worth using?

Clue: it runs in a case where neither of the other two blocks does everything you need.

8. Does `console.assert()` stop your program when the check fails?

Clue: this one catches people out, and the chapter says both things in different places.

9. EXERCISE. Write a `divide(a, b)` function that throws an `Error` when `b` is zero, then call it inside a `try...catch` and print the message.

Clue: the throw goes in the function, the try goes around the call.

10. EXERCISE. Write a `ValidationError` class that extends `Error`, throw one, and print both its name and its message.

Clue: two lines in the constructor, one of which is `super`.

11. EXERCISE. Write a `try...catch` that tells a `TypeError` apart from any other error.

Clue: one operator does the checking.

12. EXERCISE. Write two manual tests for an `add(a, b)` function — one with an `if`, one with `console.assert()` — and make the second one fail on purpose so you can see what happens.

Clue: question 8 tells you what to expect from the failure.

ANSWERS

1. Because of **when** it runs. While the page is still loading, `document.write()` simply adds content. But once the browser has finished loading and closed the document stream, calling it **reopens that stream — which wipes the entire page** and starts fresh.

So the same line behaves completely differently depending on timing, which is a horrible property for a function to have.

The replacement is `innerHTML`, and specifically `+=` rather than `=`:

```
document.body.innerHTML += "My data here";
```

The `+=` appends. A plain `=` would replace everything already inside the element — which, if you are writing values in a loop, means you end up seeing only the last one.

2. The second argument is the **default value** of the input box.

```
prompt('Enter surname', 'Enter surname here');
```


Calling it a placeholder is misleading because the two behave differently in a way that matters:

- A **placeholder** is grey hint text that **vanishes the moment you type**. It is never a value.
- A **default** is real text **already sitting in the box**. If the user presses OK without touching it, that text is exactly what `prompt()` returns to you.

So a default can arrive in your program as though the user had typed it, and your code should be ready for that.

3. Use a plain `if` for the ordinary, expected checks — is the list empty, is this number too big, did the user leave the field blank. These are things you can see coming.

Reach for an **exception** when the problem is **mission-critical** or unforeseeable: the database is unreachable, a user is trying to open an account that is not theirs, a file that must exist does not. These are cases where carrying on is not an option, and where the code that *notifies* the problem is usually not the code that should *decide what to do* about it.

That last point is the real dividing line. An `if` handles a problem where it finds it. An exception hands the problem upwards, to whoever is in a position to respond.

4. A `throw` lives **inside the function that detects the problem** — not in the `try` block.

```
function viewAccount(account, user_id) {
  if (account.user_id !== user_id) {
    // detected here
    throw new Error("Not your account");
  }
}

try {
  // called here
  viewAccount(account, user_id);
} catch (error) {
  console.error(error.message);
  // handled here
}
```

The `try` block is where you **call** the risky function; the `catch` block is where the thrown value arrives. That separation is the whole point: one piece of code knows something is wrong, another decides what to do about it.

5. You can throw:

- an **Error object** (built-in or your own subclass) — `throw new Error("...")`
- an **object literal** — `throw { code: 401, message: "Account not theirs" }`
- a **plain string** — `throw "something went wrong"` (least common)

Normally you should throw an **Error object**. You get `.message` and `.name` for free, you get a stack trace showing where it happened, and `instanceof` works so a catch block can tell error types apart. A thrown string gives you none of that.

6.

`TypeError null.someMethod()` — doing something to a value that cannot do it
`ReferenceError` using an undeclared — the name does not exist at all
`variable`
`RangeError (5).toFixed(-1)` — a number outside an allowed range
`SyntaxError` a missing bracket — the code cannot even be parsed

`SyntaxError` is the odd one out. The others happen while your program is *running*; a `SyntaxError` happens when JavaScript *reads* your file, which means the program never starts at all — so you cannot catch it in a `try...catch` in the same file.

7. `finally` runs **whether or not an exception was thrown**. It runs after the `try` succeeds, and it runs after the `catch` handles a failure.

```
try {
  openConnection();
} catch (error) {
  console.error(error.message);
} finally {
  // happens either way
  closeConnection();
}
```

It is worth using when there is **cleanup that must happen regardless** — closing a connection, hiding a loading spinner, releasing a lock. Put that in `try` and it is skipped when things go wrong; put it in `catch` and it is skipped when things go right. `finally` is the only place that always runs.

8. **No.** `console.assert()` logs an error message to the console and then **carries straight on**.

```
console.assert(1 === 2, 'this will not stop anything');
console.log('and this line still runs');
```

Output:

Assertion failed: this will not stop anything and this line still runs

That is worth being clear about, because "assert" sounds forceful and in some other languages a failed assertion does halt the program. In JavaScript it is purely a reporting tool. If you want execution to stop, you need to `throw`.

```
9. function divide(a, b) {
    if (b === 0) {
        throw new Error("Cannot divide by zero");
    }
    return a / b;
}

try {
    console.log(divide(10, 2));
    // 5
    console.log(divide(10, 0));
    // throws
} catch (error) {
    console.error("Error:", error.message);
    // Error: Cannot divide by zero
}
```

Note that the first call prints `5` before the second one throws. Everything in a `try` block up to the point of the throw runs perfectly normally — the block is abandoned from the throw onwards, not from the beginning.

```
10. class ValidationError extends Error {
    constructor(message) {
        super(message);
        this.name = "ValidationError";
    }
}

try {
    throw new ValidationError("Username is too short");
} catch (error) {
    console.error(error.name, "-", error.message);
    // ValidationError - Username is too short
}
```

`super(message)` passes the message up to `Error`, which is what makes `.message` work. Setting `this.name` is what makes the error identify itself properly — without it, the name would still say "Error", which rather defeats the purpose of having made your own.

A custom error also means a catch block can pick it out specifically with `error instanceof ValidationError`.

```

11. try {
    null.someMethod();
  } catch (error) {
    if (error instanceof TypeError) {
      console.error("TypeError caught:", error.message);
    } else {
      console.error("Some other error:", error.message);
    }
  }
}

```

JavaScript has only one `catch` block per `try` — unlike some languages, you cannot write several catches for different types. So `instanceof` inside a single catch is how you tell them apart.

```

12. function add(a, b) {
    return a + b;
  }

  // Test 1: the manual if
  if (add(2, 3) === 5) {
    console.log('Test passed');
  } else {
    console.log('Test failed');
  }

  // Test 2: console.assert, deliberately wrong
  console.assert(add(1, 2) === 4, 'Test failed: add(1, 2) should equal 4');

  console.log('Still running');

```

Output:

Test passed Assertion failed: Test failed: add(1, 2) should equal 4 Still running

The last line is the point of the exercise. The assertion failed, said so, and the program carried on regardless — which is exactly why `console.assert` is fine for a quick check but not something to build a real test suite on. That is what Jest, Mocha and Vitest are for.

Chapter 22 — Extensions (APIs & Libraries)

- APIs
 - Introduction to APIs and HTTP Requests
 - REST API Requests
 - How to send a REST request
 - Understanding SOAP APIs (and why they're usually handled outside the browser)
 - XML namespaces and why we need a namespace resolver
 - Must you use a Namespace in every SOAP request?
 - How to send a SOAP request
 - CORS work-arounds for making SOAP requests from frontend
 - Setting up a local SOAP server with Node.js
 - To use or not to use a WSDL file
 - Why Use APIs
 - How JavaScript Communicates with APIs
 - Sending HTTP Requests
 - What Happens on the server-side
 - HTTP Status Codes
- AJAX
 - How does it relate to JavaScript?
 - AJAX in Action (Examples)
 - Old Way (Using XMLHttpRequest)
 - The responseType property
 - The readyState property
 - Modern Way (Using fetch())
 - Asynchronous programming
 - a) Callback
 - b) A promise
 - promise.all()
 - c) Async/Await

- Using Axios (External Library)
- WebSockets
 - Why Use WebSockets
 - Why it works
- Where networking actually lives
 - Networking in the browser
 - Networking in Node.js
- Libraries
 - `notie`

Here we talk about external tools in the ecosystem of your programming language that make it even more powerful. We are talking here about any existing APIs, development frameworks, and off the shelf libraries available for you to use extend any kind of functionality.

APIs

Introduction to APIs and HTTP Requests

Let's start by talking about what an API is. API stands for Application Programming Interface. It's like a messenger that allows different pieces of software to talk to each other. In JavaScript, when we talk about APIs, we usually mean web APIs. These are systems created and put out there on the internet by people-companies, or other programmers, so that your browser can send requests to and receive data from them. These APIs often provide data in a format like JSON (JavaScript Object Notation), which is easy to work with in JavaScript. However, vanilla JavaScript cannot create a server-side API. This is because client-side JavaScript, which runs in a browser (frontend) is not a server-side programming language, as you already know. It therefore lacks access to server resources like databases, full access to file systems etc. To create an API, you need a server-side programming language-also referred to as a backend language. Examples of backend languages are Python, PHP, Node.js, Ruby, Java etc and they can all be used to build APIs. Frontend JavaScript can only consume APIs. We will therefore dwell here only on how to make requests to consume APIs, and how to handle the data that we get back. Having said this, I must say that there are different types of APIs-notably, REST and SOAP APIs. Whilst we are not concerned in this book about how to create the APIs, knowing the type of API we are trying to consume is important because it will determine how we make the requests to them. When consuming APIs

in JavaScript, whether they are REST or SOAP, you typically use tools like `fetch()`, which is more modern and promise-based, or `axios`, a third-party library with more features, or `XMLHttpRequest`, which is older but still used in some cases. These tools are the same, but the key differences in how you make the requests to the different API types lie in the content of the request, what you send in the HTTP headers, and the format of the data you are sending and receiving. Let's look at REST and SOAP APIs, and see how there is a difference in how to make requests to them and handle the response we get back.

REST API Requests

REST stands for Representational State Transfer-an architectural style for designing networked applications. It relies on stateless, client-server communication, typically over HTTP. The frontend (client) and the backend (server) operate independently. Rest resources are identified by their URLs or endpoints. For example: `/users`. Data is usually transmitted using JSON format e.g., `{"name": "Alice"}` You send data to the API server in JSON format using different HTTP methods, depending on the kind of request being made. Here is a list of the HTTP methods supported in a REST API:

Method	Purpose
GET	Read. Ask the server for data
POST	Create. Send some data to the server to create a new resource
PUT	Update, by replacing the entire existing resource (data)
PATCH	Update. Partially update an existing resource.
DELETE	Delete existing resource (data)

These are the HTTP/1.1 protocol methods defined in the RFC 7231 specification. There are a few more, like HEAD, OPTIONS, TRACE, and CONNECT, but these last four are used only in specific circumstances and are less common. Of the five (5) HTTP request methods listed above, GET and POST are the most commonly used in the browser. You will see them in action when I talk about AJAX, which is an easy way to send HTTP requests asynchronously from a browser to a server. The PUT and PATCH methods handle update operations, but as you can see from their purposes, while PUT entirely replaces what existed of the record previously, PATCH will only update some fields of that existing data.

How to send a REST request

Here is an example POST request to a REST API using `fetch()`:

```

fetch(
  "https://api.example.com/users",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ name: "Jane", age: 30 })
  }
)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error', error));

```

The first argument to `fetch()` is the URL or endpoint of the resource. This is basically the path of the API server we are sending the request to. You can usually tell from the URL what resource is being targeted—in this case, the resource we need in this users (`/users`). The second argument passed to `fetch()` is an object which usually contains things like the HTTP method (eg `POST`, `UPDATE` etc), headers, and the body. This object is referred to as the init object, or options object. In some documentations, it is referred to as the configuration object that controls settings like:

- The HTTP method eg `GET`, `POST`, etc
- Request headers as an object or instance of the built-in JavaScript Headers class. Here is an example of how you can use a Headers class instead of an object literal like in our example above:

```

// create the headers object
const myHeaders = new Headers();
myHeaders.append('Content-Type', 'application/json');
myHeaders.append(
  'Authorization', 'Bearer your-token-here');

// define the fetch options
const requestOptions = {
  method: 'POST',
  // use the Headers instance
  headers: myHeaders,
  body: JSON.stringify(
    {key: 'value'}
  ), // request data (aka payload)
  mode: 'cors' // enable CORS
}

// make the request
fetch(
  "https://api.example.com/users",
  requestOptions
)

```



```

.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error('Error', error));

```

- The body is also sent within the request's options object. This body option contains whatever data you are sending to the server. This data is also referred to as the payload (of the request). It is typically sent with requests like: POST, PUT, and PATCH. Sometimes data is sent with DELETE requests too, and that will only contain the id of the resource to be deleted. For the PATCH request, only data intended for the field or fields to be updated are sent.

It is okay for no request object to be sent with an API request. A GET request does not need it, and so none is sent with GET requests. Actually, a request is assumed to be a GET request by default, unless you pass in an options object to specify another request method. Whenever you pass a JavaScript object as the body in the body of a `fetch()` request, you must wrap it in a `JSON.stringify()` function, because:

- HTTP Requests only accept strings (or binary data). The body property in `fetch()` cannot handle JavaScript objects. The HTTP protocol needs the request body (data) to be sent as either of these:
 - A string which can be a JSON string, or a URL-encoded form data.
 - A Blob/Buffer (for binary data like files)
- Another reason to use `JSON.stringify()` on the `fetch()` body is because `JSON.stringify()` converts objects to JSON strings.
- The data type declared as the type of data being sent in the headers section must match the type of the data being sent in the body, obviously. Therefore, because we set the request data type as JSON

```

headers: {
  "Content-Type": "application/json"
},

```

We therefore have to make sure what is being sent via the body matches that. Hence we use `JSON.stringify()` to convert that data into a JSON string. With that said, if you are not sending JSON data, you do not have to use `JSON.stringify()` on the body. This also means you would not set the datatype as JSON ("application/json") in the headers section, obviously.

Understanding SOAP APIs (and why they're usually handled outside the browser)

While modern frontend development mostly revolves around REST APIs and JSON, there's another important type of web service called SOAP—short for

Simple Object Access Protocol, and it works differently. SOAP uses XML to structure messages used for communication between applications over a network. It was widely adopted since the early 2000s in enterprise software systems for building web services, before REST became popular. Although SOAP is still in use today (especially in finance, healthcare, telecom and enterprise software, where strict message formats and contracts are important.), it's important to know that most SOAP APIs are not accessible directly from the browser. This is because browsers block cross-origin requests unless the API explicitly allows it through something called CORS (Cross-Origin Resource Sharing)—which SOAP APIs often don't. So why learn about SOAP in a JavaScript book? Because understanding SOAP helps you:

- Work with XML-based data formats (which are still common),
- Understand legacy systems in enterprise environments,
- Parse XML using XPath and namespaces (a skill useful beyond SOAP),
- Recognize when and why you'd use tools like Postman or server-side code to interact with APIs.

This section will show you how SOAP works, what its XML messages look like, and how you'd parse a SOAP response using JavaScript's XML tools—not necessarily to run it in your browser, but to prepare you for when you'll encounter it in the real world. SOAP messages are typically sent over HTTP and have a predefined envelope structure that looks like this:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetPriceResponse>
      <Price>29.99</Price>
    </GetPriceResponse>
  </soap:Body>
</soap:Envelope>
```

This XML structure is what you parse when working with SOAP responses.

XML namespaces and why we need a namespace resolver

In XML, namespaces are used to avoid name conflicts when different elements share the same name. They do this by associating elements with a unique URI (a web-like address). For example:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  ...
</soap:Envelope>
```

This means that every element with the `soap:` prefix (like `soap:Envelope`, `soap:Body`) belongs to the SOAP XML namespace, which ensures it's interpreted correctly. Without the correct namespace, XPath queries (a technology for reading XML data, which we will use to read SOAP responses to extract the data we need) won't match anything. See Chapter 15 (DOM manipulation) where I talk about XPath and `document.evaluate()`—which is a DOM method meant for running XPath queries to read data from XML and HTML objects. JavaScript's `document.evaluate()` function however, doesn't automatically understand what "soap:" means in your XPath query string. You must help it by supplying a namespace resolver: a function that maps prefixes (like "soap") to their full namespace URI. Here is an example of how to create a namespace resolver for your SOAP request:

```
const nsResolver = (prefix) => {
  if (prefix === "soap") {
    return "http://schemas.xmlsoap.org/soap/envelope/";
  }
  return null;
};
```

Then use this resolver when calling `document.evaluate()`:

```
const result = xmlDoc.evaluate(
  // your XPath query
  "///soap:Body//Price/text()",
  xmlDoc,
  nsResolver,
  XPathResult.STRING_TYPE,
  null
);
```

Must you use a Namespace in every SOAP request?

You do not need it sometimes. If your XML doesn't use prefixes (e.g. just `<Envelope>` instead of `<soap:Envelope>`), you don't need a resolver. But in most real-world SOAP responses, you will need one, because namespaces are standard practice.

How to send a SOAP request

SOAP messages are not something you "guess"—they follow a predefined structure called a WSDL, or Web Services Description Language file. A WSDL is an XML document provided by the API provider that describes the operations (functions or methods) the service offers, and how to structure the request you send. It will also tell you about the structure of the response to expect back. For example, the WSDL might tell you:

- The name of the operation (e.g. GetPrice)
- The expected input and its format
- The required XML namespaces
- The response structure

Always check if the service provides a .wsdl file or documentation. It helps you to build the correct SOAP message. Here's a simple SOAP request message structure:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetWeather>
      <City>London</City>
    </GetWeather>
  </soap:Body>
</soap:Envelope>
```

This structure must match what the server expects. If you get it wrong, you'll usually get a "SOAP fault" (an error message in XML format). Let us look at a complete example of how you would prepare and send a SOAP request including a namespace resolver, and how you would parse (extract) the XML response returned using document.evaluate(). In the following example, we will send the SOAP request using an XMLHttpRequest object. For the response, we expect XML data, so we will handle that:

```
const xhr = new XMLHttpRequest();
xhr.open('POST', 'https://example.com/soap-endpoint');

// Important: Set the Content-Type
// to indicate a SOAP message
xhr.setRequestHeader('Content-Type', 'text/xml');

// Optional but useful: expect an XML document
xhr.responseType = 'document';

// Build your SOAP XML string
const soapMessage = `
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetPriceRequest/>
  </soap:Body>
</soap:Envelope>`;

// When the response arrives...
xhr.onload = () => {
  const xmlDoc = xhr.response;
```

```

const nsResolver = (prefix) => {
  if (prefix === "soap") {
    return "http://schemas.xmlsoap.org/soap/envelope/";
  }
  return null;
};

const result = xmlDoc.evaluate(
  "//soap:Body//Price/text()",
  xmlDoc,
  nsResolver,
  XPathResult.STRING_TYPE,
  null
);

console.log("Price: $" + result.stringValue);
};

xhr.send(soapMessage);

```

In the SOAP message—which is the XML string stored in the variable `soapMessage`, the body or contents of the message you are sending to the server are within the `<soap:Body>` section or element. In this example, we are making a call to a function named `GetPriceRequest` which is located on the API server. We would have known about the existence of this function from the WSDL (Web Service Description Language) that the API providers supplied to us. The WSDL which is the documentation of the API would also contain information about arguments we need to pass to the function, if any is required. The first argument passed to the `.evaluate()` method is the XPath query string (`"//soap:Body//Price/text()"`). XPath stands for XML Path Language. It's a way to navigate and select parts of an XML document, like querying a database but for XML. In the example above, the XPath string `"//soap:Body//Price/text()"` tells JavaScript:

- Look anywhere (`//`) for the `<soap:Body>` element
- Then go inside it and look for the `<Price>` element
- Extract its text content (using `/text()`)

You use `document.evaluate()` to run XPath queries in JavaScript. It requires a valid XML document (not just a string), which is why using either `xhr.responseType = 'document'` or using `DOMParser` is essential because they first of all convert the returned data into a document (DOM) object before you can use DOM methods like `document.evaluate()` on it. Again, visit chapter 15 (DOM Manipulation) for a full explanation of XPath and the `document.evaluate()` method.

Note on CORS: CORS is a security mechanism implemented by browsers to control how web pages from one domain (origin) can request resources (e.g. APIs,

fonts, images etc) from another domain. This is for security reasons—imagine if anyone was allowed to make their application request and use data from the application of another business, like a bank without their permission. This would facilitate the work of malicious users and hackers. That is why CORS was introduced. If you try to send a SOAP (or REST) request to an external API from your browser and see a CORS error, it's not a problem with your code. For example, trying to run the code in this example above might give you this error in your console:

"Access to XMLHttpRequest at '<https://example.com/soap-endpoint>' from origin '<http://127.0.0.1:5500>' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource."

This is because your browser is protecting your local environment from making requests to a different origin (i.e., <https://example.com/soap-endpoint>). This is a very common issue when testing SOAP or REST APIs locally. Modern browsers block requests made from one origin (like your localhost) to another (like example.com) unless the server explicitly says it's okay by sending (enabling) CORS headers like:

Access-Control-Allow-Origin: *

But in most real SOAP APIs, especially enterprise-grade ones, CORS headers are not enabled, because they assume you are calling from server-side code, not from a browser. This makes them inaccessible directly from browser code, which is what JavaScript is.

Browsers block such requests by default for security. You can work around this by using:

- A local proxy server
- Tools like Postman or Insomnia
- Server-side code (Node.js, PHP, Python, etc.)

APIs are usually meant to be accessed from backend systems, which do not have CORS restrictions like browsers do. Let's look at the ways to work-around CORS errors when making test API requests from the frontend.

CORS work-arounds for making SOAP requests from frontend

- 1. Set up a local proxy server that makes the SOAP request on your behalf (from Node.js, PHP, etc.). Your browser talks to your proxy, and the proxy talks to the API. This bypasses CORS. That is beyond the scope of this

book. But I make you aware of these solutions just to deepen your knowledge of APIs.

- 2. Use a Tool Like Postman or Insomnia (No CORS in Desktop Tools). If you're just testing SOAP requests, use Postman or Insomnia instead of your browser. These are tools meant to help you test API's, by manually sending and inspecting SOAP messages. One thing great about them is that they don't enforce CORS
- 3. Host your code on a server that allows CORS. If you're not just testing but building a real application, host your frontend and backend on the same domain or configure the backend to allow CORS (only if you control it). We will test this option below under the "Setting up a local SOAP server with Node.js" section. There I will show you how to use Node.js to set up a simple SOAP service that you can send requests to from the frontend code, and it will work because both your frontend and SOAP (backend) service are running in the same domain.

In conclusion, we must remember that browser JavaScript cannot reliably consume SOAP APIs. Unlike REST, SOAP APIs have the following limitations:

- They require XML-formatted requests with strict headers (e.g. SOAPAction).
- They block browser requests due to CORS restrictions (unless the server explicitly allows it).
- SOAP APIs often lack HTTPS (which triggers mixed content errors in browsers).

Due to all of that, here are the best practices or recommendations:

- In production (real-life applications), SOAP requests should be handled by a backend server using for example: Node.js, Python, or PHP.
- Such a backend server will process the SOAP request securely
- Such a backend server will also return a simplified response-for example in JSON format, to your frontend code.

As I indicated earlier, setting up such a backend server is beyond the scope of this book. That is why all the examples I have given you are focused on REST APIs-the modern standard of web development. My coverage of SOAP here is to prepare you and make you well balanced in your knowledge of the APIs, even if you will rarely ever consume a SOAP API directly from JavaScript.

Setting up a local SOAP server with Node.js

If you have a local server and hosted a SOAP API on your local server, it will be possible to access this SOAP API from frontend code running on the same server.

Let us just say you host a SOAP API on your local server (e.g., `localhost:3000/soap-api`) and your frontend code (HTML/JS) is also served from the same origin (e.g., `localhost:3000/index.html`), you can access the SOAP API without CORS issues. The reason why it works is simple;

- The Same-Origin Policy (SOP) allows frontend code to freely interact with APIs on the same domain/port/protocol.
- There are no CORS restrictions applicable, since there's no cross-origin request.

Let's see how to set a Node.js server on your local machine to host a simple SOAP API that we can send a request to from the frontend and get a response back. In this example, we will be using Express, a very popular Node.js framework that makes setting up and working with servers in Node.js very easy. It comes packed with libraries that provide services to handle most things to do with servers so that you don't have to build them by yourself-like security, validation, receiving and responding to HTTP requests, and much, much more. This is not a book on Node.js, but I want to leave you with a clear understanding of what happens on the side of a server when it comes to APIs. If you ever decide to go on and learn Node.js, it will be a great choice after mastering vanilla JavaScript. Node.js is a JavaScript engine that runs on a server. Thanks to it, JavaScript is no longer limited to a browser. It is one of the biggest reason's for JavaScript's popularity soaring in the last decade. It means with your current JavaScript knowledge, you will be able to build a full stack (frontend and backend) application that is fast and reliable. You will even be able to build backend systems for mobile applications as APIs, and much more. It's very powerful. Let's get right into building our SOAP API. Follow these steps, and remember, you should be in the root of your project in the Terminal when you run all of these commands:

- Make sure you have Node.js installed on your local machine, and if not, install it. Open the Terminal application on your local machine and navigate to your project folder, or in your IDE (code editor eg VSCode), open the integrated (built-in) Terminal application in it by going to Terminal (menu bar above), then choose New Terminal. Now that you're in the terminal, check if Node.js is installed by running this command:

```
node --version
```

If it is installed, you should get a response with the version number like this: `v20.12.2` If it's not installed, go ahead and install Node.js. The best way to do this is to visit their website: <https://node.js.org> and download the LTS (Long-Term Support) version for your system (Windows/ macOS). Run the installer and follow the instructions-the default settings are fine. It will install both Node.js and npm

(Node Package Manager) on your computer. Once done, confirm the installation by running the same command above:

```
node --version
```

- Next, setup a package manager file named `package.json`. All Node.js applications need you to have this file in the root of your application. Node uses it to keep track of, and manage which packages (libraries) your application is using, and their versions. In web development, these packages are also known as dependencies, because they are libraries that your application depends on to do its job. You do not have to create this file yourself, Node has a command that will do it for you and place the necessary code to start you off with a server that you can build on. But this basic one will be sufficient to handle our example application right now. Run this command to create the `package.json` file:

```
npm init -y
```

It will create a `package.json` in the root of your project, and place the following code in it:

```
{
  "name": "js-test",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

- Install Express. Do so by running the following command:

```
npm install express
```

This will add `express` as a dependency in your `package.json` and create a `node_modules/` folder.

- Install the following two Node.js packages that we will be needing:
 - `'xmldom'` (the Node.js equivalent of `DOMParser`)
 - `'xpath'` (the Node.js package used to perform XPath operations on files on the server-side.) Install them both using these commands:

```
npm install xmldom npm install xpath
```

- Optionally, modify this package.json file slightly by adding the following line in the “scripts” section if it doesn’t exist in it:

```
"start": "node index.js"
```

Your package.json file should now look like this:

```
{
  "name": "js-test",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "express": "^5.1.0",
    "xmldom": "^0.6.0",
    "xpath": "^0.0.34"
  }
}
```

Note how the new section "dependencies":... has been added. It was added when you ran the command to install Express. Then, henceforth, every time you install new dependency for you project, they will be added in there. For example, notice how both 'xmldom' and 'xpath' are all listed in there. They are all dependencies used by your application. Adding the 'start' entry to the 'scripts' section is optional, because it just allows you to be able to start your server by typing in the Terminal:

```
npm start
```

It will run this command specified in it for you: node index.js If not, you can run that command yourself whenever you want to start your server: node index.js To stop the server, just hit Control+c. Note that what this start command does is to serves a file index.js where your Node.js code to start a server should be. Let's go ahead and write the code that will create the server.

-Next, let's proceed to write the code on both the backend to create a server in Node.js and Express that will receive API (in this case SOAP) requests and send back responses, and also the frontend code that will send the requests to the back-end and get back the response. Before we do that, let's establish what the file structure of the application should look like in your project folder.

```

your-project-folder/
├── index.js      ← Your SOAP server
├── index.html    ← Your frontend
├── package.json  ← Auto-generated by npm init -y
└── node_modules/ ← Created by npm install express

```

Here is the code to go in the files. We have already seen what goes in the package.json file above:

FRONTEND CODE (index.html)

```

<!DOCTYPE html>
<head>
  <title>The JavaScript Blueprint</title>
</head>
<body>

  <h1>SOAP Request for Product Price</h1>

  <input type="text"
        id="product"
        placeholder="Enter product name (e.g., Apple)">

    <button onclick="getPrice()">Get Price</button>

<script>
  function getPrice() {
    const product = document.getElementById('product').value;

    const soapBody = `
      <soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
        <soap:Body>
          <GetPriceRequest>
            <Product>${product}</Product>
          </GetPriceRequest>
        </soap:Body>
      </soap:Envelope>
    `;

    fetch('/soap-api', {
      method: 'POST',
      headers: {
        'Content-Type': 'text/xml',
        'SOAPAction': 'GetPrice'
      },
      body: soapBody
    })
    .then(response => response.text())

```

```

.then(str => {
    const parser = new DOMParser();
    const xml = parser.parseFromString(str, "text/xml");

    const product = xml.evaluate(
        "//Product",
        xml,
        null,
        XPathResult.STRING_TYPE,
        null
    ).stringValue;

    const price = xml.evaluate(
        "//Price",
        xml,
        null,
        XPathResult.STRING_TYPE,
        null
    ).stringValue;

    console.log(`Product: ${product}`);
    console.log(`Price: ${price}`);
})
.catch(err => console.error(err));
}

</script>
</body>
</html>

```

BACKEND CODE (index.js)

```

// server.js (SOAP API running on localhost:3000)
const express = require('express');
const path = require('path');
// npm install xmldom
const { DOMParser } = require('xmldom');
// npm install xpath
const xpath = require('xpath');

const app = express();
const PORT = 3000;

// This line says "Please treat any request with
// Content-Type: text/xml as text and populate req.body"
app.use(express.text({ type: 'text/xml' }));

// Dummy function to return product prices
function getProductPrice(productName) {
    // in a live app, products & their prices
    // would ideally come from a database
}

```

```

    "Banana": 0.5,
    "Orange": 0.8
  };

  // convert the first character
  // of product name to uppercase
  // & return its price
  return prices[productName.charAt(0).toUpperCase() + productName.slice(1)]
    || 0;
}

// Serve frontend HTML
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'index.html'));
});

// Serve WSDL hint (optional)
app.get('/wsdl', (req, res) => {
  res.sendFile(path.join(__dirname, 'service.wsdl'));
});
/*
OR
app.get('/wsdl', (req, res) => {
  res.type('text/xml').send(`<!-- Insert WSDL XML as string here -->`);
});
*/

app.post('/soap-api', (req, res) => {
  console.log('SOAP Request Received:', req.body);

  const xml = req.body;
  const doc = new DOMParser().parseFromString(xml);
  const node = xpath.select1('//Product/text()', doc);
  const productName = node ? node.nodeValue : undefined;

  console.log('Product requested:', productName);

  const price = getProductPrice(productName);

  const soapResponse = `
    <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
      <soap:Body>
        <GetPriceResponse>
          <Product>${productName}</Product>
          <Price>${price}</Price>
        </GetPriceResponse>
      </soap:Body>
    </soap:Envelope>
  `;

  res.set('Content-Type', 'text/xml');

```

```
});
```

```
app.listen(PORT, () => console.log(`Server running on  
http://localhost:${PORT}`));
```

Let me explain what the code does more clearly. The frontend code sends a SOAP request, specifying that it wants to make a call to the SOAP function `getProductPrice()`. We make this request using JavaScript's Fetch API.

```
const soapBody = `  
  <soap:Envelope  
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">  
  <soap:Body>  
    <GetPriceRequest>  
      <Product>${product}</Product>  
    </GetPriceRequest>  
  </soap:Body>  
</soap:Envelope>  
`;  
  
fetch('/soap-api', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'text/xml',  
    'SOAPAction': 'GetPrice'  
  },  
  body: soapBody  
})
```

The server (backend) extracts the values sent in the SOAP message-which in this case is the name of a product the client wants to get the price of, calls a function `getProductPrice()`, then builds and returns a proper SOAP response containing the price of the requested product. On the frontend, we parse (extract) that response using `xpath`, and log the data to the console as we have been doing in many examples before. I wish to pull your attention here to a key learning point-which is how on the backend, we extract the data sent via the SOAP message to use. The message comes available in the request object of the `post()` function, and can be accessed from the `body` property like so:

```
const xml = req.body;
```

We then use `DOMParser` which we installed into our project using `npm` earlier-specifically, its `parseFromString()` method to convert the SOAP message from a string to a DOM object so we can run DOM (document) functions on it.

```
const doc = new DOMParser().parseFromString(xml);
```

Next, we extract the data we need from the SOAP message, which in this case is the product name, and we know it is in a `<Product>` tag. We use `xpath` (which we

installed into our project using npm earlier)-specifically its `select1()` method. Here is how we do it:

```
const productName = xpath.select1('//Product/text()', doc)?.nodeValue;
```

Once we got the product name, we used it to get the price of that product, then built and returned a SOAP response:

```
const price = getProductPrice(productName);

const soapResponse = `
  <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
      <GetPriceResponse>
        <Product>${productName}</Product>
        <Price>${price}</Price>
      </GetPriceResponse>
    </soap:Body>
  </soap:Envelope>
`;
```

Back to the frontend, we receive this response, and extract its text:

```
.then(response => response.text())
```

Then, we use skills we have seen before when we learned all about DOM manipulation, the `DOMParser` object, and XPath using the DOM's `evaluate()` method in chapter 15. To refresh your memory, `DOMParser` converts the text (string) into a DOM-like object so that we can manipulate the data as easily as we manipulate other DOM elements using DOM properties-which `.evaluate()` is part of.

```
const parser = new DOMParser();
const xml = parser.parseFromString(str, "text/xml");
```

We then use XPath with `evaluate()` to extract the product name and its price sent back from the XML data of the SOAP response.

```
const product = xml.evaluate(
  "//Product",
  xml,
  null,
  XPathResult.STRING_TYPE,
  null
).stringValue;

const price = xml.evaluate(
  "//Price",
  xml,
  null,
  XPathResult.STRING_TYPE,
  null
).stringValue;
```

```
).stringValue;
```

We then use that data—in our example, we simply log them to the console:

```
console.log(`Product: ${product}`);  
console.log(`Price: ${price}`);
```

If you got this to work, congratulations, you have had a crash course on Node.js and have already understood the underlying principles behind backend API programming. You are well on your way to becoming a full stack JavaScript developer. This great example demonstrates how a soap server can have a function that returns something (in this case the price of a product). Finally, I give you a hint as to how a WSDL file could be provided for your service clients to use. I do not go into showing you what the contents of the WSDL file for such an application should look like, because;

- browsers do not support consuming a WSDL on the frontend
- Using WSDL is optional

However, I will still explain to you how it is used. To have one in our above example, create the file in your application folder, and give it a name with ideally, a .wsdl extension like so:

service.wsdl or soap.wsdl

It's recommended to name the file with an .wsdl extension because:

- It would help text editors recognize and highlight XML syntax.
- External tools (like SOAP clients or validators) properly detect file type.

Once you have created the service.wsdl file in your project folder, serve it like this:

```
app.get('/wsdl', (req, res) => {  
  res.sendFile(path.join(__dirname, 'service.wsdl'));  
});
```

Your clients can then retrieve it from your application endpoint which is being served from your backend:

<http://localhost:3000/wsdl>.

To use or not to use a WSDL file

A WSDL file is not required for a SOAP service to function. You will find that this example will work, and you can send and receive SOAP envelopes manually. This is called using SOAP without contract (WSDL-less) — and it also works just fine for tightly coupled or controlled integrations. A WSDL is required when the client is

auto-generating code from the WSDL (e.g., using `wsimport` in Java or Add Service Reference in .NET), or when you want to make your service discoverable and self-documented for external consumers, or when you're using tools like SoapUI, Postman (for SOAP), or API Gateways that need to understand your contract. Internal SOAP services may skip WSDL, as well as local servers like the one we just setup to use. Public/external SOAP services on the other hand (e.g., airline APIs, payment gateways, legacy ERPs) almost always provide a WSDL.

Why use APIs

Imagine you're building a weather app. You probably don't want to build your own satellite or temperature sensor. Instead, it would be wise for you to use an API provided by a weather service. They have already done the work to gather all the data that you need, probably more efficiently than you would if you tried to, because they may have more expertise in that domain. Using their data would allow you to deliver your application faster. Examples of what APIs provide:

- Weather data
- Currency exchange rates
- News headlines
- User authentication (e.g., login with Google)
- Product details from a store

How JavaScript Communicates with APIs

To communicate with an API, your JavaScript code sends an HTTP request to a server and waits for a response. This is the same principle that web browsers use when you visit a website. The browser sends a request, and the server sends back HTML, CSS, JavaScript, or data.

Sending HTTP Requests

JavaScript gives you tools to send HTTP requests. The modern way to do that is through using `fetch()`. Here is an example of how you would send a GET request to a web application hosted on the following domain: <https://api.example.com/users>

```
fetch('https://api.example.com/users')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Here is an explanation of the request:

- Your web application sends a GET request by default.
- `.json()` reads the response body and parses it as JSON.
- `.catch()` handles any potential errors, such as if the server is down.

Keep in mind that using `fetch()` is the modern way of sending HTTP requests, but the traditional way for a long time, was to use the `XMLHttpRequest` object to do so, and that still works today. It is just more popular to use `fetch()` due to its simpler syntax of `fetch()`, and its promise-based syntax.

What Happens on the server-side

Although we're focusing on JavaScript in the browser, it helps to know what happens on the backend (also often referred to as the server-side), when a request is sent to the server from the frontend (browser). The following are the things to know, and the chain of events that unfold in the process:

- Server-side languages like PHP, Python, Node.js, Java, or Ruby etc live on the server.
- They receive your request, process it (e.g., look up something in a database), and send back a response, often in JSON format.
- JavaScript on your page then receives this response and can update the page dynamically without reloading.

You don't need to know the server-side code. Just know that it's doing the work behind the scenes. If you go on later to learn one of those server-side languages-or maybe you are a backend (server-side) programmer already, you will be able to write the code to handle such requests coming from the frontend.

HTTP Status Codes

When your code talks to a server, it gets back a status code—also known as a response code—to indicate how the request went. Here is a list of the most common HTTP response codes, and what they mean:

HTTP Code	Meaning
200	OK (everything worked)
201	Created (something was added)
400	Bad Request (your request had an error)
401	Unauthorised (you need to log in)
404	Not Found (the thing/resource doesn't exist)
500	Internal Server Error (server crashed)

Here is how you can check the status from the response you get back after a `fetch()` HTTP request:

```
fetch('https://api.example.com/users')
  .then(response => {
    if (!response.ok) {
      throw new Error('Request failed with status ' + response.status);
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

In conclusion, the take-away points on APIs can be summed up as such. APIs allow your JavaScript code to talk to external systems (like databases or services). JavaScript uses HTTP requests (usually through `fetch()`, but also using the older `XMLHttpRequest`) to send or get data. GET and POST are the most common request methods. The server code—which may well be written in a different language—processes your request and sends back a response. JavaScript reads that response (usually in JSON format) and can use it to update the web page. As a developer, you can use `console.log()` or browser tools to inspect the results and debug your requests.

AJAX

AJAX stands for Asynchronous JavaScript and XML. AJAX is a technique that allows web pages to communicate with a server without reloading the page. This makes web applications faster, more dynamic, and user-friendly. For example, when you type in Google Search, the search suggestions appear without refreshing the whole page—that's AJAX at work!

How Does AJAX Relate to JavaScript? AJAX is not a programming language. It's just a way of using JavaScript to send and receive data from a server in the back-

round. JavaScript provides different tools to implement AJAX, such as:

- XMLHttpRequest (XHR) — the old way (still works).
- Fetch API — a modern, simpler way to make requests.
- Axios — a popular external library that simplifies AJAX requests.
- Promises and async/await — tools that help manage asynchronous AJAX requests more easily.

AJAX in Action (Examples)

Let's say we want to fetch user data from a server and display it on a webpage without refreshing.

Old Way (Using XMLHttpRequest)

```
let xhr = new XMLHttpRequest();

xhr.open(
  "GET",
  "https://jsonplaceholder.typicode.com/users",
  true
);
xhr.onload = function () {
  if (xhr.status === 200) {
    // Logs the response from the server
    console.log(xhr.responseText);
  }
};

xhr.send();
```

The example above shows how to make a simple HTTP request using JavaScript's XMLHttpRequest object. Let's look at how it works step-by-step whilst understanding the properties and methods of the XMLHttpRequest object:

- `let xhr = new XMLHttpRequest();` This creates a new HTTP request object. You can use this object to fetch data from a server without reloading the entire page (a key part of AJAX).
- `xhr.open("GET", "https://jsonplaceholder.typicode.com/users", true);` This sets up the request, and does the following:
 - * "GET" tells the server you want to retrieve data.
 - * The URL "<https://jsonplaceholder.typicode.com/users>" points to a demo API (more on this below).
 - * The `true` means the request should be asynchronous—it won't block the rest of your script while waiting for a response.

- `xhr.onload = function () { ... }` This is an event listener on the AJAX request, that calls or runs this anonymous function when the event `onload` occurs on the request. This event (`onload`) always fires (occurs) when the AJAX request is finished. The code we place in the function to be ran is as follows: `* xhr.status === 200` checks if the request was successful. `* xhr.responseText` holds the raw data sent back from the server, usually as a JSON string. It gets printed to the browser console with `console.log()`.
- `xhr.send()`; This `send()` method sends the actual request to the server.

Here is the meaning of the `"https://jsonplaceholder.typicode.com/users"` URL which I used in the `xhr.open(...)` code line above. It is a free fake online REST API used for testing and learning. It's not connected to any real user database. It's perfect for trying out AJAX, `fetch()`, or `XMLHttpRequest()` without needing to set up your own backend. In this example, the `/users` endpoint simply returns a list of fake user profiles in JSON format—ideal for practice.

Note: This URL is safe to use in demos and practice code. However, in real applications, you would replace it with your actual server or API endpoint. The drawback of using `XMLHttpRequest` (XHR) is that XHR has a syntax that is a little more complex compared to the more modern approaches. However, it is still effective and very much supported by all browsers. The above example is a GET request, here is an example making a POST request with the `XMLHttpRequest` object:

HTML code

```
<input type="text" id="username" /><span id="info"></span>
```

JavaScript code

```
let username = document.getElementById('info').value;

const request = new XMLHttpRequest();
const params = "username=" + username.value;

request.open("POST", "auth/checkUsername", true);
request.setRequestHeader(
  "Content-type", "application/x-www-form-urlencoded"
);

request.onreadystatechange = function () {
  if (this.readyState === 4) {
    if (this.status === 200) {
      if (this.responseText !== null) {
```

```

    }
        else
        {
            alert("Ajax error: No data received");
        }
    }
    else
    {
        alert("Ajax error: " + this.statusText);
    }
}
};

request.send(params);

```

This JavaScript code sends a POST request to a server using the XMLHttpRequest object (also known as AJAX). The goal is to send a username to the server and check if it's already taken, then display a message back in the web page.

This line builds the data we want to send to the server.

```
params = "username=" + username.value
```

We take the value from an input field called username and create a string like this: "username=JohnDoe"

This string will be sent to the server just like a form submission.

```
request = new XMLHttpRequest()
```

This creates a new XMLHttpRequest object.

We are opening a new request here:

```
request.open("POST", "auth/checkUsername", true)
```

- "POST" means we're sending data (not just fetching).
- "auth/checkUsername" is the URL (a file or route on the server) that will handle the request.
- true, the second argument passed to request.open() means the request is asynchronous-it will run in the background so the page doesn't freeze while waiting.
- This next block of code sends a Content-Type value meant to be sent as part of the header of the request. This particular Content-Type is important for this request. It tells the server how the data it is sending across is formatted:

```

request.setRequestHeader(
    "Content-Type", "application/x-www-form-urlencoded"
)

```

We're saying: "Hey server, I'm sending form-style data — just like a normal HTML form would."

-This next line sets a function to run every time the state of the request changes:

```
request.onreadystatechange = function () {  
    ...  
}
```

- The request goes through different "states" (0 to 4) as it progresses-and these are stored in the readyState property of the XMLHttpRequest object. When it gets to state 4, it means the response has been fully returned from the server. So, this is where we check if the request is finished:

```
    if (this.readyState == 4) {  
        ...  
    }
```

See below, the different values of the readyState property and what they mean. So here, we confirm if the request is done. If it is, we move on to check if it worked properly.

- This next line checks if the server responded with success:

```
    if (this.status == 200) {  
        ...  
    }
```

Universally, in API programming, the group of status codes normally returned are as follows:

- 200 = Success
 - 404 = Not found
 - 500 = Server error
 - ...and so on.
- Now we check: Did the server actually send us back some text?

```
    if (this.responseText != null) {  
        // use this.responseText here  
    }
```

The "this.responseText" line is referencing the responseText property of the current object, which is XMLHttpRequest object. Basically, if its value is null, therefore, nothing was returned from the server. That is why we check to see if its value is not equal to null (this.responseText !== null) before we proceed to use the data returned within that block. In this example, we take the data returned by the server, and inject it into the DOM as the value of an element on our web page with the id attribute of "info".

```
document.getElementById('info').innerHTML = this.responseText
```

For example: If the server replies with "Username already taken", that message will appear in the page. On the other hand, if `this.responseText` contains nothing (is null), the code in the else block will be run, and we display something on the screen, an alert popup message to inform the user that no data was received:

```
if (this.responseText !== null) {  
    ...  
}  
else  
{  
    alert("Ajax error: No data received");  
}
```

- If the the server returned another status code other than an OK status (200), it means the AJAX request was not successful, hence we also inform the user using a popup alert that there was an Ajax error and whatever the the text that was returned with the status code (`this.statusText`):

```
} else alert("Ajax error: " + this.statusText)
```

-This is how we send the request to the server — along with the params we built earlier (`username=JohnDoe`):

```
request.send(params)
```

- That's all the JavaScript frontend code needed to make the AJAX request. But let's talk a little about how the backend (server-side) will work, so you have a full-rounded perspective about how this works. The server-side code that receives, processes the parameter passed in (`username` intros case), and sends back the response could be handled by any one of the following server-side programming languages:

- PHP file like `checkUsername.php`
- Node.js route like `/auth/checkUsername`
- Python/Django/Flask handler

That server script receives the username, checks it (e.g. in a database), and sends back a message like:

"Username is available" or "Username is already taken"

In the early days of the web, developers had to jump through hoops to make AJAX calls that worked in different browsers-especially Internet Explorer. But modern browsers now universally support `XMLHttpRequest`. Although `XMLHttpRequest` is mostly used to fetch plain text or JSON today, it was originally built for XML data. You can still use it to:

- Send or receive XML from a server
- Parse XML using responseXML

To receive an XML response, you would use the responseXML property of the XMLHttpRequest object instead of the.responseText. For example:

```
if (this.responseXML !== null) {
    // Get data from this.responseXML,
    // not this.responseText e.g.
    let xmlData = this.responseXML
}
else
{
    alert("Ajax error: No data received");
}
```

Basically, the property that will contain the returned data always depends on the type of the data that is returned from the server. Newer APIs like fetch() make asynchronous calls even easier to write and manage. I will talk about how to use the built-in JavaScript fetch() API next.

The responseType property

Sometimes, you will see developers use the .responseType property when sending an AJAX request with the XMLHttpRequest object. This specifies the type of the data you want to get back as the response from the server. In our example above we did not use it, but if we did, we would have added it like this:

```
let xhr = new XMLHttpRequest();

xhr.open(
    "GET",
    "https://jsonplaceholder.typicode.com/users",
    true
);
// specifying doc type wanted back
xhr.responseType = 'document';
xhr.onload = function () {
    if (xhr.status === 200) {
        // Logs the response from the server
        console.log(xhr.responseText);
    }
};

xhr.send();
```

The new line we have added is:

```
xhr.responseType = 'document';
```

Here we are specifying that the data type of the response we expect back should be a document. This ‘document’ means a Document Object Model (DOM) so the data should be parsed (converted) into DOM format for us. We would then be able to directly use DOM methods we are already familiar with on it to extract the data we need, or even inject it into the DOM if it is an HTML document, and we need to inject HTML and not text to any part of a web page. Using the `responseType` property in your request is good because it will determine how the `XMLHttpRequest` object formats the data it gets back to get it ready for you to use. You do not have to send it with every request. Let me show you why. When you use the `XMLHttpRequest` object to fetch data from a server, the browser usually treats the response as plain text by default. But sometimes, you may want the response to be treated as a different type of data, like XML or JSON. That’s where the `.responseType` property comes in. Setting `xhr.responseType` tells the browser how to handle and interpret the response. For example If you are loading an XML file, and you want the browser to automatically parse it into a usable document (just like an HTML or XML page), you can do it like this:

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'books.xml');

// Ask the browser to treat the response as a document
xhr.responseType = 'document';

xhr.onload = () => {
  // Now this is a parsed XML document
  const xmlDoc = xhr.response;
  const titles = xmlDoc.getElementsByTagName('title');

  for (let i = 0; i < titles.length; i++) {
    console.log("Book Title:", titles[i].textContent);
  }
};

xhr.send();
```

By setting `xhr.responseType = 'document'`, we tell the browser to treat the response like a real XML or HTML document, which allows us to use powerful DOM methods on it. Notice that we retrieved the response in this case from the `.response` property of the `XMLHttpRequest` object.

```
const xmlDoc = xhr.response;
```

We are then able to use a DOM method `getElementsByTagName()` on the parsed data like this:

```
const authors = xmlDoc.getElementsByTagName('author');
```

The `xhr.response` property is not meant to hold XML data, but because we requested for it to be parsed into a doc (`xhr.responseType = 'document';`), That data was made available on the `.response` property (`const xmlDoc = xhr.response;`).

However, you do not have to set the `.responseType` with every request. If the server returns the XML with the correct Content-Type (like `application/xml` or `text/xml`), then the browser will automatically parse the result and make it available for you to use on the `.responseXML` property-a property which is reserved to hold data if only XML is returned. You can grab the returned data to use like so:

```
const xmlDoc = xhr.responseXML;
```

Here's a version that works without setting `responseType`. Note that you have to retrieve the data on the `.responseXML` property:

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'books.xml');

xhr.onload = () => {
  // Also gives you a usable XML
  const xmlDoc = xhr.responseXML; document
  const authors = xmlDoc.getElementsByTagName('author');

  for (let i = 0; i < authors.length; i++) {
    console.log("Author:", authors[i].textContent);
  }
};

xhr.send();
```

Notice you can use a document object method like `getElementsByTagName()`, and if you try to try to get the data from the `.response` property instead of `.responseXML`, you will get an error:

```
"xmlDoc.getElementsByTagName is not a function at xhr.onload"
```

This works as long as the server sends the XML file with the correct content type.

Your take-away from here should be that

- Setting `.responseType = 'document'` is helpful when loading XML or HTML and you want to work with it using DOM methods.
- But if you're loading XML from a well-configured server that communicates back the right headers-that include the response data type it is sending, using `xhr.responseXML` will also work just fine even without setting `responseType`.

I though I should let you know about this `responseType` property so you're prepared when you need more control over how data is handled in the browser.

The readyState property

The `readyState` property of an `XMLHttpRequest` object tells you the current status or stage of the request. It changes as the request progresses. There are five (5) different states the request goes through from start to finish, and they are all recorded on the `readyState` property. To know what state the request is in, at any given time, you just have to check for the value of `readyState`. Here's a complete list of the 5 `readyState` values and what they mean:

Value	Name	Meaning
0	UNSENT	The request has been created, but <code>.open()</code> has not been called yet
1	OPENED	<code>.open()</code> has been called. You can now set headers or call <code>.send()</code>
2	HEADERS_RECEIVED	<code>.send()</code> has been called, and the response headers have been received
3	LOADING	The browser is receiving the response body (data is loading)
4	DONE	The request is complete, and the response is fully received

You can use an event listener to track the changes in the value of this `readyState` property as the request progresses, and react to them. This is very powerful because even though your AJAX request calls are happening behind the scenes (asynchronously) of your application without your user being aware, you still have complete control over their progress because you can track these changes, and update your users on what's going on at any given point.

To do this tracking, you do so using another property of the `XMLHttpRequest` object known as the `onreadystatechange` which is an event listener, albeit a JavaScript built-in one. Here is an example of how you can use the `onreadystatechange` event to track these stages of your AJAX request:

```
let xhr = new XMLHttpRequest();

xhr.onreadystatechange = function () {
    console.log("readyState:", xhr.readyState);

    if (xhr.readyState === 4 && xhr.status === 200) {
        console.log("Response:", xhr.responseText);
    }
};
```

```
xhr.open("GET", "https://jsonplaceholder.typicode.com/users", true);

xhr.send();
```

This code will make an AJAX request to the same URL we used above to get sample user data, but it will print the value of the `readyState` property of the request object (`XMLHttpRequest`) each time it changes, so you can observe how the request progresses from start to finish.

Modern Way (Using `fetch()`)

The `fetch()` function is a built-in JavaScript function used to make HTTP requests. It allows you to retrieve data from APIs, send data to servers, and handle responses asynchronously. Unlike `Axios`, `fetch()` returns a `Promise` and does not automatically convert JSON responses—you need to call `.json()` manually. It is widely used because:

- It's native to JavaScript (no need to install anything).
- It supports modern `async/await` syntax for cleaner code.
- It provides fine control over requests, including headers and methods.

However, `fetch()` does not reject on HTTP errors (e.g., 404 or 500), so you must manually check `response.ok` to handle errors properly. Despite this, it remains a powerful and lightweight tool for making AJAX requests in JavaScript. Here is an example of using `fetch()` to make an Ajax request to the following endpoint: `"https://jsonplaceholder.typicode.com/users"`, logging the result to console, and handling any errors that occurred in the request:

```
fetch(
  "https://jsonplaceholder.typicode.com/users"
)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error:", error));
```

This line converts the response to JSON: `.then(response => response.json())`

Let's convert the exact same example we wrote above for the `XMLHttpRequest` object using `fetch`. Just like before, we're sending a POST request with the username, and showing the server's response inside an element with `id="info"`.

```
const params = new URLSearchParams();
params.append("username", username.value);

fetch("auth/checkUsername", {
```

```

"Content-Type": "application/x-www-form-urlencoded"
  },
  body: params
})
.then(response => {
  if (!response.ok) {
    throw new Error("Network response was not OK");
  }
  // Read the response as plain text
  return response.text();
})
.then(data => {
  document.getElementById("info").innerHTML = data;
})
.catch(error => {
  alert("Fetch error: " + error.message);
});

```

Explanation:

- We create a `URLSearchParams` object — a fancy way to build form-style data. It's like saying: `username=JohnDoe`.
- This tells the browser where to send the request — to a server file or route (same as before).
- This waits for the server's reply:

```

.then(response => {
  if (!response.ok) {
    throw new Error("Network response was not OK");
  }
  return response.text();
})

```

If the response is not OK (e.g. 404 or 500), we throw an error, otherwise, If all is good, we grab the response as plain text and display it inside the `#info` element in the HTML.

```

.then(data => {
  document.getElementById("info").innerHTML = data;
})

```

If something goes wrong anywhere above (bad URL, server offline, etc.), we catch the error and alert the user.

```

.catch(error => {
  alert("Fetch error: " + error.message);
});

```

Some reasons why you may want to use `fetch()` over `XMLHttpRequest` are as follows:

- `fetch()` uses promises (cleaner code) but `XMLHttpRequest` does not
- `fetch()` has less code, and is easier to write
- `fetch()` has built-in error handling while `XMLHttpRequest` does not

Asynchronous programming

Asynchronous programming is when code runs but the rest of your program does not have to wait for it to finish. You can still interact with your application program in different ways while waiting for the result of the running code. There would naturally be some kind of code that listens for the event of the result of the asynchronous (background) task returning being completed. That listener will take some action based on that, even if that action is simply to display a notification on screen to inform the user of the background task's completion.

If a function is run synchronously-and this is the default behaviour, the code parser will wait at that line for it to run completely before moving on to the next line. Asynchronous is the opposite of that. If a function is run asynchronously, it means that the JavaScript parser will not stop at that line and wait for it to finish its job before proceeding. Rather, that function will simply be made to run in the background while the parser will carry on executing the rest of the lines of code after that line. JavaScript has some of its own (built-in) asynchronous functions, one of which you have seen above; the `setTimeout()` function. So in brief, an asynchronous function schedules a task to run later without blocking the execution of other code. This is where callbacks come in handy, hence why they have always been used in JavaScript. A callback is usually a function (usually anonymous) to be run when a program has finished running. It is therefore ideal in asynchronous (background) programs. In fact, call backs was always the de-facto way to write these kind of programs. ES6 (ECMAScript 2015) saw the introduction of promises which replaced callbacks. Then in 2017 came the even more refined way to do the same thing that Promises do, and that was `async/await`. `Async/await`, though not meant to replace Promises, was a different way to do what promises do. It deals with the response in such an elegant and seamless way that it looks more like synchronous programming. JavaScript has many built-in asynchronous functions that are useful to know. Here is a list of the important ones with simple examples. You may not understand the examples now, but as we delve into those concepts immediately following below, all will be explained

-1) `setTimeout()` – Runs a function after a delay (already mentioned).

```
setTimeout(
  () => console.log("This runs after 2 seconds"),
  2000
```

```
);
```

-2) setInterval() – Repeats a function at a fixed time interval.

```
let interval = setInterval(  
  () => console.log("Runs every second"),  
  1000  
);
```

```
// Stops it when needed.  
clearInterval(interval);
```

- 3. fetch() – Makes network requests (AJAX calls, APIs).

```
fetch("https://jsonplaceholder.typicode.com/posts")  
  .then(response => response.json())  
  .then(data => console.log(data));
```

-4) Promise-based APIs – Many modern APIs return Promises, which are inherently asynchronous.

```
navigator.geolocation.getCurrentPosition(  
  position => console.log(position),  
  error => console.log(error)  
);
```

-5) requestAnimationFrame() – Optimised function for animations.

```
function animate() {  
  console.log("Frame rendered");  
  requestAnimationFrame(animate);  
}
```

```
requestAnimationFrame(animate);
```

- 6. WebSockets & Event Listeners – These run asynchronously, waiting for events to happen.

```
document.addEventListener("click", () =>  
  console.log("Clicked!"));
```

- 7. async/await functions – Used to simplify Promises.

```
async function getData() {  
  let response = await fetch(  
    "https://jsonplaceholder.typicode.com/posts"  
  );  
  
  let data = await response.json();  
  console.log(data);  
}
```

```
getData();
```


Before we look at promises and `async/await`, it's essential to understand the concept of a call back, which existed to solve more or less the same problem that promises are solving.

The same wait, written three ways

All three do exactly the same thing. Each generation made the previous one easier to read.

1. CALLBACK — pass a function to be called when it is done

```
createPost(post, getPosts);
```

Works, but nest a few of these and you get the "pyramid" that made promises necessary.

2. PROMISE — `.then()` for success, `.catch()` for failure

```
fetch(url)
  .then(response => response.json())
  .catch(error => console.error(error));
```

3. ASYNC / AWAIT — the same promise, read top to bottom

```
try {
  const response = await fetch(url);
  const data = await response.json();
} catch (error) { console.error(error); }
```

- Figure 22.1 — The same wait, written three ways* Let us dive right in, and demonstrate with examples, a callback, then move on to show how a promise would be used, and finally, how Async/Await would be used to solve the same problem.

a) Callback

```
<!doctype html>
<html>
  <head>
    <title></title>
  </head>
  <body>
    <ul id="gusUl">
      </ul>
  </body>
</html>

const posts = [
  { title: 'post one', body: 'This is one'},
  { title: 'post two', body: 'This is two'}
];

let ul = document.querySelector("#gusUl");
```

```

let output = '';

    posts.forEach((post, index) => {
        output += `<li>${post.title}</li>`;
    });

    return output;
}

function createPost(post)
{
    setTimeout(() => {
        posts.push(post);
    }, 2000);
}

createPost(
    { title: 'post three', body: 'This is three' }
);

// display the post data in the UI (web page)
ul.innerHTML = getPosts();

```

You have here two functions, `getPosts()` that reads data from an array `posts`, and renders it as HTML list tags to display the posts. The other function, `createPost()` adds more data to the `posts` array. If you ran the code now and looked in your HTML you would find that only post one and post two are displayed as `<li>` elements. Though we first of all call `createPost()` before we call `getPosts()`, you would think that we will get three `<li>` tags displayed, but we only get two. The `<li>` element post three created by `createPost()` above did not appear because the code in `createPost()` runs in a `setTimeout()` function which ensures that its task is only executed after a two seconds delay. The issue is that `setTimeout()` is asynchronous, meaning `getPosts()` returns before it completes. This means that the next function we call, `getPosts()`, runs and fetches the current `posts` data before the additional post is created, hence, it only finds two items in the `posts` array. To resolve this issue, you need to use callbacks, Promises, or `async/await`. Let's see how a callback can fix the issue. We will modify the `createPost()` function to accept a callback function as its second argument. This callback function is `getPosts()` which should then be called by `createPost()` after it has finished doing its task of adding data to the `posts` array. This is to ensure that after updating the data in `posts`, the list of posts displayed on screen is refreshed by `getPosts()` to reflect that update. Be sure to pass the `getPosts()` function then as the second argument to `createPost()` when you call it. The modified code will look like this:

```

function getPosts()
{
    let output = '';

    posts.forEach((post, index) => {
        output += `<li>${post.title}</li>`;
    });

    ul.innerHTML = output;
}

function createPost(post, callback)
{
    setTimeout(() => {
        posts.push(post);

        // call the callback function
        callback();
    }, 2000);
}

// pass getPosts as a function reference, not as a
// function call meaning getPosts, not getPosts()
createPost( { title: 'post three', body: 'This
    is three'}, getPosts);

// Initially display posts
getPosts();

```

Now when we call `createPost()` as we do above, because we pass it a callback, we get the three list items displayed correctly in the browser. It is crucial to understand why the callback solution works:

- Note that earlier, we called both the `createPost()` and `getPosts()` functions separately, first to create a new post, and then to fetch all the posts to update the list in the UI like so:

```

createPost(
    { title: 'post three', body: 'This is three' }
);

ul.innerHTML = getPosts();

```

But that was because `getPosts()` was returning data (output), and we were using that to update the list in the UI outside of `getPosts()`. Well, we now know that it obviously did not work because `getPosts()` was returning posts data (output) which it was fetching before `createPost()` had done its job. This time we call both functions in one line, by calling `createPost()` and passing it `getPosts()` as a callback, then we call `getPosts()` one more time in the end just to get the initial,

out-of-date posts, before the update happens in `getPosts()` thanks to the callback we are sending:

```
createPost( { title: 'post three', body: 'This
            is three'}, getPosts);

// Initially display posts
getPosts();
```

Note that we are now calling `getPosts()` like so:

```
getPosts();
```

And not like so:

```
ul.innerHTML = getPosts();
```

When `getPosts()` finishes its job, it is updating the UI update inside itself, because having been passed as a callback into `createPost()`, it has the updated post data from the delayed task of `createPost()`. This is the crucial part of how this code is made to work.

-We have now changed the code that calls `getPosts()` to be the callback that we pass to `createPost()`. That is why we get rid of the return statement from `getPosts()`. We will not need it to return the output variable, because by the time it does that within the `setTimeout()` function, the other code will have ran and so we will never see any update in the browser. To see that update, we need to make the UI update directly inside `getPosts()` so that the callback function will run that. That is why in `getPosts()` we replace the return statement with this code to update the UI:

```
ul.innerHTML = output;
```

-Note that whenever you call another function and pass a function as a callback argument to it, that function being passed as a callback should be passed as a reference only. This means that you should not include parenthesis after the function name as you would do if you were running it directly. That is why when we call `createPost()`, we pass it the `getPosts` callback function as a reference like so `getPosts` and not `getPosts()`:

```
createPost(
  { title: 'post three', body: 'This is three'},
  getPosts
);
```

b) A promise To convert the above example to use a promise, we will leave the `getPosts()` function exactly the same as when using the callback function above, but we will modify the `createPost()` function. The final code will look like this:

```

// keep the same
function getPosts()
{
    let output = '';

    posts.forEach((post, index) => {
        output += `<li>${post.title}</li>`;
    });

    ul.innerHTML = output;
}

function createPost(post)
{
    return new Promise((resolve, reject) => {
        //resolve is for when things go right, &
        // reject is for when the request fails
        setTimeout(() => {
            posts.push(post);
            const error = false;

            if (!error)
            {
                resolve();
            }
            else
            {
                reject('Error: Something went wrong');
            }
        }, 2000);
    });
}

// pass the callback function (getPosts) inside .then()
createPost( { title: 'post three', body: 'This is
three'} ).then(getPosts)
    .catch(err => console.log(err));

// Initially display posts
getPosts();

```

Note that the `then()` will only be called if it resolves. But if it fails and is rejected, it will be caught by the `catch()` section. Many get APIs now, as well as the Mongoose library, node.js etc use the promise feature.

-`Promise.all()` The `promise.all()` function is used to handle several promises, for example, replace the call to `createPosts()` above with the following:

```

const promise1 = Promise.resolve('Hello world');
const promise2 = 10;
const promise3 = new Promise(
  (resolve, reject)
    => setTimeout(resolve, 2000,
      'Goodbye')
);

const promise4 =
  fetch('https://jsonplaceholder.typicode.com/users').then(res =>
    res.json());

Promise.all(
  [promise1, promise2, promise3,
    promise4])
  .then(values => console.log(values)
);

```

Note that promise4 uses the fetch API, an online free test API for json data. That is how it gets its json data. This data coming from the fetch API already uses promises, so you just get the response with then(). Promise3 uses the setTimeout() function to call after 2 seconds, so the total amount of time promise.all() takes to run is 2 seconds, as it always uses the longest time that any of its promises takes to resolve.

c) Async/Await

Async/await literally stands for asynchronous and await. You use them together on a block of code to get the desired result. Start by declaring it using the 'async' keyword one space before the 'function' keyword to mark that function so JavaScript knows that it should be run asynchronously. That is exactly its purpose, to allow you to create your custom asynchronous functions. For example:

```

async function myFunction() {
  // await ... means wait for this code to be run before proceeding
  await functionToCall(anyArguments);
}

```

Async/await uses a significantly more simple and elegant syntax than promises:

```

// then call it like so
myFunction();

```

Let's modify our code above to use promises:

```

const posts = [
  { title: 'post one', body: 'This is one'},
  { title: 'post two', body: 'This is two'}
];

```

```

function getPosts()
{
    let output = '';

    posts.forEach((post, index) => {
        output += `<li>${post.title}</li>`;
    });

    ul.innerHTML = output;
}

function createPost(post)
{
    // Make createPost() return a Promise
    return new Promise((resolve) => {
        setTimeout(() => {
            posts.push(post);
            // Resolves the promise after pushing the post
            resolve();
        }, 2000);
    });
}

async function init() {
    await createPost(
        { title: 'post three', body: 'This is three' }
    );

    // Runs after createPost() is done
    getPosts();
}

init();

```

We're basically saying that we will wait (await) for the new post to be created, then (before we) call `getPosts()` to get the update. This makes for cleaner and more readable code, hence it is said that `async/await` is the more elegant way to do a promise. Notice however, that though we used `async/await` to call the `createPost()` function, we still use a promise within `createPost()` so it returns a promise

```

function createPost(post)
{
    return new Promise((resolve) => {
        ...
    });
}

```

This is because you cannot use pure `async/await` to resolve the issue without using a promise in combination with it. `Async/await` is built to work with Promises. If

there's no Promise, await does nothing. This is because async/await is just syntactic sugar over Promises, and so they only work with Promises. If a the function being called (like `createPost()` in our example) does not return a Promise, await has nothing to wait for, and execution therefore will move forward immediately with no asynchrony. The other thing to understand is that the line within `createPost()` that calls `resolve()` is so important. Calling `resolve()` ensures async/await works. It is the thing that makes a promise return itself. Any value to be returned by the function should be passed into `resolve()` as an argument eg:

```
resolve(arguments);
```

With that in place, only then will the await code that calls `createPost()`

```
await createPost(...)
```

be able to correctly pause execution until the Promise resolves. Here is a modified example of using async/await with promises, structured slightly differently to achieve the same thing. Hopefully by now, you can read through the code and understand everything about it without my explanation. Read the inline comments as a guide.

```
const posts = [
  { title: 'post one', body: 'This is one' },
  { title: 'post two', body: 'This is two' }
];

function getPosts() {
  return new Promise((resolve) => {
    setTimeout(() => {
      let output = '';
      posts.forEach((post) => {
        // List items are created
        // as a template literal
        output += `<li>${post.title}</li>`;
      });

      // Return the final string after the delay
      resolve(output);
    }, 1000);
  });
}

function createPost(post) {
  return new Promise((resolve) => {
    setTimeout(() => {
      posts.push(post);

      // Ensure it waits before proceeding
    });
  });
}
```



```

});
}

// Wait for createPost() first, then update the UI
async function updateUI() {
    // Wait for post creation
    await createPost(
        { title: 'post three', body: 'This is three' }
    );

    let ul = document.querySelector("#gusUl");

    // Wait for posts and update UI
    ul.innerHTML = await getPosts();
}

updateUI();

```

Let me point out that, as you can see, you may use `await` as often as needed whenever you have to wait for some asynchronous code to return data—just make sure that the function being awaited returns a Promise. On the asynchronous side, the `resolve()` function inside a Promise is key: it signals that the operation is complete. Without `resolve()`, the Promise remains pending indefinitely, causing the awaiting code to hang. Additionally, `resolve(value)` is how data is passed back from the Promise, so make sure to pass any return values as arguments to `resolve()`. For error handling, `reject(error)` can be used to signal a failure, which can then be caught with `.catch()` or a `try...catch` block on the calling side (the script that calls the promise function). You should use a `try-catch` block when making a request to a promise that you know may send back an error (via its `reject()` function) if an error occurs. Here is an example of how to make a request to a promise using a `try` and `catch` to handle any returned errors:

```

function fetchData() {
    return new Promise((resolve, reject) => {
        // Simulate success or failure
        let success = false;

        if (success) {
            resolve("Data loaded successfully!");
        } else {
            reject("Error: Failed to fetch data.");
        }
    });
}

async function loadData() {
    try {

```

```

// This runs if the promise resolves
    console.log(data);
  } catch (error) {
    // This runs if the promise rejects
    console.error(error);
  }
}

loadData();

```

To reinforce the point, this is the reason why most modern API's implement promises, to make them easier to consume using `async/await`. Here is an example of how to use `async/await` to consume an API using the `fetch` API of JavaScript. To use `async` with the `fetch` API, do it like so:

Without `async/await`:

```

fetch("https://jsonplaceholder.typicode.com/users")
  // Convert resp to JSON
  .then(response => response.json())
  .then(data => console.log(data))
  // Log the response
  .catch(error => console.error("Error:", error));

```

With `async/await`:

```

async function fetchUsers()
{
  try {
    const res = await fetch(
      'https://jsonplaceholder.typicode.com/users'
    );

    const data = await res.json();
    console.log(data);
  } catch (error) {
    console.error("Error:", error);
  }
}

fetchUsers();

```

We call `await` the second time on the response from the `fetch` API

```

await res.json();

```

This is because `.json()` is a function that converts the data into json format, and it might need time to do its work too. We therefore want to wait for that to happen before we use the data.

Using Axios (external library)

Axios is a popular JavaScript library used for making HTTP requests (GET, POST, etc). It simplifies fetching data from APIs, sending data to servers, and handling responses. Axios works in both browsers and Node.js. When used in Node.js, it is installed as any of the other Node.js packages using the `npm ...` command (e.g. `npm install axios`). When used in the browser, it can be included via a CDN, or just installed on a machine. Axios is often preferred over the built-in `fetch()` because:

- It is promise-based, which is cleaner than callbacks
- It automatically transforms JSON data (no need for `.json()`).
- It handles errors better by rejecting failed requests.
- It supports request timeouts, making it easy to prevent long waits.
- It allows interceptors, which help modify requests or responses globally.
- It works on browsers and Node.js, unlike `fetch`, which is browser-only

With Axios, you can make GET, POST, PUT, DELETE requests easily, making it a great tool for handling AJAX operations in modern web development. Here is a simple example of how to make an Ajax request to the following endpoint: `"https://jsonplaceholder.typicode.com/users"`, log the response to the console and handle any errors that occur:

```
axios.get("https://jsonplaceholder.typicode.com/users")
  .then(response => console.log(response.data))
  .catch(error => console.error("Error:", error));
```

Here is another example of using Axios with `async/await`:

CSS code

```
body {
  font-family: Arial, sans-serif;
  padding: 20px;
  background: #f7f7f7;
}

h1 {
  font-weight: bold;
  color: dodgerblue;
}
```

HTML code (eg index.html)

```
<!DOCTYPE html>
<head>
  <title>The JavaScript Blueprint</title>
  <link rel="stylesheet" href="index.css" type="text/css">
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
</head>
<body>
  <h1 id="myHeading">The JavaScript Blueprint</h1>

  <h2>Read CSV File and Display as Table</h2>
  <button onclick="fetchPhotos()">Fetch Photos</button>
  <pre id="output"></pre>

  <script type="module" src="/index.js"></script>

</body>
</html>
```

JS code (eg index.js)

```
async function fetchPhotos() {
  try {
    const response =
      await axios.get(
        'https://jsonplaceholder.typicode.com/photos'
      );

    // Display in the page
    document.getElementById('output')
      .textContent = JSON.stringify(response.data, null, 2);
  } catch (error) {
    console.error('Error fetching photos:', error);
  }
}

window.fetchPhotos = fetchPhotos;
```

In this example, we use Axios to access the publicly available endpoint for API testing. We specify that the data we want to get back as a response is photos. This is because we know the endpoint has that resource, and the API providers or their documentation has told us so. It is the same endpoint we saw in a previous example which returned a collection of dummy users' data.

```
const response = await axios.get(
  'https://jsonplaceholder.typicode.com/users'
);
```

In this case, however, we wanted to get data on photos, hence we used this endpoint:

```
await axios.get(
  'https://jsonplaceholder.typicode.com/photos'
);
```

Notice that for axios to work, we needed to reference the library's Content Delivery Network (CDN) in our HTML code (index.html) like this:

```
<script
  src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
```

If you test this in your browser, you will find that it gets and returns all photos, that the endpoint can return. You can narrow down the result you get back by optionally passing an extra parameter or parameters to the `axios.get()` method. How you pass the parameter depends on what you want to get from the API. The following are two ways to pass in parameters with the API request, which determine the response returned.

a) You can choose to fetch and return only photos having an `albumId` that matches the `albumId` passed in. This means you get back only photos that belong to the album whose id you passed in. You can do that by passing an object literal (`{...}`) as the second argument to the `axios.get()` function. For example:

```
const response = await axios.get(
  'https://jsonplaceholder.typicode.com/photos',
  {
    params: { albumId }
  });
```

This object literal sent as a second argument to `axios.get()` is also known as the request configuration object.

b) You can also choose to fetch and return a specific (single) photo by passing in the unique id of the photo, if you know it. This id argument is not passed as a second argument to `axios.get()`. Rather, you have to do so by changing the last value in that endpoint path string-which in this case is `/photos`. Change it to the id of the photo eg `/3` if the id is 3. For example:

```
const response = await axios.get(
  'https://jsonplaceholder.typicode.com/photos/3'
)
```

Or, if you have the id of the photo as a variable, you can pass it in dynamically like so:

```
let photoId = 6;
const response = await axios.get(`https://
  jsonplaceholder.typicode.com/photos/${photoId}`);
```

Notice that because we are mixing a variable with a string here, we have to let JavaScript know that `photoId` is a variable so it can parse. We do this by wrapping the whole API endpoint path string in backtick (``) and place the variable within `${}` characters. This is a template literal, which is what you use in JavaScript whenever you wish to display a string that contains dynamic variable, and you want to make it clear to the JavaScript parser (interpreter) which of the elements in the string are variables. See chapter 23 (Templates), where I talk in depth about String Literals & Template Strings. The variables must be wrapped in `${}` like so:

```
This is a string ${variableName} and more text
```

Of course you do not need to use a string literal, especially if there is only one variable. In our case, we can also simply concatenate the variable to the end of the string like so, and it will work just the same:

```
const response = await axios.get(
  'https://jsonplaceholder.typicode.com/photos/' + photoId
);
```

Now you know how to pass data to an API endpoint when using Axios. Remember that if you have many variables to send through, it's usually easier for you to send that data in the second argument of `axios.get()` as an object literal. This is usually the case with POST requests when you want to create some resource on the server you are passing data to. This data could be user data you have collected from a signup form on your website, or their shopping cart data after they click on Checkout etc. This data, which is sent as an object literal, also known as a request configuration object can also just contain different kinds of data to be used by the receiving server to determine how it will respond to the request. Whatever the case, it is up to the creators of the API to determine what kind of data, or format of it is to be sent as data in requests to the API endpoints, and which endpoints (URLs) to use for each request type. Once more, here is how to fetch only one piece of data, the data of a photo with the id of 8:

```
async function fetchPhotos() {
  try {
    const photoId = 8;

    const response = await axios.get(
      'https://jsonplaceholder.typicode.com/photos/' + photoId
    );
```

```

document.getElementById('output').textContent =
    JSON.stringify(response.data, null, 2);
} catch (error) {
    console.error('Error fetching photos:', error);
}
}

window.fetchPhotos = fetchPhotos;

```

Another thing to be aware of is that you, as the consumer of the API are not responsible for what the endpoint, or the URL of the API should look like, or what kind of data is to be sent with the request you make to the API, or what kind of data it will return. All that is the job of the creators of the API service. Your responsibility is to ask for, or look up the documentation online, so you can get all this information.

WEBSOCKETS

Have you ever used a chat app where new messages just pop up instantly — without refreshing the page? Or maybe you've seen live sports scores update in real-time on a website? That kind of instant, back-and-forth communication is made possible by a powerful tool called WebSockets. So, what is exactly a WebSocket? A WebSocket is like opening a direct phone line between your browser and a server. Unlike regular HTTP requests (which are one-way and short-lived), a WebSocket connection is two-way and always open — until you decide to close it.

- With HTTP: The browser makes a request → the server sends a response → done.
- With WebSockets: The browser and server can both send messages to each other at any time-like chatting over a walkie-talkie.

Why Use WebSockets

WebSockets are useful when you need:

- Real-time updates (e.g., chats, online games, live notifications)
- Continuous data exchange (e.g., sensor readings, stock prices, collaborative apps)
- Lower latency (no repeated "asking" the server — data just flows when needed)

Why it works

Here is how it works, in a nutshell:

- Your browser opens a WebSocket connection to a server.
- The server accepts and keeps that connection alive.
- Both sides can now send and receive data as needed — no reloading, no waiting.

Think of it like opening a tunnel between your app and the server where they can toss messages back and forth instantly. Earlier in chapter 18 (File Management), we learned how to send binary file data over WebSocket. We saw how WebSockets can be used to send files—whether it is through the uploading of images or documents in real-time, or sending chunks of data such as audio or video. We also saw how using a WebSocket gives you a lot more control over a file transfer process, than traditional uploading does.

Where networking actually lives

We have now met every tool this chapter set out to cover — `fetch()`, `XMLHttpRequest`, `Axios` and `WebSockets` — so this is a good moment to step back and ask a question that is easy to get wrong: which of these is actually part of JavaScript? The answer is none of them. JavaScript itself has no networking. The language has no way to open a socket, no way to send a packet, nothing that knows what HTTP is. What it has instead is the ability to use whatever networking the surrounding platform hands it — and that platform is either the browser or Node.js. This matters more than it sounds, because it explains why the same language can feel so different in the two places.

Networking in the browser

In the browser, every networking tool you have used in this chapter is supplied by the browser itself:

- `fetch()` — for making HTTP requests
- `XMLHttpRequest` — the older way of doing the same
- `WebSocket` — for a two-way connection that stays open
- `<script src="...">` — loading an external script

Notice what is missing from that list. There is no way to open a raw TCP or UDP connection, no way to listen on a port, no way to act as a server. The browser deliberately does not give you those, because a web page is untrusted code running on somebody else's machine. Everything you are allowed to do goes through the browser, and the browser applies its rules — which is exactly why CORS kept turning up earlier in this chapter.

Networking in Node.js

On a server, the restrictions fall away. Node.js gives JavaScript the same networking a language like Python or Go would have:

- open TCP and UDP sockets
- run HTTP and HTTPS servers, not just make requests to them
- call other servers' APIs
- use modules such as `http`, `net`, `dns` and `ws` for full control

This is why the SOAP examples earlier had to be routed through a Node.js server, and why the WebSocket server in Chapter 18 was a Node.js program rather than a browser one. In Node.js, JavaScript can act as a full networking language. So the honest summary is this: JavaScript is not a networking language, and it is also not not one. It borrows the networking of wherever it is running. In the browser you get a careful, guarded subset, and everything in this chapter has been about using that subset well.

LIBRARIES

Notie, is an easy to use notification library

- Note: see how i implemented it in the Golang 'Hotel-booking' app on github.
- Get it from:

<http://github.com/jaredreich/notie>

-Paste the provided JS & CSS cdn references on your web page. Then write custom function like this to use it:

```
function notify(msg, msgType)
{
    notie.alert({
        type: msgType,
        text: msg
    });
}
```

-Finally; wherever you wish to use it in your code, call your custom function to notify the user of something in the browser like so:

```
notify("Thanks for confirming", "success");
```

QUIZ — Extensions (APIs and Libraries)

This page contains the Q & A (questions and answers) for this chapter — Chapter 22: Extensions (APIs and Libraries). Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 9 to 13 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. What is an API, in one sentence, and what does an "endpoint" mean?

Clue: the chapter's restaurant analogy is the one to reach for.

2. What are the main differences between a REST API and a SOAP API, and why is SOAP usually handled outside the browser?

Clue: one speaks JSON, one speaks XML, and only one of them trips over browser security.

3. Match these status codes to their meanings: 200, 201, 400, 401, 404, 500.

Clue: they come in bands, and the first digit tells you who is at fault.

4. `response.ok` is true for which status codes? Why is checking it not the same as checking whether `fetch()` succeeded?

Clue: a 404 is a perfectly successful request.

5. What does AJAX stand for, and why is the "X" a little out of date?

Clue: what format do APIs actually return these days?

6. Put these in order from oldest to newest, and say what each improved on: `async/await`, `callbacks`, `XMLHttpRequest`, `promises`, `fetch()`.

Clue: two are about *making requests*, three are about *handling the waiting*.

7. In the chapter's posts example, `createPost()` adds a post inside a `setTimeout()`, and the new post never appears on the page. Why?

Clue: nobody waited.

8. What does `Promise.all()` do, and what happens if one of the promises rejects?

Clue: it is all or nothing.

9. EXERCISE. Write a function that returns a promise resolving to "Done!" after a short delay, then use `.then()` to print it.
- Clue: `new Promise` takes a function with two parameters.
10. EXERCISE. Rewrite that same call using `async/await` inside a `try...catch`.
- Clue: `await` only works inside an `async` function.
11. EXERCISE. Fix the posts example using a callback, so that the third post appears.
- Clue: pass the function itself, not the result of calling it.
12. EXERCISE. Use `Promise.all()` to wait for two promises and print both results together.
- Clue: what comes back is an array, in the order you passed them.
13. EXERCISE. Write a `fetch()` call that checks the status code and throws if the response was not OK.
- Clue: question 4 tells you which property to test.

ANSWERS

1. An **API (Application Programming Interface)** is an agreed way for one program to ask another program for something, without needing to know how that other program works inside.

The chapter's analogy is the right one: you are at a table with a menu. You do not go into the kitchen and cook — you tell the waiter what you want and food arrives. The kitchen is the application, the menu and the waiter are the interface.

An **endpoint** is one particular thing you can ask for, expressed as a URL:

<http://my-api.com/shoes> <http://my-api.com/clothes>

Same API, same domain, different endpoints — one item on the menu each.

2.

- **REST** uses ordinary HTTP methods (GET, POST, PUT, PATCH, DELETE) against URLs, and normally exchanges **JSON**. It is simple, lightweight, and what you will meet almost everywhere today.
- **SOAP** is a stricter, older protocol built on **XML**, with a fixed envelope structure and usually a WSDL file describing the service. It is still common in banking, insurance and other enterprise systems.

SOAP is usually handled outside the browser mainly because of **CORS**. A SOAP service on another domain will not normally send the headers a browser demands before it will let your JavaScript read the response, so the request is blocked. Backends have no such restriction, which is why SOAP calls are typically made server-side and the result passed on to the frontend.

There is also the XML itself: you need `DOMParser` and usually `XPath` with a namespace resolver to dig a value out of a SOAP envelope, where a REST reply is one `response.json()` away.

3.

200	OK	it worked
201	Created	it worked, and something new exists
400	Bad Request	your request was malformed
401	Unauthorised	you need to log in
404	Not Found	that resource does not exist
500	Internal Server Error	the server broke

The first digit is the useful part. **2xx** means success. **4xx** means the problem is at

- your* end — you asked wrongly, or you are not allowed. **5xx** means the problem is at the
- server's* end and there may be nothing wrong with your request at all.

That distinction decides what you do next: a 4xx usually means fix your request, a 5xx usually means try again later.

4. `response.ok` is **true for status codes in the 200–299 range**, and false for everything else.

It is not the same as `fetch()` succeeding, and this catches almost everyone once:

```
fetch('https://api.example.com/nope')
  .then(response => {
    // we ARE here, even for a 404
    console.log(response.ok);
    // false
    console.log(response.status);
    // 404
  });
```

`fetch()` only rejects when the request itself could not be made — no network, DNS failure, CORS refusal. A 404 or a 500 is a **successful round trip** that happ-

ened to carry bad news, so the promise resolves and your `.catch()` never runs.

That is why the chapter checks explicitly:

```
if (!response.ok) {
  throw new Error('Request failed with status ' + response.status);
}
```

5. Asynchronous JavaScript And XML.

The X is dated because almost nothing returns XML any more — modern APIs return **JSON**. The name stuck from the early 2000s, when `XMLHttpRequest` really was fetching XML.

The important half of the name is **Asynchronous**: the page carries on working while the request is in flight, rather than freezing or reloading. That is what AJAX actually gave us, and it is still exactly what `fetch()` does.

6.

<code>XMLHttpRequest</code>	the original way to make a request without reloading
<code>callbacks</code>	the original way to handle "when it finishes"
<code>promises</code>	fixed callback nesting; gave us <code>.then()</code> and <code>.catch()</code>
<code>fetch()</code>	a cleaner request API, promise-based from the start
<code>async/await</code>	made promise code read like ordinary sequential code

Two of them (`XMLHttpRequest`, `fetch`) are about **making the request**. Three (`callbacks`, `promises`, `async/await`) are about **handling the wait**.

They are not mutually exclusive. `fetch()` returns a promise, and `await` is just a nicer way to unwrap that same promise.

7. Because **nothing waited for the timeout to finish**.

```
// schedules a push in 2 seconds
createPost({ title: 'post three' });
// runs immediately
ul.innerHTML = getPosts();
```

`setTimeout()` is asynchronous. `createPost()` returns straight away, having only *scheduled* the work. So `getPosts()` runs about two seconds too early, reads an array that still has two posts in it, and renders those.

The fix — in any of its three forms — is to make the display step happen **after** the push, rather than merely after the call.

8. `Promise.all()` takes an array of promises and returns a single promise that resolves once **every one** of them has resolved. What you get back is an **array**

of results, in the order you passed them in — not the order they finished.

```
Promise.all([p1, p2, p3])
  .then(results => console.log(results));
// [r1, r2, r3]
```

If **any one of them rejects**, the whole thing rejects immediately with that first error, and you never see the results of the ones that succeeded. It is all or nothing.

That makes it right for "I need all of these before I can carry on", and wrong for "get me whatever you can" — for which there is `Promise.allSettled()`.

```
9. function getData() {
    return new Promise((resolve, reject) => {
      setTimeout(() => {
        resolve("Done!");
      }, 1000);
    });
}

getData().then(result => console.log(result));
// Done!
```

The function you hand to `new Promise` receives two functions of its own: call **resolve** with the value on success, or **reject** with an error on failure. Nothing happens until one of them is called — which is exactly why the promise can sit and wait.

```
10. async function showData() {
    try {
      const result = await getData();
      console.log(result);
      // Done!
    } catch (error) {
      console.error("Failed:", error);
    }
  }

  showData();
```

`await` pauses inside the `async` function until the promise settles, so the code reads top to bottom even though it is asynchronous. It only works inside a function marked `async`.

Note that `try...catch` here does the job `.catch()` does in the `.then()` style — which is one of the nicest things about `async/await`, since it means asynchronous errors are caught the same way as ordinary ones.

```

11. function createPost(post, callback) {
    setTimeout(() => {
        posts.push(post);
        // now the list refreshes
        callback();
    }, 2000);
}

createPost({ title: 'post three' }, getPosts);

```

Result: all three posts appear.

The detail worth getting right is on the last line. You pass `getPosts`, not `getPost() .` With the brackets you would call the function immediately and hand `createPost` whatever it returned. Without them you hand over the function itself, for `createPost` to call when it is ready.

```

12. const promise1 = Promise.resolve('Hello world');
    const promise2 = new Promise(resolve =>
        setTimeout(() => resolve('Second result'), 500)
    );

    Promise.all([promise1, promise2])
        .then(results => {
            console.log(results);
            // ['Hello world', 'Second result']
            console.log(results[0]);
            // Hello world
        });

```

Notice that `promise1` was ready instantly and `promise2` took half a second, yet the results come back in the order they were **passed in**, not the order they finished.

```

13. fetch('https://api.example.com/users')
    .then(response => {
        if (!response.ok) {
            throw new Error('Request failed with status ' +
                response.status);
        }
        return response.json();
    })
    .then(data => console.log(data))
    .catch(error => console.error(error));

```

Throwing inside a `.then()` sends control to the `.catch()` at the end, so both kinds of failure — a dead network and a 404 — end up in the same handler. Without the `!response.ok` check, a 404 would sail past and you would try to read JSON from an error page.

Chapter 23 — TEMPLATING

- Introduction to Templates
- String Literals & Template Strings
- JavaScript Templating Engines
- Popular Modern Approaches
- Implementing a template with Handlebars.js

This is about the available templating engine(s) available for your programming language, and how it works with them.

Introduction to Templates in JavaScript Templates in JavaScript help dynamically generate HTML content by injecting variables into a predefined structure. They make it easier to manage dynamic UI updates and are widely used in modern web development.

String Literals & Template Strings

JavaScript introduced template literals (backticks: ``...``) in ES6, allowing easy interpolation using `${}`. Example:

```
const name = "Alice";

// Output: Hello, Alice!
console.log(`Hello, ${name}!`);
```

While useful for small templates, they are limited for larger or more complex UI generation.

JavaScript Templating Engines

Templating engines provide more powerful ways to structure dynamic HTML. Popular options include:

- EJS (Embedded JavaScript) – Works similarly to PHP or JSP, embedding JavaScript into HTML.
 - Mustache – deliberately logic-less templates using `{{ }}` placeholders.
 - Handlebars – a superset of Mustache. It keeps the same `{{ }}` syntax and adds helpers, conditionals and loops. You use one or the other, not both together.

- Pug – A whitespace-sensitive, minimalistic templating engine.

Of these, Handlebars is the one you are most likely to meet, because it keeps Mustache's simplicity while giving you just enough logic to be practical.

Popular Modern Approaches

In modern JavaScript, frameworks handle templating in different ways:

- React uses JSX, which blends HTML inside JavaScript.
- Vue.js uses a declarative template syntax with directives.

These frameworks essentially replace traditional templating engines in many projects.

Implementing a template with Handlebars.js

Step 1: Install Handlebars

- You can include it via a CDN or install via npm. Install it by running this command:

```
npm install handlebars
```

Or use a CDN in the head tag of your HTML page:

```
<head>
<script
  src="https://cdn.jsdelivr.net/npm/handlebars/dist/handlebars.min.js">
</script>
</head>
```

Step 2: Define a Template in HTML

Here is an example code for creating a template. Place this code in your HTML file:

index.html

```
<!doctype html>
<html>
  <head>
    <title>Templates</title>
    <script
```

```

</script>
</head>
<body>

  <script
    id="template"
    type="text/x-handlebars-template">

      <h1>{{title}}</h1>
      <p>{{message}}</p>
    </script>

    <div id="output"></div>

    <script type="module" src="/index.js"></script>
  </body>
</html>

```

When using handlebars, a template is created within a script tag. This is the attribute that converts this script tag into a template: `type="text/x-handlebars-template"`. Once that template has been created, you will select it, inject data into it using the variables, and then insert the template into any element of your HTML you desire.

Step 3: Compile & Render the Template in JavaScript

This is referring to how you would go about passing pieces of data dynamically using JavaScript, that will be assigned to those variables (title and message in this case) that are being used in the HTML template. Note that in Handlebars, to parse variables or display their data within HTML code, you do so by wrapping the variables within double curly brackets.

```
{{myVar}}
```

The following is an example of how to create and inject the values for the variables used in your template. Do this in your JavaScript file:

index.js

```

const source =
  document.getElementById("template").innerHTML;

// template will become a function template()
const template = Handlebars.compile(source);

// prepare data for our template variables
const data = {
  title: "Hello World",
  message: "This is a Handlebars template!"
}

```

```
};

document.getElementById("output").innerHTML =
    template(data);
```

Explanation

In this example, we select our template script tag, which has the id attribute value of “template” which we assign to a variable named “source”. Next, we convert it into a template using the `compile()` method of Handlebars.

```
Handlebars.compile(source);
```

Next, we prepare our data to inject into our template, to be assigned to the template variables, which in our case are title and message. We assign this data to a variable named data.

Next, we select the target element where we wish to inject the template on our web page. In our case, that target HTML element is the div with the id of “output”. As we select this target div, we inject our template and data into it by assigning to it the `template()` function (which our template has now become), that in turn takes our data as its argument. For clarity, it would also work if we did it like so:

```
// our template is now a function that accepts our data
const templateAndData = template(data);
const targetDiv = document.getElementById("output");

// inject the data
targetDiv.innerHTML = templateAndData;
```

Here is where the data was created:

```
const data = {
    title: "Hello World",
    message: "This is a Handlebars template!"
};
```

The values of the variables in this data which was injected into the template (as its argument)

```
...template(data);
```

will now replace `{{title}}` and `{{message}}` in the `<script>` template

Handlebars and Mustache are still used, but in modern apps they are largely being replaced by React, Vue and other frameworks. They are useful for server-side rendering and simpler projects that don’t need full-fledged frameworks.

QUIZ — Templating

This page contains the Q & A (questions and answers) for this chapter — Chapter 23: Templating. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. A template literal lets you drop a variable straight into a piece of text. Two things make that work. What kind of quote mark has to wrap the whole string, and what do you put around the variable name inside it?

Clue: the quote mark is neither the single one nor the double one — it is the third kind, and it usually shares a key with ~. The variable sits inside braces, with one symbol in front.

2. A Handlebars template is written inside a script tag, like this:

```
<script id="template" type="text/x-handlebars-template">
  <h1>{{title}}</h1>
</script>
```

Why does that tag need `type="text/x-handlebars-template"`? What would the browser try to do with the tag if you left it out?

Clue: a script tag normally holds one particular kind of thing, and the browser does not just store it — it runs it. Ask yourself what would happen if it tried to run `<h1>{{title}}</h1>`.

3. Look at this line:

```
const template = Handlebars.compile(source);
```

What kind of thing is `template` now?

Clue: the very next line in the chapter uses it as `template(data)`. Those round brackets are the giveaway.

4. EXERCISE. You have this template already on the page, and this data in your JavaScript:

```
<script id="template" type="text/x-handlebars-template">
  <h1>{{title}}</h1>
</script>
```

```
<div id="output"></div>
```

```
const data = { title: "Hello World" };
```

Write the JavaScript that reads the template, turns it into something you can call, and puts the finished HTML inside the div whose id is output.

Clue: three steps, in this order — read the template's innerHTML, compile it, then assign the result of calling it with your data to the div's innerHTML.

5. Mustache and Handlebars both use the same `{{ }}` placeholders. What is the relationship between the two, and do you need to include both in a project?

Clue: the chapter describes one of them as a superset of the other, and says plainly which you should reach for.

ANSWERS

1. The string must be wrapped in backticks, and the variable goes inside `${ }`.

```
const name = "Alice";

// Hello, Alice!
console.log(`Hello, ${name}!`);
```

Both parts are required. With ordinary single or double quotes, `${name}` is just five characters of text and nothing is substituted. This is why the chapter says template literals are fine for small pieces of text but awkward once you are building a larger chunk of a page.

2. Without that type, the browser treats whatever is inside a script tag as JavaScript and tries to run it. `<h1>{{title}}</h1>` is not JavaScript, so it would fail.

Giving the tag a type the browser does not recognise tells it to leave the contents alone. It does not run them; it simply holds them as text. That is exactly what you want, because you are going to read that text yourself with innerHTML and hand it to Handlebars.

So the attribute is not decoration. It is the thing that turns a script tag into a place to keep a template.

3. It is a function.

`compile()` takes the template text and gives you back a function that is ready to be called. You then call it with your data, and it returns the finished HTML as a string:

```
const template = Handlebars.compile(source);
const html = template(data);
```

That is why the chapter's comment says the template "will become a function template()".

4. Here it is:

```
const source = document.getElementById("template").innerHTML;
const template = Handlebars.compile(source);

const data = { title: "Hello World" };

document.getElementById("output").innerHTML = template(data);
```

Reading it in order: take the template's text off the page, compile that text into a function, then call the function with your data and put what comes back inside the target div. The `{{title}}` in the template is replaced by "Hello World".

Written out one step at a time, the last line is the same as this:

```
const finishedHTML = template(data);
const targetDiv = document.getElementById("output");
targetDiv.innerHTML = finishedHTML;
```

5. Handlebars is a superset of Mustache. It keeps Mustache's `{{ }}` syntax and adds helpers, conditionals and loops on top.

You use one or the other, not both together. Of the two, Handlebars is the one you are more likely to meet, because it keeps Mustache's simplicity while giving you just enough logic to be practical.

Worth remembering alongside that: in modern applications, both are largely being replaced by React, Vue and similar frameworks. They remain useful for server-side rendering and for simpler projects that do not need a full framework.

Chapter 24 — EVENTS HANDLING

- Event listeners
 - Inline event listener attribute
 - The `addEventListener()` method
- Common JavaScript Events
- Keyboard key detection
 - Listening for a key press
- Triggering events dynamically
 - Dynamically triggering a click
 - Dynamically triggering a form's submit event
 - Dynamically triggering a focus on an input field
 - Events with no trigger methods
- A deep dive into event mechanics
 - Event bubbling
 - Event capturing (aka capture phase)
 - How to switch from bubbling to capturing
 - Bubbling vs Capturing, which is better?
 - Why ever use Event Capturing
 - Event delegation
 - Custom events
- Conclusion

One of the biggest reasons for JavaScript's popularity is how well it handles events—those moments when users interact with your page. Think about it:

- A click on a button.
- A hover over an image.
- A keydown when typing into an input.
- A submit of a form.
- A drag and drop of a file.

All of these are events. JavaScript listens for these user actions and gives you the power to respond immediately—updating the page, showing messages, processing data, and so much more. In this chapter, we'll look at:

- How to listen for events.
- Different types of events you can respond to.
- How to trigger events yourself from your JavaScript code.
- How events travel through the DOM (bubbling and capturing).
- How to delegate events smartly.
- How to create your own custom events!

EVENT LISTENERS

JavaScript gives you a few simple ways to listen for events and respond to them. There are two major ways to do so.

i) Inline event listener attribute

This works in two steps; you add the event listener attribute on the target element's tag, then get that event listener to call an inbuilt or your custom function in response to that event by passing the custom function as the value of the attribute. Here is an example:

```
<button
  id="myButton"
  class="btn btn-primary"
  onclick="myFunction(event)"
>
Click me</button>

function myFunction(e) {
  // stop event propagation
  e.preventDefault();

  let buttonId = e.target.id;
  console.log("ID is: " + buttonId);
}
```

Notice that the code that listens for the click event is `onclick=""`. Its value `"myFunction()"` is a function which should exist in your code. This function will automatically be called. Also notice how as the function is called within the attribute value, an 'event' string is passed to it (`onclick="myFunction(event)"`). This 'event' is the global event that is generated every time an event occurs, and so by passing it to the function you are calling, you are thereby making that event object available to the function. This is very important because that is how that function will know which specific element triggered the event among all the many elements you have on your webpage. It will then be able to attend to that specific field and accurately

respond to the event. Basically, the special event object holds details about what triggered the event (which button, what key, which field, etc). From inside your function, you can inspect the event object. For example, `e.target.id` gives you the ID of the button that was clicked in our above example:

```
function myFunction(e) {
    // optionally stop event propagation
    e.preventDefault();

    // handle the click on myButton
    let buttonId = e.target.id;

    // this will write "ID is: myButton"
    console.log("ID is: " + buttonId);
}
```

ii) The `addEventListener()` method

Instead of adding event listeners inside your HTML, JavaScript also provides a cleaner, more powerful way: programmatically adding an event listener on any element of your choice, and assign it an event handler function which will take the action in response to that event. You call the `addEventListener()` method on any element. It takes two main arguments:

- The event type (like "click", "submit", "mouseover", etc.).
- A function (event handler) to run when the event happens.

This function being passed as the second argument can be implemented in two ways;

a) Anonymous Function (Closure) This is a piece off code in the function's block to be executed right there as a closure (anonymous function). Let's see `addEventListener()` in action:

In a closure

```
<button
  id="myButton"
  class="btn btn-primary">
  Click me</button>

document.getElementById("myButton")
  .addEventListener("click", function(e) {

    // Prevent page refresh
    e.preventDefault();
```

```

let buttonId = e.target.id;

    console.log("THE BUTTON ID IS: "+
        buttonId);

});

```

Notice how the function ran here in response to the click event is an anonymous function. It is known as an anonymous function or closure because it has no name, and it's passed directly into the `addEventListener()` function, instead of being defined separately and then called (by its name) from the `addEventListener()` function.

In an external named function

```

// define the function
function myFunction(e) {
    // optionally stop event propagation
    e.preventDefault();

    let buttonId = e.target.id;
    console.log("THE BUTTON ID IS: " + buttonId);
}

```

```

let myButton =
    document.querySelector("#myButton");

```

```

// invoke the function
myButton.addEventListener("click", myFunction);

```

OR

```

document.getElementById("myButton")
    .addEventListener("click", myFunction);

```

Again, `addEventListener()` takes two arguments, the element the event is expected to be triggered on, and the function to run (which is basically the action to take when that happens). But I will like to draw your attention to two things about how this external function is called, and how the event object is passed through to your function. First, when you specify the function, you need to give only the function name, just like you would pass in a variable. It should not be wrapped in quotes for it is a function, and not a string. Though it is a function you are referencing here, do not place the parenthesis after the function name. If you do that, then JavaScript will attempt to call that function immediately, instead of waiting for the event, and so will fail.

```

// no parenthesis after function name
myButton.addEventListener("click",
    myFunction);

```

The second thing to note is how the event object is passed through to your function. When we passed the function name to `addEventListener()`, unlike the case of the event listener attribute where we pass in 'event' to our event-handler function as we reference it, with `addEventListener()`, you do not do that. You just need to reference the name of the function, just like you would write a variable. The `addEventListener()` method will capture that triggered event object and automatically pass it to the target event- handler function you are referencing. That is why the handler function should accept an event object (e) within its parenthesis, eg:

```
function myFunction(e) {  
    ...  
}
```

Beside the JavaScript events for HTML elements mentioned above like click, or submit, mouseover, hover, keydown, etc, there are many more. Just read the online JavaScript documentation to learn more.

Common JavaScript Events

Here's a list of the most commonly used events in everyday web development:

Event name	When it happens
<code>blur</code>	Element loses focus. An element loses focus when the cursor is no longer active on it
<code>change</code>	When the value of the field changes
<code>click</code>	User clicks an element
<code>dblclick</code>	User double-clicks an element
<code>drag</code>	Item is being dragged
<code>dragend</code>	Dragging ends
<code>dragenter</code>	Item is dragged into a droppable area
<code>dragleave</code>	Item is dragged out of a droppable area
<code>dragstart</code>	Dragging of an item starts
<code>dragover</code>	Item is being dragged over a droppable area
<code>drop</code>	Item is dropped into a droppable area
<code>focus</code>	Element gains focus. The cursor is active on it.
<code>input</code>	User types into a field
<code>keydown</code>	Key is pressed down
<code>keyup</code>	Key is released
<code>load</code>	Page or resource finishes loading
<code>mousedown</code>	Mouse button is pressed down
<code>mouseenter</code>	Mouse first enters an element (does not bubble)
<code>mouseout</code>	Mouse moves out of an element
<code>mouseleave</code>	Mouse leaves an element (does not bubble)
<code>mouseover</code>	Mouse moves over an element
<code>mouseup</code>	Mouse button is released
<code>resize</code>	Window size changes
<code>scroll</code>	User scrolls the page or an element
<code>submit</code>	Form is submitted

Tip: Some mouse events (`mouseenter`, `mouseleave`) do not bubble, meaning they don't pass upward through the DOM like `mouseover` and `mouseout` do. (We'll talk

about bubbling shortly!)

Keyboard key detection

The keydown event will fire when any key on your keyboard is pressed. It will therefore be useful to know how to detect which specific key was pressed. JavaScript provides you a way to do this. Having this skill can be invaluable when building applications that involve users controlling objects on the screen with the keyboard. This is a vital skill when creating games, which is something JavaScript shines in. Another great use for keyboard detection is if you wish to provide the users of your application with shortcuts for certain tasks or functionalities.

When your program needs to react to keyboard input (like moving a ball with arrow keys), JavaScript gives you special keyboard events such as:

- keydown – when a key is pressed down
- keyup – when a key is released
- keypress – similar to keydown (but now discouraged for most uses)

Listening for a key press

You can use `document.addEventListener()` to listen for key events on the entire page. Here's an example:

```
document.addEventListener("keydown", (e) => {  
    console.log("You pressed:", e.key);  
});
```

Let's create an example involving detecting a press on the arrow keys. We used this same example logic in Chapter 15 to move a ball on the HTML5 canvas based on the direction of the arrow key that was pressed.

```
document.addEventListener("keydown", (e) => {  
    let dx = 0;  
    let dy = 0;  
  
    if (e.key === "ArrowUp") dy = -ball.speed;  
    if (e.key === "ArrowDown") dy = ball.speed;  
    if (e.key === "ArrowLeft") dx = -ball.speed;  
    if (e.key === "ArrowRight") dx = ball.speed;  
  
    // Move the ball if the direction is valid  
});
```

The way this code logic works is as follows

- `e` is the event object passed to your function. It has properties in which important things about the event that just occurred are recorded. The property that will contain information varies depending on the event that occurred. The event in this example is `keydown` (or `keypress`), and therefore, the properties you should be checking to get the details of the event would be properties like `.key`, `.code`, or `.keyCode` (not recommended as it is deprecated).
- `e.key` will always contain the name of whatever key on your keyboard was pressed (like `"ArrowUp"` or `"a"` or `"k"` etc).
- `e.code` always contains a code string associated with, and represent any key which you can use instead of the key name stored in `e.key`. It is similar to the `.key` value, but meant to be a code string.
- `e.keyCode`. Older JavaScript applications (or game engines) sometimes use these. They are number codes associated with all keys, that you can use instead of the string values. It is deprecated but still works as of the date of this writing.

The following table shows you the values recorded for the key names on the `.key` property, and the corresponding codes stored in the `.code` property of the event object when the arrow keys are pressed. To give you variety on alternative keys, I have also thrown in there, the names and codes recorded when the space bar and letter a keys are pressed.

e.key	e.code	e.keyCode (old)
"ArrowLeft"	"ArrowLeft"	37
"ArrowUp"	"ArrowUp"	38
"ArrowRight"	"ArrowRight"	39
"ArrowDown"	"ArrowDown"	40
" "	"Space"	32
"a"	"KeyA"	65

If for example I wanted to detect if the `arrowUp` key was pressed when `keyDown` event occurred by using the number code (`.keyCode`) instead of the `.key` value, I would do it like so:

```
document.addEventListener("keydown", (e) => {
  if (e.keyCode === 38) {
    console.log("Up arrow!");
  }
});
```

Visit the online documents for the codes of any other keys you are interested in detecting.

Triggering events dynamically

Dynamically doing something in programming terms means you are making something happen entirely in code, without any manual user effort. Sometimes you don't want to wait for a user's action before you execute an action. You can programmatically trigger certain events by calling specific methods on DOM elements. All HTML elements have certain events associated with them, and so they each have methods provided by JavaScript to deal with these events. For example, the form element has a `submit()` method which can be used to trigger the submit event (for submission) on the form or to check if it has been submitted. An anchor tag, which is used to create links has a `click()` method for handling a click event on it etc. As you get familiar with different events, you will come to be familiar with which events belong to which elements. Let's learn how to dynamically trigger some events in code:

Dynamically triggering a click

```
<a id="myLink" href="https://example.com" target="_blank">Visit
  Example</a>

<script>
  const a = document.getElementById("myLink");

  // Triggers the click programmatically
  a.click();
</script>
```

Here we select the anchor tag by its id attribute 'myLink', and store it in the variable `a`. Then we trigger a click event on it by calling the `click()` method that belongs to all HTML anchor elements.

Dynamically triggering a form's submit event

```
<form id="myForm" action="https://example.com/submit"
  method="POST">

  <input type="text" name="username" value="john_doe">

</form>

<script>
```



```
// Programmatically submits the form
    form.submit();
</script>
```

Here we select the form element by its id 'myForm'. Next, we submit the form dynamically by calling the submit() method that we know belongs to all HTML form elements.

Dynamically triggering a focus on an input field

```
<input id="myInput" type="text" placeholder="Type here...">

<script>
    const input = document.getElementById("myInput");

    // Puts cursor into the input field
    input.focus();
</script>
```

Here we select the input field by its id attribute 'myInput'. Next, we call the focus() method on it to place an active cursor in it.

Events with no trigger methods

As we have seen, some events have special methods you can directly call to trigger events like:

- click()
- submit()
- focus()
- blur()
- play(), pause() (for media elements)

However, for most other events, there are not specialised methods you can just call to handle them. For such events, the way to trigger them is to manually create and dispatch an event. For example, the following is how to trigger a mouseover event manually:

```
<button id="myButton">My Button</button>

<script>
    const element = document.getElementById("myButton");

    // Listen for the custom event
    element.addEventListener("mouseover", function(e) {
```

```
// Create a new event
const myMouseoverEvent = new MouseEvent("mouseover");

// Dispatch the event
element.dispatchEvent(myMouseoverEvent);
</script>
```

In this example, notice how you have to create the `mouseover` event by instantiating the `MouseEvent` class by passing to its constructor, the mouse event name as a string-which in this case is `'mouseover'`.

Let's understand an important distinction here. If you take out these last two lines:

```
const myMouseoverEvent = new MouseEvent("mouseover");
element.dispatchEvent(myMouseoverEvent);
```

You would notice that the event still fires. But there is a difference. The code will still work, and the `"mouseover"` event will still be triggered, however, it will be triggered only when the user hovers over the button with the `id` attribute of `"myButton"`. In this case, there will be no need to create or dispatch anything yourself. The browser itself creates and dispatches the event for you when the user interacts. That's default browser behaviour-you don't manually fire anything. Basically, here, you are waiting for the user to naturally trigger the event (move mouse) before the event will fire. Here, the browser automatically creates and dispatches event when user moves mouse

But if you put back the two lines, you will also notice that even if the user does nothing (no mouse movement over the `myButton` element), the event handler will still run. It's like you simulating or faking the user action by force. At page refresh, it fires, no need to hover over the button. Basically, here you are manually firing the event immediately, even if no user action happens. Here, you create the event yourself and dispatch it programmatically.

Following this same example, you can create and dispatch events like `dragstart`, `drag`, `mouseenter`, `mouseleave`, and even custom events this way. You may be asking yourself what the use case for triggering events dynamically will be. It is a good question. You would want to trigger an event yourself in code (maybe for automation, testing, user assistance, simulations, etc.). Imagine that you are testing your website and you want to simulate a user hovering without actually moving the mouse. Instead of saying "Hey, test person, move your mouse here!", you just use:

```
element.dispatchEvent(new MouseEvent("mouseover"));
```

And, boom! The event fires.

Deep dive into event mechanics

Event bubbling

When an event happens, it first occurs on the target element (where the user acted). Then, it "bubbles up" through the parent elements all the way to the root (highest parent element) (`<html>`). Example:

- You click a button inside a div.
- The click event first fires on the button.
- Then it bubbles up to the div.
- Then bubbles up to the body, and so on.

Why it matters: you can listen for an event higher up in the DOM, instead of on every tiny element individually. Event bubbling is the default propagation pattern.

Event capturing (aka capture phase)

This is the event propagation pattern that is directly opposite to bubbling. In event capturing mode, instead of bubbling up, events will first travel downward from the root (top-most parent) downward to the target (the element on which the event was triggered). This is called capturing. By default, most event listeners listen during the bubbling phase. But you can make a listener listen during capture phase by setting `{ capture: true }`. This is an advanced concept to grasp for less advanced programmers, so let me take the time to break it down. If I seem to be repeating some words, just follow along, it's to drive this important topic home in your mind. In the context of event capturing, "the root" simply means the very top parent element. In a web page, the biggest root is usually the document or `<html>` tag. But inside a smaller section, like a `<div>`, the root could be that `<div>` when you're only considering its children. Let me demonstrate:

```
<div id="parentDiv">
  <ul>
    <li id="item1">Item 1</li>
    <li id="item2">Item 2</li>
  </ul>
</div>
```

In this example; `#parentDiv` is the root inside this small piece of DOM. The `<li>` elements are children (targets you might click on). Again, the target is the specific element that the user actually interacts with. If you click on Item 2, then the target is the `<li>` with `id="item2"`. If you click on Item 1, the target is `<li>` with `id="item1"`.

Here is how capturing actually works. With event bubbling, the event starts at the target and bubbles up toward the root. But with capturing, the event first starts at the root and travels down the DOM tree — down through each parent, until it reaches the target. Let me demonstrate with a practical working example:

HTML code

```
<div id="parentDiv" style="padding:20px; border:2px solid black;">
  Parent Div
  <ul style="margin-top:10px;">
    <li id="item1" style="padding:10px; border:1px solid red;">
      Item 1</li>
    <li id="item2" style="padding:10px; border:1px solid blue;">
      Item 2</li>
  </ul>
</div>
```

JavaScript code

```
const parentDiv = document.getElementById("parentDiv");
const item1 = document.getElementById("item1");
const item2 = document.getElementById("item2");

// Add event listener to the parent div (capturing phase)
parentDiv.addEventListener("click", function(e) {
  alert(
    "Parent DIV handled the click FIRST during CAPTURING phase!"
  );
// <- This 'true' enables capturing
}, true);

// Add event listener to item1 (normal bubbling)
item1.addEventListener("click", function(e) {
  alert("Item 1 was clicked (handled during BUBBLING phase)");
});

// Add event listener to item2 (normal bubbling)
item2.addEventListener("click", function(e) {
  alert("Item 2 was clicked (handled during BUBBLING phase)");
});
```

Here is what happens when you run this code; If you click on Item 1:

- First, the Parent DIV's capturing handler fires (because we set {capture: true}).
- Then, the Item 1 click handler fires during bubbling.
- The same happens when you click on Item 2.

As a final analogy; imagine the Root (Parent Div) to be the CEO, and the Target (Item clicked) to be a Worker. When a user clicks,

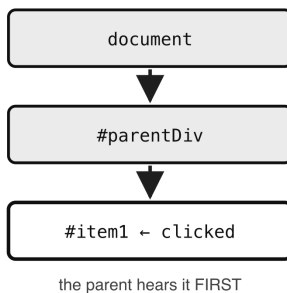
- In bubbling, the Worker notices first, then tells the Supervisor, then the Manager, up to the CEO.
- In capturing, the CEO checks first ("is it mine?"), then passes it down Manager to Supervisor to Worker, until finally the exact clicked item is found.

One click, two journeys through the same elements

The click happens once. What differs is WHEN each listener is told about it.

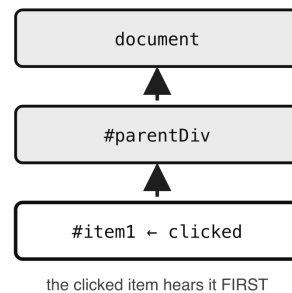
CAPTURING — top down

`addEventListener(..., true)`



BUBBLING — bottom up (the default)

`addEventListener(...)` — no third argument



Bubbling is the default, and it is what makes event delegation possible: one listener on the parent catches clicks from every child, including children added later.

Reach for capturing only when you must intercept an event BEFORE it reaches its target.

- Figure 25.1 — One click, two journeys through the same elements*

How to switch from bubbling to capturing

You have just seen how to do that already, but I am making this sub-heading to separately emphasise on how it's done again because it is very essential knowledge. Thus far, I have told you about two arguments you need to pass to the `addEventListener()` method whenever you use it to listen for an event on an HTML element. Well, there is a third argument, and that is a boolean (`true` or `false`) which determines if the event propagation is going to be in the capturing phase or not. This third argument is `false` by default, so even when you do not pass anything into it, event capturing will be set to `false`. That is why the argument is optional, and that is why event bubbling is on by default. If you want to handle an event in the capturing phase therefore, you have to explicitly pass in a value of `true` for that third parameter. This is how to do that:

```
element.addEventListener("click", function, true);
```

The `true` tells JavaScript: "I want this function to run during the capturing phase (traveling from top/root down to the target), not during the default bubbling phase (from target upward)."

Bubbling vs Capturing, which is better?

The answer is quite simple. Bubbling is generally better and more widely used, and here is why:

- 1. Bubbling is the default behaviour
 - Bubbling happens automatically in JavaScript.
 - You don't have to set anything special (like `{ capture: true }`).
 - It's the natural way the browser handles events.
- 2. Bubbling is more Predictable
 - Developers are more familiar with bubbling.
 - When you write event handlers on child elements, you expect the event to "bubble up" to parent elements.
- 3. Bubbling is what makes event delegation possible
 - Event delegation (which we talked about) depends on bubbling.
 - It lets you attach one event listener to a parent instead of many listeners on children.
 - This could be a huge performance boost when you have many child elements.
- 4. Using bubbling makes for less overhead
 - The browser is heavily optimised to handle bubbling efficiently.
 - Capturing is a bit less commonly used, so it's slightly less optimised in some browsers (but in practice, this difference is very tiny).
- 5. Using bubbling makes for cleaner and simpler code
 - This is because without capturing, you avoid complicating the event flow unnecessarily.

Why ever use Event Capturing

As bad as it seems to be from the ton of points above, capturing does have its own uses. It would be useful in the following scenarios:

- When you need to intercept an event before it reaches its target.
- When you want a parent to prevent something before the child can react.
- You are dealing with security-sensitive or priority-sensitive actions.

Take for example, you may have a parent form that needs to stop certain clicks from even reaching inner elements (e.g., security features in apps). However, we

can all agree that these occasions are rare. The rule to go by is this; always go with bubbling unless you have a very special reason to use capturing.

Event delegation

Event Delegation is a smart technique:

- Instead of attaching event listeners to hundreds of child elements,
- You attach one listener to a common parent,
- Then use the `event.target` inside your function to know exactly which child triggered the event.

Let's see an example:

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>

<script>
  document.getElementById("myList")
    .addEventListener("click", function(e) {
      if (e.target.tagName === "LI") {
        alert("You clicked " + e.target.innerText);
      }
    });
</script>
```

This works well, and with just one event listener on the `<ul>`, you handle clicks for any `<li>`

Custom events

JavaScript even allows you to create your own custom events. Example: creating and dispatching a "userLoggedIn" event:

```
// Create a new event
const loginEvent = new CustomEvent("userLoggedIn", {
  detail: { username: "john_doe" }
});

// Listen for the custom event
document.addEventListener("userLoggedIn", function(e) {
  alert("Logged in user is: " + e.detail.username);
});

// Dispatch the custom event
document.dispatchEvent(loginEvent);
```

When you comment out this last line that dispatches the event:

```
document.dispatchEvent(loginEvent);
```

Nothing happens. But as soon as you comment out the line, the event is dispatched, and your listener code picks it up.

Custom events are powerful for building your own systems inside an app. Let's look at a scenario that demonstrates how to create and dispatch a custom event in a real-world-like situation. Imagine you're building a web app. When a user successfully logs in, several parts of your app should update:

- A welcome message should show.
- A logout button should appear.
- The user profile section should refresh.

Here is the kicker; instead of calling all these updates manually in the login function, you can dispatch a custom `userLoggedIn` event, and let each part of the app listen for that event and respond. Here is some code to do just that:

```
<!DOCTYPE html>
<html>
<head>
<title>Custom Event Demo</title>
</head>
<body>

<button id="loginBtn">Login</button>

<div id="welcomeMsg" style="display: none;">Welcome, user!</div>
<button id="logoutBtn" style="display: none;">Logout</button>

<script>
  // 1. Components listening for custom event
  document.addEventListener("userLoggedIn", function (e) {
    document.getElementById("welcomeMsg").style.display = "block";
  });

  document.addEventListener("userLoggedIn", function (e) {
    document.getElementById("logoutBtn").style.display = "inline-block";
  });

  // 2. Simulate login and dispatch custom event
  document.getElementById("loginBtn").addEventListener("click", function
() {
    // Do login work here...

    // Now dispatch your custom event
    const loginEvent = new CustomEvent("userLoggedIn");
    document.dispatchEvent(loginEvent);
```



```
});  
</script>  
</body>  
</html>
```

Once more, here are the key points of what it does:

- CustomEvent is used to create an event with a name you choose.
- You can dispatch it using `element.dispatchEvent(...)`.
- Any part of the app that cares about that event can simply listen for it and take its own action.
- This promotes decoupled design – components don't need to know about each other.

Passing data with your custom event

You can pass data via your events so that subscribers to the event (event listeners) can retrieve and use it. This will of course be information relevant to the event that occurred, or information specific to the target element (the element that triggered the event). If for example, you wanted to pass in some user data, you could use detail:

```
const loginEvent = new CustomEvent("userLoggedIn", {  
  detail: { username: "coder123" }  
});  
  
document.dispatchEvent(loginEvent);  
  
// In the listener:  
document.addEventListener("userLoggedIn", function(e) {  
  console.log("Logged in user is:", e.detail.username);  
});
```

Notice how the listener retrieves and uses that data from the event object stored in `e`. Note very carefully here that in a custom event, this key that holds the data object you are passing through has to be named `'detail'`. It has to be exactly that as required by the JavaScript specification. You cannot change it to anything else like `data`, `info`, or `payload`. JavaScript looks specifically for `event.detail` when you're dealing with custom events. This is because when you create a `CustomEvent`, you're using the built-in `CustomEvent` constructor. This constructor accepts an optional second argument—an object—and it has a predefined structure. One of the allowed keys is `detail`, and it's the only place where you can safely pass your own custom data.

Conclusion

In this chapter, you learned:

- What events are and why they matter.
- Two ways to listen to events: `Inline` and `addEventListener()`.
- A huge list of useful everyday events.
- How to trigger events dynamically.
- How events travel through the DOM (bubbling, capturing).
- How to smartly delegate event handling.
- How to create and fire your own custom events!

By mastering events, you can make your web pages truly interactive and responsive to users. Events are truly the heartbeat of the modern web.

QUIZ — Events Handling

This page contains the Q & A (questions and answers) for this chapter — Chapter 24: Events Handling. Work through these after reading the chapter, while the material is fresh — recall practice is what cements new knowledge into long-term memory.

Try every question before you look below. Each one carries a clue, so nothing here should leave you stuck. Questions 8 to 11 are proper exercises where you write and run real code. The answers are all together in the Answers section further down, numbered to match the questions.

QUESTIONS

1. There are two ways to attach an event handler to an element. Name both, and say which you should prefer and why.

Clue: one lives in your HTML, the other in your JavaScript.

2. What is event bubbling, and what is event capturing?

Clue: same journey, opposite directions.

3. How do you make a listener run during the capture phase instead?

Clue: `addEventListener` takes a third argument you have probably never used.

4. A parent has a capturing listener and a child has an ordinary one. You click the child. Which handler runs first?

Clue: think about which direction the capture phase travels.

5. What is event delegation, and why does it depend on bubbling?

Clue: one listener instead of fifty — and it keeps working for elements that do not exist yet.

6. Inside a handler, what is the difference between `e.target` and `e.currentTarget`?

Clue: with delegation these are usually two different elements.

7. What does `e.stopPropagation()` do, and how is it different from `e.preventDefault()`?

Clue: one stops the journey, the other stops the browser's own reaction.

8. EXERCISE. Attach a click listener to a button that prints the button's id, using `addEventListener`.

Clue: the event object knows which element was clicked.

9. EXERCISE. Prove the bubbling order: put a listener on a parent and on a child, click the child, and print the order they fire in.
- Clue: no third argument on either.
10. EXERCISE. Now make the parent's listener capture instead, and show that the order reverses.
- Clue: one word changes.
11. EXERCISE. Use event delegation: put a single listener on a `<ul>` that reports which `<li>` was clicked — and show it still works for an `<li>` added afterwards.
- Clue: question 6 tells you which property identifies the actual item.

ANSWERS

1. The two ways are:

- **The inline HTML attribute:**

```
<button onclick="myFunction(event)">Click me</button>
```

- **`addEventListener()` in JavaScript:**

```
button.addEventListener("click", myFunction);
```

Prefer `addEventListener()` . Three reasons:

- It keeps your JavaScript out of your HTML, so the markup stays about structure and the script stays about behaviour.
- You can attach **several** listeners for the same event on the same element. An `onclick` attribute can only hold one thing; adding a second replaces the first.
- You can remove a listener again with `removeEventListener()` .

Inline handlers are still worth recognising, because you will meet them in older code.

2. They are the same journey through the same elements, travelling in opposite directions.

- **Bubbling** starts at the **target** — the element actually clicked — and travels **upward** through its parents to `document` . This is the default.
- **Capturing** starts at the **root** and travels **downward** through the parents until it reaches the target.

Every click actually does both: the browser runs the capture phase down, reaches the target, then bubbles back up. Which of your handlers fires when depends on which phase each one registered for.

3. Pass `true` as the third argument to `addEventListener()`:

```
element.addEventListener("click", myFunction, true);
```

That third argument defaults to `false`, which is why bubbling is what you get when you do nothing. You may also see it written as an options object, which reads more clearly:

```
element.addEventListener("click", myFunction, { capture: true });
```

Both mean the same thing.

4. **The parent's capturing handler runs first**, then the child's.

That surprises people, because the parent is "further away" from the click. But capturing travels **downward from the root**, so the parent is reached on the way to the target:

document → #parentDiv (capturing handler fires) → #item1 (target, its handler fires)

If the parent's listener had been an ordinary one, the order would be the other way round — child first, then parent on the way back up.

5. **Event delegation** is putting one listener on a **parent** and using it to handle events from any of its children, instead of attaching a listener to each child individually.

```
list.addEventListener("click", function (e) {  
  if (e.target.tagName === "LI") {  
    console.log("You clicked:", e.target.textContent);  
  }  
});
```

It depends on bubbling because that is what carries the event **up** from the clicked child to the parent where your listener is waiting. With no bubbling, the parent would never hear about it.

Two reasons it is worth knowing. It is far cheaper than fifty listeners on fifty items. And more importantly, it **keeps working for elements added later** — a new `<li>` needs no listener of its own, because the one on the parent was never about any particular child.

- 6.

- **e.target** is the element the event actually **happened on** — the thing the user clicked.
- **e.currentTarget** is the element whose **listener is currently running** — the one you called `addEventListener` on.

When you put a listener directly on a button and click it, these are the same element, which is why the difference is easy to miss.

With delegation they differ, and that difference is the whole point:

```
list.addEventListener("click", function (e) {
    console.log(e.target);
    // the <li> you clicked
    console.log(e.currentTarget);
    // the <ul> holding the listener
});
```

7.

- **e.stopPropagation()** stops the event travelling any further through the DOM. Handlers on parents (or on children, if capturing) will not run. It affects **your other handlers**.
- **e.preventDefault()** stops the **browser's own default reaction** — a link navigating, a form submitting, a checkbox ticking. It does not stop the event moving through the DOM at all.

They solve different problems and are not interchangeable:

```
form.addEventListener("submit", function (e) {
    // do not reload the page
    e.preventDefault();
});

button.addEventListener("click", function (e) {
    // do not let the parent hear this
    e.stopPropagation();
});
```

A common mistake is reaching for `stopPropagation()` when a form keeps re-loading. That will not help — the reload is the browser's default behaviour, so `preventDefault()` is what you need.

8. `<button id="myButton" class="btn btn-primary">Click me</button>`

```
document.getElementById("myButton")
    .addEventListener("click", function (e) {
        console.log("ID is: " + e.target.id);
        // ID is: myButton
    });
```

The handler receives the **event object** as its first parameter — usually called `e` or `event` — and `e.target` is the element the click landed on.

```
9. <div id="parent">
    <button id="child">Click me</button>
</div>

document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked");
});

document.getElementById("parent").addEventListener("click", () => {
  console.log("Parent clicked");
});
```

Output when you click the button:

Child clicked Parent clicked

The child fires first, then the event bubbles up to the parent. Neither listener passed a third argument, so both are bubbling-phase listeners.

```
10. document.getElementById("parent").addEventListener("click", () => {
      console.log("Parent clicked");
      // <- the only change
    }, true);

document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked");
});
```

Output:

Parent clicked Child clicked

Exactly reversed, from adding one word. The parent is now listening on the way **down**, so it hears the click before the click has even reached the button.

```

11. <ul id="list">
    <li>Item 1</li>
    <li>Item 2</li>
</ul>

const list = document.getElementById("list");

// ONE listener, on the parent
list.addEventListener("click", function (e) {
    if (e.target.tagName === "LI") {
        console.log("You clicked:", e.target.textContent);
    }
});

// now add a third item, AFTER
// the listener was set up
const newItem = document.createElement("li");
newItem.textContent = "Item 3";
list.appendChild(newItem);

```

Clicking any of the three — including the one added afterwards — prints its text.

That last part is the real reward. If you had attached a listener to each `<li>` in a loop, "Item 3" would have been dead on arrival, because the loop ran before it existed. The delegated listener never knew about individual items in the first place, so a new one needs no special treatment.

The `if (e.target.tagName === "LI")` check matters: clicks on padding inside the `<ul>` also reach the listener, and without the check you would report those too.

Chapter 25 — ASSET MANAGEMENT

Here we will explore the subject of asset management in JavaScript, and talk about when and why it is needed. In larger web projects, developers often use special tools to help manage all the extra files that come with building a website—things like images, JavaScript files, stylesheets, fonts, and more. These tools are part of what's called assets management. They can automatically combine multiple files into one (to make your site load faster), shrink file sizes (to save space and time), and even help your browser remember the newest versions of your files. Tools like Webpack, Vite, and Parcel are great for this—but they're mostly used in big or complex projects, especially when working with frameworks like React, Vue, or backend environments like Node.js.

For this book and your learning journey with vanilla JavaScript, you don't need to worry about any of that. Our code examples and demonstration projects have been small and simple—just a few files you can organise in folders by yourself. You have been writing basic HTML, CSS, and JavaScript, and learning how to link everything together manually. This is not only enough for learning the core concepts, but also an important step in becoming a better developer. Once you start building larger apps or using frameworks, that's when asset management tools start to make sense—and by then, you'll have the skills to explore them with confidence.

What next

Congratulations! 🎉 You've just taken your first real steps into the world of coding using JavaScript — one of the most powerful and widely-used programming languages in the world today. Now you're probably wondering: "Where do I go from here?" Let me give you a roadmap. Here it is, in a nutshell:

- Learn a JavaScript Framework or Library
- Learn TypeScript
- Explore Backend Development
- Go Deeper with Databases

If you've enjoyed learning JavaScript so far, you're ready to level up. JavaScript truly begins to shine when combined with frameworks and tools that let you build full, powerful apps. Here's what you should explore next:

- Learn a JavaScript Framework or Library JavaScript frameworks help you build real-world apps much faster and more efficiently. Start with any of the following:
 - React - the most popular library for building dynamic user interfaces.
 - Vue.js - a beginner-friendly framework that's flexible and fun to use.
 - Angular - a very powerful JavaScript framework for building enterprise applications.
- Learn TypeScript TypeScript deserves a mention of its own, because it is very commonly misunderstood. It is not a framework or a library, and it is not something you use instead of JavaScript. TypeScript is a programming language of its own, built by Microsoft, and what it really is, is JavaScript with types added on top. That is why you will hear it called a superset of JavaScript—every valid piece of JavaScript you have written in this book is already valid TypeScript. Microsoft built it to solve a problem that only shows up as applications get large. As we learned in Chapter 10 (Data Types), JavaScript is loosely typed, so a variable that held a number can quietly hold a string later on. That flexibility is wonderful when you are writing a small script, and painful when you are one of thirty developers working on the same codebase. TypeScript lets you declare what type a thing is meant to be, and then tells you off before you ever run the code if you break your own rule. In other words, it catches a whole class of bugs while you are still typing, rather than in the browser at midnight. Here is the part a lot of developers never quite realise: browsers cannot run TypeScript. Not one of them. TypeScript compiles down to plain JavaScript first —the very

JavaScript you have been learning—and it is that output the browser actually runs. All the type information is stripped away in the process. So TypeScript is not a replacement for what you have learned here. It sits on top of it, and everything in this book still applies underneath. It has become popular enough that serious JavaScript developers tend to learn it sooner or later. Angular is written in it, React and Vue both support it, and a great many job adverts now list it beside JavaScript. Learn JavaScript properly first—which is what you have just done—and TypeScript will feel like a small step rather than a new language.

- Explore Backend Development While this book focused on frontend JavaScript, you'll eventually want to build apps that can store and retrieve data, handle user accounts, and do powerful things behind the scenes. For that, learn:
 - Node.js - JavaScript's way of running on the server.
 - Or any other backend (server side) programming language like PHP, Python, Java, or Golang etc depending on what is popular and on demand in your geographical location.
- Go Deeper with Databases You already met databases in Chapter 13 (Databases and Storage), where you saw that this is where real-world data lives. You know what they are and how to store data from the browser. The next step is to take that further and learn how to:
 - Connect a backend language (like Node.js) to a database (like MongoDB or PostgreSQL),
 - Then request for that data from your frontend using APIs, and on the backend, validate that request, fetch the data from the database and send it back to the frontend.

Once you can build a full loop — frontend → API → backend → database (and back) — you'll officially be on your way to becoming a full stack developer, one of the most in-demand roles in tech.

Here's something cool: once you learn one programming language (like JavaScript), it becomes much easier to pick up others. You've already done the hard part — learning how to think like a coder. Don't be afraid to try out new languages and technologies later on. You'll notice that many of the skills you've learned here carry over.

Practice, practice, and practice

Let me share something important. When I started learning to code, I read so many books packed with theory. I understood the concepts, but I wasn't improving.

Why? Because reading about code isn't the same as writing code. What finally made me good at coding was simple: hands-on practice. That's why I wrote this book, to teach you programming the way I would have wanted to be taught, when I was first starting out. The approach I take is to show you, not just tell you. To guide you through actual projects you can type out, run, and learn from. So here's my advice to you.

Typing things out, breaking things, fixing them, building real stuff, are all part of the process you have to go through to become a solid programmer. Learn Git (<https://github.com>) so you can deposit the code for the projects you work on there. It is very common practice today for employers to hire a developer after visiting their account on Github to see the kind of applications they have built. It could be a good way to showcase your prowess in any particular programming language, and keep solutions to problems you figured out so you can re-use the same code in other applications.

- Don't just read — code.
- Build small projects. Then build bigger ones.
- Experiment. Explore. Break things and fix them again.

It will be fun

Then it will be hard

But that is the same for any art

You will resolve code, and feel like a seer

That is the way of an engineer.

Written in Oshawa, Canada. Finished 2026.

ABOUT THE AUTHOR

Gustav N. is passionate about software development. He has been coding as a backend developer using JavaScript (vanilla and its many libraries and frameworks), PHP and database systems for over 15 years. His other languages of expertise are Java, Golang and Python. Holder of a BA Hons in Media Production, and an MSc in Computing — Software Engineering from the University of Northampton, UK, he is currently based in Toronto, Canada.